

Manual

MES Development Suite MDS-BAS 8.1

Version 1.6.23049

Last changed on: 01.09.2020

Copyright

©Copyright 2020 All rights reserved.

SAP® and R/3® are registered trademarks of SAP AG.

WINDOWS® is a registered trademark of Microsoft Corporation.

MPDV® and HYDRA® are registered trademarks of MPDV Mikrolab GmbH.

ORACLE® is a registered trademark of ORACLE Corporation, California, USA.

Copying and distribution of this documentation or any part thereof, for any purpose or in any form, is prohibited without prior written permission from MPDV Mikrolab GmbH.

The information contained in this documentation is subject to change without prior notice.

Contents

1	Overview	21
2	General Notes	22
2.1	Overview	22
2.2	Namespaces and scopes	22
2.2.1	Namespaces	22
2.2.2	Scopes	22
2.3	Important notes	23
3	MES Development Suite	25
3.1	Activating the MES Development Suite	25
3.2	Applications on the MOC.....	25
3.3	Meaning of customization.....	26
4	MOC configuration settings.....	29
4.1	Configuration settings and configuration levels.....	29
4.2	Activation of a configuration scope	29
4.3	Storage locations for configuration settings	30
4.3.1	User data.....	31
4.3.2	System-wide (local) changes	31
4.3.3	Customizations by MPDV	32
4.3.4	Notes on application configurations	32
4.4	Distribution of configuration settings.....	33
4.5	Configure syntactic types	33
4.6	Change the MOC logging	39
4.6.1	Change the storage location of the log file	40
4.6.2	Change the log level.....	40
5	MOC Update Package Creator	41
5.1	Overview	41
5.2	Generate MOC packages.....	42
5.3	Generate server packages	45
6	Update Packages for the Maintenance Manager.....	48

6.1	Overview	48
6.2	Black list for MOC updates using Maintenance Manager 2.....	49
6.3	Structure of MOC Client Package.....	50
6.4	Structure of Java Server Package	52
6.5	Structure of Server Package	55
7	MOC Layout Configuration.....	59
7.1	Changing the size of detail applications.....	59
7.2	Docking	59
7.3	Simple customization of table views	65
7.4	Customization of symbols.....	66
8	Multilingual texts in the MOC using language keys	69
8.1	Using the MPDV Dictionary	70
8.2	Notes on editing and translating label texts	72
8.3	Identifying Language Keys in the GUI	74
8.4	Tips on how to translate detailed dialogs.....	75
9	Customization of Applications	81
9.1	Overview	81
9.2	Customization of main applications	81
9.2.1	Creating an application.....	82
9.2.2	Customization of applications	82
9.2.3	Toolbar	83
9.2.4	Selection panel.....	86
9.2.5	Data sources	89
9.2.6	Detail applications	96
9.3	Customization of editing applications	100
9.3.1	Application generator.....	101
9.3.2	Additional customization options.....	101
9.3.3	Process configuration	103
9.4	Calling external programs.....	103
9.4.1	Menu	103
9.4.2	Toolbar	103
9.4.3	Quick launch bar	104
9.4.4	Parameter	104
9.5	Tips & Tricks	105

9.5.1	Deleting items from detail applications.....	105
10	Process Configuration.....	108
10.1	Processes	108
10.2	Structure of a process	109
10.3	Structure of <code>ProcessConfigurationItems</code>	110
10.4	Special case: Activation of a process from another application	110
11	Components.....	112
11.1	Table view (grid).....	112
11.2	Master Detail Grid	118
11.3	Master Detail Grid (without reloading)	125
11.4	Charts	131
11.5	Layout	132
11.6	Pivot.....	133
11.7	<code>FileAttachmentPlugin</code>	137
11.8	Special table views.....	140
11.9	Special charts.....	141
11.10	Grouping application	142
12	Input and Output fields	144
12.1	Customization	144
12.2	Properties of Input and Output Fields	145
12.3	Examples of customization settings.....	150
13	User Fields	153
14	Authorization	155
14.1	Activate authorization keys.....	155
14.1.1	How it works	155
14.1.2	Function authorizations	155
14.1.3	Licenses	156
14.1.4	Feature sets	156
14.2	Authorization of functions and fields	157
14.2.1	Authorization of Fields	157
14.2.2	Authorization of Field Groups	157
14.2.3	Authorization of applications.....	159

14.2.4	Authorization of Detail Applications	159
14.2.5	Authorization of Functions	160
14.2.6	Authorization of ReferenceData entries	161
15	Application Generator	163
15.1	Instructions.....	163
15.2	Description of the fields	165
16	Customization of the MOC Menu	169
16.1	Requirements.....	169
16.2	Overview	169
16.3	Customization file MenuAdjustments.json	169
17	MDS Extensibility Framework	174
17.1	Overview	174
17.2	Dependency Injection.....	174
17.3	Extended application configuration.....	175
17.3.1	Configuration	175
17.3.2	Available standard extensions	177
17.3.1	Creating your own extensions	182
17.4	Extensions of the toolbar	192
17.4.1	Configuration	192
17.4.2	Available standard extensions	193
17.4.3	Creating your own extensions	193
17.5	Creating programmed applications.....	197
17.5.1	Definition of a new application	197
17.5.2	Integration into the MOC menu.....	201
17.5.3	Integration into the toolbar	202
17.5.4	Calling "real MOC applications" from own applications	202
18	MOC application based on an SQL query (outdated)	204
18.1	Overview	204
18.2	Restrictions	204
18.3	How to proceed	204
19	MOC Development – "Cookbook"	208
19.1	Overview	208

19.2	Enable the MES Development Suite	208
19.3	Formatting of durations as axis values in the chart.....	208
19.4	Configuration of selection fields.....	209
19.5	Deployment of customizations.....	212
19.6	Generate a new application.....	213
19.7	Defining a conditional formatting in the grid	214
19.8	Configuration of column totals in the grid.....	215
19.9	Grouping fields within a detail application	216
19.10	Creating and modifying label texts.....	217
19.11	Logging and debugging	219
19.12	Adding and customizing fields in a detail application	219
19.13	Configuration of data sources.....	221
19.14	Configuration of detail applications	222
19.15	Adding and customizing fields in the selection panel	224
19.16	Configuration of a dependent data source	226
19.17	Creating or editing the menu	227
19.18	Customizing application layout via docking	229
19.19	Customizing symbols and images	230
19.20	Adding and customizing buttons in the toolbar	230
19.21	Configuring a detail application of type "chart".....	232
19.22	Configuring a detail application of type "table".....	233
19.23	Configuring a detail application of type "pivot"	235
19.24	Protecting a field via authorization	236
20	Customer-specific Database Contents.....	238
20.1	HYDRA SQL syntax	238
20.2	HYDRA SQL Interpreter hysql.....	240
20.3	Namespaces for customer-specific database objects	240
20.4	Conventions for names in the DB (tables, columns, ...)	241
20.5	Supported data types	241
20.5.1	Overview	241
20.5.2	Data type SERIAL	242
20.6	Creating functions and triggers.....	243
20.7	Example	244
20.7.1	Creating database objects	244
20.7.2	Inserting data in a table	246

20.7.3	Changing data in a table.....	247
20.7.4	Selecting data from a table	248
20.8	HYDRA SQL syntax reference	248
20.8.1	Maximum length of SQL statements.....	248
20.8.2	Name of database objects	249
20.8.3	Strings	252
20.8.4	Transactions.....	253
20.8.5	Current date, current time.....	254
20.8.6	Date format	254
20.8.7	Date functions	255
20.8.8	Other functions	255
20.8.9	Query of NULL values	256
20.8.10	Sort by NULL values.....	256
20.8.11	Sorting with union select.....	256
20.8.12	Group by calculated expressions.....	257
20.8.13	Outer Join.....	257
20.8.14	Temporary tables	257
20.8.15	unique / distinct	258
20.8.16	like / matches	259
20.8.17	Loading and unloading data	259
20.8.18	create table as select.....	260
20.8.19	CASE in the select clause	260
1.24	Integer division	260
20.8.20	Changing tables	261
20.8.21	Reserved keyword "key".....	261
20.8.22	Default values in the database schema	261
20.8.23	Process "clustered index"	263
20.8.24	Optimizing "update statistics" under ORACLE	263
20.9	Notes on the performance	264
20.9.1	Union versus union all	264
20.9.2	Substrings in the WHERE clause	264
20.9.3	truncate table.....	265
20.10	Access to several databases.....	265
20.10.1	Syntax	265
20.10.2	Restrictions	266

21 The Repository	267
21.1 Overview	267
21.2 Domain.....	267
21.3 Service	268
21.3.1 Name	268
21.3.2 Function	268
21.3.3 ServiceType	268
21.3.4 ListMode.....	269
21.3.5 DLG.....	269
21.3.6 SystemCall	269
21.4 ServiceGui	269
21.4.1 Name	269
21.4.2 Package	269
21.4.3 Extended	269
21.4.4 AdditionalDataLogics.....	270
21.4.5 ApplicationID	270
21.4.6 ApplicationTitle	270
21.4.7 ApplicationHelpFile.....	270
21.4.8 ApplicationHelpIndex.....	270
21.4.9 Description	270
21.5 ServiceParameter	271
21.5.1 Acronym.....	271
21.5.2 ResultSet.....	271
21.5.3 WebServiceType	271
21.5.4 DefaultValue.....	271
21.5.5 IsResult	271
21.5.6 IsDynamicResult	271
21.5.7 InputAsArray.....	272
21.5.8 IsSpecialParameter	272
21.5.9 IsFilterParameter	272
21.5.10 IsMandatory.....	272
21.5.11 Can* (filter) operators	272
21.5.12 HydraAcronym.....	273
21.5.13 HydraResultAcronym.....	273
21.5.14 TransferEmptyValuesToHydra	274
21.5.15 HydraShiftPart	274

21.5.16 Reference.....	274
21.5.17 TransformationType	274
21.5.18 PlugName	274
21.5.19 DBField	275
21.5.20 DBAlias	275
21.5.21 DBTabelle	276
21.5.22 DBFieldAlternative.....	276
21.5.23 DataObjectName.....	276
21.5.24 ConditionalFieldKey.....	276
21.5.25 Constraints	277
21.6 ServiceParameterGui	277
21.6.1 Acronym	278
21.6.2 ResultSet.....	278
21.6.3 Label	278
21.6.4 Tooltip	278
21.6.5 FormatType	278
21.6.6 ClientDefaultValue.....	279
21.6.7 IsKey	281
21.6.8 ShowInGrid	281
21.6.9 ShowInDetail	281
21.6.10 ShowInSearch	281
21.6.11 ColumnCategory	281
21.6.12 Category1, Category2, Category3	282
21.6.13 TabOrder	282
21.6.14 ColumnOrder	282
21.6.15 ShowSecondControllnSearch.....	282
21.6.16 SearchTabOrder.....	282
21.6.17 SearchCategory1, SearchCategory2	283
21.6.18 ControlType.....	283
21.6.19 ControlTypeMode	283
21.6.20 ControlParameter	285
21.6.21 ControlDataSource	285
21.6.22 ControlDataSourceMode	285
21.6.23 ControlDataSourceParameter	285
21.6.24 ControlDataSourceResult	285
21.6.25 VisibleCondition.....	286

21.6.26 EditableCondition	286
21.6.27 ScriptId	287
21.7 Property	287
21.7.1 Acronym	287
21.7.2 WebServiceType	287
21.7.3 NETType	288
21.7.4 SemanticType	288
21.7.5 SyntacticType	288
21.7.6 Label	289
21.7.7 DefaultTooltip	289
21.7.8 UnitLabel	289
21.7.9 OutputFormat	289
21.7.10 InputFormat	290
21.7.11 Length	290
21.7.12 Rules for the input/output formatting	290
21.7.13 FillChar	295
21.7.14 Calculation	295
21.7.15 Further fields see ServiceParameterGui	295
21.8 ControlDataSource	296
21.8.1 Name	296
21.8.2 Source	296
21.8.3 Parameter	296
21.8.4 Columns	297
21.8.5 Result	297
21.9 ReferenceData	297
21.9.1 ref_data_key	297
21.9.2 Type	298
21.9.3 db_key	298
21.9.4 is_default	298
21.9.5 Designation	298
21.9.6 sort_key	298
21.10 Authorization	298
21.10.1 Authorization type	298
21.10.2 Authorization Context	299
21.10.3 Authorization ID	299
21.10.4 Authorization key	299

21.10.5 Authorization Designation.....	299
22 Using the Repository as Development Tool.....	300
22.1 Use of transformation types for the dynamic conversion of DB or BAPI values.....	300
22.1.1 Overview	300
22.1.2 Standard transformation functions	300
22.2 Checklist: Repository data.....	311
22.3 InterpretedWrapper: Transfer of fixed values to PDM dialog	314
23 Repository Client.....	316
23.1 Quick start.....	316
23.2 Start and exit Repository Client	318
23.3 The Application Window	319
23.4 Grids/table views	320
23.5 The application menu	322
23.6 Workset.....	325
23.7 Relations	328
23.8 References.....	329
23.9 Service documentation.....	330
24 Using the Repository Client as Development Tool	332
24.1 How to create new contents	332
24.2 Context menu of the table view/grid	334
24.3 Export.....	341
24.4 Validation	342
25 Interpreted Java Service2	343
1.1 Introduction	343
25.1 Availability	343
25.2 Definition	343
25.3 Storage in a server	343
25.4 Available Special Parameters.....	343
25.5 Repository data.....	344
25.5.1 Tab Services	344
25.5.2 Tab ServiceParameter.....	345
25.5.3 Tab Dataobjects	347

25.6	Exits	351
25.6.1	Available user exits.....	352
25.6.2	Available program exits	352
25.6.3	Specifications for the implementation class of the exit	352
25.6.4	Interfaces	354
26	Interpreted Java Service	364
26.1	Introduction	364
26.2	Definition	364
26.3	Storage in a server	364
26.4	Available Special Parameters.....	365
26.5	Repository data	366
26.5.1	Tab Services	366
26.5.2	Tab ServiceParameter.....	366
26.5.3	Tab Dataobjects	368
26.6	Exits	372
26.6.1	Available user exits.....	372
26.6.2	Available program exits	373
26.6.3	Specifications for the implementation class	373
26.6.4	Interfaces	375
27	Interpreted BAPI Services.....	384
27.1	Introduction	384
27.2	Features.....	384
27.2.1	Processing steps of the Bapi Interpreter	384
27.2.2	Basic processing of the individual processing steps	385
27.2.3	Bapi functions.....	385
27.3	Service definition.....	388
27.4	Storage in a server	389
27.5	Repository data	389
27.5.1	Tab Services	389
27.5.2	Tab ServiceParameter.....	389
27.6	User exits	394
27.6.1	Available user exits.....	394
27.6.2	Specifications for the implementation class	395
27.6.3	Interfaces	397

28 System Utilities.....	403
28.1 Introduction	403
28.2 Schnittstellenbeschreibung.....	403
28.3 Interface ISystemUtilFactory	403
28.3.1 Utility list	404
28.4 Interface IDbConnectionProvider.....	406
28.5 Interface ISdiLoggerProvider.....	406
28.6 Interface ISdiLogger	407
28.7 Interface IToNativeSqlConverter	407
28.8 Interface IToNativeSqlWithParamsConverter	407
28.8.1 Data class MpdvSqlParam	408
28.8.2 Data class NativeSqlParam	408
28.8.3 Data class NativeSqlData	409
28.9 Interface IToNativeSqlWithInlinedParamsConverter	409
28.10 Interface IServiceConfigProvider	410
28.11 Interface IServiceConfig	411
28.12 Interface IHydraCaller	411
28.12.1 Data class HydraCallResult	419
28.12.2 Data class HydraReturnValue	419
28.13 Interface IPdmStringParser	420
28.14 Interface IDataTableBuilder	421
28.15 Interface IDataTableModifier.....	424
28.16 Interface IFeatureSetChecker	427
28.17 Interface IDbTypeRetriever	427
28.18 Interface IUserExitProvider.....	428
28.19 Interface IUserExitFromScopeProvider	430
28.20 Interface IUserExit.....	431
28.21 Interface IAcronymMappingProvider	432
28.21.1 Interface IAcronymMapping	432
28.22 Interface IFormulaProvider	434
28.22.1 Interface IFormula	434
28.22.2 Interface IFormulaEvaluationResult	436
28.23 Interface IServiceCaller	438
28.23.1 Interface IServiceCallResult	439
28.23.2 Class FilterParam	439
28.24 Interface IDataAvailabilityChecker	440

28.24.1 Class DataAvailabilityCheckData.....	440
28.24.2 Class DataAvailabilityObject.....	441
28.24.3 Class DataAvailability	442
28.25 Interface IHydradirPathProvider	442
28.26 Interface IHydradirInstancePathProvider	443
28.27 Interface IHydradirTempPathProvider	443
28.28 Reading settings from the configuration files	444
28.28.1 WSP configuration files	444
28.28.2 Interface IGlobalConfigValueProvider.....	444
28.28.3 Interface IInstanceConfigValueProvider.....	445
28.29 Interface IHydraPathReadFileAction	445
28.30 Interface IHydraPathWriteFileAction.....	445
28.31 Interface IHydraPathRenameFileAction.....	446
28.32 Interface IHydraPathDeleteFileAction.....	446
28.33 Interface IHydraPathFileExistsAction.....	447
28.34 Interface IHydraPathCreateFolderAction	447
28.35 Interface IHydraPathDeleteFolderAction	448
28.36 Interface IHydraPathDownloadContentAction.....	448
28.37 Interface IHydraPathUploadContentAction	449
28.38 Interface IHydraPathListContentAction.....	449
28.39 Interface IHydraPathIsDirectoryAction.....	450
28.40 Interface IPersonCardIdChecker	450
28.40.1 Interface IPersonCardIdCheckResult	451
28.40.2 Enum PersonCardIdCheckState.....	452
28.41 Enum CheckResponsibilityAreaMode.....	453
28.42 Interface IResponsibilityAreaChecker.....	453
28.42.1 Class ResponsibilityAreaCheckerParam	454
28.43 Interface IPersonResponsibilityAreaChecker.....	454
28.43.1 Enum PersonResponsibilityAreaCheckerResult	454
28.44 Interface ISqlGenerator	455
28.44.1 Class SqlGeneratorParam.....	457
28.44.2 Class AcronymMergeStruct	457
28.44.3 Interface ISqlSpecialFilter.....	458
28.44.4 Class SqlSpecialFilterResult.....	458
28.44.5 Class ASqlFilterExpression	459
28.44.6 Enum SqlFilterType	459

28.44.7 Class SqlFilterComplexExpression	460
28.44.8 Enum SqlFilterConjunction	460
28.44.9 Class SqlFilterSimpleExpression	460
28.44.10 Class SqlFilterRawExpression	461
28.45 Interface IMemoryChecker	461
28.46 Interface IMandatorySpecialParameterValidator	462
28.47 Interface IExitMethodExecutionManager	463
28.48 Interface ISdiDataRowCopyUtil	466
28.49 Interface IDataTableToStreamConverter	467
28.50 Interface IStreamToDataTableConverter	467
28.51 Interface IResultTransformationManager	467
28.52 Auxilliary classes	469
28.52.1 Class SdiEagerDataRowStream	469
29 GlobalExits	471
29.1 Introduction	471
29.2 Availability	471
29.3 Exits	471
29.3.1 Available user exits.....	471
29.3.2 Specifications for the implementation class	472
29.3.3 Interfaces	473
29.4 HowTos.....	479
29.4.1 Redirecting a service request to another service	479
29.4.2 Creating additional result rows	480
30 Simple External Services	482
30.1 Development process.....	482
30.1.1 Repository	482
30.1.2 Specifications for the implementation class	482
30.1.3 Create mapping.....	483
30.2 Interface description	484
30.2.1 Interface ISimpleExternalService	484
30.2.2 Interface ISystemUtilFactory.....	485
30.2.3 Data types	485
30.2.4 Builder	491
30.3 Best Practices	495

30.3.1	Exception Handling in Simple External Services.....	495
30.3.2	Creating a DataTable	496
31	ExternalUtils	498
31.1	Introduction	498
31.2	Availability	498
31.3	ExternalUtils	498
31.3.1	Directory ext_util_jar.....	498
31.3.2	Directory ext_util_classes	499
32	MDS-WebUtil	500
32.1	Purpose.....	500
32.2	Requirements.....	500
32.3	Code example	500
32.4	API reference	500
32.4.1	CommonHttpClient	500
32.4.2	HttpException	505
33	JAVA REST Client: Instruction.....	506
33.1	Purpose.....	506
33.2	Requirements.....	506
33.3	Task	506
33.4	Activate the development mode	506
33.5	Create system paths	507
33.6	Create user exit class.....	507
33.7	Simple version.....	508
33.7.1	Extend user exit by REST call	508
33.7.2	Compile user exit.....	508
33.7.3	Test by calling the service MDUnits.list.....	509
33.8	Version with storage of REST service response in JSON file	509
33.8.1	Extend the user exit by file storage.....	509
33.8.2	Compile user exit.....	510
33.8.3	Test by calling the service MDUnits.list.....	510
33.9	Version with use of trust store	511
33.9.1	Create the server certificate in PEM encoding	511
33.9.2	Create the trust store.....	512
33.9.3	Extend user exit by trust store	513

33.9.4	Compile user exit.....	514
33.9.5	Test by calling the service MDUnits.list.....	514
34	MDS-MleOutboundUtil	515
34.1	Purpose.....	515
34.2	Requirements.....	515
34.3	Code example flat output data.....	515
34.4	Code example hierarchical output data	517
34.5	API reference	520
34.5.1	MleOutboundUtil.....	520
34.5.2	MleOutboundException	524
34.5.3	MleFileData	525
34.5.4	RecordData	526
34.5.5	BeginOutboundTaParam.....	526
34.5.6	EndOutboundTaParam.....	527
34.5.7	BeginOutboundTaPacketParam	530
34.5.8	EndOutboundTaPacketParam	531
34.5.9	ControlRecordData.....	535
34.5.10	ProtocolFileData.....	535
34.5.11	DataRecordClassifier.....	536
34.5.12	ProcessingState	537
34.5.13	FetchOutboundConfigStructureParam.....	537
34.5.14	MleOutboundConfigStruct	538
34.5.15	ResetAllInProcessDataRecordsParam	548
34.5.16	ResetInProcessDataRecordRecursiveParam	548
35	Instructions Java Userexits mit IntelliJ IDEA.....	549
35.1	Purpose.....	549
35.2	Requirements.....	549
35.2.1	Versions	549
35.2.2	Basics.....	549
35.2.3	Licenses	549
35.2.4	Software and files.....	549
35.3	How to proceed	550
35.3.1	Install JDK Amazon Corretto.	550
35.3.2	Provide MPDV Compile Libraries	550

35.3.3	Install and setup the IntelliJ IDEA Community	550
35.3.4	The first project.....	553
35.3.5	Create user exits	554
35.3.6	Test the user exits	555
35.4	Result.....	557
36	Instructions Java User exit Hide columns	558
36.1	Purpose.....	558
36.2	Requirements.....	558
36.2.1	Versions	558
36.2.2	Licenses	558
36.2.3	Basics.....	558
36.2.4	Java development environment.....	558
36.3	How to proceed	559
36.3.1	Task	559
36.3.2	Source code boperson.BopersonList.....	559
36.3.3	Source code explanations	560
36.3.4	Create a class	562
36.3.5	Test the user exits	563
36.4	Result.....	565
36.5	Further information	566
37	Add Specific Metrics.....	567
37.1	General Notes	567
37.1.1	Introduction	567
37.1.2	Requirements	567
37.1.3	Naming.....	567
37.2	Example	569
37.2.1	Purpose.....	569
37.2.2	Task	569
37.2.3	Activate the development mode.....	569
37.2.4	Create user exit class	570
37.2.5	Version with timer	570
37.2.6	Version with timer and LongTaskTimer.....	572
38	Calling Services via PDM Dialogs	574
38.1	Overview	574

38.2 Tutorial	574
---------------------	-----

1 Overview

MES Development Suite Business Applications & Services provides functions to adjust the client MES Operation Center to your specific requirements.

This document describes the functions provided by the product MES Development Suite Business Applications & Services.

2 General Notes

2.1 Overview

This document provides general information on the development and configuration of software using the MES Development Suite.

You can use the MDS to customize existing software or to create your own applications.

2.2 Namespaces and scopes

2.2.1 Namespaces

Use namespaces to ensure that software of MPDV and of customers can be installed and operated free of conflicts in the system. Namespaces are implemented using naming conventions of identifiers.

If the customer makes customizations or extensions, the customer must use new identifiers in the customer namespace "u_???".

Category	Namespace	Explanatory notes
Customer	u_???	Names of artifacts, objects or other identifiers, which are created by the customer or specifically for the customer, must start with the prefix "u_". Case-sensitivity is not globally specified. You use upper or lower case letters according to the context. Example of a service name (upper case letter): U_MyObjectName.listObjects Example of a database table (lower case letter): u_myobjectname
MPDV	All other	Artifacts, objects or other identifiers, which are not included in the namespace of customers, are reserved for components of the standard system.

2.2.2 Scopes

The system contains different customization levels (scopes). You can therefore customize standard applications without changing the actual application. With this concept, you store the software artifacts in customization levels (scopes) that overwrite the artifacts of the standard.

The following scopes are provided:

Standard

MPDV delivers the standard applications in the standard scope.

Custom

In the custom scope, MPDV delivers customizations of the standard software in the standard namespace or customer-specific software in the customer namespace "u_???".

Local

Customers use the local scope to customize the applications. In the local scope, you can customize all applications, regardless of the namespace. The end customer is responsible for the content of the local scope.

Software artifacts of the special scopes take priority over the artifacts of the general scopes. The local scope has the highest priority, the standard scope the lowest priority.

Software artifacts of the special scopes override the artifacts of the general scopes. There are different forms of overriding artifacts:

- **Sequential execution:** With program code (e.g. user exits of services), all existing program parts of the different scopes are executed **one after the other**. First the program code of the general scope is executed, then the code of the special scope. In an ideal case, the underlying data structures are built in a way so that the more special scopes can undo actions of the general scopes.
- **Merge:** A file or object of the special scope is added to the object of the general scope. The individual contents of the special scope overwrite and add to the defined contents of the file or the object of the general scope. Example 1: you can add further columns to a service configured in the standard scope. Here, only the additional columns are listed in a configuration file in the custom scope. Example 2: the permitted operators of a service parameter can be overwritten because the service parameter of the general scope is completely overwritten with the contents of the more special scope.
- **Replace:** A file or object of the special scope completely **replaces** the object of the general scope, e.g. configuration files.

2.3 Important notes



Respect the namespaces and scopes.



Only use latin letters, underlines, numbers and dots in identifiers. Do not use a number as first character of an identifier. Only use dots if this is necessary to structure an identifier, e.g. to structure service parameters (e.g. "person.name").

Do not use special characters!

Special characters like hyphen, slant line or umlauts or characters outside of the ASCII character set can lead to errors in the system.



To finally install your extensions, always use an Update Package and the Maintenance Manager. If you have only deployed your extensions via "deploy by copy", the extensions can be deleted in the course of a later update using the Maintenance Manager.

3 MES Development Suite

The MES Development Suite provides functions to customize the HYDRA Client MES Operation Center according to your requirements. The sections in the following provide general background information that you require for customizations and other extensions.

3.1 Activating the MES Development Suite

To activate or deactivate the MES Development Suite, use the main menu, menu item *Extras* → *MES Development Suite*.

You can only activate the MES Development Suite (or the menu item), if the required function authorization and licenses are available. The licenses must be installed in the relevant system.

To activate the MES Development Suite, you must assign the function authorization "mds" to the user.

The following products must be purchased to activate the MES Development Suite.



For historical reasons, the **products** of the MES Development Suite are different to the **licenses**, which must actually be available in the system.

Product (price list)		License system) (in the
MDS-BAS	MES Development Business Applications & Services	MDS-BAP
MDS-RPD	MES Development Suite Report Design	MDS-RPB

One of the above licenses is enough to activate the menu item, but only with MDS-BAS, all available functions are available. With the product MDS-RPB, only the functions are available after activation that are required for the report design.

3.2 Applications on the MOC

The MOC provides many different functions. The functions are made available via applications. Applications can offer very different functions, but their structure is always the same. This is true for complex evaluation applications like the *Workplace overview* and for a simple editing dialogs like the one to edit *Units*.

These are the basic elements of an application:

- Toolbar with buttons to call functions

- Selection area or selection panel to parameterize data queries
- Area for detail applications where one or any number of detail application(s) may be presented.
- Data sources or DataControllers that provide data for the detail applications, i.e. that call (web) services on the server supplying the relevant data.

From a technical point of view, editing dialogs for creating, editing and copying of data records are also applications. But editing dialogs normally do not provide detail applications. They only have a single (selection) area to enter parameters that are used to call the relevant editing function (i.e. the relevant web service).

Note: Editing applications are normally generated using the application generator (included in the product "MES Development Suite – Business Applications").

3.3 Meaning of customization

You can use the MOC to create new applications and change existing functions via customization. You do not control the available functionality via programming, in particular with applications, but you edit customization files (.config), which are usually changed via specifically developed customization dialogs. These dialogs are integrated into the software and are enabled when you activate the MES Development Suite.

A separate section below provides general background information on customization settings and their distribution to the clients. Some special features of customization settings are described here, for example of applications or menu items.

The available authorizations of the user or the available licenses specify the changes that can be made to the customization settings. But in general, a normal user can also change these files and save a table layout that has been changed according to their own requirements, for example.

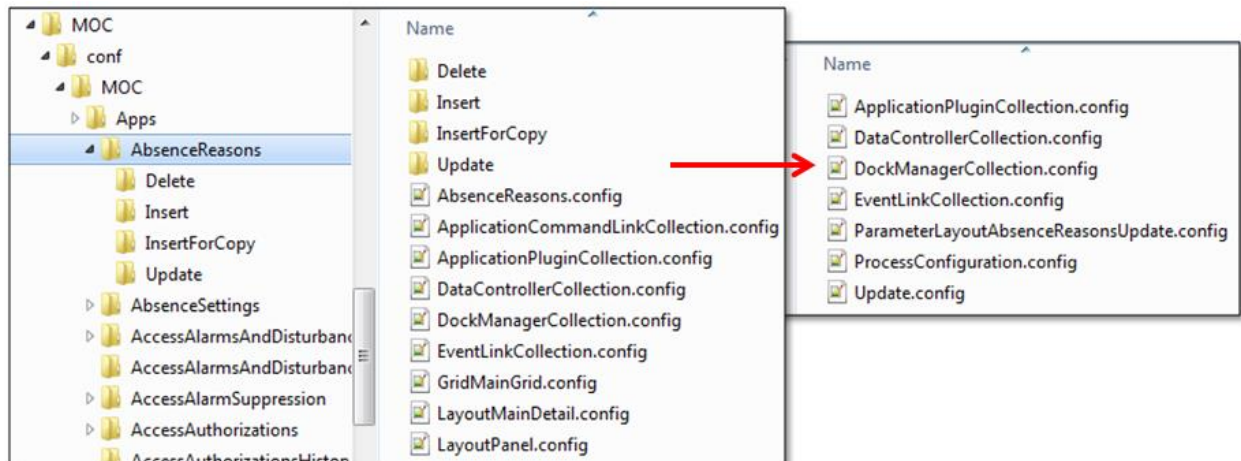
Customization files for applications

Some (main) applications are used to display data and other applications are used to edit data. For each main application, the directory %scope%\conf\Moc\Apps contains a directory with an unambiguous (English) name of the application. Editing applications are always assigned to a main application and are stored in a sub folder of this main application.

Example: The application *Absence reasons* is filed as a customization file in the "application" folder in the sub folder with the name of the related application ID, in this case AbsenceReasons. The editing dialogs required to edit absence reasons are stored in the sub folders "delete", "insert" and "update".

Main applications

The application directory contains all customization files that you require to customize an application. The specific files and the number of files are different for each application. This largely depends on the number of detail applications (tables, charts etc.) included in an application.



Contents of an application directory including editing applications.

The list below provides an overview of the most important files:

- <Id of the application>.config → Customization data for the application (title, help file, ...)
- LayoutPanel.config → Customization of the selection panel
- DockManagerCollection.config → Customization of the layout of detail applications
- DataControllerCollection.config → Customization of the data sources of the application
- ApplicationPluginCollection.config → Customization of the detail applications
- EventLinkCollection.config → Customization of the relations between data sources and detail applications
- ApplicationCommandLinkCollection → Customization of the toolbar.

For each detail application of a main application, a separate customization file is available. You can usually identify this customization file via the file prefix. For example

- Grid*. config -> detail application with table
- *Chart*.config -> Detail application with chart
- Pivot*.config -> Detail application with pivot
- Layout*.config → detail application for detail views

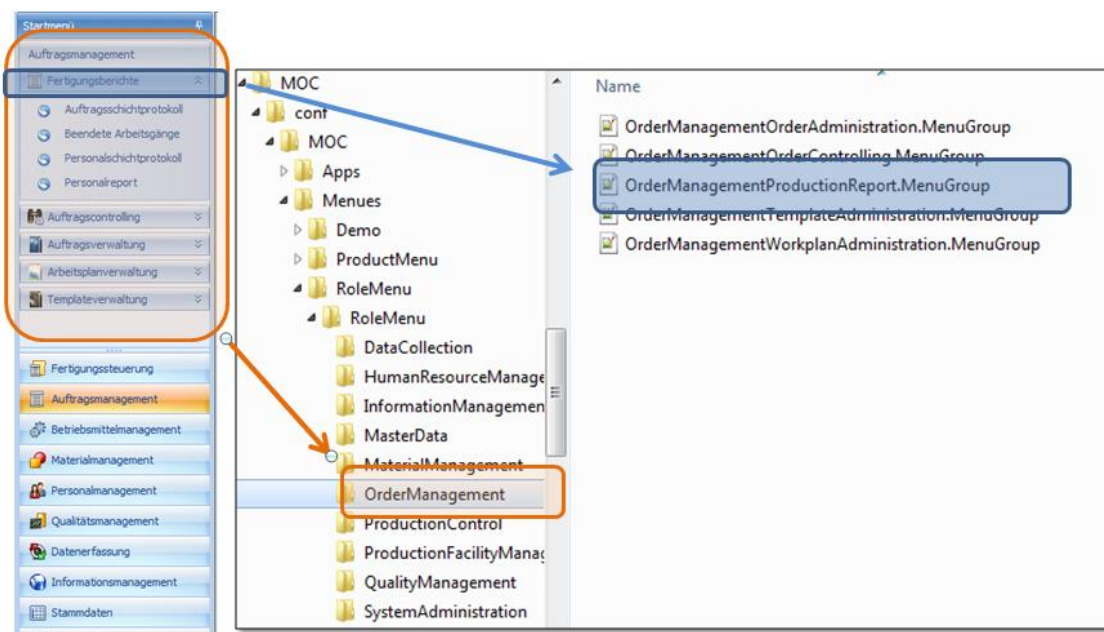
The application directory can include further files that are not listed here if special developments or plug-ins are available.

Editing applications

Editing applications are a special type of applications (see above). Their customization files are managed in sub folders of a main application. In addition to the files described above, the application directory of the main application contains an additional directory for each editing application including the customization files. Each editing application includes an additional file `ProcessConfiguration.config` that includes information on the process of calling this editing application. And the main customization file also includes additional and specific information.

Customization files for menus

On the MOC, you can use the menu item [Extras](#) → [Menu editor](#) to create and manage any number of menus. Each menu includes (main) menu items including sub items storing the actual functions. The customization files of the different menus are stored in the `%scope%\conf\Moc\Menues` directory. Each main menu is managed in a separate sub folder. The sub menus are mapped by a separate file containing the functions.



Example of a menu structure: The folder of the menu "RoleMenu" includes sub folders, e.g. the "OrderManagement" folder that manages the structure of the "order management" menu. The sub menu "production reports" and its functions are managed in the file "OrderManagementProductionReport.MenuGroup".

4 MOC configuration settings

Overview

A large number of configuration settings specify the layout and functions of the MES Operation Center (MOC): this includes, for example, the current window size, the language used or the number and order of columns in a table of a specific application. This section provides background information on the management of the MOC configuration settings and describes how custom configuration settings can be shared in the entire company.

4.1 Configuration settings and configuration levels

Every configuration setting may have up to four different values subject to the currently valid configuration level (scope). MOC has the following configuration levels (scopes):

- The “standard” configuration level (scope) includes values that MPDV provides for all users.
- The “custom” configuration level (scope) includes customized values that MPDV provides for all users.
- The “local” configuration level (scope) includes customized values that are provided locally for all users (e.g. by an administrator or key user).
- The “user” configuration level (scope) includes the values that a user has created individually.

At runtime, the MOC imports all values and selects the most specific value. If the “user” configuration level (scope) includes a value, this one will be used instead of the values from the levels “local”, “custom” or “standard”. This rule always applies up to the currently active configuration level, i.e. if the “Local” level is active, only values from the “Local”, “Custom” or “Standard” levels are used, but not from the “User” level.



Use the “local” configuration level (scope) to make global configurations intended for the use throughout the entire system.

If you want to make changes in the “local” configuration level (scope), activate the “local scope” at first. Then all changes, for example, to applications are written in the scope folder after saving, i.e. in the subfolder custom\conf of the MOC program directory.

4.2 Activation of a configuration scope

In general, the MOC is operated with the configuration scope “user”.

Use the system option “DefaultSettingScope” or the function “MES Development Suite”, “System Information Center” to set the configuration scope. (Note: the latter function is only available if corresponding licenses have been purchased.)

Set the system option "DefaultSettingScope" in the file MOC.ApplicationSettings.config or via a command line parameter.

The following row in the file MOC.ApplicationSettings.config specifies the option:

```
<add key="DefaultSettingScope" value="User" />.
```

Allowed values are "standard", "local", "custom" and "user". Restart the MOC, once you have made any changes.

As an alternative, set the configuration scope via the command line parameter

DefaultSettingScope=<scope> (e.g. in a link to moc.exe). Example

```
C:\Programme\MOC.exe DefaultSettingScope=Local
```



Only MPDV staff are permitted to use the configuration scopes "standard" and "custom". Normally, users do not need to make changes in these configuration scopes, as this would endanger system stability and, in particular, the system's ability to be upgraded.

4.3 Storage locations for configuration settings

All configuration values are stored in files, whereby a file usually combines a whole series of configuration values. The files of a configuration level are always compiled in a folder structure. By default, the following paths are used:

- The configuration values of the "user" configuration level (scope) are filed in the subfolder *user\conf* of the Windows user folder of the MOC application. You can find the folders here:
Windows 7: [LocalApplicationData]\MPDV\MOC\user\conf
- The configuration values of the "local" configuration scope are stored in the subfolder *local\conf* of the MOC program directory.
- The configuration values of the "custom" configuration scope are stored in the subfolder *custom\conf* of the MOC program directory.
- The configuration values of the "standard" configuration scope are filed in the subfolder *conf* of the MOC program directory.

You can find the currently applicable storage locations via the MOC function "Help" => "System information" => "System" tab => "Configuration scopes"

You can change the storage locations for configuration scopes by configuring the system options in the file MOC.ApplicationSettings.config. The sections that follow describe these options.



Note that the MOC update process only uses the standard values for storage locations. If you change the storage locations, you are responsible for making sure that software updates are also installed in the new folders.



If you change the storage location, please note that the application start might be slowed down when files are loaded via network.

4.3.1 User data

The system option “UserDataDirectory” specifies where user data, i.e. the configuration values changed by the user are stored. You can use placeholders when you define paths.

Default value:

```
<add key="UserDataDirectory" value="$ApplicationData\user\" />
```

Note: \$ApplicationData refers to the data directory of the MOC application. In Windows 7 this is the folder C:\Users\<user>\AppData\Roaming\MPDV\MOC\.

Allowed placeholders are:

- %HYDRAUSER%: name of the logged user
- %WINDOWSUSER%: the name of the registered Windows user
- %HYDRASYSTEM%: the name of the system the user is logged on to.

Example:

```
<add key="UserDataDirectory" value="\\dataServer\moc\users\%hydrauser%" />
```

4.3.2 System-wide (local) changes

The system option “LocalConfigurationDirectory” determines where configuration data of the “local” configuration scope is stored. This configuration scope includes individual or local changes applicable to the entire system.

Example:

```
<add key=" LocalConfigurationDirectory" value="\\dataServer\moc\local\" />
```

Note: In this case, you cannot use the placeholders described in section 4.3.1!

4.3.3 Customizations by MPDV

The "CustomConfigurationDirectory" system option controls where the configuration data of the "Custom" configuration level is stored that contain the customizations provided by the MPDV.

Example:

```
<add key=" CustomConfigurationDirectory" value="\\dataServer\moc\custom\" />
```

Note: In this case, you cannot use the placeholders described in section 4.3.1!

4.3.4 Notes on application configurations

Each application has a separate subfolder for configuration files in the subfolder conf\MOC\Apps of the corresponding "scope folder". For example, the settings of the application "Workplaces / Resources" with the application ID "workplaceoverview" are located in the following folders:

Configuration scope	Folder
User	C:\Users\<cbu>\AppData\Roaming\MPDV\MOC\user\conf\Moc\Apps\WorkplaceOverview
Local	C:\Programme\MPDV\MOC\local\conf\MOC\Apps\WorkplaceOverview
Custom	C:\Programme\MPDV\MOC\custom\conf\MOC\Apps\WorkplaceOverview
Standard	C:\Programme\MPDV\MOC\conf\MOC\Apps\WorkplaceOverview

The scope of a configuration value depends on the folder. Consequently, you can also transfer changed values to the "local scope" by copying files from the "user" directory to the "local" directory.

If you click on the save function in an application, only the changes made are saved. This means that the folder of the "local" configuration scope does not include the entire application, but only the contents deviating from the "standard" configuration scope.

If you load configurations, the folder that includes the settings file determines the configuration scope of a configuration value. If you copy files from a "user" directory to the "local" directory, you can transfer changed values to the "local" configuration scope.

4.4 Distribution of configuration settings

To deploy an application configuration changed in the “local scope” to other MOC installations, copy this configuration to the folder of the “local” configuration scope of the required installations.

1. Save the changes in your MOC installation.
2. Create update package:
Use the MOC Update Package Creator to create an update package and to deploy the new configuration. Start the MOC Update Package Creator via the MOC function [Extras](#) → [Generate update package](#).
3. Install update package on the server:
Use the Maintenance Manager to install the created update package.
4. Update the other MOC installations:
Use the MOC updater to update the MOC clients.

You can find further information in the sections dealing with [MOC Update Package Creator](#) and [Maintenance Manager](#).

4.5 Configure syntactic types

Overview

Menu	-
Transaction code	syty
Function authorization	syty

Syntactic types control at the MOC the standardized presentation of data at all locations via a central definition. Consequently, you can use the syntactic type for quantities to specify the number of decimal places. This setting affects all quantities displayed in the MOC.

Most syntactic types can only be changed by MPDV or customers/partners if they use the MES Development Suite.

However, the application "Configure syntactic types" additionally offers the option to configure selected important syntactic types directly on the customer system without having to order customizations or use the MES Development Suite.



The application "Configure syntactic types" is an expert application and only available in English. Usually, MPDV consultants use this application to change specific syntactic types of MOC data according to the customer's requirements.

The application "Configure syntactic types" uses the methods of the MES Development Suite.

The document "MES Development Business Applications & Services" (MDS-BAS) provides further basic information on the MES Development Suite. You require this knowledge when working with the application "configure syntactic types".

The screenshot shows the 'Property Configuration' window. It has two main sections: 'Load from' and 'Save to'. Both sections have radio buttons for 'Standard', 'Custom', and 'Local'. The 'Load from' section has a dropdown menu showing 'ConfigurableSyntacticType.Properties.xml' and a 'Load' button. The 'Save to' section has a dropdown menu showing 'ConfigurableSyntacticType.Properties.xml' and a 'Save' button. There is also a 'Create update package' button. Below these sections is a table with the following columns: Acronym, Label, Unit Label, Output Format, Input Format, and Length. The table contains 12 rows of data.

Acronym	Label	Unit Label	Output Format	Input Format	Length
> cyde	lkcycle	lkHrsPer 1000	{0:mpdv_cycletime}		10
quantity	lkquantity		n0	[0-9]{0,10}	10
quantity_negative				-?[0-9]{0,10}	0
st_iw_te	lkoperation.plan.te	lkHrsPer 1000	{0:mpdv_te}		10
st_iw_teb	lkoperation.plan.teb	lkHrsPer 1000	{0:mpdv_te}		10
st_iw_tr	lkoperation.plan.tr	lkHrs	{0:mpdv_te}		10
st_iw_trb	lkoperation.plan.trb	lkHrs	{0:mpdv_te}		10
st_mf_te	lkoperation.plan.te	lkHrsPer 1000	{0:mpdv_te}		10
st_mf_teb	lkoperation.plan.teb	lkHrsPer 1000	{0:mpdv_te}		10
st_mf_tr	lkoperation.plan.tr	lkHrs	{0:mpdv_te}		10
st_mf_trb	lkoperation.plan.trb	lkHrs	{0:mpdv_te}		10

This document illustrates the procedure step by step. Consequently, even unexperienced users should be in the position to make the most important changes independently.



You can configure the syntactic types included in the client file *ConfigurableSyntacticType.Properties.xml*.

Definition of terms

Technical terms from the MES Development Suite and system administration are used here in connection with this application.

Scope

A scope describes a level at which configuration and programming can be performed in the system:

Standard

MPDV uses the *standard* scope to deliver standard products.

Custom

MPDV uses the *custom* scope to deliver customizations that complement or overwrite the standard.

Local

Customers or partners can use the *local* scope to make changes that complement or overwrite the *custom* and the *standard* scope.

Update package

An update package includes programs and configurations the Maintenance Manager first installs on the server. Then the MOC updater distributes the MOC data from the server to the other MOC clients. Usually, this is an automatic process.

Update Package Creator

The Update Package Creator is a tool used with the MOC to pack locally created customizations in an update package. The application "configure syntactic types" uses a simplified and restricted version of the Update Package Creator.

Maintenance Manager

Web application used to install update packages on the server. Usually, your IT department is familiar with the procedure as MPDV regularly sends updates that are installed via the Maintenance Manager.

Overview of the procedure

This section provides a brief overview of the steps you have to carry out to change syntactic types. The sections that follow provide further details.

1. Load configuration: load the existing configuration of syntactic types from the system.
2. Change table data: change the configuration.
3. Save the changes.
4. Create update package: create an update package to distribute the new configuration.
5. Install update package: use the Maintenance Manager to install the update package.
6. Update MOC clients: use the MOC updater to update the MOC clients.

1) Load configuration

Load the syntactic types from the system before you can change them. You can select the scope:

Local

Loads the syntactic types you have already changed. In case you have not changed the syntactic types, the system loads the syntactic types provided by MPDV from the *custom* scope or the *standard* scope.

Custom

Loads the changed syntactic types provided by MPDV. In case MPDV has not changed the syntactic types, the system loads the standard configurations. This process does not include the syntactic types you have already changed.

Standard

Loads the syntactic types from the standard scope. This process does neither include the customizations provided by MPDV nor the changes you made.

As a general rule, you should choose the following methods for loading syntactic types:

- Create or change the customizations you made: load configurations from the *local* scope.
- Discard the changes you made and reset configurations to the version delivered by MPDV: load configurations from the *custom* scope.

2) Change table data

You can make changes to the table. In most cases only changes in the columns *OutputFormat*, *InputFormat* and *Length* are required.

Table columns

Acronym

Name of the syntactic type. You should not change this value.

Label

Labeling of input fields displayed in front of the input fields. By default, "language keys" are used for the labels. These keys are translated depending on the language set in the MOC. If required, you can also enter customer-specific texts instead of "language keys". But these texts will not be translated. Customers using the product MES Development Suite (MDS-BAS) can define their own language keys. There are some rare places in the MOC that are not affected by changed labels. But the entire system will be affected if you use the MES Development Suite (MDS-BAS) to customize the *language key* of the label.

UnitLabel

The UnitLabel is the labeling displayed behind the input field. The same rules apply as for the label.

OutputFormat

Output format for data. A separate section describes expedient output formats.

Times and durations are internally stored in the system as integer seconds. If you convert times or durations during input or output formatting to formats other than hours, minutes and seconds (HH:MM:SS), the conversion may not be possible without losses. For example, this applies to the use of the "mpdv_calc" format and the classic industry minute display:



When converting from seconds to hours (division by 3600), decimal numbers with an infinite number of decimal places can occur, which inevitably have to be rounded when displayed on the client. Example: 20 minutes = 1200 seconds = 0.333333... hours. If the value is rounded to three decimal places, you calculate backward as follows: $0.333 \times 3600 = 1198.8$ seconds.

Depending on how the client rounds, the internal value is then no longer 1200 seconds, but 1199 or 1198 seconds.

InputFormat

Input format to check user input. Normally, the MOC automatically defines the appropriate input format that matches the output format. You should only indicate the InputFormat if additional checks are required. So-called "regular expressions" specify the InputFormat. Regular expressions are standard in software development. You can find further information on regular expressions on the Internet.

Length

Field length: number of characters.

Configuration of quantities

You can change the output and input formats for quantity fields:

Output format

n<x>: shows the number with thousands separator. <x> indicates the number of decimal places.
e. g. n1, n2 ...

f<x>: shows the number without thousands separator. <x> indicates the number of decimal places.
e. g. f1, f2 ...

Input format (examples)

[0-9]{0,10}

Integer with up to 10 digits. No minus sign allowed; only positive values supported.

-?[0-9]{0,10}

Integer with up to 10 digits and optional, leading minus sign.

-?[0-9]{0,9}\R.?[0-9]{0,3}

Optional sign, up to 9 places before decimal point and optionally up to three decimal places.

Configuration of cycles

You can edit the label, UnitLabel, OutputFormat and length. Meaningful values are assigned by default to "UnitLabel" and "OutputFormat". The following configurations are useful:

Unit Label	Output Format
lkHrsPer1000	{0:mpdv_cycletime}
lkUnitsSecondsPerOne	{0:mpdv_cycletime_sec_cycle}
lkUnitPiecePerHour	{0:mpdv_cycletime_piece_hour}

Configuration of single piece specifications (te, teb)

- Syntactic types starting with "st_iw_" affect incentive pay applications.
- Syntactic types starting with "st_mf_" are used in all other applications.

You can edit the label, UnitLabel, OutputFormat and length. The MOC automatically assigns expedient values to the *InputFormat*.

You can use the format "mpdv_calc..." to perform calculations for the output format. The basis of the calculation is the value available in the database, which is always in "seconds per 1000 pieces".

UnitLabel	OutputFormat
lkHrsPer1000 = [h/1000]	{0:mpdv_te}
lkUnitsSecondsPerOne = [sec/1]	mpdv_calc;MULT=1;DIV=1000;INVERSE=false;FORMAT=f3
lkUnitMinutesPerPiece = [min/1]	mpdv_calc;MULT=1;DIV=60000;INVERSE=false;FORMAT=f3
lkUnitPiecePerHour = [1/h]	mpdv_calc;MULT=1;DIV=3600000;INVERSE=true;FORMAT=f3
[1/min]	mpdv_calc;MULT=1;DIV=60000;INVERSE=true;FORMAT=f3
...	...

Output formats with calculation allow you to calculate the inverse by setting "INVERSE=true". Consequently, you can display data as "quantity per time" instead of "time per quantity".



First use the multiplier and divisor. Then calculate the inverse. If you want to display the *pieces per minute*, you have to divide the t_e in [sec/1000] by 60000 to get [*minutes/piece*]. Then calculate the inverse to indicate [*pieces/minute*].

Configuration of setup specifications (tr, trb)

- Syntactic types starting with "st_iw_" affect incentive pay applications.
- Syntactic types starting with "st_mf_" are used in all other applications.

You can edit the label, UnitLabel, OutputFormat and length. The MOC automatically assigns expedient values to the *InputFormat*.

You can use the format "mpdv_calc..." to perform calculations for the output format. The value existing in the database is the basis for calculations. This value is stored in "seconds".

UnitLabel	OutputFormat
lkHrs	{0:mpdv_te}
[min]	mpdv_calc;MULT=1;DIV=60;INVERSE=false;FORMAT=f3
...	...

3) Save changes

Click the "save" button to save changes to the local MOC client in the local scope. You should not change the file name.

4) Create update package

Click the "Create update package" button to start the Update Package Creator. The input fields are populated with default values. Enter the following settings:

Path (folder that includes the generated update)

Specify the folder where the update package is stored. The folder must exist already.

Update name

We recommend appending a unique ID to the update name. You can add date and time, for example: "SyntacticTypes20170726_1342".

Version number

Optionally, you can indicate a version number that will be displayed in the Maintenance Manager.

5) Install update package

The created Update Package is installed like any other update using the Maintenance Manager.

6) Update MOC clients

Usually, the MOC updater automatically downloads the updates to the MOC clients. You can use the menu to search immediately for updates (Help --> Search for updates). Once all MOC clients have been updated, the changes to the syntactic types take effect.

4.6 Change the MOC logging

By default, the client log files are stored in the user directory of the Windows user who runs the MOC. The log files are stored in the following directory, if this has not been changed:

[LocalApplicationData]\MPDV\MOC\log\

4.6.1 Change the storage location of the log file

We recommend separating the log entries of different MOC instances in order to facilitate the failure analysis. To change the storage location, create a new file named "NLog.user.config" or "NLog.local.config" with the following content in the main directory of the affected MOC installation (e.g. "C:\Program Files (x86)\MPDV\MOC") :

```
<?xml version="1.0" encoding="utf-8"?>
<nlog xmlns="http://www.nlog-project.org/schemas/NLog.xsd" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" autoReload="true">
  <variable name="logpath" value="${specialfolder:folder=ApplicationData}\MPDV\MOC\log" />
</nlog>
```

Change the path highlighted in red and enter your required storage location (e.g. \${specialfolder:folder=ApplicationData}\MPDV\MOC\log). Ensure that the local user has write access to this folder.

Use the file NLog.user.config for customizations that only apply to this specific workplace (client). Whereas you should use the file NLog.local.config to distribute the modifications to all workstations (clients) via the update package.

4.6.2 Change the log level

In some rare cases, you might have to increase the MOC log level. To do so, create the file "NLog.user.config" with the following content as described in the previous section:

```
<?xml version="1.0" encoding="utf-8"?>
<nlog xmlns="http://www.nlog-project.org/schemas/NLog.xsd" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" autoReload="true" globalThreshold="Trace">
</nlog>
```

Once you have sent the log file generated with the increased log level, you should delete this file because the increased log level can have a negative effect on the performance.

5 MOC Update Package Creator

5.1 Overview

Menu	Extras → Generate update package
Transaction code	
Function authorization	mupc

You can generate update packages with the "MOC Update Package Creator". You can create packages for the MOC client and for the server. You use the Maintenance Manager to install the generated update packages on the server. The MOC updates are distributed via the MOC updater.

You mainly use the MOC Update Package Creator to generate updates for the deployment of applications and services that you have developed using the MES Development Suite. You can also use the generated update packages to distribute the configurations that you have made locally in the MOC.

The MOC Update Package Creator is available in German and English. The MOC Update Package Creator is an independent Windows application. Therefore, the Update Package Creator does not use the language set in the MOC, but the language set in the operating system.

Experienced users can start the MOC Update Package Creator via the Windows command line from the installation directory of the MOC or via a link. Enter the language using command line parameters.



MOCUPC.exe path=<MOC_inst_path> l=<language> s=<scope>

<MOC_inst_path>: path of the MOC installation

<language>: language ("en" or "de")

<scope>: "local" or "custom" (custom only for MPDV or partner)

Example: D:\Moc>MOCUPC.exe path=d:\moc l=en s=local

After start of the MOC Update Package Creator, two tabs are available: One tab for the generation of MOC packages and one for server packages. You must generate separate update packages for MOC and server.

5.2 Generate MOC packages

The screenshot shows the 'MOC Update Package Creator' window. It has two tabs: 'MOC-Package' (selected) and 'Server-Package'. The main title is 'Create MOC update package'. The 'Path-Selection' section contains two text boxes with browse buttons: 'Folder, which contains the relevant files for the update:' with path 'D:\MOC\' and 'Folder that contains the generated update, once it has been created:' with path 'D:\tmp\MOC-Packages'. The 'Update-Information' section has 'Update filename:' (U_Units), 'Version number' (1.0.MOC.10), and 'Brief description of the update:' (Customized App Units). Below these are 'Load', 'Save', and 'Delete' buttons. The 'Domain-Information' section has 'Configuration level:' with radio buttons for 'local' (selected) and 'custom'. Below is a tree view showing a hierarchy: 'local' (expanded) -> 'conf' (checked) -> 'MOC' (checked) -> 'Apps' (checked) -> 'TerminalConfiguration' (unchecked) -> 'Units' (checked) -> 'MainMenu.config' (unchecked) -> 'System.config' (unchecked). There is also a 'resources' folder (unchecked). At the bottom is a 'Generate update package' button.

For the generation of MOC packages, you must select the files in a tree view that you want to include in the update. You can save the selected files in "update package profiles". Later, you can reload these profiles and generate a new package with the same files.

The packages include a version number. The last created version number is saved in the "update package profile". If you create a new version of the same package, you can easily identify the last version and increase the number by one.

Field description

Folder, which contains the relevant files for the update

This field shows the installation directory of the MOC, which includes the configurations or adjustments. On start of the MOC Update Package Creator, the system populates this field with the correct entry. The folder with the relevant files for the update includes the subfolders with the selected configuration scope ("local" or "custom").

Folder that contains the generated update, once it has been created

The generated updates are stored in this folder. This folder must already exist. It is not created automatically.

Update file name

Name of the update file. The file name of the update is also the name of the update package profile if you save the profile.



The update name must not include blank characters or umlauts.

Brief description of the update

This description is, e.g., displayed in the update overview of the Maintenance Manager.

Version number

MOC packages include version numbers. You can only install packages with the same name in the Maintenance Manager, if they have a higher version number. The version number is automatically populated. The version number that has been created last is entered here. The update overview of the Maintenance Manager also shows the version.

Version numbers should have the following format: "<x>.<y>.<customer abbreviation>.<zzzz>", e.g. "1.1.CUST.17".

Configuration scope

The configuration scope "local" includes the developments of the customer. The configuration scope "custom" is used by MPDV or the MPDV partners.

File selection (tree view)

Select the files in the tree view that you want to include in the update.

Load/Save/Delete Update Package Profile

The system uses the name of the field "Update file name" for the administration of the update package profiles. The saved profiles include the following information:

- Folder that contains the generated update once it has been created
- Update file name
- Short description of the contents

- Last created version number
- Configuration scope
- The files that you have selected in the tree view

If you load an existing profile, you should check in the tree view, if new files have been added since the last time the profile was saved in the directory structure. If required, you must activate the new files and add them to the profile by saving the profile again.

Click the button "Generate update package" to create the update package. The Maintenance Manager can then install the update package in the server.

In the MOC clients, the updates are usually installed with the MOC Updater. You can immediately search for updates, if you select in the menu: Help → Search for updates.

5.3 Generate server packages

The screenshot shows the 'MOC Update Package Creator' window with the 'Server-Package' tab selected. The window is titled 'Create server update package'.

Path-Selection

Development directory, which contains the domains for the update:
Path: ...

Folder that contains the generated update, once it has been created:
Path: ...

Update-Information

Package name: Brief description of the update:

Domain-Information

Domain(s) in the development directory: Set standard version:

	Domain	Add	Allocate version (only by adding)
☐	AbsenceReasons	☐	
☐	IncentiveBonuses	☐	
☒	MyTest	☒	1.1.CUST.1

You use "Server packages" to distribute server data of the MPDV repository. You can also use server packages to distribute JAVA artifacts that you have developed yourself using the MES Development Suite (MDS). A list view shows the domains created in the repository.

Activate the domains in the list view that you want to include in the update package. The system assigns version numbers to the domains. With server packages, each domain gets an own version number. The installation in the Maintenance Manager can only be performed, if the domains in the server package have a higher version number than the domain versions that are already installed.

Field descriptions

Development directory, which contains the domains for the update

This field shows the directory where the MPDV Repository Client (MRC) stores the server domains. It is the same path that is entered as "Server source" in the MRC workset.

If you want to use server packages to distribute configuration files and JAVA artifacts that you have developed yourself using the MES Development Suite (MDS), you must store these files and artifacts in the domain directory structure of the MRC.

<Source directory>/

<Domain1>/

Interpreter/

List Interpreter and BAPI interpreter configurations

ReferenceData/

Reference data configurations

ExtSvc/

JAVA Class files of the Simple External Services (including package structure)

ExtSvcMapping/

Mapping configuration files of the SimpleExternalServices

User exit/

JAVA Class files of the User Exits (including package structure)

<Domain2>/

...

Folder that contains the generated update, once it has been created

The generated updates are stored in this folder. This folder must already exist. It is not created automatically.

Package name

Name of the update file. The Update Package Creator automatically adds "-server" to the file name. You can therefore use the same name for the client and the server package.



The update name must not include blank characters or umlauts.

Brief description of the update

This description is, e.g., displayed in the update overview of the Maintenance Manager.

Set standard version,**Complete column "Version" in the domain table**

With the server package, a version is required for each domain. If you enter a version number in the field "Set standard version", this version number is assigned to all domains that are activated in the list. You can also assign individual version numbers for activated domains. If the field "Set standard version" is left empty, you must assign an individual version to each activated domain.

Version numbers should have the following format: "<x>.<y>.<customer abbreviation>.<zzzz>", e.g. "1.1.CUST.17".

Domain selection (list view)

The list shows the domains that are included in the development directory. Activate the domains in the list view that you want to include in the update package. The system assigns version numbers to the domains.

Click the button "Generate update package" to create the update package. The Maintenance Manager can then install the update package in the server.

The system stores the values of the two fields "Development directory, which contains the domains for the update" and "Folder, which contains the relevant files for the update". When the MOC Update Package Creator is started the next time, the application populates these two fields with the values that were last entered in this field.

6 Update Packages for the Maintenance Manager

6.1 Overview

Update Packages are used to distribute new features via the Maintenance Manager. The following chapter describes the structure of these files.

An update package is an archive file (zip) and can have any name. The file extension *upd* is set by default.

The package can include the following subfolders:

client: MOC client package in **.upd* format (0-n)

→ You can also deploy these **.upd* files individually.

java: java server package in **.upd* format (0-n)

→ You can also deploy these **.upd* files individually.

server: server packages in **.upd* format (0-n)

→ You can also deploy these **.upd* files individually.

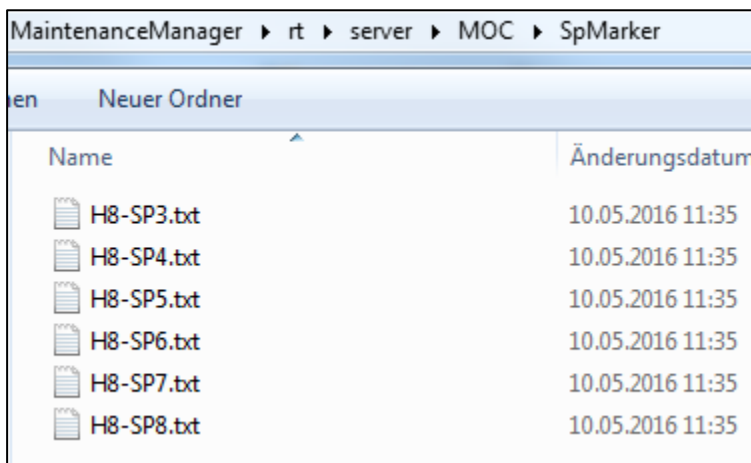
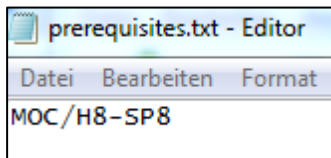
You may find a *prerequisites.txt* file in addition to the folders mentioned above. The *prerequisites.txt* file includes information on the required service pack or hotfix version.

You can install update packages via the *Package Deployment* in the Maintenance Manager.

Name	Änderungsdatum	Typ
 example_pack.upd	23.05.2016 11:48	UPD-Datei
Name  client  java  prerequisites.txt		

prerequisites.txt

The file *prerequisites.txt* describes the requirements, i.e. which service pack is needed. You can check in *MaintenanceManager\rt\server\MOC\SpMarker* if the required file exists.



6.2 Black list for MOC updates using Maintenance Manager 2

The update process and update behavior of an MOC installation on a workstation PC have changed if you use Maintenance Manager 2 and the MOC Updater. In contrast to the previous MOC update, where files were only supplemented or updated, the new update process also deletes files.

During the update process, the local MOC installation is compared/synchronized with the reference version in *Maintenance Manager 2*. All files that do not correspond to the server's reference version are overwritten or deleted. This also applies to files created or modified as part of the development of customizations with the MES Development Suite.

To avoid data loss, you can exclude directories or files from the update process. For this purpose, enter the relevant files or directories in an MOC black list. You can only enter files and directories that are located in the MOC main directory!

You can create the black list using any text editor. Save the file as "Blacklist.txt" in the home directory of the MOC Updater `<MOC installation directory>\update\` so that the MOC Updater can process the file.

The file structure must be in JSON format. Enclose each entry in quotation marks. Separate multiple entries via comma.

Example of a *Blacklist.txt* file:

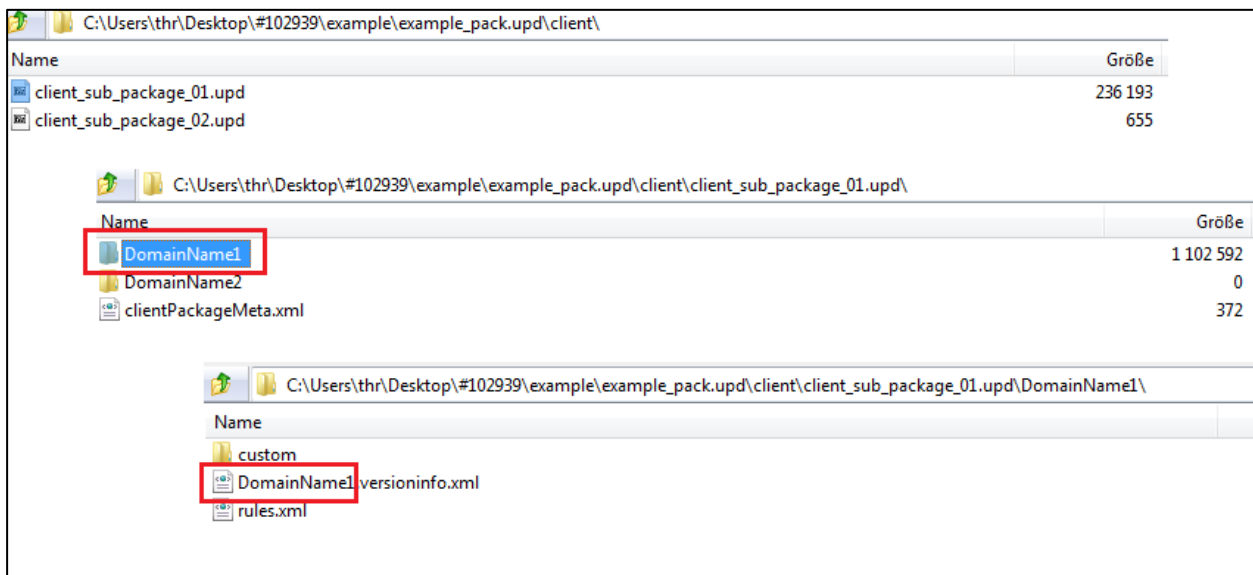
```
{
  "DirectoryBlacklist":
    ["local\\",
     "custom\\",
     "conf\\MOC\\Apps\\"],
  "FileBlacklist":
    ["conf\\DoNotDelete.txt",
     "conf\\DoalsoNotDelete.txt"]
}
```

Important: If you develop own applications in the local scope, enter the directory of the local scope in the black list to prevent the local developments from being deleted by the MOC Updater:

```
{
  "DirectoryBlacklist": ["local\\"]
}
```

6.3 Structure of MOC Client Package

An MOC update package is structured as follows:



clientPackageMeta.xml:

The *clientPackageMeta.xml* in the root directory of the *.upd folder includes information on the contents of the update package: name of the update package without file extension, description, date of creation, name of the application, 1-n domains.

```
<?xml version="1.0" encoding="utf-8"?>
<packageMeta>
  <name>client_sub_package_01</name>
  <description>Description package 01</description>
  <creationDate>2016-05-23T09:30:08</creationDate>
  <applicationName>MOC</applicationName>
  <domain version="1.0.CUST.00001">DomainName1</domain>
  <domain version="1.0.CUST.00002">DomainName2</domain>
</packageMeta>
```

#.versioninfo.xml

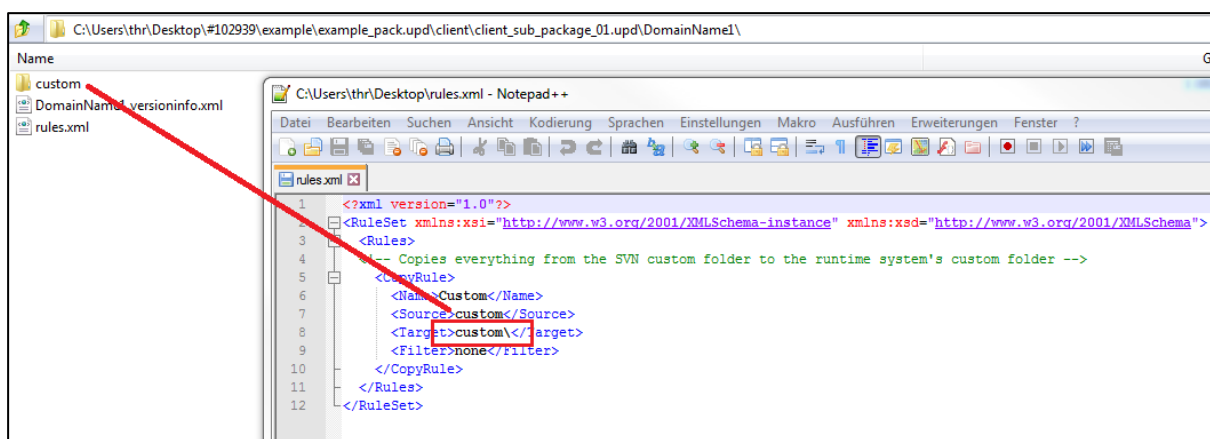
You can find the **#.versioninfo.xml** below the higher-level domain. Enter the domain name for the placeholder "#". Enter the correct customer ID and the domain as object ID in this file.

```
<?xml version="1.0"?>
<mpdvVersion xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <CustomizedVersion>0</CustomizedVersion>
  <MajorVersion>1</MajorVersion>
  <MinorVersion>0</MinorVersion>
  <Revision>45661</Revision>
  <CustomerId>CUST</CustomerId>
  <ObjectId>DomainName1</ObjectId>
</mpdvVersion>
```

rules.xml:

You can find the *rules.xml* below the higher-level domain. This file includes 1-n copy rules. These rules define which file / which directory (source) is stored in which target directory. Use the filter to select specific files. If you only want to copy *xml* files, enter the following filter: "<filter>*.xml</filter>". This example copies the complete contents of the *custom* folder into the MOC runtime directory. Use the placeholder **#SERVER#** in the target, to store the files directly in *JHYDRADIR* after activation in the Maintenance Manager.

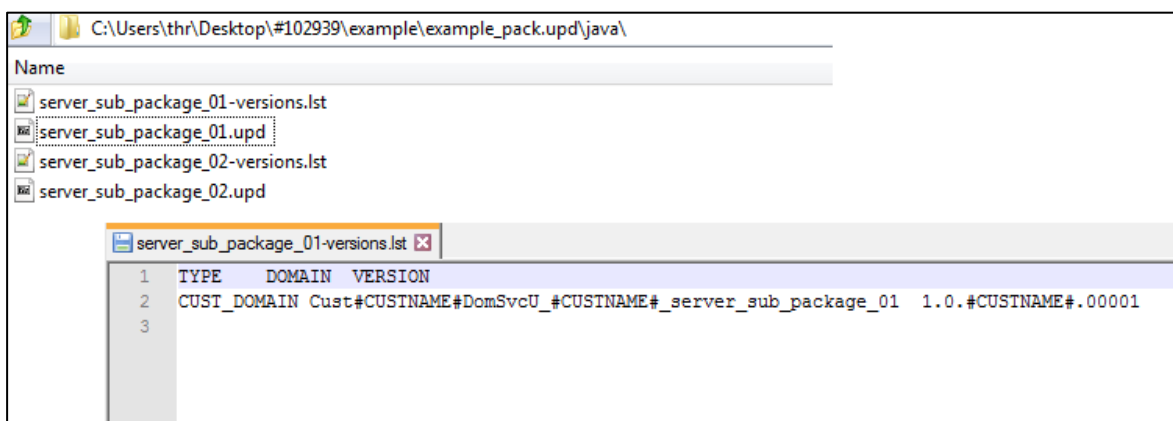
You can find further copy rules in the description of the java server packages.



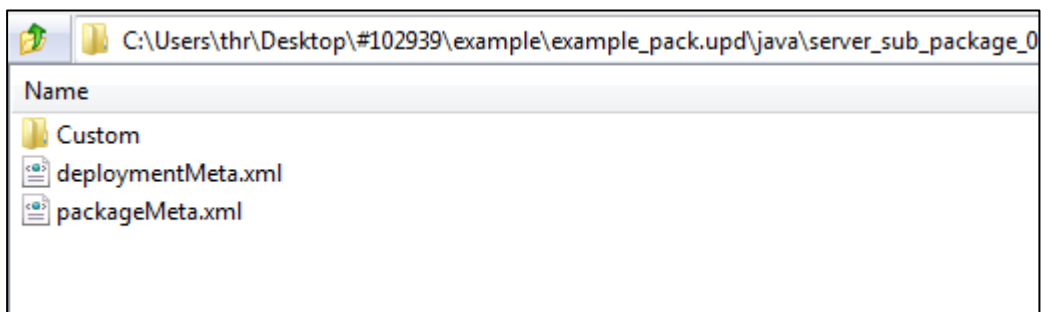
MaintenanceManager ▶ rt ▶ MOC ▶ custom ▶	
Neuer Ordner	
Name	Änderungsdatum
resources	09.05.2016 15:48
readme.txt	10.05.2016 11:37

6.4 Structure of Java Server Package

A server update package is structured as follows (the examples mentioned below sometimes include the placeholder #CUSTNAME#; replace this placeholder with the relevant customer name):



*.lst files are not relevant for the update package and are created for internal purposes only. This file is not mandatory.



deploymentMeta.xml:


```
<?xml version="1.0" encoding="UTF-8"?>
- <deploymentMeta>
  <warName>MocServices</warName>
  <confContextName>MOC</confContextName>
</deploymentMeta>
```

packageMeta.xml:

The *packageMeta.xml* in the root directory of the *.upd folder includes information on the contents of the update package: name of the update package without file extension, description, date of creation, 1-n domains including version, customer, type, path and name.

```
<?xml version="1.0" encoding="UTF-8"?>
- <packageMeta>
  <name>server_sub_package_01</name>
  <applicationName>MOC</applicationName>
  <description>Custom Service for sub package 01</description>
  <creationDate>2016-05-23T11:30:43</creationDate>
  <packageElement version="1.0.#CUSTNAME#.00001" customer="#CUSTNAME#" type="CUST_DOMAIN" path="Custom/Domains/Services/U_#CUSTNAME#_DomainName1"
    name="Cust#CUSTNAME#DomSvcU_#CUSTNAME#_DomainName1"/>
  <packageElement version="1.0.#CUSTNAME#.00002" customer="#CUSTNAME#" type="CUST_DOMAIN" path="Custom/Domains/Services/U_#CUSTNAME#_DomainName2"
    name="Cust#CUSTNAME#DomSvcU_#CUSTNAME#_DomainName2"/>
  <packageElement type="THIRD_PARTY_LIBS" path="ThirdPartyLibs" name="thirdpartylibs"/>
</packageMeta>
```

C:\Users\thr\Desktop\#102939\example\example_pack.upd\java\server_sub_package_01.upd\Custom\Domains\Services\U_#CUSTNAME#_DomainName1\	
Name	Größe
ExtSvc	0
ExtSvcMapping	0
Interpreter	0
MpdvCust#CUSTNAME#DomSvcU_#CUSTNAME#_DomainName1.xml	132
rules.xml	3 680

MpdvCust#CUSTNAME#DomSvcU_#CUSTNAME#_DomainName1.xml:

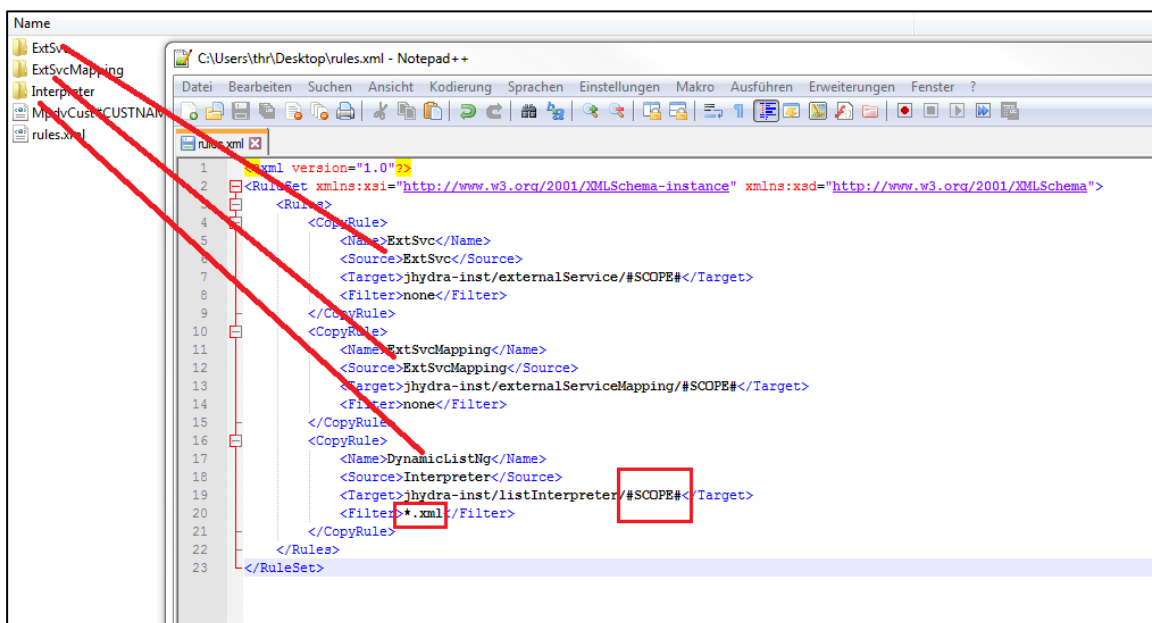
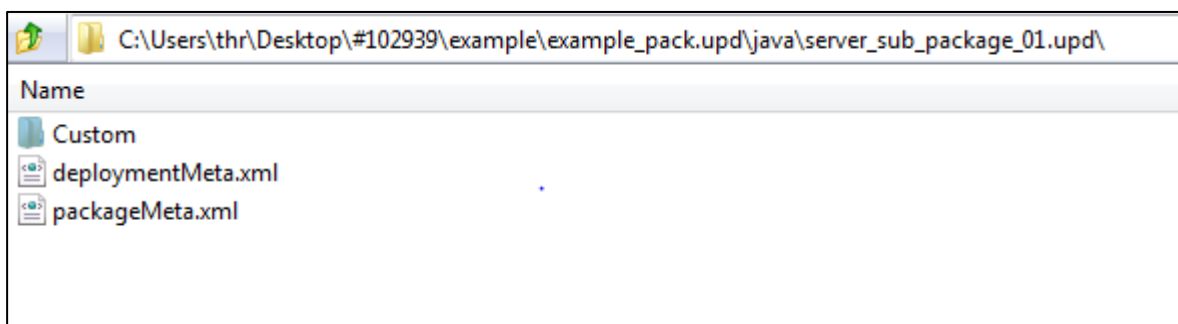
```
<?xml version="1.0" encoding="UTF-8"?>
- <projectMeta>
  <projectType>library</projectType>
  <useInWar>false</useInWar>
</projectMeta>
```

rules.xml:

You can find the *rules.xml* below the higher-level domain. This file includes 1-n copy rules. These rules define which file / which directory (source) is stored in which target directory. Use the filter to select specific files. If you only want to copy *xml* files, enter the following filter: "<filter>*.xml</filter>".

This example copies *ExtSvc*, *ExtSvcMapping* and the folder *Interpreter* to the *JHYDRADIR* (runtime directory of the Maintenance Manager) of the predefined subdirectory. The placeholder *#SCOPE#* is then replaced with the directory created in the root directory of the update package (e.g. custom, standard, local).

Use the placeholder *#CLIENT#* in the target to store the files directly in the runtime directory (MOC) after activation in the Maintenance Manager.

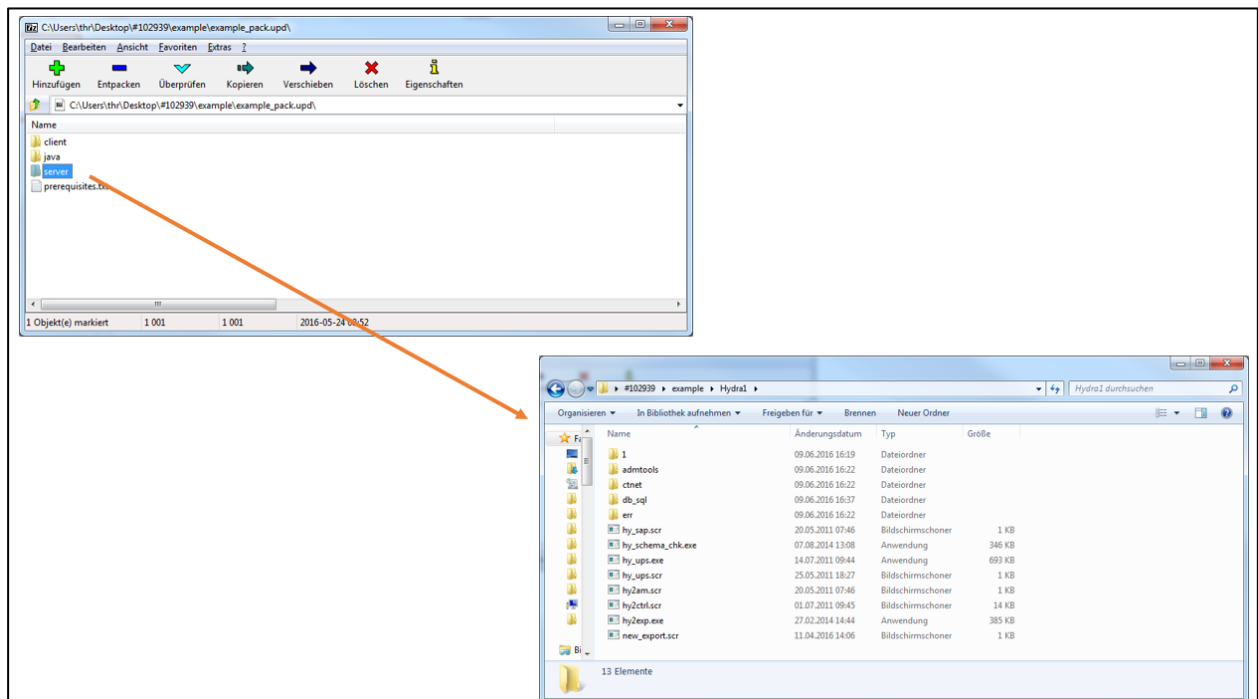


MaintenanceManager ▶ rt ▶ server ▶ MOC ▶ jhydra-inst ▶ listInterpreter ▶		
ür ▼ Brennen Neuer Ordner		
Name	Änderungsdatum	Typ
custom	12.05.2016 16:08	Dateiordner
standard	10.05.2016 11:36	Dateiordner

Interpreter copied with custom scope to the runtime directory.

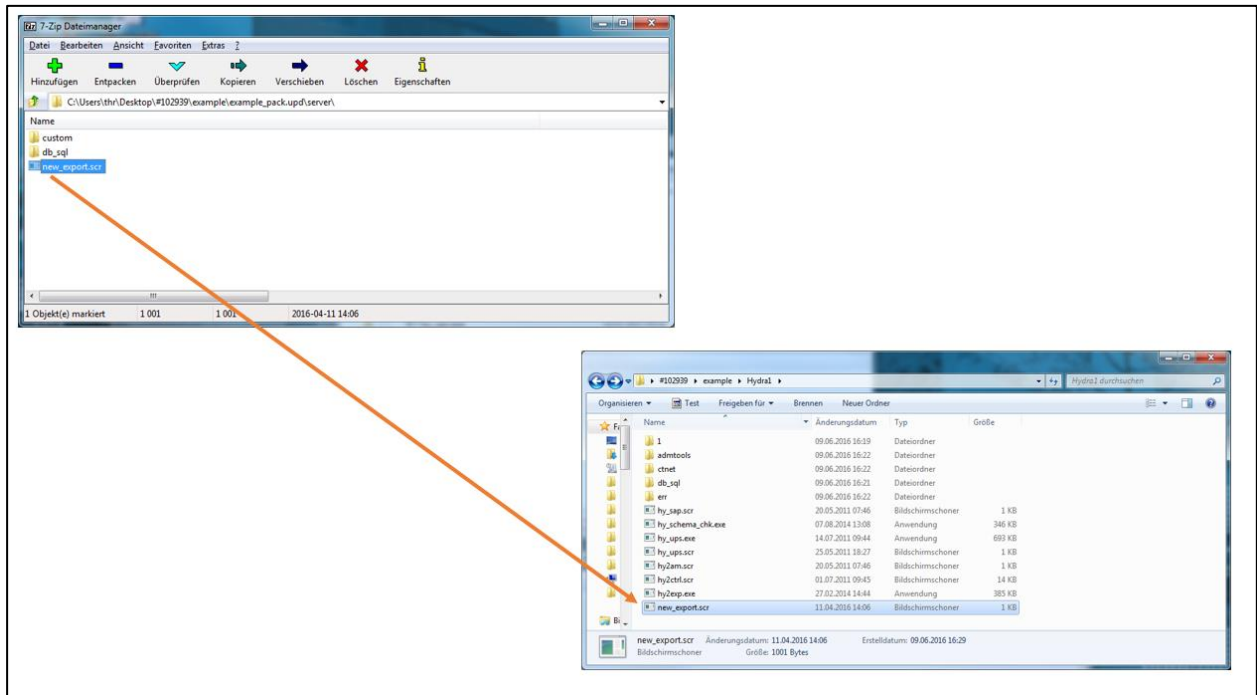
6.5 Structure of Server Package

The system copies the directory structure of the root directory of the update package one-to-one into the HYDRA directory (all subfolders of the server directory).

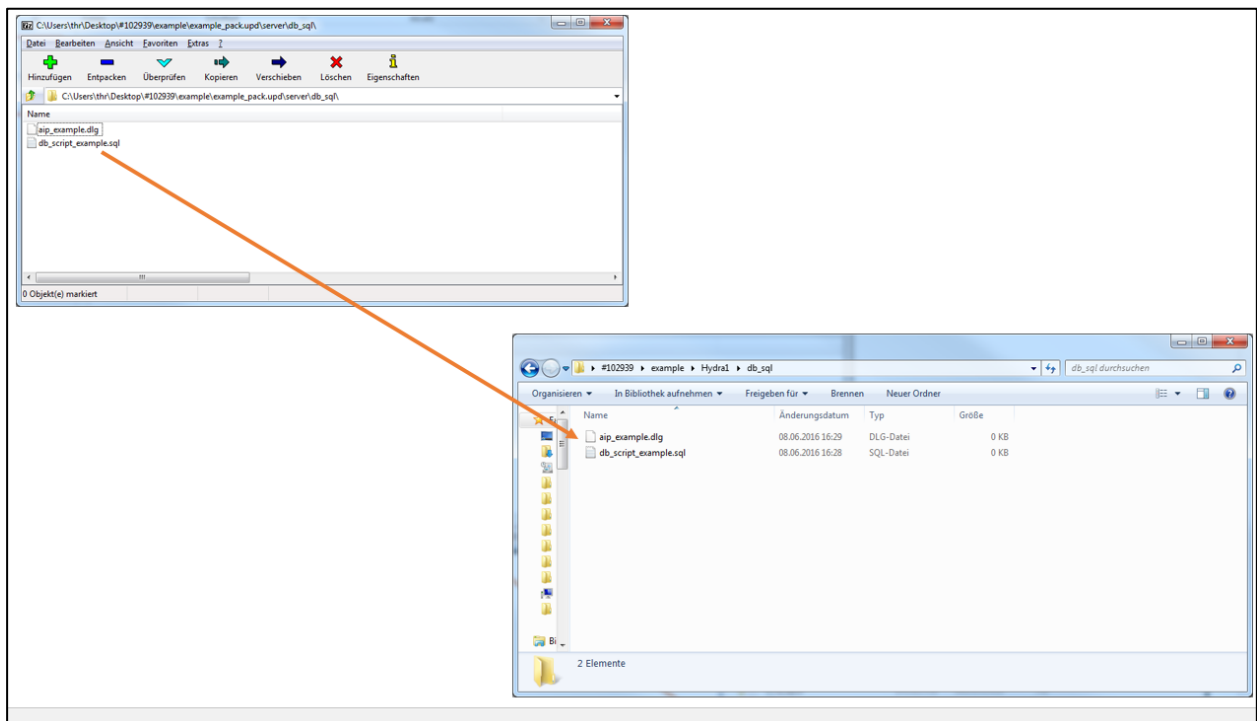


The following example shows a server update package:

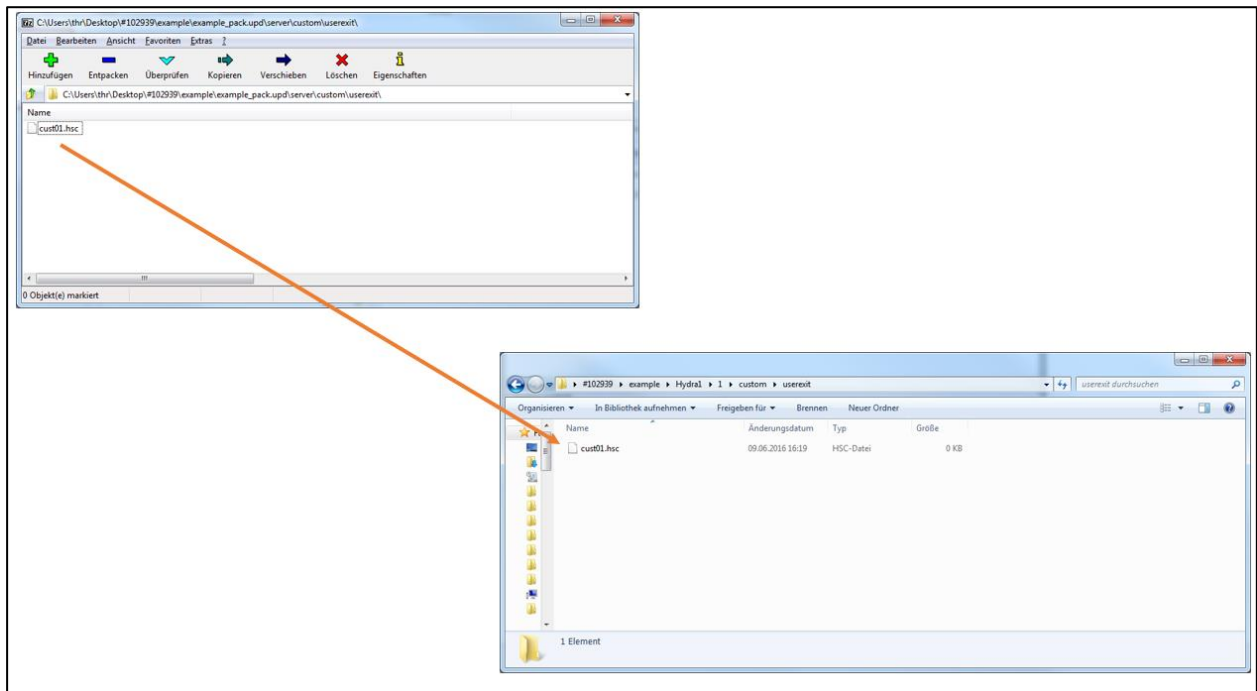
Store server scripts (.scr), programs (.exe/.out), etc. directly in the root directory of the update package. These are stored one-to-one in the HYDRA root directory as described above.



DB patches, SQL scripts, SQL files, dialog files are stored in the subfolder `db_sql`. These are also stored one-to-one (including subfolders) in the HYDRA directory.



Customizations in the form of user exits (terminal scripts, server scripts, SVG files for the upload interface) are stored in the subfolder *custom/userexit*.



Further examples:

Label design: Reports are stored in *custom/reports* (.il / .qr3 files).

Terminals: Customer-specific INI files are stored in *custom/aip* or *custom/aip2*.

Customer-specific language files are stored in *custom* (hycust.mld).



The directory structure in the update package must be identical to the HYDRA directory structure on the server without leading system number. I.e. in the update package, the path is

/custom/userexit

and not

1/custom/userexit.

If the installation is performed using the Maintenance Manager, the files are automatically copied to the subdirectory with the correct system number.



With update packages of MPDV (e.g. as part of a service pack), files can also be contained in a subdirectory with system number 1 (e.g. *1/custom/userexit*). Also these files are automatically

copied to the subdirectory with the correct system number if the installation is performed using the Maintenance Manager. This structure is not recommended any more.

7 MOC Layout Configuration

7.1 Changing the size of detail applications

You can change the size and place of each detail application using drag and drop in the space reserved for the detail applications.

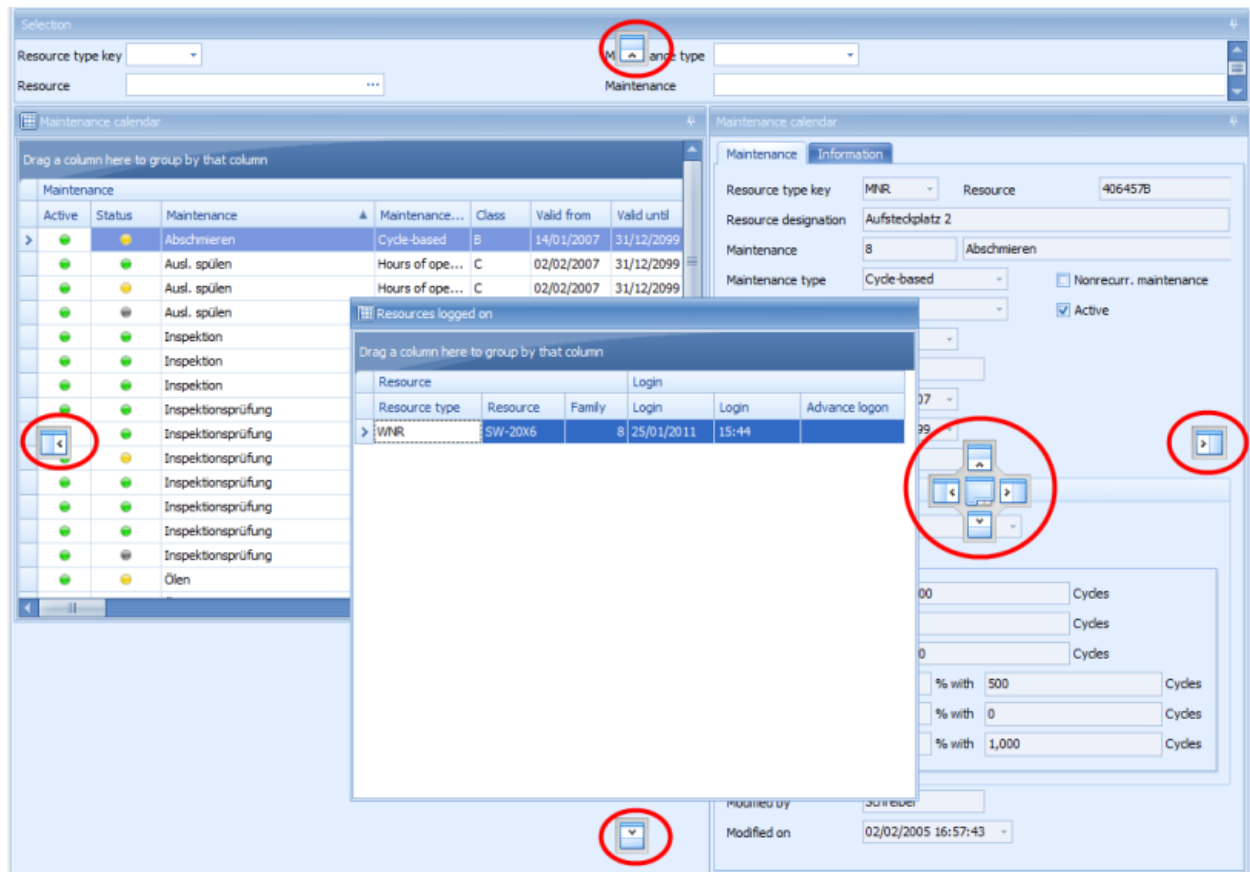
To change the size, move the mouse pointer to the edge of an application, click and hold the mouse button, drag the application to the required size and then release the mouse button.

7.2 Docking

You can place the detail applications next to or below each other or you can dock them. The term "Docking" results from the fact that separate applications are always docked to either the edge of a (main) application or next to or above each other, i.e. a single detail application always has a reference to the related main or affiliated application.

By activating the MES Development Suite, the docking mechanism is automatically activated. Using this mechanism, you can place the detail application where you like using drag and drop. Click the title bar of a detail application, hold the mouse button and move the mouse.

The mouse movement releases the current docking state of the application and the available docking options are shown by arrow symbols (see screenshot). If the released application is now moved towards one of the symbols, a color change indicates the effect of a shift. When you release the mouse button, the detail applications will dock at the required location.



You can use other detail applications or the main application as possible docking locations. A special feature is provided by the icon for "tabbing" detail applications, where applications are placed one above the other. You then use the tabs to display a detail application in the foreground.



From a technical point of view, it is generally possible to arrange detail applications "undocked" or "floating", but these detail applications will then leave the "context" of the MOC and are treated as separate windows by Windows. For this reason, the undocked detail application is not recommended.

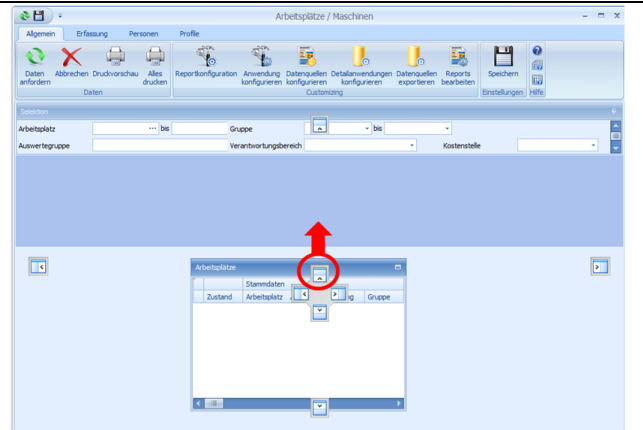


When you create a docking configuration, you must configure the location with reference to the lines because if you change the size of the main application, the detail applications are changed accordingly (this is important when you change to the "maximize" window mode). See the examples below:

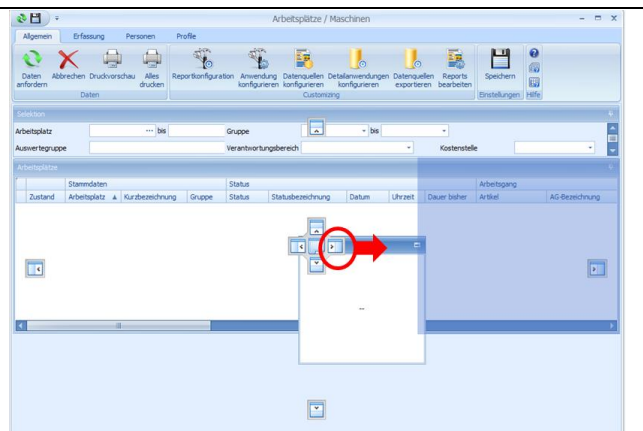
Example 1: Docking in a multi-line application

The following example describes how a multi-line docking configuration is created and how it can be used, for example, in the application *Workplace overview*.

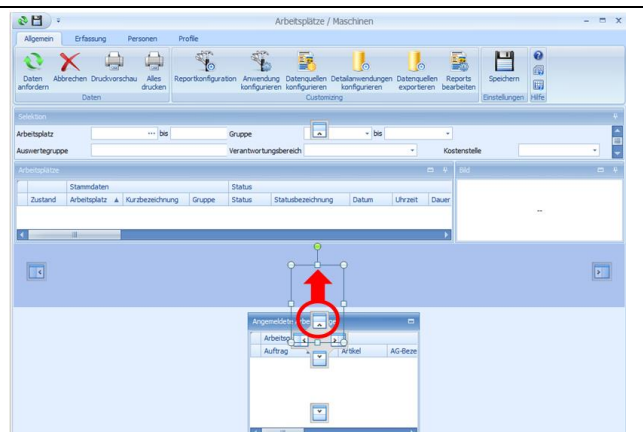
Step 1: Detail application 1 is always docked to the selection panel of the main application.



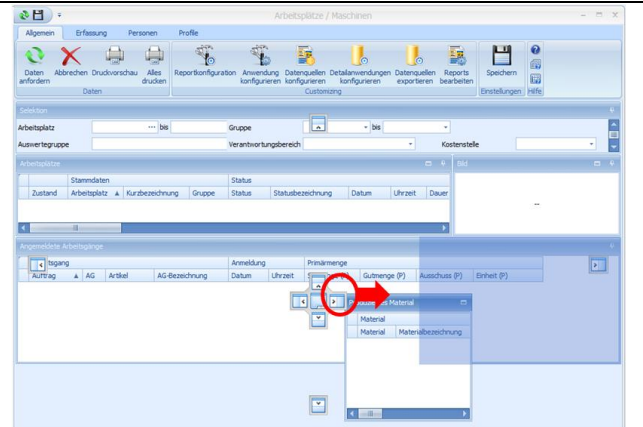
Step 2: In the example, detail application 2 must be displayed to the right of detail application 1. For this reason, it is docked to the right of the detail application. An invisible "virtual docking panel" opens that includes both applications. This virtual docking panel makes sure that both detail applications always have the same height – this is then called a line with detail applications.



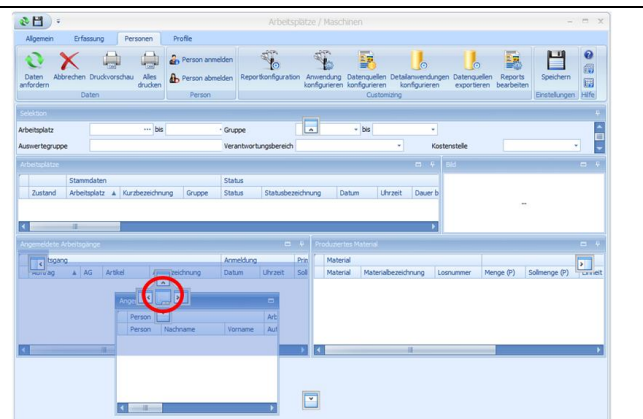
Step 3: You want to display detail application 3 below the existing applications. For this reason, it is docked to the bottom of the "virtual docking panel".



Step 4: Detail application 4 is again docked on the right hand side of detail application 3. Another "virtual docking panel" is created that ensures the correct height of detail applications 3 and 4.

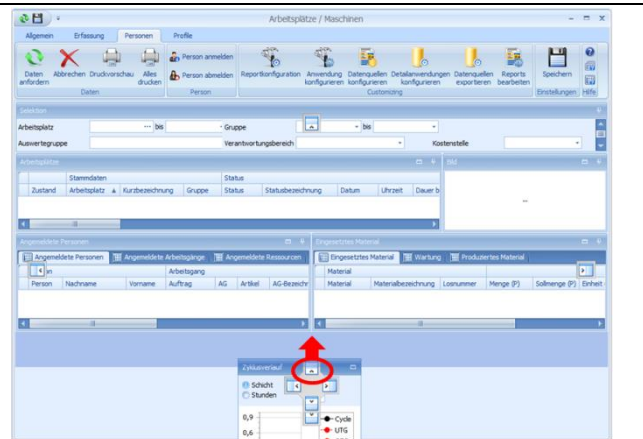


Step 5: You want to customize detail application 5 as "tabbed detail application" with detail application 4. To generate tabbed detail applications, new detail applications are simply dragged "into" an existing application.

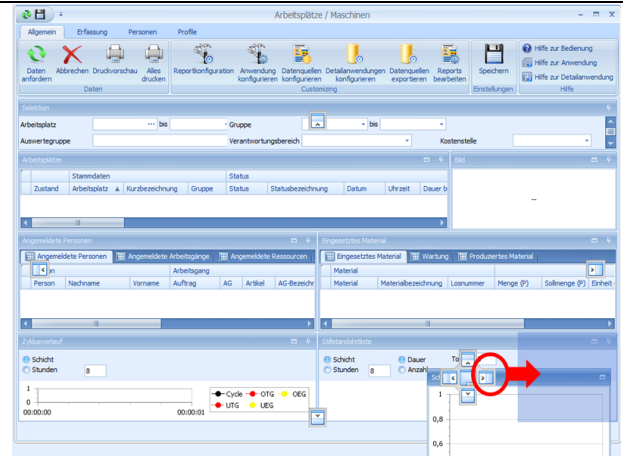


Tabbed detail applications behave like single applications if their size is changed. As many detail applications as required may be tabbed.

Step 6: A new line is generated - the new detail application is docked to the "virtual docking panel" of the second line and, as a result, the next detail application is docked to the right of it into the new, third line.



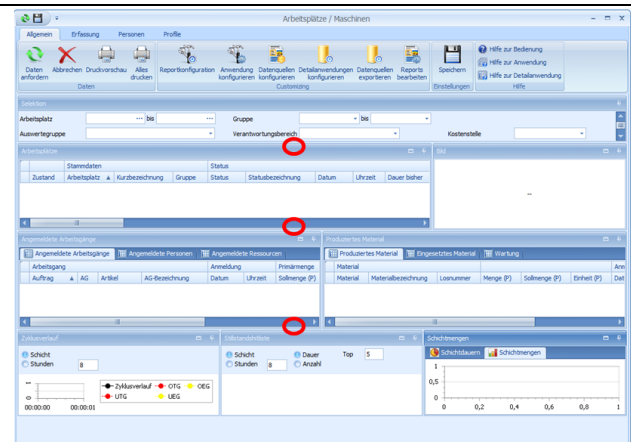
Step 7: The third line is to include three detail applications next to each other. The third application is docked to the right of the second application, and as a result, automatically to the "virtual docking panel" of the third line.



This results in an application that is docked line by line. The height of the rows can be changed at the marked positions.

The MOC provides a mechanism for changes of size. If the size of a detail application is changed, the detail applications at the bottom automatically change their size, too, and empty spaces are avoided.

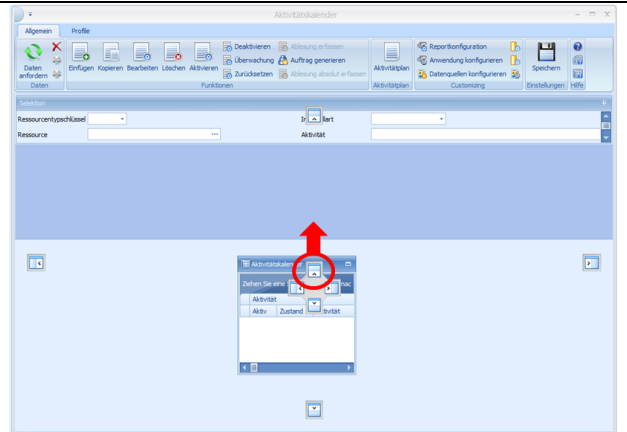
The empty space shown in the picture on the right is filled automatically when the size of the application is changed.



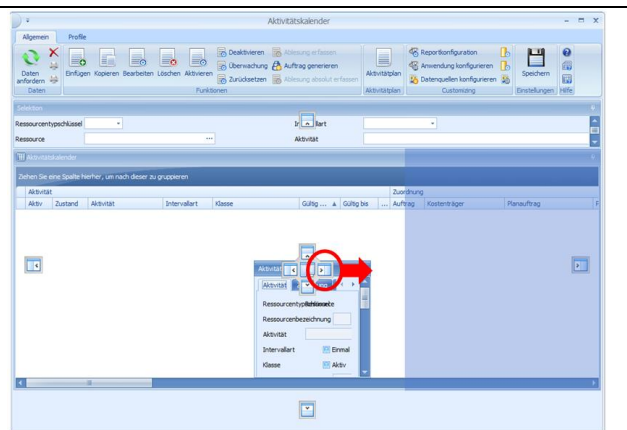
Example 2: Docking in an application with detail applications of different heights

The next example shows how an application is created that includes detail applications with different heights. The activity calendar is an example for such an application.

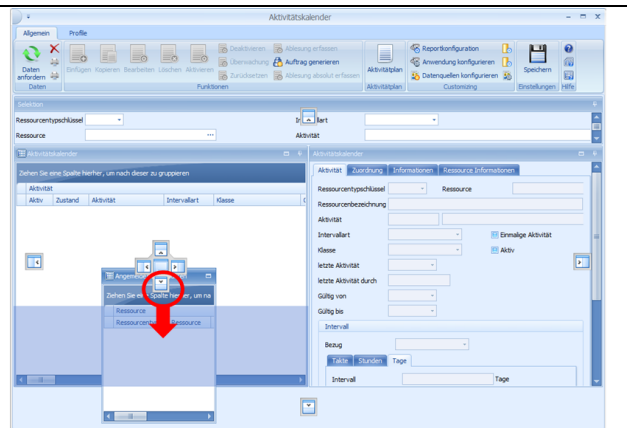
Step 1: Detail application 1 is docked to the selection panel of the main application.



Step 2: Detail application 2 is docked to the right of detail application 1. This results again in a "virtual docking panel" that includes the two detail applications.

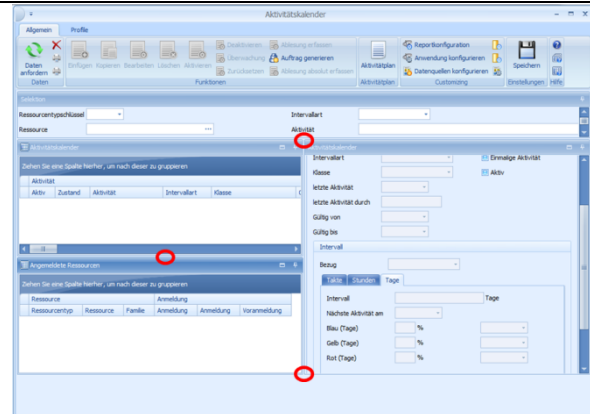


Step 3: Detail application 3 is docked *inside* of detail application 1 to the bottom. This generates another "virtual docking panel" at the location where detail application 1 has been before. This "virtual docking panel" now includes detail application 1 and detail application 3 and together with detail application 2 it also represents the content of the first "virtual docking panel".



The **result** is an application where you can change the height of the lines at the highlighted points.

The next time sizes are changed, the free space below the applications is automatically filled so that the two lower applications fully use it.



7.3 Simple customization of table views

Most applications use table views ("grids") to display data. Tables have several columns. The columns are grouped in so-called categories.

Show and hide columns and categories

Using drag and drop, you can move columns within categories and you can move the entire categories. Any user can individually make this kind of changes for any table view in any application.

You can also use the layout configuration to remove columns and categories or show hidden columns and categories using the context menu (right-click the column header to open the context menu).

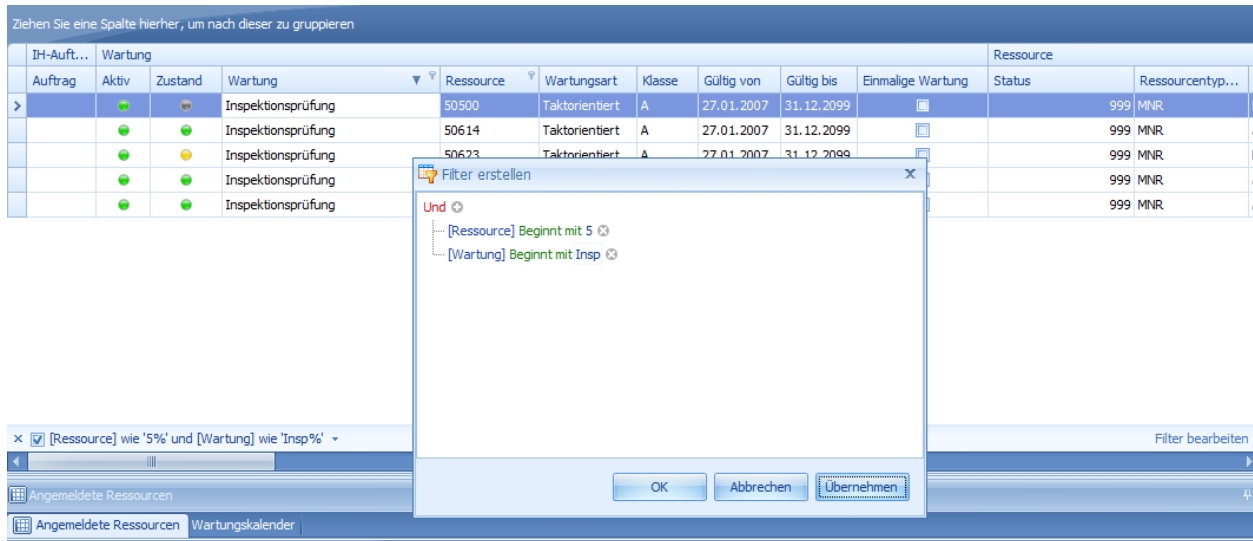


If you want to show a column of a hidden category, you must first show and add the category to the table.

Filtering table results

In the grid, you can filter by specific values in the columns. Just click the pin next to the column header. A selection list of all values in this column is shown. This is the same procedure as filtering in MS Excel.

If you have set a filter for a column, this filter is displayed at the bottom left of the grid. Click the checkbox to delete the filter. You can edit and refine the filter subsequently. For this purpose, the button "Edit filter" is shown at the bottom right of the grid. Click to open the window with the filter editor. Optionally, you can open the filter editor using the function *Filter editor* in the context menu. In the dialog, you can select columns and specify comparison operators and values for each column.



Red text: if you click the red text, a list of possible link methods for several filters is shown.

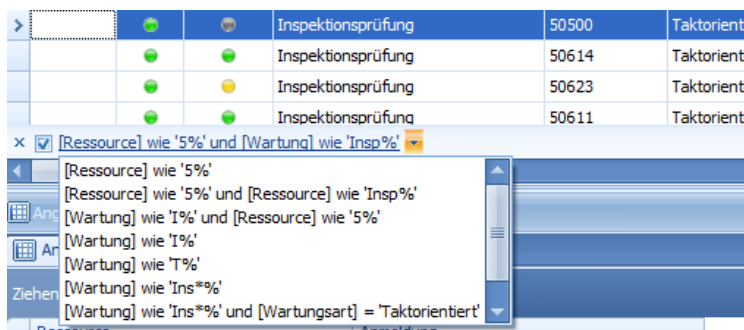
Blue text: A list of columns included in the grid is shown. You can define a filter for these columns.

Green text: The list of available comparison operators for a filter is shown.

Plus: A new filter is added.

Cross: The relevant filter is removed.

You can subsequently extend an existing filter. The column of the column header, which has been clicked to show the dialog, is always suggested as comparison value.

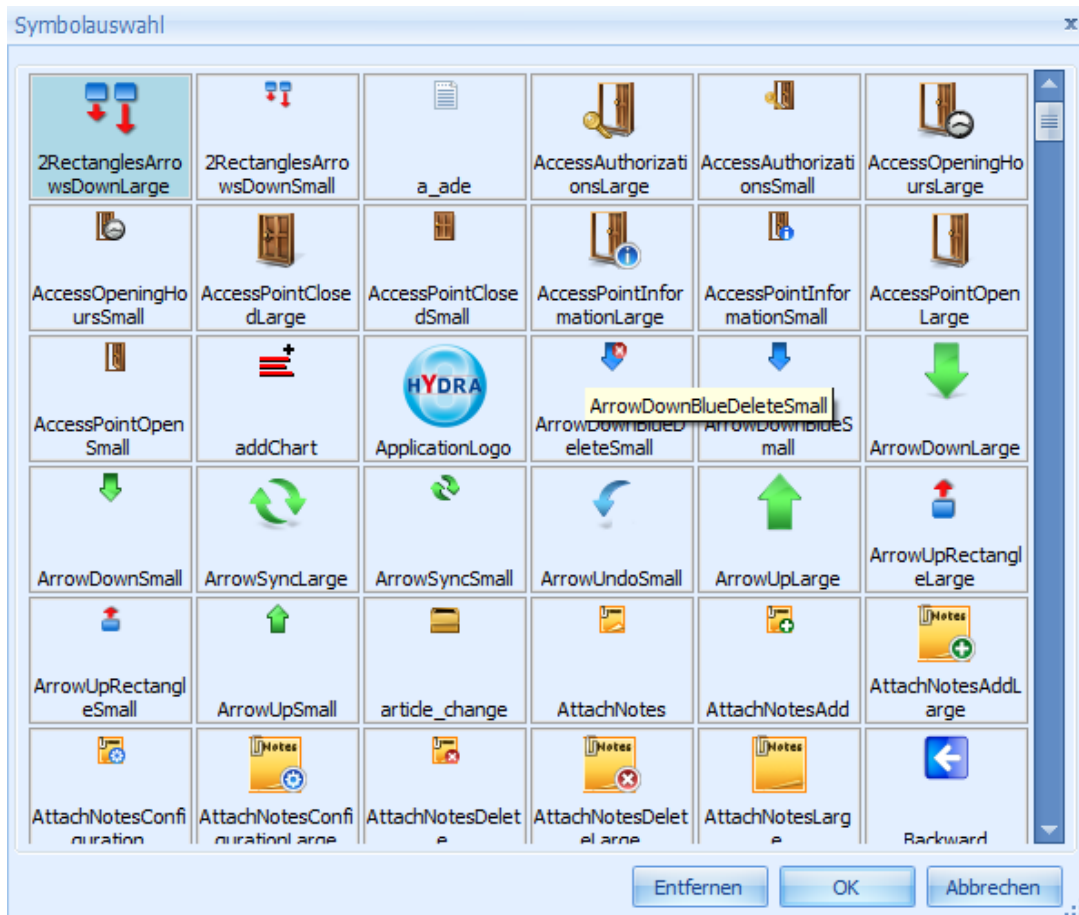


Filters entered once are saved and can be called again. You can therefore create a list of filters. When the list has been stored in the customization level "Local", the list can be distributed to the users.

7.4 Customization of symbols

You can not only translate label texts, but you can also replace MOC symbols. You only need to store an image file with the required name and in a format that is supported (PNG, JPG, ICO) in the folder %scope%/resources/images.

You can identify the symbols and images used and their names using the dialog *Selected icons*. Example: To replace the "ApplicationLogo" icon, an image file with precisely this name must be stored in the "resources" folder.



Most symbols are available in the resolutions 32x32 (xxxLarge) and 16x16 (xxxSmall). The table below shows some exceptions.

Symbol / image	Name	Resolution
Start screen (splash screen)	splash.jpg	480x285
MOC logo (taskbar on the bottom right)	Logo.png	69x25
Start button (taskbar on the bottom left)	StartButton.png	104x20



If an image has a different resolution than the one mentioned above, unwanted effects in the display can occur, for example on the MOC desktop.

You can use the MOC "Update Package Creator" ([Extras](#) → [Generate Update Package](#)) to create update packages with own icons. This update package can be loaded in the Maintenance Manager and distributed to all clients of the system.

8 Multilingual texts in the MOC using language keys

Texts such as field labels, column names, menus, buttons, dialog texts, etc. displayed on the screen of the HYDRA 8 MOC client, are not directly entered in the code (or configuration files) of an MOC application but referenced by “language keys” (Languagekeys). The software automatically searches the correct label text for the language key of the currently active language, e.g. the language key “lkWorkplace” is translated as label text “Arbeitsplatz” in German and “Workplace” in English.

The Excel file “mpdvDictionary.xlsm” is part of the HYDRA Language Manager and has been designed to translate the HYDRA 8 client MOC.

The languages are exported from the Excel file. These (resource) files are imported and evaluated in the application at run time. Consequently, you do not have to modify the software in order to add a new language to the application or to change a label text.

The following cases can occur during the localization process.

Editing of standard language versions

MPDV supports a number of standard language versions, i.e. language versions edited by MPDV. At the moment, these are the language versions German, English and French. Software development maintains these language versions in the file “mpdvDictionary.xlsm”.

Relevant resource files are provided along with software delivery.

Editing of customer-specific label texts by MPDV

MPDV maintains customer-specific label texts in customer-specific copies of the file “mpdvDictionary.xlsm”. These copies only include the new or adjusted entries.

Customers are provided with relevant resource files along with software delivery.

Editing new language versions by customers or partners

A customer or partner who has purchased the license for the HYDRA Language Manager receives a copy of the file “mpdvDictionary.xlsm” and this description. Consequently, customers or partners can create and edit individual language versions.

Editing of customer-specific label texts by customers/partners

A customer or partner is allowed (e.g. as part of the MES Development Suite) to add individual label texts to existing language versions and to translate existing terms differently. To do so, the customer or partner receives a customer-specific copy of the file "mpdvDictionary.xlsm". The customer or partner can create new entries in the table entitled "CustomDictionary". In this case, the customer or partner can use the entries included in the "dictionary" workbook as copy template, but should not change them.

8.1 Using the MPDV Dictionary

Structure

The MPDV Dictionary includes the following spreadsheets:

- "Dictionary" includes the standard label texts edited by MPDV. Columns:
 - "Date" (Datum): when was the key created.
 - "Origin" (Herkunft): who has created the key.
 - "Status": "new": key has not yet been translated; "ok": has been translated; or "changed": key changed after translation.
 - "Key" includes the language key (e.g. "lkWorkplace") that will be replaced in the application with a label text of the current language version.
 - Columns including the label texts for the corresponding language version (German, English, French, ...)
 - "Comment": can include complementary information and notes for translators.
- "CustomDictionary" includes customer-specific label texts supplementing the "Dictionary" or overwriting the label texts included in the "Dictionary".
- The "Languages" workbook includes the list of language versions that should be exported. You can start the export by clicking the function "Create resource files. Columns
 - "Create": if this is checked (enter an "x") a file named "mpdvTexte.<culture>.resx" is generated.
 - "Language": Column heading in the (Custom) dictionary.
 - "Culture": name of a culture supported by Microsoft .NET Framework.
 - "English Name": culture name
 - "Comment": additional note.

Generating and importing resource files

If you click "Create resource files" in the "Dictionary" or "CustomDictionary" spreadsheet, a resource file is generated for every language version selected in the "Languages" spreadsheet. The generated resource files are named according to the pattern "mpdvTexte.<culture>.resx", e.g. mpdvTexte.de-DE.resx.

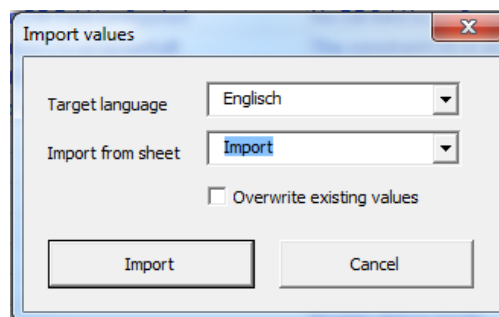
The standard files are copied into the folder "%moc%\resources\languages\", customized files are stored in the folder "%moc%\custom\resources\languages\" or "%moc%\local\resources\languages\". Entries of customized files overwrite default entries.



Resource files generated by MPDV are delivered via the software distribution process. If customers generate resource files, customers have to ensure the files are integrated in the corresponding client installations using an appropriate distribution mechanism. The MOC "Update Package Creator" (MOC: *Extras* → *Generate Update Package*) allows you to create update packages with individual texts. You can load these texts into the Maintenance Manager and distribute these among the clients throughout the system.

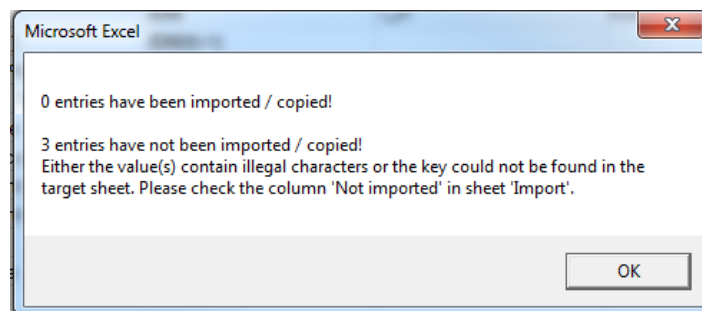
Importing translation

You can use the function "Import values" to import translated language keys from another sheet into the Dictionary (e.g. in case an external service provider translated texts).



Existing label texts are overwritten if you check the option "overwrite existing values".

After the import, a dialog appears indicating the number of transferred label texts and, if necessary, also the number of label texts that have not been transferred.



In case a label text cannot be taken over as it includes e.g. invalid characters (see below), a corresponding note is shown in the "Not imported" column in the line of the affected entry.

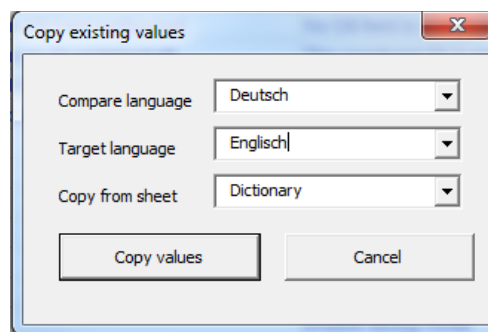
Datum	Herkunft	STATUS	Key	Deutsch		
02.03.2012	TEST	xxx	lkQMCalibrationCalendar2	Eintrag mit ungültigem Zeichen <	Illegal value	
02.03.2012	TEST	xxx	lkCalibrationHistory2	Eintrag mit ungültigem Zeichen &	Illegal value	
02.03.2012	TEST	xxx	lkGageStatus2	Eintrag nicht vorhanden	Key not found	

Accepting existing translations

You can use different language keys for the same label texts.

Key	Deutsch	Englisch	Französisch
lkWorkplan	Arbeitsplan	Work plan	Plan travail
lkworkplanorder.id	Arbeitsplan	Work plan	Plan travail
lkworkplanorder.order	Arbeitsplan	Work plan	Plan travail

You can use the function “Copy existing values” to translate duplicates automatically. Consequently, you do not have to translate such label texts multiple times.



Find duplicates

Double entries included in one column may be identified by the “Find duplicates” function. To do so, sort the column alphabetically and position the cursor at the place from where you want to start searching.

8.2 Notes on editing and translating label texts

You should adhere to the following notes when editing and translating label texts.

No duplicate language keys!

You must not generate duplicates when you create new language keys, i.e. each key may only exist once. Otherwise all previous entries will be overwritten.

Please note: Duplicates are highlighted automatically in the "Key" column. Keys are not case sensitive. In general, you are allowed to write keys differently. For the sake of clarity, however, you should use uniform notation.

Datum	Herkunft	Status	Key	Deutsche
19.03.2012	xxx	new	IkTest	
19.03.2012	xxx	new	IkTest	
19.03.2012	xxx	new	IkTEST	

Special characters

Resource files are in the XML format. Therefore, label texts must not include values interpreted as XML. This includes, in particular, the following characters: ">", "<" and isolated "&" characters. Use the following strings instead:

Characters	Character string.
">"	">"
"<"	"<"
"&"	"&"

While translating, you should make sure these special characters are not separated by blanks. As errors might occur during file import if "& lt;" is used instead of "<".

Wildcard characters

Wildcard characters enable the display of variable values in a language key and have the format {0}, {1}, {2}, etc. While translating, you have to make sure that blanks are not inserted between brackets and figures.

Example: Calling "Translate("IkUserName", "peter")" in the source code with language key "IkUserName" and translation "User {0}" returns the character string "User peter".

Adding new language versions

In order to add new language versions, you have to create a new column for the label texts in the Dictionary. The column title must match the entry in the "Language" column on the "Languages" spreadsheet.



If you want to generate different "Cultures" of a common language from one column in the dictionary (e.g. "de-DE" for Germany and "de-AT" for Austria), you have to modify the "Language" value for new languages in the "Languages" workbook.

You have to enter an "x" in the "Create" column in order to export the required language.



Previous versions of the dictionary did not include all languages available in .NET in the "Languages" spreadsheet. If you use such a version, you can find all possible entries, e.g. here: <https://msdn.microsoft.com/en-us/goglobal/bb896001.aspx>. Please note case-sensitivity when adding new languages.

8.3 Identifying Language Keys in the GUI

The translation process normally takes place in the Excel file "mpdvDictionary.xlsm". The translator does not know where in the GUI the result will be visible. Consequently, some translations are longer than the native text and do not fit in the dialog. In such cases, it is useful if the translator knows the associated language key.

To do so, use an "empty" language file (without any translated entries), as the MOC shows the language key whenever it does not find a translation.

To create and import an empty file, just follow the instructions described in the paragraph.

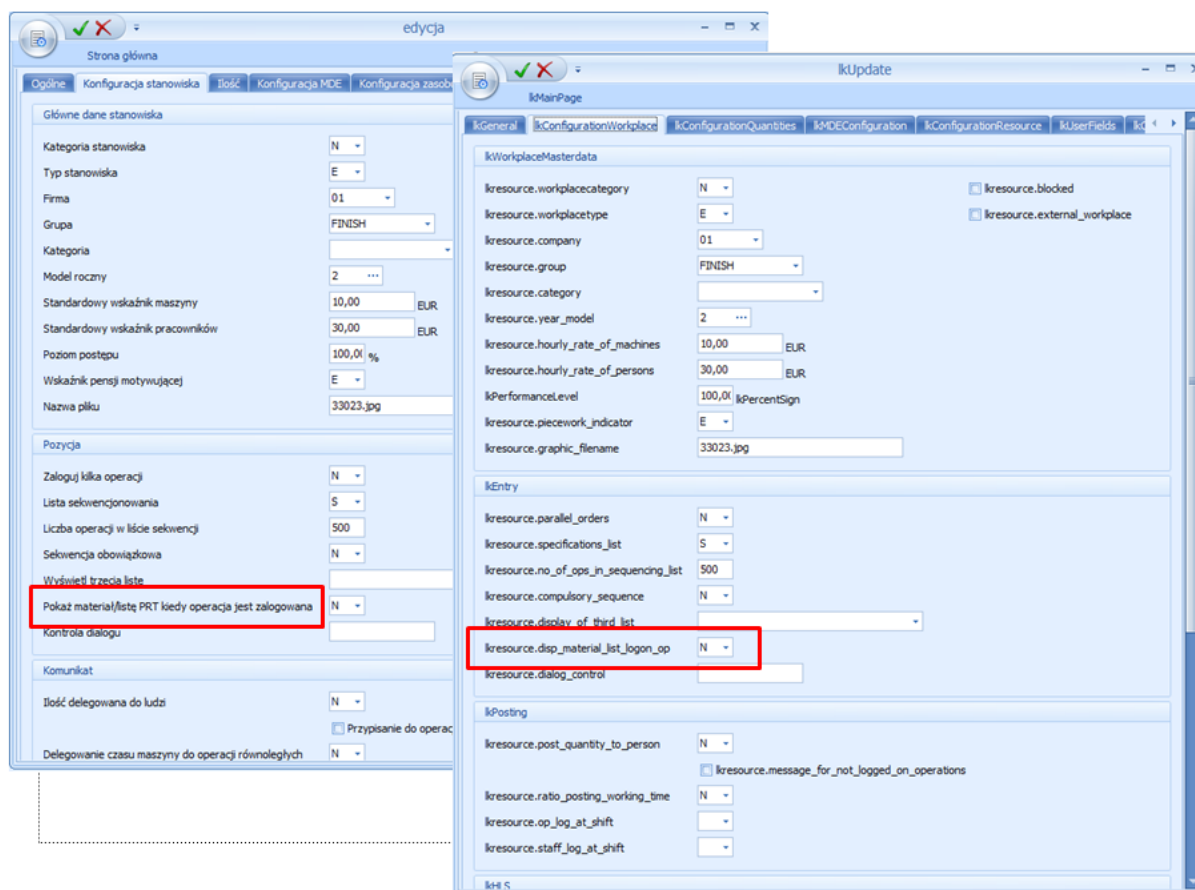


Figure 1: This screenshot shows the same editing dialog in different MOC instances. The first instance shows a new target language. The second instance shows a language with "empty" resource file. If you compare both dialogs, you can identify problematic language keys you can correct in mpdvDictionary.

8.4 Tips on how to translate detailed dialogs

MOC uses a large number of detailed dialogs where data can be viewed and edited using "controls". These controls support different types of input and output options: text fields, selection lists, checkboxes, etc.

The following paragraphs illustrate how translations can affect the design of such dialogs.

Structure of detailed dialogs

All controls are aligned in a column-based structure. The number of columns may vary from line to line.

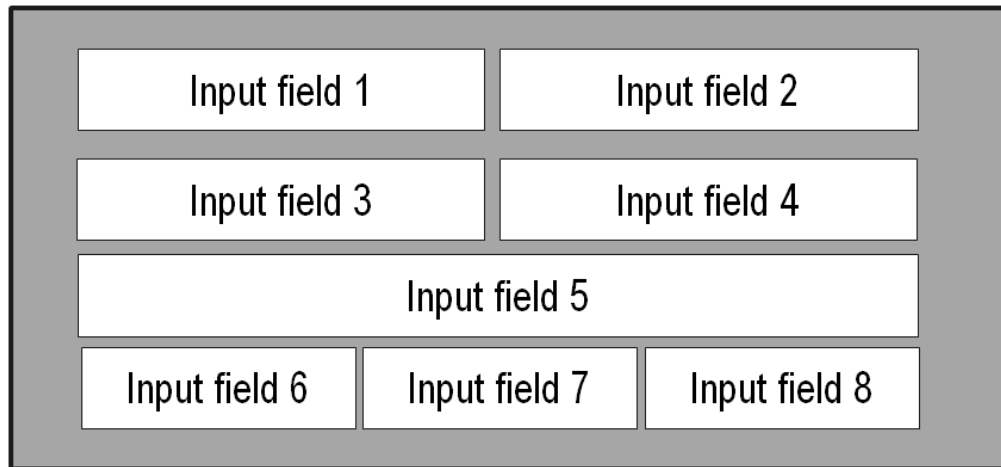


Figure 2: This figure shows the structure of a dialog with two lines displaying controls in two columns, one line using only one column and one line with three columns.

Most controls show labels. These controls include two parts: the label and the (data) "control". Such controls have two columns: one column for the label and one for the control.

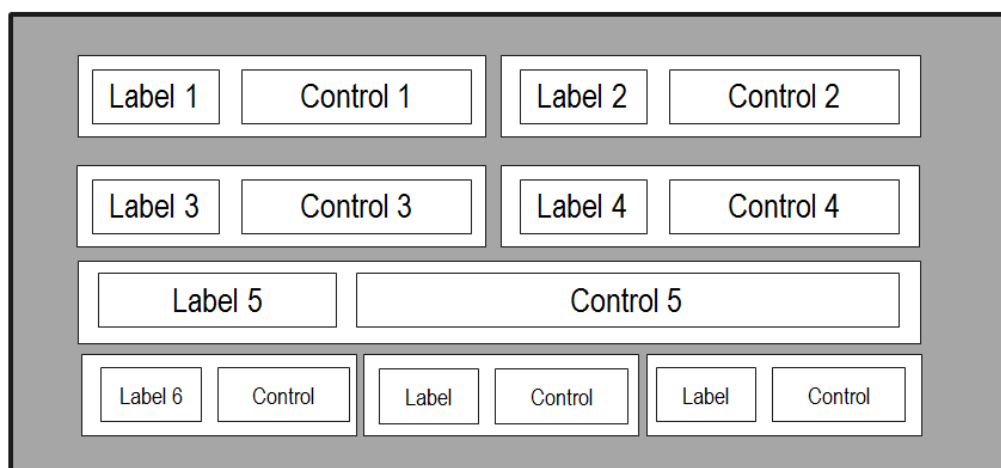


Figure 3: This screenshot shows the internal dialog structure from Figure 2.

All controls of a column have the same label width. The label width for the elements 1 and 3 of Figure 3 are identical.

Label widths are calculated dynamically depending on text length. As the label of control 1 increases, control 3 also requires a wider label in Figure 4.

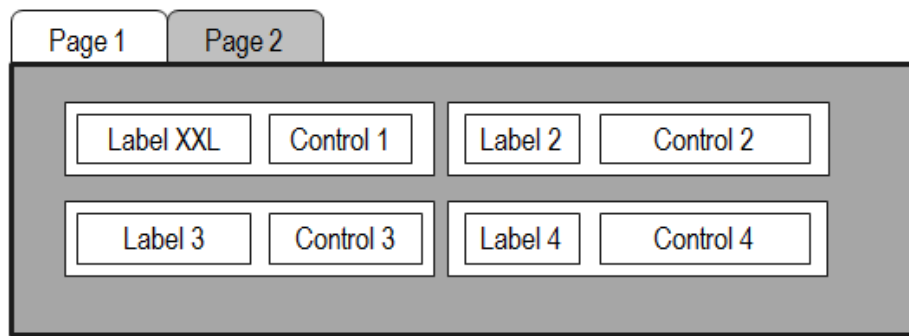


Figure 4: Due to a longer label text in control 1, control 3 also requires a wider label.

The columns of controls extend over all dialog "tabs". This means the label width of one control can be affected by the label width of another element. Even if this element is not visible as it is shown on another dialog tab.

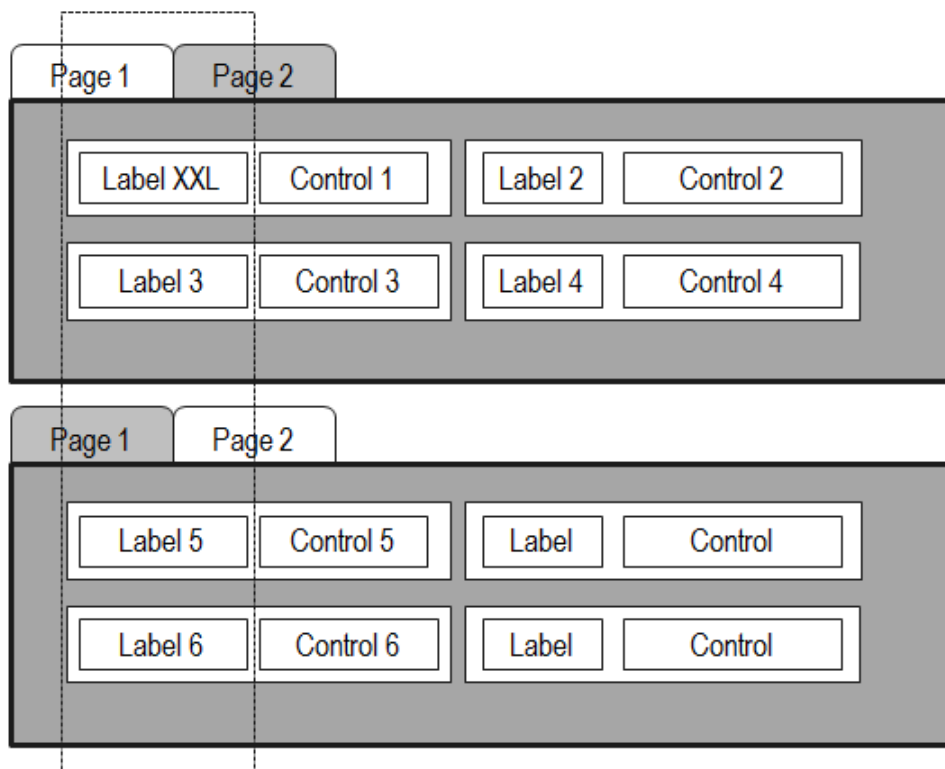


Figure 5: This figure shows two dialog tabs one below the other. The width of label 1 specifies the width for the label columns of the elements 5 and 6.

Problems caused by (too) long labels

Most controls have a fixed width. If the label width increases, the space for the actual control is reduced. This can even result in the control being displayed only to some extent or not at all. Due to the above-described column structure, this might also occur with controls with labels that are not too large.

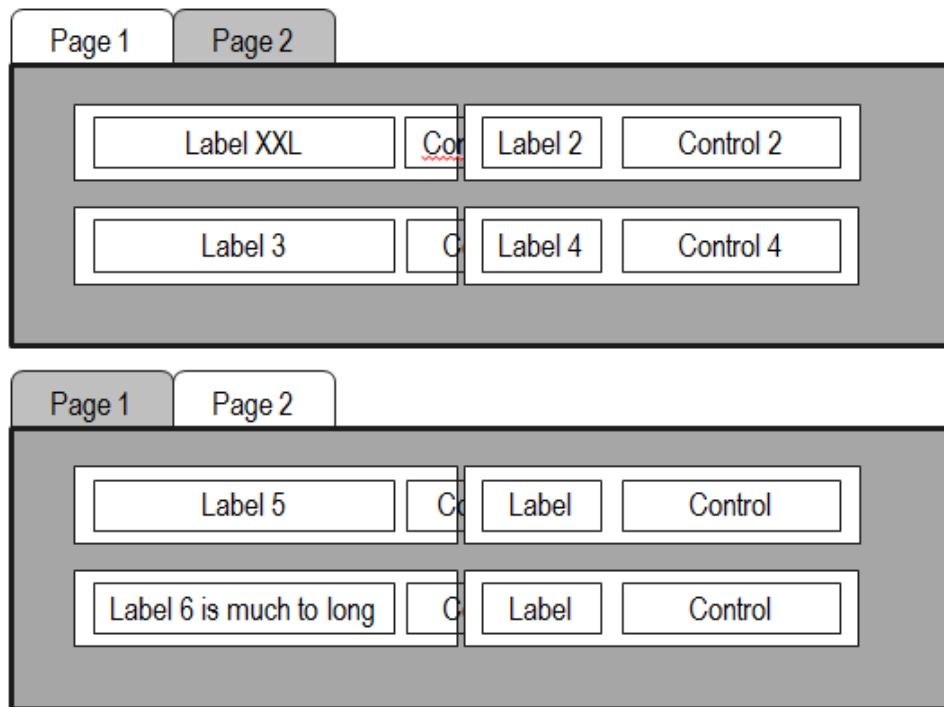


Figure 6: This figure illustrates how a single label with too long text might affect the entire dialog layout.

The following screenshots illustrate the issue.

Solution: identify the control elements having too long label texts on the single tabs and shorten the text.
Please note: An empty language file (see section 8.3) can help you identify the affected language keys.

edycja

Strona główna

Opólne Konfiguracja stanowiska Ilość Konfiguracja MDE Konfiguracja zasobu Pola użytkownika Komentarz

Główne dane stanowiska

Kategoria stanowiska N

Typ stanowiska E

Firma

Grupa HANDARB

Kategoria

Zablokowana

Stanowisko zewnętrzne

Kontrola dialogu

Komunikat

Ilość delegowana do ludzi N

Delegowanie czasu maszyny do operacji równoległych N

Zaloguj operację automatycznie przy zakończeniu zmiany

Automatycznie wyloguj osobę wraz z zakończeniem zmiany

HLS

Figure 7: The above figure shows how large control labels...

edycja

Strona główna

Opólne Konfiguracja stanowiska **Ilość** Konfiguracja MDE Konfiguracja zasobu Pola użytkownika Komentarz

Ręczne wprowadzanie wartości

Składowa ilość w wyniku kalkulacji

Jednostki i współczynnik konwersji dla ilości bazowej

Podstawa konwersji ilości HYDRA-MDE	M		
Jednostka Ilości (P)	S	Mianownik, pierwsza wartość	0
Jednostka Ilości (S)		Licznik, druga wartość	0
Jednostka Ilości (T)		Mianownik, druga wartość	0
Jednostka Ilości (B)		Licznik, trzecia wartość	0
Licznik, druga wartość		Mianownik, trzecia wartość	0
	ST		...
			0

Figure 8: ... affect elements of other tabs. In this case, labels of the first tab overlap input fields of the second tab.

9 Customization of Applications

9.1 Overview

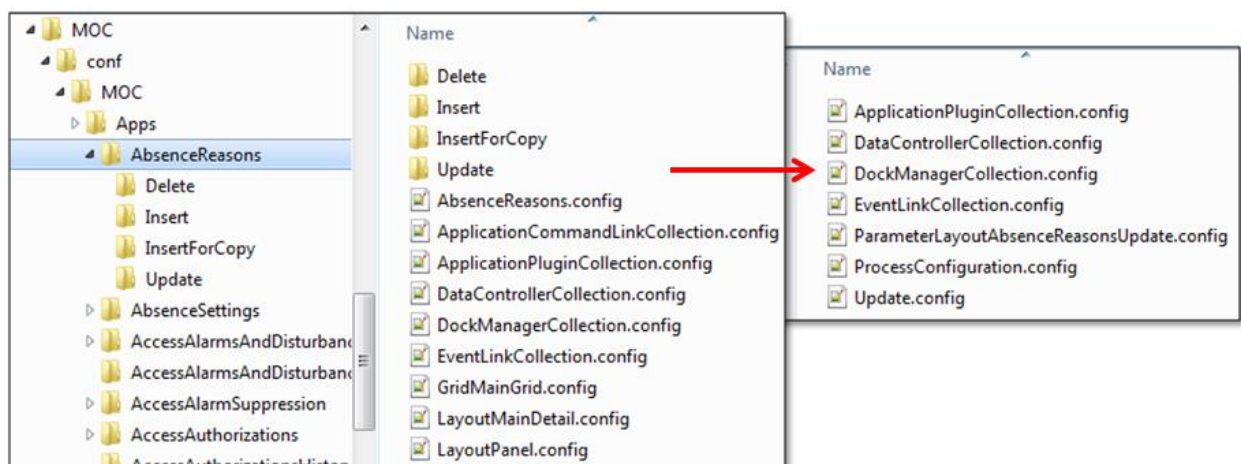
The MOC provides two types of applications:

- Main applications to present data
- Editing applications to edit data

There is a directory with the clear/unambiguous (English) application name for each main application. The application directory includes all files required to customize an application. The specific files and the number of files are different for each application. This largely depends on the number of detail applications (tables, charts etc.) included in an application.

Editing applications are always assigned to a main application. The files for the customization of detail applications are therefore managed in sub folders of a main application. In addition to the customization files of the main application, the application directory contains an additional directory including the files of the detail application.

Example: The files of the application *Absence reasons* are stored in the folder of the applications with the name of the relevant application ID, i.e. in this example "AbsenceReasons". The editing dialogs required to edit absence reasons are stored in the sub folders "delete", "insert" and "update".



9.2 Customization of main applications

Applications are the central elements in order to access the variety of functions of the MES Operation Center. All applications have the same structure, i.e. they contain

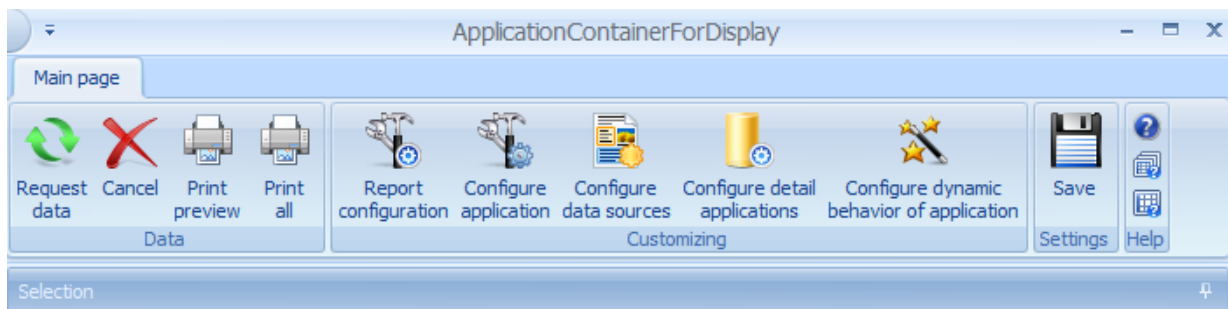
- the main application, i.e. the frame of the application (also called "Application Container")

- a toolbar with preset buttons to activate standard functions (Request data, Print preview, Save configuration, Help, etc.) and a deliberate number of buttons added via customization.
- a selection panel to enter selection or filter criteria with a configurable number of input fields
- data sources (DataControllers) to supply detail applications with (web service) data,
- detail applications embedded in the application via customization, and
- a deliberate number of report configurations to activate the defined reports.

The different components of an application and their customization options are described in detail in the sections below.

9.2.1 Creating an application

You use the menu item *MES Development Suite – New application* to create a new application. A new application opens that can be edited. Save the application to persist the changes.



You cannot create applications that include editing dialogs using this procedure. You then require a specific application generator, as described in section "9.3.1 Application generator".

9.2.2 Customization of applications

You can use the button *Configure application* in the toolbar to edit specific properties of an application. You can edit the following values.

ID

Clear ID of the application that can be used to identify the relevant customization file, for example.

Caption

Text displayed in the title bar of the application. Specify a language key ("lkxxx") for this text so that the caption is translated.

Help file and Help index

Name of file where help subjects for this detail application are listed and/or description of the entry point within this file.

Update rate

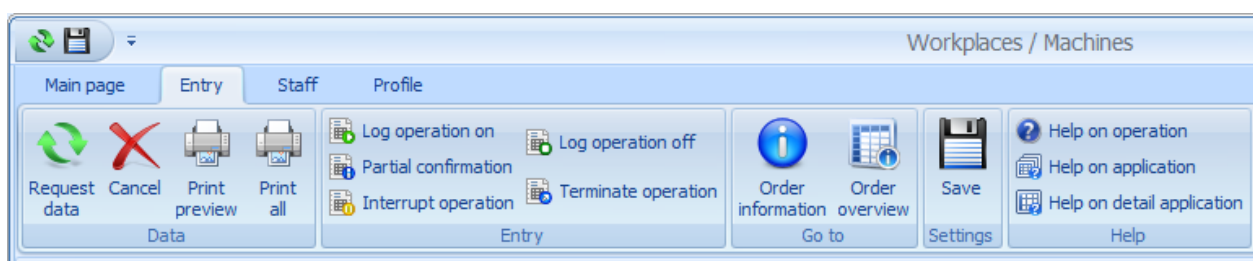
Time in seconds after which this data source will automatically and regularly request data. This is the default value for the entire application. This value is used for all detail applications containing the value 0 as update rate. If this value is 0, data is not automatically updated.

Version

Application version

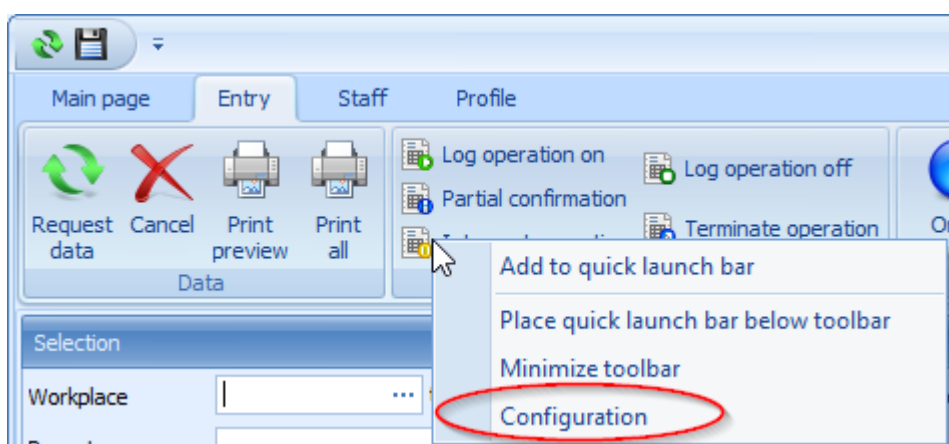
9.2.3 Toolbar

The toolbar provides a quick access to context-related functions. The toolbar can consist of several tabs which can include several categories each.



The categories "Data" and "Help" are given on each tab in each application. The "Data" category includes the buttons *Data request*, *Cancel*, *Print preview* and *Save*. The categories "Data" and "Help" and the buttons contained cannot be changed.

If the MES Development Suite is activated, another non-adjustable category, *Customizing*, is shown. This category includes all important functions to change the application (please also refer to 9.2.4 et sqq.).



The *Configuration* menu item in the context menu of the toolbar calls the dialog to edit the functions. Here, you can create new buttons and edit existing buttons that can be changed.

The following customization options are available:

ID	Clear ID of the ApplicationCommandLink that is called
Command	Deliberate identification of the ApplicationCommandLink
Function	Function that you want to execute. Possible values are e.g. callScript, openApplicationContainerForEdit
Parameter	Parameters passed to the function
Authorization key	Is used to authorize this function via function authorizations.
Title	Language key for display
Category	ID of the category (is a language key). If the category does not exist, it is created.
Ribbon page	ID of menu bar tab (is a language key). If the tab does not exist, it is created.
Comment	Comment
Keyboard shortcut	Keyboard shortcut Valid keyboard shortcuts : CTRL+A to CTRL+Z CTRL+F1 to CTRL+F12 SHIFT+F1 to SHIFT+F12
Image (small)	Image used to present the function in the toolbar
Image (large)	Image for presentation. If a large image is available, it will be used; otherwise the small image is used.

There is no customization file for default categories that already exist. Additional functions are saved in the file ApplicationCommandLinks.config. This file is only created if required (if functions are added and the application is then saved).

Important functions

Link to a main application

ID = application ID

Command = application ID

Function = openAppContainer

Parameter = id=<application ID>

The below parameters can be transferred additionally (added at the end, separated by blanks):



The parameters are case sensitive!

- autorequest=true/false → if true, data is automatically requested after the application has been opened
- windowOpenMode=MultiWindow/SingleWindow → if MultiWindow, a new instance of the application is opened every time it is linked; if SingleWindow the same instance of the application is always opened. MultiWindow is always set by default to link one application to another application. If SingleWindow is set explicitly via parameter transfer, all other transferred values are also reset automatically. Data is only requested automatically if autorequest=true is passed.
- Modal=true/false → if true, the application is opened modally
- selectedprofile=<Profile> → if a defined profile is transferred, it is automatically preassigned when the application is opened.

If the name of the selected profile includes blank characters, the profile name has to be enclosed by "inverted commas".

You can also transfer values from the application (also applies for opening of editing applications):

- zielacronym=DC (the relevant value of the same acronym from DatenController is transferred to the target acronym)
- zielacronym=DC:quellacronym (the relevant value of the source acronym from DatenController is transferred to the target acronym)
- zielacronym=SP (the relevant value of the same acronym from SelectionPanel is transferred to the target acronym)
- zielacronym=SP:quellacronym (the relevant value of the source acronym from the SelectionPanel is transferred to the target acronym)

Opening an editing application

ID = ID of the editing application

Command = ID of the editing application

Function = openApplicationContainerForEdit

Parameter = id=<ID of the editing application>

Calling a script

ID = Script ID

Command = Script ID

Function = callScript

Parameter = id=<script ID>

9.2.4 Selection panel

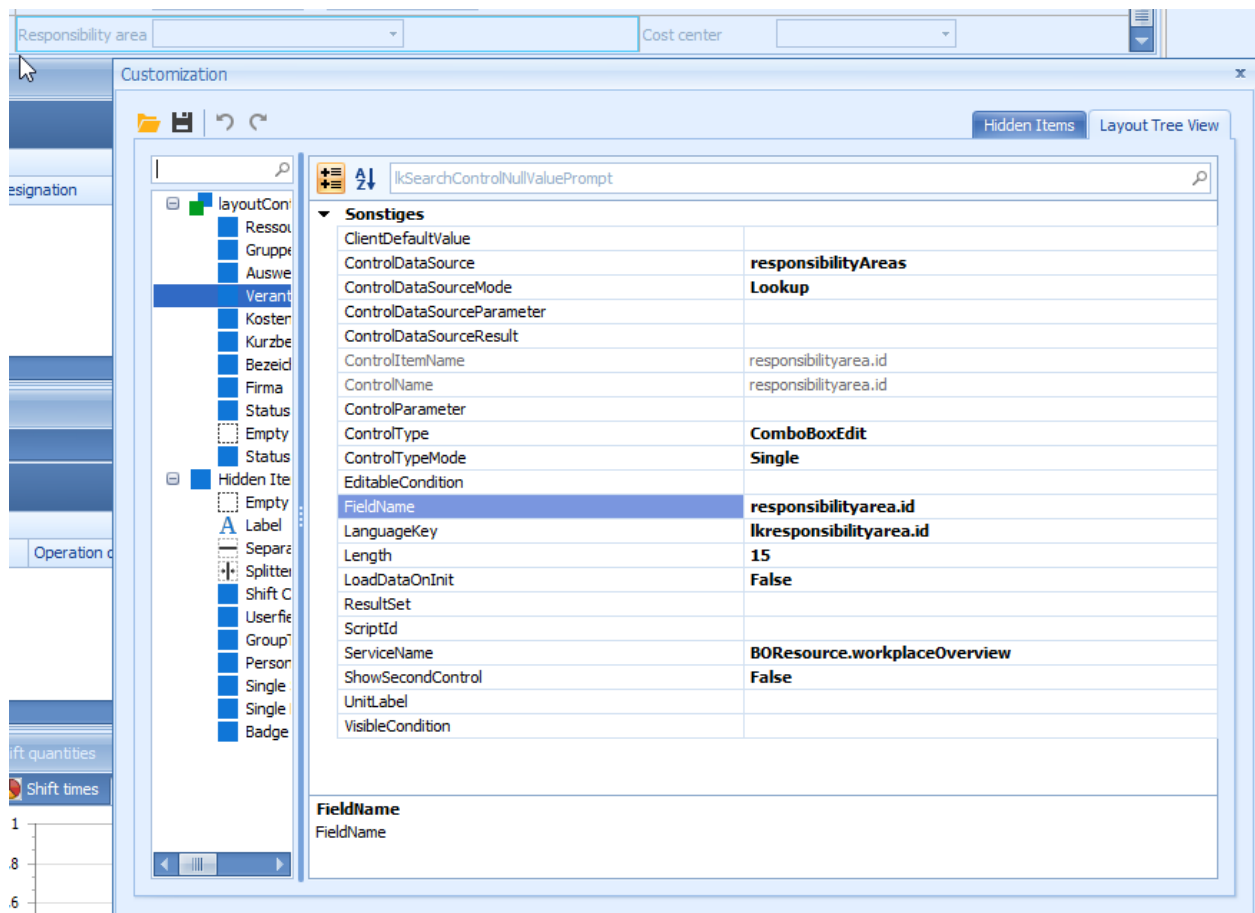
The selection panel contains input fields (controls) to enter filter criteria. To use input fields as filter criteria, the following steps are required.

- Adding of input fields to the selection dialog and
- assigning input fields as parameters for the data sources (this step is described in section 9.2.5)

Activating the editing mode

If the MES Development Suite is activated, right-click the empty space of the selection panel or the label of a control to open the context menu. Use the context menu to add new controls, delete existing controls or to activate the mode to edit layouts and controls (*Customize layout*).

You can use the editing mode to adjust the layout and to specifically edit specific controls in the "Customization" dialog.



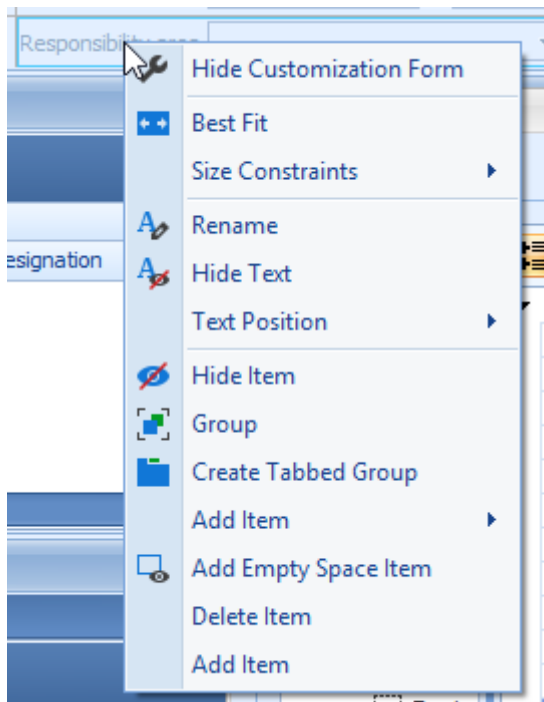
In addition, you can move specific control types to the dialog using drag and drop in editing mode. These can be

- UserField-Control: a control to present user fields (see section "User fields").
- Shift Control: a control to enter shift times
- Person selection: a control to configure person selections in a very flexible manner.

Changing the layout

In the editing mode, you can change the layout of a selection panel. Click the controls and move them using drag & drop. Please note that the layout function will attempt to arrange the controls in columns in order to achieve a very uniform presentation.

In the editing mode, the context menu includes additional functions to edit controls.



To access this context menu, click *Customize layout* in the context menu of the control in the application, and in the *Customization* window go to tab *Layout Tree View* and right-click an item.

Tips for the layout:

- Use *Create EmptySpaceItem* to create forced spaces between controls.
- You can use the values in *Size constraints* to fix height and/or width of controls. After editing, we recommend to fix the values.
- Use *Text Position* to place the label text and *Hide Text* to hide or show the label text.
- Use the item *Group* to combine several fields to an area or *Create Tab Group* to distribute fields to tabs. It is best to use the dialog *Customization* to this end. Here, you can select the controls that you want to group in the tree view and group the controls using the entry of the context menu.

Editing input fields

Controls usually have the type "mpdvEdit". You can add the controls of this type to a layout using *Add item* and delete the controls using *Remove item*. If the editing mode is activated, click a control to show its properties on the right hand side of the dialog *Customization*.

The detailed properties of the controls are described in the "Input and Output Fields" section. Please find some notes on particularly important properties in the context of the selection panel below:

- The "FieldName" is used to map the control value on a service parameter on the one hand, but on the other hand, it may also be used to adopt existing customization settings from other controls automatically, if the field name corresponds to a property and/or a service parameter. It is often enough to select a service parameter to get a completely defined control. The selection list of the service parameters is specified via the contents of the "ServiceName" property - only those service parameters are displayed that are known in the service context. If ServiceName is empty, all known properties are shown.
- Use the "ControlDataSource" to specify a data source suggesting selection values. This can be a selection list or a selection dialog that is opened to select a value.
- The "ControlType" specifies which type of control will be used (e.g. text, date or list control).
- If "ShowSecondControl" is set to "true", the control is shown as a "From To" field.
- The "LanguageKey" is used to set the control label.



Numerous changes in controls are only shown, when you save, close and restart the application!



Once you have created an item, e.g. input field, you cannot completely delete this item using the graphic configuration because the framework used does not support this function. To definitely delete an item that you have accidentally created, refer to section "9.5.1 Deleting items from detail applications".

9.2.5 Data sources

Data sources provide data for detail applications. This connection to a data source can be used to read data or to create, change or delete data. The repository defines for each data source the type of access provided to specified data.

Each application must include at least one data source, but can also include any number of data sources.

Adding/editing data sources

Use the button *Configure data sources* in the toolbar to create a new data source. Already existing data sources are edited via the same button. The dialogs used and the procedure are almost identical for both activities. It is therefore only described here how to create a new data source. You can easily use the description to edit an existing data source, too.

The "Configure data sources" dialog shows the list of the already existing data sources. If no data source was created, the list is empty.

By clicking on "Add", a new data source may now be created. Another dialog to specify the properties of the new data source is now opened¹. The following properties are available:

Data logic

The data logic specifies where the data source gets data. For this purpose, so-called data logics serving as connection to web services exist. The list shows for each data logic the name and a short description.

For each data source, you must specify exactly one data logic. The application designer must know which data logic provides the data required or performs the required functionality using the data (create, edit or delete).

Events

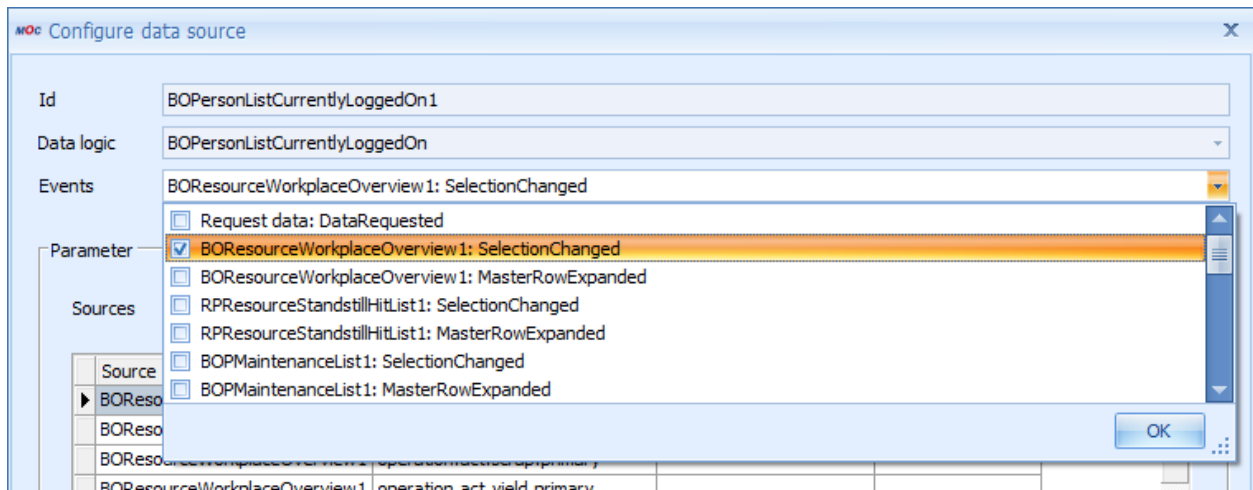
This field specifies the event. The data source then responds to this event. If this event occurs, the data source requests data from its data logic.

By default, only the event "Request data: DataRequested" can be selected. This means that the data source requests data when the green arrow in the toolbar is clicked.

In addition, each data source can respond to the SelectionChanged event of all other data sources created until then. This event occurs if one or more data records are selected in this data source (e.g. if you select a row in the annexed grid). This way, you integrate a master detail connection between the separate data sources. You often use this method to show detailed information for a row selected in the grid, e.g. all operations assigned to an order.

The list of available events is extended with each new data source. If a data source for the data logic WorkplaceOverview was created before, the selection looks as follows:

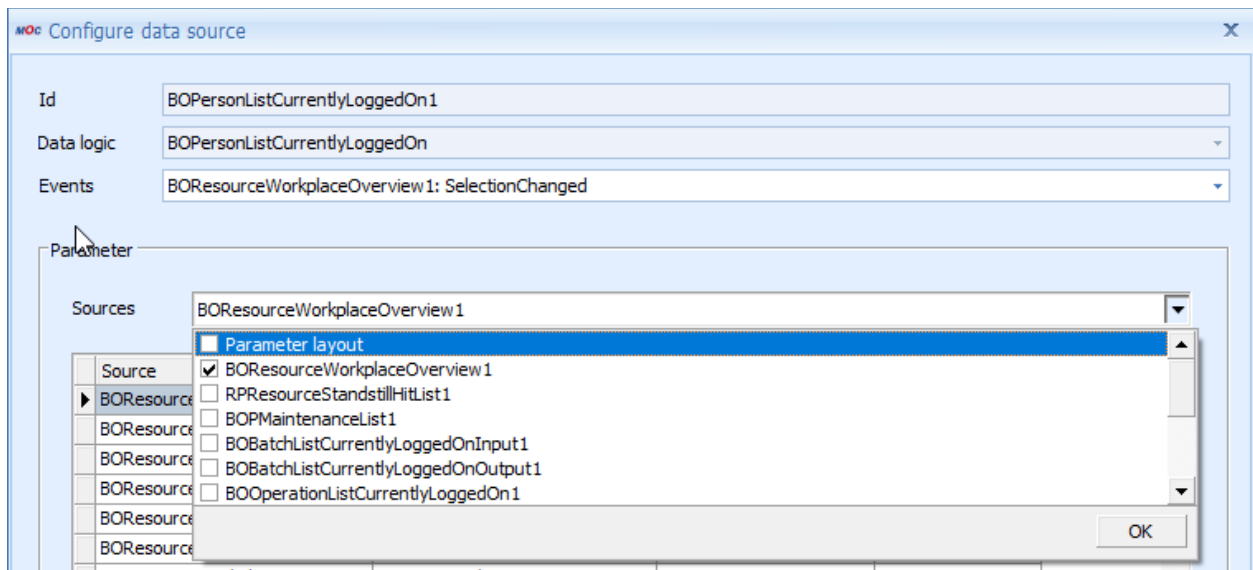
¹ By clicking on "Change", an already existing data source may be edited. A dialog is opened – the same dialog is used to create a data source. The already defined properties of the data source are pre-allocated.



Parameters: Sources²

You can parameterize the connection to the database that is created via data source. Using parameters, you can limit the requested data quantity or define the values of a new data record. The precise use of parameters depends on the type of data source.

The parameter sources specify where the data source gets the parameters. Each data source can get parameters from any number of sources.



If an application only includes one data source, you can only select the selection panel "Parameter Layout" as source. This means that the selection panel provides a parameter from the data source for each field (input field, checkbox, combo box, ...) of the selection panel.

² The parameter source is also called ParameterContainer.

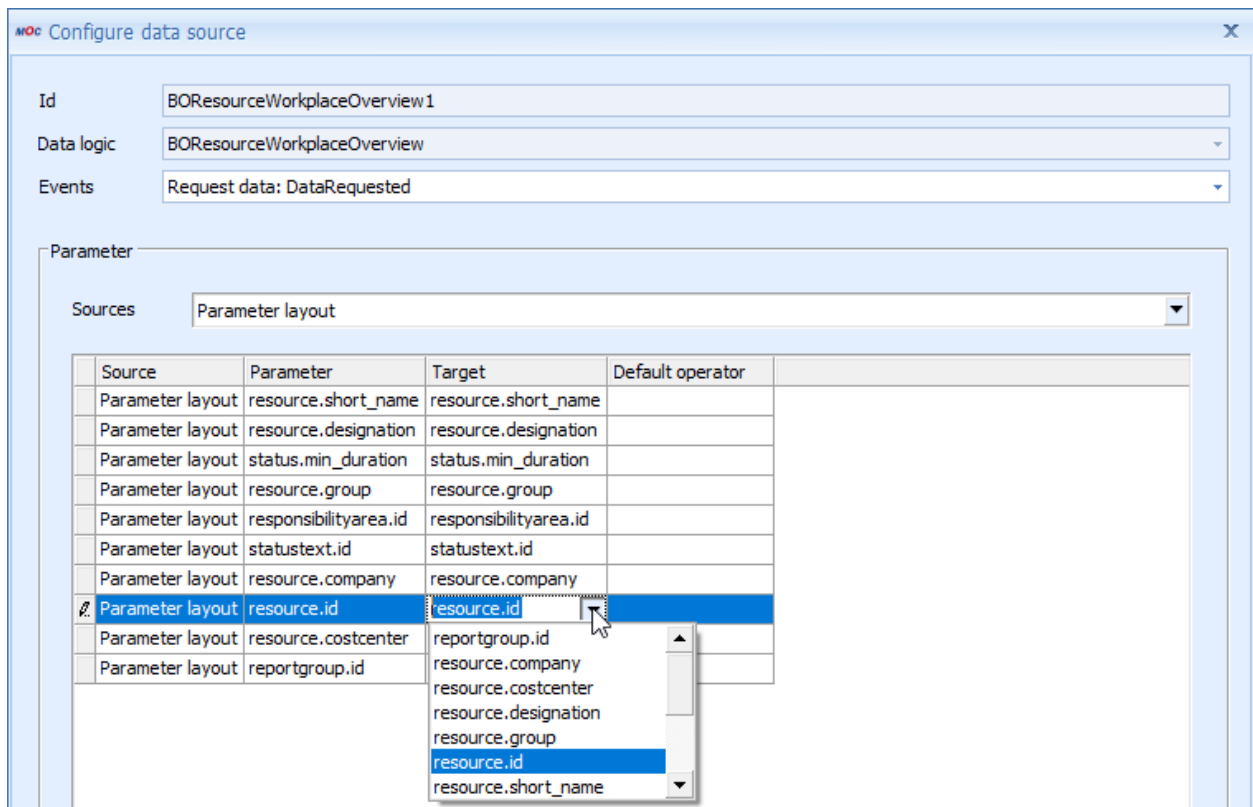
If several data sources are available, each data source can also obtain its parameters from the selected data records of all other data sources created until then. In this case, a parameter is created for each column of the dataset. The list of available parameter sources is extended with each new data source.

Parameters: Mappings

The parameters that a data source gets from different sources are not transferred to the data logic as they are. The user can define which parameters are assigned to parameters that are expected by the data logic. The user can also define how the parameters are mapped. An example can best illustrate the procedure.

Example 1:

In the selection panel, there is a field for the selection criterion *Workplace* with acronym "resource.id". The selection panel was selected as parameter source of the new data source.



Source	Parameter	Target	Default operator
Parameter layout	resource.short_name	resource.short_name	
Parameter layout	resource.designation	resource.designation	
Parameter layout	status.min_duration	status.min_duration	
Parameter layout	resource.group	resource.group	
Parameter layout	responsibilityarea.id	responsibilityarea.id	
Parameter layout	statustext.id	statustext.id	
Parameter layout	resource.company	resource.company	
Parameter layout	resource.id	resource.id	
Parameter layout	resource.costcenter	reportgroup.id	
Parameter layout	reportgroup.id	resource.costcenter	

In the left column you see where the parameters are from. In this case, it is the selection panel. The second column shows how the parameters are identified within the selection panel. In the third column, a list of all parameters expected from the web service is shown. Now the requested target parameter may be selected for each parameter from the selection panel.

It is possible to map up to two parameters on the same target parameter. This will automatically be interpreted as from...to relation, e.g. to limit time ranges. At present you are not allowed to allocate the same target parameter to more than two parameters. In this case, the relevant parameters are not used in a data request.

The parameters of the selection panel can be allocated freely when fields are created. Note: due to the mapping, names of source and target parameters need not be identical. But the property `FieldName` of fields of the selection panel is used as key to identify specific properties like input screen, field width, label, etc. Carefully select the `FieldName` for this reason.

Example 2:

A data source for the WorkplaceOverview data logic has already been created and selected as parameter source for the new data source.

Source	Parameter	Target	Default operator
BOResourceWorkplaceOverview1	resource.group		
BOResourceWorkplaceOverview1	resource.id	bookingrelation.workplace	
BOResourceWorkplaceOverview1	resource.predicted_downtime		
BOResourceWorkplaceOverview1	resource.remaining_downtime		

Here, too, the two first columns show where the parameters are from and how they are called. In this case, one parameter, each, is offered for each field provided by the WorkplaceOverview data source. Since these fields, however, should normally not be included in the requests of the new data source as parameters completely, only those also allocated to a target parameter are considered.

Typical case in practice: WorkplaceOverview is used as data source for a grid. The new data source requires a machine number as parameter to identify all operations logged on to a specific machine, for example. In this case it is not reasonable to transfer all available parameters. Only the machine number (`resource.id`) is therefore mapped on the target parameter.

Note: In many cases, the acronyms of the source and target parameters are identical. This is often the case, but not always.

You can enter the operator in the fourth column that is used to map input to service parameter. In the example, the definition remains empty; this is the default operator "equals". Other options that are used less often are "like", "in", "lessThan", "greaterThan", etc.



If a parameter defined in the layout is not displayed as data source "Parameterlayout" in the list of available values, you must check if the relevant control has been properly configured. For example, if the data type of the parameter does not support an array (e.g. datatype "boolean" or .NET type "duration") and if the control has been defined using "ShowSecondControl=True", then the parameter is not included in the selection list because this assignment is not valid.

Cache time

This is used to specify the time in seconds which is added to the data logic when data is requested. Within this period, new requests with identical parameters are served from the cache. This value will overwrite a default value for the entire MOC.

Update rate

Time in seconds after which this data source will automatically and regularly request data. If the value is 0, the default value of the total application (see above) is used. If data is requested manually, the time is reset.

Column configurator

The column configurator is used to limit the data requested. Each data logic usually provides a very large number of fields (sometimes up to 1,000) because it is of universal use. But not the total number of fields needs to be displayed in a detail application. For reasons of performance, it is not reasonable to request data for fields which are not required anyway, the column configurator is used to specify the columns where data is actually requested.

The column configurator may be maintained via an additional dialog for each data source. All fields provided by the selected data logic are listed.

When creating a new data source, all fields appropriately identified in the repository are selected by default. If the data source is saved and the column configurator is not opened, then this preset selection is automatically used. To change the selection, select the required fields and close the dialog via OK.

Default values

A default value may be defined for each parameter expected from the selected data logic. This value is automatically allocated to the relevant parameter if no value is supplied by any parameter source.

Default values are also maintained via an additional dialog. All parameters expected from the web service are listed here.

The default value may be edited as free text. It is important, however, that the expected data type of the parameter is observed. The following values are possible:

Data type	Possible values
string	Deliberate character strings
integer	Only numerical values
decimal	Numerical floating point values
boolean	0, 1, false, true
datetime	Valid date character strings, e.g. 2009-02-01 or 02-01-2009.

If the "Array allowed" column includes the value "true", then a list of values can also be transferred to the parameter. You can enter a list of values; separate the values via semicolons (;).

If an invalid value is entered, this is notified to the user. The value will not be adopted in this case.

ResultSet

There are also data sources which may return several ResultSets. In this case, there are the following possibilities for customization:

- The data source is used by a specialized detail application plug-in capable of handling the supplied data structure (e.g. machine time profile, PDV evaluations/reports).
- In a detail application that can only process simple ResultSets, the first ResultSet is automatically displayed (important: the selection of the first set can be a selection by chance!).
- To specify the ResultSet that the DataController provides to "simple" detail applications, you can specify the name of the ResultSet to be used in the customization file. Currently, you can only do this by manually editing the file. You can find the name of the ResultSet in the repository column with the same name.

```

<DataControllerDescriptor>
  <Id>RPPProductionReportingStatusAnalysisByClassification</Id>
  <DataLogic>RPPProductionReportingStatusAnalysisByClassification</DataLogic>
  <ParameterContainers>
    <Mappings>
    <DefaultValues>
    <ColumnConfigurator>
    <CacheTime>0</CacheTime>

```

```
<RefreshRate>0</RefreshRate>  
<UpdateParametersByService>false</UpdateParametersByService>  
<ResultSet>profile</ResultSet>  
</DataControllerDescriptor>
```

Important: If the result for the data source is specified, the same ResultSet is made available for all detail applications using this data source. If a specific detail application requires a different ResultSet, you must specify this via script.

Deleting data source

To delete a data source, the requested data source must be selected in the list of all data sources already created.

Click *Remove* to remove this data source from the application. For security reasons, the user must confirm the operation.



You can only delete a data source that is not used yet.

You can use a data source as

- Event "provider" for other data sources;
- Parameter source for other data sources;
- Data source for one or more detail applications.

In this example, all three conditions are true. The data source is actually used and you must check again if you really want to delete the data source. If yes, you must first remove all connection to the data source. With the data source "PersonListCurrentlyLoggedIn2, you must remove the data source from the list of parameter sources and events. The "Workplace overview" detail application must also be deleted because no other data source can be selected for it subsequently.

9.2.6 Detail applications

Detail applications are used to present data in an application. There are different types of detail applications. Some of them are very universal and are therefore used quite frequently, e.g. grid, chart, detail display or pivot grid. Some of them refer to a special application case, e.g. cycle progression chart.

When you create a detail application, you specify its type, i.e. how data is presented. This specification is fundamental and cannot be changed once a detail application has been created.

Creating / editing detail applications

To create a new detail application, click the toolbar button *Configure detail applications*. Already existing detail applications are edited via the same button. The activities of creating and editing dialogs are very similar and the same dialogs are used. Below it is therefore only described how to create a new detail application. You can easily use the description to edit an existing data application, too.

The dialog *Configure detail applications* opens and a list of the already existing detail applications is shown. If no detail application has been created before, the list is empty.

Click *Add* to create a new detail application. Another dialog to specify the following properties is now opened.

ID

The ID must be clear within the application because the ID is used to clearly identify the detail application.

Caption

The text entered specifies the title of the detail application. The detail application is displayed within a dockable panel. The text entered here is displayed in the title bar of the panel.

Note: enter a language key here to guarantee the correct translation when changing to another language.

Please note: When the detail application has been created, the translation of the language key will directly be displayed in the list of existing detail applications. Requirement: the language key entered must already exist.

Application category

The application category specifies the form used to display the data. There are general application categories used for numerous different detail applications (chart, table view,...). There are also very special application categories that are only used for very special detail applications. It is possible that an application category is specified for one single detail application only. In this case it is possible that the data source, which provides the data, is specified in a fixed manner for the detail application. Example: the cycle progression. If the cycle progression is added to an application, the assigned data source ResourceCycleProgression with default values is automatically created, too. You can edit the data source later on when the application has been created.

It is also possible that the data source has already been created. In this case, the data source is not created again, but the already existing one is automatically assigned to the detail application.



In case of already existing detail applications, the application category cannot be changed subsequently.

The cycle progression, which includes its own data source, is a special case. This application category does not only include a chart for the display of data, but also additional control elements to limit data (e.g. radio buttons to select shifts/hours). This detail application therefore does not only provide its own data source but also its own parameters and its own events. The user expects that the data displayed is adapted to the selection upon a click on a radio button.

Events

Like data sources, some detail applications can respond to events. This always depends on the application category. If you select a relevant application category, this is possible. The selection list is then activated and the user can select all events where the detail applications of this type can respond.

Important difference to events with data sources: a data source always responds to an event in the same way, i.e. data is requested. With detail applications, the response is not clearly specified, but depends on the category of the detail application. For information on the response of a detail application to an event, refer to the documentation of the relevant detail application (see **Error! Reference source not found.**).

Data source

If the data source has not been specified automatically with the selection of an application category, you can select a data source here. All data sources are available, which have been created as described in 0. Select the relevant data sources. The new detail application will then get its data from the selected data source.



In case of already existing detail applications, you cannot change the data source subsequently.

Ribbon page

The ribbon page entered here is activated as soon as the detail application is focused.

Additional data sources

Some detail applications support the use of several data sources. If you select such an application category, this selection list automatically becomes active.

Authorization key

Authorization keys to control authorizations that are required to display this detail application. If no authorization key is defined, the detail application will always be opened.

Help file and Help index

Name of file where help subjects for this detail application are listed and/or description of the entry point within this file.

Icon

Selection of the icon displayed in the title bar of the detail application. Select the icon using an image selection dialog. After having selected an icon, the icon is displayed on the button that opens the dialog.

Show selected data records only

Usually, all data records of the selected data source are displayed in a detail application. In some cases this is not reasonable. Example: the application category "Detail view" is normally used to display specific values of a single data record in different fields.

Typical application case: several workplaces are shown in a table view. Next to the table and for reasons of overview, you want to show the ID, status, group, cost center, etc. of a single workplace in a detail view. The detail view must always provide details on the workplace selected in the table.

Here, both detail applications show data of the WorkplaceOverview data source, either all data or only one data record.

The same data source is assigned to both detail applications. But in the detail view, you only want to show the selected data record. You therefore enable the option *Show selected data records only*. All data records highlighted in the grid are selected. If several data records are highlighted, the data of the data record highlighted last is displayed in the detail view.

Column configurator

Use the column configurator to narrow down the data available in the detail application. In an ideal case, the relevant data source has already been configured accordingly and provides only the fields required. But some applications are very special and cannot "handle" all fields of their data source (e.g. specific charts).

You can edit the column configurator via an additional dialog for each detail application. All fields provided by the selected data source are listed (see screenshot 0).

When you create a new detail application, all fields are selected by default. If the detail application is saved then without opening the column configurator dialog, this pre-setting is automatically adopted. To change the selection, select the required fields and close the dialog via OK.



If no data source has been selected yet, this list is empty because the fields displayed are based on the data source.



If an additional column is added to the column configurator, a field is not automatically added to the detail application!

Deleting detail applications

To delete a detail application, select the required detail application in the list of all detail applications already created.

Click *Remove* to completely delete this detail application from the application. For security reasons, the user must confirm the operation.



You cannot use a detail application as a data source. In general, you can always delete a detail application. But if a data source has been created automatically for a detail application, this data source is not automatically deleted, too. If it is no longer required, you must delete the data source manually (see 0).

Showing detail application

When you have created a detail application, it is automatically loaded into a dockable panel and displayed in the application. You can place and resize the panel using drag and drop.

The detail application is automatically "initialized". For example, this can mean that in a grid, all columns are automatically added and shown that are available in the data source of the application.

Configuring contents of the detail applications

You are free to configure the contents of the different detail applications. With very simple detail applications like the type *Layout*, the fields displayed are configured like the selection panel (see 9.2.4). The configuration of more complex detail applications is described in the following sections.

9.3 Customization of editing applications

Editing applications are very similar to main applications. Their structures are nearly identical and they provide almost the same configuration options.

This section describes the customization options provided by editing applications. Many customization options are identical to those provided by main applications. This section therefore focuses on the differences or additional possibilities.

9.3.1 Application generator

Editing applications are usually created along with the relevant main application in an automated manner using the application generator.

For a detailed description of the procedure, refer to the document [MDS-ApplicationGenerator.pdf](#).

9.3.2 Additional customization options

In general, editing applications provide the same configuration options that are described above. Some further settings are only available for editing applications and are only active if an editing application is called.

When you create the application, some reasonable values are preassigned to these additional settings.

Currently, you can only perform a subsequent customization if you edit the main customization file of the application (<NameOfEditingApplication>.config in the application folder of the editing application).

ParentController

Name of the relevant data source of the higher level main application.

The application generator initially sets this setting automatically. To make subsequent changes, you must manually change the above-mentioned customization file.

Subject to the `ParameterProcessingMode`, the values from the currently selected row of this data source are transferred to the editing application and assigned to its fields.

Subject to `UpdateMode/UpdateSourceMode`, the data of this data source is updated after performing the editing function.

ParameterProcessingMode

Specifies if and how parameters from the calling main dialog are transferred to the editing dialog.

The application generator initially sets this setting automatically. To make subsequent changes, you must manually change the above-mentioned customization file.

The following options are available:

- `NoParameter`: no parameters are transferred.
- `SingleParameter`: transfers the selected grid row (mandatory); an error message is displayed if no row is selected.

- OptionalSingleParameter: transfers the selected grid row (optional); no error message.
- MultiParameter: transfers one or several grid rows (mandatory); an error message is displayed if no row is selected.
- OptionalMultiParameter: transfers one or several grid rows (optional); no error message.
- SingleSelectionParameter: transfers the selected grid row. If no row is selected, the values of the selection panel are passed. If the selection panel neither transfers values --> an error message is displayed.
- OptionalSingleSelectionParameter: transfers the selected grid row. If no row is selected, the values of the selection panel are passed. No error message.
- SelectionOnlyParameter: transfers the values from the selection panel. If no values are transferred → error message.
- OptionalSelectionOnlyParameter: transfers the values from the selection panel. No error message.

UpdateMode

Specifies how data is changed via editing function.

The application generator initially sets this setting automatically. To make subsequent changes, you must manually change the above-mentioned customization file.

The following options are available:

- Insert: new data records are added
- Update: existing data is changed / edited
- Delete: existing data is deleted

UpdateSourceMode

Specifies how data is updated in the main application, once the editing function has been executed.

You can make this setting initially via the application generator. To make subsequent changes, you must manually change the above-mentioned customization file.

The following options are available:

- All: all data of the main application is completely refreshed (using the values entered in the selection panel). This is required, for example, for separate delete and copy dialogs. Because here, you cannot identify the relevant data records and only request these data records.
- OnReturnValues: the service called returns values that are used to request only the relevant data. This may be the case, e.g. for "Insert". In this context, it is often the case that a new key field is generated that is required to request the new data record.

Important: if the service called does not return any fields that can be used to clearly identify a relevant data record, then this setting does not make sense. This setting is set by default for *Insert* and must be changed, if necessary.

- OnKeyValues: the key fields of the list service are used to request only the relevant data. This is the ideal procedure for the *Update*.

Important: the service of the main application and the editing service must have the same key fields; otherwise it does not work.

- OnScript: for future use

9.3.3 Process configuration

To configure the processes for editing functions as precisely as possible, a process configuration exists for each editing application in the file ProcessConfiguration.config.

The application generator automatically generates this file when the application is created. The file is directly stored in the application directory of the application.

Another section of this document describes the process configuration in detail.

9.4 Calling external programs

You can call external programs from the MOC via the menu or from an entry in the toolbar of an application.

9.4.1 Menu

To call an external program from the menu, you make an entry in the menu editor:

The following values are relevant:

- Enter "StartExternalProcess" as command and ID
- Enter the program to be run in "Parameter"

For further details, refer to section "Parameters"

Command	Labeling	ID	Command	Parameter	Im
Test Notepad	Test Notepad	StartExternalProcess	StartExternalProcess	path="notepad" example.txt	

9.4.2 Toolbar

To start an external program from an application, you can call the program via toolbar

You use the link editor for customization.

- Function: callCommandObject
- Parameter
Program to be executed including parameters
For further details, refer to section "Parameters"

ID	Call external Program
Command	StartProcess
Function	callCommandObject
Parameter	StartExternalProcess path="notepad" example.txt
Authorization key	
Visibility	
Title	lkEditExampleTxt
Tooltip	lkEditExampleTxt
Category	
Ribbon page	lkMainPage
Context	

9.4.3 Quick launch bar

Directly enter CommandLink to call the program

Example:

StartExternalProcess path="notepad" example.txt

9.4.4 Parameter

To call external programs, the following parameters are supported

Path

This parameter defines the program that is called

-> The "path" parameter **must** be specified.

-> The transferred value has to be enclosed by double inverted commas.

The specified program must be executable and/or if the program cannot be called automatically, the program including the entire program path must be entered

All other parameters are optional and transferred to the executing program as parameters.

Parameter

Variables can be transferred in square brackets.

List of parameters

These parameters are possible:

UserName:

logged on user

MesInstanceName:

registered system

MesInstanceUrl:

Url of the registered system

Language:

Language selected by the user (format: en-US)

All Application Settings

If you put [CONF:] in front, you can transfer any ApplicationSettings in the call parameters,

e.g.:

CONF:LookAndFeelSkin

the skin selected by the user

The complete list of possible application settings can be shown if you enter the transaction code "syssettings" on the MOC

Examples:

Starting the MaintenanceManager for the system logged on:

StartExternalProcess path=" iexplore.exe" "http://[MesInstanceUrl]/MaintenanceManager/"

Starting any program and transfer of skin

StartExternalProcess path="irgendeine.exe" Skin=[CONF: LookAndFeelSkin]

9.5 Tips & Tricks

9.5.1 Deleting items from detail applications

Background

You can use the context menu of the graphic configuration to delete items, e.g. input fields. But the framework used does not definitely delete the items. It only hides the items and they still exist as "hidden item". The framework does not provide a technical option to definitely delete an item created for a detail application.

Solution

If it is not enough to hide the items and if the items created must be deleted definitely, you must manually change the configuration files. To do so, you must delete XML entries in the layout configurations of the detail applications using a text editor or an XML editor.

Layout files are files with the extension "*.config" that include the text "layout" in the name. These files are stored in the directory of the application configuration of the MOC in the relevant scope. Example:

- d:\Moc\local\conf\Moc\Apps\Units\LayoutPanel.config
- d:\Moc\local\conf\Moc\Apps\Units\LayoutMainDetail.config
- d:\Moc\local\conf\Moc\Apps\Units\MDUnitsInsert\ParameterLayoutMDUnitsInsert.config

In the relevant XML file, search for the "FieldName" of the item that you want to delete (in our example "example.item.to.delete") and delete an XML node at two places.



When you edit XML files, the MOC must not be started. The changes are enabled with the next start of the MOC.

Deleting the item

```
<property name="Text"></property>
<property name="CustomizationFormText">SI-Einheit</property>
<property name="StartNewLine">false</property>
<property name="Visibility">Always</property>
<property name="TextLocation">Left</property>
</property>
<property name="Item7" isnull="true" iskey="true">
  <property name="TypeName">LayoutControlItem</property>
  <property name="ControlName">example.item.to.delete</property>
  <property name="AllowHtmlStringInCaption">false</property>
  <property name="TextAlignMode">UseParentOptions</property>
  <property name="SizeConstraintsType">Custom</property>
  <property name="Image" isnull="true" />
  <property name="ImageIndex">-1</property>
  <property name="ImageAlignment">MiddleLeft</property>
  <property name="ImageToTextDistance">5</property>
  <property name="OptionsPrint" isnull="true" iskey="true">
    <property name="TextToControlDistance">-1</property>
    <property name="AllowPrint">true</property>
  </property>
  ...
  <property name="TextLocation">Default</property>
</property>
...
```

Delete the complete item property (in the example Item7) from the opening to the closing tag (underlined).



The items contain numbers (e.g. "Item7") The numbers need not be consecutive numbers. It is therefore not necessary to assign new numbers to the succeeding items when an item has been deleted.

Deleting the settings of an item

```
<Setting Key="mpdvEdit example.item.to.delete" Description="" LastChanged="2017-07-27T08:16:55.8650007Z" Value="<Value>
  <ConfigurationItem xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns="http://www.mpdv.de">
    <Acronym>example.item.to.delete</Acronym>
    <ServiceName />
    <Label>lkTest</Label>
    <Length>0</Length>
    <FillChar xsi:nil="true" />
    <ShowInGrid xsi:nil="true" />
    ...
    <CanNotEqualOrNull xsi:nil="true" />
    <TransferEmptyValuesToHydra xsi:nil="true" />
    <IsResult xsi:nil="true" />
    <ShowSecondControlInSearch xsi:nil="true" />
    <LoadDataOnInit xsi:nil="true" />
  </ConfigurationItem>
</Value>
</Setting>
...
```

Delete the complete setting from the opening to the closing tag (underlined).

10 Process Configuration

Processes by means of which one or more similar service starts may be performed can be configured in the MOC.

Please note: At present, such processes are exclusively used in the course of maintenance applications. The application generator creates standard processes for the activation of maintenance applications.

For this purpose, the processes support the following steps

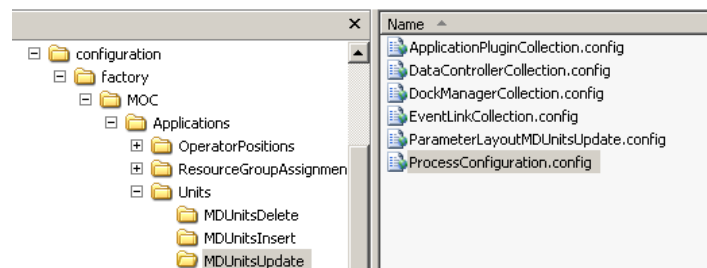
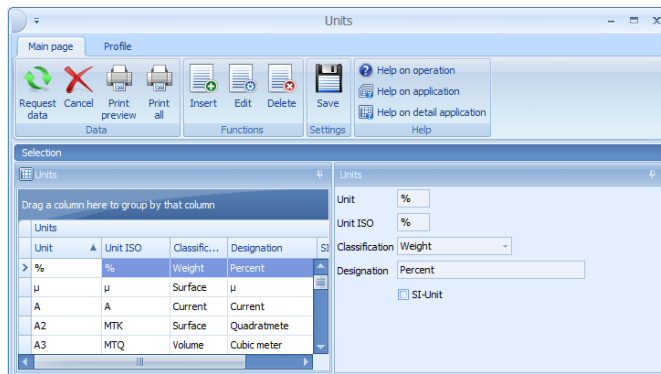
- Transfer of one or more parameter records from a data source. A parameter record describes the parameters to be transferred to the service. If more than one parameter record is transferred, the service is performed several times.
- Display of a dialog requesting the user to confirm the service activation (e.g. confirmation before activating DELETE services).
- Modification of a parameter record by a script or collection of a parameter record by a service activation.
- Activation of preparatory services (e.g. LOCK before UPDATE or NEW before INSERT).
- Processing or entry of parameter records in a dialog by the user (e.g. entry of values for INSERT services).
- Activation of a service with the parameter record.
- Activation of a script if the service returns an error (e.g. if a key value is missing) or general activation of a downstream service (e.g. UNLOCK for UPDATE services).

10.1 Processes

Each process requires one or more data sources provided with parameters from different parameter sources as a minimum requirement. For this purpose, an `ApplicationContainerForEdit`, via which the data sources and the relevant parameter mappings can be configured, is used as a basic component for each process.

This also applies to processes for services not requiring any explicit input dialog, e.g. the activation of DELETE services. As regards service activations with an input dialog (e.g. INSERT), this dialog is implemented as the plugin of an `ApplicationContainerForEdit` (usually a `LayoutApplication` plugin).

The process is configured by means of the `ProcessConfiguration` containing a specified number of `ProcessConfigurationItems`, which are each evaluated in relation to specified points. The `ProcessConfiguration` is a characteristic of the `ApplicationContainerForEdit` and is saved in a separate configuration file (`ProcessConfiguration.config`).



The figure shows the distribution of the different components of an application to the configuration files. In the example, the three functions "Insert", "Process" and "Delete" were additionally configured for the application "Units". The application is saved in the folder ".\MOC\Applications\Units". Each of the functions has its own `ApplicationContainerForEdit`, saved in a separate subfolder. In the figure, the subfolder "MDUnitsUpdate" has been selected, so that its configuration files are visible. The file "ProcessConfiguration.config" defines the process by means of which an update can be executed.

When the application generator is used, standard processes for Insert, Update, Delete and Copy are automatically predefined if necessary. At present, the configuration of other functions and/or the adaptation of the generated processes is implemented by editing the file `ProcessConfiguration.config` (there may be an editor for this in the future).

10.2 Structure of a process

At present, all processes have the same structure, i.e. they work according to the same pattern. A `ProcessConfiguration` consists of a list of `ProcessConfigurationItems`, run in a fixed sequence.

The process of an `ApplicationContainerForEdit` is started by the command `<container>.ProcessExecute`. Subsequently, the following steps are run through.

- Identification of the selected data in the `ParentController`, i.e. the "relevant" `DataController` of the activating `ApplicationContainer`. In general, the user will have highlighted one or more lines in a grid; these will be identified by this.

- Activation of `ProcessConfiguration.Prerequisite`, e.g. used to check whether the process is to be implemented.
- Identification of the number of steps - if more than one data record is selected, the process is run through the corresponding number of times.
- In each step
 - Activation of `ProcessConfiguration.StepParameterModification`, to check, modify or initially identify parameters (e.g. NEW Service).
 - Activation of `ProcessConfiguration.StepPrerequisite`, to check whether a process may be implemented or whether other preconditions (e.g. LOCK Service) are executed.
 - Depending on the type of process, either the dialog is opened (`ProcessTypes.Dialog`), or the process is implemented directly (`ProcessTypes.Direct`). In both cases, `ProcessConfiguration.StepExecute` is implemented; this is where the "main service" is activated.
 - Activation of `ProcessConfiguration.StepFinish`, in order to perform a follow-up treatment (e.g. UNLOCK Service).
- After all steps are completed, activation of `ProcessConfiguration.Finish`.

If exceptions are generated in the process, they are either intercepted at the end of the step loop, so that the user can decide whether to continue with the next steps, or the entire process is canceled.

10.3 Structure of `ProcessConfigurationItems`

The steps described above are each configured by `ProcessConfigurationItems`. The following characteristics can be defined for each item.

- **Type**: a value of `ProcessItemTypes`, e.g. `Prerequisite`, `StepExecute`, `StepFinish`. This value is automatically set to a `ProcessConfiguration` when an item is assigned and is primarily used for information and debugging purposes.
- **PreScriptName**: Name of a script to be executed before the execution of a service. If no name is indicated, the script is searched on the basis of the default name.
- **SingleItemText** and **MultiItemText**: Text (and/or language key) displayed in a Yes/No dialog before the service is activated. If the user clicks No, the entire process step is interrupted. If 0 or 1 parameter was transferred, the `SingleItemText` is displayed; if several parameters were transferred, the `MultiItemText` is displayed.
- **ServiceName**: Name of the service to be activated in this process step.
- **PostScriptName**: (Optional) name of a script provided by MPDV, to be executed before the execution of a service.

When a `ProcessConfigurationItem` is "executed", the above-stated characteristics are run through in the sequence presented, if they have been defined.

10.4 Special case: Activation of a process from another application

Some processes are needed within different applications, e.g. 'Set operation status' is to be activated from the order overview AND the operation maintenance. In this case, you do not want to define this process repeatedly and then have to update it at different locations in the case of modifications.

For this reason, it is also possible to activate a process already defined for an existing application from another application. For this purpose, the configuration has to be adapted as follows:

- Add a new button to the toolbar, by means of which the process can be activated.
- Function: `openApplicationContainerForEdit`
- Parameter: string with the following format: `id=<Value> parentSetting=<Value> callerController=<Value>`
 - o `id`=Name of the process to be executed
 - o `parentSetting`=application to which the process to be activated is associated
 - o `callerController`=main controller of the application from where this process is to be activated

Example: The process `operationsetStatus` of the application `OrderOverview` is to be activated from the screen `EditOperations`. In this case, the correct string for the parameter entry of the toolbar button would be:

`Id=operationsetStatus parentSetting=OrderOverview callerController=BOOperationList`

Important: It does not, of course, makes sense to use this randomly, but only if the data of the activating application indeed corresponds to that of the maintenance dialog.

11 Components

The following sections describe the components (application plug-ins) that you can integrate in detail applications to display data.

11.1 Table view (grid)

The grid is used to display data in a tabular form. It provides several columns, which are grouped in so-called categories.

If you place a detail application of the "Table view" type on an application, all columns and categories are created automatically using the specified data source and according to the Data Description. On basis of this "default setting", you can then configure the grid.

Most configuration options are provided in the main context menu. Right-click a column header to open this context menu. If the Customizing mode is not enabled, some of the options are not available. The sections in the following mainly concentrate on the functions of this main context menu.

Grid properties

You can use the dialog "grid properties" to make advanced configurations for the column properties. To edit the properties of a column, select the column in the selection list (default is the column clicked on). The most important properties are:

- **LanguageKey:** Language key used for the column header. Important: Changes of the column header via the "Caption" property are invalid because they are overwritten with this configuration. Changes only become visible after closing and re-opening the application.
- **Tooltip:** Text displayed when the mouse is moved over the column header.
- **FieldName:** Name of the field from DataDescription, whose values are displayed in this column.
- **SummaryItem:** Formatting of column totals (see 0)
- **BackColorField:** Field name of column that specifies the color code for the background color. There are columns that include numerical color codes. If you specify their field names for a column, the respective color code is used for the background color of the column. You can use the configurator to hide the color column.
- **ForeColorField:** see BackColorField. Here, you specify the font color.
- **HyperLinkField:** Field name of column used for the hyperlink of this column. Some columns include URLs. If you enter their field name for a column, a double click on a field of the original column opens a URL. If the URL is a mail address, a "mailto" is automatically added. You can hide the actual hyperlink column during this process.

Moving columns/categories

You can move all columns and categories in the grid using drag and drop. Use the column configurator to hide columns or categories (main context menu, *Column chooser*). When the configurator is open, you can simply move the columns and categories that you want to hide. You can also use the configurator to show hidden columns and categories.

Sorting

To sort the grid by specific columns, simply click the header of the respective column. If you click the same column a second time, sorting is changed from ascending to descending. If you click a new column, the previous sorting is discarded unless the shift button is pressed at the same time.

You can also use the menu for sorting. The options *Sort ascending*, *Sort descending* and *Clear sorting* have the same effect.

Filter

In the grid, you can filter by specific values in the columns. Just click the pin next to the column header. A selection list of all values in this column is shown. This is the same procedure as filtering in MS Excel.

If you have set a filter for a column, this filter is displayed at the bottom left of the grid. Click the checkbox to remove the filter.



You can edit and refine the filter subsequently. For this purpose, the button "Edit filter" is shown at the bottom right of the grid. Click the button to open the filter editor that you can use to combine several filter criteria.

- Red text: Click the red text to open a list of possible linking methods for several filters.
- Blue text: Click to show a list of columns included in the grid. You can define a filter for these columns.
- Green text: Click to show the list of available comparators for a filter.
- Plus: A new filter is added.
- Cross: The respective filter is removed.

You can also open the filter editor via the menu.

Grouping

You can group the grid by one or several columns using drag and drop. To this end, drag a column header to the field on top of the grid

Right-click the "group by" field to open a dialog. *Maximize/Minimize* expands/collapses all groups. Click "Clear grouping" to delete the complete grouping. You can also drag and drop the column back to the grid.

You can also use the menu for grouping. Select *Group by this column*. You can change the visibility of the field by selecting *Show/hide group by box*.

Add/remove columns

You can manually add new columns to a grid. If you click *Add column*, a dialog opens. Enter the following values to create new columns:

- Name: Clear name of column
- Field name: Name of the field from *DataDescription*, whose values are displayed in this column.
- Header: Language key used for the column header.
- Category: Column category to which the column is to be allocated. The category must already exist. You can also move the column to another category subsequently.

Click *Remove column* to remove columns from the grid.

Add/remove categories

Use the menu function *Add category* to add new column categories. The name entered should be a valid language key in order to ensure correct translation.

Use *Remove category* to remove a category from the grid. Note: This is only possible with categories that do not include any columns.

Use *Rename category* to change the name of a category.

Fix categories

You can fix column categories on the left hand side of the grid using the menu function "Fix category". This means that the fixed category is no longer affected when scrolling through the grid. This is useful if you want to permanently show one category, irrespective of the scrolling position.

The category might be moved to the left if it is fixed.

Column totals

In the grid, you can show the sum total for groups (*Show column totals for group*) and for overall columns (*Show overall column totals*).

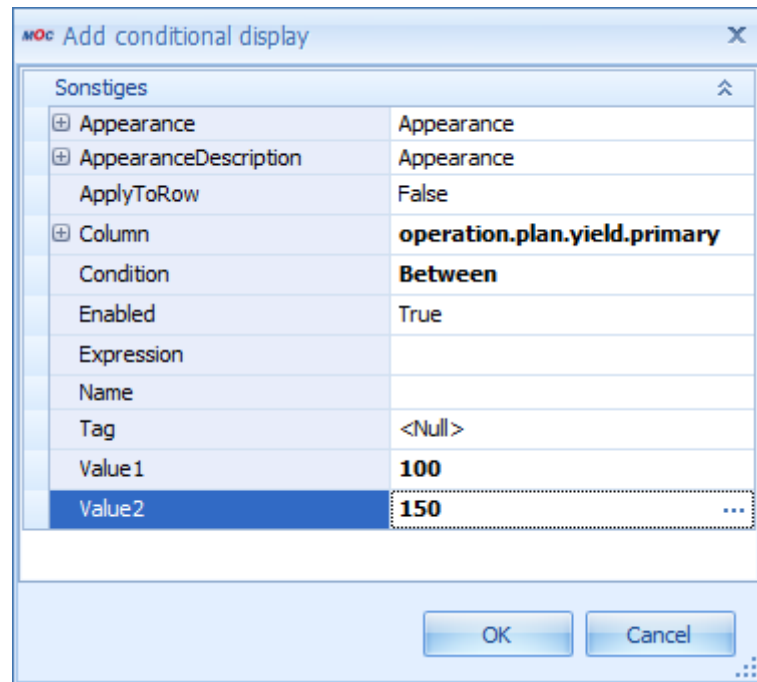
You can define the calculation of the sum totals via grid properties in *SummaryType* of the required column:

SummaryItem	DevExpress.XtraGrid.GridColumnSummaryItem
DisplayFormat	{0:mpdv_timespan}
FieldName	bookingrelation.operatorposition
SummaryType	Custom
Tag	TimeSpanSum

- **DisplayFormat:** Format string used to format the display. Possible formats are standard formats, e.g. {0:n0}, or the custom format {0:mpdv_timespan}. You can use the latter to specify a time for a value in seconds.
- **FieldName:** Name of the field from DataDescription, whose values are displayed in this column.
- **SummaryType:** there are different types of summaries. The most common ones are Sum, Count and Custom (see DisplayFormat and Tag). **Tag:** Only required if the custom format {0:mpdv_timespan} is used. In this case, TimeSpanSum must be entered here.

Conditional display

You can change the presentation of a grid column depending on the value in the cells. Open the dialog *Configure conditional display* that lists the conditions that are already configured. The name of a condition is composed of the major properties of that condition. Click *Add* or *Edit* to open the following dialog where you can edit a new or an existing condition:

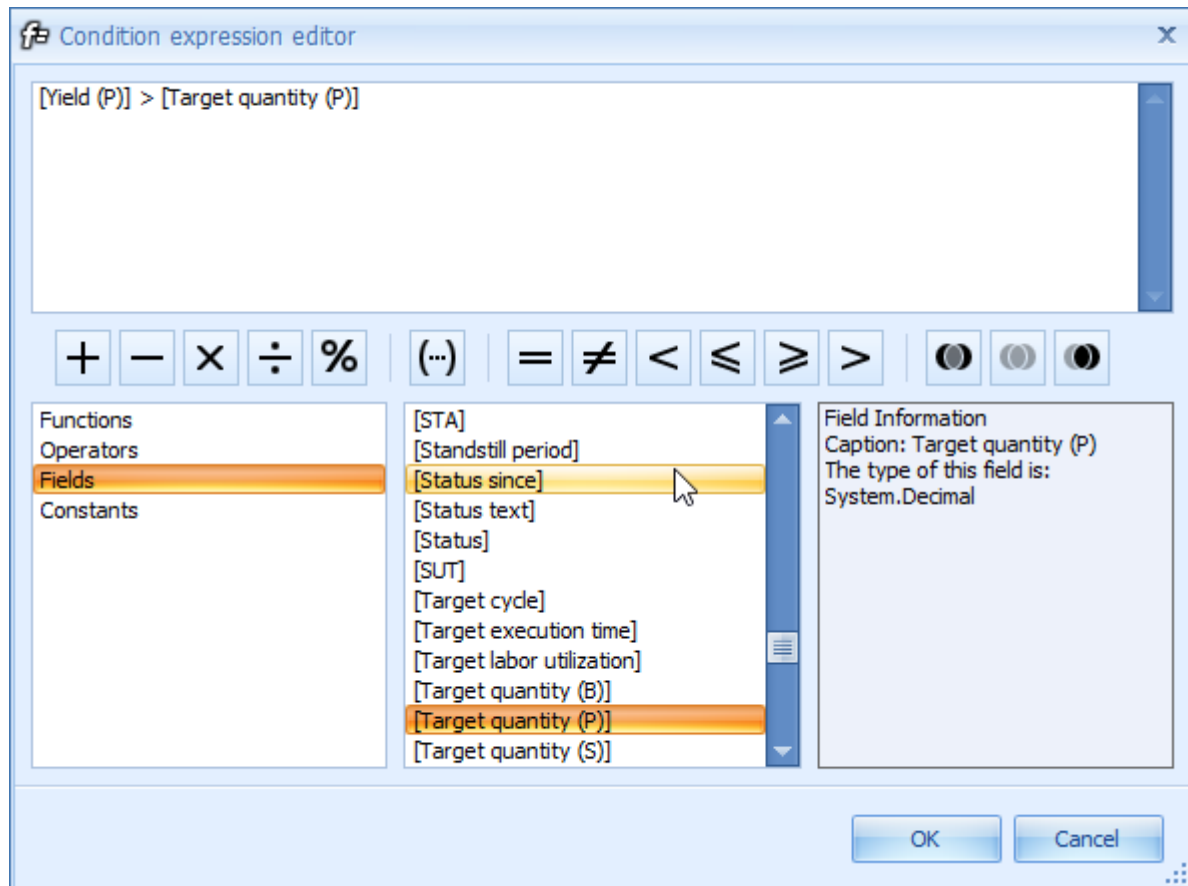


Sonstiges	
Appearance	Appearance
AppearanceDescription	Appearance
ApplyToRow	False
Column	operation.plan.yield.primary
Condition	Between
Enabled	True
Expression	
Name	
Tag	<Null>
Value1	100
Value2	150 ...

OK Cancel

In the image, all cells of the column operation.plan.yield.primary are colored in red, if their value is between 100 and 150.

- **Appearance:** Appearance summarizes all properties that affect the display of a grid column. For example, you can change the text type or the background color. If a cell of the selected column fulfills the condition defined below, the specified appearance is used for this cell.
- **Column:** Column the condition is defined for.
- **Condition:** Condition to be met.
- **Value1:** First comparative value.
- **Value2:** Second comparative value (e.g. if Condition = Between)
- **Expression:** You can define an expression that specifies the condition for the formatting. In this case, Expression must be specified for the Condition (Condition = Expression). You use the "Condition expression editor" to define an expression (to open the editor, click the three dots in field "Expression"):



The following objects are available for the definition of the expression:

- Functions: selection of mathematical functions and standard functions to edit date values, numeric values and character strings
- Operators: selection of logical operators
- Fields: available columns
- Constants: constant values

It is useful to create the required expression by double-clicking the different functions, operators, fields and constants, instead of editing them manually because this is less error-prone.

On the right hand side of the editor, the individual functions, operators, fields and constants are described.

An error message is displayed when the dialog is left if the expression defined does not meet the specified syntax.

Column width

You can change the column width if you simply "drag" the column. Move the mouse pointer over the edge of a column. The mouse pointer changes and the edge can be dragged to change the column width.

You can use the menu to set the optimum column width of a column (*Optimum column width*) or of all columns (*Optimum column width - all columns*). To identify the optimum width of a column, the system uses the contents of the relevant column.

Selection of Several Lines

If the menu item *Allow for several lines to be selected* is marked, the user can select several lines in the grid at the same time.

Configuration file

If a new application of the grid type is created, a separate configuration file is generated - as with most other detail application types, too. This file is stored in the application directory and is called Grid<Name_of_application>.config. All customizing possibilities described above have an effect on the contents of this file. Optionally, manual editing is also possible. The file includes the following sections:

- Layout: Layout of grid (columns, categories, grid-internal properties): This xml structure is automatically read by the DevExpress grid and must therefore have a specific structure.
- GroupPanelShown: specifies if the field ("group by") is displayed
- SummaryShown: specifies if the totals line is displayed
- GroupSummaryShown: specifies if totals lines for groups are displayed
- MultipleRowSelection: specifies if several lines can be selected at a time
- Styles: List of conditional formattings

11.2 Master Detail Grid

The Master Detail Grid is a hierarchical grid, i.e. the data is displayed on several hierarchical levels. The grid can include any number of levels. At each level (except the top level), there might be several grids.

Drag a column here to group by that column

Inspection plan										
Area ID	Area	Insp. plan index	Article	Drawing issue nu...	Article designation	Drawing number	Article customer num...	Group 1	Group 2	Group 3
A	Ausgang	01	23 9383 712	B	Frontscheinwerfer XA 77	3230 93039		Leuchten	Frontsche...	

Insp. plan characteristics									
OP	Characteristic	Characteristic designation	Characteristic type	Inspection result base	Mandatory inspection	Process parameter	Documents available	Tolerance, normal	
50	1004	Sichtprüfung	attributiv		☒		☒		

Characteristic documents									
Document									
Position	Designation	Type	File name/...	Display wit...	Created by	Created on	Edited by	Edited on	
40	1002	Kratzer		attributiv			☒		
30	0007	Breite		variabel			☒		
20	0012	Länge		variabel			☒		
10	0010	Winkel		variabel			☒		
5	1007	Identitätsprüfung		attributiv			☒		

A	Ausgang	01	32 9382 111	G	Unterlegscheibe D5	3312 4433 3131		Stanzteile	Unterlegs...	
A	Ausgang	01	77 9938 882	C	Metallkugelschreiber 4 farbig 10000m Schrift	23213 90939	MKS 234-Z	Kugelschreiber	Kunststoff	
A	Ausgang	02	77 9938 882	C	Metallkugelschreiber 4 farbig 10000m Schrift	23213 90939	MKS 234-Z	Kugelschreiber	Kunststoff	
A	Ausgang	03	77 9938 882	C	Metallkugelschreiber 4 farbig 10000m Schrift	23213 90939	MKS 234-Z	Kugelschreiber	Kunststoff	
E	Eingang	01	04-01-01		Spiegel Ford Mondeo			Spiegel	Ford	Spiegel Ford Mondeo
EMU	Erstmus...	01	77 9938 882	C	Metallkugelschreiber 4 farbig 10000m Schrift	23213 90939	MKS 234-Z	Kugelschreiber	Kunststoff	

The following example explains the structure of the Master Detail Grid and the steps that are required to configure a detail application for this grid.

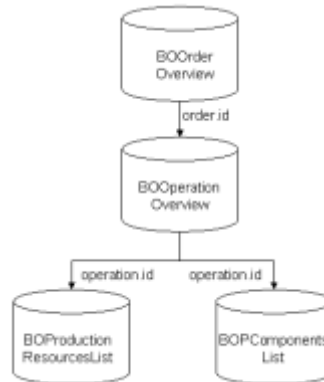
Basic data structure

The data source for a "normal" grid is a DataTable. Since a Master Detail Grid includes several levels, a DataSet, i.e. a collection of several interlinked DataTables is used here. The levels of a grid have a specific relationship to each other. Therefore, also the relations between the different DataTables within a DataSet must be specified.

The data for the grid are requested "on demand". This means that only the data for the top level are requested first. Clicking on the cross next to a data record will display another level. The data shown on this level is only requested when the level is expanded, indeed only the data displayed.

Example

The following sections follow this example: the first level shows orders, the second level shows operations. On a third level, production resources, tools and components are shown.



The image shows the structure of the DataSet with 4 DataTables from the example above.

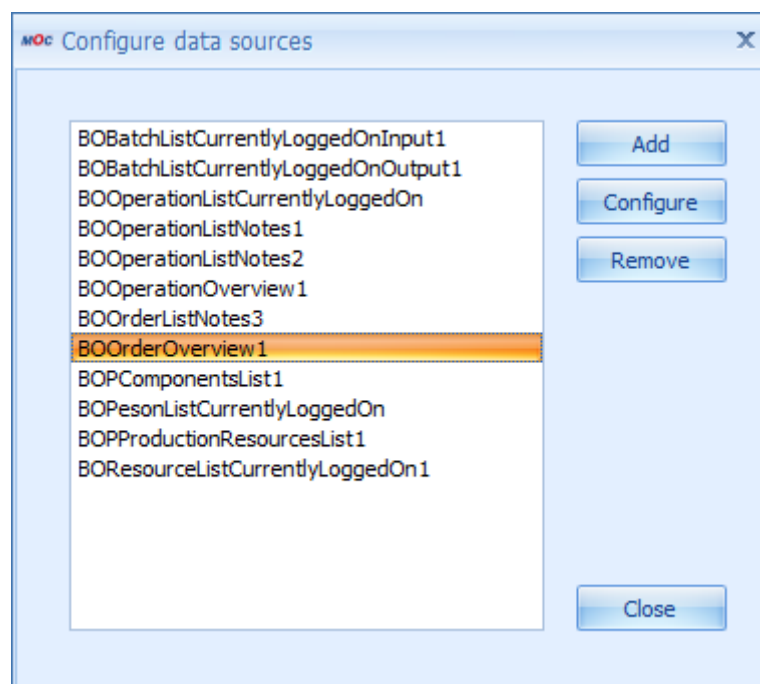
Clicking on the green arrow will consequently request all orders first. This data is inserted into the first DataTable (BOOrderOverview) within the Result DataSet. The order xyz is now expanded. All operations for this order are identified and added to the DataTable BOOrderOverview. If you also expand the order abc, the relevant operations are identified for this order, too, and the DataTable BOOperationOverview is added.

Inserting a detail application of type Master Detail Grid

If you want to add a new Master Detail Grid to an application, this application must be configured as follows:

Data sources

You must add data sources for each level of the grid.



The configured data sources of the example.

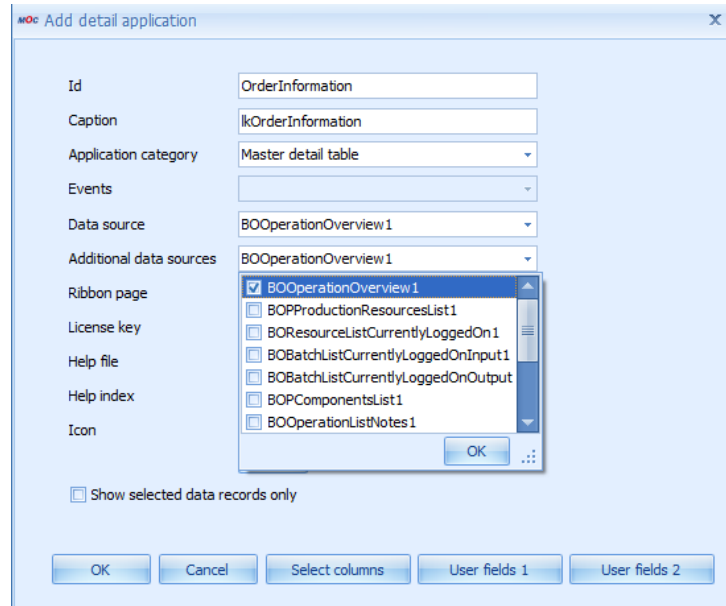
Note the following when you configure the individual data sources:

- The data source for the top level will usually react to a click on the "green arrow". The selection panel provides the required parameters. You must configure this accordingly.
- The inferior data sources must be configured as follows:
 - Events: Select the MasterRowExpanded event of the superior data source
 - Source: Select the superior data source
 - Parameter: Map all parameters that define the relationship between the two data sources (as in each master detail relationship)

Detail application

A new detail application must be added to the application. Configure as follows:

- Application category: Master Detail Table
- Data source: Select data source for top level
- Additional data sources: all other data sources whose data is to be shown in the grid



Configuration of the "Additional data sources" for the example.

Initializing the configuration file

If you create a new detail application (see step above) and you save the application, the system automatically creates a configuration file that includes the layout definition of this detail application. The name of this file is MasterDetailGrid<Name of detail application>.config.

The contents of a configuration file for a MasterDetailGrid are very similar to those of a "normal" grid. The only difference is that the settings are available several times – once for each detail grid. To uniquely identify the settings, the name of the respective data source is added.

See also: 11.1.

Important notes:

a) There are other, MDG-specific (MasterDetailGrid) properties that are missing in the configuration file of a "normal" grid. So far, side effects for the missing entry [AllowExpandEmptyDetails](#) are known. If this property is set to *false* (which unfortunately is the default setting), the MDG nodes are always only expanded upon the second click and only if data is available. The entry is as follows:

```
<property name="OptionsDetail" isnull="true" iskey="true">
  <property name="SmartDetailExpandButtonMode">AlwaysEnabled</property>
  <property name="AllowExpandEmptyDetails">true</property>
</property>
```

b) In order to hide the title line on lower levels (e.g. for reasons of space), the following must be added in the layout section of the superior level:

```
<property name="RowSeparatorHeight">0</property>
<property name="OptionsDetail" isnull="true" iskey="true">
  <property name="SmartDetailExpandButtonMode">AlwaysEnabled</property>
  <property name="AllowExpandEmptyDetails">true</property>
  <property name="ShowDetailTabs">>false</property>
</property>
<property name="ColumnPanelRowHeight">-1</property>
```

The respective entry is ShowDetailTabs = false in OptionsDetail.

c) Tab label texts: by default, the label texts of tabs are made up of the translation of "lk" + name of the respective data source. Or: If you want a different label text, you can manually add an additional setting in the configuration file.

```
<Setting      Key="TabTitle_xxx"      Description=""      LastChanged="2010-04-
13T17:42:58.0115088Z" ValueType="System.String" Version="0.0.0.0">
  <Value>
    <string>lkTitle</string>
  </Value>
</Setting>
```

d) User fields: the user fields for each individual level must currently be added manually into the configuration file. Configure as follows:

```
<Setting Key="UserFieldObjectType_xxx" Description="" LastChanged="2011-07-
25T18:49:05.2387406Z" ValueType="System.String" Version="0.0.0.0">
  <Value>
    <string>CPAN</string>
  </Value>
</Setting>
<Setting      Key="UserFieldKey_xxx"      Description=""      LastChanged="2011-07-
25T18:49:05.2387406Z" ValueType="System.String" Version="0.0.0.0">
  <Value>
    <string>FEP</string>
  </Value>
</Setting>
<Setting      Key="UserFieldPraefix_xxx"      Description=""      LastChanged="2011-07-
25T18:49:05.2387406Z" ValueType="System.String" Version="0.0.0.0">
  <Value>
    <string>inspectionrequirement</string>
  </Value>
</Setting>
```

Relations

In addition to the grid configuration for each level of the Master Detail Grid, you must define the relations between the levels. To this end, a further section is added at the end of the master detail configuration file. Here, there is NO editor and you must manually configure the file. For the manual configuration, proceed as follows (in the example):

```
<Setting Key="Relations" Description="" LastChanged="2009-07-
07T11:52:21.229117Z" ValueType="System.Xml.XmlDocument" Version="0.0.0.0">
  <Value>
    <Relations>
```

```

    <Relation Name="Operations" MasterController="BOOrderOverview1"
DetailController="BOOperationOverview1">
      <ColumnRelations>
        <ColumnRelation>
          <ForeignKeyColumn DataSource="BOOrderOverview1"
Name="order.id"></ForeignKeyColumn>
          <KeyColumn DataSource="BOOperationOverview1"
Name="order.id"></KeyColumn>
        </ColumnRelation>
      </ColumnRelations>
    </Relation>
    <Relation Name="FHM" MasterController="BOOperationOverview1"
DetailController="BOProductionResourcesList1">
      <ColumnRelations>
        <ColumnRelation>
          <ForeignKeyColumn DataSource="BOOperationOverview1"
Name="operation.id"></ForeignKeyColumn>
          <KeyColumn DataSource="BOProductionResourcesList1"
Name="operation.id"></KeyColumn>
        </ColumnRelation>
      </ColumnRelations>
    </Relation>
    <Relation Name="Components" MasterController="BOOperationOverview1"
DetailController="BOPComponentsList1">
      <ColumnRelations>
        <ColumnRelation>
          <ForeignKeyColumn DataSource="BOOperationOverview1"
Name="operation.id"></ForeignKeyColumn>
          <KeyColumn DataSource="BOPComponentsList1"
Name="operation.id"></KeyColumn>
        </ColumnRelation>
      </ColumnRelations>
    </Relation>
  </Relations>
</Value>
</Setting>

```

Note:

- For each relation between two levels, you must insert one relation.
- You are free to select any name for the relation.
- MasterController is the name selected by the superior data source itself (the name must absolutely be identical).
- DetailController is the name selected by the inferior data source itself (the name must absolutely be identical).

- You must create a ColumnRelation for each relation between columns. ForeignKeyColumn is always the column in the superior data source, KeyColumn is the column in the inferior data source.
- As title of the inferior table in the MDG, "lk" + name of relation is always shown automatically. For this purpose, a language key usually has to be created in the mpdvDictionary. To hide this title (e.g. for reasons of space), change the following in the configuration file in the layout section of the relevant level: Set ShowDetailTabs in OptionsDetail to false (see example):

```
<property name="RowSeparatorHeight">0</property>
<property name="OptionsDetail" isnull="true" iskey="true">
  <property name="SmartDetailExpandButtonMode">AlwaysEnabled</property>
  <property name="AllowExpandEmptyDetails">true</property>
  <property name="ShowDetailTabs">false</property>
</property>
<property name="ColumnPanelRowHeight">-1</property>
```

Important: Make sure that ALL connections between columns are specified for each relation, which are required for an unambiguous assignment. Otherwise it can happen that too little data records are displayed in the grid (ambiguous data records cause an exception and this exception has the effect that the data record is not used for the display).

Before saving the (manually edited) file, you must shut down the MOC completely. Otherwise the relations, if any, are removed again!

Further configuration

The further configuration of the Master Detail Grid is similar to the grid configuration. But here, for each level the configuration is performed separately. All configuration options of the grid are available at each level of the Master Detail Grid. See also: 11.1.

Tips for troubleshooting

If data records are missing in one of the MDG levels, this can have two reasons:

- For the respective level, not all key columns have been defined that are required to uniquely identify a data record (in repository isKey = true).
- For the "Relations", not all connections between columns have been specified that are required to uniquely define the connection.

11.3 Master Detail Grid (without reloading)

The display and structure of the Master Detail Grid (without reloading) is similar to the "normal" Master Detail Grid.

It is a hierarchical grid, i.e. the data is displayed in several hierarchical levels. The grid can include any number of levels. At each level (except the top level), there might be several grids.

Drag a column here to group by that column

Inspection plan										
Area ID	Area	Insp. plan index	Article	Drawing issue nu...	Article designation	Drawing number	Article customer num...	Group 1	Group 2	Group 3
A	Ausgang	01	23 9383 712	B	Frontscheinwerfer XA 77	3230 93039		Leuchten	Frontsche...	

Insp. plan characteristics									
OP	Characteristic	Characteristic designation	Characteristic type	Inspection result base	Mandatory inspection	Process parameter	Documents available	Tolerance, normal	
50	1004	Sichtprüfung	attributiv						

Characteristic documents									
Document	Position	Designation	Type	File name/...	Display wit...	Created by	Created on	Edited by	Edited on
	40	1002	Kratzer		attributiv				
	30	0007	Breite		variabel				
	20	0012	Länge		variabel				
	10	0010	Winkel		variabel				
	5	1007	Identitätsprüfung		attributiv				

A	Ausgang	01	32 9382 111	G	Unterlegscheibe D5	3312 4433 3131		Stanzstelle	Unterlegs...	
A	Ausgang	01	77 9938 882	C	Metallkugelschreiber 4 farbig 10000m Schrift	23213 90939	MKS 234-Z	Kugelschreiber	Kunststoff	
A	Ausgang	02	77 9938 882	C	Metallkugelschreiber 4 farbig 10000m Schrift	23213 90939	MKS 234-Z	Kugelschreiber	Kunststoff	
A	Ausgang	03	77 9938 882	C	Metallkugelschreiber 4 farbig 10000m Schrift	23213 90939	MKS 234-Z	Kugelschreiber	Kunststoff	
E	Eingang	01	04-01-01		Spiegel Ford Mondeo			Spiegel	Ford	Spiegel Ford Mondeo
EMU	Erstmus...	01	77 9938 882	C	Metallkugelschreiber 4 farbig 10000m Schrift	23213 90939	MKS 234-Z	Kugelschreiber	Kunststoff	

The section below explains the structure of this special form of Master Detail Grid (MDG) and the steps required to configure a detail application that is based on this MDG.

Basic data structure

The data source for a "normal" grid is a DataTable. Since a Master Detail Grid includes several levels, a DataSet, i.e. a collection of several DataTables is used here. The levels of a grid have a specific relationship to each other. Therefore, also the relations between the different DataTables within a DataSet must be specified.

The data is - and this is the difference to a "normal" MDG - NOT requested "on demand". The data is requested simultaneously for all grid levels e.g. when clicking the green arrow.

Reason for this behavior: For this detail application of the Master Detail Grid type (without reloading), a data source has been defined or selected, that provides a DataSet as result. Important: This must be discussed with the system designer in advance.

Special case: in exceptional cases it is possible, e.g. by script, to compose such a DataSet manually from the results of several data sources. An example for this is the PaymentDayTypes application with its related scripts (see also 0).

In the repository, the structure of the result DataSet of a data source is defined on the ServiceParameters tab. The ResultSet column is used to note the level where each column is located. In the result DataSet, there will later be a DataTable for all entries under ResultSet in the repository that can be used for one level of the MDG.

Inserting a detail application of type Master Detail Grid (without reloading)

If you want to add a new Master Detail Grid to an application, this application must be configured as follows:

Data sources

You must define 1 data source (providing a DataSet as result).



Special case: If such a data source is not available but if data from several different data sources (whose data source is DataTable) are presented in the grid, these data sources must be combined into a DataSet via script and assigned to the application (see also 0).

Detail application

You must add a new detail application of type "Master detail table (without reloading)" to the application. Configure as follows:

- Application type: Master Detail Table (without reloading)
- Data source: Select suitable data source (in case of more than one data source, specify the data source for the top level)

Creating/Initializing the configuration file

You can use a configuration file to define the layout of this detail application. For technical reasons, this is NOT created automatically for the MDG (without reloading). How such a file may be created and edited is explained below.

First create a text file with the name MasterDetailGridWithoutReload[name of detail application].config. Save this file in the application directory of the main application.

The configuration of a normal grid with only one level is made up of several sections (see 0). The configuration of an MDG is made up of the configuration of such a normal grid for each level. Consequently, each level must be configured separately³.

You need not edit these configurations manually. Proceed as follows:

- Create an empty test application.
- Add the data source to the test application that is also used for the MDG. Important: Select all columns in the column configurator that are required in the relevant MDG level.

³ If several grids are located in one level, one configuration each must be available for the different grids. In this case, several configurations for one level might exist.

- Add a normal grid to the test application as detail application. Configure the grid, if required.
- The last two steps are repeated for all levels of the MDG. Important: The selected columns of the data source must always have the same entry under ResultSet in the repository.
- Now save the application. For each grid, a configuration file with the name Grid<Name of detail application>.config is created.
- Copy the contents (= all sections) of all configuration files into the previously created MDG configuration file (one below the other).
- Rename all sections as follows:
<Name of section>_<Associated ResultSet from repository>

Important notes:

a) The layout section in the configuration file of a "normal" grid usually DOES NOT contain any entry for the property Name. This, however, results in errors when requesting data within an MDG. The name must absolutely correspond to the name of the associated ResultSet (see Repository). For this reason, manual modifications are required if such a layout section is copied to the configuration of an MDG.

b) There are other, MDG-specific properties also missing in the configuration file of a "normal" grid. So far, side effects for the missing entry [AllowExpandEmptyDetails](#) are known. If this property is set to *false* (which unfortunately is the default setting), the MDG nodes are always only expanded upon the second click and only if data is available. The entry is as follows:

```
<property name="OptionsDetail" isnull="true" iskey="true">
  <property name="SmartDetailExpandButtonMode">AlwaysEnabled</property>
  <property name="AllowExpandEmptyDetails">true</property>
</property>
```

c) In order to hide the title line in lower levels (e.g. for reasons of space), the following must be added in the layout section of the superior level:

```
<property name="RowSeparatorHeight">0</property>
<property name="OptionsDetail" isnull="true" iskey="true">
  <property name="SmartDetailExpandButtonMode">AlwaysEnabled</property>
  <property name="AllowExpandEmptyDetails">true</property>
  <property name="ShowDetailTabs">>false</property>
</property>
<property name="ColumnPanelRowHeight">-1</property>
```

The respective entry is ShowDetailTabs = false in OptionsDetail.

Relations

In addition to the grid configuration for each level of the Master Detail Grid, you must define the relations between the levels. To this end, a further section is added at the end of the master detail configuration file. Here, there is NO editor and you must manually configure the file. For the manual configuration, proceed as follows (in the example):

```
<Setting      Key="Relations"      Description=""      LastChanged="2009-07-07T11:52:21.229117Z"
ValueType="System.Xml.XmlDocument" Version="0.0.0.0">
  <Value>
    <Relations>
      <Relation Name="order_operation" MasterController="order" DetailController="operation">
        <ColumnRelations>
          <ColumnRelation>
            <ForeignKeyColumn DataSource="order" Name="order.id"></ForeignKeyColumn>
            <KeyColumn DataSource="operation" Name="order.id"></KeyColumn>
          </ColumnRelation>
        </ColumnRelations>
      </Relation>
    </Relations>
  </Value>
</Setting>
```

Note:

- For each relation between two levels, you must insert one relation.
- You are free to select any name for the relation.
- MasterController is the name of the superior ResultSet (the name must absolutely be identical).
- DetailController is the name of the inferior ResultSet (the name must absolutely be identical).
- You must create a ColumnRelation for each relation between columns. ForeignKeyColumn is always the column in the superior data source, KeyColumn is the column in the inferior data source.
- Before saving the file, you must shut down the MOC completely. Otherwise the relations, if any, are removed again.

Tables

You must also define all levels. To this end, a further section is added at the end of the master detail configuration file. Here, there is NO editor and you must manually configure the file. For the manual configuration, proceed as follows (in the example):

```
<Setting Key="Tables" Description="" LastChanged="2009-  
  <Value>  
    <Tables>  
      <Table>order</Table>  
      <Table>operation</Table>  
    </Tables>  
  </Value>  
</Setting>
```



Note: For each level/each grid, you must add an entry. The specified name matches the name of the associated ResultSet.

MasterTable

In addition, the name of the main level must be defined. To this end, you attach a further section at the end of the Master Detail configuration file. Here, there is NO editor and you must manually configure the file. For the manual configuration, proceed as follows (in the example):

```
<Setting Key="MasterTable" Desc  
  <Value>  
    <string>order</string>  
  </Value>  
</Setting>
```



Note: The name specified must match the associated ResultSet.

Further configuration

You should have made the configuration of the individual grid levels directly for the underlying grids, because for technical reasons it is not possible to save them. Subsequent changes must directly be made in the configuration file.

Special case: Composing DataSet from results of several data sources

There are 2 possibilities if no data source with several results (as DataSet) exists, but if the data is nevertheless to be presented in an MDG:

- Either you use a regular MDG with reload of data on demand (description in 11.2);
- Or you use the MDG without reload of data and compose the results (as DataTable) manually via script.

Note the following for the second variant:

- Currently, the only possibility to create the new ResultSet is via script. The respective script entry point is <Name of detail application> + AltSetDataSource.
- You must specify the inferior data sources as additional data sources for the detail application (as for the normal MDG).
- The name of the sections in the configuration file is the name of the data source (as for the normal MDG).

An example for this procedure is included in the configuration file of the MDG and/or in the script for the PaymentDayTypes application.

11.4 Charts

The ChartControl (Chart) is a control tool to visualize data. The chart may handle different types of data sources. In addition, a wizard is provided allowing for all major settings being made by the user and/or application designer.

Available chart types

Chart: Standard chart, completely configured through ChartWizard.

Chart with series selection: See chart, but the user can show and/or hide individual series.

Individual series chart: Special chart to display data of several columns of one data source in a pie chart. All columns configured in the column configurator are consecutively written into a series. The column header is used as an argument (label). The configuration is made using the ChartWizard.

RPA distribution: See individual series chart. RPA colors are provided as palette. Using a radio button, the user can decide if the tooltip displays times or quantities.

Cycle progression: Special chart to display a cycle progression. Data source and series are pre-configured and cannot be edited.

Downtime hit list: Special chart to display a downtime hit list. Data source and series are pre-configured and cannot be edited.

Article profile: Special chart to display an article profile.

PDV measurement analysis: Special chart to display the PDV measurement analysis.

ChartWizard

You can use the ChartWizard to edit most settings of the chart application graphically. Note: Only if the application has already requested data, a selection of data series in the wizard is possible.

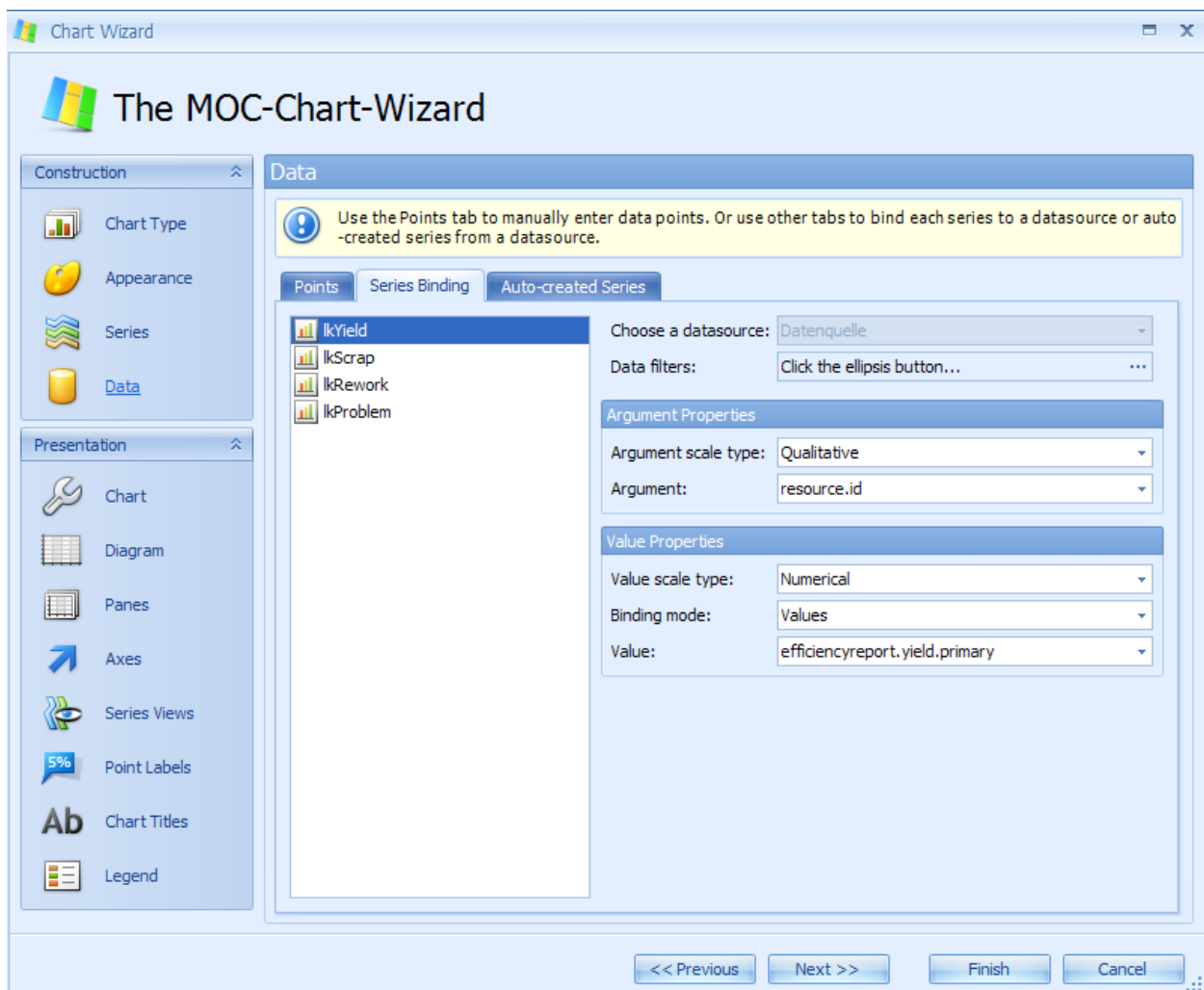
To activate the wizard, double-click a chart. The following setting options are available:

Chart type: Selection of chart type, e.g. bar chart, pie chart

Appearance: A color scheme may be selected and a color palette may be specified.

Series: Creating series. The data types of the axes must be specified. The series names must be language keys.

Data: Binding series to the fields of the linked data source.



The figure shows a sample configuration in the ChartWizard.

11.5 Layout

A detail application with the category "Layout" is used to display the contents of individual data records in fields (controls), which may be distributed among several tabs of the application. The configuration of the application is similar to the one of the selection panel.



Detailed information on maintenance is included in the sections "Configuration of applications - Selection panel" and "Input and output fields".

Special notes

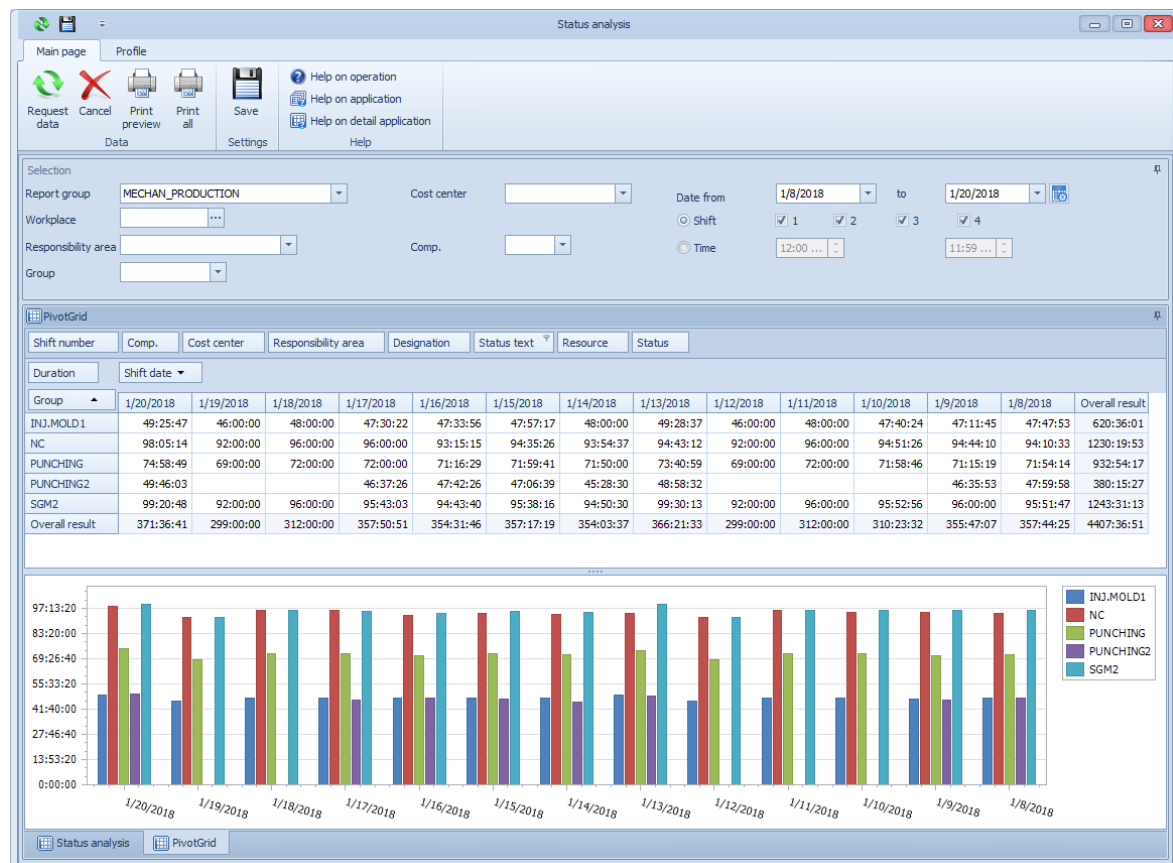
- Select "Show selected data records only" in the detail application configuration.
- The values transferred are based on the data source. The key here is the FieldName (provided by WebService, data source and must comply with the FieldName of the InputControl).
- Set the property "ShownInDetail" = "true" for fields that you want to display in application details.
- Use an entry in "Category 1" to show fields in a specific tab.



Many changes in InputControls will only be visible upon saving, closing and re-opening the application!

11.6 Pivot

Using the pivot module, you can show summaries of data from a simple table in a pivot table using the relevant columns. You can use different aggregate functions (sum total, mean value, etc.) to calculate the contents of the created table. You can place the available columns in the four areas at runtime using drag and drop in order to highlight different focuses or to identify trends.



Areas of the pivot table

The header area (where the filter fields are placed) is divided into:

- **Row Header Area**, field "Group" in the example
- **Column Header Area**, field "Shift date" in the example
- **Data Header Area**, field "Duration" in the example. Calculations are made after this field.
- **Filter area**, fields "Shift number", "Company", etc. in the example. The fields in this area may be used in order to make further restrictions, e.g. calculation of durations for a particular status only (to be selected via "Status text" field).

In the actual **Data Area**, the aggregate data is presented, including the total results according to rows and columns, as well as intermediate results if several fields are placed in the row or column area. In the example, the duration is summed up according to group and shift date.

Field list

Using the column selection of the data source of the detail application, the application designer specifies the fields that are available in the pivot detail application (according to the columns of a flat table). The field list provides these fields for the user. From there, you can transfer and/or remove the fields into or from the four header areas. The remaining fields are not involved in the calculations, but are available to the user for further compositions.

Context menus

The following different context menus are provided for the pivot table:

Context menu in empty header field (main menu)

- Show/hide all fields: all fields in the field list are added to the pivot table as fields and/or all fields of the pivot table are removed from all areas of the table.
- Hide filter area fields: all fields are removed from the pivot table filter area.
- Hide/show field list: hide/show selection list with available fields.
- Show FilterEditor: the FilterEditor may be used to implement additional filters for the pivot table.
- Show/hide settings: this menu item may be used to open an additional editing dialog (please also refer to "Settings" below).

Settings

- Top N: specifies how many data is shown in the table. Note: this setting only works if "Top N" is activated for data fields (see below) in the context menu.
- Show others: if the option "Top N" is activated, you can use this option to specify if the other data is shown in the table as summarized form.
- Totals location: specifies where the totals columns are shown in the table. Far: right; Near: left
- Chart: Is the chart area of the pivot shown?
- Selection: Is only selected data shown in the chart?
- Columns: Are the values in the chart shown in different columns or in "cumulative" form?
- Totals: Are totals additionally shown in the chart?
- Legend: Is the legend shown in the chart?

Context menu in row and column fields

- Show/hide subtotals: is used to specify if only totals or also subtotals for individual groups are shown in the table.
- Hide: This is used to remove the selected field from the relevant area.
- Sequence: This is used to specify the sequence of fields in the relevant area.
- Optimum column width / Optimum column width (all columns): see also 0. Important: in the pivot table, all columns always have the same width.

Context menu in data fields.

- Aggregate functions: is used to specify how data is aggregated in the pivot table. The following functions are available: Total, Number, Minimum, Maximum, Variance, Variance (Percent), Standard deviation, Standard deviation (Percent). Important: It is possible that specific aggregate functions are not useful depending on the data formatting.
- Additional aggregate function: The Ratio aggregate function is based on the single values of two adjacent columns (dividend and divisor).
- Example from Time and Attendance PZE: Actual time=99, target time=100. Ratio=0,99, displayed (via OutputFormat) as %actualtime/targettime=99%.
- What is special about this function is that the column values are not based on their own single values and cannot be computed from other, aggregate values.
- What is also special about the calculation: in case of a division by 0, the result is set to 0.
- Show aggregate function name: if this option is selected, the name of the selected aggregate function is displayed behind the data field.
- Display type: specifies how the data is displayed. Options:
 - Nominal: the value is displayed (default)
 - Difference (absolute): the difference to the previous value is displayed.
 - Difference (%): the percentage difference to the previous value is displayed.
- Hide: Removes field from data list
- Sequence: see above
- Optimum column width / Optimum column width (all columns): see also 0.
- Sort ascending/descending: specifies sorting of data
- Top N: specifies whether only the top n data records are shown. In addition, the settings dialog may be used to specify n.

Associated chart

The associated chart graphically presents the current data of the pivot table. An arbitrary row/column selection may also be displayed. See also "Pivot table settings". In the Customizing mode, you can configure the chart in more detail using the ChartWizard (double-click to open).

Pivot table settings

The panel for the table settings and the associated chart may be shown and hidden via context menu in the empty header field.

The panel with settings will be migrated to the application menu in a future version.

11.7 FileAttachmentPlugin

Overview

You can use the FileAttachmentPlugin to assign a file as attachment to an existing data record in a grid. The FileAttachmentPlugin can be used for the links in the toolbar. The file is uploaded to the server into a configurable path. You can also use the plug-in to show the file, to delete the file and to remove it from the data record.

Requirements

- 1) You must configure a path in the path configuration where the files are stored. You can also use an already configured path.
- 2) The data records in the grid must have a column with a unique key. This key then becomes part of the file name on the server. Example: internal ID (serial).
- 3) The data records in the grid must include a column of data type string. The name of the file that is uploaded to the server is then written into this column. If no attachment is assigned, this column must be empty. When you assign an attachment, the plug-in automatically populates this column using a generated file name. If you delete an attachment, the plug-in removes the file name from the data record.

Assigning an attachment

Configuration in the link editor

Command

Fixed "UploadAttatchment"

Function

Fixed "callCommandObject"

Parameter

Example:

```
UploadAttatchment hydraPath="MOCHRIMG"
attatchmentFileName="pequal_[personnelqualificationsassignment.internal_id]"
listDataLogic="PersonnelQualificationsAssignmentList"
updateDataLogicId="PersonnelQualificationsAssignmentUpdate"
updateDataLogicParameter="personnelqualificationsassignment.person.id,personnelqualification
sassignment.qualification.id,personnelqualificationsassignment.internal_id,personnelqualifications
assignment.valid_from,personnelqualificationsassignment.valid_to"
updateDataLogicField="personnelqualificationsassignment.filename"
```

Parameter	Explanation
-----------	-------------

UploadAttachment	Fixed CommandLinkID
hydraPath	HYDRA path used to upload the files.
attatchmentFileName	Column that includes the name pattern for the file name on the server, e.g. "pequal_[personnelqualificationsassignment.internal_id]"
listDataLogic	Data source of list service
updateDataLogicId	Data source of update service
updateDataLogicParameter	List of columns that must be passed to the update service as parameters.
updateDataLogicField	Column of update service that includes the file name.

Other fields

For the other fields, e.g. Language Keys and symbols, nothing special applies.

Process description

1. If you click the icon to upload an attachment in the toolbar, a dialog opens where you can select a file.
2. The plug-in generates a unique file name for the file on the server according to the configured pattern (parameter attatchmentFileName). The file extension of the original file is not changed.
3. The plug-in then copies the file with the generated name into the configured path.
4. As a next step, the selected data record is automatically locked by the lock service (key like update service).
5. The plug-in calls the update service to save the generated file name in the configured column of the data object. The plug-in passes the configured parameters from the list service as key to the update service and also the configured column with the file name.
6. The plug-in then automatically performs the unlock service (key as update service).

Show attachment

Configuration in the link editor

Command

Fixed "ShowAttachment"

Function

Fixed "callCommandObject"

Parameter

Example:

```
ShowAttachment hydraPath="MOCHRIMG"
listDataLogic="PersonnelQualificationsAssignmentList"
updateDataLogicField="personnelqualificationsassignment.filename"
```

Parameter	Explanation
ShowAttachment	Fixed CommandLinkID
hydraPath	HYDRA path used to upload the files.
listDataLogic	Data source of list service
updateDataLogicField	Column of service that includes the file name.

Other fields

For the other fields, e.g. Language Keys and symbols, nothing special applies.

Process description

1. If you click the icon to show an attachment in the toolbar, the file is automatically downloaded to the client. (The MOC stores the files in the directory of application data of the user; the files are automatically deleted when the user logs off or when the MOC is shut down).
2. The plug-in opens the file using the operating system. To show the file, the application is called that is assigned to the respective file extension. If no application is assigned to the file extension, Windows provides a selection of possible applications.

Delete attachment

Configuration in the link editor

Command

Fixed "DeleteAttachment"

Function

Fixed "callCommandObject"

Parameter

Example:

```
DeleteAttachment hydraPath="MOCHRIMG"
listDataLogic="PersonnelQualificationsAssignmentList"
updateDataLogicId="PersonnelQualificationsAssignmentUpdate"
updateDataLogicParameter="personnelqualificationsassignment.person.id,personnelqualification
sassignment.qualification.id,personnelqualificationsassignment.internal_id,personnelqualifications
assignment.valid_from,personnelqualificationsassignment.valid_to"
updateDataLogicField="personnelqualificationsassignment.filename"
```

Parameter	Explanation
DeleteAttachment	Fixed CommandLinkID

hydraPath	HYDRA path used to upload the files.
listDataLogic	Data source of list service
updateDataLogicId	Data source of update service
updateDataLogicParameter	List of columns that must be passed to the update service as parameters.
updateDataLogicField	Column of update service that includes the file name.

Other fields

For the other fields, e.g. Language Keys and symbols, nothing special applies.

Process description

1. If you click the icon to delete an attachment in the toolbar, a confirmation prompt opens.
2. If you click OK to confirm, the plug-in deletes the file with the name from the configured column of the list service in the configured path.
3. As a next step, the selected data record is automatically locked by the lock service (key like update service).
4. The plug-in calls the update service to set to empty the file name in the configured column of the data object. The plug-in passes the configured parameters from the list service and the configured column with the empty file name to the update service.
5. The plug-in then automatically performs the unlock service (key as update service).

11.8 Special table views

The following sections describe other table views used in very special application cases.



The special table views are intended for internal use by MPDV itself. The special table views have been developed for very specific use cases and it is normally not useful to use them in other situations. They require extensive knowledge of the configuration files on the MOC. A full use might only be possible if an additional programming (scripting) is performed. This is not possible with the tools that are available for customers.

Year model

The year model is a special grid to display and edit data in a calendar form. This type of detail application was especially developed for the year model application and is used to maintain data. Apart from the grid itself, it contains additional controls and input fields. Of these, year, model and designation/name as well as the grid contents are actually used for data maintenance. The fields below the grid are only used for editing the grid at runtime.

The year model is used for data maintenance and is therefore only placed on maintenance applications. As the data source, MDMFYearModelInsert or MDMFYearModelUpdate may be used optionally. The column configurator is empty. Further configuration is not required.

PDV ID Tracing

The grid for PDV ID tracing is a grid especially used for the ID tracing application. The only difference to the "normal" grid is that the columns are generated dynamically at runtime. The columns stored in the configuration are NOT integrated.

The columns `resource.id` and `pdvtagbasedsinglevalue.capture_ts` are permanently added at runtime. Depending on the data provided by the web service, more columns are added.

For this plug-in, only the data source `PdvTagBasedSingleValueList` can be used. In the column configurator, all columns should be selected.

The configuration is identical to the one of the grid, see 11.1.

PDV process analysis

The grid for a PDV process analysis is a grid especially used for the process analysis applications (machine-related, order-related, batch-related). The only difference to a "normal" grid is that the measured value that is displayed is taken from one of 4 different columns. The data type specifies which of the four columns is used.

For this plug-in, only the data source `PdvSingleValueListWithAdditionalInfo` can be used. In the column configurator, all columns should be selected.

The configuration is identical to the one of the grid, see 11.1.

11.9 Special charts

The following sections describe other chart types used in very special use cases.



The special charts are intended for internal use by MPDV. The special charts have been developed for very specific use cases and it is normally not useful to use them in other situations. They require extensive knowledge of the configuration files on the MOC. A full use might only be possible if an additional programming (scripting) is performed. This is not possible with the tools that are available for customers.

PDV measurement analysis

The chart for the PDV measurement analysis is a special chart that can only be used for special applications: Applications for the measurement analysis of measured values and samples, as well as for machine-related, order-related and batch-related measurement analyses.

This plug-in can include up to 3 charts that are added/removed using  . For each chart, several process parameters can be selected from the associated selection list. Machine/order/batch and time range specify the selection available in the list. Initially, the list is empty.

When configuring an application using this plug-in, note the following:

- For single values, limit values and events, you must define 3 data sources each; this means that 9 data sources must be defined in total.
- Permitted data sources are:
 - Single values: PdvSingleValueList, PdvSampleValueList
 - Limit values: PdvSingleValueSpecification, PdvSampleValueSpecification
 - Events: PdvEventList
- For each data source, all columns in the column configurator should be selected.
- For the detail application, all data sources must be set. The first data source of single values is the main data source, the remaining ones are additional data sources.

11.10 Grouping application



The grouping application is intended for internal use by MPDV. It requires extensive knowledge of the configuration files on the MOC. A full use might only be possible if an additional programming (scripting) is performed. This is not possible with the tools that are available for customers.

The grouping application does not have any data source but - as is suggested by its name - is used to group other detail applications. For this reason, it is not necessary to create a data source before a grouping application is placed on an application. After selecting the application type, an appropriate information window is therefore shown.

To add other applications to the group, double-click the application. This is only possible if the MESDevelopmentSuite is activated. The grouped applications must be docked within the application. A dialog opens. The left hand side shows all existing detail applications of the main application. The right hand side shows the list of all detail applications already included in this grouping. Applications can be moved by clicking the arrow buttons.

If detail applications are moved to the group, you can dock them again within the group. After docking, the application must be restarted. You can also use this procedure in the inverse direction, to move applications back from the group to the main application.

12 Input and Output fields

Input and output fields (controls of type `mpdvEdit`) are used in "LayoutControls" i.e. in the selection panel, in detail applications and in editing dialogs to present data (output) and/or for the entry of data by the user. The controls support a number of features, such as

- Special input controls for the entry of texts, date and/or time values, color selection, true/false values, etc.
- Selection from selection lists (data-based)
- Forwarding to selection dialogs
- Labeling and unit display
- Input of FROM and TO values.

12.1 Customization

You can control all features of an input and output field via customization.

Adopting properties from the MDS Repository

The property *Field name* of an `mpdvEdit` can reference a property or a service parameter of the MDS repository and this way inherit all properties. In the simplest case, you must only enter a property/service parameter to get a completely defined input and/or output field.

Advantage of this procedure: The relevant properties can be controlled centrally, i.e. all input fields for e.g. machine or personnel numbers have the same behavior and can be changed via central customization settings, if required.

But you can still select any "Field name" for an `mpdvEdit` and/or you can locally overwrite properties. Note: If you make a local change, you can no longer control the field centrally.

Manual customization

To manually customize an input and output field, you use the editing mode of a LayoutControl. Enable the MES Development Suite and activate the editing mode using the context menu of the selection panel, for example.. For further information, refer to the paragraph "Selection panel" in section "Customization of applications".

Note the following:

- You can assign controls of the "mpdvEdit" type to a layout using "Add Control". When the editing mode is activated, click a control to show its properties on the right hand side of the dialog "Customization".

- The selection list of the service parameters of the property "Fieldname" is specified via the contents of the "ServiceName" property - only those service parameters are displayed that are known in the service context. If "ServiceName" is empty, all known properties are displayed.



Numerous changes in controls are only shown, when you save, close and restart the application! For example, if you create a new control and assign a field name to it, you must save and restart the application so that the changes become active.

12.2 Properties of Input and Output Fields

You can control the following properties of input and output fields via a property/service parameter or manually via the editing mode:

- **FieldName:** Reference to the property or - if a ServiceName is given - to the service parameter. If a ServiceName is entered, only the parameters of this service are shown, otherwise all known properties are displayed.
- **ServiceName:** Provides the context from which service parameters can be used. The selection list shows all known services.
- **LanguageKey:** Is used to specify the label text and should be a "LanguageKey" ("lkxxx"), so that the text is displayed in the language that has been selected.
- **UnitLabel:** Text key for unit
- **Length:** (in number of characters) controls the field width. If value is 0, the control will use the entire width available to it. Important: If a width is specified that exceeds the available space, the control is cut.
- **ScriptID:** Is used to identify scripts provided by MPDV.
- **ControlType:** Controls the type of controls used. Possible values for ControlType:
 - **TextEdit:** Text field for entry / display of (short) texts. Can be extended by indicating a "ControlDataSource" with a button that opens a search dialog. If you enter the name of a DataLogic in ControlParameter and if a mapping is included in ControlDataSourceResult, data is requested when you leave the control and return values are mapped appropriately.
 - **CheckEdit:** Checkbox for the entry / display of a Boolean value (true/false) or selection of multiple values, if reference to data source is given.
 - **ColorEdit:** Control for entry / display of a color value.
 - **ComboBoxEdit:** Combo box for the input/display of values provided by the data source in "ControlDataSource".

- DateTimeEdit: Control for entry / display of a date and/or a time.
- MemoEdit: Control for entry / display of a deliberate, multi-line text.
- RadioGroup: Control for entry / display of one or more Boolean values (true/false). The values can be controlled using the ControlDataSource.
- ControlTypeMode: Can be used to control the input control. The permitted values depend on the ControlType.
 - CheckEdit: DualState (default), TriState, J;N;J (checked;unchecked;tristate)
 - ColorEdit: none
 - ComboBoxEdit: SingleEdit, Single, Multiple (multiple selection)
 - DateTimeEdit: Date (date display), time (time display), DateTime, RelativeDate (with button for opening the relative date selection, further information: MpdvEdit_RelativeDate)
 - MemoEdit: none .
 - RadioGroup: SingleColumn, SingleRow
 - TextEdit:
 - None; the search button is shown if a ControlDataSource is defined.
 - "SearchButton": Search button is shown.
 - "SearchButtonValidate": Search button is shown. If you enter an invalid value, an error is displayed.
 - "OpenFileDialog" will open a file selection dialog.
- ShowSecondControl: Specifies whether a second control for from/to inputs is shown.
- ControlDataSource: Data source for the selection of values. The data source can be:
 - a Web service. Configuration: ControlType = ComboBoxEdit, ControlDataSourceType = Lookup, ControlDataSource = Entry from tab ControlDataSources, Field Name
 - a RefDat. Configuration: ControlType = ComboBoxEdit (can be RadioGroup), ControlDataSourceType = Reference, ControlDataSource = Entry from tab ReferenceData, column type
 - a pool application. Configuration: ControlType = TextEdit, ControlDataSourceType = Lookup, ControlDataSource = Name of application
 - a script. Configuration: ControlType = ComboBoxEdit, ControlDataSourceType = Script, ControlDataSource = Name of script
- ControlDataSourceMode: mode of data source (Lookup, Reference or Script)

- **ControlDataSourceParameter:** Request parameter of the data source
- **ControlDataSourceResult:** Result columns are separated by semicolon. The values are interpreted as follows:
 - First entry: Value
 - Second entry: Labeling
 - Third entry: UnitLabel
 - As from the fourth entry, the fields will be mapped:
 - Via acronym or semantic type.
 - Mapping of fields: in **ControlDataSourceResult** a mapping in the form "FieldName=SpalteAusResult" can be entered as of the fourth entry. Example: tool.id=resource.id to fill the field "tools.id" with the value "resource.id" from the pool dialog. Several mappings are separated by ";" - spaces are not allowed.
 - Asterisk mapping: Instead of mapping, you can also enter * . Subsequently, all return columns of the pool screen are mapped. The mapping is performed as usual via ID or semantic type.
- **ClientDefaultValue:** preset default value, see next section 0
- **EditableCondition:** This value decides whether a field is editable. There are three possibilities:
 - **Boolean value:** In case of TRUE or FALSE, the field is always editable / non-editable.
 - **Binary expression:**
 - Field name must be the name of a field that is also located in the **ControlPanel**.
 - Valid operators: =, <, >, <=, >=, <>, !=
 - The value is written as a string and interpreted depending on the comparative field value.
 - Field, operator and value must be separated by a space!
 - **Concatenation of binary expressions**
 - You can concatenate an arbitrary number of binary expressions.
 - To link the expressions, you can use &&, AND, ||, OR.
 - Here, too, all components of the conditions must be separated by a space.
 - Evaluation takes place as AND and/or && before OR and/or ||. Parentheses are not allowed.
 - Example: resource.id = 12345 && resource.costcenter = 20 || resource.id = 60610



The default value of the property "ClientDefaultValue" is assigned to the field, if the result of an expression in the EditableCondition or the VisibleCondition changes from FALSE to TRUE. The expressions in the EditableCondition and the VisibleCondition are dynamically evaluated, if the fields of the application change.

- VisibleCondition: Visibility condition. For customization, see EditableCondition.
- LoadDataOnInit: This is used to specify whether the control data is loaded directly when this control is initialized. Some control types within the mpdvEdit contain a list of data (Combobox, LookupCombo, CheckedCombo, RadioGroup) which is filled from RefDat or Lookup values. For reasons of performance, these controls (except for RadioGroup) are always only filled on demand, i.e. when the control is "opened". In some cases, however, it may be reasonable and/or necessary that this takes place upon loading the control already. For this case, the property LoadDataOnInit must be set to true. This, of course, is only reasonable with regard to the controls mentioned. The default value is false.



Not all modifications in the customization files are directly adopted. You must first save and restart the application.

Use of default values in input fields

Input fields have a "ClientDefaultValue" property. The value entered here is displayed as default value when the control is initialized. "From" and "to" values are separated by semicolons.

Set checkbox

If the value for ClientDefaultValue is set to 'true' in a CheckEdit, the checkbox will be set after initializing.

Pre-allocating text fields with "From" and "To" values

You can use a TextEdit to pre-allocate the From and To fields by setting the ClientDefaultValue to 'Value1;Value2'.

Pre-allocation of date fields

With date fields, you can pre-allocate the field with an offset. If you set default values for date fields, you must absolutely specify the type of offset. The following offsets are possible:

- h (hours)
- d (days)
- w (weeks)
- m (months)

- y (years)

The 'to' value is always **relative to the 'from' value**. The default value is always a DateTime object. The presentation depends on the configuration of the relevant field.

You can put "[" and "]" in front and at the end of the relevant value to specify the start and end of a period of time. Consequently, e.g. "[0d;0d]" means that 00:00:00 is entered in the 'from' field today and 23:59:59 is entered in the 'to' field today. "[-1w;0w]" means from Monday last week up to Sunday last week.

Examples

- Current date: 0d
- From today to the day after tomorrow: 0d; 2d
- From today to one week from today: 0d;1w
- From yesterday to tomorrow: -1d;2d
- From one year ago today to one year from today: -1y;2y

Year selection lists

You can use ControlDataSource = YearList and ControlDataSourceMode = Script to create a year selection list or a normal "Service-ControlDataSource". In this case, you can use the following default values:

- Current year: 0y and/or currentyear
- Last year: -1y
- Following year: 1y
- 4 years ago: -4y
- Year that was current 10 months ago: y-10m --> this is usually the case when the relevant year field occurs in combination with a month selection list.

If you want to preallocate two fields (ShowSecondControl), you must separate both values by semicolon, e.g. -1y;1y

Month selection lists

The following default values can be used for a month selection list:

- Current month: 0m
- Last month: -1m
- Following month: 1m

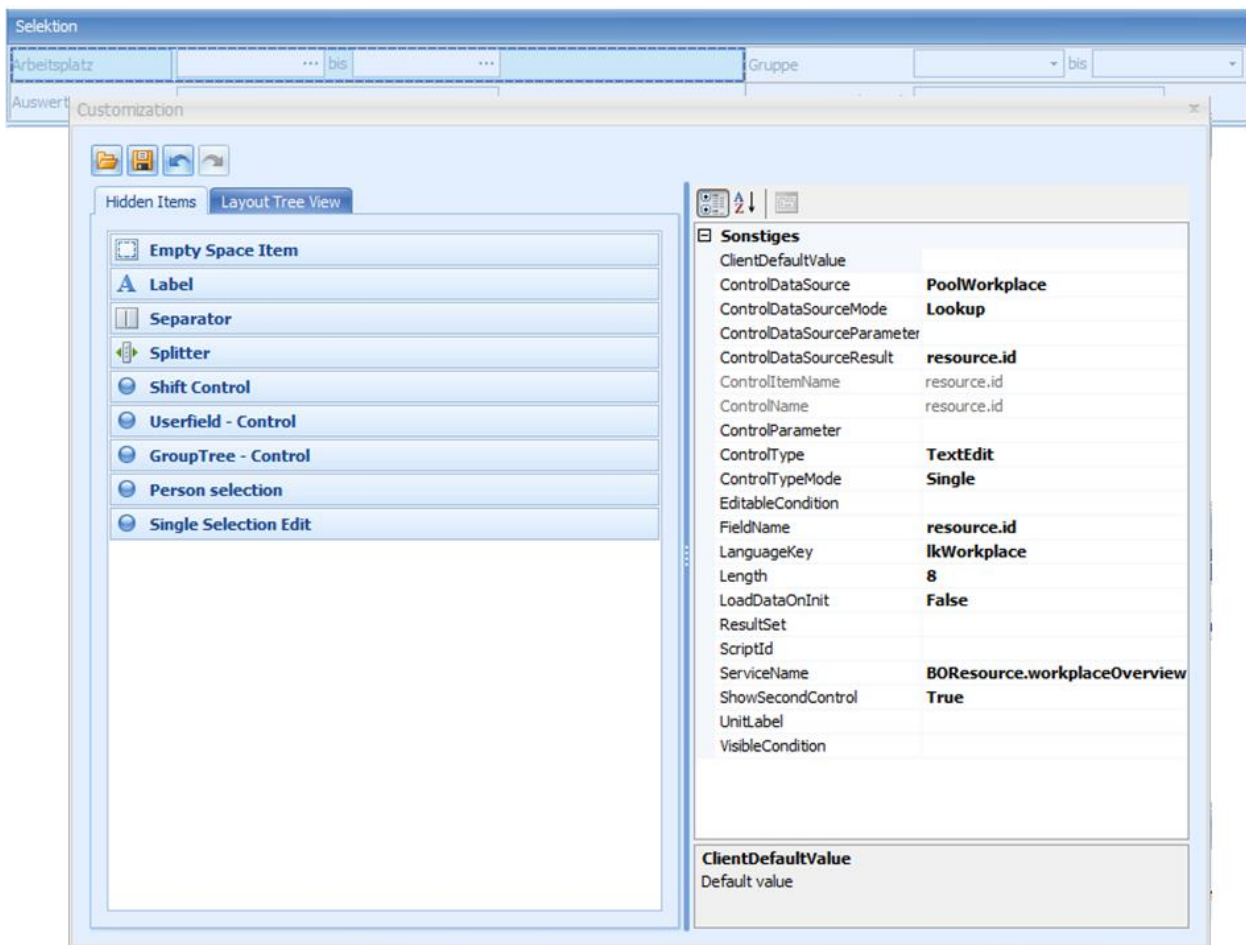
- 4 months ago: -4m

If you want to preallocate two fields (ShowSecondControl), you must separate both values by semicolon, e.g. -1y;1y

12.3 Examples of customization settings

The following examples illustrate possible customization settings of input and output fields.

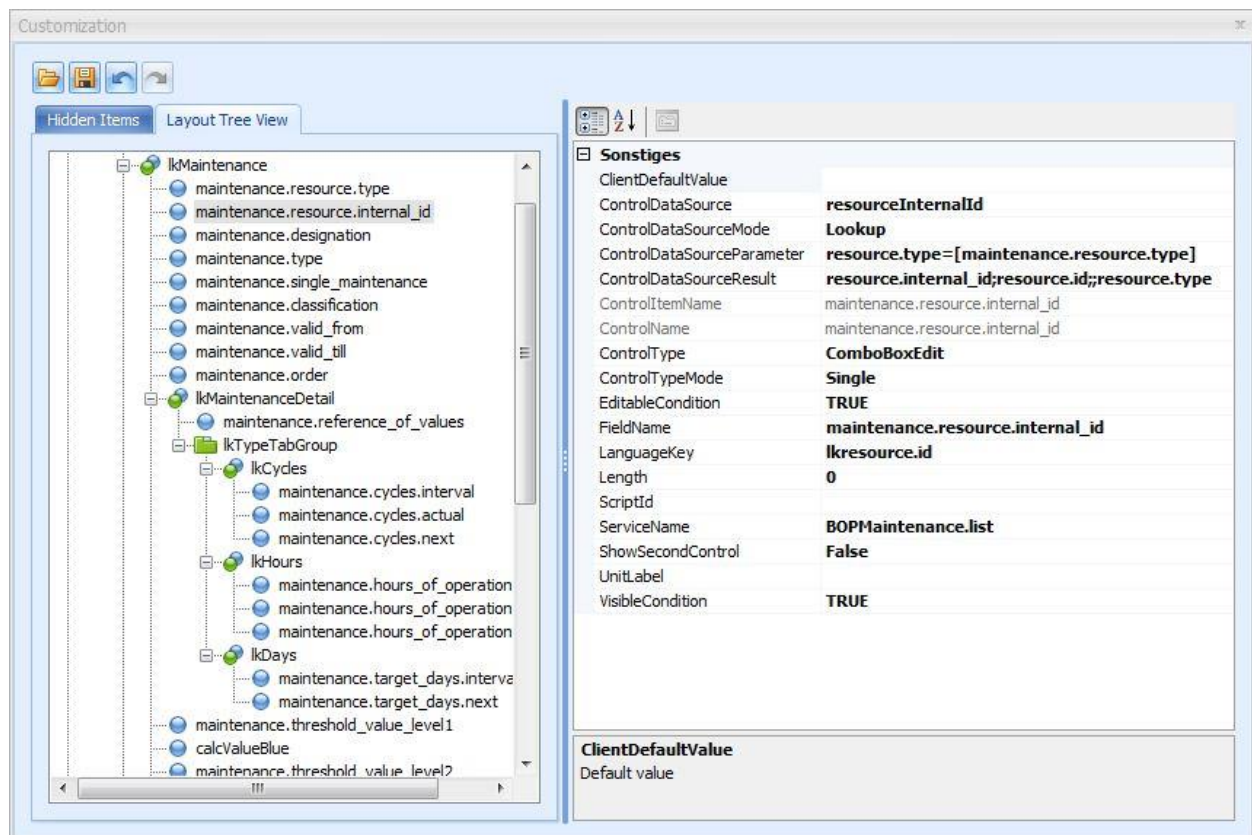
Example: From-To selection parameters with value selection



- ControlType: TextEdit
- ControlDataSource: Application with ID "PoolWorkplace" ("Workplaces" selection)
- ControlDataSourceMode: Lookup
- ControlDataSourceResult: The "resource.id" is selected from the return values of the data record selected in the selection screen and interpreted as result value.
- ShowSecondControl: True => shows the "to" control.

Note: You can only select one value at a time, i.e. a possible "Multiple" ControlType is ignored.

Example: Customization of a selection list



This example shows a complex customization setting for an mpdvEdit. It is an editable ComboBox filled with data from the "resourceInternalId" data source. As filter parameter, the selected value of the "maintenance.resource.type" is transferred to this data source on the same LayoutControl. The data source returns several columns. The column "resource.internal_id" is used as value, "resource.id" is used as display value and "resource.type" is used to fill the "maintenance.resource.type" field.

Example: Data is only requested when ComboBox is opened

Applications with many selection lists (combo boxes) can take longer to open if all combo boxes are filled when the application is requested.

Customization

- ControlDataSource: Name of an existing CDS
- ControlDataSourceMode: Lookup
- ControlType: ComboBoxEdit
- ControlTypeMode: SingleEdit

13 User Fields

Use of User fields

Detail views (e.g. table views or layouts) may be allocated to user fields by defining the object type and the user field key.

Steps

- Configure detail application
- User fields 1 / 2 command button
- Type and user field key selection
- Checkbox "Generate parameter in selection panel".
- Press "OK" => User fields are shown in table view and detail view.

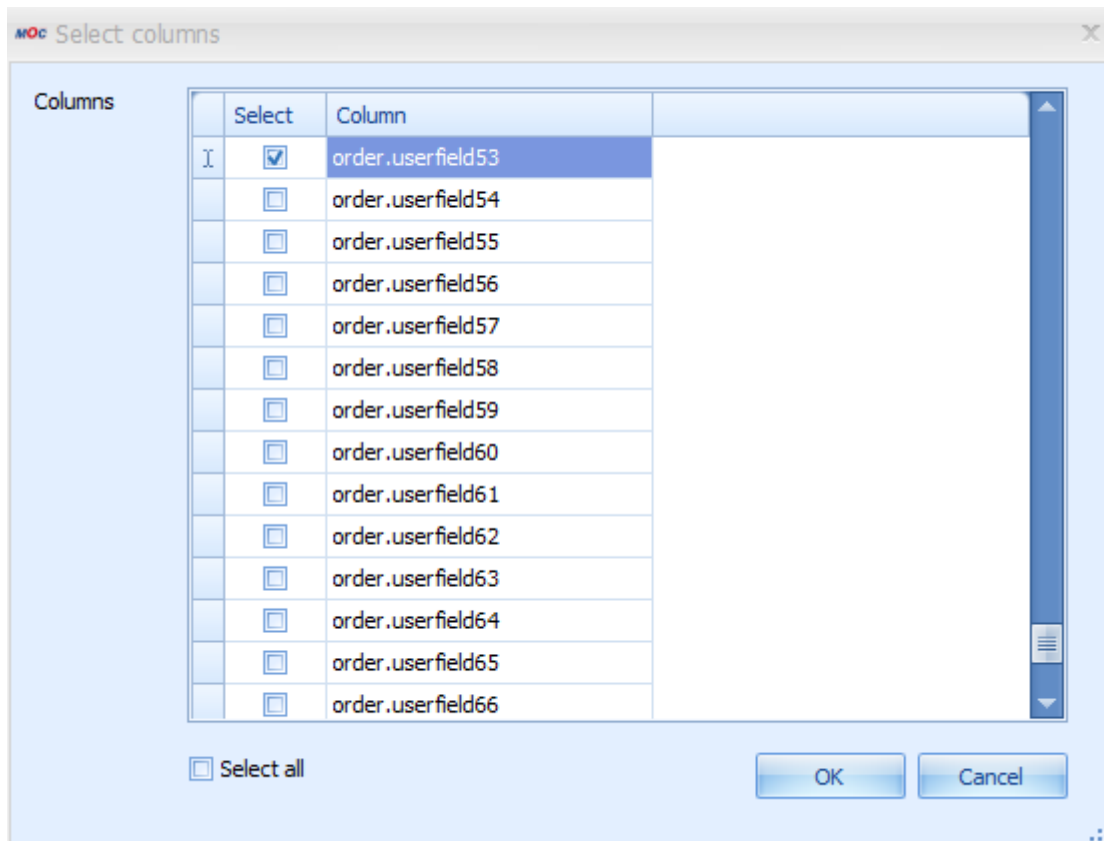


At present, the user fields are only supported in table and detail views (layout).

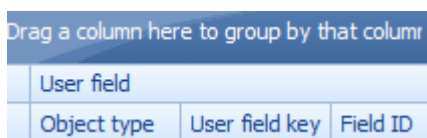
A screenshot of a 'User field configuration' dialog box. It has a title bar with 'MOC User field configuration' and a close button. The dialog contains three input fields: 'Object type' with a dropdown menu showing 'AUNR', 'Object in data source' with a text box containing 'order', and 'User field key' with a dropdown menu showing 'DEFAULT'. At the bottom, there are 'OK' and 'Cancel' buttons.

After pressing the OK button, the user configuration is read and all user fields are returned along with a configuration.

On basis of the identified user field - field names, all acronyms with the same field name will be identified in the related data source of the current detail application. The data source BOOrderOverview.list has two acronyms with "userfield53" in the following example. The user may choose which acronyms shall be used for the read user field configuration.



Upon pressing OK, a new category, "User fields" with related columns is created.



After saving the application, this information is stored in the application configuration. Should the user field configuration have changed, the procedure must be followed again.

User fields may be deleted by entering a blank object type and user field key in the user field configuration.

User fields as Independent Detail View

User fields may be shown in an independent detail view. A separate application type, **UserFieldDetail**, is available for this. This allows for displaying several user field definitions in one application.

After selecting the user fields, the standard columns of the data source are deleted and only the user fields are shown.

14 Authorization

The authorization mechanism is used to protect functions such as applications or buttons in the toolbar against unauthorized use and/or hide fields or field groups in the GUI or prevent them from being edited.

In order to protect functions or fields, you can assign an **authorization key**. Once assigned, you must explicitly **activate** the key if you want to use the functions or fields.

The following chapters describe the assignment of authorization keys to functions and fields and the assignment of authorization keys to function authorizations, licenses and feature sets.

14.1 Activate authorization keys

14.1.1 How it works

You can use three components to activate an authorization key. The authorization is therefore based on "three pillars". For the MES Development Suite, only the *Function authorization* is relevant.

Authorization key		
Function authorization	License	Feature set
Assignment to one or several function authorization(s)	Assignment to one or several licenses (mostly used by MPDV only)	Assignment to one or several feature sets (mostly used by MPDV only)

This chapter does not only treat the function authorizations, but also briefly considers the two other components in order to gain an overall understanding of the processes.

An authorization key is activated

- if it is assigned to at least one of the three components
- if the requirements of all assigned components are met

14.1.2 Function authorizations

You use the files "functionauthorisation.txt" to assign function authorizations to the authorization keys in the MOC.

If several function authorizations are assigned to one authorization key, the logged on user must have assigned at least one of the function authorizations.

Example:

...\resources\data\authorizations\FunctionAuthorisationMappings\functionauthorisation.txt

```
...  
pcccfg;pcccfg  
wpas.sug;wpas.sug  
grapt.spe;grapt.spe  
chkresp;CheckResponsibilityareaSelection  
doctype;doctype  
mdres.resmgmt;mdres.resmgmt  
uslo;uslo  
pfautp;pfautp  
pfunk;pfunk  
autcockpit;autcockpit  
mslmonitoring;mslmonitoring  
...
```

The first column includes the function authorization. The authorization key is after the semicolon.



Always create your own assignment files in the "local" scope!

14.1.3 Licenses

The assignment of authorization keys to licenses is only used by MPDV. It is realized using a "functionMapping". The mapping files are stored on the server in JDIR or JHYDRADIR.

...\jhydradir\MOC\1\functionMapping\standard*.lic

...\jdir\MOC\1\functionMapping\standard*.lic

The files are generated by a tool and protected against changes by a checksum. If several licenses are assigned to one authorization key, at least one of the assigned licenses must be activated.

14.1.4 Feature sets

The assignment of authorization keys to feature sets is only used by MPDV.

You can use feature sets to activate specific functions in accordance with a product version. Each FeatureSet is stored on the server in JDIR or JHYDRADIR.

...\jdir\MOC\1\configManager\standard*.xml

...\\jhydradir\MOC\1\configManager\standard*.xml

If several feature sets are assigned to one authorization key, at least one of the assigned feature sets must be fulfilled.

14.2 Authorization of functions and fields

You assign authorization keys to functions, applications and fields in the MOC using the "*.Authorization.xml" files in the folder "resources\data\authorizations". There is one file for each domain. The files in the "Standard" scope are secured against manipulation by a checksum. If a file is identified as being manipulated, all authorizations will fail.

Authorizations are maintained in the MDS repository, i.e. the above files are created by the export function of the Repository Client for MPDV modifications or can be generated/modified manually by customers for individual modifications.

14.2.1 Authorization of Fields

In order to authorize individual fields, the domain and the AuthorizationID with the acronym to be authorized must be maintained as a minimum. AuthorizationType must be "Acronym", AuthorizationKey must comprise the authorization key. If a service is entered in AuthorizationContext, the field must only be authorized in this service; if the column remains empty, authorization is always given.

Example:

```
<Authorization Domain="BOPerson"
  AuthorizationContext="BOPerson.list"
  AuthorizationId="person.premium_indicator"
  AuthorizationType="Acronym"
  AuthorizationKey="prgrp"
  AuthorizationDesignation="Extracted from code" />
```

14.2.2 Authorization of Field Groups

Important columns for the authorization of groups are AuthorizationContext and AuthorizationID. AuthorizationType must be "AcronymGroups", AuthorizationKey must comprise the authorization key. The AuthorizationID corresponds to the group name and usually is a language key. The AuthorizationContext is comprised of ApplicationID and Plugin-ID, separated by points.

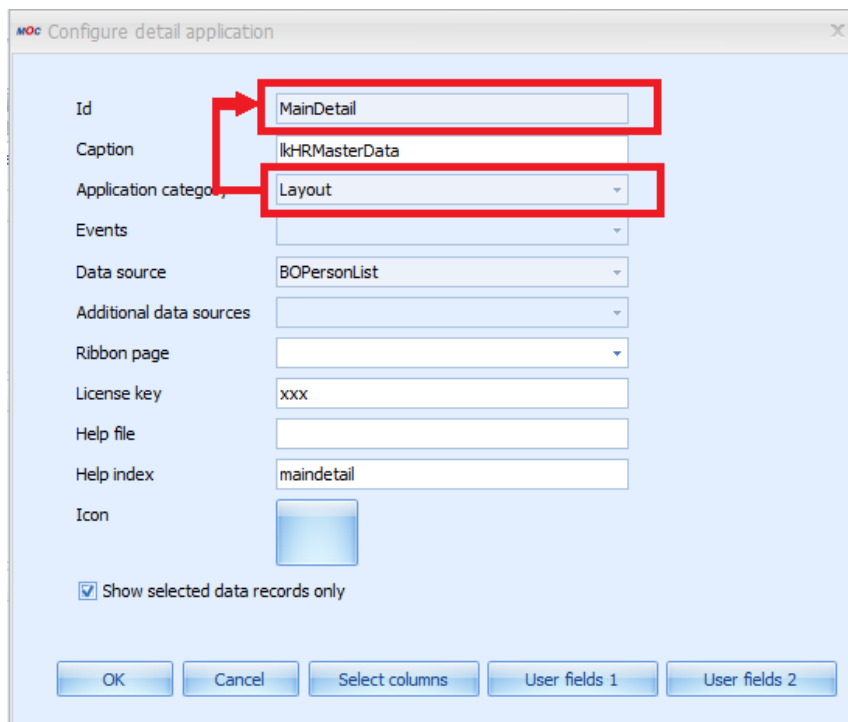
Example: Group in layout panel of main application "Persons":

```
<Authorization Domain="BOPerson"
                AuthorizationContext="Persons.LayoutMainDetail"
                AuthorizationId="lkName"
                AuthorizationType="AcronymGroups"
                AuthorizationKey="testkey"
                AuthorizationDesignation="Extracted from code" />
```

Explanation for AuthorizationContext:

"Persons" is the application ID.

The plug-in ID "LayoutMainDetail" is composed of the application category and the detail application ID. This can be read in "Configure detailed applications":



Example: Group in editing dialog under "Persons":

```
<Authorization Domain="BOPerson"
                AuthorizationContext="Update.ParameterLayoutBOPersonUpdate"
                AuthorizationId="lkPersonLLE"
                AuthorizationType="AcronymGroups"
                AuthorizationKey="testkey"
                AuthorizationDesignation="Extracted from code" />
```

14.2.3 Authorization of applications



The authorization of (standard) applications is provided by whitelist management, i.e. applications are always protected and must be enabled by an authorization key. An application without authorization key can therefore not be used.

Applications not included in the authorization configuration cannot be opened. Authorization will also have an effect on whether or not a menu entry is activated.

AuthorizationID is the application name, AuthorizationType is "Application". The AuthorizationContext may be left empty.

Example:

```
<Authorization Domain="UnsortedApplications"
  AuthorizationContext=""
  AuthorizationId="Persons"
  AuthorizationType="Application"
  AuthorizationKey="pers.★"
  AuthorizationDesignation="Imported form menu" />
```

14.2.4 Authorization of Detail Applications

An authorization key is assigned in the detail application configuration.

The screenshot shows a Windows-style dialog box titled "Configure detail application". It contains several configuration fields:

- Id:** MainDetail
- Caption:** lkHRMasterData
- Application category:** Layout
- Events:** (empty dropdown)
- Data source:** BOPersonList
- Additional data sources:** (empty dropdown)
- Ribbon page:** (empty dropdown)
- License key:** xxx (this field is highlighted with a thick black border)
- Help file:** (empty text box)
- Help index:** maindetail
- Icon:** (empty icon box)

At the bottom, there is a checkbox labeled "Show selected data records only" which is checked. Below the checkbox are five buttons: "OK", "Cancel", "Select columns", "User fields 1", and "User fields 2".

14.2.5 Authorization of Functions

Functions are authorized by assignment of an authorization key in the relevant CommandLink:

The 'Link editor' dialog box is shown with the 'Copy' menu item selected. The main form contains the following fields:

- ID: Copy
- Command: Copy
- Function: openApplicationContainerForEdit
- Parameter: id=Copy
- Authorization key: edcor.copy
- Title: lkCopy
- Tooltip: lkCopy
- Category: lkFunctions
- Ribbon page: lkMainPage
- Context:
- Disabled: ☐
- Shortcut: None
- Image(small):
- Image(large):

Buttons at the bottom: New, Delete, OK, Cancel.

14.2.6 Authorization of ReferenceData entries

You can authorize ReferenceData entries by creating an authorization entry of type "RefDatItem". The context corresponds to the ReferenceData type. The value (db_key) is the authorization ID.

Example:

```
<Authorization Domain="BOPMaintenance"
  AuthorizationContext="rmcal.type"
  AuthorizationId="B"
  AuthorizationType="RefDatItem"
  AuthorizationKey="rmcaltypes"
  AuthorizationDesignation="Authorization for ReferenceData-item rmcal.type:B" />
```

Domain Set	Domain	ref_data_key	type	db_key	Description	is_default	designation	sort_key	Parent
Local	WageTypeInteractions	PersonnelConfigurationSelection:NONE	PersonnelConfigurationSelection	NONE		Y	lkNoSelection	1	Local->WageTypeInteractions->ReferenceData
Local	WageTypeInteractions	PersonnelConfigurationSelection:PNR	PersonnelConfigurationSelection	PNR		N	lkPerson	2	Local->WageTypeInteractions->ReferenceData
Local	WageTypeInteractions	PersonnelConfigurationSelection:KST	PersonnelConfigurationSelection	KST		N	lkCostCenter	3	Local->WageTypeInteractions->ReferenceData
Local	WageTypeInteractions	PersonnelConfigurationSelection:BER	PersonnelConfigurationSelection	BER		N	lkArea	4	Local->WageTypeInteractions->ReferenceData
Local	WageTypeInteractions	PersonnelConfigurationSelection:ABT	PersonnelConfigurationSelection	ABT		N	lkDepartment	5	Local->WageTypeInteractions->ReferenceData
Local	WageTypeInteractions	PersonnelConfigurationSelection:PKREIS	PersonnelConfigurationSelection	PKREIS		N	lkperson.employee_subgroup	6	Local->WageTypeInteractions->ReferenceData
Local	WageTypeInteractions	PersonnelConfigurationSelection:TAETIGKEIT	PersonnelConfigurationSelection	TAETIGKEIT		N	lkperson.function	7	Local->WageTypeInteractions->ReferenceData
Local	WageTypeInteractions	PersonnelConfigurationSelection:BESCHVERH	PersonnelConfigurationSelection	BESCHVERH		N	lkperson.employment_relationship	8	Local->WageTypeInteractions->ReferenceData
Local	WageTypeInteractions	PersonnelConfigurationSelection:NSTMP	PersonnelConfigurationSelection	NSTMP		N	lkperson.DoesNotClock	9	Local->WageTypeInteractions->ReferenceData

Authorizations [2]								
	Domain Set	Domain	Authorization Type	Authorization Context	Authorization Id	Authorization Key	Authorization Designation	Parer
	Local	WageTypeInteractions	RefDatItem	PersonnelConfigurationSelection	NSTMP	PersonalTimeEvaluationControlDoesNotClock	Person does not clock	Local
	Local	WageTypeInteractions	Application		WageTypeInteractions	wtia.*	Imported from menu	Local

15 Application Generator

Applications may be generated in an automated way by the application generator. Automation refers in particular to standard features that are used in many applications (above all editing dialogs) and that may easily be standardized for this reason.

The application generator receives all its information from the repository or the files exported from there. Consequently, it is reasonable to edit all affected data in the repository before a new application is generated.

Each new application can and should be created using the generator. If the entries available in the repository are identified as being incorrect after the generation, they should be modified first before generating a new application.

Each application may still be changed later. Some changes can be made at runtime using editors and others can only be performed by directly editing the configuration files. If possible, modifications should always be made via the repository, as in this case they can also be used for other applications.

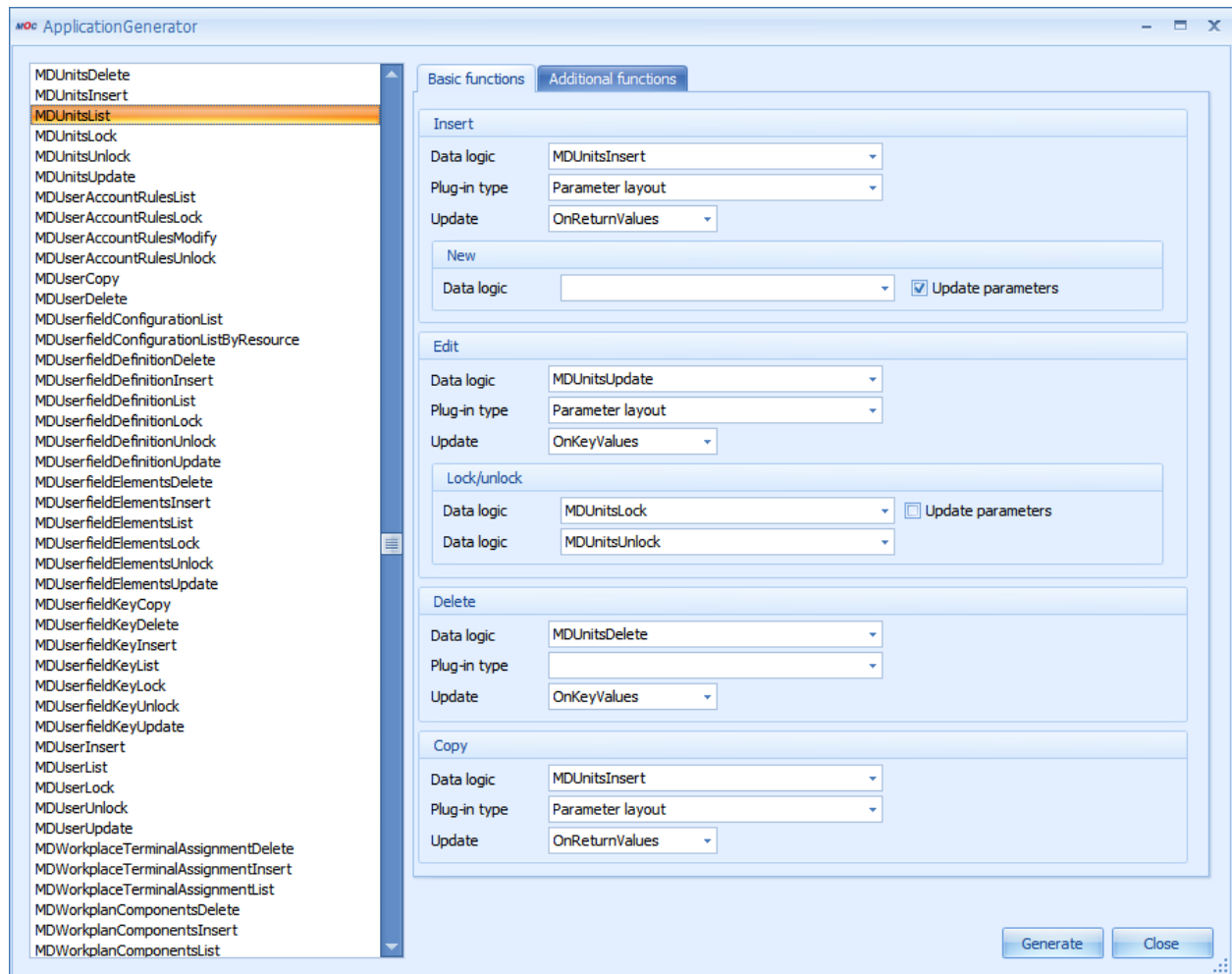
A generated main application always includes:

- a (main) data source
- a selection panel (Plug-in = ParameterLayout, Configuration file = LayoutPanel.config)
- A grid application to show the data for the data source (Plug-in = Grid, Configuration file = GridMainGrid.config)
- A detail application to show the data records selected in the grid (plug-in = Layout, configuration file = LayoutMainDetail.config)
- 0-n toolbar buttons to start editing functions (configuration file = ApplicationCommandLinkCollection.config)
- 0-n (sub-)applications to implement the editing functions

If additional editing applications are generated, they also include data sources and, if necessary, detail applications. Moreover, they can also be edited and saved at runtime, just as it is the case for "normal" main applications.

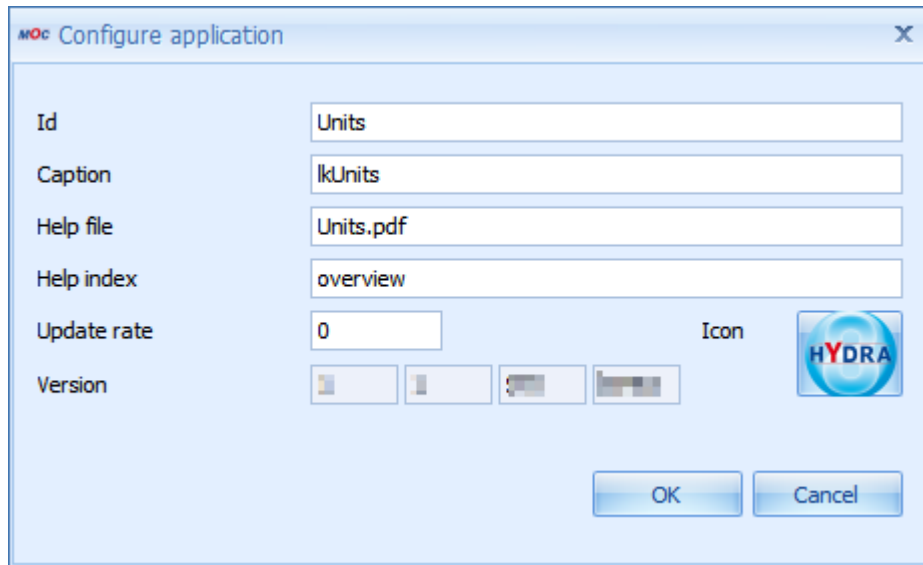
15.1 Instructions

The application generator is available if the MES Development Suite is enabled and can be started by the shortcut `_cgenapp` in the quick launch bar or by using the main menu (MES Development Suite → Generate application).



- Data logic has to be selected first from the list on the left-hand side.
 - This list includes all data logic exported from the repository into the files in <MOC>\resources\data\services\xxx.DataLogic.xml. If a specific entry is missing in the list, it should be checked whether or not the data have already been exported.
 - The selected data logic is then used as main data source within the main application (for the grid as well as for the detail application).
 - Normally, main data logic has the extension "List" or "Overview".
 - The selection process is simplified by "Incremental Search", i.e. if the first letters of the data logic name are entered, the matching entry will be selected automatically.
- Once data logic has been selected, most fields on the right-hand side of the generator are automatically assigned default values.
 - This pre-assignment refers to default values matching in many cases but certainly not in every case. Therefore, the field assignment has to be checked thoroughly as to whether it meets the requirements of the application to be generated (see the description of the fields).
 - At the moment, the basic functions Insert, Copy, Update and Delete are automatically assigned as editing functions. If the relevant service is available, the corresponding field will be assigned automatically in the generator.
- Besides the standard editing functions, up to five further editing functions can be defined on the tab "additional functions" (e.g. Activate, Deactivate, etc.). No additional editing functions will be created if nothing is edited here.

- A dialog to configure the main dialog opens by clicking the "generate" button. This dialog is already assigned default values within the repository, if data logic has been configured correctly. However, the relevant specifications may also be entered manually.



- Once this dialog has been closed, the main dialog and, if necessary, editing dialogs of the application are generated and directly opened.

15.2 Description of the fields

The individual editing functions can be edited on the right-hand side of the generator. As already mentioned, the default functions are "insert", "copy", "edit" and "delete". Up to five further functions may be defined on the second tab. An editing application is generated automatically for each editing function for which data logic has been selected.

The following specifications can be made for each editing function:

Data logic

Data logic selects the service that is respectively started/used for the editing of data.

An editing application is generated automatically, once data logic has been selected. If the data logic name is empty, the created application will not include the editing application.

Data logic (New), which is executed before the actual insert process, may be defined additionally for "insert". Data logic for the new service is automatically assigned to default values when selecting list data logic, provided the relevant service is available.

Additional data logic may be specified for "update". This data logic is executed before the actual "update" process before (Lock) or after (Unlock). Data logic for Lock/Unlock is also assigned default values when selecting list data logic, provided the relevant services are available.

Most applications do not have a separate copy service. If, however, a copy function is still to be used, the insert dialog will normally be used and its fields will be assigned to default values. If a copy service exists, it will be assigned by default. Otherwise, the insert service is assigned automatically. Data logic has to be deleted here if no copy function is required.

Plug-in type

The plug-in type specifies which application plug-in is used for editing data.

Any application plug-in may be selected for each editing dialog. The only condition: the plug-in has to implement `IParameterContainer`. The selection list only includes appropriate plug-ins. The plug-in `ParameterLayout` is always set by default.

It is also possible to NOT select a plug-in. Consequently, a dialog without plug-in is created automatically. However, this dialog will not be opened by clicking the relevant toolbar button. Then the application folder includes a configuration that may be edited. Example: "standard delete" without dialog.

Update (UpdateSourceMode)

Specifies how data are to be updated in the main application, once the editing function has been executed. You may choose from the following options:

- All: all data of the main application are completely refreshed (taking the values entered in the selection panel into account). This is required, for example, for separate deletion and copy dialogs. As they do not allow for the affected data records to be determined specifically and requested only.
- OnReturnValues: the called service returns values that are used to request only the concerned data. This may be the case, e.g. for "Insert". In this context, it is often the case that a new key field is generated that is required to request the new data record.

Please note: However, this setting is not reasonable if the called service does not return any fields that are suitable for a unique identification of the affected data record. This setting is set by default for "inserting" and might be changed, if necessary.

- OnKeyValues: the key fields of the list service are used to request only the affected data. This is, for example, the ideal procedure for the "update".

Please note: In this case, the service of the main application and the editing service should have the same key fields; otherwise it does not work.

- OnScript: for future use

This configuration should be checked thoroughly. As subsequent changes can only be made manually in the main configuration file of the editing dialog (e.g. `EditOrders\Insert\Insert.config`, section: `UpdateSource`).

Update parameter

This entry specifies whether or not the return value of a service is to be used to replace the original parameters. These new parameters will then be used during the course of the process.

Example: Calling the new service before the "insert" is normally used to request data that will then be assigned by default in editing dialogs. Therefore, the data provided by this service are required during the course of the process. Consequently, it is reasonable to set the check box to "true". For this reason, this is the default value assigned for "New".

Name

Name of the editing function. This value has already been assigned as the default value for the basic functions and can only be assigned individually for the additional functions. This name is used for the following positions:

- The editing application will be named like that as a part of the main application. Consequently, the folders included in the configuration also have this name. For this reason, the name has to be clear and unique within the main application.
- Ik + Name = labeling of the created toolbar button
- The command link on which the toolbar button is based is named like that.

Small/large

Icons for the application may be selected by image dialogs. These icons are used for the following positions.

- Small icon (16X16): presented top left of the editing application; possibly also in the toolbar.
- Large icon (32X32): might be displayed in the toolbar (both icons should be defined as the ribbon defines whether the large or the small icon is shown)

Transfer of parameters (ParameterProcessingMode)

Please note! This configuration can neither be set by the generator nor by a wizard!

The configuration `ParameterProcessingMode` specifies if and how parameters from the calling main dialog are transferred to the editing dialog. You may choose from the following options:

- `NoParameter`: no parameters will be transferred
- `SingleParameter`: obligatory transfer of the selected grid row; an error message occurs if no row is selected.
- `OptionalSingleParameter`: optional transfer of the selected grid rows; no error message
- `MultiParameter`: obligatory transfer of one or several grid rows; an error message occurs if no row is selected.
- `OptionalMultiParameter`: optional transfer of one or several grid rows; no error message.
- `SingleSelectionParameter`: transfer of the selected grid row. If no row is selected the values from the selection panel will be transferred. If no values are transferred from there --> error message.
- `OptionalSingleSelectionParameter`: transfer of the selected grid row. If no row is selected the values from the selection panel will be transferred. No error message.
- `SelectionOnlyParameter`: the values from the selection panel are transferred. If no values are transferred --> error message.

- OptionalSelectionOnlyParameter: the values from the selection panel are transferred. No error message.

As a part of the generation process, the following values are always set automatically for editing applications:

- Insert: NoParameter
- Update: SingleParameter
- Copy: SingleParameter
- Delete (including message box, without editing dialog): MultiParameter
- Delete (including editing dialog): SingleParameter

At the moment, these values can only be changed manually and directly in the configuration (e.g. EditOrders\Insert\Insert.config, section: ParameterProcessingMode).

16 Customization of the MOC Menu

16.1 Requirements

- The customization of the MOC menu using the customization file is available as of service pack 15 (end of 2019).
- If you want to make customizations, you require a development license for developments on the MOC and the relevant runtime licenses for further systems, if required.

16.2 Overview

You can not only edit and save complete sub menus using the [Menu editor](#), you can also customize single menu items and change, delete or remove the entries via configuration files without having to copy the entire sub menu.

- You can change existing menu items (e.g. call an application that you created yourself instead of the standard application or change the label text).
- You can add a new menu item before or after an existing menu item.
- You can delete an existing menu item.

You use a JSON file for customization, which is stored in the MOC directory.

16.3 Customization file MenuAdjustments.json

You can create the customization file using a simple text editor. The file is deployed to the MOC instances using the existing tools.

Storage of files

Only the files in the scopes "Local" and "Custom" are integrated:

```
[MOC_ROOT] local\conf\MOC\MenuAdjustments\[MenuName]\MenuAdjustments.json
```

```
[MOC_ROOT] custom\conf\MOC\MenuAdjustments\[MenuName]\MenuAdjustments.json
```

Example:

```
c:\Moc\local\conf\MOC\MenuAdjustments\RoleMenu\MenuAdjustments.json
```

Example file

```
{
  "Adjustments": [
    {
      "RefElementInternalId": "3afbdf93-cd1a-4b08-ada2-e73d2cbb32c8",
      "AdjustmentType": "ADD",
      "Position": "BEFORE",
    }
  ]
}
```

```

    "CommandLink": {
      "LinkId": "beforeFuncAuthorizations",
      "Id": "cmdNewApplicationContainerForDisplay",
      "FunctionName": "beforeFuncAuthorizations",
      "Label": "lkAnyAppBeforeFuncAuthorizations",
      "FunctionParameter": "windowId=test windowOpenMode=SingleWindow id=Units",
      "AuthorisationKey": "mdunit.*",
      "SmallImage": "PushRightSmall",
      "LargeImage": "PushRightLarge"
    }
  },
  {
    "RefElementInternalId": "3afbdf93-cd1a-4b08-ada2-e73d2cbb32c8",
    "AdjustmentType": "ADD",
    "Position": "AFTER",
    "CommandLink": {
      "LinkId": "afterFuncAuthorizations",
      "Id": "cmdNewApplicationContainerForDisplay",
      "FunctionName": "afterFuncAuthorizations",
      "Label": "lkAnyAppAfterFuncAuthorizations",
      "FunctionParameter": "windowId=test windowOpenMode=SingleWindow id=Units",
      "AuthorisationKey": "mdunit.*",
      "SmallImage": "PushRightSmall",
      "LargeImage": "PushRightLarge"
    }
  },
  {
    "RefElementInternalId": "a7954dec-59fb-4454-858f-7f15bfb14805",
    "AdjustmentType": "MODIFY",
    "CommandLink": {
      "LinkId": "modifiedFunctionAuthorizationProfile",
      "Id": "cmdNewApplicationContainerForDisplay",
      "FunctionName": "beforeFuncAuthorizations",
      "Label": "replaceFunctionAuthorizationProfileByAnyApp",
      "FunctionParameter": "windowId=test windowOpenMode=SingleWindow id=Units",
      "AuthorisationKey": "mdunit.*",
      "SmallImage": "PushRightSmall",
      "LargeImage": "PushRightLarge"
    }
  },
  {
    "RefElementInternalId": "41185cd9-e089-4385-8fb4-6c27af6464cb",
    "AdjustmentType": "DELETE",
    "MyOptionalComment": "Delete app 'locked data records'"
  }
]
}

```

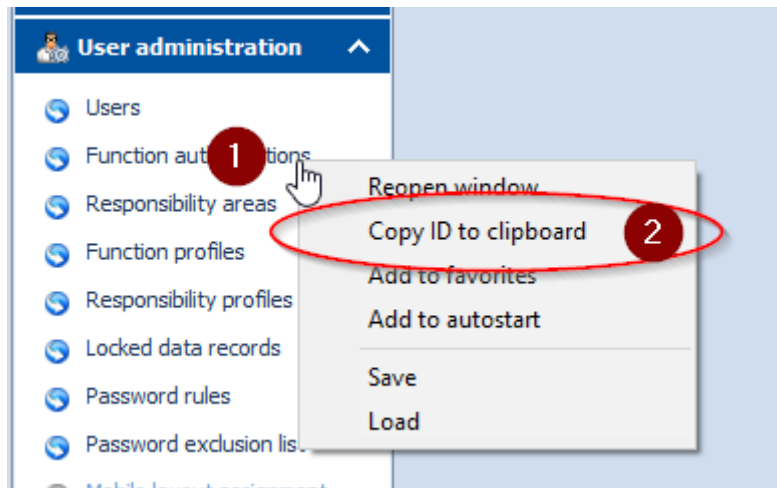
Explanation of the customization file

The file includes data in JSON format.

To insert, change and delete menu items, you always require the reference to an existing menu item. The reference is called "RefElementInternalId" in JSON.

You can identify the "RefElementInternalId" of a menu item on the MOC:

1. Enable the MES Development Suite on the MOC.
2. Right-click on the required menu item to open the context menu.
3. Select *Copy ID to clipboard*. The "RefElementInternalId" is now contained in the clipboard and can be pasted to the customization file.



The JSON file includes the array "Adjustments".

```
{  
  "Adjustments": [ ... ]  
}
```

The array includes one or several rules that are processed by the MOC.

The MOC first loads the menus created via menu editor. The MOC only shows the menus when the rules of the customization file have been processed. First the rules of the "custom" scope are processed, then the rules of the "local" scope. The changes caused by the rules only change the display on the MOC. The original menu saved using the menu editor is not changed.

Each rule is a JSON object with the following attributes:

RefElementInternalId

Unique ID (UUID) of the existing menu item. The rule applies to this ID.

AdjustmentType

ADD: This rule adds a new menu item before or after the menu item specified via "RefElementInternalId".

MODIFY: The rule changes the menu item specified via "RefElementInternalId".

DELETE: The rule deletes the menu item specified via "RefElementInternalId".

Position

With rules of type "ADD", the *Position* specifies if the new menu item is added before or after the existing menu item.

CommandLink

With rules of type ADD or MODIFY, you must specify a CommandLink. You must entirely specify the CommandLink in each scope with all attributes. The system does not perform a merge with the CommandLink of the more general scope. The attributes of the CommandLink are identical to the properties of buttons in the toolbar or the columns in the menu editor:

```
...
"CommandLink": {
  "LinkId": "modifiedFunctionAuthorizationProfile",
  "Id": "cmdNewApplicationContainerForDisplay",
  "FunctionName": "beforeFuncAuthorizations",
  "Label": "replaceFunctionAuthorizationProfileByAnyApp",
  "FunctionParameter": "windowId=test windowOpenMode=SingleWindow id=Units",
  "AuthorisationKey": "mdunit.*",
  "SmallImage": "PushRightSmall",
  "LargeImage": "PushRightLarge"
}
...
```

CommandLink.LinkId

Assign a unique ID to the ApplicationCommandLink that is called (e.g. ID of the application that is called).

CommandLink.Id

Command that is executed. To open applications, use "cmdNewApplicationContainerForDisplay".

CommandLink.FunctionName

Assign a unique name to the ApplicationCommandLink that is called (e.g. ID of the application that is called).

CommandLink.Label

Text of menu item. You can either enter a text or a language key. Use language keys if you want to operate the MOC in several languages.

CommandLink.FunctionParameter

Parameter that calls the command of "CommandLink.Id". For cmdNewApplicationContainerForDisplay, this is for example:

id=<ID of application>

In addition, the below parameters can optionally be transferred (added at the end, separated by blanks):



The parameters are case sensitive!

autorequest=true/false: If true, data is automatically requested after the application has been opened.

windowOpenMode=MultiWindow/SingleWindow: If MultiWindow, a new instance of the application is opened every time it is linked; if SingleWindow, the same instance of the application is always opened.

Modal=true/false: If true, the application is opened modally.

selectedprofile=<selection profile>: If an existing selection profile is transferred, it is automatically preassigned when the application is opened. If the name of the selected profile includes blank characters, you must use "inverted commas" for the profile name.

CommandLink.AuthorisationKey

You can optionally enter an authorization key here. This key controls the visibility of the menu item according to the user's authorizations (e.g. function authorizations). If you do not want to assign an authorization key, do not use this attribute.

CommandLink.SmallImage**CommandLink.LargeImage**

Icon (small or large) displayed next to the menu item. Menu items usually get the icons "PushRightSmall" or "PushRightLarge". If you want to use other icons for exceptions, you can find the names of the available icons on the MOC using the picture selection during configuration of CommandLinks of the toolbar.



You do not necessarily require an entry in the files "functions.xml" and "functionauthorisation.txt" for new CommandLinks in the customization file MenuAdjustments.json for a new MOC application.

- You require the file "functions.xml" if an application is also used in the menu editor of the MOC.
- You require the file "functionauthorisation.txt" if you want to call an application using the transaction code.

17 MDS Extensibility Framework

17.1 Overview

You use the Extensibility Framework to create extensions that you have programmed using the .NET Framework. You can also use the predefined extensions that are provided in the standard.

Via extensions, you can add functions to the MOC that are not available via simple configuration. You can call the extensions in the MOC toolbar if configured accordingly. You can integrate separate applications into the MOC menu.

The programming environment for customizations is the .NET Framework of Microsoft. You can use any programming language that is supported by .NET Framework. The programming language used in the examples of this document is C#.

Examples

To customize the MOC, the following options are available:

1. Dynamic behavior of application
 - Specific logic to visualize/enable command buttons in the toolbar
 - Specific logic to validate input fields before requesting data
 - Specific logic with/after entry of values into an input field
2. Extension of the toolbar with external self-programmed functions
3. Development of entirely own applications (free programming) that can be integrated into the MOC (menu and transaction code). Internal functions of the MOC are provided, e.g. web services, error logging, system information, reporting, translation libraries, etc. You can use all possibilities of the .NET Framework and you can also use third-party components.

17.2 Dependency Injection

If you develop own customizations, you require objects that provide basic functions (logging, calling web service, printing report, etc.).

These objects are passed in the constructors of the classes. Contrary to the *classic* object-oriented programming, you need not define the objects in a fixed order and number. The objects that are passed to the constructor are dynamically "injected".

For further information on the *Dependency injection*, refer to the relevant literature.

Examples:

```
// Constructor without parameter.
public constructorExample()
{
}

// Constructor with parameter, IRequestFactory is injected at run time.
public constructorExample1 (IRequestFactory requestFactory)
{
    this.requestFactory = requestFactory;
}
```

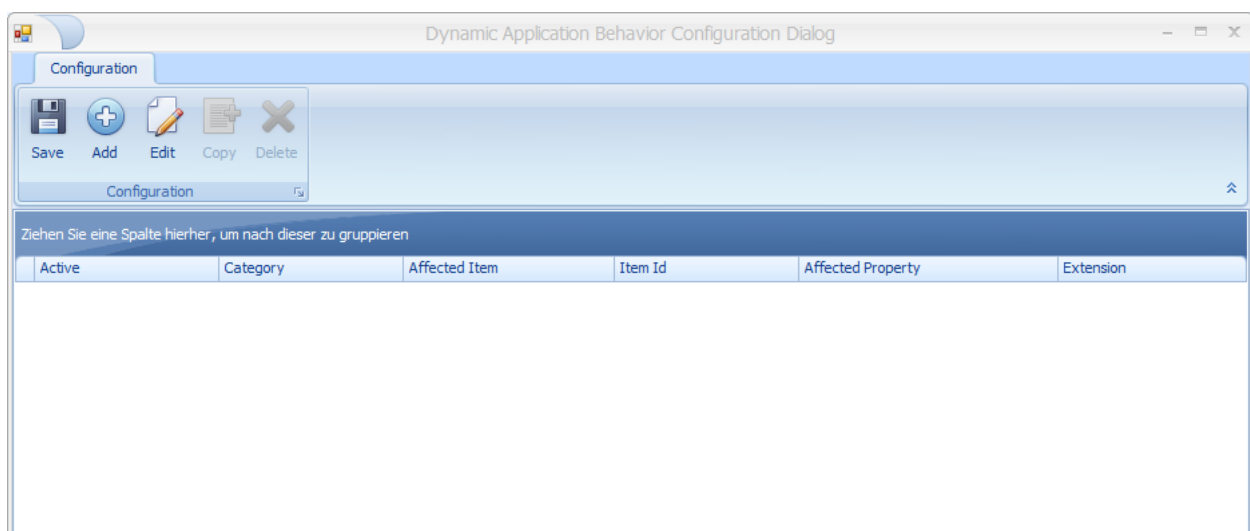
17.3 Extended application configuration

17.3.1 Configuration

You can configure an extension using the button *Configure dynamic behavior of application* in the toolbar of an application. This function is only available if the MES Development Suite is available and enabled.

The configured extensions made to an application are saved in the file ***DynamicApplicationBehaviors.config*** in the directory of the respective application.

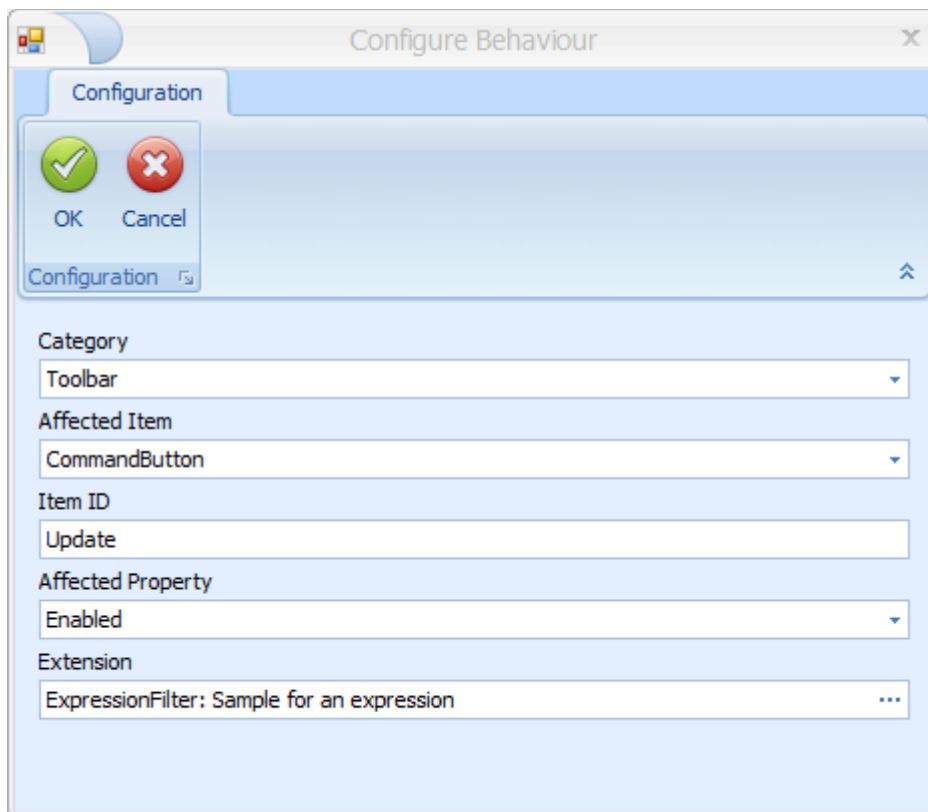
Example: <MOC directory>\local\conf\MOC\Apps\OrderOverview\DynamicApplicationBehaviors.config



Add/Change a configuration

- **Category:**
 - Toolbar
 - SelectionPanel (input fields selection criteria)
- **AffectedItem** (depending on Category):
 - Toolbar:
 - CommandButton
 - SelectionPanel:
 - Validation (check when data is requested)
 - EditControl (check when data is entered)
- **Item ID:** ID of the affected item (depending on AffectedItem)
 - CommandButton: ID of the function in the link editor (e.g. OrderInformation in the Order overview)
 - EditControl: Fieldname of the input field (e.g. order.id in the Order overview)
- **Affected Property** (only with **AffectedItem** = CommandButton)
 - Visible
 - Enabled

- **Extension:** Selection of the extension in a further dialog



17.3.2 Available standard extensions

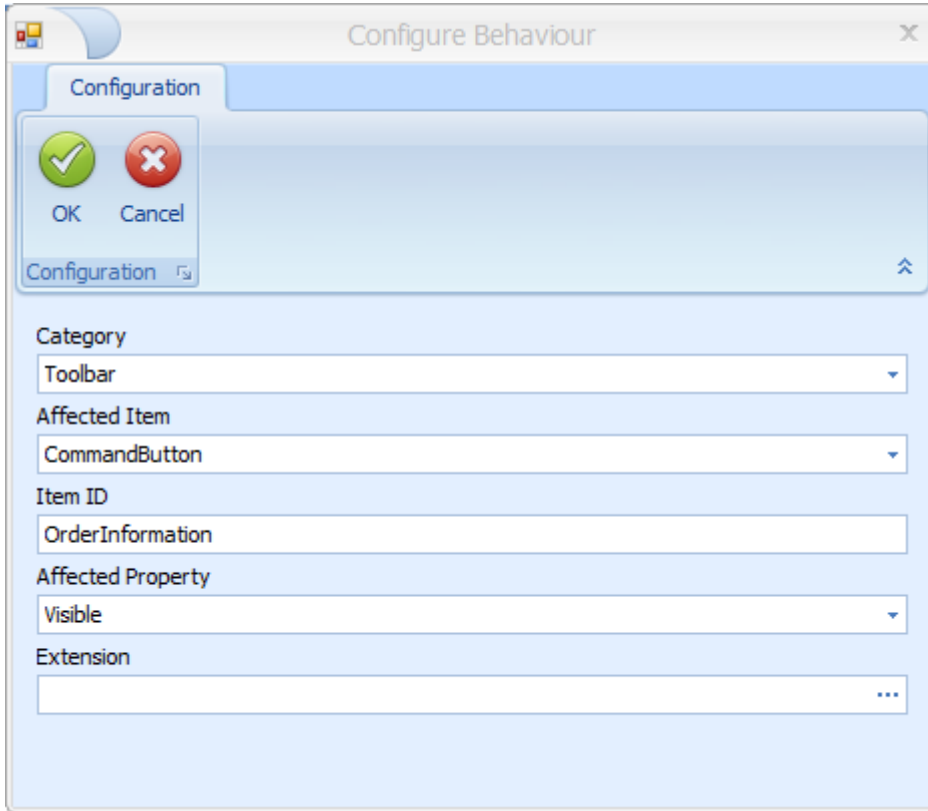
17.3.2.1 Visible/Enabled for toolbar functions

- **Category:** Toolbar
- **AffectedItem:** CommandButton
- **Item ID:** ID of the function (e.g. "OrderInformation" in the Order overview)
- **Affected Property:** Visible or Enabled
 - Visible
 - Enabled
- **Extension:**
 - ExtensionType: IApplicationContextFilter
 - Implementation Type:
 - SingleSelectionType
 - ActivePluginFilter
 - SelectedRowCountFilter
 - AnySelectionFilter
 - MultiSelectionFilter
 - InverseAuthorisation

- IniDataValueFilter

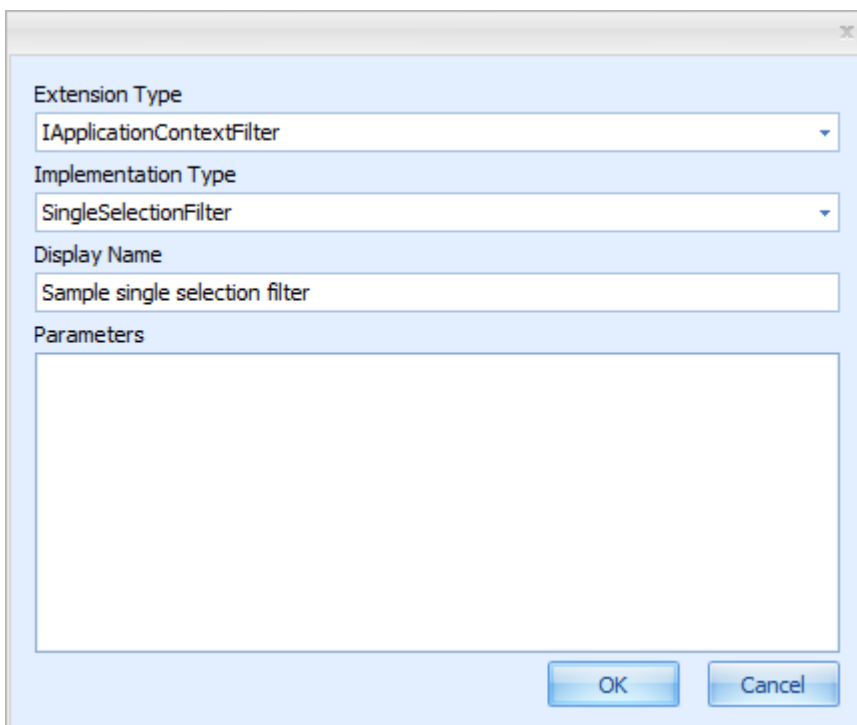
Example 1:

The function *Order information* in the *Order overview* is not visible if more than 1 order is selected.



The 'Configure Behaviour' dialog box is shown. It has a 'Configuration' tab and a 'Configuration' section with a 'Configuration' button and a 'Configuration' icon. The 'Configuration' section contains the following fields:

- Category: Toolbar
- Affected Item: CommandButton
- Item ID: OrderInformation
- Affected Property: Visible
- Extension: (empty)



The extension configuration dialog box is shown. It contains the following fields:

- Extension Type: IApplicationContextFilter
- Implementation Type: SingleSelectionFilter
- Display Name: Sample single selection filter
- Parameters: (empty)

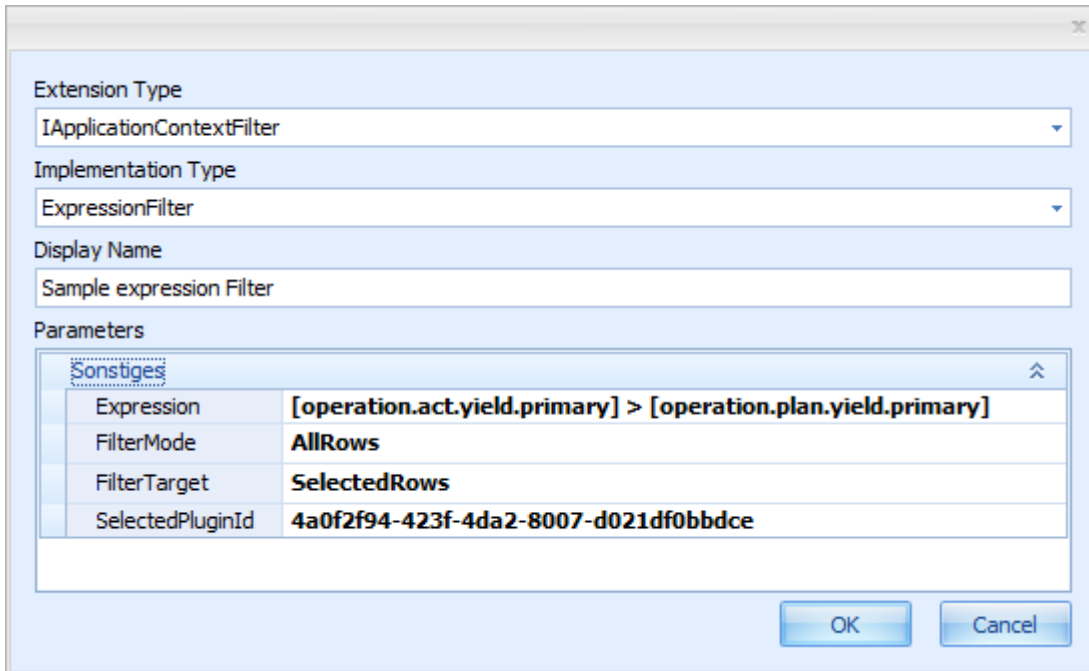
Buttons: OK, Cancel

Type Name	Extension Name	Description
Namespace: Mpdv.ApplicationExtensions.Core.Filters		
SingleSelectionFilter	SingleSelectionFilter	Returns true, if a single row is selected.
ActivePluginFilter	ActivePluginFilter	Returns true, if the supplied plugin id matches the currently active plugin's id (cas...
SelectedRowCountFilter	SelectedRowCountFi...	Returns true, if the exact number of configured rows is selected.
ExpressionFilter	ExpressionFilter	A filter that can evaluate DevExpress-Style expressions on the selected data rows.
AnySelectionFilter	AnySelectionFilter	Returns true, if a single row or multiple rows are selected.
MultiSelectionFilter	MultiSelectionFilter	Returns true, if more than one row is selected.
InverseAuthorization	InverseAuthorization	Returns true if the authorization key is not authorized.
IniDataValueFilter	IniDataValueFilter	Checks if a specified value is set in INI-configuration

Example 2:

The function *Terminate operation* in the *Order overview* is only enabled if the yield posted (P) is greater than the target quantity (P). You use the filter options of the table view (grid) to this end (ExpressionFilter). Extension: ExpressionFilter.

The screenshot shows the 'Configure Behaviour' dialog box with the 'Configuration' tab selected. The dialog contains a list of configuration items. The 'Item ID' field is set to 'acDynDlgTerminateOperation' and the 'Extension' field is set to 'ExpressionFilter: Sample expression Filter'. The 'Affected Property' is set to 'Enabled'. The 'Category' is set to 'Toolbar' and the 'Affected Item' is set to 'CommandButton'. The 'OK' button is highlighted.



Parameters	
Expression	[operation.act.yield.primary] > [operation.plan.yield.primary]
FilterMode	AllRows
FilterTarget	SelectedRows
SelectedPluginId	4a0f2f94-423f-4da2-8007-d021df0bbdce

- Expression: [operation.act.yield.primary] > [operation.plan.yield.primary]
- FilterMode: AllRows (valid for all data records)
- FilterTarget: SelectedRows (for the selected data records)
- SelectedPlugin: ID of the detail application that includes the data for the expression. In this case, the table Order progress of the application Order overview. You can identify the ID of the detail application using the function (button) *Configure detail application*.

17.3.2.2 Validation before requesting data

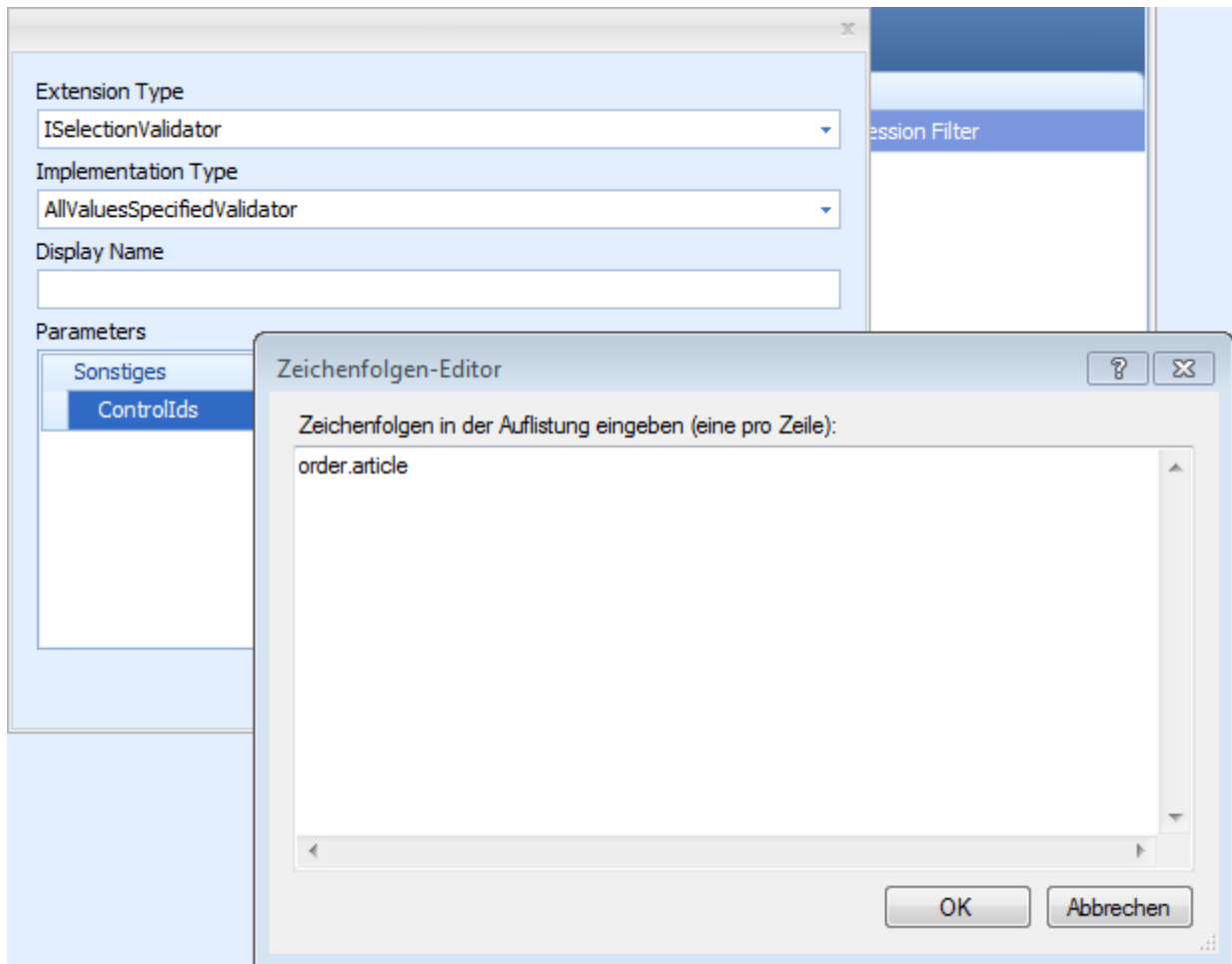
- **Category:** SelectionPanel
- **AffectedItem:** Validation
- **ItemID:** leer
- **Affected Property:** empty
- **Extension:**
 - ExtensionType: ISelectionValidator
 - Implementation Type:
 - AllValuesSpecifiedValidator
 - AtLeastOneValuesSpecifiedValidator

Example: You must enter a value in the field *Final article* in the *Order overview*.

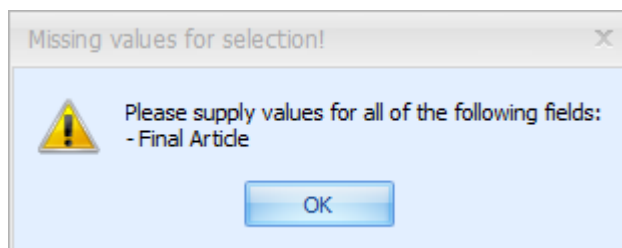
The image shows a 'Configure Behaviour' dialog box with the following fields and values:

- Category:** SelectionPanel
- Affected Item:** Validation
- Item ID:** (empty)
- Affected Property:** Event
- Extension:** AllValuesSpecifiedValidator: (This field is circled in red)

At the top left of the dialog, there are 'OK' and 'Cancel' buttons with corresponding checkmark and cross icons. A 'Configuration' label with a folder icon is also present.



If the standard extension is saved and the user does not enter a value in the selection field *Final article* before requesting data, the following error message is output:



17.3.1 Creating your own extensions

You can create your own extensions using the .NET Framework (e.g. C#, VB.NET).

17.3.1.1 Requirements

- Visual Studio Project or other .NET development environment.

- Output type: Class Library
- Output path: <MOC>\local\extensions)
- Target framework: as of 4.5.2
- Added references to Contract Libraries
 - Mpdv.Communication.Contracts.dll
 - Mpdv.Extensibility.Contracts.dll
 - Mpdv.Extensibility.Moc.Contracts.dll
 - Mpdv.ExtensionFramework.Contracts.dll
 - Mpdv.Integration.Moc.Contracts.dll
 - Mpdv.Utilities.Contracts.dll
 - Mpdv.Utilities.FileAccess.Contracts.dll
- Entry in the AssemblyInfo.cs: [assembly: ExtensionAssembly]

17.3.1.2 Creating an extension step by step

17.3.1.2.1 Requirements

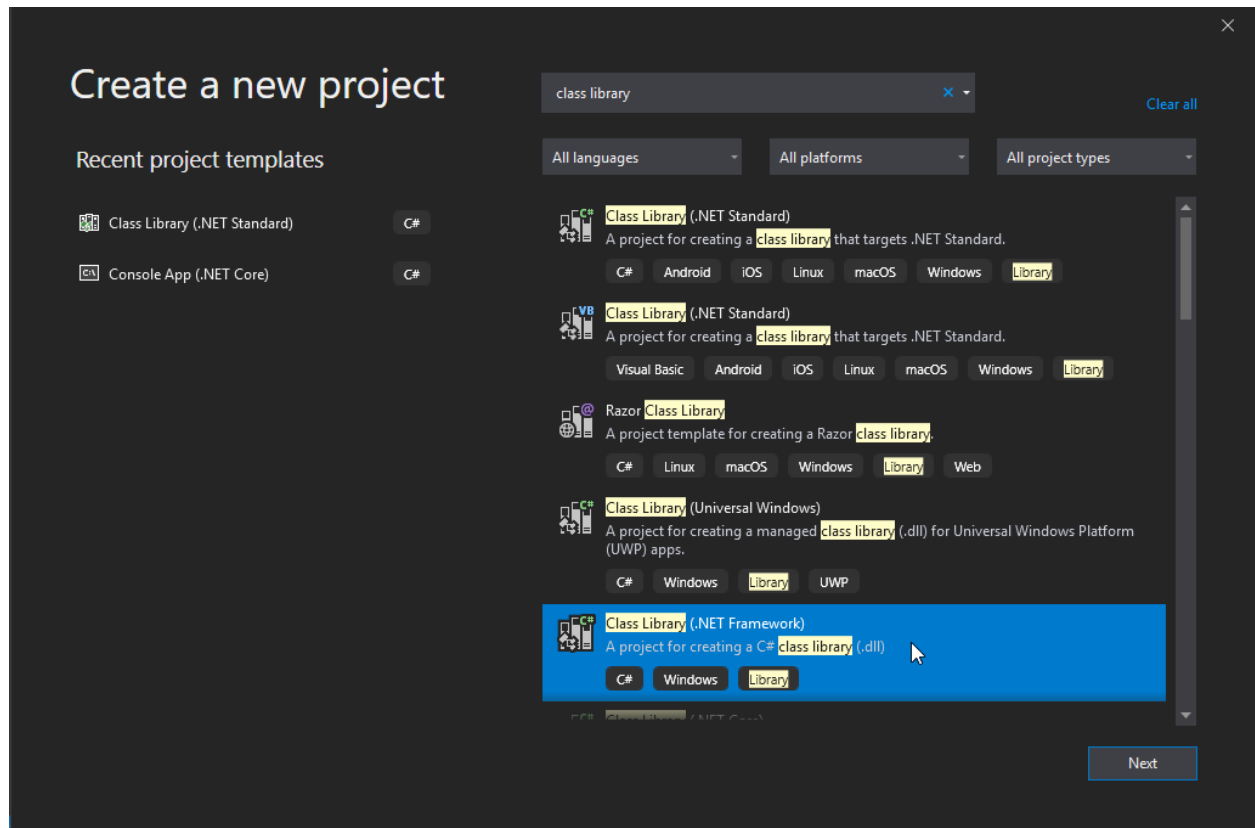
- MOC installation at the workstation (in the example in directory e:\MocNF\)
- Create the directory for the extensions in the MOC installation. In the example, this is "e:\MocNF\local\extensions\".
- Installation of "Visual Studio" at the workstation. The example was generated with an installation of Visual "Studio Community 2019".
- -
 -

If "Visual Studio" is not available, you can also create extensions using other .NET development environments. You can also use the free development environment "SharpDevelop". The settings that you must make are similar to the settings of this instruction.

17.3.1.2.2 Create a new project

Menu: File / New / Project

Select "Class Library (.NET Framework)"



Click [Next]

Project name: Ext_FieldUpperCase

Location: Storage location of project. In the example: "e:\DevSrc\vc\source\repos\Ext_FieldUpperCase"

Framework: .NET Framework 4.5.2

Click [Create]

17.3.1.2.3 Add references

Menu: Project / Add Reference...

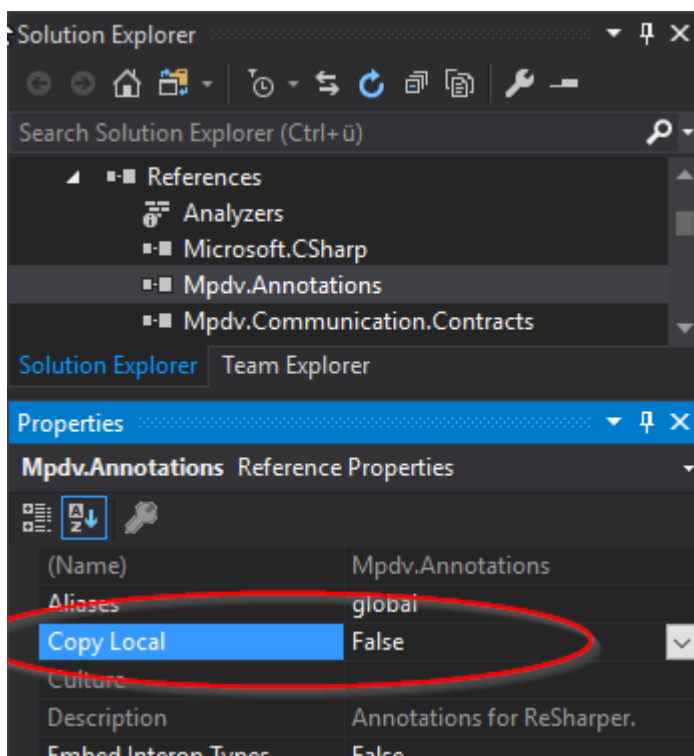
Click [Browse]

Change to the directory of the MOC installation, which contains the contracts. In the example, this is "E:\MocNF\contracts".

Add all DLLs contained in the directory. At least

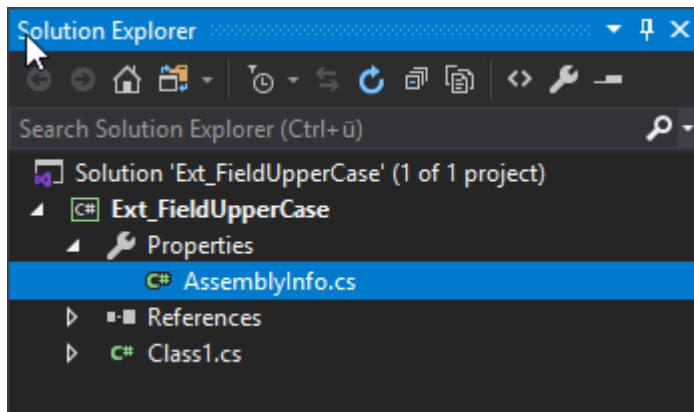
- Mpdv.Communication.Contracts.dll
- Mpdv.Extensibility.Contracts.dll
- Mpdv.Extensibility.Moc.Contracts.dll
- Mpdv.ExtensionFramework.Contracts.dll
- Mpdv.Integration.Moc.Contracts.dll
- Mpdv.Utilities.Contracts.dll
- Mpdv.Utilities.FileAccess.Contracts.dll

For all references "Mpdv.*", set the option "CopyLocal" to **False**. Otherwise, these libraries are copied into the "extensions" directory and then exist in two places.

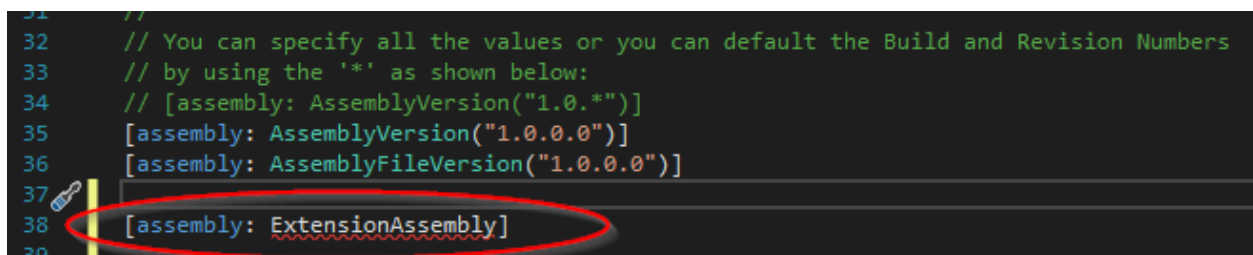


17.3.1.2.4 AssemblyInfo.cs

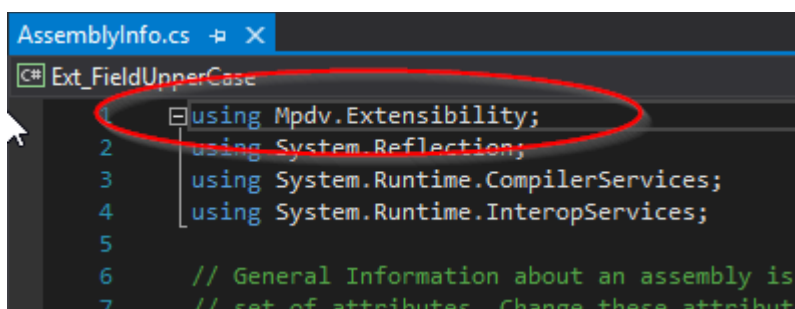
Open the file "AssemblyInfo.cs" that is included in the project.



Add a new row with the content "[assembly: ExtensionAssembly]". This row is necessary. The MOC only loads this assembly if this row is available.



At the beginning of the file, add a row with the content "using Mpdv.Extensibility" if the compiler cannot correctly interpret the previously added row.



17.3.1.2.5 Solution Properties

Menu: Project / Ext_FieldUpperCase Properties...

Select the category "Application".

Assembly name: FieldUpperCase.Extension

Default namespace: Ext.Extensions.Moc.FieldUpperCase

Target framework: Check whether the value is ".NET Framework 4.5.2".

Output type: Check whether the value is "Class Library".

Select the category "Build".

Output path: Set the output path to the sub directory "local/extensions" of your MOC installation (in the example "e:\MocNF\local\extensions\").

Save the settings (disk icon).

17.3.1.2.6 C# class

Create the C# class.

- Add the attribute [Extension]. Example: [Mpdv.Extensibility.Extension(Name = "name of extension",Description="description..")]
- Derive the class from the respective interface (see developer documentation, e.g. `IControlEnterListener`).
- Implement the methods of the interface.

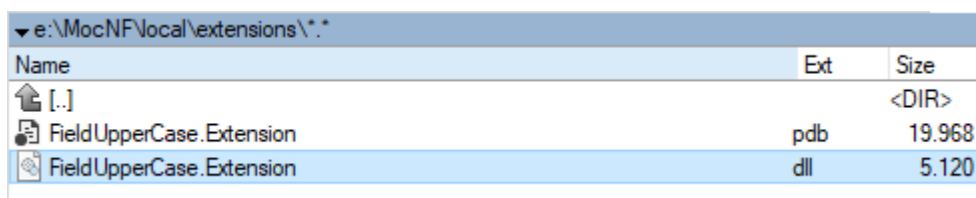
You can use one of the following examples as basis.

17.3.1.2.7 Test

Build the solution (menu: Build / Build Solution F6)

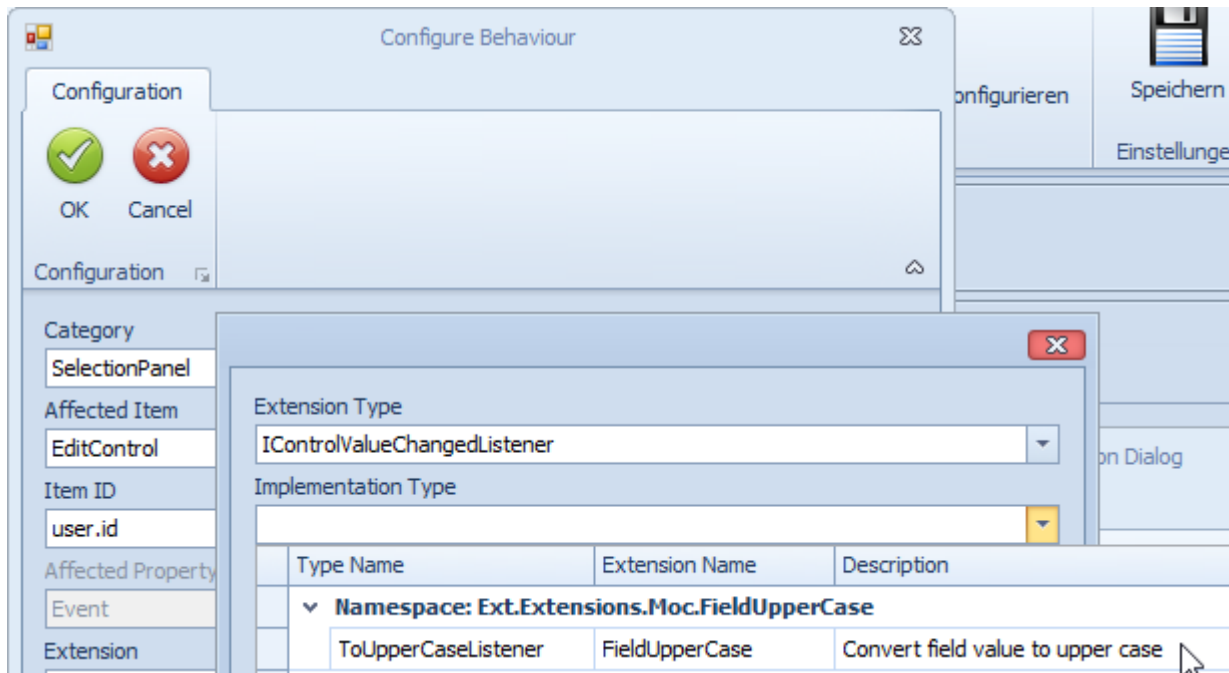
Check and correct the errors and warnings of the compiler.

After successful Build, a DLL with the name of the extension must be available in the MOC installation (in the example in directory "e:\MocNF\local\extensions\"):



▼ e:\MocNF\local\extensions\.*		
Name	Ext	Size
[.]		<DIR>
FieldUpperCase.Extension	pdb	19.968
FieldUpperCase.Extension	dll	5.120

After restart of the MOC, you can enable the *MES Development Suite* and configure the new extension.



17.3.1.3 Examples

The developer documentation describes the different interfaces. Find two examples in the following:

17.3.1.3.1 The value of an input field is converted into capital letters.

When you enter a value, the value is automatically converted into capital letters. To convert the value, the system uses the interface *IControlValueChangedListener*. This interface has the method *OnEditValueChanged(..)*.

Assembly name : ValueChanged.Extension.dll

```
using System;
using System.Windows.Forms;
using Mpdv.Extensibility.Moc;
using Mpdv.Integration.Moc.Authorization;

namespace Ext.Extensions.Moc.ValueChanged
{
    [Mpdv.Extensibility.Extension(Name="FieldValueToUpper",Description="value to upper")]
    public class ToUpperCaseListener : IControlValueChangedListener
    {
        public void OnEditValueChanged(IControlContext context)
        {
            var value = context.AffectedControl.EditValue;
            if (value is String)
            {
                context.AffectedControl.EditValue = value.ToString().ToUpper();
            }
        }
    }
}
```

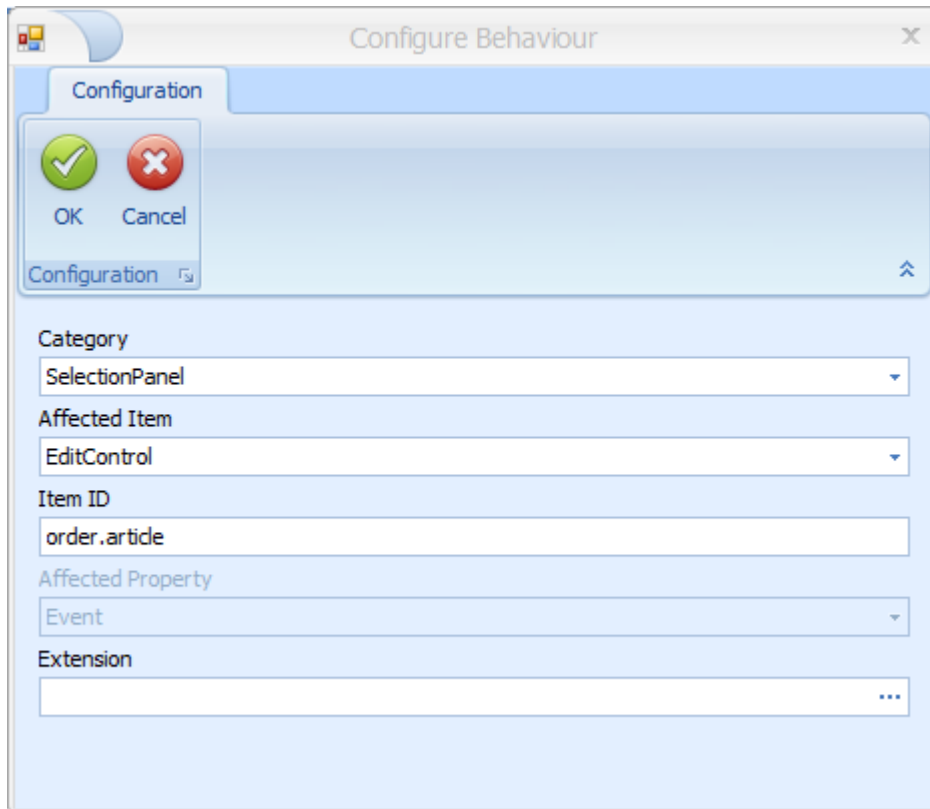


```
}
```

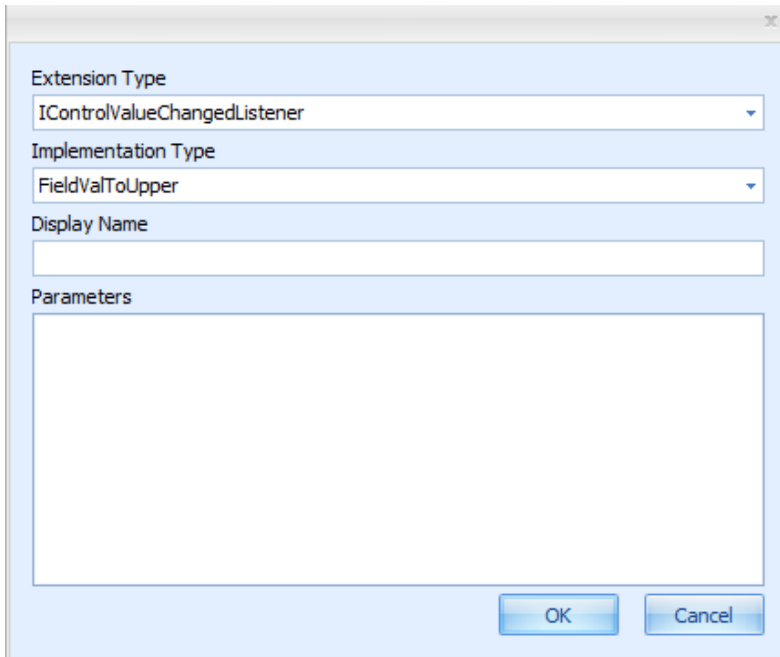
Compile into the "local\extension" directory of the MOC (ValueChanged.Extension.dll).

MOC Configuration

You want to convert the input field *Final article* (order.article) into capital letters.



The extension *FieldValToUpper* is now available as implementation type.



When you have saved the extension and restarted the application, you can test the behavior.

17.3.1.3.2 The user may only edit the own data record in the user administration

Assemblyname : MyApplicationContextFilter.extension.dll

Source code:

```
namespace Extension.MyApplicationContextFilter
{
    [Extension(Description = "True, if current equals selected user")]
    [ConfigParameter("User", typeof(string))]
    public class EnableButtonIfITsMe : IApplicationContextFilter, IConfigurable
    {
        private readonly ICurrentUserProvider currentUserProvider;
        private string userColumn;

        // constructor of class
        public EnableButtonIfITsMe(ICurrentUserProvider currentUserProvider)
        {
            this.currentUserProvider = currentUserProvider;
        }

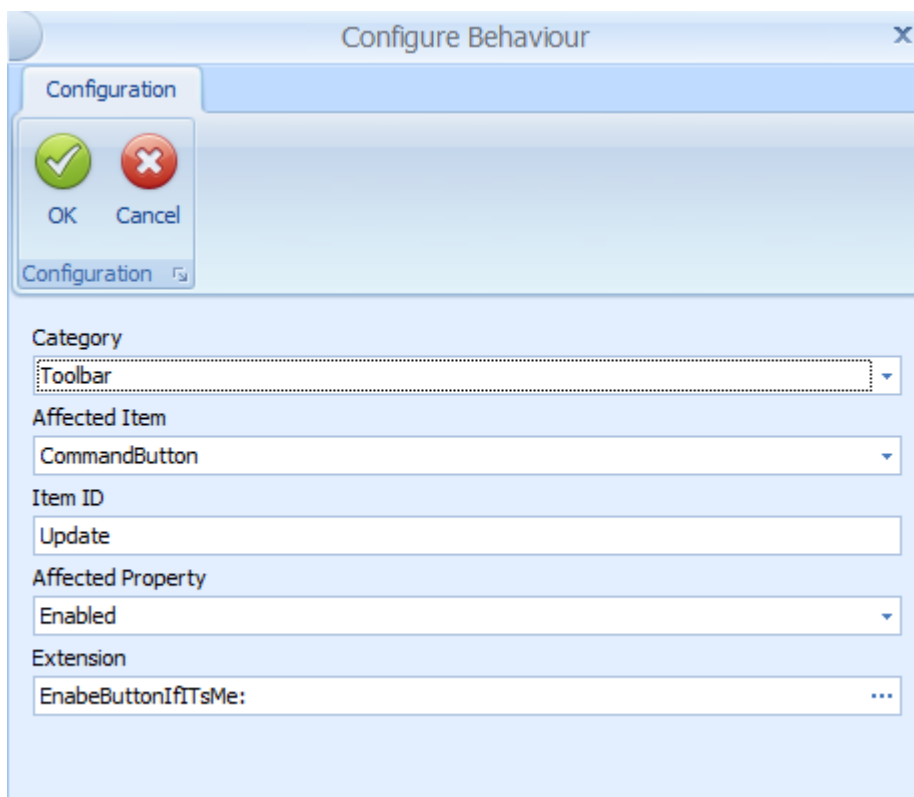
        // Read Parameter
        public void Configure(IConfigParameters parameters)
        {
            userColumn = parameters.GetParameter<string>("User").ToString();
        }

        // Implementation of IApplicationContextFilter method IsMatch
        public bool IsMatch(IApplicationContext context)
        {
            if (context.SelectedDataRows.Count <= 0) return false;
        }
    }
}
```

```
var selectedUser=context.SelectedDataRows[0][userColumn];  
if (selectedUser.Equals(currentUserProvider.GetUser().UserName))  
    return true;  
else  
    return false;  
}  
}
```

Configuration of dynamic behavior of the application in the user administration on the MOC

The *Update* button may only be enabled if the user logged on to the MOC wants to edit the own data record.



The new extension "EnableButtonIfItsMe" is now available.

Extension Type
IApplicationContextFilter

Implementation Type
EnableButtonIfItsMe

Display Name

Parameters

Sonstiges

User user.id

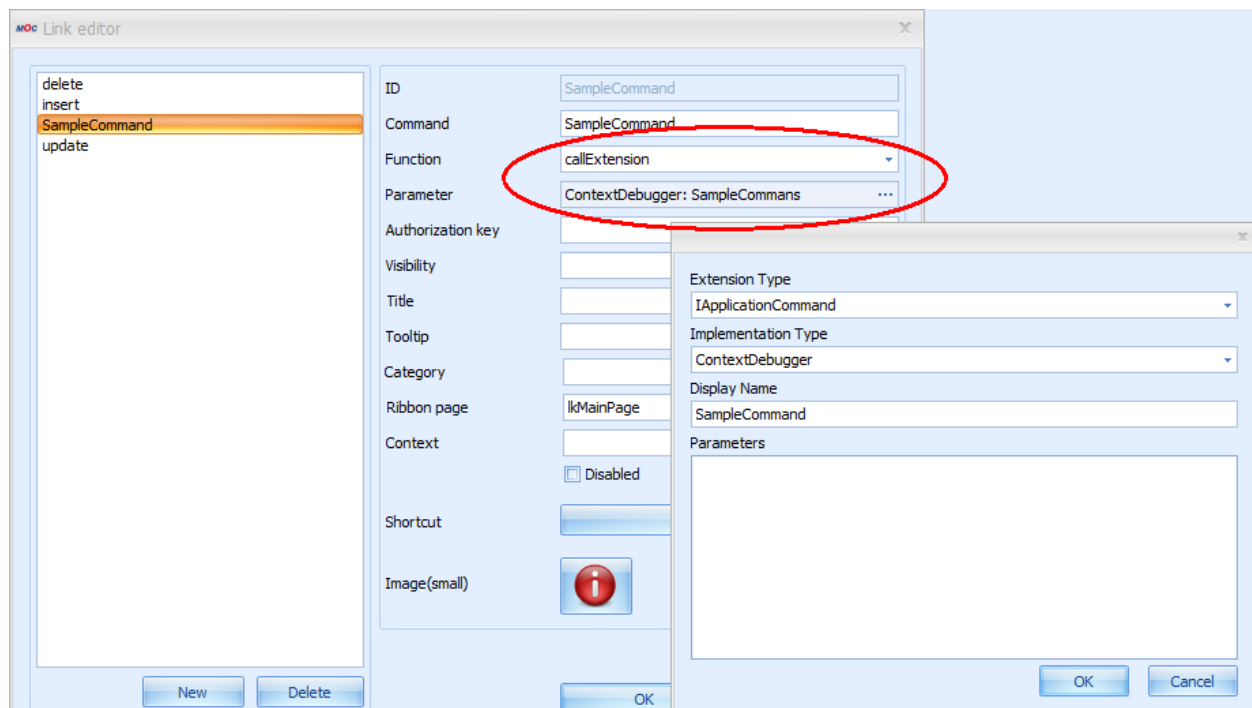
OK Cancel

17.4 Extensions of the toolbar

If you use extensions in the toolbar, you can quit the MOC to perform external actions and then return to the MOC.

17.4.1 Configuration

You configure extensions in the toolbar using the link editor. Select the function "callExtension" and configure an extension via parameter configuration.



17.4.2 Available standard extensions

17.4.2.1 ContextDebugger

The ContextDebugger is a sample implementation that shows the current application context in a window. The application context displayed can be helpful for the development of complex applications.

17.4.3 Creating your own extensions

You can create your own extensions using the .NET Framework (e.g. C#, VB.NET).

17.4.3.1 Requirements

The same requirements apply as for the extended application configuration (section 17.3.1.1 Requirements).

17.4.3.2 Creating an extension

To extend the toolbar, you must implement the interface *IApplicationCommand*.

The interface requires the method **"Execute"** where the external processing is performed. The object *IApplicationContext* includes the information required for the request context (e.g. selected data records).

The method "Execute" returns the object [ApplicationAction](#):

- The application is to be closed ([ApplicationAction.Close](#)).

- The application is to "Request data" (`ApplicationAction.RequestData`).
- Nothing is to happen (`ApplicationAction.None`).

You can inject additional objects in the constructor of the class that perform specific tasks (web services, localize, print/show report, logging). See also the API documentation. The API documentation is handed out as part of the training. You can also download the current version of the API documentation from the Support Portal.

17.4.3.3 Examples

The API documentation describes the different interfaces. Find two examples in the following:

17.4.3.3.1 HelloWorld

This example outputs a message box "HelloWorld". You can also add a parameter.

AssemblyName: HelloWorld.Extension

```
namespace Ext.Extensions.Moc.HelloWorld
{
    [Extension(Description = "Hello World"), ConfigParameter("val", typeof(String))]
    public class HelloWorldCommand : IApplicationCommand, IConfigurable
    {
        private string name = "";

        //constructor of class
        public HelloWorldCommand()
        {
        }

        /// <summary>
        /// Configures the instance.
        /// </summary>
        /// <param name="parameters">The parameters.</param>
        public void Configure(IConfigParameters parameters)
        {
            name = parameters.GetParameter<string>("val");
        }

        public ApplicationAction Execute(IApplicationContext applicationContext)
        {
            MessageBox.Show(String.Format("Hello {0}", name));

            return ApplicationAction.None;
        }
    }
}
```

MOC Configuration

ID

helloWorld

Command

helloWorld

Function

callExtension

Parameter

HelloWorldCommand: ...

Authorization key

Visibility

Title

helloWorld

Tooltip

helloWorld

Category

Ribbon page

MyButtons

Context

☐ Disabled

Shortcut

None

Image(small)

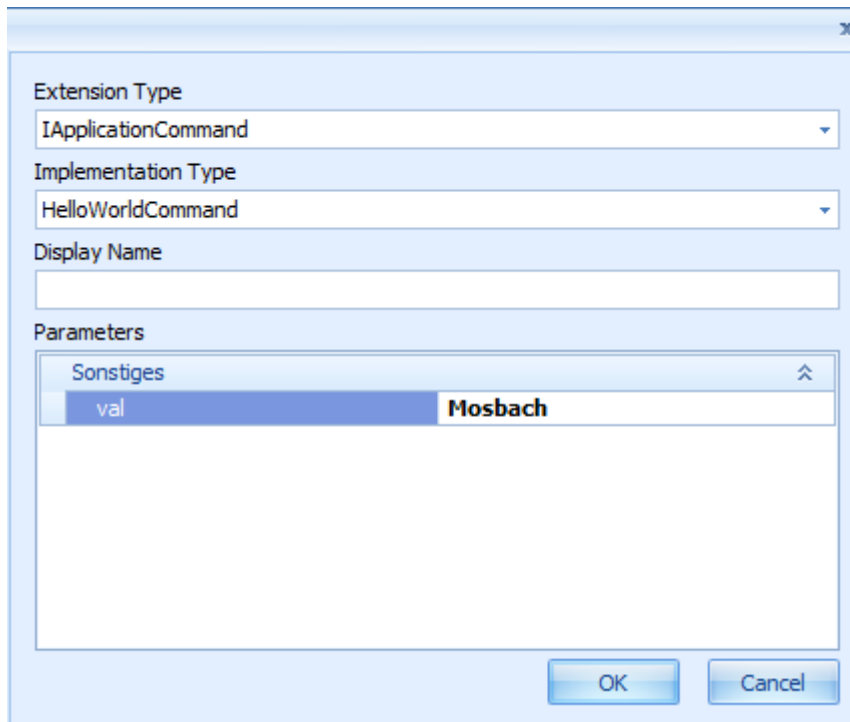


Image(large)



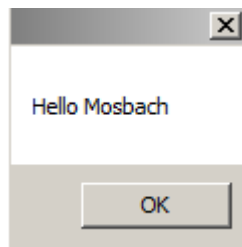
OK

Cancel



Parameters	
val	Mosbach

Result



17.4.3.3.2 Example including web service call

This example shows how you can call a web service. You can call a web service using the **IRequestFactory**. You can use all web services available.

AssemblyName: WebServiceExample.Extension

```
[Extension(Description = "WebService Example")]
public class WebServiceAndReportExample : IApplicationCommand
{
    private readonly IRequestFactory requestFactory;

    public WebServiceAndReportExample(IRequestFactory requestFactory)
    {
        this.requestFactory = requestFactory;
    }

    /// <summary>
```



```

/// Execute Method
/// </summary>
/// <param name="applicationContext"></param>
/// <returns>ApplicationAction.RequestData</returns>
public ApplicationAction Execute(IApplicationContext applicationContext)
{
    foreach (var r in applicationContext.SelectedDataRows)
    {
        string order = r["order.id"].ToString();

        // Update Userfield 29 of order
        var config = requestFactory.GetDefaultConfiguration("B00order.update");
        config.Parameters.SetOrAdd("order.id", Operator.EqualTo, order);

        config.Parameters.SetOrAdd("order.userfield29",
            Operator.EqualTo, "PRINTED");

        IAsyncRequest request = requestFactory.CreateAsyncRequest(config);

        try
        {
            request.GetResultAsync();
        }
        catch (Exception e)
        {
            logger.Trace(e.Message);
        }
    }

    return ApplicationAction.RequestData;
}

```

17.5 Creating programmed applications

Independent of the Application Framework, you can program own applications using the .NET Framework (e.g. C#, VB.NET, etc.). That means: You must program selection panel, DataController, docking, grids, charts, etc. yourself. The basic functions of the MOC are available in self-programmed applications (web services, reports, dictionary, system information, etc.).

You can use all functions and components of the .NET Framework. You can also integrate third-party components.

To this end, the interfaces **IMocApplication** (or **IParameterizedMocApplication**) and **IMocApplicationDescriptor** are available.

17.5.1 Definition of a new application

You can completely define the application in source code. You do not require further entries in other files.

17.5.1.1 IMocApplication

You use the interface "IMOCApplication" to define the own MOC application. Using this interface, you can start your own MOC application via the transaction code or via the menu. You cannot transfer parameters to the application. Use the interface "IParameterizedMocApplication" if you want to transfer parameters.

Method	Description
RunApplication(Form mdiParent)	Main function to start own application in MOC Parameter: mdiParent: The MDI parent to attach to. If the app should not open as a child this value can be null.
Constructor of own class	Possible Parameters: IReportHelper IRequestFactory ICurrentLocalizerProvider ICurrentUserProvider IAuthorizationManager IExtensionLogger

This interface has the single method RunApplication(Form mdiParent). In this method, you can instantiate and call a self-programmed form. The constructor of the class provides the basic functions/information of the MOC.

Example:

```
[Extension]
public class ExampleApplication : IMocApplication
{
    IReportHelper reportHelper = null;
    IRequestFactory requestFactory = null;
    ICurrentCultureProvider cultureProvider = null;
    ICurrentLocalizerProvider localizeProvider = null;
    ICurrentUserProvider currentUser = null;
    IAuthorizationManager authorisationManager = null;

    public ExampleApplication( IReportHelper reportHelper,
                             IRequestFactory requestFactory,
                             ICurrentCultureProvider cultureProvider,
                             ICurrentLocalizerProvider localizeProvider,
                             ICurrentUserProvider currentUser,
                             IAuthorizationManager authorisationManager
                             )
    {
        this.reportHelper = reportHelper;
        this.requestFactory=requestFactory;
        this.cultureProvider = cultureProvider;
        this.localizeProvider = localizeProvider;
    }
}
```

```

        this.currentUser = currentUser;
        this.authorisationManager = authorisationManager;
    }

    public void RunApplication(Form mdiParent)
    {
        var form = new Form1(reportHelper, requestFactory)
        {
            MdiParent = mdiParent
        };

        form.enableTimer(true);
        form.Show();
    }
}

```

17.5.1.2 IParameterizedMocApplication

Use the interface "IParameterizedMocApplication" to define the own MOC application if you want to add parameters to the application when it is called. Using this interface, you can start your own MOC application via the transaction code or via the menu. If you do not want to pass parameters, you can optionally use the interface "IMocApplication".



The start of your own MOC applications including parameters via the interface "IParameterizedMocApplication" is available on the MOC as of service pack 15 (fall 2019).



If you implement both interfaces („IMocApplication“ and „IParameterizedMocApplication“), your MOC application is compatible with the service pack 15 status and with older MOC installations. If the own C# application implements both interfaces, the C# application is called via the method RunApplication() of the interface "ParameterizedMocApplication".

Method	Description
void RunApplication([CanBeNull] Form mdiParent, [CanBeNull] string[] args)	Main function to start own application in MOC. Parameters: mdiParent: The MDI parent to attach to. If the app should not open as a child this value can be null. Args: Parameters that are passed to the application. Can be null, in case where no parameters are passed.
Constructor of own class	Possible Parameters: IReportHelper IRequestFactory ICurrentLocalizerProvider ICurrentUserProvider IAuthorizationManager

	IExtensionLogger
--	------------------

This interface has the single method `RunApplication(Form mdiParent, string[] args)`. In this method, you can instantiate and call a self-programmed form. The constructor of the class provides the basic functions/information of the MOC. The parameters of the request are passed in parameter "args".

The interface "IParameterizedMocApplication" is implemented like the interface "IMocApplication".

17.5.1.3 IMocApplicationDescriptor

The interface "IMocApplicationDescriptor" describes the properties of the MOC application (application ID, language keys, transaction codes).

Field name/Method	Description
ApplicationId	application identifier
ApplicationNameLanguageKey	application name language key
ApplicationDescriptionLanguageKey	application description language key
AuthorizationKey	authorization key
TransactionCode	transaction code
ApplicationExtensionType	type of the application extension. Type of class of the programmed application

Note: The authorization key must have been created for the current user. Otherwise, the application is not available.

Example:

```
[Extension]
public class ExampleApplicationDescriptor : IMocApplicationDescriptor
{
    public string ApplicationDescriptionLanguageKey
    {
        get { return "lkExample"; }
    }
}
```

```

    }

    Type IMocApplicationDescriptor.ApplicationExtensionType
    {
        get
        {
            return typeof(ExampleApplication);
        }
    }

    public string ApplicationId
    {
        get { return "Example"; }
    }

    string IMocApplicationDescriptor.ApplicationNameLanguageKey
    {
        get { return "lkExample"; }
    }

    string IMocApplicationDescriptor.AuthorizationKey
    {
        get { return "u_test"; }
    }

    string IMocApplicationDescriptor.TransactionCode
    {
        get { return "u_test"; }
    }
}

```

For further details, refer to the API documentation.

17.5.1.4 Storage location

The MOC directory includes the directory "applications" in each of the scopes. You must create a directory named ApplicationId in this directory. Compile the class library in this directory.

Example: ApplicationId = Example (scope Local)

<MOC directory>\local\applications\Example\Example.MocApplication.dll

17.5.2 Integration into the MOC menu

You can integrate programmed applications into the MOC menu using the menu editor. In the menu editor, enter the transaction code of your own MOC application as command.



If you have implemented the interface "IParameterizedMocApplication" in your own MOC application, then you can enter the parameters in the column "Parameter" of the menu editor that are passed to the application via the method "RunApplication" in parameter "args".

Requirements:

- The application must implement the interfaces of the Extensibility Framework.
- The user must be authorized for the authorization key of the application (function authorization).

17.5.3 Integration into the toolbar

You can call your own MOC application created using C# via a button in the toolbar.

Make the following settings in the link editor:

Command

You can use any text for the ApplicationCommandLinks, e.g. "callCommandObject".

Function

Fixed: callCommandObject

Parameters

"ApplicationId" of the implementation of the interface "IMocApplicationDescriptor". If you implement the interface "IParameterizedMocApplication", you can optionally pass further static parameters separated by blanks.



It is currently not possible to pass values from the calling application to the own MOC application called. The keys "DC" and "SP" are not dynamically resolved.

17.5.4 Calling "real MOC applications" from own applications



You can call "real MOC applications" from own MOC applications on the MOC as of service pack 15 (fall 2019).

The class "**MocCommandHelper**" is integrated in the Extension Framework, which implements the interface "**ICommandHelper**" and provides the relevant methods for the request of MOC applications. You can inject "MocCommandHelper" in a C# application if you specify the "ICommandHelper" interface in the constructor.

You can use the method "**OpenApplication()**" to call MOC applications that have the type MOC application. These are all applications that can be listed on the MOC using the transaction code "_cselectApp". You can also call own C# MOC applications. When you call the relevant variant, you can also pass static parameters to the MOC application or the C# application.

Use the method "**RetrieveApplicationData()**" to call an MOC application as pool application. In the call, you can optionally pass static parameters separated by blanks to the pool application. The data selected in "pool mode" is returned as result of the function call to the calling application.

```

using System.Collections.Generic;
namespace Mpdv.Integration.Moc.Execution
{
    /// <summary>
    /// Class for accessing functions of the command framework.
    /// </summary>
    public interface ICommandHelper
    {
        /// <summary>
        /// Opens the MOC application to the specified identifier.
        /// </summary>
        /// <param name="id">The identifier.</param>
        void OpenApplication(string id);

        /// <summary>
        /// Opens the MOC application to the specified identifier.
        /// </summary>
        /// <param name="id">The identifier of the application.</param>
        /// <param name="parameters">Parameters that are passed to the application (multiple parameters
        /// must be delimited with blank character).</param>
        void OpenApplication(string id, string parameters);

        /// <summary>
        /// Opens the MOC application to the specified identifier as pool application and returns the selected
        /// values of the application.
        /// </summary>
        /// <param name="id">The identifier of the application.</param>
        /// <param name="parameters">Parameters that are passed to the application (multiple parameters must be delimited with
        /// blank character).</param>
        /// <returns>The selected values as key/value pair where the key contains the name and the value contains
        /// the value of the field.</returns>
        IEnumerable<KeyValuePair<string, object>> RetrieveApplicationData(string id, string parameters);
    }
}

```

Example:

```

using System.Collections.Generic;
using System.Linq;
using System.Windows.Forms;
using Mpdv.Integration.Moc.Execution;

namespace MyParameterizedSampleMocApplication
{
    public partial class MainForm : Form
    {
        private readonly ICommandHelper _commandHelper;
        private string _arguments;

        public string Arguments
        {
            get => _arguments;
            set
            {
                _arguments = value;
                argumentsRichTextBox.Text = value;
            }
        }

        public MainForm(ICommandHelper commandHelper)
        {
            _commandHelper = commandHelper;
            InitializeComponent();
        }

        private void OpenApplicationButton_Click(object sender, System.EventArgs e)
        {
            if (string.IsNullOrEmpty(applicationParameterTextBox.Text))
            {
                _commandHelper.OpenApplication(applicationIdTextBox.Text);
                return;
            }

            _commandHelper.OpenApplication(applicationIdTextBox.Text, applicationParameterTextBox.Text);
        }

        private void RetrieveDataViaApplicationButton_Click(object sender, System.EventArgs e)
        {
            var retrieveDataViaApplication = _commandHelper.RetrieveApplicationData(applicationIdTextBox.Text,
            applicationParameterTextBox.Text);
            dataGridView1.DataSource = retrieveDataViaApplication?.
                Select(pair => new KeyValuePair<string, string>(pair.Key, pair.Value?.ToString())).ToList();
        }
    }
}

```

18 MOC application based on an SQL query (outdated)

18.1 Overview

Table views with simple database queries can be created with little time and effort as „Applications based on SQL Queries“. The design for this application is restricted as the function is outdated.



Do not create a new application with this technology.

Better create a service and an application based on this service.

Due to outdated technology and restrictions, this feature may not be available in a future version of the MOC.

18.2 Restrictions

Applications based on SQL queries have some restrictions compared to the usual application:

- You cannot use the grid configuration in the application because the properties of the columns are defined by the SQL result.
- You can only define column headings with an alias in the SQL statement by using the heading as an alias.

Example:

```
select masch_nr as Machine ID from machines;
```

- The services used in the other data sources offer security mechanisms, such as area of responsibility authorization. These are often not included in simple SQL statements.

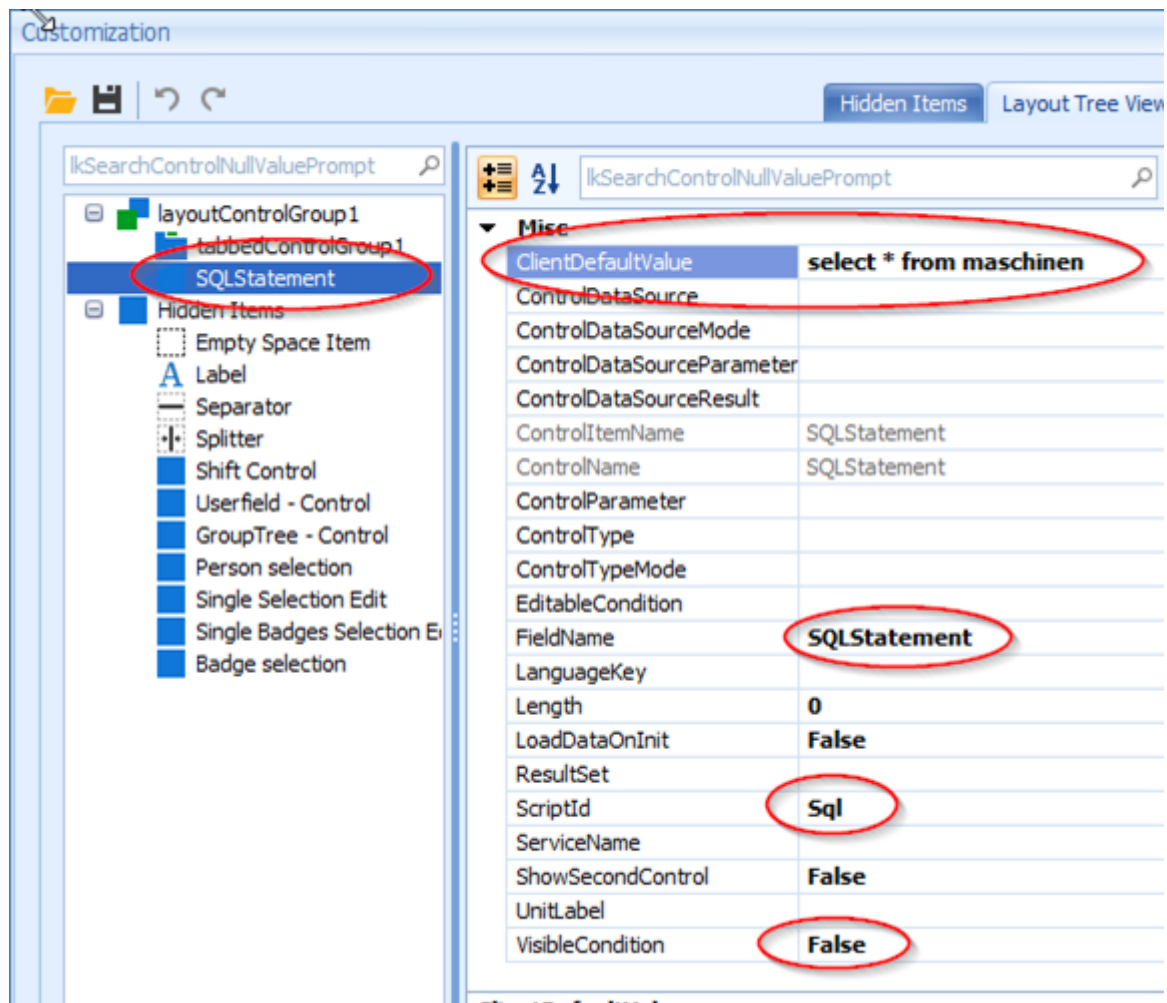
18.3 How to proceed

Create a new application using the menu point *New application*

Add a new data source with the data logic "SQL"

Create a new field on the selection panel with the following content:

- ClientDefaultValue: <SQL-Statement>e.g. *Select * from machines*
- FieldName: *Fix SQLStatement*
- Scriptid: *Fix Sql* (it is case sensitive)
- VisibleCondition: *False* (field is hidden)



Add a new detail application with the application type *SQL grid*. You may have to activate the switch *Show internal application types* in the configuration of the detail applications.

Dock the SQL grid below the selection area.

Configure data source: Link the SQL statement in the hidden input field to the data source.

Processing of parameter in the SQL statement

The SQL statement is entered in the hidden input field in the ClientDefaultValue field.

You can also process other fields from the selection area in the statement. Enter the field name with a placeholder in the SQL:

```
Select <fields> from <table> where <field> = '[<acronym_from_selection_panel>]'
```

Example (for field name = resource.group):

```
Select masch_nr, description  
      from machines  
where mgruppe = '[resource.group]'
```

19 MOC Development – "Cookbook"

19.1 Overview

You use the "Cookbook" for the development on the MOC. It explains many use cases step by step. Some of the use cases occur frequently, others rarely. The requirements and all steps of the solution are listed in short form. Frequent errors are mentioned so that you can easily avoid these errors. Also note the further information specified in each case.

19.2 Enable the MES Development Suite

Enable the MES Development Suite to make changes on the MOC.

Requirements

- MDS license
- Function authorization "mds"
- Select the correct scope

Solution: Steps to follow

- Go to the main menu "Extras" - "MES Development Suite" and enable the MES Development Suite.
- Reopen the application you want to configure.
- Make changes and save the changes made.

Restrictions

The available MDS license specifies the scope of changes you can make.

19.3 Formatting of durations as axis values in the chart

In the chart, the axis labeling shows numeric values. Actually, the values are durations.

Requirements

- Application including configured chart.

Solution: Steps to follow

1. Open the configuration file of the chart in a text editor.
2. Edit the value for "CustomFunction" and change it to "Duration".

```
<Setting Key="CustomFunction" Description="" LastChanged="2015-07-15T12:51:32.8967665Z" ValueType="System.String"
Version="0.0.0.0">
  <Value>
    <string>Duration</string>
  </Value>
</Setting>
```

3. Save the configuration file.
4. Restart the MOC.

Restrictions

The described solution only works with values that you can format with the output format `mpdv_timespan` e.g. in the grid.

Further information

The respective configuration file is stored in the application directory and uses the naming convention `[PluginId]Chart.config`.

19.4 Configuration of selection fields

You can store search applications or selection lists for a field in order to simplify the selection of entries. Depending on the use, three options are available for search applications and selection lists:

- Use **search applications** to select a data record from a large quantity of data, e.g. all orders. Search applications include filter criteria that help to narrow down the data.
- For small quantities of data, e.g. all order statuses, define a **dynamic selection list** including the request of a service. The static selection lists do not include filter criteria. These lists always show all data for selection.
- Define a **static selection list** with a `ReferenceData` if you only want to offer few specified values.

Requirements

- "MDS Development Suite" is enabled to configure applications.
- You have selected the correct scope to configure applications.
- You have created the field you want to configure in the detail application (see tutorial: "Adding and customizing fields in a detail application").
- The data you want to store (search application, data of selection list) must be available in the server or client.

Solution: Steps to follow

Configuration of a search application

Store a button for the field to open an application and select an entry from an existing application, e.g. open the order overview to transfer an order into the field.

1. Open the application you want to modify.

2. Right-click the layout of the detail application - "Customize layout"
3. Select the field you want to configure.
4. "ControlDataSource": Assign the application to be opened, e.g. OrderOverview.
5. "ControlDataSourceMode": Set Lookup
6. Select the parameters "ControlDataSourceParameter" that are transferred to the application to be opened, e.g. a specified order number via `order.id=12002500`
7. Set the "ControlDataSourceResult", e.g.
`order.id;order.id;;order.articledesignation;order.act.status`
 - o the internal (unique) ID, here `order.id`
 - o the value to be transferred into the field, here `order.id`
 - o the text to be placed behind the field, usually units like e.g. n hours, here empty
 - o further parameters that can be displayed in other fields of the application, here `order.articledesignation;order.act.status`
8. [Optional] "ControlParameter": Assign the DataLogic of the service that is to be called when the user directly types into this field and the field of the service where the entries are made, here e.g. `BOOrderList;order.id`
9. "ControlType": Assign `TextEdit`
10. Save the application by clicking the button "Save".

Note: If you want to use the field including search application at different places on the MOC, it might be useful to make the described configuration in the higher-level repository client, and not in the client.

Configuration of a dynamic selection list with data of a service

Store a selection list for a field to select a value from a limited number of database entries, e.g. all order statuses. The entries in the list are requested dynamically from the service.

1. Define the ControlDataSource in the repository.
 1. Create a new data record in the view "ControlDataSource".
 2. Select the domain and service (as DataLogic in the field Source), which provide the data, e.g. `MOrderStatus` and `MOrderStatsList`
 3. Define the "name" of the ControlDataSource. Then this name will be used in the client, e.g. `orderStatusList`
 4. Define the "columns" to request, e.g. `orderstatus.status;orderstatus.text`
 5. Define the "parameters" that limit the results, e.g. `orderstatus.statustype=A`
 6. Specify the "Result"
 - the internal (unique) ID, here `order.id`
 - the value to be transferred into the field, here `order.id`
 - the text to be placed behind the field, usually units like e.g. n hours, here empty
 - further parameters that can be used in the application, here `order.articledesignation;order.act.status`

7. Test deployment of the ControlDataSource (see tutorial "Deployment of customizations")
2. Open the application that you want to modify on the MOC.
3. Right-click the layout of the detail application - "Customize layout"
4. Select the field you want to configure.
 1. "ControlDataSource": Assign the name of the ControlDataSource defined in the repository.
 2. "ControlDataSourceMode": Set `Lookup`
 3. Set the "ControlDataSourceResult", if it is not identical to the data defined in the repository (syntax as described above).
 4. "ControlType": Set `ComboBoxEdit`
 5. "ControlTypeMode": Set e.g. `SingleEdit` for single selection or `Multiple` for multiple selection.
5. Save the application by clicking the button "Save".

Configuration of a static selection list with predefined selection

Store a predefined selection list in a field as `ReferenceData`, e.g. order types. Unlike the `ControlDataSource`, the `ReferenceData` is a predefined list of values that the user cannot change.

1. Define the `ReferenceData` in the repository.
 1. Create a new data record for each required selection option in the view "ReferenceData".
 2. Select the domain providing the data, e.g. `MDOOrderType`
 3. Define the "type" of the `ReferenceData`. This type will be used as `ReferenceData` name in the client, e.g. `checksendaheadtype`.
 4. Define the "db_key" as the actual value of the concrete data record, e.g. `N`.
 5. Define the "ref_data_key" as the combined value of "type" and "db_key", e.g. `checksendaheadtype:N`
 6. Define the default value ("is_default"), the designation ("designation") and the sort sequence ("sort_key")
 7. Test deployment of the `ReferenceData` (see tutorial "Deployment of customizations")
2. Open the application that you want to modify on the MOC.
3. Right-click the layout of the detail application - "Customize layout"
4. Select the field you want to configure.
 1. "ControlDataSource": Assign the name of the `ReferenceData` defined in the repository.
 2. "ControlDataSourceMode": Set `Reference`
 3. Set the "ControlDataSourceResult", if it is not identical to the data defined in the repository (syntax as described above).
 4. "ControlType": Set `ComboBoxEdit` or `RadioGroup`

5. "ControlTypeMode": Set e.g. `Single` for single selection or `Multiple` for multiple selection.
5. Save the application by clicking the button "Save".

Affected files

The changes described here affect the following files:

- In the relevant application directory `<MOC>\local\conf\MOC\Apps\`, the file `Layout<ID of the detail application>.config` of the respective detail application.
- In the MOC main directory, the file `<MOC>\custom\resources\data\controldatasources\<ID of the application>.ControlDataSources.xml` when defining `ControlDataSources`
- There is this file on the server `<InstallDir>\jdir\MOC\1\referenceData\local\<Service-Domain>.xml` if you configure `ReferenceData`

Possible problems

- In case of configuration errors, you can usually find the reason in the log files, e.g. typing errors in service names, etc.

Further information

- MDS-BAS_81: Customizing of main applications, section "Selection panel"
- MDS-BAS_81: Input and output fields (to assign field properties)
- MDS-BAS_81: Sections "ControlDataSource" and "ReferenceData"
- Tutorial: "Adding and customizing fields in a detail application"
- Tutorial: "Deployment of customizations"
- Tutorial: "Adding and customizing fields in a detail application"

19.5 Deployment of customizations

Further information

- CUT-MOC: Documentation of the training CUT-MOC: Deployment for artifacts of client and server.
- MDS-BAS_81: MOC Update Package Creator
- MDS-BAS_81: Update Packages for the Maintenance Manager

19.6 Generate a new application

Create a new application or create an application including editing applications (insert, update, delete, copy)

Requirements

- "MDS Development Suite" is enabled to configure applications.
- You have selected the correct scope to configure applications.

Solution: Steps to follow

Create new main application

1. Go to: MOC "MES Development Suite" - "New application"
2. Save the new application by clicking the button "Save".
 - Define a unique ID for the application, e.g. `U_myOrders`
 - Define a caption including language key, e.g. `lkU_myorders`
 - (Optional) Specify a help file and index, define the update rate to reload data and application icon.
3. Customize the application and define selection panel, data sources, detail applications, toolbar (see the corresponding tutorials).
4. Save the application by clicking the button "Save".

Creating an application with editing dialogs

1. Go to: MOC "MES Development Suite" - "Generate application"
2. Select the underlying DataLogic (usually of type list or overview)
3. Check the predefined fields for the editing dialogs you want to create
4. If necessary, define additional dialogs in the tab "Additional functions"
5. Generate the application by clicking "Generate".
6. Define the application properties.
 - Define a unique ID for the application, e.g. `U_myOrders`
 - Define a caption including language key, e.g. `lkU_myorders`
 - (Optional) Specify a help file and index, define the update rate to reload data and application icon.
7. Save the application by clicking the button "Save".

Affected files

If you generate an application, a new directory with the ID of the new application is created in the directory `<MOC>\local\conf\MOC\Apps\`. All files an application requires are stored here.

If you generate an application including editing dialogs, the application directory will include an additional directory for each editing dialog. All configuration files required for the respective editing application are stored in these directories.

Possible problems

- New applications are not automatically integrated into the menu. By default, you can only call the application using the transaction code `_capp id=<ID of the application>`. The tutorial "Create or edit the menu" describes how to integrate an application into the menu.
- You cannot add editing dialogs subsequently to an application. This is only possible using the application generator. You can directly change the configuration files if you only want to make minor changes to the editing applications, . After having changed the files, you must reload the configuration data (MOC menu "MES Development Suite" - "Reload configuration data").

Further information

- MDS-BAS_81: Customizing of main applications, section "Detail applications"
- MDS-BAS_81: Customizing of editing applications

19.7 Defining a conditional formatting in the grid

Configure a conditional presentation in the grid to simplify the display of information.

Requirements

- "MDS Development Suite" is enabled to configure applications.
- You have selected the correct scope.
- The detail application of type "grid" is configured.

Solution: Steps to follow

1. Open the application.
2. Right-click the grid column - "Configure conditional display".
3. "Add" a new configuration.
 - Configure the required display via "Appearance", e.g. red as "BackColor".
 - Configure the condition via "AppearanceDescription":
 - "Column": The condition is valid for the selected columns.
 - "Condition": Defines the type of condition to be checked, e.g. *Between*
 - "Value1" or "Value2" define the limit values, e.g. 100 as lower, 300 as upper limit.
4. Save the changed application by clicking the button "Save".

Refer to the documentation for detailed information on how to configure the conditions to be checked.

Affected files

The changes described here affect the following files in the respective application directory

<MOC>\local\conf\MOC\Apps\:

- The file Grid<ID of the grid detail application>.config as layout file of the detail application of type "grid".

Further information

- MDS-BAS_81: Components for detail applications, section "Table view (grid)"

19.8 Configuration of column totals in the grid

Configure totals per grid column or per grid column and group to simplify the display of information.

Requirements

- "MDS Development Suite" is enabled to configure applications.
- You have selected the correct scope.
- The detail application of type "grid" is configured.

Solution: Steps to follow

1. Open the application.
2. Right-click the grid column - "Grid properties"
3. Configure the property "SummaryItem"
 - SummaryType: Sum (most common), max, min, etc.
 - Special case: Use the following configuration with durations: Display format: {0:mpdv_timespan}; SummaryType: Custom; Tag: TimeSpanSum
4. Right-click the grid column - "Show overall column totals" to obtain a total of all data.
5. Or: Right-click the grid column - "Show column totals for group" to obtain a total per group.
6. Save the changed application by clicking the button "Save".
7. Reopen the application and request data.

Affected files

The changes described here affect the following files in the respective application directory

<MOC>\local\conf\MOC\Apps\:

- The file Grid<ID of the grid detail application>.config as layout file of the detail application of type "grid".

Further information

- MDS-BAS_81: Components for detail applications, section "Table view (grid)"

19.9 Grouping fields within a detail application

Group individual fields to improve the presentation within a detail application of type "layout".

Requirements

- "MDS Development Suite" is enabled to configure applications.
- You have selected the correct scope to configure applications.
- The fields you want to group are defined in the detail application of type "layout".

Solution: Steps to follow

Group fields in groups or tabs.

1. Open the application you want to modify.
2. Right-click the layout of the detail application - "Customize layout"
3. In the dialog "Customization", change to "Layout tree view".
4. In the tree view, select all fields that you want to group (e.g. via CTRL + click).
5. In the context menu, select "Group" or "Create tabbed group".
6. Define a language key for the newly created group in the field `CustomizationFormText` of the respective control.
7. Save the application by clicking the button "Save".

Remove grouping of fields

1. Open the application you want to modify.
2. Right-click the layout of the detail application - "Customize layout"
3. In the dialog "Customization", change to "Layout tree view".
4. Select the group or the tab you want to remove.
5. In the context menu, select "Clear grouping" or "Clear tabbed group".
6. Save the application by clicking the button "Save".

Affected files

The changes described here affect the following files in the respective application directory

<MOC>\local\conf\MOC\Apps\:

- The file `Layout<ID of the detail application>.config` if the detail application has been customized.

Further information

- MDS-BAS_81: Customizing of main applications, section "Selection panel"
- Tutorial: "Adding and customizing fields in a detail application"

19.10 Creating and modifying label texts

You can create or modify label texts for MOC applications to integrate customer-specific terminology or translations.

Requirements

- "MES Development Suite" is enabled.
- The MPDV custom dictionary `mpdvDictionaryCustomer.xlm` is available.

Solution: Steps to follow

Customize an existing language key.

1. Identify the language key you want to edit in the respective MOC application.
 - The key refers to the name of the application:
 - Open the dialog "Configure application" by clicking the button in the toolbar.
 - The field value of "caption" is e.g. `lkOrderOverview`
 - The key refers to the name or the tooltip of a button:
 - Open the "Link editor" by calling the context menu in the toolbar and clicking "Configuration".
 - The field value of "Title" or "Tooltip" is e.g. `lkInterrupt`
 - The key refers to a field label in the selection panel or in the detail application of type "layout":
 - Open the dialog "Customization" by right-clicking the field label and selecting "Customize layout".
 - Click and select the field while "Customization" dialog is open.
 - The field value of "LanguageKey" is e.g. `lkOrder`
2. Enter the new text for the identified language key in the custom dictionary in form of a new entry.
3. Create the language resource files by clicking "Create resource" in the Excel file and store the files in `<MOC directory>\local\resources\languages`
4. Restart the MOC.

Define a new language key

1. Create a new entry in the custom dictionary and assign a unique key, e.g. `lkU_inspection`
2. Check if a duplicate of the key exists by clicking "Check dup. keys" in the Excel file.

3. Create the language resource files by clicking "Create resource" in the Excel file and store the files in <MOC directory>\local\resources\languages. Refer to the documentation for further details on the languages you can select.
4. Enter the new language key in the MOC application.
5. Restart the MOC.

Affected files

The changes described here affect the following files:

- Language files in <MOC directory>\local\resources\languages

Restrictions

- You cannot assign language keys to user fields in the individual applications. This is only possible in the user field configuration.
- You must change the language keys of column headers of a grid in the grid configuration. Right-click the grid column and select "Grid properties". Change the property "LanguageKey" of the respective field in the configuration. If you want to change the language key for all identical fields, we recommend to make the configuration in the repository.

Possible problems

- Problem: The newly configured language key is not translated in the application, but `lKU_XXX` is displayed.
 - Proceed as follows: Make sure that the language key is spelled properly (case sensitive) and that the language files are exported to the correct directory. Then restart MOC.
- Problem: The language key is translated, but is displayed in the wrong language.
 - Proceed as follows: Make sure that the language selected in the MOC has been exported and that the file is stored in the correct directory.
 - Note: We distinguish between several language localizations, e.g. `de-AT` and `de-DE`. If no translation is available for the localization selected in the MOC, the system does not select an alternative localization, but displays the standard language (generally English).

Further information

- MDS-BAS_81: Multilingual texts in the MOC using language keys
- MDS-BAS_81: Activating the MES Development Suite
- MDS-BAS_81: Configuration settings and configuration levels

19.11 Logging and debugging

Further information

- CUT-MOC: Documentation of the training CUT-MOC: Information on the troubleshooting on the client and server.

19.12 Adding and customizing fields in a detail application

You can change fields in a detail application to improve the presentation of contents.

Requirements

- "MDS Development Suite" is enabled to configure applications.
- You have selected the correct scope to configure applications.
- The detail application that you want to change has been generated (by default, the selection panel is included in an application).
- The field you want to add must be available in the service, i.e. the field is an existing field or a customer-specific field that has been added to the service definition in the repository client.

Solution: Steps to follow

Add a field

1. Open the application you want to modify.
2. Right-click the layout of the detail application - "Add item".
3. Right-click the layout of the detail application - "Customize layout"
4. Select the newly created element (usually at the bottom of the detail application or the selection panel).
5. In the dialog "Customization", change to "Layout tree view".
6. Move the new field by drag and drop to the required position in the dialog "Customization" or directly in the application.
 - Change the width or height of a field by right-clicking the field in the list of the "Customization" dialog. Select "Size constraints" and "Free sizing".
 - Once you have adjusted the fields, set the "Size constraints" of all fields to "Lock size".
 - You can create an element at the bottom ("Create empty space item") to avoid that you unintentionally change the size of the last field.
7. Assign a "FieldName" in the dialog "Customization".
 - Use the service acronym to complete the configuration options automatically.
 - If necessary, you can define a context in form of a "ServiceName".
 - Or you must manually fill in all properties (detailed description in the documentation).
8. Save the application by clicking the button "Save".

9. If necessary, close and reopen the application to load the field configuration via service documentation.

Request field data from service

Note: If you proceed as follows, the name of the new field must match the name of the service field.

1. Entry in the data source
 10. Open the dialog "Configure data sources"
 11. Select the data source that should provide the requested data.
 12. Click "Configure".
 13. Click "Select columns".
 14. Select the column for which you have newly defined the field in the application.
2. Entry in the detail application
 10. Open the dialog "Configure detail application".
 11. Select the detail application that includes the newly defined field.
 12. Click "Configure".
 13. Click "Select columns".
 14. Select the column for which you have newly defined the field in the application.
3. Save the application by clicking the button "Save".

Edit an existing field

1. Open the application you want to modify.
2. Right-click the layout of the detail application - "Customize layout"
3. Select the field you want to modify.
4. Change field parameters in the dialog "Customization" (detailed description in the documentation).
5. Reposition the field in the application by drag and drop.
6. If required, edit the configuration of the data sources or the detail application (see above).
7. Save the application by clicking the button "Save".

Delete an existing field

1. Open the application you want to modify.
2. Right-click the field you want to delete and select "Delete item".
3. Save the application by clicking the button "Save".

Affected files

The changes described here affect the following files in the respective application directory

<MOC>\local\conf\MOC\Apps\:

- The file `Layout<ID of the detail application>.config` if the detail application has been customized.

Restrictions

You can only configure fields of detail applications of type "layout" and of the selection panel. If you want to change other types of detail applications like charts or table views, please refer to the respective tutorials.

Possible problems

Note: The mere configuration of a new field is usually not sufficient. In addition, you must edit linked data sources or you must make available the new field in the detail application (see above).

Further information

- MDS-BAS_81: Customizing of main applications, section "Selection panel"
- MDS-BAS_81: Input and output fields (to assign field properties)
- Tutorial "Configuration of data sources"
- Tutorial "Configuration of selection fields"

19.13 Configuration of data sources

You can create or modify label texts for MOC applications to integrate customer-specific terminology or translations.

Requirements

- "MDS Development Suite" is enabled to configure applications.
- You have selected the correct scope to configure applications.

Solution: Steps to follow

1. Open the application you want to modify.
2. Open the dialog "Configure data sources" by clicking the button.
3. Click the required button, e.g. "Add".
 1. Enter "ID" of the data source as unique key.
 2. Select the "Data logic". It defines the requested web service.
 3. "Events" specify when the data source is requested, e.g. line break in the grid (`SelectionChanged` of another data source) or when clicking the button "Request data" (`DataRequested`).
 4. The "Source" defines possible selection parameters, e.g. "Parameter layout" to transfer data from the selection panel.
 1. For the input from the selection panel, "Parameter" specifies the field name in the selection panel, i.e. which user input is used.
 2. "Target" specifies the service field to which the user input is forwarded.

3. "Default operator" defines the type of operator; if nothing is entered, the `EqualOperator` is used.
5. Click the button "Select columns" to define the columns requested from the server.
6. Confirm by clicking "OK".

Affected files

The changes described here affect the following files in the respective application directory

<MOC>\local\conf\MOC\Apps\:

- The file `EventLinkCollection.config` to map events within the application.
- The file `DataControllerCollection.config` which includes the configuration of data sources and the requested columns.

Possible problems

- Problem: You cannot delete the data source.
 - Proceed as follows: Before trying to delete the data source, make sure that the data source is not in use.
 - Note: Data sources are not only used in detail applications, they can also be referenced by other data sources to provide parameters or in an event.

Further information

- MDS-BAS_81: Customizing of main applications, section "Data sources"

19.14 Configuration of detail applications

Modify detail applications of an application to implement customer-specific layouts or other contents.

Requirements

- "MDS Development Suite" is enabled to configure applications.
- You have selected the correct scope to configure applications.
- You have configured the data source for the detail application (see tutorial "Configuration of data sources").

Solution: Steps to follow

Create a new detail application

1. Open the application you want to modify.
2. Add a new detail application.
3. Open the dialog "Configure detail application" by clicking the button.

4. Click "Add".
 1. Define a unique ID.
 2. Define a caption according to the schema `lkU_XXX`.
 3. Select the application category. The category specifies the design of the detail application. The most common designs are: layout, grid, chart (refer to the documentation for further information).
 4. Select the configured data source.
 5. In case of a layout application showing a detail view of a table: Enable the option "Show selected data records only".
 5. Click "Select columns" to define the available fields of the data source for the application.
 6. Create the new detail application by clicking "OK".
3. Position the new detail application within the application via docking (see tutorial "Customize application layout via docking")
4. Save the application by clicking the button "Save".

Edit existing detail applications

1. Open the application you want to modify.
2. Open the dialog "Configure detail application" by clicking the button.
3. Select the respective detail application.
4. Click "Configure" to edit the detail application.
 1. Edit the configuration (refer to the documentation for further information).
 2. Click "OK" to confirm your changes.
5. Save the application by clicking the button "Save".

Delete an existing detail application

1. Open the application you want to modify.
2. Undock the detail application by drag and drop.
3. Open the dialog "Configure detail application" by clicking the button.
4. Select the respective detail application.
5. Click "Remove" to delete the detail application.
6. Save the application by clicking the button "Save".

Affected files

The changes described here affect the following files in the respective application directory

<MOC>\local\conf\MOC\Apps\:

- The file `DataControllerCollection.config` which includes the configuration of the application's data sources.
- The file `ApplicationPluginCollection.config` which includes the configuration of the application's detail application.

- The file `Layout<ID of the detail application>.config` as layout file of the respective detail application.
- If required, the file `DockManagerCollection.config` which saves the docking of the new detail application.

Further information

- MDS-BAS_81: Customizing of main applications, section "Detail applications"
- Tutorial "Customize application layout via docking"

19.15 Adding and customizing fields in the selection panel

You can change fields in the selection panel to improve the presentation of contents.

Requirements

- "MDS Development Suite" is enabled to configure applications.
- You have selected the correct scope to configure applications.
- The field you want to add must be available in the service, i.e. the field is an existing field or a customer-specific field that has been added to the service definition in the repository client.

Solution: Steps to follow

Add a field

1. Open the application you want to modify.
2. Right-click the selection panel - "Add item".
3. Right-click the selection panel - "Customize layout".
4. Select the newly created element (usually at the bottom of the detail application or the selection panel).
5. In the dialog "Customization", change to "Layout tree view".
6. Move the new field by drag and drop and assign the properties (for details, refer to the tutorial "Adding and customizing fields in a detail application").
7. Save the application by clicking the button "Save".
8. If necessary, close and reopen the application to load the field configuration via service documentation.

Use entered values as filter parameters

1. Entry in the data source
 1. Open the dialog "Configure data sources"
 2. Select the data source you want to query when clicking "Request data".
 3. Click "Configure".
 4. In the group "Parameter", the source "Parameter layout" is selected.

5. In the grid, select the row with the values "Source" = `ParameterLayout` and "Parameter" = Name of the newly defined field.
6. In the column "Target" of this row: Enter the service parameter to which this parameter value is to be transferred.
2. Save the application by clicking the button "Save".

Edit an existing field

1. Open the application you want to modify.
2. Right-click the selection panel - "Customize layout".
3. Select the field you want to modify.
4. Change field parameters in the dialog "Customization" (detailed description in the documentation).
5. Reposition the field by drag and drop.
6. If required, edit the configuration of the data source (see above).
7. Save the application by clicking the button "Save".

Delete an existing field

1. Open the application you want to modify.
2. Right-click the field you want to delete and select "Delete item".
3. Save the application by clicking the button "Save".

Affected files

The changes described here affect the following files in the respective application directory

<MOC>\local\conf\MOC\Apps\:

- The file `LayoutPanel.config` which is the configuration file of the selection panel.

Possible problems

Note: The mere configuration of a new field is usually not sufficient. In addition, you must edit linked data sources (see above).

You cannot use all parameters that are available in a service as filter parameters in the selection panel. In the service definition of the repository, set the value `isFilterParameter` to `true`.

Further information

- MDS-BAS_81: Customizing of main applications, section "Selection panel"
- MDS-BAS_81: Input and output fields
- Tutorial "Configuration of data sources"
- Tutorial: "Adding and customizing fields in a detail application"

19.16 Configuration of a dependent data source

Configure a data source that refers to another data source to request data (master-detail relationship).

Requirements

- "MDS Development Suite" is enabled to configure applications.
- You have selected the correct scope to configure applications.
- The master data source has already been created (see tutorial "Configuration of data sources").

Solution: Steps to follow

Example "Order overview": Selection of all operations (detail data source) of an order (master data source). The order number (order.id) is used for mapping. All operations of the selected order are then displayed in the detail table.

1. Open the application you want to modify.
2. Open the dialog "Configure data sources" by clicking the button.
3. Click the required button, e.g. "Add".
 1. Enter "ID" of the data source as unique key.
 2. Select the "Data logic". It defines the requested web service.
 3. The "Events" specify when the data source is requested. Here, you must select `<master data source>: SelectionChanged`.
 4. As "Source" it might be useful to select the master data source including the corresponding parameter mapping.
 5. Click "OK" to confirm data input.
4. Save the application by clicking the button "Save".

Affected files

The changes described here affect the following files in the respective application directory

`<MOC>\local\conf\MOC\Apps\:`

- The file `EventLinkCollection.config` to map events within the application.
- The file `DataControllerCollection.config` which includes the configuration of data sources and the requested columns.

Further information

- MDS-BAS_81: Customizing of main applications, section "Data sources"
- Tutorial "Configuration of data sources"

19.17 Creating or editing the menu

You can create a custom menu to have quick access to important entries and integrate custom applications into the menu.

Requirements

- You have selected the correct scope.

Solution: Steps to follow

Create a custom menu

1. In the MOC, select "Extras" - "Menu editor".
2. Select the menu you want to edit in the drop-down list. Open this menu by clicking "Load".
3. We recommend not to overwrite the two standard menus, but to create a custom menu.
 1. Load the menu you want to edit.
 2. Define a name in the drop-down.
 3. Save
4. Edit the loaded menu (refer to the documentation for further information):
 - via buttons (add menu, delete entry),
 - edit text directly in the tree structure or
 - add new applications from the function list on the left by drag and drop.
 - Define parameters by clicking "Advanced settings".
5. Save the modified menu.
6. Select the menu via "Extras" - "Configuration" - "Select menu". The selected menu is filed in the user scope.

Add a custom application to the menu

1. Create a custom application (see tutorial "Generate applications")
2. Call the new application via transaction code `_capp id=<application name>` for test purposes.
3. Enter an authorization key if you want to avoid that any user can open the application.
 - Create a new entry in the file `<MOC directory>\local\resources\data\authorizations\LocalApps.Authorization.xml`.
 - AuthorizationId: Assign the ID of the application; AuthorizationKey: Assign an authorization key of max. 15 characters.
 - If the file or folder structure is not available, use the menu template of the training documentation to create it.
4. Enter a function authorization for the authorization key in order to assign authorizations to users.

- Create a new entry in the file <MOC directory>\local\resources\data\authorizations\FunctionAuthorisationMappings\functionauthorisation.txt.
 - The syntax is: <function authorization>;<authorization key>
 - If the file or folder structure is not available, use the menu template of the training documentation to create it.
- 5. Enter the application in the list of available applications.
 - Create a new entry in the file <MOC directory>\local\resources\data\functions\functions.xml.
 - Use the ID of the application as LinkId and the name as Label. As Parameter, enter the id=<application ID>. Optionally, you can define a transaction code.
 - If the file or folder structure is not available, use the menu template of the training documentation to create it.
- 6. Optional: Test the defined transaction code (an error message pops up because of missing authorization).
- 7. Optional: Assign the required authorization to the current user for test purposes.
 1. On the MOC: "System administration" - "User administration" - "Function authorizations"
 2. Add a new entry to the current user and assign the function authorization from
functionauthorisation.xml.
- 8. Restart the MOC.
- 9. Add the application to the menu, see "Creating a custom menu".

Affected files

Each menu is stored in a specific directory in <MOC directory>\local\conf\MOC\Menues. Each menu includes a main file that references the files of the individual groups of the menu. Each group again includes a subdirectory with one file per menu entry.

Possible problems

- Problem: You cannot select the new menu in "Extras" - "Configuration".
 - Proceed as follows: Make sure that you are in the correct scope (local or user).
- Problem: After having edited the XML files of the menu manually, the MOC does not start.
 - Proceed as follows: Make sure that the configuration files do not include inconsistencies, e.g. ensure that the indicated number of elements matches the actual number of existing entries.
 - Note: We generally recommend to use the menu editor to avoid corruption of the menu configuration.
- Problem: The menu editor does not show your custom application.

- Proceed as follows: Make sure that you have added the correct entries to all three files (authorization key, mapping to function authorization and you have made the application available for the menu). It is of special importance that the authorization keys of `LocalApps.Authorization.xml` and `functionauthorisation.xml` are identical.
- Tip: Assign a transaction code for test purposes and check, if you can call the application using this code (see above).
- Note: You must restart the MOC to enable the changes in these files.

Further information

- MDS-BAS_81: Customizing files for menus
- Tutorial "Generating applications"

19.18 Customizing application layout via docking

You can position the detail applications in an MOC application and thus create a customer- or user-specific application layout.

Requirements

- "MES Development Suite" is enabled.
- You have selected the correct scope to configure applications.

Solution: Steps to follow

1. Move the detail application by drag and drop to the required position.
2. Save the changed application layout by clicking the toolbar button "Save".

Affected files

The changes described here affect the following files:

- <MOC directory>\local\conf\MOC\Apps\<application name>\DockManagerCollection.config

Possible problems

- Problem: The application layout is misconfigured and you want to return to the original state. The application is still open.
 - Proceed as follows: Close the application without clicking the "Save" button. All changes since the last time the application was saved are discarded.
 - Note: We recommend to make a backup copy of the application directory before editing the layout.

Further information

- MDS-BAS_81: Customization of application layout
- MDS-BAS_81: Activating the MES Development Suite
- MDS-BAS_81: Configuration settings and configuration levels

19.19 Customizing symbols and images

You can replace symbols and images with customer-specific icons, start screens and logos.

Requirements

- You have selected the correct scope.

Solution: Steps to follow

1. Adjust size of the new graphic (refer to the documentation for details).
2. Save the graphic with an appropriate name (case sensitive; refer to the documentation for specifications).
3. File in the directory `<MOC directory>\local\resources\images`.
4. (Re)start the MOC.

Affected files

When customizing symbols and images, the original files are not overwritten. Only the files in the directory `<MOC directory>\local\resources\images` are modified or added.

Possible problems

- Problem: The graphic has been copied to the correct storage location but it is not loaded.
 - Proceed as follows: Make sure that the file name is properly spelled (case sensitive, file type) and that the file is stored in the local or user scope.

Further information

- MDS-BAS_81: Customization of symbols
- MDS-BAS_81: Configuration settings and configuration levels

19.20 Adding and customizing buttons in the toolbar

You can add a new button to the toolbar or configure an existing button.

Requirements

- "MES Development Suite" is enabled.
- You have selected the correct scope to configure applications.

Solution: Steps to follow

1. Open the "Link editor" by calling the context menu in the toolbar and clicking "Configuration".
2. Add a new `ApplicationCommandLink` by clicking "New".
 1. Assign any ID and command.
 2. Select function. Possible values: see documentation.
 3. Enter the parameters matching the function.
 4. Specify via the "Authorization key" if a button is enabled or grayed out.
 5. The authorization key entered in the field "Visibility" controls if a button is displayed or not.
 6. You must enter a language key in the field "Title". If the field "Tooltip" is empty, the value of "Title" is taken over when saving the changes.
 7. "Category" and "Ribbon page" specify the position of the button within the toolbar.
 8. "Disabled".
 9. The button "Shortcut" opens a dialog to define a shortcut that runs the `ApplicationCommandLink` when the application is active.
 10. If you check the option "Disabled", you can hide an existing button (of a higher scope). If you want to reactivate the button, you must do this in the configuration file.
 11. You can configure an image from the MOC resources for the button layout. It depends on the size of the toolbar and the number of buttons to be shown, if the small or the large image is displayed. You cannot influence or change this.
3. Confirm the changes by clicking "OK".
4. You must save the application configuration. Restart the application to enable the configuration.

Affected files

The button configuration is stored within the application configuration in the file `ApplicationCommandLinkCollection.config`. Each button requires separate configuration.

Restrictions

You cannot customize buttons of the groups "Data", "Customizing", "Settings" and "Help" as described above.

Further information

- MDS-BAS_81: Toolbar

- MDS-BAS_81: Activating the MES Development Suite
- MDS-BAS_81: Configuration settings and configuration levels
- MDS-BAS_81: Authorization

19.21 Configuring a detail application of type "chart"

You can configure a detail application as chart to visualize data in a graphic manner.

Requirements

- "MDS Development Suite" is enabled to configure applications.
- You have selected the correct scope.
- You have configured the data source providing data for the chart.

Solution: Steps to follow

1. Open the application you want to modify.
2. Open the dialog "Configure detail application" by clicking the button.
3. Create a new detail application by clicking "Add".
 - Define an ID that is unique in the application.
 - Assign a caption as language key, e.g. `lkU_myChart`.
 - Select the application category `Chart`.
 - Select the data source that provides the necessary data to fill the chart.
4. Select the columns you want to integrate in the application by clicking "Select columns".
5. Position the chart in the application via docking.
6. Save the changed application by clicking the button "Save".
7. Before configuring the chart, request data once.
8. Open the chart wizard by double-clicking the chart.
 1. Select the chart type and confirm by clicking "Next".
 2. Select the "appearance" and confirm by clicking "Next".
 3. Define the "series" to be displayed. Define one series for each data source field you want to display. Confirm by clicking "Next".
 4. Configure "Data":
 - Define the "Value" of the field to be shown for this series, e.g. yield of an order.
 - Define the "Argument" for the grouping of values, e.g. grouping of yield per person.
 5. [Optional] Define further layout properties of the chart.
 6. Confirm by clicking "Finish".
9. Save the application by clicking the button "Save".

Refer to the documentation for details on the configuration of special chart types.

Affected files

The changes described here affect the following files in the respective application directory <MOC>\local\conf\MOC\Apps\:

- The file `ApplicationPluginCollection.config` which includes the configuration of the application's detail application.
- The file `DockManagerCollection.config` which saves the docking of the new detail application of type "chart".
- The file `Chart<ID of the detail application>.config`: the configuration of the detail application.
- The file `DataControllerCollection.config` which includes the configuration of the application's data sources if the service requests further columns.

Further information

- MDS-BAS_81: Components for detail applications, section "Charts"
- Tutorial "Configuration of data sources"
- Tutorial: "Configuration of detail applications"

19.22 Configuring a detail application of type "table"

You can configure a detail application as grid to visualize data in tabular form.

Requirements

- "MDS Development Suite" is enabled to configure applications.
- You have selected the correct scope.
- You have configured the data source providing data for the grid.

Solution: Steps to follow

Adding a detail application of type "grid"

1. Open the application you want to modify.
2. Open the dialog "Configure detail application" by clicking the button.
3. Create a new detail application by clicking "Add".
 - Define an ID that is unique in the application.
 - Assign a caption as language key, e.g. `lkU_myGrid`
 - Select the application category `Tabellenansicht`
 - Select the data source that provides the necessary data to fill the table.
4. Select the columns you want to integrate in the application by clicking "Select columns".
5. Position the grid in the application via docking.

6. Save the changed application by clicking the button "Save".

Refer to the documentation for details on the configuration of the table grid.

Adding a column to the grid

If a new field is available in the service and you want to display this field in the detail application of type "table", proceed as follows:

1. Open the application you want to modify.
2. Request the new field of the service.
 1. Open the dialog "Configure data sources", select the data source defined for the grid.
 2. Click "Configure" to edit the data source.
 3. Open the dialog "Select columns" by clicking the button.
 4. Select the new field and confirm the changes.
3. Make the field available in the detail application.
 1. Open the dialog "Configure detail application", select the grid.
 2. Click "Configure" to edit the grid.
 3. Open the dialog "Select columns" by clicking the button.
 4. Select the new field and confirm the changes.
4. Add the column in the grid.
 1. Right-click the grid column, "Add column".
 2. Define the new column.
 - Name: define unique name.
 - Field name: Field name of the service, e.g. `order.id`
 - Caption: The header in the grid (defined by a language key).
 - Category: to group the columns.
5. Save the application by clicking the button "Save".

Affected files

The changes described here affect the following files in the respective application directory

<MOC>\local\conf\MOC\Apps\:

- The file `ApplicationPluginCollection.config` which includes the configuration of the application's detail application.
- The file `DockManagerCollection.config` which saves the docking of the new detail application of type "table".
- The file `Grid<ID of the detail application>.config`: the configuration of the detail application which is created when you add a detail application of type "table".
- The file `DataControllerCollection.config`: the configuration of the application's data sources if further columns are added or requested by the service.

Possible problems

- Problem: A wrong language key has been assigned to the added column. The language key must be changed.
 - Proceed as follows: Right-click the grid column - "Grid properties". Select the proper field in the drop-down list. Change the property "LanguageKey" to the required value. Save the application.
 - Note: You must restart the application to enable the changes.

Further information

- MDS-BAS_81: Components for detail applications, section "Table view (grid)"
- Tutorial "Configuration of data sources"
- Tutorial: "Configuration of detail applications"

19.23 Configuring a detail application of type "pivot"

You can configure a detail application of type "pivot" to visualize data in a pivot table.

Requirements

- "MDS Development Suite" is enabled to configure applications.
- You have selected the correct scope.
- You have configured the data source providing data for the pivot table.

Solution: Steps to follow

6. Open the application you want to modify.
7. Open the dialog "Configure detail application" by clicking the button.
8. Create a new detail application by clicking "Add".
 - Define an ID that is unique in the application.
 - Assign a caption as language key, e.g. `lkU_myChart`.
 - Select the application category `PivotTabelle`
 - Select the data source that provides the necessary data to fill the pivot table.
9. Click "Select columns" to select the columns you want to transfer into the application, i.e. for the fields that are available in the pivot table.
10. Position the pivot table in the application via docking.
11. Save the changed application by clicking the button "Save".
12. Configure the pivot table.
 - Right-click "Drag filter fields here" - "Show all fields" or "Show field list". Drag and drop the fields required for the pivot table.
 - Assign the fields to column fields and row fields to group the data by these fields.

- Assign the fields to data fields to display the data in the pivot table.
- Configure the chart added below by double-clicking the chart (see tutorial "Configure a detail application of type "chart").

13. Save the changed application by clicking the button "Save".

Refer to the documentation for details on the configuration possibilities.

Affected files

The changes described here affect the following files in the respective application directory

<MOC>\local\conf\MOC\Apps\:

- The file `ApplicationPluginCollection.config` which includes the configuration of the application's detail application.
- If required, the file `DockManagerCollection.config` which saves the docking of the new detail application of type "pivot".
- The file `Pivot<ID of the detail application>.config`: the configuration of the detail application.
- The file `DataControllerCollection.config` which includes the configuration of the application's data sources if the service requests further columns.

Further information

- MDS-BAS_81: Components for detail applications, section "Pivot"
- Tutorial "Configuration of data sources"
- Tutorial: "Configuration of detail applications"
- Tutorial: "Configuring a detail application of type 'chart'"

19.24 Protecting a field via authorization

You can protect a new field of an application and a service so that a missing server component cannot lead to an error in the client.

Requirements

- The field `a_acronym` has been defined in the application `A`
- The field `a_acronym` is available in the requested service of the domain `a_dom`.

Solution: Steps to follow

1. Use an authorization key to protect the field `a_acronym`.
 1. Create a new key in the repository client in the dialog "Authorization".
 2. Domain = `a_dom`

3. Authorization Id = a_acronym (the field you want to protect)
4. Authorization type = Acronym
5. Authorization key: select an individual key a_authkey
6. Export the data into the respective domain + runtime structure.
2. Define the mapping: authorization key - FeatureSet.
 1. In the domain MOC authorization (in products), edit the file %MOC authorization%\bin\resources\data\authorizations\FeatureSetMappings\featureset.txt
 2. Add the entry a_featureSet;a_authkey. Here: a_featureSet specifies the new FeatureSet that you want to define.
3. Create the FeatureSet.
 1. Check out the corresponding server domain.
 2. Create a new FeatureSet mapping in the directory %Service%\ConfigManager a_featureSetMappingFile.xml
 3. Define contents according to the documentation, e.g. check in the FeatureSet a_featureSet if a database patch is available. The field a_acronym that you want to protect is only activated, if the patch assigned in the FeatureSet has actually been executed.
 4. Edit the FeatureSet Excel file as list of all available FeatureSet mappings.
4. Deployment of the FeatureSet configuration on the server to %InstallDir%\JDIR\MOC\1\configManager\standard
5. Restart the Tomcat.

The client field is protected by the defined authorization key and only visible, if this key is enabled. The authorization key is enabled if the corresponding FeatureSet mapping is found and enabled. This is only the case if the customization has been imported into the server and the feature set is thus available.

Further information

- Documentation MDS-BAS

20 Customer-specific Database Contents

20.1 HYDRA SQL syntax

The system supports different database systems: Microsoft SQL Server and Oracle. These two databases do not always use the same SQL syntax and the data types required are partly different.

MPDV software therefore uses a special SQL syntax. The internal database interfaces convert the MPDV SQL syntax dynamically into the required syntax depending on the actually used database system.

With customer-specific solutions, the HYDRA SQL syntax is less important when you perform SQL queries and statements for Data Manipulation (DML) because these statements only have to work on the customer's database system. With customer-specific developments, you can therefore use the native SQL syntax of this database system.

But you must use the HYDRA SQL syntax for Data Definition (DDL) statements so that the objects created for the customer are compatible with all areas and tools of the MPDV software.

If you create objects like tables, columns, views, triggers, indices and functions, you must use the HYDRA SQL syntax. For this purpose, use an SQL client which converts the statements into the required SQL dialects.

For further information on the HYDRA SQL syntax, refer to one of the following sections.

Only the following command line tools are authorized SQL clients for the Data Definition Language (DDL) used to create and modify data objects:

- HYDRA SQL Interpreter `hysql.exe` (Windows) or `hysql.out` (Linux)
- HYDRA Script Interpreter `hydscr.exe` (Windows) or `hydscr.out` (Linux)

HYDRA SQL Interpreter can process SQL statements including DDL in an interactive prompt or it processes text files including SQL statements one after the other. The procedure is described in detail in the following sections.

MPDV uses HYDRA Script Interpreter to execute database patches, e.g. for product upgrades.

Use the defined SQL clients and respect the conventions mentioned in the following to ensure that the customer-specific data objects are compatible with all MPDV tools.



If you do not respect these rules, serious problems might be the result when you process data objects using MPDV tools, e.g. data inconsistency, loss of DB objects like views, triggers or indices, or even loss of data. This applies in particular with upcoming release upgrades, data transfers between different systems and required database reorganizations.

20.2 HYDRA SQL Interpreter hysql

The HYDRA SQL interpreter is a command line tool on the server. To launch it, you have to start the command line for the selected system number on a Windows server. On a Linux server you have to change to the required system number with hsys.scr.

Start as follows, if you want to execute text files including SQL statements:

Windows: `hysql -r statements.sql`

Linux: `hysql.out -r statements.sql`

Start as interactive SQL command line interpreter:

Windows: `hysql -r -`

Linux: `hysql.out -r -`

The parameter "-r" returns a result row after each SQL statement. This result row contains the SQL code, the number of affected data records and in case of SELECT statements the first selected row with columns separated by the pipe character "|".

The single minus sign as last parameter starts the interactive mode.

Find examples in the sections that follow.

20.3 Namespaces for customer-specific database objects

We have defined a separate namespace for customer-specific objects in order to avoid conflicts between MPDV standard objects (e.g. new features) and customer-specific objects. MPDV standard objects do not use the customer-specific namespace.

- Prefix all customer-specific objects in the database (tables, columns, views, indices,...) with "u_".
- Columns in a customer-specific table need not start with "u_" , as the table itself is located in the customer's namespace.
- Customer-specific columns are not allowed for the standard table. Better create a customer-specific table. Maintain this table in parallel to the standard table and join it for SELECT statements. If you cannot avoid to insert customer-specific columns in standard tables, the columns must have the prefix "u_".
- If necessary, MPDV also uses the customer-specific namespace for customizations to customer-specific tables. These tables are listed in the customer documentation.

20.4 Conventions for names in the DB (tables, columns, ...)

- Names for tables, columns, etc. can be made up of the Latin letters "a" to "z", the underscore "_" and the numbers "0" to "9". The name must start with a letter. Using special characters, umlauts or characters from other character sets is not allowed.
- In the HYDRA SQL syntax, write all identifiers (table name, column name, indices, views,...) in **lower case letters**.
- The identifiers' length should not exceed 30 characters so all MPDV tools can process the objects properly.**
- Do not use other data types** than the data types specified. If you use data types that are not implemented in the MPDV SQL clients, the system will probably react with undefined and incorrect behavior.
- Use the **customer's namespace** ("u_").

20.5 Supported data types

20.5.1 Overview

The supported data types are in some kind the "lowest common denominator". These data types are accepted in all supported database systems and implemented in all MPDV database clients.

The following table shows the data types used in HYDRA SQL and how these data types are implemented in the databases:

HYDRA SQL	Comment	Oracle	Sql Server
smallint	Integer ranging between -32768 and 32767. The value -32768 stands for an empty column (null).	NUMBER(37)	SMALLINT
integer	Integer ranging between -2147483648 und 2147483648. If the value is -2147483647, the column is empty (null).	NUMBER(22)	INTEGER
serial	See below	NUMBER(36)	INTEGER IDENTITY
decimal(m,n)	Decimal(18,6) is used by default.	NUMBER(m,n)	DECIMAL(m,n)
smallfloat	This data type is rarely used and should be avoided.	FLOAT(125)	smallfloat (→ REAL, see below)
float		FLOAT	FLOAT
float(n)	This data type is not used in the standard. Avoid this data type.	FLOAT(n)	FLOAT(n)

char or char(1)		CHAR (1)	CHAR (1)
char(n)		NVARCHAR2(n) with n > 4000 LONG	NVARCHAR(n) with n > 4000 TEXT
text	Storing large text fields. This data type is not used in the standard. Avoid this data type.	NCLOB	TEXT
date	Date without time	DATE	hydate (→ DATETIME, see below)
datetime	Timestamp including date and time.	TIMESTAMP(3)	DATETIME
image	Storing binary objects	BLOB	IMAGE

With a Microsoft SQL server, the following data types are created as user-defined data types in order to distinguish them:



hydate: EXEC sp_addtype date, datetime, 'NULL'

smallfloat: EXEC sp_addtype smallfloat, real, 'NULL'

The system automatically creates the user-defined data types when you create the database during the default installation process of the system.

20.5.2 Data type SERIAL

The data type SERIAL contains a numeric key that the database automatically assigns in ascending order. You use this data type, if a table does not contain a unique key.

ORACLE implements this data type by creating a sequence named *S_<tablename>*. This sequence provides the unique values. In addition, a trigger named *T_<tablename>* is created. If a data record is inserted into the table, this trigger reads the next value from the sequence and adds the value to the relevant column. The same trigger also guarantees that the value of the SERIAL column cannot be changed.



When you create a table including a SERIAL column, you must create a UNIQUE INDEX for this column.

20.6 Creating functions and triggers

To create functions and triggers, you also use an own syntax in HYDRA SQL for Oracle and Microsoft SQL Server because the native syntax can be quite different. MPDV therefore always lists the two statements for both SQL Server and Oracle in the corresponding SQL or HYDRA script files for functions and triggers of the standard. The clients HYDRA SQL Interpreter or HYDRA Script Interpreter execute only the relevant statement depending on the actually used database system.

You must query functions as qualified identifier preceded by database user "mipadm" (or "hydadm" preceding MW 4.0 pe) .

See also the following example:

20.7 Example

In the following example, a customer-specific table is created that includes columns with the most important data types. An index is created for the table. In addition, a function, a trigger and a view are created.

Further examples show how to write SQL statements for hysql to insert or change data in a table. The example shows how to write values for columns including data types like date, datetime and float or decimal in the statements.

20.7.1 Creating database objects

An SQL file is created in the subdirectory "db_sql" on the server:

```
create table u_machine_detail
(
  machine_nbr char(20),
  detail_text char(100),
  room_nbr integer,
  purchase_price float,
  last_maintenance_date date,
  last_maintenance_time integer,
  last_maintenance_ts datetime,
  modified_by char(10),
  modified_ts datetime,
  internal_id serial not null
);
revoke all on u_machine_detail from "public";

create unique index u_mdet_m on u_machine_detail (machine_nbr);
create unique index u_mdet_id on u_machine_detail (internal_id);

define function u_get_timestamp for oracle as
create function u_get_timestamp( p_d in date, p_n in number) return timestamp as
begin
  return cast((cast(p_d as timestamp) +
    case when (MONTHS_BETWEEN(cast(p_d as timestamp),sysdate)/12)>2000 and p_n=86400 then 0
      else p_n/86400 end) as timestamp) <EOS>
end;

define function u_get_timestamp for sqlserver as
CREATE FUNCTION u_get_timestamp( @p_d hydate, @p_n int ) RETURNS datetime AS
BEGIN
  RETURN @p_d + case when @p_d=convert(datetime, 2958463) and @p_n=86400 then 0 else ((@p_n + 0.001)/86400.0) end <EOS>
END;

define trigger u_t_machine_detail_mt_ts for oracle as
CREATE TRIGGER u_t_machine_detail_mt_ts BEFORE INSERT OR UPDATE ON u_machine_detail FOR EACH ROW
BEGIN
  :new.last_maintenance_ts := get_datetime(:new.last_maintenance_date, :new.last_maintenance_time)<EOS>
END;

define trigger u_t_machine_detail_mt_ts for sqlserver as
CREATE TRIGGER u_t_machine_detail_mt_ts ON u_machine_detail FOR INSERT, UPDATE AS IF (@@ROWCOUNT = 0) RETURN
BEGIN
  SET NOCOUNT ON
  UPDATE u_machine_detail
  SET last_maintenance_ts =
    u_machine_detail.last_maintenance_date + ((u_machine_detail.last_maintenance_time + 0.001)/86400.0)
  FROM inserted
  WHERE u_machine_detail.internal_id = inserted.internal_id
  SET NOCOUNT OFF
END;

create view u_v_machine_detail_only_ts as
select machine_nbr,
  detail_text,
  room_nbr,
  purchase_price,
  hydadm.u_get_timestamp( last_maintenance_date, last_maintenance_time ) as mt_ts,
  modified_by,
  modified_ts,
  internal_id
  from u_machine_detail;
```


The SQL file is then started from the command line for the relevant system using the HYDRA SQL Interpreter hysql. The following example shows the query from a Windows server. The grayed out sections are keyboard inputs. With a Linux server, the command is "hysql.out -r db_sql/u_table_example.sql".

```

hydadm:3:F:\hydra3>hysql -r db_sql\u_table_example.sql

01.03.2017 08:56:39 PROCESSING db_sql\u_table_example.sql...

create table u_machine_detail ( machine_nbr char(20), detail_text char(100),
    room_nbr integer, purchase_price float, last_maintenance_date date, las
t_maintenance_time integer, last_maintenance_ts datetime, modified_by char(1
0), modified_ts datetime, internal_id serial not null );
OK. NR OF ROWS 0.
RESULT:
|0|0|0|0|

revoke all on u_machine_detail from "public";
OK. NR OF ROWS 0.
RESULT:
|0|0|0|0|

create unique index u_mdet_m on u_machine_detail (machine_nbr);
OK. NR OF ROWS 0.
RESULT:
|0|0|0|0|

create unique index u_mdet_id on u_machine_detail (internal_id);
OK. NR OF ROWS 0.
RESULT:
|0|0|0|0|

define function u_get_timestamp for oracle as create function u_get_timestamp( p
_d in date, p_n in number) return timestamp as begin return cast((cast(
p_d as timestamp) + case when (MONTHS_BETWEEN(cast(p_d as timestamp),s
ysdate)/12)>2000 and p_n=86400 then 0 else p_n/86400 end) as time
stamp) <EOS> end;
OK. NR OF ROWS 0.
RESULT:
|0|0|0|-1|

define function u_get_timestamp for sqlserver as CREATE FUNCTION u_get_timestamp
( @p_d hydate, @p_n int ) RETURNS datetime AS BEGIN RETURN @p_d + case when @p
_d=convert(datetime, 2958463) and @p_n=86400 then 0 else ((@p_n + 0.001)/86400.0
) end <EOS> END;
OK. NR OF ROWS 0.
RESULT:
|0|0|0|-1|

define trigger u_t_machine_detail_mt_ts for oracle as CREATE TRIGGER u_t_machine
_detail_mt_ts BEFORE INSERT OR UPDATE ON u_machine_detail FOR EACH ROW BEGI
N :new.last_maintenance_ts := get_datetime(:new.last_maintenance_date, :new.l
ast_maintenance_time)<EOS> END;
OK. NR OF ROWS 0.
RESULT:
|0|0|0|-1|

define trigger u_t_machine_detail_mt_ts for sqlserver as CREATE TRIGGER u_t_mach
ine_detail_mt_ts ON u_machine_detail FOR INSERT, UPDATE AS IF (@@ROWCOUNT = 0)
RETURN BEGIN SET NOCOUNT ON UPDATE u_machine_detail SET las
t_maintenance_ts = u_machine_detail.last_maintenance_date + ((u_machin
e_detail.last_maintenance_time + 0.001)/86400.0) FROM inserted WHERE u
_machine_detail.internal_id = inserted.internal_id SET NOCOUNT OFF END;
OK. NR OF ROWS 0.
RESULT:
|0|0|0|-1|

```

```
create view u_v_machine_detail_only_ts as select machine_nbr, detail_text, room_nbr, purchase_price, hydadm.u_get_timestamp( last_maintenance_date, last_maintenance_time ) as mt_ts, modified_by, modified_ts, internal_id from u_machine_detail;
OK. NR OF ROWS 0.
RESULT:
|0|0|0|0|
hydadm:3:F:\hydra3>
```

20.7.2 Inserting data in a table

You can also start the HYDRA SQL Interpreter in the interactive input mode. This is useful with small SQL statements or with statements inserted via clipboard. The example also shows the output of result data with SELECT statements. Start the interactive mode using the command "hysql -r -" (Windows) or "hysql.out -r -" (Linux). (Note: the minus sign at the end!) End the interactive mode using the command "exit;".

The grayed out sections are keyboard inputs.

```
hydadm:3:F:\hydra3>hysql -r -
02.03.2017 08:40:23 PROCESSING STDIN...
SQL> insert into u_machine_detail
(
  machine_nbr,
  detail_text,
  room_nbr,
  purchase_price,
  last_maintenance_date,
  last_maintenance_time,
  modified_by,
  modified_ts
)
values
(
  '00000100',
  'Detail text for machine 00000100',
  1020,
  125000.000,
  '12/31/2016',
  14.5*3600,
  '12345',
  '12/31/2016 14:25:17.123'
);
insert into u_machine_detail ( machine_nbr, detail_text, room_nbr, purchase_price, last_maintenance_date, last_maintenance_time, modified_by, modified_ts ) values ( '00000100', 'Detail text for machine 00000100', 1020, 125000.000, '12/31/2016', 14.5*3600, '12345', '12/31/2016 14:25:17.123' );
OK. NR OF ROWS 1.
RESULT:
|0|5|1|0|
SQL>
```

The example shows:

- You can write strings in single or in double quotes.
- The dot is the decimal separator for float and decimal.
- You store time of the type *Integer* on the database tables. The content refers to "seconds since midnight". You must therefore multiply time by 3600. In the example above, the time is 14:30. Instead of entering "14.5*3600", you can directly enter 52200.
- Dates are in the 'MM/DD/YYYY' format.
- Timestamps for columns of the type datetime are in the 'MM/DD/YYYY hh:mm:ss.ccc' format. 'ccc' are milliseconds.
- A continuous number is automatically assigned to columns of the type serial, if data is inserted. The columns are not indicated in the statement.
- The created trigger automatically assigns a timestamp to the column last_maintenance_ts which is calculated from last_maintenance_date and last_maintenance_time.

20.7.3 Changing data in a table

The grayed out sections are keyboard inputs.

```
hydadm:3:F:\hydra3>mysql -r -

02.03.2017 08:40:23 PROCESSING STDIN...

SQL> update u_machine_detail set
  detail_text = 'Test machine for training',
  room_nbr = 1021,
  purchase_price = 27235.50,
  last_maintenance_date = '02/28/2017',
  last_maintenance_time = 21600,
  modified_by = 'trainee',
  modified_ts = '03/02/2017 08:54:30.468'
where machine_nbr = '00000100';
update u_machine_detail set detail_text = 'Test machine for training', room_
nbr = 1021, purchase_price = 27235.50, last_maintenance_date = '02/28/2017',
last_maintenance_time = 21600, modified_by = 'trainee', modified_ts = '03
/02/2017 08:54:30.468' where machine_nbr = '00000100';
OK. NR OF ROWS 1.
RESULT:
|0|0|1|0|

SQL>
```

Also on changing data, the created trigger automatically assigns a timestamp to the column last_maintenance_ts which is calculated from last_maintenance_date and last_maintenance_time.

You cannot change columns of the type serial. The columns can only be used as key in the WHERE clause.

20.7.4 Selecting data from a table

The grayed out sections are keyboard inputs.

```
hydadm:3:F:\hydra3>hysql -r -

01.03.2017 09:04:07 PROCESSING STDIN...

SQL> select personalnummer, name, kostenstelle, eintritt from personen where personalnummer =
40256;
select personalnummer, name, kostenstelle, eintritt from personen where personal
nummer = 40256;
OK. NR OF ROWS 1.
RESULT:
|0|0|1|0|2|4|40256|0|83|Pernikova, Lisa|0|10|105|7|10|10/09/1993|

SQL> exit;
EXIT FOUND.

hydadm:3:F:\hydra3>
```

Example for an SQL command including an error:

```
hydadm:3:F:\hydra3>hysql -r -

01.03.2017 09:11:53 PROCESSING STDIN...

SQL> select error from error where error = 'TRUE';
select error from error where error = 'TRUE';
^
ERROR -208, CISAM 0, OFFSET 0: [42S02][208][Microsoft][ODBC SQL Server Dri
ver][SQL Server]Unknown object name 'error'.
RESULT:
|-208|0|0|0|

SQL> exit;
EXIT FOUND.

hydadm:3:F:\hydra3>
```

The database system provides the error message text (e.g. MS SQL server or Oracle) and is not within the control of MPDV.

20.8 HYDRA SQL syntax reference

20.8.1 Maximum length of SQL statements

The maximum length of SQL statements is 32000 characters. Longer statements are automatically cut.
Note: necessary implementations for individual databases can increase the length.

20.8.2 Name of database objects

Do not use the following names for tables because these names have a special meaning under ORACLE: evt_*, sm*_s, sm*_x, sm*links, smc*, smp_*. In addition, the following names are not allowed for views: sm*_v, smp_*

20.8.2.1 Data type IMAGE

The system support the data type IMAGE (SQL server: IMAGE, Oracle: BLOB) for the following application cases:

- Creation of customer-specific tables via hysql that include data type BLOB
- Export and import of tables including BLOB data

Requirements:

- Oracle: as of MW 3.0
- SQL Server: as of MW 3.0, hyaccsql.dll version 8.1.2.10

Create tables with column type IMAGE

Requirements

Tables including one or more IMAGE columns must have a SERIAL column named **VERWEIS** (= "reference") (because of Oracle database).

Implementation

Use the tool hysql.out|exe if you want to create tables including the HYDRA data type IMAGE. The IMAGE column has data type IMAGE without any indication of size.

Example:

```
create table u_document
(
    verweis serial not null,
    ...
    opticalfingerprint image,
    ...
);
revoke all on u_document from "public";

create unique index u_document_vw on u_document (verweis);
```

Export

Store the schema of tables including IMAGE columns in the export SQL file as described in the example of section 3.1.

For each IMAGE field (per row/per column), an own IMAGE file is stored in the subdirectory <table name>, if the database field contains an IMAGE information (size > 0). The reference to the relevant file is stored in the UNLOAD file <TABLE NAME>.UNL.

All IMAGE files of a table are stored in the sub directory <TABLE NAME>.

Use the following rule to name IMAGE files:

<TABLE NAME>/<COLUMN NAME>_<current number per table>.IMG

Notes:

- Separator with WINDOWS systems is the backslash "\".
- The "current number per table" starts at 1 and is issued with 10 digits and leading zeros.
- The files have the extension: **IMG**
- All database fields in the relevant UNL file are populated with the references to the possible IMAGE files.
- Only IMAGE files that contain IMAGE information are exported to IMAGE fields.

Export example (hyexport.exe blob):

```
File blob.sql:

...
create table u_document
(
    verweis serial not null,
    charge_id char(20),
    documentorderid char(20),
    run char(12),
    sex char(1),
    countrycode char(3),
    nationalitycode char(3),
    documentserial char(9),
    dateofbirth datetime,
    dateofissue datetime,
    dateofexpiry datetime,
    faceimage image,
    signatureimage image,
    destinationoffice char(3),
    surname char(40),
    secondsurname char(40),
    givennames char(35),
    countrynationality char(12),
    nationality char(22),
    placeofbirth char(22),
    opticalfingerprint image,
    applicationnumber integer,
    civilbirthregistry char(30),
    diasbilityreg char(1),
    serialnumber char(10),
    residencetype char(10),
    visa char(20),
    profession char(30),
    mainfingerprtmin image,
    secfingerprtminu image,
    facialrecpattern image,
    authority char(50)
);
revoke all on u_document from "public";
```

```
create index u_document_cid on u_document (charge_id);
create unique index u_document_vw on u_document (verweis);
...
```

File `u_document.unl`

```
$COLUMN$VERWEIS|...| FACEIMAGE| SIGNATUREIMAGE|...|
24055|...| faceimage_0000000001.img| signatureimage_0000000002.img|...|
24056|...| faceimage_0000000003.img| signatureimage_0000000004.img|...|
.....
```

Storage of the **IMAGE** files

```
<HYDRADIR>
  → blob.exp [directory]
    → u_document [directory]
      faceimage 0000000001.img
      faceimage 0000000003.img
      signatureimage_0000000002.img
      ...
```

Import

If you import tables including IMAGE columns, create the tables as described in section 3.1. Perform the internal processing as described in the following:

Oracle:

- On reading the UNL files, all columns are read that are not of database type IMAGE and inserted via "INSERT INTO <TABLE>".
- The IMAGE columns are initialized using the ORACLE function **EMPTY_BLOB()**.
- Each IMAGE file (if existing) is read and inserted into the respective column.

SQL Server:

- Each IMAGE file is read before the "INSERT INTO <TABLE>" and is directly bound to the SQL statement.

20.8.2.2 Data type TEXT

The columns of HYDRA data type **char** and a length of more than 1999 characters are stored under ORACLE as data type NCLOB. The HYDRA data type is **TEXT**. You can store data up to 4Gbytes in this data type.

Restrictions / Conditions

This data type does not support the following SQL operations:

SQL Operations Not Supported	Example of unsupported usage
SELECT DISTINCT	SELECT DISTINCT clobCol from...
SELECT clause ORDER BY	SELECT... ORDER BY clobCol
SELECT clause GROUP BY	SELECT avg(num) FROM... GROUP BY clobCol
UNION, INTERSECT, MINUS (Note that UNION ALL works for LOBs.)	SELECT clobCol1 from tab1 UNION SELECT clobCol2 from tab2;
Join queries	SELECT... FROM... WHERE tab1.clobCol = tab2.clobCol
Index columns	CREATE INDEX clobIndx ON tab(clobCol)...

20.8.3 Strings

20.8.3.1 Constant strings

Use single or double quotes to limit constant strings in HYDRA SQL. ORACLE and SQL Server automatically replace double quotes by single quotes.

20.8.3.2 Substrings

Substrings with SELECT

Following the example of the previously available database INFORMIX, you can select string parts with HYDRA SQL using square brackets. Separate the first and the last digit by comma within the square brackets. ORACLE and SQL Server use the functions SUBSTR or SUBSTRING to which the first digit and the length of the result string are transferred.

HYDRA SQL: `select column_name[3,4] from table_name;`
 ORACLE: `select substr( column_name,3,2) from table_name;`
 SQL Server: `select substring(column_name,3,2) from table_name;`

Substrings with UPDATE

Also with UPDATE, you can access string parts with HYDRA SQL. The automatic implementation with ORACLE and SQL Server is as follows:

HYDRA SQL: `update table_name set column_name[3,4] = "ab";`
 ORACLE: `update table_name set column_name =
 substr(column_name, 1, 2) || 'ab' || substr(column_name, 5);`
 SQL Server: `update table_name set column_name =
 substring(column_name, 1, 2) + 'ab' + substring(column_name, 5, 2000);`

With SQL Server, you must always specify a length with the function SUBSTRING. But as on implementing the statement the field length is not known, a length of 2000 is assumed to add the rest of the field.

20.8.3.3 Concatenation of strings

You concatenate strings in HYDRA SQL using the operator '||'. With the SQL Server, this operator is automatically replaced with '+':

HYDRA SQL: `select "string" || column || "string" from tablename;`

SQL Server: `select 'string' + column + 'string' from tablename;`

20.8.4 Transactions

Transactions are explicitly started with HYDRA SQL. ORACLE automatically starts a new transaction once a transaction has been finished using *commit*. If there is no active transaction, ORACLE automatically performs a *commit* after each data change:

HYDRA SQL: `begin work;`

SQL Server: `begin transaction;`

and

HYDRA SQL: `commit [work];`

SQL Server: `commit transaction;`

and

HYDRA SQL: `rollback [work];`

SQL Server: `rollback transaction;`

In case of an SQL server with default settings, a session processes a data record in a transaction and another session must wait until the transaction is finished before it has **read** access. And vice versa, **also the read access (open cursor) locks the data and the data cannot be updated by another user.**

If you use the command "**set transaction isolation level read uncommitted**" with SQL server, the second session does not wait but reads the changed (possibly not consistent) data that has not yet been "committed". This is different with ORACLE. Here, the (consistent) data is issued before the beginning of the transaction. **You nevertheless use this isolation level with SQL Server as otherwise there might be time delays and deadlocks.**

SQL Server processes nested transactions. As this is not possible with ORACLE, HYDRA SQL declines a *begin work* in a current transaction and creates an SQL error.

20.8.5 Current date, current time

HYDRA SQL uses *today* and *current* to query the current date and the current time. SQL Server offers the function *getdate()*, which returns date and time just like *sysdate* with ORACLE. When implementing *today*, the time must be cut (set to midnight) as otherwise in case of queries similar to "... where datum between today and today + 1" the current day is not included in the selection.

HYDRA SQL	ORACLE	SQL SERVER
today	trunc(sysdate,'DD')	cast(convert(char,getdate(),101) as datetime)
current	systimestamp	getdate()

Note:

Both functions always provide date and time in relation to the time of the operating system.

If you use the HYTIMEZONE functionality, the values are therefore not correct. For this reason, you may not use these two functions in the standard.

20.8.6 Date format

HYDRA SQL uses the date format "MM/TT/YYYY".

20.8.7 Date functions

The following table includes the available date functions and their implementation for ORACLE and SQL Server.

HYDRA SQL	ORACLE	SQL SERVER
year(...)	to_number(to_char(to_date(...),'YYYY'))	year(...)
month(...)	to_number(to_char(to_date(...),'MM'))	month(...)
day(...)	to_number(to_char(to_date(...),'DD'))	day(...)
weekday(...)	to_number(to_char(to_date(...),'D'))	datepart(dw,...) – 1
date(...)	to_date(...)	cast(... as datetime)
get_date(...)	trunc(...,'DD')	cast(convert(char,..., 101) as datetime)
get_time(...)	to_number(to_char(...,'SSSS'))	(datepart(Hh,...) * 60 + datepart(Mi,...) * 60 + datepart(Ss,...)

The function get_date(...) returns the column date of type *DATETIME*. The function get_time(...) returns the column time in seconds since midnight of type *DATETIME*. Other functions

20.8.8 Other functions

Implement other functions for ORACLE and SQL Server as follows:

HYDRA SQL	ORACLE	SQL SERVER
length(...)	(no implementation)	len(...)
nvl(...)	(no implementation)	isnull(...)
trim(...)	rtrim(ltrim(...))	rtrim(ltrim(...))
rtrim(...)	(no implementation)	(no implementation)
ltrim(...)	(no implementation)	(no implementation)
string(...)	to_char(...)	ltrim(str(...))
value(...)	to_number(...)	cast(... as integer)

The functions **trim(...)**, **rtrim(...)** and **ltrim(...)** currently only work independently of databases if a space character is used. The transfer of another character, which should be cut off on the left and/or right of a string, is not supported.

When sorting is concerned, the function **nvl(...)** has a special meaning in HYDRA SQL (see chapter "Sorting of NULL values").

20.8.9 Query of NULL values

SQL Server makes a difference between character strings including an empty string and the ones with a NULL value. At the same time, SQL Server saves the NULL values as empty string after export and subsequent reimport. Therefore, you must always combine these two queries:

HYDRA SQL: ... where (*column* is null **or** *column* = "")

With fields of type *char(1)*, you must additionally include a space character in the query:

HYDRA SQL: ... where (*column* is null **or** *column* = "" **or** *column* = " ")

The same applies, if you query not NULL:

HYDRA SQL: ... where (*column* is not null **and** *column* != "")

or

HYDRA SQL: ... where (*column* is not null **and** *column* != "" **and** *column* != " ")

20.8.10 Sort by NULL values

Contrary to SQL Server where NULL values are returned on top of the column on sorting, ORACLE sorts the NULL values at the bottom. Use the function *nvl()* to change this:

HYDRA SQL: ... order by **nvl(char_column, " ")**, **nvl(number_column, -1)**;

To enable this change with ORACLE, enter a space character (not an empty string!) as replacement value for columns of type *char*. The example above states -1 with numeric columns. If necessary, you must select a lower value. With SQL Server, this function is therefore deleted in the statement (only in *order by*).

20.8.11 Sorting with union select

If you use *union select* in a statement for sorting, you can use either the column number, the column name or the alias in the *order by* clause.

We recommend to create and use an alias with columns of a *union* that you want to sort.

20.8.12 Group by calculated expressions

If you want to group by a calculated expression, ORACLE and SQL Server expect the calculated expression in the *group by* clause. To integrate this conveniently in HYDRA SQL and to be independent of databases in the future, HYDRA SQL uses the alias to state calculated expressions in *group by*:

HYDRA SQL:	select <i>column</i> + 2 <i>alias</i> from <i>table</i> group by <i>alias</i> ;	ORACLE +
SQL Server:	select <i>column</i> + 2 <i>alias</i> from <i>table</i> group by <i>column</i> + 2 ;	

20.8.13 Outer Join

Use the ANSI syntax for outer joins. All databases can process this syntax and for this reason it need not be implemented:

HYDRA SQL: ... from *table1* a **left outer join** *table2* b on a.column = b.column

If you use an outer join to read in another table the name of a value of the first table, you can use a so-called lookup:

```
select column1, (select column2 from table2 b where a.column1 = b.column1) from table1 a ...
```

A lookup is a subselect, which replaces a column. You **must** ensure with this subselect that at least 1 data record is found, otherwise an SQL error is created. If no data record matches the condition, a NULL value is returned (similar to an outer join).

To provide compatibility, an outdated independent HYDRA SQL syntax is supported, which combines the outdated syntaxes of INFORMIX and older Oracle versions:

HYDRA SQL:	... from <i>table1</i> a, outer <i>table2</i> b where a.column = b.column (+)
ORACLE:	... from <i>table1</i> a, <i>table2</i> b where a.column = b.column (+)
SQL Server:	... from <i>table1</i> a, <i>table2</i> b where a.column *= b.column
[INFORMIX:	... from <i>table1</i> a, outer <i>table2</i> b where a.column = b.column]

20.8.14 Temporary tables

20.8.14.1 Overview

Temporary tables are tables that are only valid for the current database connection and include an intermediate result. The tables are automatically deleted when the program is finished, if not yet done. The following two sections show 2 possibilities how to create a temporary table.

20.8.14.2 Restrictions

For technical reasons, the number of temporary tables is limited to 12 for one database connection.

Also programs running as services or deamons have this limit; for example the "hymw" services of the data collection. This means: When an input dialog is processed, a maximum of 12 temporary tables can be created and used at a time. If you do not explicitly drop these temporary tables, these tables decrease the number of temporary tables available in the program until the service or daemon is stopped. In this case, for example in user exits of the data collection, it is absolutely necessary to drop the temporary tables when they are not required any more to avoid problems with other software parts that also require temporary tables.

If more than 12 (or almost 12) temporary tables are required at the same time, it is more secure to use normal permanent tables. If you use permanent tables, the tables must either be dynamic tables with table names that are unique in the entire system or the data records included must be clearly linked to the database connection. To create a unique table name, you can add the terminal number to the table name or add it as additional column in the data records, for example.

20.8.14.3 create temp table

Create a temporary table using the command *create temp table*:

```
HYDRA SQL:  create temp table tablename (...);
ORACLE:     create global temporary table tablename_<pid> (...)
              on commit preserve rows;
SQL Server: create table #tablename (...);
```

With ORACLE, the *global temporary tables* are available for all users. Add the PID (process ID) to the table name separated by an underscore so that the relevant process can distinctly identify the table.

20.8.14.4 select into temp

The second possibility to create a temporary table is a *select into temp*:

```
HYDRA SQL:  select ... from ... where ... into temp tablename;
ORACLE:     create global temporary table tablename_<pid>
              on commit preserve rows as select ... from ... where ...;
SQL Server: select ... into #tablename from ... where ...;
```

20.8.15 unique / distinct

„*select unique*“ and „*select distinct*“ both work with HYDRA SQL. With SQL Server, *unique* is replaced in the statement by *distinct*.

20.8.16 like / matches

You can use *matches* as synonym of *like* in HYDRA SQL. With both comparison operators, '*' and '%' are processed as wildcard for any number of characters. '?' and '_' substitute any other character.

20.8.17 Loading and unloading data

Use the command *unload* to unload data from a database into a file. The columns of a data record are written in a row of the file separated by '|'. The command *xunload* additionally writes the row names in the first row. Use this command, if the number or the order of the columns do not coincide with those in the table into which you want to load the data. Example:

HYDRA SQL: **xunload to** *filename* select *columns* from *table*;

Use the command *load* to load such data back into the database. Use the command *fload* to load a great number of data records faster with ORACLE if the table includes a column of data type *serial*. To this end, delete the trigger and the sequence to create distinct values and recreate them after having loaded the data. Also use the command *fload* if the data to be loaded include very high values in the *serial* column because if you insert these high values, the sequence must be counted up to this value.

HYDRA SQL: **load from** *filename* insert into *table*;

Unload files may include comments with additional information. This way, you might store the table schema and the indices into the unload file. If you want to store comments, the following rules apply:

- You can identify the beginning of a comment because the row begins and ends with the character '\$'. Comment rows are therefore different to data rows which always end with the character '|'.

Each comment ends with "\$END\$" in a the separate row. Comment that only include one row are not possible.

The comments may include any information.

All comments must be at the beginning of the file and precede the row "\$COLUMN\$".

Example:

```
$SCHEMA$
create table tablename ( column integer );
create unique index indexname on tablennname ( column );
$END$
$VERSIONS$
hymw 8.1.1.417
$END$
$DBPATCHES$
dbp_mw30 18.12.2010
$END$
$COLUMN$column|
0|
1|
```

20.8.18 create table as select

With ORACLE and SQL Server, you can create a table from a query:

HYDRA SQL: **create table** *tablename* **as** select ... from ...;

SQL Server: select ... **into** *tablename* from ...;

20.8.19 CASE in the select clause

In the SQL-92 standard, the CASE function is implemented in SQL. Using the CASE function, you can make decisions on result set level.

Example:

In MDE log records ("ereignis" table), yield and scrap quantities may only be taken from the end-of-shift records. However, the duration of statuses has to be determined from the "N" and "P" records. Therefore, our previous programs have cumulated the quantities in a separate UNION. This problem can be solved by an SQL statement using CASE syntax. This provides fundamental performance benefits.

```
select masch_nr,
       sum(dauer),
       sum(case when (satzart = 'N') then zaehler1 else 0 end) gut,
       sum(case when (satzart = 'N') then zaehler3 else 0 end) aus
from ereignis
where masch_nr like '%'
      and bmktonr between 1 and 11
      and satzart in ('P', 'N')
group by masch_nr;
```

The following example returns name and first name separated by comma but only if a first name exists in the HR master.

```
select case when (person_vorname is null) or (person_vorname = '')
           then person_name
           else person_name || ', ' || person_vorname
           end
from personalstamm;
```

1.24 Integer division

ORACLE returns a float if you perform a division of two columns including integer data types. With SQL Server, the result and the two operands are integers:

HYDRA SQL: select 215 / 10 from setup;

ORACLE: Result = 21.5

SQL Server: Result = 21

You can avoid this difference by replacing one of the two numbers by a decimal value:

HYDRA SQL: `select 215 / 10.0 from setup;`

20.8.20 Changing tables

20.8.20.1 Adding columns

The different databases use different syntaxes to add columns:

HYDRA SQL: `alter table tablename add ( column1 integer, column2 char(1) );`

SQL Server: `alter table tablename add column1 integer, column2 char(1);`

20.8.20.2 Changing columns

The different databases use different syntaxes to change columns:

HYDRA SQL: `alter table tablename modify ( column1 char(20), column2 char(40) );`

SQL Server: `alter table tablename alter column column1 varchar(20);`
`alter table tablename alter column column2 varchar(40);`

20.8.20.3 Deleting columns

The different databases use different syntaxes to drop/delete columns:

HYDRA SQL: `alter table tablename drop ( column1 char(20), column2 char(40) );`

SQL Server: `alter table tablename drop column column1 varchar(20);`
`alter table tablename drop column column2 varchar(40);`

20.8.21 Reserved keyword "key"

The keyword "key" is reserved with SQL Server and DB2. If you select a column named "key", "key" must be set in double quotes.

HYDRA SQL: `select key from tabelle;`

SQL Server: `select "key" from tabelle;`

20.8.22 Default values in the database schema

If you create a table, you can define a default value for the columns.

HYDRA SQL: `create table tabelle ( column char(1) default "N" not null );`

SQL Server: `create table tabelle ( column char(1)`
`constraint df_tabelle_column default "N" not null );`

Using a default value can help to avoid unnecessary *or* statements that might create errors. In addition, you can avoid NULL values in the relevant column if you add *not null*.

If you add a column with a default value to a table, this column is automatically populated in the existing data records.

HYDRA SQL: alter table *tabelle* add (*column* char(1) **default** "N" not null);
 SQL Server: alter table *tabelle* add *column* char(1)
 constraint df_*tabelle_***column default** "N" **with values** not null);

If you subsequently add a default value to an existing column, the statement must include (because of former DB Informix) the current data type. You cannot change the data type and the default value at the same time (because of SQL Server).

HYDRA SQL: alter table *tabelle* modify (*column* char(1) **default** "N" [not null]);
 SQL Server: alter table *tabelle* **add constraint df_***tabelle_***column default** "N"
 for *column*;

Note:

With ORACLE, if you change a column that already is "not null", you must not state "not null" because this would create SQL error 1442.

You can delete the default value of a column with the following statement. Note that you must first reset the attribute *not null* for the column:

HYDRA SQL: alter table *tabelle* modify (*column* char(1) **default** null);
 SQL Server: alter table *tabelle* **drop constraint df_***tabelle_***column**;

Notes:

You may only include one column per statement if you add and delete default values.

If you want to assign a default value to several columns of a table, you must use several individual SQL statements.

If you want to change the default value of a column, you must first delete the default value and then add the new default value. Here, you need not reset the attribute *not null*.

If you want to delete a column including a default value, you must first delete the default value and then drop the column (with SQL Server).

20.8.23 Process "clustered index"

With SQL Server, you can create a "clustered index" per table. With ORACLE, the keyword "clustered" is deleted from the statement:

HYDRA SQL: create [unique] **clustered** index *indexname* on ...;

ORACLE: create [unique] index *indexname* on ...;

20.8.24 Optimizing "update statistics" under ORACLE

Function

As of the below mentioned program version of the ORACLE backend, you can control the functioning of the command "update statistics" via environment variables. Using these variables, you can change from COMPUTE to ESTIMATE processing and vice versa. COMPUTE uses all data records of a table or an index to generate the statistics. ESTIMATE only uses parts of the data records (depending on the configuration).

Configuration

You control the activation of the extended functionality via environment variables.

UPD_STAT_NUM_ROWS ... Once this number of entries in a table is reached, it is changed to ESTIMATE.

UPD_STAT_ESTIMATE_PERCENT ... Percentage for ESTIMATE (value range between 1 and 100)

Example for Windows

```
set UPD_STAT_NUM_ROWS=10
set UPD_STAT_ESTIMATE_PERCENT=20
```

Example for Unix

```
export UPD_STAT_NUM_ROWS=10
export UPD_STAT_ESTIMATE_PERCENT=20
```

You can set the environment variables as described above in the script hy_env.scr (UNIX) or under Windows as system variable or entry in the registry.

Activation and default values

UPD_STAT_NUM_ROWS	UPD_STAT_ESTIMATE_PERCENT	Executed syntax
not set	not set	Old syntax (corresponds to COMPUTE)
Value greater than or equal to 0	not set → Default: 10 (%)	New syntax (corresponds to ESTIMATE with 10 % for tables with more than UPD_STAT_NUM_ROWS data records)

not set → Default: 0	Value between 1 and 100	New syntax (corresponds to ESTIMATE with defined percentage for all tables)
Value greater than or equal to 0	Value between 1 and 100	New syntax (corresponds to ESTIMATE with defined percentage for tables with more than UPD_STAT_NUM_ROWS data records)

Note

If the number of data records in the relevant tables is smaller than **UPD_STAT_NUM_ROWS**, the "new syntax" with estimate_percent with **NULL** (→ corresponds to COMPUTE) is used.

20.9 Notes on the performance

20.9.1 Union versus union all

Use *union* to summarize data of several tables to a result set. This *union* also removes duplicate data records. To do so, the database sorts the result set by all columns. If you do not want to remove the duplicate data records with *union* or if there cannot be duplicate data records, use the syntax *union all* instead of *union*. You save a lot of time because *union all* does not sort and remove the duplicate data records.

20.9.2 Substrings in the WHERE clause

If you use substrings in the WHERE clause, you must bear in mind that ORACLE does not use an existing index for the relevant column.

ORACLE: select ... from auftrags_bestand **where** auftrag_nr[1,8] = „...“;

In this example, ORACLE does not use the index for the column auftrag_nr and performs a sequential scan in worst-case.

A possible change of the statement is:

ORACLE: select ... from auftrags_bestand
 where auftrag_nr like „...%“
 and auftrag_nr[1,8] = „...“;

20.9.3 truncate table

If you want to delete all data records of a table, use *truncate table* instead of "delete from..." with HYDRA SQL. Using this command with ORACLE, the data records to be deleted are not stored in log files and processing is accelerated.

HYDRA SQL: **truncate table** *tablename*;

20.10 Access to several databases

20.10.1 Syntax

Currently, the access to several databases is only realized for ORACLE under Linux. Open a connection to **one** additional database using the statement:

connect <connectstring> [user <username>] [password <password>];

Example: connect linux1ora user hydadm password mpdv;

Here, the parameters *user* and *password* are optional. The default value for both parameters is "hydadm". Following the example of the local database, you can override the default values using environment variables. You can use the environment variables HYDBUSER and HYDBPW to define user and password of the local database. But you must add the connect string in capital letters separated by an underscore to add an additional database (example: HYDBUSER_DEC1ORA).

Notes:

You may only open the additional database using "connect..." once the default database has been opened.

You may not use bind variables to pass <connectstring>, <username> and <password>.

To perform statements on the second database precede the statement by

at <connectstring> ...

Example: at linux1ora select projekt from setup;

Close the additional database using the statement

close database <connectstring>;

Example: close database linux1ora;

Note:

You must close the connection to the additional database before closing the default database.

20.10.2 Restrictions

The following restrictions apply if you want to access the additional database:

- You must not create, change and delete tables (mainly because of the SERIAL columns). This also applies for the use of temporary tables.
- The command fload (fast load) is not allowed.
- You may only perform *update statistics* on the local database.
- Currently, this extension is only available under Linux (under Windows you still use HOLD_CURSOR=YES).

21 The Repository

21.1 Overview

The data of the repository is used in multiple ways:

- The repository defines and describes the **interface between client and server**. The input parameters and the result sets of service requests are described.
- In case of interpreted service types, the **processing and the business logic of a service** is specified via configuration in the repository. Only in exceptional cases, an actual programming in the server is required.
- For the client, the repository defines **how the data is displayed on the client** and which GUI elements are used to enter data. The repository also defines how the client checks the user input. You can generate most of the applications on the client using the configurations of the repository. Here, programming on the client is not required.

The repository data is grouped and structured using **domains**. A domain summarizes all data that logically belongs to an application.

The domain contains hierarchically structured and typed data. A domain includes *services* and *service parameters*, the respective *GUI settings*, *properties*, *authorizations*, *ReferenceData* and *ControlDataSources*.

Find below a detailed description of the repository elements.

21.2 Domain

Domains have properties and provide services within the domain context.

A domain is the smallest software unit. You can update the domain using an update package. Create a separate domain for each application. This domain then includes the services implemented for this application. You can also use the services and client attributes of a domain in applications of other domains. For example, a client application in its own domain can use a service of a different domain.

You can assign global contents to a global domain: for example, client menu configurations or separate global syntactic types.

Name

Each domain has a unique name. For the name, you use the notation "UpperCamelCase".

21.3 Service

Services have transfer parameters and return values, which are often identical to the properties of the domains.

21.3.1 Name

Name of a service. The service name usually consists of the domain name that includes the service and the **function**, separated by a dot.

21.3.2 Function

This field describes the requested service function. Typical functions are list, update, insert, delete, new, ...

21.3.3 ServiceType

There are several service types.

InterpretedJavaService2: Services of this type are used to display lists and evaluations. The services are interpreted at runtime using repository data. Contrary to the *InterpretedJavaService*, the services of type *InterpretedJavaService2* are prepared to stream data and provide more elegant options for Java user exits.

***InterpretedJavaService* (obsolete)**: Services of this type are interpreted at runtime using repository data. These services have been replaced with the service type *InterpretedJavaService2*.

InterpretedBAPIService: You use services of this type to edit data. The services are interpreted at runtime using repository data.

ExternalJavaService: Services of this type are completely implemented in Java. You can use these services to implement lists or editing functions. You use these services if the possibilities of the interpreting service types are not sufficient and the logic must be converted into Java programming.

InterpretedWrapper: Services of this type are interpreted at runtime using the repository data. The service is implemented as wrapper of an existing PDM dialog and is therefore subject to specific limitations, e.g. it does not support any dynamic Where.

***Wrapper* (obsolete)**: Services of this type are programmed and wrap an existing BAPI function. They are therefore subject to specific limitations, e.g. no dynamic Where.

***JavaService* (obsolete)**: Services of this type are completely implemented in Java.

Recommendation:

- The type *InterpretedJavaService2* is recommended for services that you use to read data.
- The type *InterpretedBAPIService* is recommended for services that you use to write data.
- If the interpreted service types cannot meet the requirements (or only with great effort) even if they include Java user exits, you should use the services implemented in Java of type *ExternalJavaService*.
- The other service types are older technologies and should not be used for new developments.

21.3.4 ListMode

For services of type *Wrapper* or *InterpretedWrapper*: This column must be populated for each service. The column specifies whether the requested PDM dialog returns a file as result or whether it is only a return string. "Y" => The result is a file, otherwise only a string.

21.3.5 DLG

For all services of **ServiceType** *InterpretedWrapper* or *Wrapper*, you must fill in either **DLG** or **SystemCall**. You fill in DLG, if **ServiceType** is *Wrapper* or *InterpretedWrapper* and if the service requests a PDM dialog with the structure "DLG=<content in this column>|..."

21.3.6 SystemCall

For all services of **ServiceType** *InterpretedWrapper* or *Wrapper*, you must fill in either **DLG** or **SystemCall**. Fill in SystemCall, if the you want to run a program in the server. In the column, the name of the external program is specified. The result is a PDM dialog with the structure: "DLG=SYSTEM.CALL|PROG=<content of this column>|..."

21.4 ServiceGui

The ServiceGui data define the use and the presentation of the services on a client. You can clearly allocate the ServiceGui to a service via their name.

21.4.1 Name

The name of the service for which this data record provides presentation information.

21.4.2 Package

This field is obsolete and must be left empty.

21.4.3 Extended

This field is obsolete and must be left empty.

21.4.4 AdditionalDataLogics

This field is obsolete and must be left empty.

21.4.5 ApplicationID

Application ID used for generating applications in the client. In case of editing applications, the ApplicationID is edited with the main data source of the application that you want to generate.

21.4.6 ApplicationTitle

Language key for the title of the generated application. In case of editing applications, the ApplicationTitle is edited with the main data source of the application that you want to generate.

21.4.7 ApplicationHelpFile

File name of help file (including file extension) of the generated applications. In case of editing applications, the ApplicationHelpFile is edited with the main data source of the application that you want to generate.

The name of the help file should be independent of the technology of a used client. The client should therefore put a prefix in front of the file name. You can then design the help file displayed according to the client's technology.

Example for the client MOC: In ApplicationHelpFile, you enter "Article.pdf". The client MOC then loads the document "MOC_Article.pdf" as online help. The client automatically uses the prefix "MOC_".

21.4.8 ApplicationHelpIndex

Bookmark that is activated when Help is opened. In the main application, it is usually "Overview". You must only edit this bookmark for the main data source of the application that you want to generate.

21.4.9 Description

21.4.9.1 General

Language key for short description of service.

You can show this description on the client when the selection of services is displayed.

21.4.9.2 Processing in the MOC client

The MOC shows the description if you add a data source while configuring an application.

21.5 ServiceParameter

ServiceParameters specify the parameters of a service. They provide information on the data source and value ranges.

The service parameters include selection criteria and the columns of the result set. A service parameter can be a selection criterion or be included in the result set. The attributes described below specify if a service parameter is used as selection criterion and/or is included in the result set.

21.5.1 Acronym

Name of the parameter. The combination of **Acronym** and **ResultSet** must be unique for each service.

21.5.2 ResultSet

If the associated service returns more than one **ResultSet**, a name must be indicated here. This way, you can return results in parallel that have been calculated at the same time but have a different structure. The combination of **Acronym** and **ResultSet** must be unique for each service.

21.5.3 WebServiceType

Data type of the parameter (*decimal, integer, string, boolean, binary, datetime*). This value must be identical to the configured value of the property configuration. **IMPORTANT:** *binary* parameters are not supported by default. You can only use these parameters in user exits.

21.5.4 DefaultValue

Specifies a service default value for a parameter.

21.5.5 IsResult

Specifies whether this service parameter is part of the ResultSet (return value). If you want to use the **DefaultValue**, do not set this field (IsResult).

In case of services of type InterpretedWrapper, you must only set the column IsResult to "Y" for UPDATE, LOCK, UNLOCK, DELETE, INSERT and COPY, if the BAPI actually returns a value, e.g. a new internal_id when you create new data records.

21.5.6 IsDynamicResult

Required for the generation of the Java function (for dynamic ResultSets, the column number must automatically be extended to the fixed number). Missing columns are added as empty columns (i.e. these columns are not computed).

21.5.7 InputAsArray

The client must transfer values in form of an array. **InputAsArray** is only reasonable in case of a quantity input parameter, i.e. if at least one of the two columns, **IsSpecialParameter** and **IsFilterParameter**, is set and a quantity operator such as BETWEEN or IN is possible.

Specify if a field is an array or not (with filters always yes except for Boolean type).

If true and no array or empty, then exception. Is currently only verified in case of mandatory special parameters.

21.5.8 IsSpecialParameter

Specifies whether or not the parameter is a special type controlling the service functionality (i.e. is not a filter parameter). For the **ServiceType Wrapper**, this is the only possible parameter type. In case of the **ServiceType JavaService**, it represents a special parameter not directly included in the WHERE condition but with different "controlling" effects. If you want to use the *Default Value* on the server side, do not set this field. In addition to the defined special parameters of standard processing, you can also use other special parameters in user exits.

21.5.9 IsFilterParameter

Specifies whether it is a filter parameter. If you want to use the **DefaultValue** on the server side, do not set this field.

21.5.10 IsMandatory

Specifies whether it is a mandatory parameter for the service. If true and parameter is missing, an exception is thrown. Is currently only checked for special parameters.

21.5.11 Can* (filter) operators

This option specifies whether the service supports the relevant filter operator for this parameter. Set the "Can*" fields for filter parameters.

Available operators:

- **CanEqual**
- **CanLike**
- **CanBetween**
- **CanIn**
- **CanNotEqual**
- **CanLt (Can Less Than)**

- **CanLte (Can Less Than or Equal To)**
- **CanGt (Can Greater Than)**
- **CanGte (Can Greater Than or Equal To)**

For technical reasons, each operator has a second operator that you should select is a data record must be selected, if the operator is applicable or if the comparative value is NULL. The operator **CanEqual** will only return a data record in case of equal values, **CanEqualOrNull** in case of equal values or if the data record value is NULL. Accordingly, there are the following operators:

- **CanEqualOrNull**
- **CanLikeOrNull**
- **CanBetweenOrNull**
- **CanInOrNull**
- **CanNotEqualOrNull**
- **CanLtOrNull**
- **CanLteOrNull**
- **CanGtOrNull**
- **CanGteOrNull**

Especially with List Services you should make sure that generally all parameters support all operators in order to achieve the highest possible selectivity. In general, the framework supports this for Java services.

- You may only set **CanIn**, **CanBetween**, **CanBetweenOrNull** and **CanInOrNull**, if **InputAsArray** is also set.
- **CanLike** is only useful if the **WebServiceType** is *string*.
- With **WebServiceType** boolean, only **CanEqual** is useful.
- With **WebServiceType** string, all operators are possible.
- With all other types, all operators except for **CanLike** and **CanLikeOrNull** are useful.

Before you set wrappers, you must check which operators are actually supported by the PDM dialog or the system command.

21.5.12 HydraAcronym

With service type *InterpretedWrapper*, the HYDRA acronym is specified.

21.5.13 HydraResultAcronym

If the acronym of the selection criterion is different to the acronym in the result file, you can enter an acronym that is different to the HydraAcronym for the service type *InterpretedWrapper* and **ListMode=Y**.

21.5.14 TransferEmptyValuesToHydra

Specifies whether blank values, too, are to be transferred to the server, or whether the ID is simply omitted. "Y" => blank values are transferred, otherwise => ID is completely omitted.

Note: You must set this field for Insert and Update (editing screens). Only then, you can enter blank values and/or overwrite existing values with blank values.

21.5.15 HydraShiftPart

The following components are combined with the **Reference** field: Start of shift date, start of shift time, end of shift, end of shift time stamp, start of shift time stamp. These components are marked as belonging together. The column "HydraShiftPart" can include the following values:

- beginDate
- beginTime
- beginDatetime
- endTime
- endDatetime

Important: The column can only be populated if the parameter is part of a group that includes the following five components: Start of shift date, start of shift time, end of shift, end of shift time stamp, start of shift time stamp. The column must not be populated if it is only a group of three components including date, time and date + time field. In this case, ONLY populate the **Reference** column.

21.5.16 Reference

Is used to generate a DateTime data type from one field each for the date and the time (in seconds after midnight) and to identify the shift parameters.

21.5.17 TransformationType

Use this field to specify transformations for input and result parameters for List Services/wrappers (e.g. convert Bool to J/N and vice-versa or correct filtering for DateTime fields that consist of two fields in the database). For further details on this field, refer to section 21.10.

21.5.18 PlugName

Specifies whether the result parameter for this service is directly derived from the specified DataObject or whether it is added to the DataObject via plug.

Example:

Service A.List uses a plug of service B.List in the service parameter b. Consequently, the following configuration applies to service A.List:

ServiceParameter	DataObjectName	PlugName
a	A.List	
b	A.List	B.List

If the field *PlugName* includes a value, the Interpreter replaces the values of the *ServiceParameter* with those values of the plugged service when creating the SQL statement.

In the special case where an interpreted List Service does not use an own table but only plugs, and subsequently adds fields via user exit, these fields should state USEREXIT!



If you create new services, it is recommended to avoid plugs and to provide data directly via the DataObject via Join. Dependencies between several services are thus avoided.

21.5.19 DBField

Database field that you use to make a selection. Write the database field in lower case. You can either enter simply the field name or (for complex expressions) the expression with placeholders for the alias (e.g. `hydadm.get_datetime(%1$s.bearb_date,%1$s.bearb_time)` or `{fn substring(%1$s.field,2,1)}`).

Proceed as follows for joins to other tables:

Entry: <ALIAS>.<DBfield>

Example:

DB field: STA1.status_bez

Acronym: gage.status.designation

Table: caq_status (STA1)

Conditions: status_typ = 'PMSTATUS', status_nr = status

21.5.20 DBAlias

The alias for the table that is used to select the value for the acronym.

21.5.21 DBTabelle

The table that is used to select the value for the acronym.

21.5.22 DBFieldAlternative

If you cannot use the **DBField** because the **ConditionalFieldKey** is not applicable, you use the **DBFieldAlternative**.

You can enter a number, 'null', 'string', {fn ...} or another field / subselect. If it is another field or subselect, you MUST enter %1\$s for the alias of the table.

If **DBFieldAlternative** is empty, but you require an alternative field, NULL is selected.

21.5.23 DataObjectName

If a service uses several data sources to identify its data, you can store the data source (= DataObject = DO) that issues the result parameter in this field. For example: A service includes the parameters a, b and c:

- a is computed,
- b is identified using data object (DO) F and
- c is identified using data object (DO) G.

For a: the field is blank. For b: the field contains F. For c: the field contains G. Is used as reference for the ...do.xml configuration.

21.5.24 ConditionalFieldKey

This field specifies if a DB field is only conditionally available. The ConfigurationManager checks the condition for the existence of the field. Enter the feature key of the Configuration Manager (**feature set**) in this repository field to enable the check.



If a parameter is a conditional field and the condition is not fulfilled, the entries for the MOC acronym are removed from the ComplexSelectMap and the SpecialFilterMap.

As a result, the changes in the Special Filter Map via user exits and transformation type are also lost!

21.5.25 Constraints

Constraints are processing parameters that are used for **ServiceType** *InterpretedBAPIService*. Constraints are structured as keys with optional values. The separator between keys is the pipe character (|). You use a semicolon to separate various values. You use the equal sign (=) to separate key and value. The general structure is as follows:

Key1=Value;Value;Value|Key2|Key3=Value|

The following constraints are available:

Constraint Key	Constraint value(s)	Description
KEY	exactly one number between 1 and 5	Define field as key including key number for hyd_lock table
SERIAL	none	Field is a SERIAL (and/or auto-increment)
SEP_DATETIME	1st parameter refers to the date field 2nd parameter refers to the time field	Allows processing of separate date and time fields
BOOL	1st parameter is the value to be entered into the DB if true 2nd parameter is the value to be entered into the DB if false 3rd parameter is the value to be entered into the DB if null (null for null) 4th parameter is the type of DB field, e.g. BOOL=J;N>null string BOOL=1;0>null;integer	Use this to write Boolean values into a string or Integer Field.
MODIFY_TS	None	
MODIFY_BY	None	
CREATE_TS	None	
CREATE_BY	None	

21.6 ServiceParameterGui

The ServiceParameterGui define how ServiceParameters are displayed on the client. Use **Acronym** and **ResultSet** to clearly allocate ServiceParameterGui to a service parameter.

A number of settings exist in the `ServiceParameterGui` and in the properties. You normally use the properties to define how data is displayed on the client. You only fill the respective field in the `ServiceParameterGui` if you want to display specific services on the client in a way that is different to the settings in the properties. The `ServiceParameterGui` fields overwrite the property fields of the same name.

21.6.1 Acronym

Name of the parameter for which this data record provides presentation information. There must be a corresponding property for each **acronym** of a parameter.

21.6.2 ResultSet

See **ResultSet** with `ServiceParameter`.

21.6.3 Label

21.6.3.1 General

The label includes a language key. Using this language key, the parameter on the user interface is identified (by default), e.g. as label text of a column header. Overwrites the value from the property configuration.

21.6.3.2 Processing in the MOC client

The label is displayed as label text of a field or a column title.

21.6.4 Tooltip

Specifies a specific tooltip for the parameter in the service context. Entry as language key.

21.6.5 FormatType

Use this field to overwrite specific values of a property in relation to the service (currently **Label**, **Length**, **ControlType**, **ControlTypeMode**, **ControlDataSource**, **ControlDataSourceMode**, **ControlDataSourceResult**).

For example: If you enter *workplace.id* as **FormatType** for the parameter *resource.id*, you can define for the parameter to be a *resource.id* in this service, however its length, label and control properties are taken from *workplace.id*.

In this case (other than in case of semantic and syntactic types), the value from **FormatType** takes priority. For this reason, we have a new hierarchy:

- Value from **FormatType**
- Value from ServiceParameterGUI
- Value from Property
- Value from **SemanticType**
- Value from **SyntacticType**

21.6.6 ClientDefaultValue

Input fields have a **ClientDefaultValue** property. The value entered here is displayed as default value when the control is initialized. "From" and "to" values are separated by semicolons.

Set checkbox: If the value of this field is set to *true* during a CheckEdit, the checkbox is set after initializing.

Preallocation of text fields with "from" and "to" values (InputAsArray): set value1;value2 to prepopulate the 'from' and 'to' fields during a text edit.

Date fields: In case of date fields, the field can be preallocated with an offset. If you set default values for date fields, you must absolutely specify the type of offset. The following offsets are possible:

- h (hours)
- d (days)
- w (weeks)
- m (months)
- y (years)

The 'to' value is always **relative to the 'from' value**. The default value is always a DateTime object. The presentation depends on the output format of the relevant field.

You can put "[" and "]" in front and at the end of the relevant value to specify the start and end of a period of time. Consequently, e.g. "[0d;0d]" means that 12:00:00 AM is entered in the 'from' field today and 11:59:59 PM is entered in the 'to' field today. "[-1w;0w]" means from Monday last week up to Sunday last week.

Examples

Current date:

0d

From today to the day after tomorrow:

0d;2d

From today to one week from today:

0d;1w

From yesterday to tomorrow:

-1d;2d

From one year ago today to one year from today:

-1y;2y

Year shortlists: You can configure a year shortlist by **ControlDataSource** = YearList and **ControlDataSourceMode** = Script, or even by standard "Service-ControlDataSource". In this case, you can use the following default values:

- Current year: 0y and/or currentyear
- Last year: -1y
- Following year: 1y
- 4 years ago: -4y
- Year that was current 10 months ago: y-10m → this is mostly the case when the relevant year field is used in combination with a month shortlist.

If you want to preallocate two fields (**ShowSecondControl**), you have to separate both values by semicolon, e.g. -1y;1y

Month shortlists: You can use the following default values for a month shortlist:

- Current month: 0m
- Last month: -1m
- Following month: 1m
- 4 months ago: -4m

If you want to preallocate two fields (**ShowSecondControl**), you have to separate both values by semicolon, e.g. -1y;1y.

21.6.7 IsKey

The IsKey column is very important and should be occupied for all key columns of a service, since otherwise data records cannot be clearly identified. Columns including the value 'null' may NOT be defined as keys. The IsKey columns should be identical for all services (insert, update, delete, lock, unlock, copy). These entries are important, so it is best to verify them twice.

This field specifies the positioning of the cursor after an editing operation. If the positioning option OnKeyValue is selected, the client should only request one new data row after editing. You also use values that are marked **IsKey** as selection criteria. **IsKey** must also be set for *delete*, since this data record must be deleted from the view.

IsKey must also be indicated for *list*. If no sorting is given in the *list*, sorting takes place according to **IsKey** fields.

Every parameter which is **IsKey** MUST always be **IsMandatory**. This rule has two exceptions:

- List service
- Wrappers with composed keys.

21.6.8 ShowInGrid

Specifies whether the parameter is to be displayed in tables by default.

21.6.9 ShowInDetail

Specifies whether the parameter is to be displayed in detail views by default.

21.6.10 ShowInSearch

Specifies if the parameter is to be used as selection criterion (i.e. in selection panels) by default.

21.6.11 ColumnCategory

21.6.11.1 General

In the tabular view, the client should provide the option to summarize the columns in the table to categories. You specify a language key that is displayed as title of the summarized columns.

21.6.11.2 Processing in the MOC client

The ColumnCategory is used to assign the parameter to a "strip" in the grid (table view).

21.6.12 Category1, Category2, Category3

21.6.12.1 General

The client processes the columns Category1, Category2, Category3 in order to group fields in applications. The grouping can be performed via tabs or frames for a group of fields. You specify a language key that is displayed as title or label text of the grouped elements.

21.6.12.2 Processing in the MOC client

Category1: Assigns the parameter to a tab in the detail view.

Category2: Grouping options for detail screens.

Category3: Currently not used.

21.6.13 TabOrder

You specify the order of tabs for detail views.

21.6.14 ColumnOrder

You specify the order of columns in tabular views.

21.6.15 ShowSecondControllnSearch

21.6.15.1 General

Specifies whether a second control is to be displayed (from/to). You can use this setting with selection criteria that include a value range via the operator CanBetween, e.g. "date from/to".

21.6.15.2 Processing in the MOC client

The MOC provides two adjoining fields. The label text of the second field is automatically "to". If it is a field of "date" type, you can predefine a relative date for both fields.

21.6.16 SearchTabOrder

Specifies the tab sequence for the selection panel.

21.6.17 SearchCategory1, SearchCategory2

21.6.17.1 General

The client processes the columns SearchCategory1 and SearchCategory2 in order to group fields in selection panels. The grouping can be performed via tabs or frames for a group of fields. You specify a language key that is displayed as title or label text of the grouped elements.

21.6.17.2 Processing in the MOC client

SearchCategory1: You allocate the parameter to a tab in the selection panel.

SearchCategory2: Grouping options for the selection panel.

21.6.18 ControlType

Use the ControlType to specify which control should be used for the relevant parameter. The client will map the abstract type onto a specific control class. If you do not specify a type, the client uses the data type to decide on the ControlType. Possible values for the ControlType:

CheckEdit: Selects a Boolean value (true/false) or multiple values if a reference to a data source is given.

ColorEdit: Selects a color value.

ComboBoxEdit: Combobox with selection of values from web service or data reference.

DateTimeEdit: Enter a date and/or a time.

MemoEdit: Enter an arbitrary text.

RadioGroup: Selects a Boolean value (true/false) or one of multiple values if a reference to a data source is given.

TextEdit: Standard text input. You can add a button opening a search dialog to this control, if you add a reference to a service in **ControlDataSource**. If you enter the name of a DataLogic in **ControlParameter** and if a mapping is included in **ControlDataSourceResult**, data will be requested upon leaving the control and return values will be mapped appropriately.

21.6.19 ControlTypeMode

21.6.19.1 General

Allows for controlling the input control.

CheckEdit: DualState (default), TriState, J;N;J (checked;unchecked;tristate)

ColorEdit: none

ComboBoxEdit: SingleEdit, Single, Multiple (multiple selection)

DateTimeEdit: Date (date display), Time (time display), DateTime, RelativeDate, RelativeDateTime

MemoEdit: none .

RadioGroup: SingleColumn, SingleRow

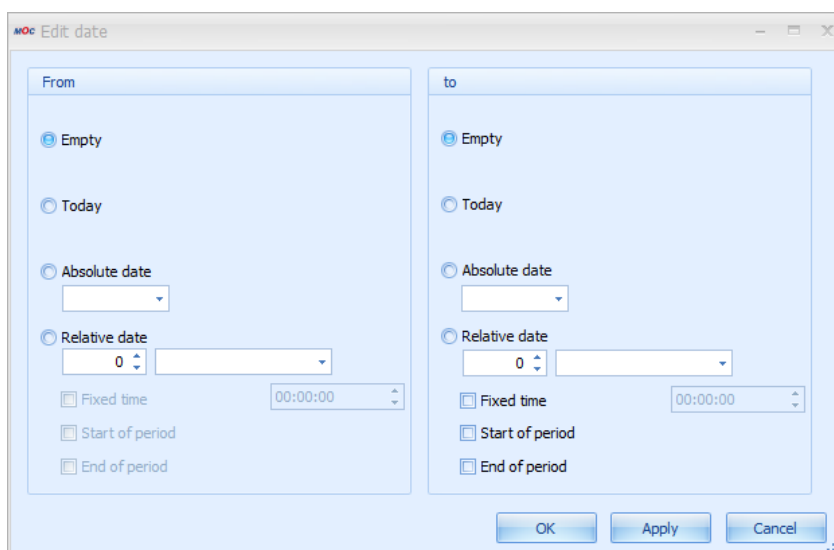
TextEdit:

- Empty: the search button is shown if a **ControlDataSource** is defined.
- "SearchButton": Search button is shown.
- "SearchButtonValidate": Search button is shown. If you enter an invalid value, an error is displayed.
- "OpenFileDialog": opens a file selection dialog.

21.6.19.2 Processing in the MOC client

If you use **DateTimeEdit** including the definition of a relative date (**ControlTypeMode:** RelativeDate or RelativeDateTime), you can enter a relative date.

If **ShowSecondControl** = true, you can predefine the complete relative value range. In this case, a button is displayed behind the second input control. You can use this button to open the following dialog:



Use this dialog to customize the values for **ClientDefaultValue** . The following entries are possible:

- **Empty:** no value is adopted

- **Today:** the current date is adopted
- **Absolute date:** you can select a fixed date value via a calendar control
- **Relative date:** you can select and adopt a date relative to the current date. In this context, "Start of period" means that you additionally go to the start of the selected period. Example: current date is 20-MAY-2010. If you select "- 1 month", 20-APR-2010 is adopted. If you also select "Start of period", the date is changed to 01-APR-2010. The same applies to "End of period". These settings are saved in the mpdvEdit or the selection profiles as **ClientDefaultValue**.

21.6.20 ControlParameter

See **ControlType** → **TextEdit**

21.6.21 ControlDataSource

Data source for the selection of values. The data source can be:

- **Web service** (configuration: **ControlType** = *ComboBoxEdit*, **ControlDataSourceMode** = *Lookup*, **ControlDataSource** = **Name** of a **ControlDataSource**. See also section "21.8 ControlDataSource")
- **ReferenceData** (configuration: **ControlType** = *ComboBoxEdit*, **ControlDataSourceMode** = *Reference*, **ControlDataSource** = **Type** of **ReferenceData**)
- **Search application** (configuration: **ControlType** = *TextEdit*, **ControlDataSourceMode** = *Lookup*, **ControlDataSource** = application name)
- **Script** (configuration: **ControlType** = *ComboBoxEdit*, **ControlDataSourceMode** = *Script*, **ControlDataSource** = Name of script)

21.6.22 ControlDataSourceMode

Data source mode (*Lookup*, *Reference* or *Script*).

21.6.23 ControlDataSourceParameter

Optional setting of parameters of a **ControlDataSource**. If you make settings here, these settings overwrite the settings in the **ControlDataSource**.

See also the description **ControlDataSource - Parameter**

21.6.24 ControlDataSourceResult

Optional setting of the result of a **ControlDataSource**. If you make settings here, these settings overwrite the settings in the **ControlDataSource**.

The settings in this field provide more options than the **Result** in the **ControlDataSource**:

Result columns are separated by semicolon. Field mapping:

- First entry: Value
- Second entry: Labeling
- Third entry: UnitLabel
- As of the fourth entry, the fields are mapped:
 - o Via acronym or semantic type.
 - o Field mapping: in **ControlDataSourceResult**, you can enter a mapping in the form "FieldName=ColumnFromResult" as from the fourth entry. For example, you can specify tool.id=resource.id in order to fill the field "tools.id" with the "resource.id" value from the search application. Several mappings are separated by ";" - spaces are not allowed.
 - o Asterisk mapping: Instead of mapping, you can also enter * . Subsequently, all return columns of the search application are mapped. The mapping is performed as usual via ID or semantic type.

21.6.25 VisibleCondition

This value decides whether an input field is visible on the client. For customization, see EditableCondition.

21.6.26 EditableCondition

This value decides whether you can edit an input field on the client. There are three possibilities:

- Boolean value: In case of *TRUE* or *FALSE*, the field is always editable / non-editable.
- Binary expression:
 - o Field name must be the name of a field that is also located in the ControlPanel.
 - o Valid operators: =, <, >, <=, >=, <>, !=
 - o The value is written as a string and interpreted depending on the comparative field value.
 - o Field, operator and value must be separated by a space!
- Concatenation of binary expressions:
 - o You can concatenate an arbitrary number of binary expressions.
 - o You can use the operators "&&", "AND", "||", "OR" to link expressions.
 - o Here, too, all components of the conditions must be separated by a space.
 - o Priority of operators: "AND" or "&&" are evaluated first, then "OR" and "||".
You cannot use brackets.
 - o Example: resource.id = 12345 && resource.costcenter = 20 || resource.id = 60610



The client assigns the default value of the property "ClientDefaultValue" to the field, if the result of an expression in the EditableCondition or the VisibleCondition changes from FALSE to TRUE. The client dynamically evaluates the expressions in the EditableCondition and the VisibleCondition, if the fields of the application change.

21.6.27 ScriptId

21.6.27.1 General

The ID of the script that is allocated to the parameter. If you set the ID, the relevant script is performed upon various events (at present EditValueChanged and Leave).

21.6.27.2 Processing in the MOC client

The method name of the script is ScriptId+EditValueChanged and/or ScriptId+Leave. The script can be included in any DLL that is read by the CodeManager.

21.7 Property

For the acronyms, properties include information on data types, input and output formats, display options, a name (that can be localized) and other settings specifying how ServiceParameters are displayed in the client. Each property has a system-wide unique acronym.

A number of settings exist in the ServiceParameterGui and in the properties. You normally use the properties to define how data is displayed on the client. You only fill the respective field in the ServiceParameterGui if you want to display specific services on the client in a way that is different to the settings in the properties. The ServiceParameterGui fields overwrite the property fields of the same name.

21.7.1 Acronym

Clear identification of the property across all domains.

21.7.2 WebServiceType

Describes the data type used to transfer the property between client and server. The currently supported WebServiceTypes are exclusively

- *binary*
- *boolean*
- *datetime*
- *decimal*
- *integer*

- string

Important: the types **date* and **time* are internal types which are not transferred.

21.7.3 NETType

The data type used by the client. If NETType is empty, the **WebServiceType** is used to automatically identify the data type used by the client. At present, NETType supports the following entries:

- **color:** Use *color* to convert the transferred integer into an RGB code. In this case, the conversion is implemented by the grid.
- **duration:** creates a duration from an integer.
- **image:** either creates an image from a transferred byte array or interprets a transferred string as image name. For example, the *maintenance.active.led* property is transferred as a string including the name of an icon.
- **preview:** Specifies that the contents in the client may be displayed as "preview" (similar to auto-preview outlook) (application e.g. in DevExpress grid).
- **timestamp:** Use *timestamp* to automatically create an additional column for date values in the client in order to process time and date separately.

21.7.4 SemanticType

Use semantic types to inherit semantic properties. The "order.id" is therefore used to identify orders (semantic meaning). The acronym *operation.order.id* includes such an order identification and therefore has the semantic type *order.id*. If an attribute of the property is not set (empty), the respective value from the semantic type is used for the processing in the client.

For example: You must set the semantic type if you want to adopt a value from a lookup screen in the field. For the *workplace* field, enter e.g. *resource.id* as semantic type in order to adopt the selected workplace from a search screen for workplaces. Refer to the description of the SyntacticType for further information on the priority used to specify the attributes of a Property, the SemanticType and the SyntacticType.

21.7.5 SyntacticType

You mainly use a syntactic type for a uniform presentation of the different properties. The syntactic type does not include any semantic content. For example: The properties *booking.begin_ts* and *booking.shift.start_ts* have different semantic meanings, but are presented in a uniform format that can be controlled centrally.

Syntactic types are used to control the characteristics of a *Property*: for example length, input and output screen, tooltip, label, etc. To select the valid value for a characteristic, the client proceeds as follows:

- If the characteristic (e.g. length) is set in Property, the client uses this value.
- Or: If a semantic type is available and the characteristic is set, the client uses this value.
- Or: If a syntactic type is available and the characteristic is set, the client uses this value.

Note:

- You must always enter a **description** for syntactic and semantic types.
- Syntactic types can reference other syntactic types so that "inheritance hierarchies" can be created.
- Create syntactic types as property of the *SyntacticType* domain.
- Semantic types are usually "real" properties of a "normal" domain that are used as semantic type at other places.

21.7.6 Label

21.7.6.1 General

The label includes a language key. Using this language key, the parameter on the user interface is identified (by default), e.g. as label text of a column header. Overwrites the value from the property configuration.

21.7.6.2 Processing in the MOC client

The label is displayed as label text of a field or a column title.

21.7.7 DefaultTooltip

Specifies the default tooltip for the property as language key.

21.7.8 UnitLabel

Text key for unit. The unit is displayed to the right of the input field.

21.7.9 OutputFormat

This field specifies the format that is used to display a value (e.g. for date or quantity values). If you do not enter an **InputFormat** in the repository, the MOC tries to develop an appropriate format from the **OutputFormat**. Enter the value **InputFormat** in the repository only if special masking is required. Find further details in section "21.7.12 Rules for the input/output formatting".

21.7.10 InputFormat

Equivalent to OutputFormat. You can enter a valid regular expression in the field InputFormat. Other entries that are not regular expressions are not permissible. Find further details in section 21.7.12.

21.7.11 Length

The client shows the control for this acronym in the specified width (i.e. the specified number of characters). With Length=0, the control uses the entire width available. If a width is specified but the space available is not sufficient, the control is cut off.

This field also specifies the number of characters that you can enter in an input field with **ControlType = TextEdit**, if no other **InputFormat** is specified.

21.7.12 Rules for the input/output formatting

Overview

In the repository, you define the formatting of the data output and the input dialogs to edit data. The "Properties" of the different acronyms include an **OutputFormat** and an **InputFormat**. The **OutputFormat** defines formatting if you display a value.

Important: If you do not enter an **InputFormat** in the repository, the MOC uses the **OutputFormat** to generate an appropriate formatting. Enter the value InputFormat in the repository only if special masking is required.



In case of strings, you cannot enter the special characters asterisk (*) and pipe (|), if you have not defined any input format. As you use these two special characters as separator and control character, they can cause problems if they are written in the database.



With strings, the maximum number of characters that you can enter is defined by the attribute **Length**, if no other input format is defined.

Syntactic types

The Properties provide so-called "syntactic types" in order to make groups (similar to field types in Delphi). Syntactic types have the same properties as real properties. The real properties have a syntactic type. For example, if the **output format** of the syntactic type includes a value, this value is used wherever this syntactic type is entered.

Example: Industrial minutes

The syntactic type "Durations" has the format {0:mpdv_timespan}. With the different properties showing durations, "Durations" is entered in the column **SyntacticType** and no entries are made in the columns "output format" and "input format". When the property is read - and if no **output format** is available in the property - the format of the syntactic type is used.

If a system displays industrial minutes (no standard function!) and if the syntactic type "Durations" is specified, the **output format** is automatically changed from {0:mpdv_timespan} to {0:mpdv_industrialMinutes}. As a result, all formats including the syntactic type "Durations" are shown in industrial time units.

Times and durations are internally stored in the system as integer seconds. If you convert times or durations during input or output formatting to formats other than hours, minutes and seconds (HH:MM:SS), the conversion may not be possible without losses. For example, this applies to the use of the "mpdv_calc" format and the classic industry minute display:



When converting from seconds to hours (division by 3600), decimal numbers with an infinite number of decimal places can occur, which inevitably have to be rounded when displayed on the client. Example: 20 minutes = 1200 seconds = 0.333333... hours. If the value is rounded to three decimal places, you calculate backward as follows: $0.333 \times 3600 = 1198.8$ seconds. Depending on how the client rounds, the internal value is then no longer 1200 seconds, but 1999 or 1998 seconds.

If you use less than three decimal places, the conversion error gets even greater:

The system recorded a duration of 123 seconds. The client displays 0.03 hours. If you calculate backwards, the result is 108 seconds.

Output formats

OutputFormat	Automatically created masking (input format)	Examples	Description
Numeric data			
f(number)	None	f3, f1	Numeric value without thousands separator. The number specifies the number of decimal places.
n(number)	None	n0, n2, n5	Numeric value with thousands separator. The number specifies the number of decimal places, even if the data type to be displayed is an integer type. In case of n0, no decimal places will be shown.
#. (##) , #. (0)	None	#.####, #.0000	Arbitrary format
{0:mpdv_timespan}	None	2:33:30	MPDV format provider. Conversion of seconds to hh:mm:ss and vice-versa.

{0:mpdv_timespan_short}	None	2:33	MPDV format provider. Conversion of seconds to hh:mm and vice-versa.
{0:mpdv_timespan_minutes}	None	45	MPDV format provider. Conversion of seconds to minutes and vice-versa.
{0:mpdv_cycletime}	None	1:30:00	MPDV format provider. Hours per 1000 pieces. Conversion into seconds and vice versa.
{0:mpdv_te}	None	2.00	MPDV format provider. Hours per 1000 pieces. Conversion into seconds and vice versa.
Strings			
empty	[^*]]*		Illegal characters begin with ^. In this example * and * No limitation in length
empty	[^*]]{0.10}		Illegal characters begin with ^. Max. length: 10 characters
empty	[0-9a-fA-F]		Allowed characters 0 through 9, a through f, A through F.
Special formats			
{0:mpdv_cycletime_sec_cycle}	None	29 sec/cycle	MPDV format provider. Seconds per cycle. Conversion into seconds per 1000.
{0:mpdv_IndustrialMinutes}	None	1.50	MPDV format provider. Conversion of seconds into industrial minutes and vice versa.
{0:mpdv_leadingzeros_order}	ORDER		You must combine this output format with the input format ORDER. The combination is used in the syntactic type "order_id". The basic settings are used to automatically specify the length.
{0:mpdv_leadingzeros_operation}	ORDER		You must combine this output format with the input format ORDER. The combination is used in the syntactic type "operation". The basic settings are used to automatically specify the length.
{0:mpdv_leadingzeros_sequence}	ORDER		You must combine this output format with the input format ORDER. The combination is used in the syntactic type "ordersequence_id". The basic settings are used to automatically specify the length.

Input formats

The following definitions are available for the input format:

- Leave empty: The input format is implicitly defined using the output format. See table above.
- Use of logical input formats
- Use of regular expressions

Logical input formats

To simplify the definition of input formats and limit the variety of entries in the repository, the logical input formats are provided. These input formats are permanently implemented in the client and can directly be used in the repository. Input formats are customized in the properties. But service parameters specify whether wildcards are allowed. For this reason, the input format actually used can vary depending on the allocated service.

In order to use logical input formats, define the name of the input format in the affected property in the repository. The following formats are currently available:

Name	Input format without wildcard	Input format with wildcard
CHARACTER	[^*][LENGTH]	[^][LENGTH]
NUMBER_N0	[0-9][LENGTH]	[0-9][LENGTH]
NUMBER_N1	[0-9][LENGTH]\R.[0-9]{0,1}	[0-9][LENGTH]\R.[0-9]{0,1}
NUMBER_N2	[0-9][LENGTH]\R.[0-9]{0,2}	[0-9][LENGTH]\R.[0-9]{0,2}
NUMBER_N3	[0-9][LENGTH]\R.[0-9]{0,3}	[0-9][LENGTH]\R.[0-9]{0,3}
NUMBER_N6	[0-9][LENGTH]\R.[0-9]{0,6}	[0-9][LENGTH]\R.[0-9]{0,6}
TIMESPAN_SHORT	[0-9][LENGTH]\R:[0-9]{2,2}	[0-9][LENGTH]\R:[0-9]{2,2}
TIMESPAN	[0-9][LENGTH]\R:[0-9]{2,2}\R:[0-9]{2,2}	[0-9][LENGTH]\R:[0-9]{2,2}\R:[0-9]{2,2}
ORDER	[0-9a-zA-Z.+][LENGTH]	[0-9a-zA-Z.+*][LENGTH]

The placeholder [LENGTH] is replaced with the configured field length at runtime. If the defined length is '0', an '*' is entered. With the logical format "ORDER", the system automatically changes the [LENGTH] according to the basic settings when the output format changes.

Input/Output formats including calculation

If you specify the **output format** *mpdv_calc*, you can include calculations in the formatting. In the format, you can specify a divisor and multiplier and an identifier that specifies if a reciprocal is calculated. You can also specify the number of decimal places. The **OutputFormat** *mpdv_calc* implicitly defines the **InputFormat**. If an input of values is made, the reciprocal value is calculated.

Example: "mpdv_calc;MULT=5;DIV=2;INVERSE=false;FORMAT=n3" (the value is multiplied by 5, divided by 2, then the reciprocal is calculated and the result is displayed with 3 decimal places).

The input/output format including calculation is normally used for the display of cycle times or specifications of single pieces. In the database, these times are always saved in seconds per 1000 pieces. If an input/output format including calculation is used, you can convert the times to hours per 1000 pieces, minutes per piece or with reciprocal also to piece per hour.

Overview of regular expressions

You can find a large amount of information on regular expressions using the search engines on the internet. In the following, the most important aspects are presented.

Meta characters

Represent a range of characters.

Character	Description
.	Matches any character.
[aeiou]	Matches any single character included in the specified set of characters.
[^aeiou]	Matches any single character, which is not included in the specified set of characters.
[0-9a-fA-F]	Use of a hyphen (–) allows specification of contiguous character ranges.
\R.	Matches the decimal separator specified by the System.Globalization.NumberFormatInfo.NumberDecimalSeparator property of the current culture.
\R:	Matches the time separator specified by the DateTimeFormatInfo.TimeSeparator property of the current culture.

Quantifier

Repetition, number of characters

Quantifier	Description	Samples
*	Specifies zero or more matches.	The "\w*" mask matches a string consisting of zero or more letter characters. It's equivalent to the "\w{0,}" mask.
+	Specifies one or more matches.	The "\w+" mask matches a string consisting of one or more letter characters. It's equivalent to the "\w{1,}" mask.
?	Specifies zero or one match.	The "\w?" mask matches zero or one letter character. It's equivalent to the "\w{0,1}" mask.
{<i>n</i>}	Specifies exactly <i>n</i> matches.	The "\d{4}" mask matches exactly four digits.
{<i>n</i>,}	Specifies at least <i>n</i> matches.	The "\d{2,}" mask matches two or more digits.
{<i>n</i>,<i>m</i>}	Specifies at least <i>n</i> , but no more than <i>m</i> , matches.	The "\d{1,3}" mask matches either one, or two, or three digits.

Special characters

Special characters

Character	Description	Samples
 	Alternation symbol. This can be used to implement a choice between two or more alternatives.	The "1 2 3" mask matches either "1" or "2" or "3". The "abc 123" mask matches either "abc" or "123". The "\d{2} \p{L}{2}" mask matches either two digits or two letters.
()	Grouping. You can use parentheses to create sub-expressions, or to limit the scope of the alternation.	The "(an ba)t" mask matches either "ant" or "bat". The "(net)+" mask matches "net", "netnet", "netnetnet", ... strings. Compare with the "net+" mask which matches the "net", "nett", "nettt", ... strings. The "(0 1)+" mask matches a string of indeterminate length, consisting of "0" and "1".

Examples

Input 1..9999 => Input format for property : ([1-9][1-9][0-9][1-9][0-9][0-9][1-9][0-9][0-9][0-9])

Input 0..999 => Input format for property : ([0-9][1-9][0-9])100)

Best practice: input of long string fields

The client identifies the width of an input field using the attribute **Length**. In case of long string fields with more than 20 characters, the layout can become confusing because these string fields use the complete width of the layout and are very long compared to other input fields. Very long string fields are cut off on the right-hand side, if the available space is not enough. To avoid this behavior, you can control the displayed field width regardless of the number of characters that you can enter.

- Use the attribute **Length** to specify the **width** of the input field.
- You can use a regular expression in the **InputFormat** to specify the **number of characters that you can enter**.

If you enter strings that are larger than the displayed field, the input field automatically scrolls horizontally.

Examples:

Attribute	Length	InputFormat	Effect
article.designation	50	.{0.250}	The field is displayed with a width of 50 characters. You can enter up to 250 characters.
operation.input_component_list	25	.{0.40}	The field is displayed with a width of 25 characters. You can enter up to 40 characters.

21.7.13 FillChar

Obsolete. This field must be left empty.

21.7.14 Calculation

Obsolete. This field must be left empty.

21.7.15 Further fields see ServiceParameterGui

For a description of the following fields, refer to the data types of the ServiceParameterGui:

ControlType, ControlTypeMode, ControlParameter, ControlDataSource, ControlDataSourceMode, ControlDataSourceParameter, ControlDataSourceResult, VisibleCondition and EditableCondition.

21.8 ControlDataSource

A ControlDataSource defines a data source that you can use to fill selection lists in controls, for example. These can be data logics (service requests) or reference values (see also ReferenceData).

Reference values are usually required to fill selection lists (and/or RadioGroups) with static contents.

You use data logics to request services that identify selection lists (or RadioGroups) dynamically. For example, these lists can include master data that are configured in the database.

The settings made in the columns **Parameter** and **Result** can be overwritten in a **Property** or **ServiceParameterGui**.

21.8.1 Name

Name of the ControlDataSource. The name should be composed of English terms clearly describing the data source. You usually use the camelCase notation.

21.8.2 Source

If the data source is a web service, this field contains the name of the client's data logic. You derive the data logic from the service name. To do so, remove the dot between domain and function and use a capital letter for the first letter of the function:

Service	Data logic
MDUser.list	MDUserList
MDUnits.list	MDUnitsList

In case of reference values, this field includes the **Type** of a **ReferenceData**.

21.8.3 Parameter

A list of parameters. The list does not include spaces, use semicolons to separate parameters. This field is only allowed in combination with web service data sources. A parameter can be allocated dynamically or permanently.

Permanent parameters appear as <acronym>=<value>, e.g.

"dialogconfiguration.type=AIPDEF;dialogconfiguration.type=AIPTNR".

Dynamic parameters are specified as a pair of <acronym1>=[<acronym2>]. e.g.

"resource.id=[resource.id];pdvprocessparameter.evaluation_ts=[pdvsinglevalue.evaluation_ts]"

The acronym in square brackets is replaced with the acronym values from the ControlPanel.

21.8.4 Columns

A list of requested columns. The list does not include spaces. To separate columns, semicolons are used. This is only permissible for web service data sources.

21.8.5 Result

You can enter 1-n acronyms separated by semicolon. The sequence used specifies the importance.

- Position 1 (Value): Name of acronym whose value is entered in the input field.
- Position 2 (ControlValue): Name of acronym whose value is displayed in the selection list. If you do not specify position 2, the acronym of position will be displayed.
- Position 3 (LabelValue): If you specify position 3, the value of the acronym is entered in the label field of the input field and also displayed in the selection list.
- Position 4-n: Use these positions to define additional return values, which are then used to update "dependent" controls in the client ("lookup").

Only with web service data sources:

Optional return columns of the data source, separated by semicolons. Without spaces. The return has the format <acronym>=<value> - for acronym pairs, the second acronym is therefore replaced with the result value (e.g. if you enter "operation.resource.id=resource.id", this results in "operation.resource.id=4711").

21.9 ReferenceData

Reference values are usually required to fill selection lists (and/or RadioGroups) with static contents. In contrast to values provided by web services, reference values are fixed and do not change. For this reason, reference values can be entered once in a list and are delivered in this form.

21.9.1 ref_data_key

The **ref_data_key** must be unambiguous for each entry. In special cases, this key is used in the source code (at least in the server).

Usually, the **ref_data_key** is composed of **type** + : + **db_key**; this facilitates its allocation to type and key. An exception occurs if the **db_key** includes a German expression. The **ref_data_key** must then be formed differently. For example, *pwdexclusion:person.firstname* is a super **ref_data_key** for the type *pwdexclusion.pwd* and **db_key** *PNR.PVORNAME*.

21.9.2 Type

Use this field to summarize various ReferenceData entries to a list.

21.9.3 db_key

The **db_key** is the actual value that is selected in the list. This key identifies an entry unambiguously within a **Type**. You cannot freely select the key because the key is often transferred to services and can correspond to the content of a configuration identifier in the database, for example.

21.9.4 is_default

The entry with this key is preallocated as default.

21.9.5 Designation

Text displayed in the selection list. A language key is specified.

21.9.6 sort_key

Specifies the sequence that is used to display the entries in the selection list.

21.10 Authorization

The authorization mechanism

- protects applications and functions against unauthorized use on the client,
- hides fields or field groups on the GUI,
- prevents these fields from being edited.

21.10.1 Authorization type

Controls the type of authorization. Possible values:

- Acronym: enables the authorization of individual fields (properties)
- AcronymGroups: enables the authorization to group fields
- Application: enables the authorization of applications
- Functions: enables the authorization of functions which are e.g. requested from the application toolbar.

21.10.2 Authorization Context

Context where the authorization is intended. If the field is left empty, authorization is always granted, irrespective of the context. You normally use this field to control the authorization of acronyms in the context of special services.

21.10.3 Authorization ID

Identifies the object to be authorized, i.e. the name of the acronym or the ID of an application.

21.10.4 Authorization key

The authorization key that is used to protect the object.

21.10.5 Authorization Designation

(Optional) text description of the authorization.

22 Using the Repository as Development Tool

If you use the MDS Repository as development tool to create new services, you must mind the information given in the following to correctly use the tool.

22.1 Use of transformation types for the dynamic conversion of DB or BAPI values

22.1.1 Overview

With interpreted services, you use transformation types to transform service parameters for the integration in database tables or for the integration as PDM dialog parameters.

Example: A Boolean service parameter can be mapped in the database as 1/0 and/or as J/N as parameter of the PDM dialog.

You can use transformation types for interpreted wrappers (service calling a PDM dialog) and for interpreted list services (service directly accessing the database).

The runtime interpreters integrate particular "special treatments" in their standard form (e.g. converting the string fields returned by the PDM into the data type according to repository). Other things cannot be generally integrated because they do not always work the same way (e.g. conversion of Boolean values. These values are sometimes mapped by J/N or by 1/0 in the PDM dialog or database).

The service parameters in the repository provide a column named Transformation Type. Enter the definition of the transformation in a key/value format in this column.

Example:

```
FCT=BOOLTRANSFORMATION|TRUEINVAL=N|FALSEINVAL=J|TRUERESVAL=N|FALSERESVAL=J|
```

You must always specify the value **FCT=...** that assigns the function. Depending on the function, other values must additionally be specified to configure the function.

22.1.2 Standard transformation functions

HYCOLORTORGBTRANSFORMATION

With wrappers and list services, you use this transformation to convert fields, which contain a color in PDM color code (1-16), for the return into the relevant RGB presentation as integer.

FCT-Id

HYCOLORTORGBTRANSFORMATION

Configuration parameters

Name	Description
------	-------------

Supported transformations

Transformation	Description
PDM Result	Conversion of PDM color codes from the string field into RGB as integer
List Result	Selection of the PDM color code as integer from the database and then conversion into RGB as integer

DATETIMEFILTERTRANSFORMATION

This transformation is used with list services to deposit a filter for fields, which are selected as datetime via the database function get_datetime, but which consist of two separate fields including the date and the time component in the database.

FCT-Id

DATETIMEFILTERTRANSFORMATION

Configuration parameters

Name	Description
DBFIELDDATE	Name of database field with date component (without alias)
DBFIELDTIME	Name of database field with time component (without alias)

Supported transformations

Transformation	Description
List Call	Adding a SeparateDateAndTimeFilter for the field, configured with the two specified database fields

BOOLTRANSFORMATION

You use this transformation to process Boolean fields for wrappers and list services. The processing integrates the implementation for the PDM call, the list service filter and the result conversion with both service types.

FCT-Id

BOOLTRANSFORMATION

Configuration parameters

Name	Description
TRUEINVAL	(OPTIONAL) Value for Yes (Ja) when calling PDM dialog/list filter (default J or if REALDATATYPE=integer then 1)
FALSEINVAL	(OPTIONAL) Value for No when calling PDM dialog/list filter (default N or if REALDATATYPE=integer then 0)
TRUERESVAL	(OPTIONAL) Value for Yes (Ja) when returning a PDM dialog/the selection from the database (default J or if REALDATATYPE=integer then 1)
FALSERESVAL	(OPTIONAL) Value for No when returning a PDM dialog/the selection from the database (default N or if REALDATATYPE=integer then 0). Must include a value matching the specified REALDATATYPE (J for REALDATATYPE=integer is an error)
REALDATATYPE	(OPTIONAL) Specifies the real data type of the field; integer and string are supported (default string)
NULLHANDLING	(OPTIONAL) Specifies the interpretation of null values. Possible values are none (ignore null), true (interpret null as true) and false (interpret null as false) (default none)
OTHERVALHANDLING	(OPTIONAL) Specifies the interpretation of other values (than null, the value for true and the value for false). Possible values are none (ignore others), true (interpret others as true) and false (interpret others as false) (default none)

Supported Transformations

Transformation	Description
PDM Result	Converts the string value provided by the PDM Result into Bool
List Result	Converts the selected value from the DB (integer or string) into Bool
PDM Call	Converts the true/false from the client call into the configured true/false values
List Call	Adds a filter for the DB field to filter the SQL according to data type and configured true/false values.

Examples:

Real value of string type, true=Y and false=N; null is interpreted as null (NULLHANDLING, REALDATATYPE, FALSEINVAL and FALSERESVAL need not be specified because default values are correct)

```
FCT=BOOLTRANSFORMATION | TRUEINVAL=Y | TRUERESVAL=Y |
```

Real value of integer type, true=1 and false=0; null is interpreted as null (NULLHANDLING, TRUEINVAL, TRUERESVAL, FALSEINVAL and FALSERESVAL need not be specified because default values are correct)

```
FCT=BOOLTRANSFORMATION|REALDATATYPE=integer|
```

DECIMALPLACESTRANSFORMATION

Converts the decimal places of a wrapper, e.g. "2" into a FormatString, in this case "#####0.##"

FCT-Id

DECIMALPLACESTRANSFORMATION

Configuration parameters

Name	Description
(none)	(-)

Supported Transformations

Transformation	Description
PDM Call	Conversion of a number into a FormatString, "2" -> "#####0.##".

DECIMALPLACESNUMBERTRANSFORMATION

With a list service, converts a FormatString, e.g. "0.00" into an integer, in this case "2".

FCT-Id

DECIMALPLACESNUMBERTRANSFORMATION

Configuration parameters

Name	Description
(none)	(-)

Supported Transformations

Transformation	Description
----------------	-------------

List Result	Converts a FormatString, e.g. "0.00" into an integer, "0.00" -> "2".
-------------	--

CASEINSENSITIVEFILTERTRANSFORMATION

String filter on a DB field regardless of upper/lower case

FCT-Id

CASEINSENSITIVEFILTERTRANSFORMATION

Configuration parameters

Name	Description
(none)	(-)

Supported Transformations

Transformation	Description
List Call	String Filter Case insensitive

SHIFTENDFILTERTRANSFORMATION

This transformation is used to filter by the time stamp of shift end in list services. Here, the date field or the date field + 1 day must be used, depending on whether the shift end is larger or smaller than the shift start.

FCT-Id

SHIFTENDFILTERTRANSFORMATION

Configuration parameters

Name	Description
DBFIELDDATE	Name of database field with shift date (without alias)
DBFIELDBEGIN	Name of database field with shift start (without alias)
DBFIELDEND	Name of database field with shift end (without alias)

Supported Transformations

Transformation	Description
List Call	Adds a ShiftEndDateFilter using shift date, start and end

DATATYPECONVERSIONTRANSFORMATION

This transformation is used to convert the data type between DB and web service with list services.

At present, the following are supported:

- string to integer
- string to decimal
- integer to string
- integer to decimal
- decimal to string
- decimal to integer

FCT-Id

DATATYPECONVERSIONTRANSFORMATION

Configuration parameters

Name	Description
DBTYPE	Data type in DB
WSTYPE	Data type for web service

Supported Transformations

Transformation	Description
List Call	If filters are specified in web service type, the filters are converted into DB type.
List Result	Conversion of DB type field into web service type in result.

TSPARTTRANSFORMATION

This transformation is used to identify the components of a time stamp in list services. Components are, e.g. year, day, months, calendar week,

FCT-Id

TSPARTTRANSFORMATION

Configuration parameters

Name	Description
MODE	Component to be identified (see separate table)

Supported Transformations

Transformation	Description
List Result	Conversion of DB type field datetime into the required component (as integer) in the result.

Valid Values for MODE

Name	Description
DAY	Day of month
DOY	Day of year
DOW	Day of week (0=Sunday ... 6=Saturday)
MONTH	Month
YEAR	Year
CWJ	Calendar week acc. to JAVA standard (CW1 = first complete week in year)
CWD	Calendar week acc. to DIN 1355/ISO 8601 (CW1 = week including January 4th)
CWU	Calendar week acc. to USA (CW1 = week including January 1st)
QUART	Quarter
HR	Hour
MIN	Minute
SEC	Second
MIL	Millisecond
MONTHB	Month of business year. The first month of the business year is identified via CAQ Option 1018.
QUARTB	Quarter of business year. The first month of the business year is identified via CAQ Option 1018.

Examples:

Domain	Service Function	Acronym	Is Result	Web Service Type	Conversion Method	Data Object Name	DB Alias	DB Field
								
Click here to add a new row								
TsPartTest	list	a.ts	☒	datetime		TsPartTest.list	ut	hydadm.get_datetime(%1\$.bearb_date,%1\$.bearb_time)
TsPartTest	list	a.day	☒	integer	FCT=TSPARTTRANSFORMATION MODE=DAY	TsPartTest.list	ut	hydadm.get_datetime(%1\$.bearb_date,%1\$.bearb_time)
TsPartTest	list	a.doy	☒	integer	FCT=TSPARTTRANSFORMATION MODE=DOY	TsPartTest.list	ut	hydadm.get_datetime(%1\$.bearb_date,%1\$.bearb_time)
TsPartTest	list	a.month	☒	integer	FCT=TSPARTTRANSFORMATION MODE=MONTH	TsPartTest.list	ut	hydadm.get_datetime(%1\$.bearb_date,%1\$.bearb_time)
TsPartTest	list	a.year	☒	integer	FCT=TSPARTTRANSFORMATION MODE=YEAR	TsPartTest.list	ut	hydadm.get_datetime(%1\$.bearb_date,%1\$.bearb_time)
TsPartTest	list	a.cwj	☒	integer	FCT=TSPARTTRANSFORMATION MODE=CWJ	TsPartTest.list	ut	hydadm.get_datetime(%1\$.bearb_date,%1\$.bearb_time)
TsPartTest	list	a.hr	☒	integer	FCT=TSPARTTRANSFORMATION MODE=HR	TsPartTest.list	ut	hydadm.get_datetime(%1\$.bearb_date,%1\$.bearb_time)
TsPartTest	list	a.min	☒	integer	FCT=TSPARTTRANSFORMATION MODE=MIN	TsPartTest.list	ut	hydadm.get_datetime(%1\$.bearb_date,%1\$.bearb_time)
TsPartTest	list	a.sec	☒	integer	FCT=TSPARTTRANSFORMATION MODE=SEC	TsPartTest.list	ut	hydadm.get_datetime(%1\$.bearb_date,%1\$.bearb_time)
TsPartTest	list	a.mil	☒	integer	FCT=TSPARTTRANSFORMATION MODE=MIL	TsPartTest.list	ut	hydadm.get_datetime(%1\$.bearb_date,%1\$.bearb_time)
TsPartTest	list	a.dow	☒	integer	FCT=TSPARTTRANSFORMATION MODE=DOW	TsPartTest.list	ut	hydadm.get_datetime(%1\$.bearb_date,%1\$.bearb_time)
TsPartTest	list	a.quart	☒	integer	FCT=TSPARTTRANSFORMATION MODE=QUART	TsPartTest.list	ut	hydadm.get_datetime(%1\$.bearb_date,%1\$.bearb_time)
TsPartTest	list	a.cwd	☒	integer	FCT=TSPARTTRANSFORMATION MODE=CWD	TsPartTest.list	ut	hydadm.get_datetime(%1\$.bearb_date,%1\$.bearb_time)
TsPartTest	list	a.cwu	☒	integer	FCT=TSPARTTRANSFORMATION MODE=CWU	TsPartTest.list	ut	hydadm.get_datetime(%1\$.bearb_date,%1\$.bearb_time)
TsPartTest	list	a.monthb	☒	integer	FCT=TSPARTTRANSFORMATION MODE=MONTHB	TsPartTest.list	ut	hydadm.get_datetime(%1\$.bearb_date,%1\$.bearb_time)
TsPartTest	list	a.quartb	☒	integer	FCT=TSPARTTRANSFORMATION MODE=QUARTB	TsPartTest.list	ut	hydadm.get_datetime(%1\$.bearb_date,%1\$.bearb_time)

ALPHAPERSONIDTRANSFORMATION

This transformation is used with list services to convert an alphanumerical personnel number into a numerical number for the result, and/or to filter the alphanumerical field using the numeric number.

FCT-Id

ALPHAPERSONIDTRANSFORMATION

Configuration parameters

Name	Description
------	-------------

Supported Transformations

Transformation	Description
List Call	Adds a special filter in order to filter the alphanumerical field using the numeric person.id.
List Result	Conversion of DB type string field into numeric personnel number

FIXCUTOFFNUMWORKPLACEIDTRANSFORMATION

This transformation is used to fill the result field with leading zeros. This is required with systems using numeric machine numbers and wrappers that have cut off the leading zeros. But if the client requires the complete machine number like it is included in the database, this transformation is used.

FCT-Id

FIXCUTOFFNUMWORKPLACEIDTRANSFORMATION

Configuration parameters

Name	Description
------	-------------

Supported transformations

Transformation	Description
PDM Result	Completes the relevant result field with leading zeros until 8 characters are reached, if the result field is shorter and numeric machine numbers are active.

CATEGORYLEDTRANSFORMATION

With wrappers and list services, this transformation is used to convert fields, which contain the name of an order category bitmap, for the return into the LED constant.

The assignment is as follows:

Bitmap	LED constant
fa.bmp	LED_FA
gk.bmp	LED_GK
kp.bmp	LED_KP
na.bmp	LED_NA
pj.bmp	LED_PJ
pm.bmp	LED_PM

FCT-Id

CATEGORYLEDTRANSFORMATION

Configuration parameters

Name	Description
------	-------------

Supported transformations

Transformation	Description
PDM Result	Conversion of the bitmap name from the string field of the PDM result into the LED constant
List Result	Selection of the bitmap name as string from the database and subsequent conversion into LED constant

STATUSLEDTRANSFORMATION

With wrappers and list services, this transformation is used to convert fields, which contain the name of a status bitmap, for the return into the LED constant.

The assignment is as follows:

Bitmap	LED constant
x.bmp	LED_RED
v.bmp	LED_GREY
u.bmp	LED_YELLOW
p.bmp	LED_BLUE
n.bmp	LED_PINK
l.bmp	LED_LIGHT_GREEN
f.bmp	LED_YELLOW_GREEN
e.bmp	LED_GREEN
a.bmp	LED_BLACK

FCT-Id

STATUSLEDTRANSFORMATION

Configuration parameters

Name	Description
------	-------------

Supported transformations

Transformation	Description
PDM Result	Conversion of the bitmap name from the string field of the PDM result into the LED constant
List Result	Selection of the bitmap name as string from the database and subsequent conversion into LED constant

LEGACYFULLTSTRANSFORMATION

With wrapper services, this transformation is used to map a complete time stamp to a single acronym of the dialog string. Using the default functions of the wrapper interpreter, you can only map the date component to an acronym or date and time each to separate acronyms.

The values of the time stamp are assigned in the format MM/dd/yyyy HH:mm:ss to the acronym.

FCT-Id

LEGACYFULLTSTRANSFORMATION

Configuration parameters

Name	Description
------	-------------

Supported transformations

Transformation	Description
PDM Call	Setting the complete time stamp to an acronym

LEGACYARRAYPARAMETERTRANSFORMATION

This transformation is used to support an "IN" and "BETWEEN" with wrapper services. Using the default functions of the wrapper interpreter, you can only map single values to PDM acronyms.

The list of values is converted into a string separated by separators. Each single value is also embraced by single inverted commas for the "string" web service type. The separator used between the single values can be configured. By default, it is a comma.

FCT-Id

LEGACYARRAYPARAMETERTRANSFORMATION

Configuration parameters

Name	Description
SEPARATOR	Optional: separator used, if not specified, then comma

Supported transformations

Transformation	Description
PDM Call	Conversion of value lists into a string separated by separators



Only the data types "string" and "integer" are supported!

DATEONLYFILTERTRANSFORMATION

With list services, you use this transformation to only use the date component as filter and to ignore a time component that might exist. This way, you can document in the interface that only a date component is processed and a client need not remove the time component itself.

FCT-Id

DATEONLYFILTERTRANSFORMATION

Configuration parameters

Name	Description
------	-------------

Supported transformations

Transformation	Description
List Call	Adds a filter that only uses the date component and ignores the time component.

22.2 Checklist: Repository data

The correct completion of the MDS repository is a complex task, which is occasionally prone to errors, too. The following sections might help you to avoid typical mistakes.

Term definition: *Input parameter* is an acronym, if **isFilterParameter** or **isSpecialParameter** is set. *Output parameter* or *Return value* is an acronym, if **isResult** is set to Y.

ServiceParameter: An input parameter must define at least one operator

Effect: Wrapper generator stops with the following error message: Input Parameter specified with no operator (with specification of relevant acronym and services)

Reason: An input parameter must define at least one operator.

Solution: Set at least one of the Can columns (usually CanEqual) to Y

ServiceParameter: Operator cannot exist without input parameters

Effect: Wrapper generator stops with the following error message: Operator specified for parameter that is no input parameter (with specification of relevant acronym and services)

Reason: An input parameter must define at least one operator.

Solution: Set at least one Can column (usually CanEqual) to Y.

ServiceParameter: Acronym must be input or output parameter

Description: it is not possible that an acronym is neither input nor output parameter, exception: acronym with fixed value for the wrapper.

Effect: Wrapper generator stops with the following error message: Neither input nor output parameter found (with specification of relevant acronym)

ServiceParameter: Acronym with fixed value for the wrapper

Description: Normally, an acronym is at least one of both: input or output parameter. If a wrapper must transfer a parameter with fixed value after a BAPI call and if the client does not know this, then all three columns for input and output parameters may remain unset.

Check: isFilterParameter, isSpecialParameter, isResult blank, DefaultValue must be set; HydraAcronym must be set.

Effect: if DefaultValue is not set, the wrapper generator stops with the following error message: Neither input nor output parameter found (with specification of relevant acronym)

For details, refer to section 17.1.

ServiceParameter: Acronym with fixed value for wrapper must have string data type

Description: WebServiceType for acronym with fixed value may only be string, even if DB column has a different data type.

Effect: Wrapper generator stops with the following message: java.lang.IllegalStateException: Fix value parameter with other datatype than string found: INTEGER

Solution: Declare data type in repository as string.

ServiceParameter: Reference field for *date, *time and datetime must be identical

Effect: Wrapper generator stops with the following error message: At least one component of date/time triple parameter is missing

Reason: The entries with the * types are specified for the wrapper service and must have the same reference value as the corresponding datetime entry. The reference value must be unique within a service.

Solution: set the three reference values to an equal value.

ServiceParameter: Hydra acronym must be available for an input parameter of a wrapper service

Effect: Wrapper generator stops with the following error message: Input parameter with empty Hydra-Acronym found.

Reason: The wrapper service must be able to map an acronym to the HYDRA acronym of the BAPI

Solution: Add HYDRA acronym

ServiceParameter: A wrapper service must have at least one parameter

Effect: an error is only produced if you call update, delete, lock, or unlock in the client application

Reason: Functions as delete or lock usually require a key field

ServiceParameter: Wrapper services do not have any output parameters

Effect: Description: isResult must not be set

Solution: empty isResult

ServiceParameter: WrapperServices do not have any filter parameters

Description/Solution: Set isSpecialParameter to Y, empty FilterParameter

ServiceParameter: No double acronyms within the same service

Exception: multiple ResultSets

Effect: DataLogic generator stops with the following error message: ERROR: Acronym duplicate in non-multiple result set: <acronym>! Please ensure the services.xml export doesn't contain duplicate entries!

Solution: Check each service for clear acronyms

ServiceParameter: Specify key field also for list service

Effect (e.g.): If you use Delete in the MOC application, this causes the exception "MissingPrimaryKeyException". But isKey is available in a correct form in repository, wrapper, servicex.xml and DataLogic of the delete service. The reason is that a key is missing in the list service so that the data record can be identified in the grid.

Solution: Specify isKey and isMandatory for the key fields of the list

ServiceParameter: Specify mandatory fields for wrapper

Effect: Insert service of client application complains and shows an error returned from BAPI.

Solution: Set isMandatory for the respective mandatory fields (see relevant SystemDesign document)

Service: Set service type correctly

Specific case: all services of the domain (list, insert, copy, update, delete, lock, ...) are set to JavaService although wrappers are planned.

Effect: ProjectManager/SVN Working Copy/Commit does not suggest the newly "created" WrapperServices for check-in.

Reason: The exported services.xml is empty when it is created; WrapperGenerator runs without error message, but does not generate any source code, which is completely correct.

Solution: visual diagnosis. List is often a JavaService, but the other functions of the domain are wrappers.

All tables, all columns with specific value stock: only particular specified values are permitted

Typical: V instead of Y. This is very difficult to identify in a visual check.

This can happen with Insert/Paste (Ctrl-V) into the repository.

Solution: Check columns for not permitted values (in case of visual diagnosis, use column filter selection).

22.3 InterpretedWrapper: Transfer of fixed values to PDM dialog

If fixed values must be transferred (e.g. MOD=E) with a service of type InterpretedWrapper to the PDM dialog (independent of client call), then the values must be entered as follows:

- WebServiceType: set to string.
- DefaultValue: (here the default value must be entered, E for example)
- HydraAcronym: (here the acronym must be entered, MOD for example)

The following columns MUST be empty:

- InputAsArray
- IsSpecialParameter
- IsFilterParameter
- IsMandatory
- Can....

- IsResult

23 Repository Client

You use the MPDV Repository Client MRC to display and edit repository data. It provides a user-friendly access.

23.1 Quick start

This section provides a quick overview of how to work with the Repository Client. The individual steps are only briefly described. For further information on the individual steps, refer to the sections in the following, if required.

Installation

Requirements

To use the Repository Client, you must have installed the Microsoft DotNet framework (at least version 4.5.2).

Program installation

To install the program, just copy the folder including the binary files into your system. An installation program is not required.

Installation of developer license

If the developer license is not available, you can only read the data. You cannot save or export the data.

The developer license is handed out once you have attended a respective Customizing Training. The developer license is provided as *.lic file. This file is included in the data medium that you have received during the training: Folder "Repository Client", subfolder "tools/licence", e.g.

`x:\CUT-MOC_81_files\Tools\MPDVRepositoryClient\tools\licence\mpdvWrite.lic.`

Copy the folder "licence" with its content into the folder of the Repository Client in the roaming directory of the Windows user, e.g.:

`C:\Users\%User%\AppData\Roaming\MPDV\RepositoryClient\licence\mpdvWrite.lic`

This folder is automatically created on the first start of the Repository Client.

Before you start

Before you start working with the Repository Client, you must make sure that the Repository Client has been installed according to the installation instructions and that the required license files are stored as described there.

In general, the repository is empty when you start work. However, if you do not want to start with an empty repository, it is recommended to make sure that the data you want to work with is available.

First steps

Start the Repository Client.

➔ Start `.\\bin\\mrc.exe`

Now load or create a [Workset](#).

A workset specifies the sources of the repository that you want to edit.

➔ Click the button *Load work set* in the file-based repository
-> select `workingset.work`

Click the button [Repository](#) ➔ [Load Repository](#) to load the data from the sources specified in the workset.

How to proceed further depends on what you want to do via the Repository Client.

Tip: Working with perspectives

The Repository Client provides the possibility to use different perspectives for different tasks. Do not hesitate to close views you do not need or drag them to another open view in order to tab them and hence provide for more space and clarity. You can also use more than one view of the same type. Simply adapt your perspective to the requirements of your tasks.

Example:

If you create a service and you would like to check with an existing service how to populate the fields, you can simply open another service view. Thus, you do not need to destroy your current view and re-orient yourself later.

Default perspectives

The installation of the Repository Client provides default perspectives:

default

This is a good perspective to start with. It provides a combined view for server and client-related contents.

Select a domain on the top left. Via the included relations, the top right area shows the services, servicesGUI and properties of this domain. The bottom right area shows the ServiceParameters of the service selected above, the ServiceParameterGui of the ServiceGui selected above and the ControlDataSources, ReferenceData and Authorizations of the selected domain.

default client

Similar to the "default" perspective but limited to the contents concerning client development.

default server

Similar to the "default" perspective but limited to the contents concerning server development.

DB schema

Shows documentation of the database structure.

Validation

Use this perspective to validate contents.

Proceed as follows:

- Open perspective and load data from workset.
- Perform validation: tab *Repository* --> button *Validate*.
- A CSV file listing the identified irregularities is generated in the sub-directory "validation_logs" of the Repository Client's installation directory. The application that is linked to this file type in the operating system opens the CSV file.
- In the views for Domains, Properties, ServiceParameters, etc. the Repository Client only shows the entries with detected irregularities.

You should analyze and, if necessary, correct the detected irregularities. Not every irregularity leads to an error.

23.2 Start and exit Repository Client

Start the Repository Client via the Windows start menu, a link on the desktop or the command line. As soon as all required components have been loaded, the application window is shown.

You can start and run the Repository Client multiple times in parallel on a PC. You can access different repository data in each of the started instances of the Repository Client. For example, you can view several versions of the repository at the same time.

You can also start the client by opening one of the workset files. On start of the client, the system attempts to load the contents of the workset defined in this file. This option is available after the first start of the Repository Client.

Command line parameters

You may transfer parameters to the Repository Client upon the start. The following parameters are supported:

`-perspective/-p <perspectivename>` Use this parameter to start the Repository Client with a specific perspective. If you do not use this parameter, the last active perspective is started by default.

`-workset/-w <worksetfile>` Use this parameter to specify the workset to be loaded. If you do not specify the workset on start of the application, the last loaded workset is loaded.

`--autoload/--a` If you use this parameter, the Repository Client loads the repository defined in the last active workset or in the workset transferred via parameters. This repository is loaded directly on start of the application.

`--trim/--t` If you use this parameter, the Repository Client removes so-called leading and trailing "whitespaces" when loading the repository. Only select this option if needed, because the load time increases extremely.

Exit

To exit the application, click  in the title bar.



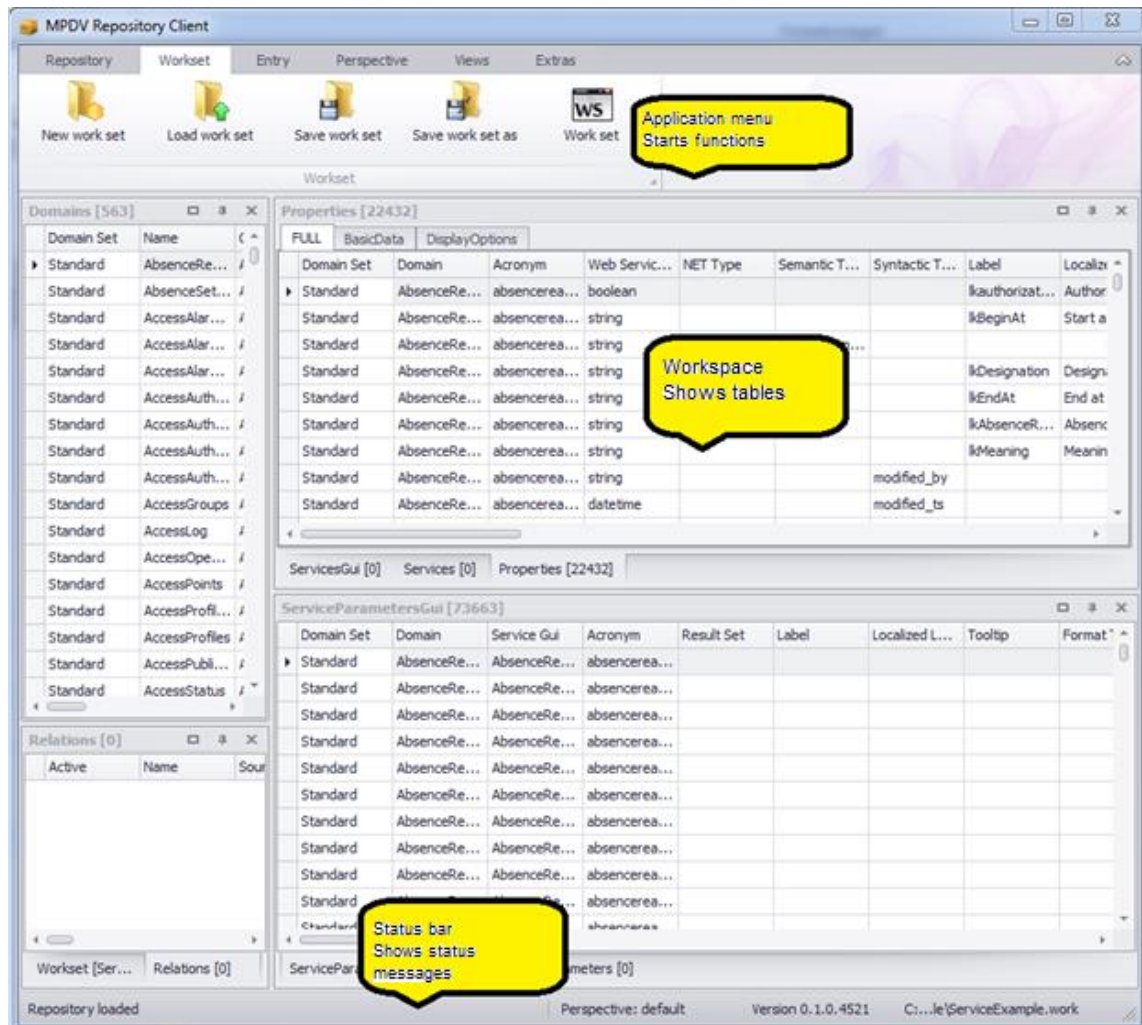
If the loaded repository includes active changes when you exit the application, a respective message is issued asking you to save the changes. If you do not want to save the changes, you can discard the changes or stop exiting the application.



Note: Changes to the workset and perspective are discarded when you exit the application, if you have not saved the changes.

23.3 The Application Window

The application window forms a framework for the display of different tables. It includes the application menu with control elements to call and control different functionalities. The menu is on top of the window. A status bar is at the bottom of the window. The status bar shows progress and event messages.



You can individually dock the grids/table views. To do so, click the title bar of a table view/grid and drag it out of the docking position. For orientation purposes, the system shows the docking positions where you can drop the table view. You can also drop a table view without docking it.

23.4 Grids/table views

Grids/table views are components to present data records in a table. You can change the tables in the Repository Client according to your requirements. For each grid/table view, the functions described below are available.



The settings, that you make in a table, are saved with the perspective. To undo changes, you can switch to the standard perspective (in the application menu: *Perspective* → *Change perspective*).

Sort table data

Click the table header to sort table data in descending order. If you click the table header once more, data is sorted in ascending order. The selected sorting option is shown.

You can sort data by several columns: Press the *Shift* key of your keyboard after sorting the first column. Then click the other column headings by which you want to sort.

You can also use the context menu of the table header to sort data.

Group data in the table

You can group table data if the *group by* area is shown. If the *group by* area is not shown, you can show the area via the context menu of the table header (*Show/Hide group by box*). To group by a column, click the column header and drag it to the grouping pane. Multiple grouping is also supported.

Optimum column width (best fit)

Select the option "*Best fit*" in the context menu of the table header to adjust the column width of the selected column to the optimum width. In this case, "optimum" means that the column is as wide as the largest entry in the selected column.

Optimum column width (all columns) / Best fit (all columns)

Click this function to adjust all columns to the optimum width.

Change column width

You can also change the column width using the mouse, i.e. move the space between two cells to the left or right.

Show and/or hide columns and entire categories

Use the context menu function *Select columns* to show and/or hide individual columns and entire categories. For this purpose, select the function in the context menu and then drag the required columns and/or categories from the table to the pool or from the pool to the table.

Change the sequence of columns and categories

Also use the mouse to change the display order of columns and categories. To do so, drag the column or category you want to move and drop it at the required location. The system will indicate the location to which the column and/or category will be allocated when you release the left mouse button.

Freezing columns to prevent horizontal scrolling

You can freeze columns at the left and right-hand side to keep the columns in view while scrolling. These column settings are included in the perspective and can be saved with the perspective. Right-click the column header and press one of the below-mentioned shortcuts to freeze columns:

- CTRL + right click: freeze at the left-hand side.
- ALT + right click: freeze at the right-hand side.
- SHIFT + right click: Unfreeze.

Filter table data

Click the filter icon  in the column where you intend to set the filter and select the required filter option from the list; i.e. you select one of the values available in the table or compose a combination of values in the *user-defined* filter.

You can also set several filters in different columns. The table footer indicates that the table has been filtered and also shows the filter criteria. Select the function *Edit filter* on the right of the footer to open the filter editor. Use the filter editor to create complex filter criteria across all columns. You may also open the filter editor via the context menu of the table header.

Search box

Use the context menu of the table header to access the option *Show search box*. This option provides a search box within the table. Use this box to quickly search and/or filter the requested data. Simply start typing in this box and the system will only show those rows matching the data you typed. The more characters you enter, the more you narrow down the result.

Filter row

Use the context menu of the table header to open the option *Show filter row*. This option provides an additional row shown below the table header. You can enter a search term in any column, and the system will narrow down the displayed rows appropriately. The system supports wildcards. You can also combine search terms in various columns to restrict the search result.

Edit rows

Double-click a row or press the "Enter" button to switch to the editing mode and edit the respective row. When you have finished editing and leave the row, the editing mode is terminated. Edited rows are highlighted in color.

23.5 The application menu

You can use the application ribbon menu of the Repository Client to control various functions of the tool. It includes several tabs that are described in the following.

Workset

Includes functions to administer worksets. A workset specifies the sources included in the repository that you want to edit. To display a workset, use the workset panel which consists of a grid/table view.



- **New workset:** This function creates a new workset. If a workset is loaded that has been modified, a dialog pops up asking you to save the changes.
Click "Yes" to save the changes, "No" will discard them. In both cases, a new workset is created. Click "Cancel" to cancel the process of creating a new workset.
- **Load workset:** This functions loads a workset from an existing file. You can select the workset to be loaded via a file dialog. If the currently loaded workset has been modified, you can save the changes as described above.
- **Save workset:** This function saves the current workset in the file from which it was loaded. If the current workset is new, a file dialog pops up asking you to select the file in which you want to save the workset.
- **Save workset in:** Use this function to save the currently loaded workset in a file. You can select the files using a file dialog.
- **Workset:** Use this button to show and/or hide the grid/table view presenting the currently loaded workset. For details on the workset table, please refer to section Workset.

Repository

Includes functions to load and save data in the repository. In addition, you may display repository-specific data here.



- **Load repository:** Use this function to load the repository. The currently loaded workset specifies the data sources that are used to load the repository. If a repository is loaded that has been modified, a dialog pops up asking you to save the changes. Click "Yes" to save the changes, "No" will discard them. In both cases, the repository will be reloaded subsequently. Click "Cancel" to cancel the process of loading a repository.
- **Save repository:** ***only available if used in development mode**
Use this function to save changes in the repository. If no changes have been made, an appropriate note will be displayed.
- **Export repository:** ***only available if used in development mode**
In contrast to saving the repository, you can use this function to export parts of the repository. For details on this function, please refer to section **Error! Reference source not found.**
- **Validate:** ***only available if used in development mode**
You can use this function to validate your data records manually. (See section "Validation").
- **Value list:** Use this menu entry to show and/or hide the value list. The list includes permissible entries for specific fields of the repository.
- **References:** Use this entry to show and/or hide the table with repository references. For details on references, please refer to section References.

- **Changes: *only available if used in development mode**

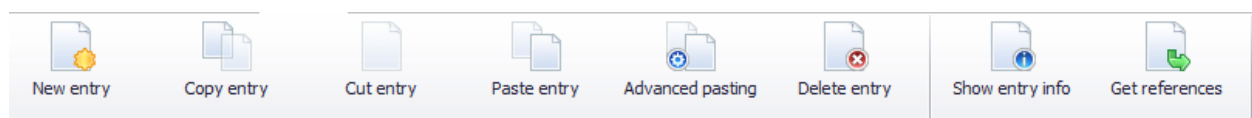
Use this button to show and/or hide the change view. This view shows the current modifications in the loaded repository.

- **Service documentation**

Use this button to show the extended documentation of selected standard services. For further information on the service documentation, refer to section "23.9 Service documentation".

Data collection

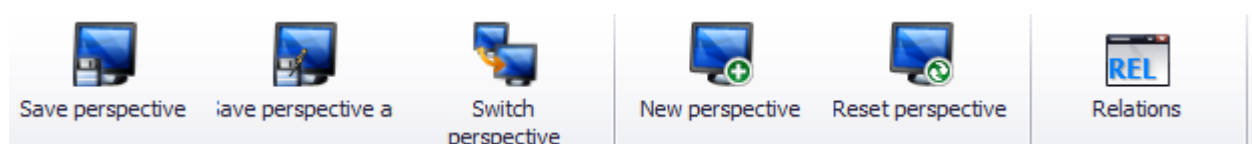
The *Entry* tab summarizes the functions that you can use to edit the loaded repository. The entries refer to the currently focused table view/grid.



- **New entry:** Use this function to create a new entry. For details on this function, please refer to [Context menu → New](#).
- **Copy entry:** Use this function to copy selected table entries. For details on this function, please refer to [Context menu → Copy](#).
- **Cut entry:** Use this function to cut selected table entries. For details on this function, please refer to [Context menu → Cut](#).
- **Paste entry:** Use this function to insert (paste) entries from the cache/clipboard. For details on this function, please refer to [Context menu → Insert](#).
- **Advanced pasting:** Use this function to edit entries in the clipboard prior to inserting them. For details on this function, please refer to [Context menu → Advanced pasting](#).
- **Delete entry:** Use this function to delete the selected entries. For details on this function, please refer to [Context menu → Delete](#).
- **Show entry info:** Use this function to open a dialog showing information on the currently selected entry. For details on this function, please refer to [Context menu → Info](#).
- **Get references:** Use this function to open a new grid/table view showing the currently selected data record including referenced values. For details on this function, please refer to [Context menu → Get references](#).

Perspective

These entries of the menu refer to the administration of perspectives. A perspective is a layout of table views/grids and includes also the associated relations between table views/grids.



- **Save perspective:** Use this function to save the currently shown arrangement of tables.
- **Save perspective as:** Use this function to save the currently shown perspective under a different name. You can enter the new name in the displayed dialog.
- **Switch perspective:** Use this function to change the perspective. A dialog with all available perspectives is shown.
- **New perspective:** Use this function to create a new perspective. Similar to the "Save perspective as" function, you can select the name of the perspective in a dialog. After entering the name, you can immediately switch to the new perspective.
- **Reset perspective:** Use this function to reset the current perspective to the status saved last.
- **Relations:** Use this menu entry to show/hide the grid/table view showing the relations between the grids/table views. For details on relations, please refer to section Relations.



Note: Changes to the perspective are discarded when you exit the application, if you have not saved the changes explicitly.

Views

Use these entries to open table views/grids. The entries will open a new grid/table view each showing the relevant data records of the repository.



For clear identification of the data records shown in the tables, the *Parent* column is included in each of the tables. The *Parent* column includes the identifier for the father node in the repository tree. The other columns of these tables are defined by the repository documentation.

The *View* area additionally includes a group with entries for the remaining grids/table views of the application.

23.6 Workset

Worksets define a set of data sources that make up the repository that you want to edit. Use the workset management function to organize your work on different projects and create an appropriate workset for each of your projects. You can show/hide the workset table via the application menu ([Workset](#) → [Workset](#)).



Note: The workset loaded last will be loaded on start of the Repository Client.

Workset [example.work]							
Name	Server Source	Client Source	Prio...	Is Writeable	Active	Overrides	M
ZIP	D:\DevRepo\ServerDomains.zip	D:\DevRepo\ClientDomains.zip	1	☐	☐		
Runtime	\\servername\hydra1\jhydradir\MOC\1	C:\Program Files (x86)\MPDV\HYDRA 8\MOC	2	☐	☒		
▶ Dev	d:\DevSrc\Repository\Data\server	d:\DevSrc\Repository\Data\client	3	☒	☒		

The workset table includes the following columns:

Name

You can specify the domain set that you want to use to load the data source. The repository data include the information on the domain set that was used to load the data. You can copy data from a domain set into another domain set. Example: Copy existing applications from the domain set "Runtime" (read only) into your development directory, e.g. the domain set "Dev" (writable) where you can make changes.

Client Source

You can specify the data source that you want to use to load client data. The following options are available:

- Load data from the runtime structure of the client reference in the server
In the server, the client configurations are stored as client reference with runtime structure. You can load the repository data from this structure.

Example:

HYDRA: x:\jhydradir\MaintenanceManager\rt\client\MOC

MIP: x:\wsp_config\MaintenanceManager\rt\client\MOC

The access is read-only.

- Load data from the runtime structure of a local MOC client
If you enter a path to an MOC installation directory, the respective client data are directly loaded from the MOC runtime installation.

Example:

C:\Program Files (x86)\MPDV\HYDRA 8\MOC

The access is read-only.

- Load data from a local directory with domain structure
You use local directories with domain structure for the administration of your own developments. Using the Repository Client, you can read data in this directory and you can also save data into this directory.

Example: d:\DevSrc\Repository\Data\client

- Load data from a ZIP archive
You can enter a ZIP file (including path) that includes the data in domain structure. MPDV provides the ZIP archives as part of the trainings or on the support portal.
The access is read-only.

Server Source

You can specify the server source that you want to use to load the server-specific data. The following options are available:

- Load data from the runtime structure in the server

You can read the configurations for the server in a server installation. To this end, the configuration directory of the web service provider (WSP) up to the instance number is specified.

Example:

HYDRA: \\<servername>\<install_dir>\jhydradir\MOC\1

MIP: \\<servername>\<install_dir>\jdir\MOC\1

The configuration is loaded from the standard scope by default. As an alternative, you can also load the configuration from the *custom* scope, if you enter the value "custom" in the field "name".

The access is read-only.

- Load data from a local directory with domain structure

You use local directories with domain structure for the administration of your own developments. Using the Repository Client, you can read data in this directory and you can also save data into this directory.

Example: d:\DevSrc\Repository\Data\server

- Load data from a ZIP archive

You can enter a ZIP file (including path) that includes the data in domain structure. MPDV provides the ZIP archives as part of the trainings or on the support portal.

The access is read-only.

Priority

The priority of a data record specifies the loading sequence of sources that are allocated to the same data record. In the above example, data is first read from the local development directory "d:\DevSrc\Repository" and then from the runtime installations "Server" and "Client". Data records with low priority are overridden and consequently not loaded.

Is Writeable

In this column you can specify if a data source grants write access. You only require write access in workset entries where you want to make local developments.



Please note that ZIP archives do not grant write access. For this reasons, you must not enable "is Writeable" in case of a ZIP data source.



Please note that it is not supported to save data in the runtime structure (client directory or server runtime directory). You must therefore not enable "Is Writable" for data sources with

runtime structure.

Overrides

You can specify in this column which domain set is overridden by the current one. This affects the resolving of references (for details please refer to section References).



An entry in the "Overrides" column does not have any effect on the loading of the data sources. See column "Priority".

Active

Use this option to enable or disable an entry.

23.7 Relations

Relations are a property of perspectives and define table filters. Tables that include relations are dynamically adapted to the selected values of another table by setting a filter. For example: Using relations, you can specify that only the service parameters of the service currently selected in another service view are displayed in a service parameter view. The *Relations* table lists the relations of the current perspective. You can call this table via the application menu (*Perspective* → *Relations*).

Relations [13]								
Active	Name	Source	Target	Filter	Var0	Var1	Var2	
☒	Authorizations for Domain	Domains	Authorizations	[Parent] = 'id->Authorizations'				
☒	ControlDataSources for Domain	Domains	ControlDataSources	[Parent] = 'id->ControlDataSources'				
☒	DataObject for Service	Services	Dataobjects	[DomainSet] = '\$0' and [Domain] = '\$1' AND [Name] = '\$2'	DomainSet	Domain	Name	
☒	Properties for Domain	Domains	Properties	[Parent] = 'id->Properties'				
☒	ReferenceData for Domain	Domains	ReferenceData	[Parent] = 'id->ReferenceData'				
☒	SchemaColumns for ServiceParameter	ServiceParameters	SchemaColumns	((TableName) = '\$0' AND [ColumnName] = '\$1' AND [TableNam...	DBTabelle	DBField	HydraAcronym	
☐	ServiceParameter for ServiceParameterGui	ServiceParametersGui	ServiceParameters	[Domain] = '\$0' AND [Service] = '\$1' AND [Acronym] = '\$2'	Domain	ServiceGui	Acronym	
☒	ServiceParameters for Service	Services	ServiceParameters	[Parent] = 'id'				
☒	ServiceParametersGui for Service	Services	ServiceParametersGui	[DomainSet] = '\$0' and [Domain] = '\$1' AND [ServiceGui] = '\$2'	DomainSet	Domain	Name	
☐	ServiceParametersGui for ServiceGui	ServicesGui	ServiceParametersGui	[Parent] = 'id'				
☒	Services for Domain	Domains	Services	[Parent] = 'id->Services'				
☐	Service forServiceGui	ServicesGui	Services	[Name] = '\$0'	Name			
☒	ServicesGui for Domain	Domains	ServicesGui	[Parent] = 'id->ServicesGui'				

The following columns are displayed:

Active

This checkbox specifies if the relation is used.

Name

The name of the relation – free choice.

Source

The table and its selection that are used to set the filters. If you edit an entry in this column, the currently possible assignments, i.e. all currently existing views are presented in a selection box.

Target

The table where the filter is applied. If you edit an entry in this column, the currently possible assignments, i.e. all currently existing views are presented in a selection box.

Filter

You can store the filter expression here that you want to apply to the target table. Variables ranging between \$0 and \$9 are supported. These variables are dynamically filled with the values of the columns Var[0-9].

Var[0-9]

In these columns, you can specify the columns of the source table that are used to adapt the filters dynamically.

As soon as a correct (and activated) relation is entered in this table, it is applied. If you close one of the referenced views of a relation, the relevant view is removed from the relation. If this results in a double entry in the *Relations* table, this entry is removed. You can therefore use this view to administer the relations between concrete table instances and to administer unbound relations that can serve as template for relations.

23.8 References

References show the inherent connections between data records of the repository. They are defined by the repository structure and cannot be edited in the Repository Client. For example: A value in the column "Syntactic Type" of the *Property* table references another data record in the *Property* table. The *References* table lists the defined references and may be activated via the application menu ([Repository](#) → [References](#)).

Name	Source	Source Column	Dependency	Condition	Target	Filter	Priority
ControlDataSource1	Property	ControlDataSource	ControlDataSourceMode	Lookup	ControlDataSource	[Name] = \$value and [Domainset] = \$domset	-1
ControlDataSource2	Property	ControlDataSource	ControlDataSourceMode	Reference	ReferenceData	[type] = \$value and [Domainset] = \$domset	-1
SyntacticType	Property	SyntacticType			Property	[Acronym] = \$value	0
SemanticType	Property	SemanticType			Property	[Acronym] = \$value	1
Acronym1	ServiceParameter	Acronym			Property	[Acronym] = \$value	3
Acronym2	GuiServiceParameter	Acronym			Property	[Acronym] = \$value	3

The following columns are displayed:

- **Name:** Name of the reference
- **Source:** The repository object type that can include this reference.
- **SourceColumn:** The source type property that can include the reference.
- **Dependency:** Source type property specifying the reference target.

- **Condition:** Value that the property specified under *Dependency* must have. Only then, the reference is pursued. For example: The value of *ControlDataSourceMode* (lookup, reference) specifies the target of the reference (*ControlDataSource* or *ReferenceData*) which is specified in the *ControlDataSource* property.
- **Target:** The repository object type that is referenced.
- **Filter:** Filter that selects the referenced data from the overall quantity of this type of data. Such a filter can include the variable *\$value* (value of column *Value* in the current row) and *\$parent* (value of column *Parent* in the current row).
- **Priority:** Specifies the priority of the reference. You can find further details in section "Get references".

References provide two general functions:

Show reference: You can use this function to display the referenced data in a new table. You can call this function via the context menu ([Context menu](#) → [Show reference](#)).



Note: This function is only available in the context menu of cells which can include references.

Get references: Use this function to complete missing values of a data record with values of referenced data records. For example: You can use this function to show the inherited values of a property of the *SemanticType* or the *SyntacticType*.

You can call this function via the context menu ([Context menu](#) → [Get references](#)) of the table views/grids. A new data record is generated and shown in a new panel. The generated data record is a copy of the currently selected data record. The values that are not filled are filled by those in the referenced data records. The reference priority specifies the filling sequence.

23.9 Service documentation

The Repository Client contains an extended documentation for selected standard services. The documentation mainly includes services released in the system and to be used with the Service Interface (SCS-SIF).

The documentation is available as of MRC version 1.8.STD.65500 (beginning of 2019).

There are two options to access the service documentation:

- In the toolbar "Repository", you can use the button *Service Documentation* to open the table of contents of the services included. Via hyperlinks, you can navigate to the different services.
- In the table views "Services" and "ServicesGui", you can use the context menu to open the documentation of the selected service if it is available.

**Printing a service documentation:**

You can print the service documentation using the shortcut Ctrl-P.

24 Using the Repository Client as Development Tool

The Repository Client is not only used as service documentation. You can also use it to edit the repository data to create new services.

24.1 How to create new contents

The data structure of the repository is hierarchical. This means that a service is always part of a domain, a service parameter is always part of a service and properties are always the children of a domain.

This again means that you always work in a "top-down" view when working in the repository. For example, if you want to create a new service, you must ensure that the domain where you want to create the service does actually exist. If you want to create a service parameter for a new service, this one should also exist at this time, etc.

If you keep this basic information in mind and design your workflow on basis of this structure, you will spare a lot of unnecessary work and frustration.

¶ If you want to make changes in the repository, it is recommended to create another domain set within the work set to manage your modifications. ¶ Do not forget to check the "IsWriteable" option – otherwise you will not be able to save any changes later on.

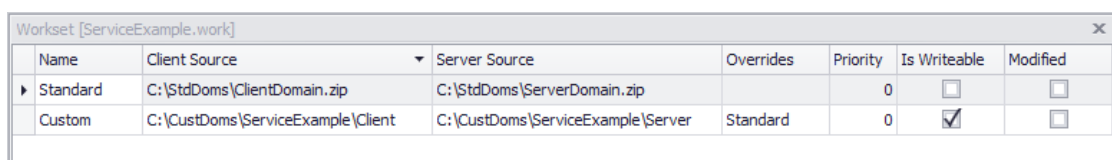
Please also note that the workset is not part of the repository data model and changes in the workset will only become effective upon re-loading the repository. We therefore recommend that the workset is defined for the imminent task, first.

Prior to starting your work, it is reasonable to make yourself familiar with the [Relations](#).

Example: Creating new services

The use case in the following illustrates the workflows that are involved in the generation of services.

1. Import the latest repository version.
2. Open the repository client.
3. Create a new [Workset](#) with two domain sets (standard, custom).



Name	Client Source	Server Source	Overrides	Priority	Is Writeable	Modified
Standard	C:\StdDoms\ClientDomain.zip	C:\StdDoms\ServerDomain.zip		0	☐	☐
Custom	C:\CustDoms\ServiceExample\Client	C:\CustDoms\ServiceExample\Server	Standard	0	☒	☐

4. Save the new workset.
5. Load the repository.

6. Create a new domain via the context menu (right click the domain view→New) and edit the domain data (Name = "U_ServiceExample").

Parent	Name	Client	Server	Modified
?				
Standard	Workplacebooking	Workplacebooking	Workplacebooking	
Standard	WorkplaceOverview	WorkplaceOverview		
Standard	WorkplaceStatusAssignment	WorkplaceStatusAssign...	WorkplaceStatusAssign...	
Standard	WorkplanOrderManagement	WorkplanOrderManage...	WorkplanOrderManage...	
Custom	U_ServiceExample			

7. Copy other services to be used as model via the context menu:

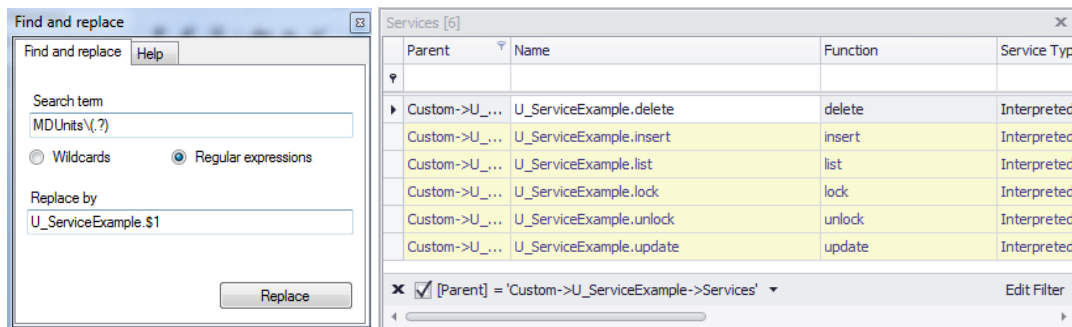
Domain Set	Domain	Name	Function	Service Type	List Mode	DLG	System Call	Parent	Description	Mo
Standard	AbsenceRe...	AbsenceRe...	delete	Interpreted...		FGR.DELETE		Standard->...	Fehlgründe	
Standard	AbsenceRe...	AbsenceRe...	insert	Interpreted...		FGR.INSERT		Standard->...	Fehlgründe	
Standard	AbsenceRe...	AbsenceRe...	list	Interpreted...				Standard->...	Fehlgründe	
Standard	AbsenceRe...	AbsenceRe...	lock	Interpreted...				Standard->...	Fehlgründe	
Standard	AbsenceRe...	AbsenceRe...	new	Interpreted...				Standard->...	Fehlgründe	
Standard	AbsenceRe...	AbsenceRe...	unlock	Interpreted...				Standard->...	Fehlgründe	
Standard	AbsenceRe...	AbsenceRe...	update	Interpreted...				Standard->...	Fehlgründe	
Standard	AbsenceSet...	AbsenceSet...	delete	Interpreted...				Standard->...	Steuerung ...	
Standard	AbsenceSet...	AbsenceSet...	insert	Interpreted...				Standard->...	Steuerung ...	
Standard	AbsenceSet...	AbsenceSet...	list	Interpreted...				Standard->...	Steuerung ...	

8. Select the U_ServiceExample domain in the domain view. The selected domain becomes the active domain which is used to filter the services. (This is only possible with an active relation from domain to service).

9. The services can be inserted using the context menu in the service view. The active filter (set before) defines which of the copied services can be added to the new domain. All included service parameters are automatically copied at the same time.

Of course you can also use proven key combinations, e.g. Ctrl+C for copying, Ctrl+X for cutting, as well as Ctrl+V for pasting/inserting.

10. At this point, an adjustment of the service names is required. For example, you can change the names using the Find and Replace dialog that is also available via the context menu:



11. Adjust / remove / add service parameters in known manner.
 12. Save.

The files have been written to the specified location in the hard disk.

13. *Optional:* Export

You can directly write into a structure, which you can use to directly test your changes.

This example could well have been extended. But to directly start work with the client, the example used illustrates the major steps to get a first idea.

At this point, you might ask how you have to proceed with the GUI part of the services. Of course you could also copy them into the new domain and change them. Other option: right-click the domain to create the GUI part of the services automatically and to add potentially missing properties from the created services.

It might be easier to copy the complete domain and to simply delete the elements that are not required.

One level further down, the properties of the structure become even more evident. If only a few service parameters of a service are required for a new one, it might be easier to copy the complete service and to delete the excessive parameters. This spares the entire "Creation" of a new service.

24.2 Context menu of the table view/grid

A context menu opens if you right-click the tables. The menu includes different entries depending on the type and status of the table view.

New

Use this function to add a new row to the table view. In the columns with set filters, the values will be set according to the filters in the new row. If, for example, a filter is set to "LIKE Test%" or "= Test", the cell value is set to "Test". Advanced filters are not supported.

Info

Click *Info* to open an InfoPanel. The panel is bound to the source table and shows information on the selected data record in the source table. In addition to a clear identification of the data record, the data source from which it was loaded is shown. The entry *Children* shows the number of data records allocated. In the given example, the service parameter has 2 children service parameters. In addition, the service attribute values are listed in a table. In the bottom area of the dialog, the description stored for the data record is shown.

The 'Information' dialog box displays the following details for a 'GuiServiceParameter':

- Type: GuiServiceParameter
- Key: Standard->AbsenceReasons->ServicesGui->AbsenceRe
- Source: AbsenceReasons\data\AbsenceReasons.GuiConfigurati
- Children: 0

Show	Key	Value
☒		
☒	DomainSet	Standard
☒	Domain	AbsenceReasons
☒	ServiceGui	AbsenceReasons.delete
☒	Version	
☒	Acronym	absencereason.company
☒	ResultSet	
☒	Label	
☒	LocalizedLabel	
☒	Tooltip	
☒	FormatType	
☒	ClientDefaultValue	
☒	IsKey	Y

Filter: ☒ [Show] = 'Checked' Edit filter

Description:

Copy

Deposits selected rows into the clipboard. This option is only offered if the view contains data.

Cut

Deposits selected rows into the clipboard and subsequently deletes them. This option is only offered if the view contains data.

Delete

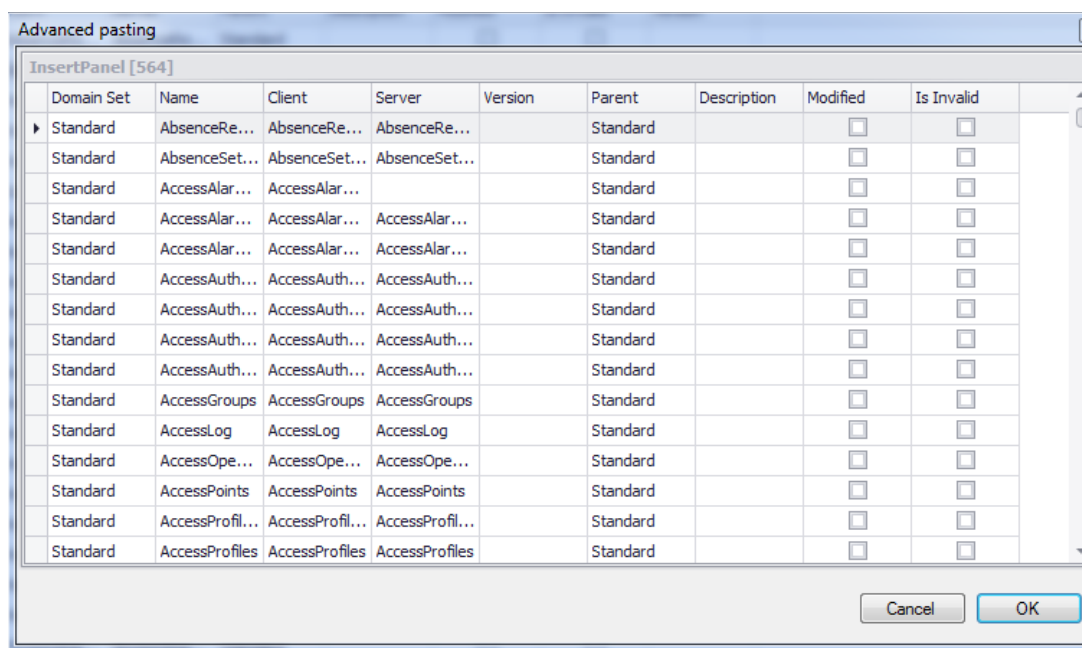
If you select this function, a dialog listing the data records to be deleted is shown. Click "Yes" to delete them; "No" will cancel the deletion process.

Insert

This function adds rows from the clipboard to the grid. This option is only offered if the cache/clipboard contains data which may be inserted in the currently shown table.

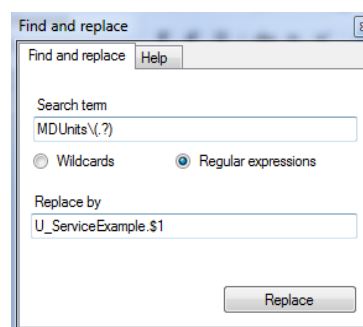
Advanced pasting

Contrary to 'Insert', this function opens a dialog that allows to edit the entries in the cache/clipboard before you insert them. It is possible to allocate new values to individual cells and to all cells of a column. You can cancel the *Insert* process. You can only select this option if the cache/clipboard contains data which may be inserted in the currently shown table.



Find and replace

Use this function to find and replace values within a column. If you select this function, the following dialog opens:



Specify the search term and the term that should replace the search term and confirm by "Replace". For example, use this dialog to replace prefixes. In addition, this function supports regular terms.

Relations

This entry leads to a list with identifiers of relations that are defined in the relations table. If you select one of these identifiers, a list of the currently shown tables opens. Select one of these tables to instantiate this relation. A new relation of this type is automatically created; the source is set on the current table and the target on the selected table, respectively. For details on the semantics of relations, please refer to section Relations.

Show reference

You can use this function to open a table view listing the data to which the entry of the current cell refers. For details on the semantics of references, please refer to section References.

Get references

This entry opens a new table which will fill the current data record with values from the referenced data records. For details on the semantics of this function, please refer to section References.

Create GUI configuration

This entry is only shown in the context menu of the domain panel. Use this function to create the ServicesGUI according to the services of the selected domain.

Create properties

Also this entry is only available in the domain panel. Use this entry to create properties according to the ServiceParameterGui.

Create service based on SQL

Also this entry is only available in the domain panel. You can use this function to generate services. You use an SQL statement to extract information on fields and tables.

- A "select" statement generates a service of type InterpretedJavaService.
- A "create table" statement generates services of type InterpretedBapiService to edit data and a list service of type InterpretedJavaService to show the respective data.

The information included in existing parameters in the repository is added to the information on the individual fields, if possible (the allocation is based on the table and the field name).

Note: This function only helps to create services. It is up to the user to ensure the correctness.

Example 1 ("select" statement)

```
select m.masch_nr,
       m.bez_lang,
       k.bezeichnung,
       k.sap_logical_system
from   maschinen m
       left outer join kostenstellen k
           on k.kostenstelle = m.kostenstelle
```

This statement is used to generate a list service. No existing acronym for the column `k.sap_logical_system` can be found. For this reason, the column is marked in the "acronym" with "<TODO>" and the acronym must be defined manually (delete <TODO>). Then you can run the list service.

Example 2 ("create table" statement)

```
create table u_test_table
(
    test_string char(20),
    test_date   date,
    test_integer integer,
    test_decimal decimal(18, 6),
    test_serial serial
);
```

This statement requires more manual rework:

- Check acronyms
- If you use the database type "serial": with database type "serial", you must assign WebServiceType=integer. For the services delete, lock, unlock and update, you must include the constraint "SERIAL|" in the serial column and specify it as mandatory parameter (IsMandatory=Y). For the service insert, make the settings IsMandatory=N, IsResult=Y and IsSpecialParameter=N.
- For the editing services, you can define key columns (except for serials) if required (Constraint "KEY=n|") and define them as mandatory parameters (IsMandatory=Y)
- The services delete, lock and unlock only require the key columns (constraint "KEY=n" or "SERIAL|"). Delete the columns that are not used.
- Check WebServiceType with all service parameters and complete, if required.
- Also respect the further notes on the generation of services in this document.

Then you can run the service.

Operator Assistant

If a service has a lot of parameters, it is a complex task to set the columns for the operators supported by the service parameter in the table view of the service parameters. To facilitate this task, an assistant exists to manage the supported operators.

Start an *Operator Assistant* in the context menu of the *ServiceParameters* view.



The *Operator Assistant* is available in the MPDV Repository Client as of version 1.8.STD.66280.

You can also copy the supported operators from one service parameter to another in one operation and use the function *Find and replace* for all operators.

Operator Assistant

To start the *Operator Assistant*, select a row in the view *ServiceParameters* and right-click to open the context menu. Select the entry *Operator Assistant*.

In this dialog, you can edit all operators of the service parameter and use the option *InputAsArray*. If you click the OK button, the settings are applied to the columns in the table view of the service parameter.

Button *Default for WebServiceType*

The options are predefined according to the *WebServiceType*.

Button *Reset all*

All options are deactivated.

Copying operators from one *ServiceParameter* to another



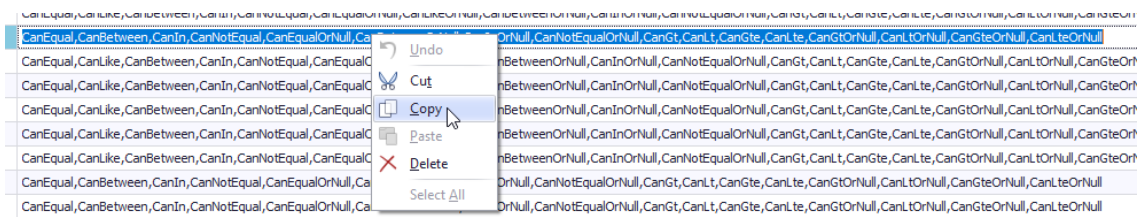
The *Operator Assistant* is available in the MPDV Repository Client as of version 1.8.STD.66280.

The table view of the *ServiceParameters* provides a column *Operators*. You can use this column to manage the options using the *Operator Assistant* and to manually edit the operators of a *ServiceParameter* in a single column of the table. Changes to the column *Operators* and to the different columns *CanEqual*, *CanLike*,... are automatically synchronized.

Acronym	Service Type	Key	Operators
bonusreason.designation	interpretedJavaS...		CanEqual,CanLike,CanBetween,CanIn,CanNotEqual,CanEqualOrNull,CanLikeOrNull,CanBetweenOrNull,CanInOrNull,CanNotEqualOrNull,CanGt,CanLt,CanGte,CanLte,CanGtOrNull,CanLtOrNull,CanGteOrNull,CanLteOrNull
incentivebonus.authorized	interpretedJavaS...		CanEqual,CanLike,CanBetween,CanIn,CanNotEqual,CanEqualOrNull,CanLikeOrNull,CanBetweenOrNull,CanInOrNull,CanNotEqualOrNull,CanGt,CanLt,CanGte,CanLte,CanGtOrNull,CanLtOrNull,CanGteOrNull,CanLteOrNull
incentivebonus.authorizingperson	interpretedJavaS...		CanEqual,CanLike,CanBetween,CanIn,CanNotEqual,CanEqualOrNull,CanLikeOrNull,CanBetweenOrNull,CanInOrNull,CanNotEqualOrNull,CanGt,CanLt,CanGte,CanLte,CanGtOrNull,CanLtOrNull,CanGteOrNull,CanLteOrNull
incentivebonus.bonusreason.id	interpretedJavaS...		CanEqual,CanBetween,CanIn,CanNotEqual,CanEqualOrNull,CanBetweenOrNull,CanInOrNull,CanNotEqualOrNull,CanGt,CanLt,CanGte,CanLte,CanGtOrNull,CanLtOrNull,CanGteOrNull,CanLteOrNull

Proceed as follows to copy the operator options from one row to another in one operation:

- Select the column *Operators* of the service parameter that includes the operators you want to copy and double-click. The text in the column is then selected.
- Copy the text to the clipboard:

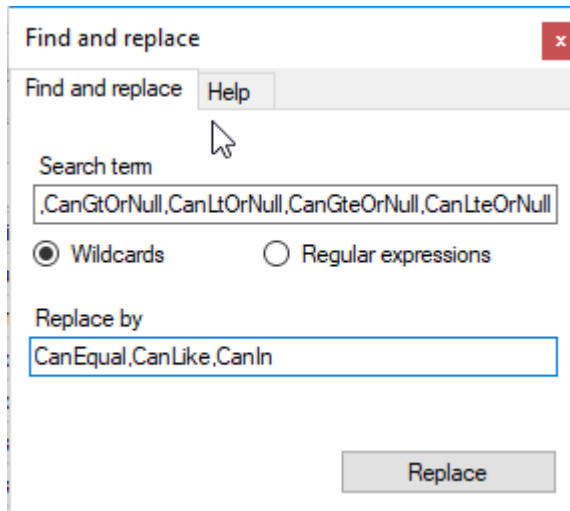


- Select the column *Operators* of the service parameter to which you want to copy the operators and double-click. The text in the column is then selected.
- Paste the text from the clipboard.
- Press the RETURN key.

Replacing operators using *Find and replace*

This function is available for the column *Operators*. You proceed in a similar way like described in the section above:

- Select the column *Operators* of the service parameter that includes the operators you want to replace by another combination.
- Select *Find and replace* in the context menu.
- The dialog *Find and replace* opens.



The value of the previously selected service parameter is preassigned in the *search term* field.

- Enter the combination that replaces the search term.
- Confirm by clicking the button *Replace*. The assistant replaces the operator combination specified in the search term by the new combination in all service parameters in the table. The change is also performed in the columns of the separate operators "CanEqual", "CanLike",...

24.3 Export

Use the application menu ([Repository](#) → [Export repository](#)) to activate the export dialog. Here, you can make a detailed selection of the data records that you want to export. Settings in this dialog are displayed again when re-opening the dialog.

In the "Domain filter" area you can specify the domain set that you want to export. If you do not make any entry here, all domain sets are exported. In addition, you can set a filter for the domains that you want to export. If you do not set any filter, all domains of the relevant domain set are exported. In the "File filter" area, you can specify which data types are to be exported.

In the "Export paths" area, you can store and select up to three paths for export. For each path, you can specify the export structure that you want to use.

- **Client Domain:** Data in this structure can be read by the Repository Client.
- **Server Domain:** Data in this structure can be read by the Repository Client.
- **Client Runtime:** Data in this structure can be read and processed by the client.
- **Server runtime:** Data in this structure can be read and processed by the server.

Start the data export via "Export". When the export is completed, a dialog opens showing the number of exported data records.

24.4 Validation

The Repository Client provides an integrated validation function checking the syntax of the columns (or property) to be edited and the syntax of a data record itself. The validation function also checks the consistency between several data records (data types), in particular master-detail relations of individual domains, and it provides a multiple validation and a validation subject to a function type or data type (e.g. Service --> ServiceType). This function is performed when you edit data or when you click the validation button.

25 Interpreted Java Service2

1.1 Introduction

Interpreted Java services version 2 are web services that are converted by an interpreter to SELECT SQL statements. The result is converted to a web service result, once the SQL statement has been executed.

The definition for the interpreter is created in XML files using the repository client.

25.1 Availability

As of SP7

25.2 Definition

An interpreted Java service is defined in the repository (for further information on the required values and their meaning, refer section "repository data" below).

The service domain can be exported as XML file, once the definition has been completed. The resulting files (the ones relevant to the interpreter) are **<Domain>.Configuration.xml** and **<Domain>.do.xml**.

25.3 Storage in a server

Both files are located on the server at:

JDIR\MOC\<SYSTEM>\listInterpreter\<Scope>.

or JHYDRADIR\MOC\<SYSTEM>\listInterpreter\<Scope>. The scope can have one of the following values: standard, custom or local.

25.4 Available Special Parameters

Each interpreted Java service includes special parameters that are always available and can always be used. Subject to how the interpreted Java service is customized in the file <Domain>.do.xml, the parameters affect processing or are ignored.

These parameters are available:

Name	Data type	Operators	Description
checkresponsibilityarea	boolean	EQUAL	Is only effective if checkRespAreaMode and checkRespAreaField are configured. If false, the

			responsibility area will not be checked. If true, it is checked. The default value is true.
checkresponsibilityareafuctions	String[]	EQUAL, IN	Is only effective, if checkRespAreaMode and checkRespAreaField are configured and checkresponsibilityarea == true. This parameter controls which functions are checked by the responsibility area. The default value is select. The following functions can be indicated: create (vab_tab.anlegen='J') delete (vab_tab.loeschen='J') select (vab_tab.anzeigen='J') update (vab_tab.aendern='J') use (vab_tab.verwenden='J')
longtermdata	boolean	EQUAL	Is only effective if tableClauseLongterm is configured. If false, no long-term data is used. If true, it is checked. The default value is false.

25.5 Repository data

25.5.1 Tab Services

Name	Meaning	Optional
Domain	The domain of the service (e.g. BOPerson)	
Name	The complete service name, i.e. Domain.Function (e.g. BOPerson.list)	
Service Function	The function of the service (e.g. list)	
Service Type	The type - for interpreted Java services: fixed InterpretedJavaService2	
Description (German)	Brief (internal) description of the service	

25.5.2 Tab ServiceParameter

Name	Meaning	Optional
Domain	The domain of the service (e.g. BOPerson)	
Service Function	The function of the service (e.g. list)	
Service	The complete service name, i.e. Domain.Function (e.g. BOPerson.list)	
Acronym	Acronyms (e.g. person.id) have to be unique for service and result set.	
Web service type	The data type of the parameter (decimal, integer, string, boolean, datetime)	
DB table	The table that is used to select the value for the acronym	
DB field	The field from which the value for the acronym is to be selected This is either just the field name or the expression (if it is a calculated field) including placeholder for the table alias (e.g. <code>hydadm.get_datetime(%1\$s.bearb_date,%1\$s.bearb_time)</code> or <code>{fn substring(%1\$s.field,2,1)}</code>). The placeholder for the table alias is always <code>"%1\$s"</code> .	
DB Alias	The table alias for the table that is used to select the value for the acronym	
Conversion Method	Here you can specify transformations for input and result parameters (e.g. conversion bool to J/N and vice versa or the correct filtering for datetime fields that consist of two fields in the database). Possible transformations are described elsewhere.	X
Can ...	Have to be set for filter parameters (for Boolean only Can Equal; for string all and for the others, everything except Can Like and Can Like or null)	X (if only Result)
IsFilterParameter	Specifies whether or not the field is a filter field. A filter parameter including its operator is directly converted into an SQL fragment	X (if only Result)

IsResult	Specifies whether or not it is a Result	X (if only Filter)
IsSpecialParameter	Specifies whether or not a field is a special parameter. At the moment, standard processing only supports the above-mentioned special parameters. Further parameters can be used in exits. Special parameters are "options" that cannot directly be converted into an SQL fragment of the type <DB field> <Operator> <Value>.	X
IsMandatory	Specifies whether or not it is a mandatory field. If TRUE and the parameter is missing, an error message is generated at runtime. Is checked with special parameters and filter parameters.	X
InputAsArray	Specifies whether the field also supports arrays as input parameters. (e.g. the operators IN or BETWEEN require an array as input parameter)	
ConditionalFieldKey	This field specifies if a DB field is only conditionally available. The condition as to whether the field is available is checked using the Configuration Manager (see Configuration Manager in the section "server"). The feature key of the Configuration Manager (feature set) has to be entered in this field within the repository for checking.	X
DBFieldAlternative	Only relevant if it is a conditional field. Here you can enter the alternative value. This value can be a figure, null, 'string', {fn ...}, or even another field / subselect. Default value is zero if nothing else is entered. Note: This field has a different behavior than "DB Field". The alias is not automatically put in front. If you want to use the alias of the field "DB Alias", you must put %1\$. in front of the field name.	X
Constraints	If the value "SKIP_INTERPRETER " is entered here, the interpreter ignores this acronym. This is useful, if you want to edit or add acronyms in an exit.	X

25.5.3 Tab Dataobjects

Name	Meaning	Optional
name	Name of the data source. Must be identical to the complete service name, i.e. domain.function (e.g. BOPerson.list).	
orderBy	Specifies sorting (with the real alias and not %1\$s. in front of the field name) Please note: Do not use if "groupByCols" is applied. This might lead to SQL errors if sorting is based on a field that is not included in the "group by" clause (in case the client has not requested it).	X
groupByCols	Specifies the "group by" fields (in the order) for the SQL statement. The value includes a list of acronyms including their database field (with the real alias and not %1\$s. in front of the field name). The interpreter only adds the fields requested by the client (including their acronyms) to the group by clause. For example: Acronym1=alias.field1 Acronym2=alias.field2	X
filterBy	Specifies fixed filters (with the real alias and not %1\$s. in front of the field name)	X
checkRespAreaMode	Checks the responsibility area of the current user. Modes: "none": no check direct: directly checked by a field of the data source (joined to vab_tab). Use of --DEFAULT-- if empty or null directnotempty: check directly via a field of the data source (joined to vab_tab) "person": check via VAB of person "machine": check via VAB of machine	X

checkRespAreaField	Specifies the field including the responsibility area for direct/directnotempty. Join field if person or machine (with the real alias and not %1\$. in front of the field name)	X
checkRespAreaDefaultValue	Specifies the default value for checking the responsibility area (if the parameter has not been specified by the client). The default value is true. Valid values are true and false.	X
dataTabLabel	Name of the result set. The field normally remains empty and is only required if special processing is performed by further user exits.	X
tableClause	Specifies the table clause without key word FROM. Also the alias must be included. Example: "fmea_eval_nbr_catalog fmeacat".	
tableClauseLongterm	Specifies the table clause for long-term data (only relevant if long-term data exist). If nothing is entered the tableClause is used as tableClauseLongterm.	X
conditionalLongtermKey	This value specifies if the long-term data tables are only conditionally available. The condition as to whether the tables are available is checked by the Configuration Manager. The feature key of the Configuration Manager has to be entered here for checking.	X
mergeOnAttributeLevel	Attribute available as of SP11: This attribute controls how individual DataObjects are merged over several scopes (a single DataObject is identified by the attribute "name"). If the attribute mergeOnAttributeLevel does not exist or the value is not equal "Y", the behavior is the same than before SP11 (backward compatibility). This means that the whole configuration of the DataObjects (not the whole file, but only the entire row) is completely overwritten by a specific scope. For example, if a filterBy is introduced in the custom scope, the complete DataObject in the standard scope is replaced. If the standard is then	

	extended, the standard extension is not applied in the custom scope. • If the attribute is set to "Y", the merge behavior changes and one attribute is merged after the other and not the complete DataObject at a time. Refer to the following subchapter for details. (This setting is the default setting for new DataObject configurations as of SP11; existing older configurations are not changed). You can find details and examples in the next subchapter.	
--	--	--

25.5.3.1 Rules for the merge of SQL attributes on attribute level

"Merging by attribute" is available as of Service Pack 11.

Only "name" is a mandatory attribute if you merge DataObjects on attribute level. The other values are acquired from the specific scope.

The following applies for most attributes:

- An **empty attribute** means that the value of the general scope **is used**.
- A **populated attribute** means that the value of the general scope is **overwritten**.

These SQL attributes are an exception:

- tableClauseLongterm
- tableClause
- filterBy
- groupByCols
- orderBy

The following applies with these SQL attributes:

- An **empty attribute** means that the value of the general scope **is used**.
- A **populated attribute** means that the value of the general scope is **written before** it (i.e. the value of the specific scope is written after the value of the general scope). To separate the content of the general scope from the content of the specific scope, different separators are used for the different attributes:
 - o tableClauseLongterm: one space
 - o tableClause: one space

- filterBy: an „AND“; also the general scope is in brackets. . If there is no keyword "\$LOWER_SCOPE_VALUE\$" in the specific scope, then the specific scope is in brackets.
- orderBy: a comma („“)
- groupByCols: a pipe („|“)
- If the attribute contains the keyword \$NO_LOWER_SCOPE\$, the value of the general scope is completely replaced by the value of the attribute from the specific scope.
- If the attribute includes the key word \$LOWER_SCOPE_VALUE\$, the general scope is copied to this position (with the specific separator) exactly.

25.5.3.2 Examples of the merge of SQL attributes on attribute level

"Merging by attribute" is available as of Service Pack 11.

The following examples only refer to the attributes tableClauseLongterm, tableClause, filterBy, orderBy and groupByCols. The other attributes follow a very simple scheme: If the attribute in more specific scope is populated, the general scope is replaced. If attribute is not filled in the specific scope, the attribute from the general scope is used.

In the following, you can find examples of the behavior, if you merge on attribute level (mergeOnAttributeLevel=Y). We used "filterBy" in our examples. The first row is the general scope (e.g. standard), the second row is the specific scope (e.g. custom) and the third row is the result of the merge.

```
"xy.field_x = '42'",
"foo.field_y = '24'",
"(xy.field_x = '42') and (foo.field_y = '24')"
```

```
"xy.field_x = '42'",
"",
"xy.field_x = '42'"
```

```
"",
"foo.field_y = '24'",
"foo.field_y = '24'"
```

```
"xy.field_x = '42'",
"foo.field_y$LANG = 'foo' $LOWER_SCOPE_VALUE$",
"foo.field_y$LANG = 'foo' and (xy.field_x = '42')"
```

```
"xy.field_x = '42'",
"foo.field_y = '24' $LOWER_SCOPE_VALUE$",
"foo.field_y = '24' and (xy.field_x = '42')"
```

```
"",
"foo.field_y = '24' $LOWER_SCOPE_VALUE$",
"foo.field_y = '24'"
```

```
"xy.field_x = '42'",
"foo.field_y = '24' and $LOWER_SCOPE_VALUE$ bar.field_z = 'world'",
"foo.field_y = '24' and (xy.field_x = '42') and bar.field_z = 'world'"
```

```
"",
"foo.field_y = '24' and $LOWER_SCOPE_VALUE$ bar.field_z = 'world'",
"foo.field_y = '24' and bar.field_z = 'world'"
```

```

"xy.field_x = '42'",
"$LOWER_SCOPE_VALUE$ foo.field_y = '24'",
"(xy.field_x = '42') and foo.field_y = '24'"

"",
"$LOWER_SCOPE_VALUE$ foo.field_y = '24'",
" foo.field_y = '24'"

"xy.field_x = '42'",
"foo.field_y = '24' $NO_LOWER_SCOPE$",
" foo.field_y = '24'"

"",
"foo.field_y = '24' $NO_LOWER_SCOPE$",
" foo.field_y = '24'"

"xy.field_x = '42'",
"foo.field_y = '24' and $NO_LOWER_SCOPE$ bar.field_z = 'world'",
"foo.field_y = '24' and bar.field_z = 'world'"

"",
"foo.field_y = '24' and $NO_LOWER_SCOPE$ bar.field_z = 'world'",
"foo.field_y = '24' and bar.field_z = 'world'"

"xy.field_x = '42'",
"$NO_LOWER_SCOPE$ foo.field_y = '24'",
" foo.field_y = '24'"

"",
"$NO_LOWER_SCOPE$ foo.field_y = '24'",
" foo.field_y = '24'"

"satz_art = 'U'",
"$NO_LOWER_SCOPE$ masch_nr = '4711' $LOWER_SCOPE_VALUE$",
" masch_nr = '4711' and (satz_art = 'U')"
```

```

"satz_art = 'U'",
"masch_nr = '4711' $NO_LOWER_SCOPE$LOWER_SCOPE_VALUE$",
"masch_nr = '4711' LOWER_SCOPE_VALUE$" => invalid SQL

"satz_art = 'U'",
"masch_nr = '4711' $LOWER_SCOPE_VALUE$NO_LOWER_SCOPE$",
"masch_nr = '4711' (satz_art = 'U') andNO_LOWER_SCOPE$" => invalid SQL

"satz_art = 'U'",
"masch_nr = '4711' $LOWER_SCOPE_VALUE$ $LOWER_SCOPE_VALUE$",
"masch_nr = '4711' (satz_art = 'U') and $LOWER_SCOPE_VALUE$" => invalid SQL

"satz_art = 'U'",
"$LOWER_SCOPE_VALUE$ masch_nr = '4711' $LOWER_SCOPE_VALUE$",
"(satz_art = 'U') and masch_nr = '4711' $LOWER_SCOPE_VALUE$" => invalid SQL

```

25.6 Exits

Exits provide the entry points to enable changes to the defined behavior by programming.



Instead of the user exits and program exits presented below, use the [GlobalExits](#). The GlobalExits ensure the greatest possible compatibility in the further development of the system, as they are supported equally for all service types.

25.6.1 Available user exits

User exits provide entry points that are not modified by releases. In case of releases, the interfaces are respected and preserved by MPDV in a backwards compatible manner.

25.6.1.1 `sdiInterpretedSqlModifyRequest`

The application developer can change the parameters and the column configurator using this exit. You can also create temporary tables as this exit includes access to the DB session which is also used by the main SQL.

25.6.1.2 `sdiAddResultTransformationCallbacks`

Using this exit, the application developer can modify the service result after having executed the SQL statement. Rows can be deleted, added or changed. The application developer registers a callback to this end, which is called for each row.

25.6.1.3 `sdiInterpretedSqlCleanup`

Using this exit, the application developer can undertake cleanup actions. This exit is always executed, whether or not errors occurred. Here you can e.g. clean up temporary tables as you can access the DB session of the main SQL.

25.6.2 Available program exits

Program exits offer extended functionality that is not backwards compatible. Changes can be made with every update. Users of program exits must test if their modifications still work after an update.

Therefore, program exits are of limited value with modifications.

25.6.2.1 `sdiAugmentSql`

With this user exit, the interpreted Java service provides the application developer with an option to extend the generated SQL shortly before it is executed.

25.6.3 Specifications for the implementation class of the exit

Package name: You must include the class in a package that consists of the domain name (in lower case letters). Further subpackages are **not** allowed.

Example: The service is requested "MDUserAccountRules.list", consequently the package is called "mduseraccountrules"

Class name: The class must have a name of the following structure: domain name in lower case letters, whereas the first letter is written in capital letters, the name of the service function follows and is written in lower case letters, whereas the first letter is once again written in upper case.

Example: The service is requested "MDUserAccountRules.list", consequently the class is called "MduseraccountrulesList"



The following definition applies for customized class names:

Customized names include "_" (see naming conventions)

After "_" the first letter is changed to upper case and the underscore is skipped

Example: U_CUST_Units_sample.list => UCustUnitsSampleList

Implemented interfaces / methods: no specifications

Other: The class must have a default constructor without parameters

Compilation: The Jar files *MpdvDomCoreSdiCompileLib.jar* and *MpdvDomCoreUserExitCompileLib.jar* must be included in the class path for the compile process.

Deployment: The class file of the exit must be stored in the directory
<JDIR>/MOC/<System>/userexit/<scope> including package directory structure.

Example: Exit sdiAugmentSql for service "MDUserAccountRules.list":

```
package mduseraccountrules;
```

```
import de.mpdv.customization.userExit.IUserExitParam;
```

```
/**
 * Sample user exit
 *
 *
 */
public class MduseraccountrulesList
{
    public void sdiAugmentSql(final IUserExitParam param)
    {
        // TODO implementation
    }
}
```

Directory structure on the server (System 1, scope custom):
<JDIR>/MOC/1/userexit/custom/mduseraccountrules/MduseraccountrulesList.class

25.6.4 Interfaces

25.6.4.1 Class: InterpretedJavaServiceUeContext

de.mpdv.sdi.data.ue.InterpretedJavaServiceUeContext
-hydraNow: Calendar -userId: String
+getHydraNow(): Calendar +getUserId(): String

This context class provides data that is generally useful for interpreted Java services as regards to exits.

Field	Description
hydraNow	The property "hydraNow" is a time stamp created at the beginning of web service processing and, as a result, can be used as reference time stamp for the current web service call.
userId	Includes the user logged on to the client.

25.6.4.2 User exit: sdiInterpretedSqlModifyRequest

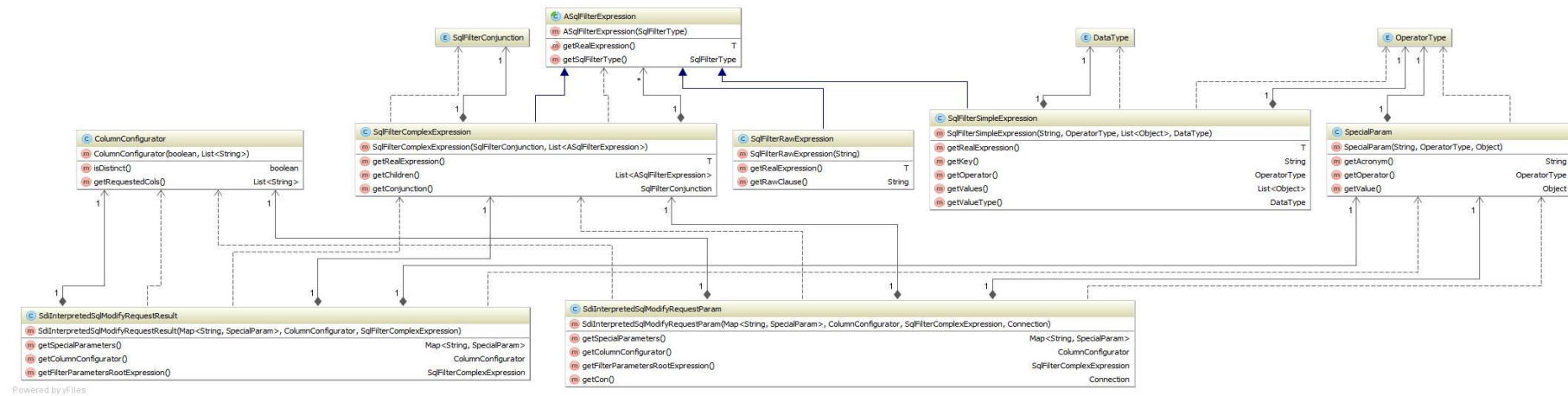
Parameter key in IUserExitParam:

Key name	Type	Description
param	SdiInterpretedSqlModifyRequestParam	Parameter structure for the user exit
context	InterpretedJavaServiceUeContext	Context structure for all exits of the interpreted Java Services
factory	ISystemUtilFactory	Utility class to access system utilities, such as logger or DB connection in exits.

Return key in IUserExitParam:

Key name	Type	Description
result	SdiInterpretedSqlModifyRequestResult	Result structure for the user exit

Class diagram of parameter and result structures:



25.6.4.3 Class *SdiInterpretedSqlModifyRequestParam*

Field	Type	Description
columnConfigurator	ColumnConfigurator	Includes the columns requested by the client
specialParameters	Map<String, SpecialParam>	Assigns acronym => SpecialParameter for all web service parameters of the type "is SpecialParameter"
filterParametersRootExpression	SqlFilterComplexExpression	Root node of the SQL filter tree. All parameters of type "isFilterParameter" are included in this tree.
con	Connection	DB session that is also used by the main SQL.

25.6.4.4 Class *SdiInterpretedSqlModifyRequestResult*

Field	Type	Description
columnConfigurator	ColumnConfigurator	Includes the modified column configuration
specialParameters	Map<String, SpecialParam>	Includes the modified SpecialParameters
filterParametersRootExpression	SqlFilterComplexExpression	Includes the modified FilterParameter

25.6.4.5 User exit: *sdiAddResultTransformationCallbacks*

Parameter key in IUserExitParam:

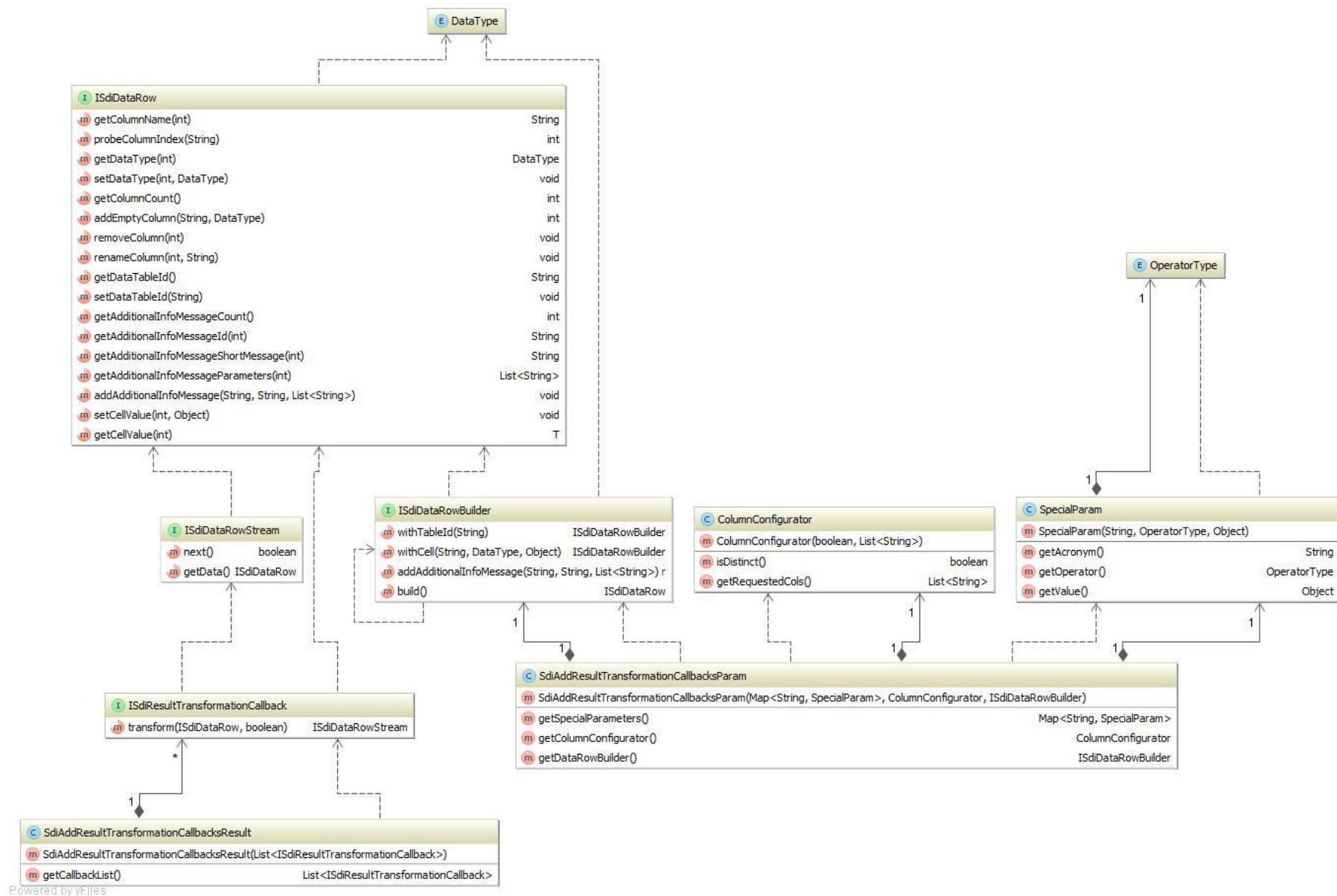
Key name	Type	Description
param	SdiAddResultTransformationCallbackParam	Parameter structure for the

		user exit
context	InterpretedJavaServiceUeContext	Context structure for all exits of the interpreted Java Services
factory	ISystemUtilFactory	Utility class to access system utilities, such as logger or DB connection in exits.

Return key in IUserExitParam:

Key name	Type	Description
result	SdiAddResultTransformationCallbackResult	Result structure for the user exit: Note: If you only want to change the input row, you need not generate a result of the class "SdiModifyResultRowResult" and therefore also the key "result" is not necessary in IUserExitParam. A result of the class "SdiModifyResultRowResult" is generated, if rows are deleted or added.

Class diagram of parameter and result structures:



25.6.4.6 Class *SdiAddResultTransformationCallbacksParam*

Field	Type	Description
specialParameters	Map<String, SpecialParam>	Assigns acronym => SpecialParameter for all web service parameters of the type "is SpecialParameter"
columnConfigurator	ColumnConfigurator	Includes the columns requested by the client
dataRowBuilder	ISdiDataRowBuilder	Using this builder, you can generate the instances ISdiDataRow. A direct implementation of ISdiDataRow is not allowed!

25.6.4.7 Class *SdiAddResultTransformationCallbacksResult*

Field	Type	Description
callbackList	List<ISdiResultTransformationCallback>	Callback list for the result transformation

25.6.4.8 Interface *ISdiResultTransformationCallback*

Method	Description
transform(ISdiDataRow dataRow, boolean isLastRow): ISdiDataRowStream	Callback method for the result transformation Return type: ISdiDataRowStream must not be NULL: Includes the rows as stream (to support streaming) once the input row has been edited in the user exit. The service then returns the stream rows as result to the client. You can use the class SdiEagerDataRowStream to modify the current row and to return few result rows. If you want to return large amounts of data, you must implement a

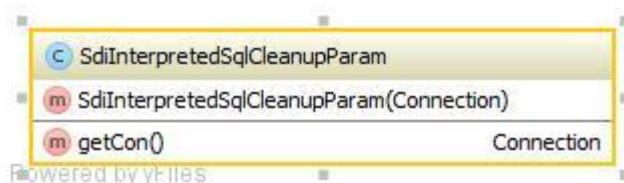
	stream that takes data rows from an external data source. If you must create ISdiDataRow instances, you must use ISdiDataRowFactory from SdiAddResultTransformationCallbacksParam. A direct implementation of ISdiDataRow is not allowed! Input: ISdiDataRow dataRow: Includes a result row that the interpreter creates as service result. isLastRow boolean: TRUE, if this row is the last row, otherwise FALSE
--	--

25.6.4.9 User exit: sdilInterpretedSqlCleanup

Parameter key in IUserExitParam:

Key name	Type	Description
param	SdiInterpretedSqlCleanupParam	Parameter structure for the user exit
context	InterpretedJavaServiceUeContext	Context structure for all exits of the interpreted Java Services
factory	ISystemUtilFactory	Utility class to access system utilities, such as logger or DB connection in exits.

Class diagram of the parameter structure:



25.6.4.10 Class SdiInterpretedSqlCleanupParam

Field	Type	Description
-------	------	-------------

con	Connection	DB session that is also used by the main SQL.
-----	------------	---

25.6.4.11 Program exit: sdiAugmentSql

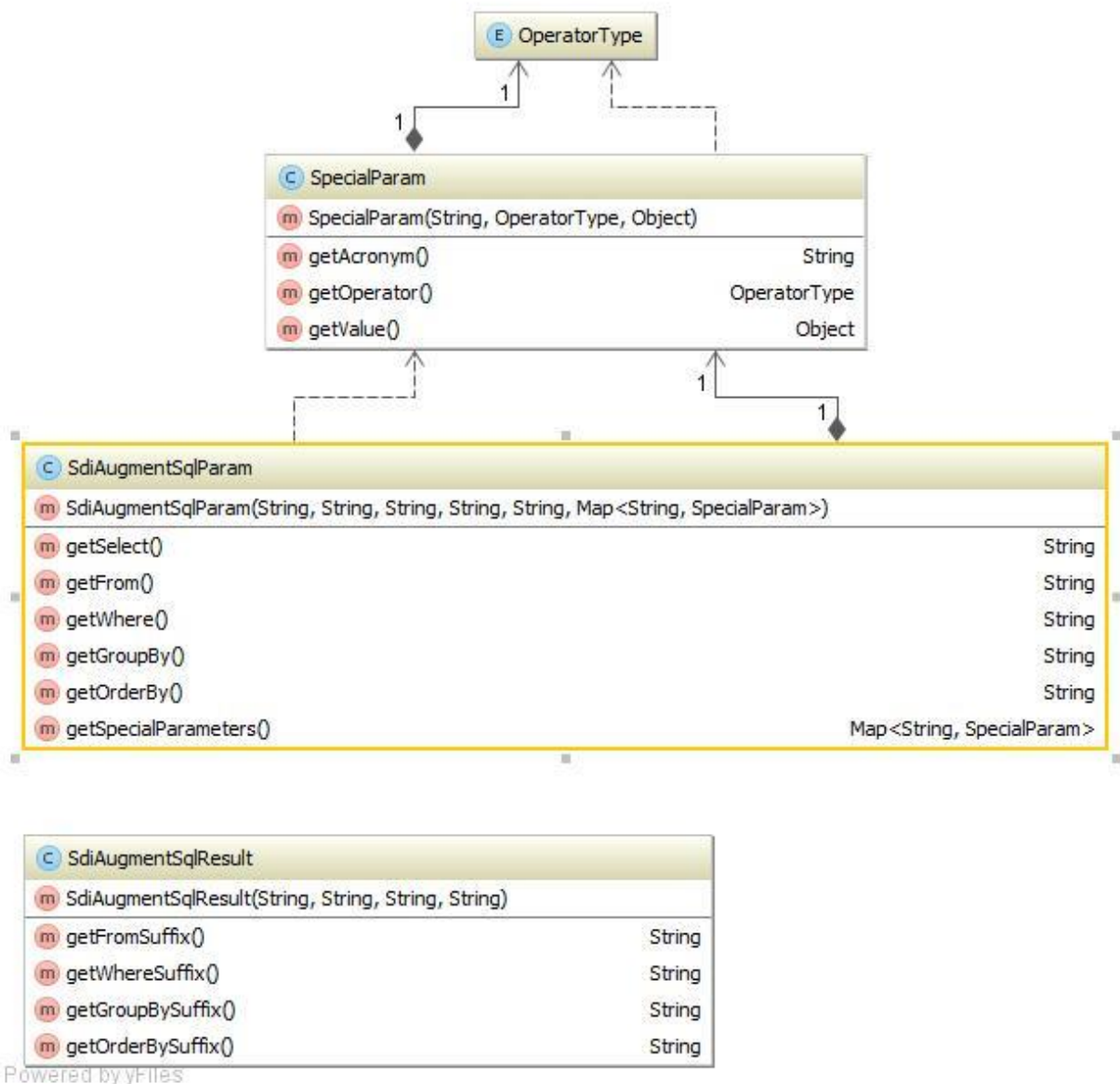
Parameter key in IUserExitParam:

Key name	Type	Description
param	SdiAugmentSqlParam	Parameter structure for the program exit
context	InterpretedJavaServiceUeContext	Context structure for all exits of the interpreted Java Services
factory	ISystemUtilFactory	Utility class to access system utilities, such as logger or DB connection in exits.

Return key in IUserExitParam:

Key name	Type	Description
result	SdiAugmentSqlResult	Result structure for the program exit

Class diagram of parameter and result structures:



25.6.4.12 Class SdiAugmentSqlParam

Field	Type	Description
select	String	SELECT clause created based on the configuration (only includes the column part up to FROM, only without the key word SELECT)
from	String	FROM clause created based on the configuration (without the key word FROM)

WHERE	String	WHERE clause created based on the configuration (without the key word WHERE)
groupBy	String	GROUP BY clause created based on the configuration
orderBy	String	ORDER BY clause created based on the configuration
specialParameters	Map<String, SpecialParam>	Assigns acronym => SpecialParameter for all web service parameters of the type "is SpecialParameter"

25.6.4.13 Class SdiAugmentSqlResult

Field	Type	Description
fromSuffix	String	Suffix for the FROM clause created in the program exit or NULL
whereSuffix	String	Suffix for the WHERE clause created in the program exit or NULL
groupBySuffix	String	Suffix for the GROUP BY clause created in the program exit or NULL
orderBySuffix	String	Suffix for the ORDER BY clause created in the program exit or NULL

26 Interpreted Java Service

26.1 Introduction

Interpreted Java services are web services that are converted by an interpreter to SELECT SQL statements. The result is converted to a web service result, once the SQL statement has been executed.

The definition for the interpreter is created in XML files using the repository client.

If you create new services, use the type `InterpretedJavaService2` instead of the `InterpretedJavaService`.

The `InterpretedJavaService2` type is prepared for the future streaming of data and offers more options for Java user exits.



As long as no Java user exits are used, it is still easy to convert the `InterpretedJavaService` type into the `InterpretedJavaService2` type by simply changing the service type. There are the following differences for services without Java user exits.

- The column `DataObjectName` for `InterpretedJavaService2` is not required in the repository data of service and should be set to empty.
- The column `parameterReference` is not required anymore for the repository data of the `DataObjects` and should be set to empty.

26.2 Definition

An interpreted Java service is defined in the repository (for further information on the required values and their meaning, refer section "repository data" below).

The service domain can be exported as XML file, once the definition has been completed. The resulting files (the ones relevant to the interpreter) are **<Domain>.Configuration.xml** and **<Domain>.do.xml**.

26.3 Storage in a server

Both files are located on a server under `jdir\MOC\<SYSTEM>\listInterpreter\<Scope>`
or `jhydradir\MOC\<SYSTEM>\listInterpreter\<Scope>`

The scope has one of the following values: standard, custom or local.

26.4 Available Special Parameters

Each interpreted Java service includes special parameters that are always available and can always be used. Subject to how the interpreted Java service is customized in the file <Domain>.do.xml, the parameters affect processing or are ignored.

These parameters are available:

Name	Data type	Operators	Description
checkresponsibilityarea	boolean	EQUAL	Is only effective if checkRespAreaMode and checkRespAreaField are configured. If false, the responsibility area will not be checked. If true, it is checked. The default value is true.
checkresponsibilityareafunctions	String[]	EQUAL, IN	Is only effective, if checkRespAreaMode and checkRespAreaField are configured and checkresponsibilityarea == true. This parameter controls which functions are checked by the responsibility area. The default value is select. The following functions can be indicated: create (vab_tab.anlegen='J') delete (vab_tab.loeschen='J') select (vab_tab.anzeigen='J') update (vab_tab.aendern='J') use (vab_tab.verwenden='J')
longtermdata	boolean	EQUAL	Is only effective if tableClauseLongterm is configured. If false, no long-term data is used. If true, it is checked. The default value is false.

26.5 Repository data

26.5.1 Tab Services

Name	Meaning	Optional
Domain	The domain of the service (e.g. BOPerson)	
Name	The complete service name, i.e. Domain.Function (e.g. BOPerson.list)	
Service Function	The function of the service (e.g. list)	
Service Type	The type - for interpreted Java services: fixed InterpretedJavaService	
Description (German)	Brief (internal) description of the service	

26.5.2 Tab ServiceParameter

Name	Meaning	Optional
Domain	The domain of the service (e.g. BOPerson)	
Service Function	The function of the service (e.g. list)	
Service	The complete service name, i.e. Domain.Function (e.g. BOPerson.list)	
Acronym	Acronyms (e.g. person.id) have to be unique for service and result set.	
Web service type	The data type of the parameter (decimal, integer, string, boolean, datetime)	
DB table	The table that is used to select the value for the acronym	
DB field	The field from which the value for the acronym is to be selected This is either just the field name or the expression (if it is a calculated field) including placeholder for the table alias (e.g. hydadm.get_datetime (%1\$s.bearb_date, %1\$s.bearb_time) or {fn substring (%1\$s.field, 2,	

	1))). The placeholder for the table alias is always "%1\$s".	
DB Alias	The table alias for the table that is used to select the value for the acronym	
Conversion Method	Here you can specify transformations for input and result parameters (e.g. conversion bool to J/N and vice versa or the correct filtering for datetime fields that consist of two fields in the database). Possible transformations are described elsewhere.	X
Can ...	Have to be set for filter parameters (for Boolean only Can Equal; for string all and for the others, everything except Can Like and Can Like or null)	X (if only Result)
IsFilterParameter	Specifies whether or not the field is a filter field. A filter parameter including its operator is directly converted into an SQL fragment	X (if only Result)
IsResult	Specifies whether or not it is a Result	X (if only Filter)
IsSpecialParameter	Specifies whether or not a field is a special parameter. At the moment, standard processing only supports the above-mentioned special parameters. Further parameters can be used in exits. Special parameters are "options" that cannot directly be converted into an SQL fragment of the type <DB field> <Operator> <Value>.	X
IsMandatory	Specifies whether or not it is a mandatory field. If this is true and the parameter is missing, an error message is generated at runtime. Is currently only checked for special parameters.	X
InputAsArray	Specifies whether the field also supports arrays as input parameters. (e.g. the operators IN or BETWEEN require an array as input parameter)	
DataObjectName	Name of the interpreted Java Service. Used as reference for the ...do.xml configuration	
ConditionalFieldKey	This field specifies if a DB field is only conditionally available. The condition as to whether the field is available is checked using the	X

	Configuration Manager (see Configuration Manager in the section "server"). The feature key of the Configuration Manager (feature set) has to be entered in this field within the repository for checking.	
DBFieldAlternative	Only relevant if it is a conditional field. Here you can enter the alternative value. This value can be a figure, null, 'string', {fn ...}, or even another field / subselect. "%1\$s." NEEDS to be entered for the alias if it is another field or subselect! The default value is null if nothing is entered.	X

26.5.3 Tab Dataobjects

name

Name of the data source. References to the field DataObjectName for the repository (to connect the ServiceParameter)

parameterReference

References to the service name in order for the correct parameters to be determined

orderBy (optional)

Specifies sorting (with the real alias and not %1\$s. in front of the field name)

Please note: Do not use if "groupByCols" is applied. This might lead to SQL errors if sorting is based on a field that is not included in the "group by" clause (in case the client has not requested it).

groupByCols (optional)

Specifies the "group by" fields (in the order) for the SQL statement.

The value includes a list of acronyms including their database field (with the real alias and not %1\$s. in front of the field name). The interpreter only adds the fields requested by the client (including their acronyms) to the group by clause.

For example:

Acronym1=alias.field1|Acronym2=alias.field2|...

filterBy (optional)

Specifies fixed filters (with the real alias and not %1\$s. in front of the field name)

checkRespAreaMode (optional)

Checks the responsibility area of the current user.

Modes:

- „none“: no check
- „direct“: Check performed via a field of the data source (joined to vab_tab). Use of -- DEFAULT-- if empty or zero.
- „directnotempty“: Check performed via a field of the data source (joined to vab_tab)
- „person“: Check performed via the responsibility area (VBA) of a person.
- „machine“: Check performed via the VAB assigned to the machine.

checkRespAreaField (optional)

Specifies the field of the main table which contains the VAB for direct/directnotempty.

Join field if person or machine

(with the real alias and not %1\$s. in front of the field name)

checkRespAreaDefaultValue (optional)

Specifies the default value for checking the responsibility area (if the parameter has not been specified by the client). The default value is true. Valid values are true and false.

dataTabLabel (optional)

Name of the result set. The field normally remains empty and is only required if special processing is performed by further user exits.

tableClause (optional)

Specifies the table clause without the key word FROM (only relevant if several tables are used). If nothing is entered here, the first table and its alias found for a service parameter will be used as the tableClause.

tableClauseLongterm (optional)

Specifies the table clause for long-term data (only relevant if several tables are used or if long-term data exist). If nothing is entered the tableClause is used as tableClauseLongterm. If this one is neither indicated, the first table and its alias found for a service parameter will be used.

conditionalLongtermKey (optional)

This value specifies if the long-term data tables are only conditionally available. The condition as to whether the tables are available is checked by the Configuration Manager. The feature key of the Configuration Manager has to be entered here for checking.

mergeOnAttributeLevel (optional)

Attribute available as of SP11:

This attribute controls how individual DataObjects are merged over several scopes (a single DataObject is identified by the attribute "name").

- If the attribute `mergeOnAttributeLevel` does not exist or the value is not equal "Y", the behavior is the same than before SP11 (backward compatibility). This means that the whole configuration of the DataObjects (not the whole file, but only the entire row) is completely overwritten by a higher scope. For example, if a `filterBy` is introduced in the custom scope, the complete DataObject in the standard scope is replaced. If the standard is then extended, the standard extension is not applied in the custom scope.
- If the attribute is set to "Y", the merge behavior changes and one attribute is merged after the other and not the complete DataObject at a time. Refer to the following subchapter for details. (This setting is the default setting for new DataObject configurations as of SP11; existing older configurations are not changed).

You can find details and examples in the next subchapter.

26.5.3.1 Rules for the merge of SQL attributes on attribute level

Only "name" and "parameterReference" are mandatory attributes if you merge DataObjects on attribute level. The other values are taken over from the lower scope.

The following applies for most attributes:

- If the attribute is empty, the value of the lower scope **is taken over**.
- If the attribute is populated, the value of the lower scope **is overwritten**.

These SQL attributes are an exception:

- `tableClauseLongterm`
- `tableClause`
- `filterBy`
- `groupByCols`
- `orderBy`

The following applies with these SQL attributes:

- If the attribute is empty, the value of the lower scope **is taken over**.
- If the attribute is populated, the value of the lower scope **is written in front** (the value of the higher scope is written behind the value of the lower scope). A specific separator is used for the separation:
 - `tableClauseLongterm`: a space character
 - `tableClause`: a space character

- filterBy: an "AND"; in addition the lower scope is put in parentheses. If the higher scope does not include a key word, also the higher scope is put in parentheses.
- orderBy: a comma (",")
- groupByCols: a pipe ("|")
- If the attribute includes the key word \$NO_LOWER_SCOPE\$, the value of the lower scope is completely replaced by the value of the attribute.

If the attribute includes the key word \$LOWER_SCOPE_VALUE\$, the lower scope is copied to exactly this position (with the specific separator).

26.5.3.2 Examples of the merge of SQL attributes on attribute level

As of SP11

The following examples only refer to the attributes tableClauseLongterm, tableClause, filterBy, orderBy and groupByCols. The other attributes follow a very simple pattern: if the attribute in a higher scope is populated, the lower scope is replaced. If the attribute in the higher scope is not populated, the lower scope is used.

In the following, you can find examples of the behavior, if you merge on attribute level (mergeOnAttributeLevel=Y). We used "filterBy" in our examples. The first row is the lower scope (e.g. standard), the second row is the higher scope (e.g. custom) and the third row is the result of the merge.

```
"xy.field_x = '42'",
"foo.field_y = '24'",
"(xy.field_x = '42') and (foo.field_y = '24')"
```

```
"xy.field_x = '42'",
"",
"xy.field_x = '42'"

"",
"foo.field_y = '24'",
"foo.field_y = '24'"

"xy.field_x = '42'",
"foo.field_y$LANG = 'foo' $LOWER_SCOPE_VALUE$",
"foo.field_y$LANG = 'foo' and (xy.field_x = '42')"
```

```
"xy.field_x = '42'",
"foo.field_y = '24' $LOWER_SCOPE_VALUE$",
"foo.field_y = '24' and (xy.field_x = '42')"
```

```
"",
"foo.field_y = '24' $LOWER_SCOPE_VALUE$",
"foo.field_y = '24'"

"xy.field_x = '42'",
"foo.field_y = '24' and $LOWER_SCOPE_VALUE$ bar.field_z = 'world'",
"foo.field_y = '24' and (xy.field_x = '42') and bar.field_z = 'world'"

"",
"foo.field_y = '24' and $LOWER_SCOPE_VALUE$ bar.field_z = 'world'",
"foo.field_y = '24' and bar.field_z = 'world'"

```

```
"xy.field_x = '42'",
"$LOWER_SCOPE_VALUE$ foo.field_y = '24'",
"(xy.field_x = '42') and foo.field_y = '24'"

"",
"$LOWER_SCOPE_VALUE$ foo.field_y = '24'",
" foo.field_y = '24'"

"xy.field_x = '42'",
"foo.field_y = '24' $NO_LOWER_SCOPE$",
" foo.field_y = '24'"

"",
"foo.field_y = '24' $NO_LOWER_SCOPE$",
" foo.field_y = '24'"

"xy.field_x = '42'",
"foo.field_y = '24' and $NO_LOWER_SCOPE$ bar.field_z = 'world'",
"foo.field_y = '24' and bar.field_z = 'world'"

"",
"foo.field_y = '24' and $NO_LOWER_SCOPE$ bar.field_z = 'world'",
"foo.field_y = '24' and bar.field_z = 'world'"

"xy.field_x = '42'",
"$NO_LOWER_SCOPE$ foo.field_y = '24'",
" foo.field_y = '24'"

"",
"$NO_LOWER_SCOPE$ foo.field_y = '24'",
" foo.field_y = '24'"

"satz_art = 'U'",
"$NO_LOWER_SCOPE$ masch_nr = '4711' $LOWER_SCOPE_VALUE$",
" masch_nr = '4711' and (satz_art = 'U')"
```

```
"satz_art = 'U'",
"masch_nr = '4711' $NO_LOWER_SCOPE$LOWER_SCOPE_VALUE$",
"masch_nr = '4711' LOWER_SCOPE_VALUE$" => invalid SQL

"satz_art = 'U'",
"masch_nr = '4711' $LOWER_SCOPE_VALUE$NO_LOWER_SCOPE$",
"masch_nr = '4711' (satz_art = 'U') andNO_LOWER_SCOPE$" => invalid SQL

"satz_art = 'U'",
"masch_nr = '4711' $LOWER_SCOPE_VALUE$ $LOWER_SCOPE_VALUE$",
"masch_nr = '4711' (satz_art = 'U') and $LOWER_SCOPE_VALUE$" => invalid SQL

"satz_art = 'U'",
"$LOWER_SCOPE_VALUE$ masch_nr = '4711' $LOWER_SCOPE_VALUE$",
" (satz_art = 'U') and masch_nr = '4711' $LOWER_SCOPE_VALUE$" => invalid SQL
```

26.6 Exits

Exits provide the entry points to enable changes to the defined behavior by programming.



Instead of the user exits and program exits presented below, use the [GlobalExits](#). The GlobalExits ensure the greatest possible compatibility in the further development of the system, as they are supported equally for all service types.

26.6.1 Available user exits

User exits provide entry points that are not modified by releases. In case of releases, the interfaces are respected and preserved by MPDV in a backwards compatible manner.

26.6.1.1 sdiModifyColumnConfigurator

With this user exit, the interpreted Java service provides the application developer with an option to modify/extend the column configurator before it is executed.

26.6.1.2 sdiModifyResultList

With this user exit, the interpreted Java service provides the application developer with an option to modify/extend the service result after it is executed.

26.6.2 Available program exits

Program exits offer extended functionality that is not backwards compatible. Changes can be made with every update. Users of program exits must test if their modifications still work after an update.

Therefore, program exits are of limited value with modifications.

26.6.2.1 sdiModifyColumnMap

With this program exit, the interpreted Java service provides the application developer with an option to modify/extend standard assignment of acronym to DB table before it is executed.

26.6.2.2 sdiAugmentSql

With this program exit, the interpreted Java service provides the application developer with an option to modify the generated SQL shortly before it is executed.

26.6.2.3 sdiModifySql

As of SP8: with this program exit, the SQL from the generator or of lower scopes can be overwritten explicitly.

26.6.3 Specifications for the implementation class

Package name: You must include the class in a package that consists of the domain name (in lower case letters). Further subpackages are **not** allowed.

Example: The service is requested "MDUserAccountRules.list", consequently the package is called "mduseraccountrules"

Class name: The class must have a name of the following structure: domain name in lower case letters, whereas the first letter is written in capital letters, the name of the service function follows and is written in lower case letters, whereas the first letter is once again written in upper case.

Example: The service is requested "MDUserAccountRules.list", consequently the class is called "MduseraccountrulesList"



The following definition applies for customized class names:

Customized names include "_" (see naming conventions)

After "_" the first letter is changed to upper case and the underscore is skipped

Example: U_CUST_Units_sample.list => UCustUnitsSampleList

Implemented interfaces / methods: no specifications

Other: The class must have a default constructor without parameters

Compilation: The Jar files *MpdvDomCoreSdiCompileLib.jar* and *MpdvDomCoreUserExitCompileLib.jar* must be included in the class path for the compile process.

Deployment: The class file of the exit must be stored in the directory
<JDIR>/MOC/<System>/userexit/<scope> including package directory structure.

Example: Exit sdiAugmentSql for service "MDUserAccountRules.list":

```
package mduseraccountrules;

import de.mpdv.customization.userExit.IUserExitParam;

/**
 * Sample user exit
 *
 *
 */
public class MduseraccountrulesList
{
    public void sdiAugmentSql(final IUserExitParam param)
    {
        // TODO implementation
    }
}
```

Directory structure on the server (System 1, scope custom):

<JDIR>/MOC/1/userexit/custom/mduseraccountrules/MduseraccountrulesList.class

26.6.4 Interfaces

26.6.4.1 Class: InterpretedJavaServiceUeContext

de.mpcdv.sdi.data.ue.InterpretedJavaServiceUeContext
-hydraNow: Calendar -userId: String
+getHydraNow(): Calendar +getUserId(): String

This context class provides data that is generally useful for interpreted Java services as regards to exits.

Field	Description
hydraNow	The property "hydraNow" is a time stamp created at the beginning of web service processing and, as a result, can be used as reference time stamp for the current web service call.
userId	Includes the user logged on to the client.

26.6.4.2 Program exit: sdiModifyColumnConfigurator

Parameter key in IUserExitParam:

Key name	Type	Description
param	SdiModifyColumnConfiguratorParam	Parameter structure for the user exit
context	InterpretedJavaServiceUeContext	Context structure for all exits of the interpreted Java Services
factory	ISystemUtilFactory	Utility class to access system utilities, such as logger or DB connection in exits.

Return key in IUserExitParam:

Key name	Type	Description
result	SdiModifyColumnConfiguratorResult	Result structure for the user exit

Class diagram of parameter and result structures:

```
de.mpdv.sdi.data.ue.SdiModifyColumnConfiguratorParam
+getColumnConfigurator():ColumnConfigurator
+getSpecialParameters():Map<String, SpecialParam>
```

```
de.mpdv.sdi.data.ue.SdiModifyColumnConfiguratorResult
+getColumnConfigurator():ColumnConfigurator
```

26.6.4.3 Class SdiModifyColumnConfiguratorParam

Field	Type	Description
columnConfigurator	ColumnConfigurator	Includes the columns requested by the client
specialParameters	Map<String, SpecialParam>	Assigns acronym => SpecialParameter for all web service parameters of the type "is SpecialParameter"

26.6.4.4 Class SdiModifyColumnConfiguratorResult

Field	Type	Description
columnConfigurator	ColumnConfigurator	Includes the requested columns after processing in user exit

26.6.4.5 Program exit: sdiModifyColumnMap

Parameter key in IUserExitParam:

Key name	Type	Description
param	SdiModifyColumnMapParam	Parameter structure for the program exit
context	InterpretedJavaServiceUeContext	Context structure for all exits of the interpreted Java Services
factory	ISystemUtilFactory	Utility class to access system utilities, such as logger or DB connection in exits.

Return key in IUserExitParam:

Key name	Type	Description
----------	------	-------------

result	SdiModifyColumnMapResult	Result structure for the program exit
--------	--------------------------	---------------------------------------

Class diagram of parameter and result structures:

de.mpdv.sdi.data.ue.SdiModifyColumnMapParam
+getColumnMap():Map<String, String>
+getSpecialParameters():Map<String, SpecialParam>

de.mpdv.sdi.data.ue.SdiModifyColumnMapResult
+getColumnMap():Map<String, String>

26.6.4.6 Class SdiModifyColumnMapParam

Field	Type	Description
columnMap	Map<String, String>	Includes the assignment of acronym => table column (including alias)
specialParameters	Map<String, SpecialParam>	Assigns acronym => SpecialParameter for all web service parameters of the type "is SpecialParameter"

26.6.4.7 Class SdiModifyColumnMapResult

Field	Type	Description
columnMap	Map<String, String>	Includes the assignment of acronym => table column (including alias)

26.6.4.8 Program exit: sdiAugmentSql

Parameter key in IUserExitParam:

Key name	Type	Description
param	SdiAugmentSqlParam	Parameter structure for the program exit
context	InterpretedJavaServiceUeContext	Context structure for all exits of the interpreted Java Services
factory	ISystemUtilFactory	Utility class to access system utilities, such as logger or DB

		connection in exits.
--	--	----------------------

Return key in IUserExitParam:

Key name	Type	Description
result	SdiAugmentSqlResult	Result structure for the program exit

Class diagram of parameter and result structures:

de.mpdv.sdi.data.ue.SdiAugmentSqlParam
+getSelect():String +getFrom():String +getWhere():String +getGroupBy():String +getOrderBy():String +getSpecialParameters():Map<String, SpecialParam>

de.mpdv.sdi.data.ue.SdiAugmentSqlResult
+getFromSuffix():String +getWhereSuffix():String +getGroupBySuffix():String +getOrderBySuffix():String

26.6.4.9 Class SdiAugmentSqlParam

Field	Type	Description
select	String	SELECT clause created based on the configuration (only includes the column part up to FROM, only without the key word SELECT)
from	String	FROM clause created based on the configuration (without the key word FROM)
WHERE	String	WHERE clause created based on the configuration (without the key word WHERE)
groupBy	String	GROUP BY clause created based on the configuration
orderBy	String	ORDER BY clause created based on the configuration
specialParameters	Map<String, SpecialParam>	Assigns acronym => SpecialParameter for all web service parameters of the type "is"

		SpecialParameter“
--	--	-------------------

26.6.4.10 Class SdiAugmentSqlResult

Field	Type	Description
fromSuffix	String	Suffix for the FROM clause created in the program exit or NULL
whereSuffix	String	Suffix for the WHERE clause created in the program exit or NULL
groupBySuffix	String	Suffix for the GROUP BY clause created in the program exit or NULL
orderBySuffix	String	Suffix for the ORDER BY clause created in the program exit or NULL

26.6.4.11 User exit: sdiModifyResultList

Parameter key in IUserExitParam:

Key name	Type	Description
param	SdiModifyResultListParam	Parameter structure for the user exit
context	InterpretedJavaServiceUeContext	Context structure for all user exits of the interpreted Java Services
factory	ISystemUtilFactory	Utility class to access system utilities, such as logger or DB connection in user exits.

Return key in IUserExitParam:

Key name	Type	Description
result	SdiModifyResultListResult	Result structure for the user exit

Class diagram of parameter and result structures:

de.mpdv.sdi.data.ue.SdiModifyResultListParam
+getColumnConfigurator():ColumnConfigurator
+getDataTables():List<IDataTable>
+getSpecialParameters():Map<String, SpecialParam>

de.mpdv.sdi.data.ue.SdiModifyResultListResult
+getDataTables():List<IDataTable>

26.6.4.12 Class SdiModifyResultListParam

Field	Type	Description
columnConfigurator	ColumnConfigurator	Includes the columns requested by the client
dataTables	List<IDataTable>	Includes the data table(s) generated as service result by the interpreter
specialParameters	Map<String, SpecialParam>	Assigns acronym => SpecialParameter for all web service parameters of the type "is SpecialParameter"

26.6.4.13 Class SdiModifyResultListResult

Field	Type	Description
dataTables	List<IDataTable>	Includes the data table(s) after processing in the user exit that the service supplies as result to the client

26.6.4.14 Program exit: sdiModifySql

Parameter key in IUserExitParam:

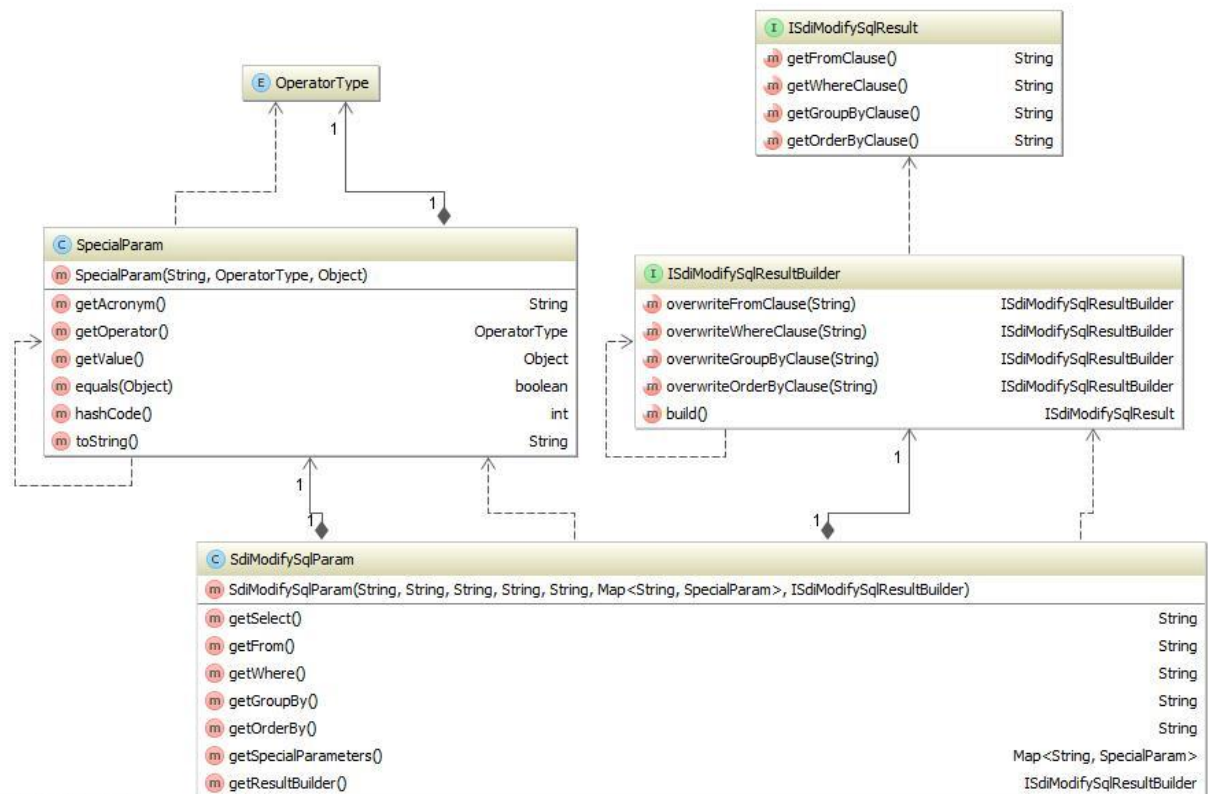
Key name	Type	Description
param	SdiModifySqlParam	Parameter structure for the program exit
context	InterpretedJavaServiceUeContext	Context structure for all exits of the interpreted Java Services
factory	ISystemUtilFactory	Utility class to access system utilities, such as logger or DB

		connection in exits.
--	--	----------------------

Return key in IUserExitParam:

Key name	Type	Description
result	ISdiModifySqlResult	Result structure of the program exit

Class diagram of parameter and result structures:



Powered by yFiles

26.6.4.15 Class SdiModifySqlParam

Field	Type	Description
select	String	SELECT clause created based on the configuration (only includes the column part up to FROM, only without the key word SELECT)
from	String	FROM clause created based on the configuration (without the key word FROM)

WHERE	String	WHERE clause created based on the configuration (without the key word WHERE)
groupBy	String	GROUP BY clause created based on the configuration
orderBy	String	ORDER BY clause created based on the configuration
specialParameters	Map<String, SpecialParam>	Assigns acronym => SpecialParameter for all web service parameters of the type "is SpecialParameter"

26.6.4.16 Interface ISdiModifySqlResultBuilder

Method	Description
overwriteFromClause():ISdiModifySqlResultBuilder	Overwrites FROM clause. Only call this method if you really want to overwrite the FROM clause!
overwriteWhereClause():ISdiModifySqlResultBuilder	Overwrites WHERE clause. Only call this method if you really want to overwrite the WHERE clause!
overwriteGroupByClause():ISdiModifySqlResultBuilder	Overwrites GROUP BY clause. Only call this method if you really want to overwrite the GROUP BY clause!
overwriteOrderByClause():ISdiModifySqlResultBuilder	Overwrites ORDER BY clause. Only call this method if you really want to overwrite the Order BY clause!
build():ISdiModifySqlResult	Creates the result structure of the program exit.

26.6.4.17 Interface ISdiModifySqlResult

Method	Description
getFromClause(): String	FROM clause of the program exit if the clause is overwritten there. Otherwise it is the original clause.
getWhereClause(): String	WHERE clause of the program exit if the clause is overwritten there. Otherwise it is the original clause.
getGroupByClause(): String	GROUP BY clause from the program exit if it was overwritten there, otherwise the original clause.
getOrderByClause(): String	ORDER BY clause of the program exit if the clause is overwritten there. Otherwise it is the original clause.

27 Interpreted BAPI Services

27.1 Introduction

Interpreted Bapi services are web services that are converted by an interpreter to INSERT, UPDATE or DELETE SQL statements. Interpreted Bapi services enable writing access to tables and, as a result, have been designed for data maintenance. Along with interpreted Java services, you can completely implement maintenance and display of data via configurations.

The definition for the interpreter is created with the Repository Client and stored in XML files.

27.2 Features

27.2.1 Processing steps of the Bapi Interpreter

The Bapi Interpreter is structured in 5 steps. The steps are:

27.2.1.1 init

- has been designed for initialization: reads out input parameters; reads the service configuration
- **plausibility checks must not take place!**

27.2.1.2 checkKeys

- has been designed to check whether or not all key fields have been specified. All plausibility checks are performed in a row and then the client receives the result of the plausibility checks.

27.2.1.3 selectData

- selects data
 - o data required for plausibility checks
 - o data that are also required in performAction
 - o etc.
- **plausibility checks must not take place!**

27.2.1.4 checkData

- All plausibility checks that do not pertain to the checking of key fields are performed here.

27.2.1.5 performAction

- the actual BAPI functions are executed here (e.g. create data record, lock data record, etc.)

27.2.2 Basic processing of the individual processing steps

27.2.2.1 init

- reads out the service configuration
- reads out request parameters

27.2.2.2 checkKeys

- checks whether at least one input parameter has been configured with the property "key field". The property "key field" is the constraint SERIAL if an input parameter including this constraint is available or it is the constraint KEY.
- checks whether at least 5 input parameters have been configured with the property "key field". The property "key field" is the constraint SERIAL if an input parameter including this constraint is available or it is the constraint KEY.
- checks whether all input parameters including the constraint KEY have a unique index between 1 and 5.
- checks whether all input parameters including the constraint KEY have been passed in the request
- checks whether all input parameters including the constraint SERIAL have been passed in the request

27.2.2.3 selectData

- selects a data record using the constraint SERIAL, if the data record and the constraint SERIAL are available
- selects data records using the constraint KEY, if data records and the constraint KEY are available
- reads internal locks
- reads external Locks

27.2.2.4 checkData

- checks whether all mandatory parameters have been transferred in the request (repository field "Is Mandatory")

27.2.2.5 performAction

- no action, because specific to the function

27.2.3 Bapi functions

27.2.3.1 LOCK

The Bapi function LOCK has been designed to lock a data record for a later processing.

init

- basic processing only

checkKeys

- basic processing only

selectData

- basic processing only

checkData

- Basic processing
- checks if exactly one data record is available matching the property "key field" (filter). The property "key field" is the constraint SERIAL if an input parameter including this constraint is available or it is the constraint KEY.

performAction

- Locks the data record, if it has not yet been locked by another user

27.2.3.2 UNLOCK

The Bapi function UNLOCK has been designed to unlock a data record after processing or once processing has been interrupted.

init

- basic processing only

checkKeys

- basic processing only

selectData

- basic processing only

checkData

- Basic processing
- checks if exactly one data record is available matching the property "key field" (filter). The property "key field" is the constraint SERIAL if an input parameter including this constraint is available or it is the constraint KEY.
- checks whether a separate lock is at all available

performAction

- unlocks the data record

27.2.3.3 INSERT

The Bapi function INSERT adds a data record to the database.

init

- basic processing only

checkKeys

- basic processing only

selectData

- basic processing only

checkData

- Basic processing
- checks that no data record could be identified using the constraint KEY

performAction

- adds the data record and identifies the serial if a result parameter has been configured using the constraint SERIAL

27.2.3.4 UPDATE

A data record is edited using the function UPDATE.

init

- basic processing only

checkKeys

- basic processing only

selectData

- basic processing only

checkData

- Basic processing

- checks if exactly one data record is available matching the property "key field" (filter). The property "key field" is the constraint SERIAL if an input parameter including this constraint is available or it is the constraint KEY.
- checks whether a separate lock is at all available

performAction

- UPDATES the data record

27.2.3.5 DELETE

The Bapi function DELETE deletes a data record from the database.

init

- basic processing only

checkKeys

- basic processing only

selectData

- basic processing only

checkData

- basic processing
- checks if exactly one data record is available matching the property "key field" (filter). The property "key field" is the constraint SERIAL if an input parameter including this constraint is available or it is the constraint KEY.
- checks whether or not an internal lock is available
- checks whether or not an external lock is available

performAction

- deletes the data record

27.3 Service definition

An interpreted Bapi service is defined in the repository (further information on the required values and their meaning: see section "repository data").

The service domain can be exported as XML file, once the definition has been completed. The resulting file (the one relevant to the Interpreter) is the file **<Domain>.Configuration.xml**.

27.4 Storage in a server

The file is located on a MW30 server under `jdir\MOC\<Instance>\listInterpreter\<Scope>` or `JHYDRADIR\MOC\<Instance>\listInterpreter\<Scope>`. The scope can have one of the following values: **standard**, **custom** or **local**.

27.5 Repository data

27.5.1 Tab Services

Name	Meaning	Optional
Domain	The domain of the service (e.g. BOPerson)	
Name	The complete service name, i.e. Domain.Function (e.g. BOPerson.insert)	
Service Function	The function of the service (e.g. insert)	
Service Type	Service type - for interpreted BAPI services, fixed: InterpretedBapiService	
Description	Brief (internal) description of the service	

27.5.2 Tab ServiceParameter

Name	Meaning	Optional
Domain	The domain of the service (e.g. BOPerson)	
Service Function	The function of the service (e.g. insert)	
Service	The complete service name, i.e. Domain.Function (e.g. BOPerson.insert)	
Acronym	Acronyms (e.g. person.id) have to be unique within a service	
Web service type	The data type of the parameter (decimal, integer, string, boolean, datetime)	
DB table	The table that is used to select the value for the acronym	X (if the field is used in UE only)
DB field	The field that is used to select the value for the acronym	X (if the field is used in UE only)
DB Alias	The table alias for the table that is used to select the value for the acronym	X (if the field is used in UE only)
Can Equal	If the field is an input parameter, this option must be enabled.	X (only if Result or Constraint "MODIFY_TS" or "MODIFY_BY" or "CREATE_TS" or "CREATE_BY" is set)
IsResult	Specifies whether or not it is a Result	X (if not SERIAL)
IsSpecialParameter	Specifies whether or not the field is an input parameter	X (only if Result or Constraint "MODIFY_TS" or "MODIFY_BY" or "CREATE_TS" or "CREATE_BY" is set)
IsMandatory	Specifies whether or not the field is a mandatory field. Only reasonable with input parameters. Mandatory fields must be sent with the request and must not be null or empty.	X

Constraints	See separate section "Constraints"	X (if no constraint is required for the current acronym)
ConditionalFieldKey	If a name of a FeatureSet is entered, this acronym is only processed, if the FeatureSet is active.	

27.5.2.1 ServiceParameter: Constraints

Constraints are processing parameters that are structured as key with optional values. You use the pipe (|) character to separate two keys. You use the equal sign (=) to separate key and value. You use a semicolon to separate various values. The general structure is as follows: e.g.
Key1=value;value;value|Key2|Key3=value|

The following constraints are available:

Constraint Key	Constraint value(s)	Description
KEY	Exactly one number ranging between 1 and 5. This number may only be used once within a service, i.e. different acronyms with the constraint KEY also must have different numbers. The configuration of this constraint for a specific acronym must be identical for UPDATE, DELETE, LOCK and UNLOCK. Example: the acronym workplace.id includes the Constraint KEY=1 in the service <domain>.update. In this case, workplace.id also must include the Constraint KEY=1 in the services <domain>.delete, <domain>.lock and <domain>.unlock!	Define field as key including key number for hyd_lock table
SERIAL	none	The field is a SERIAL (or auto-increment). The database generates this value when creating a data record. ⇒ "isResult" needs to be active for the INSERT service!

SEP_DATETIME	1st parameter refers to the date field 2nd parameter refers to the time field	Allows processing of separate date and time fields
BOOL	1st parameter refers to the value that is to be entered into the database, if true. 2nd parameter refers to the value that is to be entered in the database, if false. 3rd parameter refers to the value that is to be entered into the database, if null ("null" for writing the DB null -> default). 4th parameter refers to the type of the DB field. For example: BOOL=J;N>null string BOOL=1;0>null;integer	Allows for Boolean values to be written in a string or integer field
MODIFY_TS	None	The acronym represents the point in time of processing. -> To do so, the point in time of web service processing in the server is written into the database. ⇒ No parameter value is used!
MODIFY_BY	None	The acronym represents the user. The registered user executing the service is entered in the database. ⇒ No parameter value is used!
CREATE_TS	None	The acronym represents the point in time of generation -> To do so, the point in time of web service processing in the server is written into the database. ⇒ No parameter value is used!

CREATE_BY	None	The acronym represents the user who created the data record. The registered user executing the service is entered in the database. ⇒ No parameter value is used!
CONST_VALUE	String value for the constant e.g. CONST_VALUE=J	You can create constant values for database fields with this constraint. The parameter must be in the repository. The parameter must not be isSpecialParameter=Y or isResult=Y!
SUB_STR	. 1st parameter is the database field (without alias). 2nd parameter is the position in the database field. 3rd parameter is the parameter length in the database field. e.g.: param1;1;1	You can use this constraint to map the content of a database field using several acronyms. For writing, you must always pass all acronyms for this DB field at the same time. If this is not the case, a plausibility error is thrown that includes all acronyms for this DB field. You can easily combine this constraint with CONST_VALUE and BOOL.
STR_VALUE_RESTRICTION	List of valid values separated by semicolon ";". To allow NULL or empty string (are processed the same way), include an empty string between two semicolons ";". To allow a semicolon ";" within a valid value, you must mask it with a backslash "\". To allow a backslash "\" within a valid value, you must mask it with another backslash "\\".	You can use this constraint to implement value restrictions without exit for string parameters. If the parameter value is not included in the list of allowed values, a plausibility error is thrown.

	e.g.: NULL and empty string and J and N are allowed values: STR_VALUE_RESTRICTION=J;N;; e.g.: Semicolon and J are allowed values: STR_VALUE_RESTRICTION=\;;J e.g.: The values Foo and Bar are allowed values: STR_VALUE_RESTRICTION=Foo;Bar	
LOCK_KEY_ONLY	None	Using this constraint and the constraint CONST_VALUE, you can enter constants for the lock key that otherwise are not persisted in a DB field. You can also add this constraint to parameters. These parameters are then only stored in the lock table and not in a DB field.
SKIP_INTERPRETER	None	Use this constraint to tag fields that are only processed in exits and not by the interpreter.
LOGICAL_KEY	None	Only available for UPDATE and INSERT: You can use this constraint to tag logical keys. This only makes sense, if the technical key is a SERIAL. You can use the constraint LOGICAL_KEY to issue a plausibility error message. This error message is

		issued, if you try to create a new data record with identical logical keys or if you try to change an existing data record using the same logical keys that are used by an existing data record.
--	--	--



The database table of an interpreted Bapi service is **exclusively** taken from the first (first from the top) acronym configuration of a service that includes the constraint SERIAL or KEY. In this context, the constraint SERIAL takes priority, if it is available.

27.6 User exits

User exits provide entry points to enable changes to the configured behavior via programming.



Instead of the user exits and program exits presented below, use the [GlobalExits](#). The GlobalExits ensure the greatest possible compatibility in the further development of the system, as they are supported equally for all service types.

27.6.1 Available user exits

27.6.1.1 sdiAfterInit

Via this user exit, the application developer can use the interpreted Bapi service to change (insert, delete, change) the read request parameters.



You must **NOT** perform a plausibility check. You must perform plausibility checks in sdiAfterCheckData. The results can then be added to the result of sdiAfterCheckData.

27.6.1.2 sdiAfterSelectData

Via this user exit, the application developer can use the interpreted Bapi service to select data, e.g. for a plausibility check.



You must **NOT** perform a plausibility check. You must perform plausibility checks in `sdiAfterCheckData`. The results can then be added to the result of `sdiAfterCheckData`.

27.6.1.3 `sdiAfterCheckData`

Via this user exit, an application developer can create plausibility checks.

You must **NOT** perform an SQL query. You **must** perform SQL queries in `sdiAfterSelectData`.

You can then use the results in this user exit.



You must **NOT** use an exception to tag a plausibility error here. For each plausibility error, you must create an object `BapiInterpreterValidationMessage`.

27.6.1.4 `sdiAfterPerformAction`

Via this user exit, the application developer can use the interpreted Bapi service to perform actions after the actual Bapi processing.

27.6.1.5 `sdiBapiCleanup`

Via this user exit, the application developer can use the interpreted Bapi service to perform cleanup actions after the actual Bapi processing. This exit is always executed, whether or not errors occurred.

27.6.2 Specifications for the implementation class

Package name: You must include the class in a package that consists of the domain name (in lower case letters). Further subpackages are **not** allowed.

Example: The service is called "MDUserAccountRules.insert", consequently the package is called "mduseraccountrules"

Class name: The class must have a name of the following structure: domain name in lower case letters, whereas the first letter is written in capital letters, the name of the service function follows and is written in lower case letters, whereas the first letter is once again written in upper case.

Example: The service is called "MDUserAccountRules.insert", consequently the class is called "MduseraccountrulesInsert"



The following definition applies for customized class names:
 Customized names include “_” (see naming conventions)
 After “_” the first letter is changed to upper case and the underscore is skipped

Example: U_CUST_Units_sample.insert => UCustUnitsSampleInsert

Implemented interfaces / methods: no specifications

Other: The class must have a default constructor without parameters

Compilation: The Jar files *MpdvDomCoreSdiCompileLib.jar* and *MpdvDomCoreUserExitCompileLib.jar* must be included in the class path for the compile process.

Deployment: The class file of the user exit must be stored in the directory `jdir/MOC/<instance>/userexit/<scope>` or `<JHYDRADIR>/MOC/<instance>/userexit/<scope>` including package directory structure.

Example: User exit `sdiAfterCheckData` for service "MDUserAccountRules.insert":

```
package mduseraccountrules;

import de.mpdv.customization.userExit.IUserExitParam;

/**
 * Sample user exit
 *
 */
public class MduseraccountrulesInsert
{
    public void sdiAfterCheckData(final IUserExitParam param)
    {
        // TODO implementation
    }
}
```

Directory structure on the server (instance 1, scope custom):
`jdir/MOC/1/userexit/custom/mduseraccountrules/MduseraccountrulesInsert.class`

27.6.3 Interfaces

27.6.3.1 Class: BapiInterpreterUeContext

de.mpdv.sdi.data.ue.BapiInterpreterUeContext
-hydraNow: Calendar -userId: String
+getHydraNow(): Calendar +getUserId(): String

This context class provides data that can be used globally in interpreted Bapi services in the context of user exits.

Field	Description
hydraNow	The property "hydraNow" is a time stamp created at the beginning of web service processing and, as a result, can be used as reference time stamp for the current web service call.
userId	Includes the user logged on to the client.

27.6.3.2 Userexit: sdiAfterInit

Parameter key in IUserExitParam:

Key name	Type	Description
param	SdiAfterInitParam	Parameter structure for the user exit
context	BapiInterpreterUeContext	Context structure for all user exits of the interpreted Bapi services
factory	ISystemUtilFactory	Utility class to access system utilities, such as logger or DB connection in user exits.

Return key in IUserExitParam:

Key name	Type	Description
result	SdiAfterInitResult	Result structure for the user exit: parameter structure including the changes made in the user

		exit
--	--	------

27.6.3.3 User exit: sdiAfterSelectData

Parameter key in IUserExitParam:

Key name	Type	Description
param	SdiAfterSelectDataParam	Parameter structure for the user exit
context	BapiInterpreterUeContext	Context structure for all user exits of the interpreted Bapi services
factory	ISystemUtilFactory	Utility class to access system utilities, such as logger or DB connection in user exits.

Return key in IUserExitParam:

No return available

Class diagram of the parameter structure:

de.mpdv.sdi.data.ue.SdiAfterSelectDataParam
-specialParameters: Map<String, SpecialParam>
+getSpecialParameters(): Map<String, SpecialParam>

27.6.3.4 Userexit: sdiAfterCheckData

Parameter key in IUserExitParam:

Key name	Type	Description
param	SdiAfterCheckDataParam	Parameter structure for the user exit
context	BapiInterpreterUeContext	Context structure for all user exits of the interpreted Bapi services
factory	ISystemUtilFactory	Utility class to access system utilities, such as logger or DB connection in user exits.

Return key in IUserExitParam:

Key name	Type	Description
result	SdiAfterCheckDataResult	Result structure for the user exit: includes plausibility errors

27.6.3.5 Userexit: sdiAfterPerformAction

Parameter key in IUserExitParam:

Key name	Type	Description
param	SdiAfterPerformActionParam	Parameter structure for the user exit
context	BapiInterpreterUeContext	Context structure for all user exits of the interpreted Bapi services
factory	ISystemUtilFactory	Utility class to access system utilities, such as logger or DB connection in user exits.

Return key in IUserExitParam:

No return available

27.6.3.6 Userexit: sdiBapiCleanup

Parameter key in IUserExitParam:

Key name	Type	Description
param	SdiBapiCleanupParam	Parameter structure for the user exit
context	BapiInterpreterUeContext	Context structure for all user exits of the interpreted Bapi services
factory	ISystemUtilFactory	Utility class to access system utilities, such as logger or DB connection in user exits.

Return key in IUserExitParam:

No return available

27.6.3.7 Class SdiAfterInitParam

Field	Type	Description
specialParameters	Map<String, SpecialParam>	Assigns acronym => SpecialParameter for all web service parameters of the type "is SpecialParameter"

27.6.3.8 Class SdiAfterInitResult

Field	Type	Description
specialParameters	Map<String, SpecialParam>	Assigns acronym => SpecialParameter for all web service parameters of the type "is SpecialParameter"

27.6.3.9 Class SdiAfterSelectDataParam

Field	Type	Description
specialParameters	Map<String, SpecialParam>	Assigns acronym => SpecialParameter for all web service parameters of the type "is SpecialParameter"

27.6.3.10 Class SdiAfterCheckDataParam

Field	Type	Description
specialParameters	Map<String, SpecialParam>	Assigns acronym => SpecialParameter for all web service parameters of the type "is SpecialParameter"
originalRecord	IBapiInterpreterDbRecord	DB record that includes all fields of the DB table that must be maintained (select * from <table>)

mergedRecord	IBapiInterpreterDbRecord	DB record that includes all fields of originalRecord and that was overwritten by all changes to the current data record.
--------------	--------------------------	--

27.6.3.11 Class SdiAfterCheckDataResult

Field	Type	Description
validationMessages	List<BapiInterpreterValidationMessage>	List of all validation/plausibility errors from the user exit

27.6.3.12 Class BapiInterpreterValidationMessage

Instances of this class represent plausibility errors specific to applications.

Field	Type	Description
languageKey	String	Language key that can be translated.
parameters	List<String>	Parameter values matching the language key

27.6.3.13 Class SdiAfterPerformActionParam

Field	Type	Description
specialParameters	Map<String, SpecialParam>	Assigns acronym => SpecialParameter for all web service parameters of the type "is SpecialParameter"
createdSerial	Integer	The created Serial (only with function insert, if a serial column is available for the BAPI)

27.6.3.14 Interface IBapiInterpreterDbRecord

Method	Description
--------	-------------

fetchValueOfDbField(String dbField): Object	Fetches the DB value for the specified DB field. If the value does not exist or is NULL, NULL is returned.
---	--

27.6.3.15 Class SdiBapiCleanupParam

This class is "for future use" and does currently not include any data.

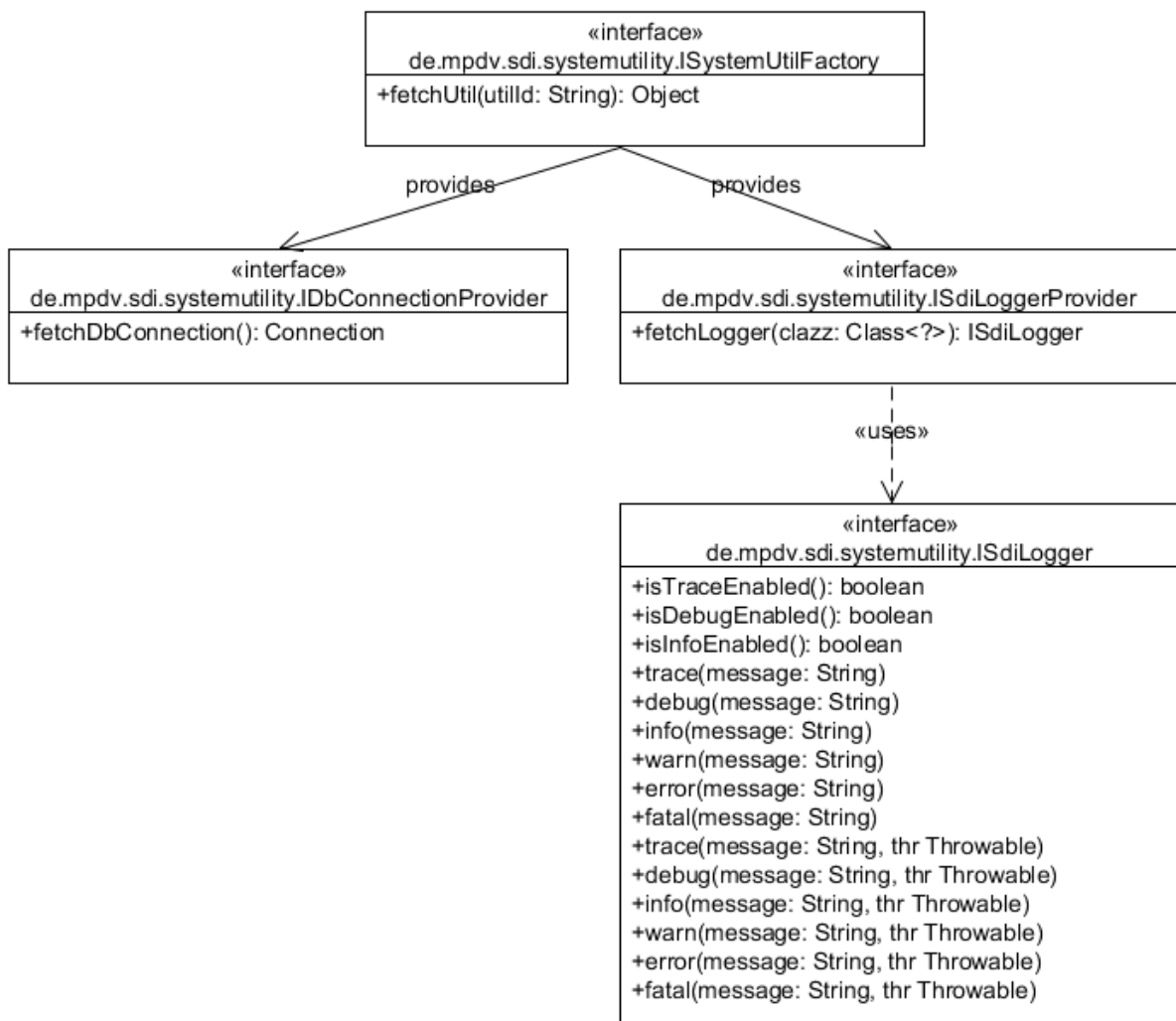
Field	Type	Description
-------	------	-------------

28 System Utilities

28.1 Introduction

The System Utilities are a pool of system functions. For the application developers, these system functions are available as part of the Simple Developer Interface (SDI).

28.2 Schnittstellenbeschreibung



28.3 Interface ISystemUtilFactory

You use this interface to get access to the system utility functions, such as logging.

You can find a list of all available utilities in the following.

Method	Description
fetchUtil()	Provides an instance of the utility for the utilID. Return type: Object - interface for the respective utility. See the below list of utilities. Input: String utilId – the ID of the required System Utility. The ID is case sensitive! A list of all available IDs can be found in the utility list below. Exception: If "utilID" is not known, an <code>IllegalArgumentException</code> is thrown.

28.3.1 Utility list

utilId	Interface of the returned utility	Available as of SPX
LoggerProvider	ISdiLoggerProvider	
DbConnectionProvider	IDbConnectionProvider	
ToNativeSqlConverter	IToNativeSqlConverter	
ToNativeSqlWithParamsConverter	IToNativeSqlWithParamsConverter	
ToNativeSqlWithInlinedParamsConverter	IToNativeSqlWithInlinedParamsConverter	
ServiceConfigProvider	IServiceConfigProvider	
HydraCaller	IHydraCaller	
PdmStringParser	IPdmStringParser	
DataTableBuilder	IDataTableBuilder	

DataTableModifier	IDataTableModifier	
FeatureSetChecker	IFeatureSetChecker	
DbTypeRetriever	IDbTypeRetriever	
UserExitProvider	IUserExitProvider (deprecated as of SP8: replaced by: IExitMethodExecutionManager)	
UserExitFromScopeProvider	IUserExitFromScopeProvider (deprecated as of SP8: replaced by: IExitMethodExecutionManager)	
ServiceCaller	IServiceCaller	
FormulaProvider	IFormulaProvider	
AcronymMappingProvider	IAcronymMappingProvider	
DataAvailabilityChecker	IDataAvailabilityChecker	
JhydradirPathProvider	IJhydradirPathProvider	
JhydradirInstancePathProvider	IJhydradirInstancePathProvider	
JhydradirTempPathProvider	IJhydradirTempPathProvider	
GlobalConfigValueProvider	IGlobalConfigValueProvider	
InstanceConfigValueProvider	IInstanceConfigValueProvider	
HydraPathReadFileAction	IHydraPathReadFileAction	
HydraPathWriteFileAction	IHydraPathWriteFileAction	
HydraPathRenameFileAction	IHydraPathRenameFileAction	
HydraPathDeleteFileAction	IHydraPathDeleteFileAction	
HydraPathFileExistsAction	IHydraPathFileExistsAction	
HydraPathCreateFolderAction	IHydraPathCreateFolderAction	

HydraPathDeleteFolderAction	IHydraPathDeleteFolderAction	
HydraPathDownloadContentAction	IHydraPathDownloadContentAction	
HydraPathUploadContentAction	IHydraPathUploadContentAction	
HydraPathListContentAction	IHydraPathListContentAction	
HydraPathIsDirectoryAction	IHydraPathIsDirectoryAction	
PersonCardIdChecker	IPersonCardIdChecker	
ResponsibilityAreaChecker	IResponsibilityAreaChecker	
PersonResponsibilityAreaChecker	IPersonResponsibilityAreaChecker	
SqlGenerator	ISqlGenerator	SP7
MemoryChecker	IMemoryChecker	SP7
MandatorySpecialParameterValidator	IMandatorySpecialParameterValidator	SP7
ExitMethodExecutionManager	IExitMethodExecutionManager	SP8
SdiDataRowCopyUtil	ISdiDataRowCopyUtil	SP9
DataTableToStreamConverter	IDataTableToStreamConverter	SP9
StreamToDataTableConverter	ISStreamToDataTableConverter	SP9
ResultTransformationManager	IResultTransformationManager	SP9

28.4 Interface IDbConnectionProvider

This interface provides database connections with connection objects.

28.5 Interface ISdiLoggerProvider

You use this interface to create loggers for a specific class. You can then address these loggers separately in the logging configuration.

28.6 Interface ISdiLogger

You can use the interface ISdiLogger to log outputs on different priority levels. The following ascending order applies for the priority: trace, debug, info, warn, error, fatal.

28.7 Interface IToNativeSqlConverter

You can use the interface IToNativeSqlConverter to convert MPDV-SQL (SQL with MPDV extensions, e.g. \$lang or individual escapes) into native SQL without bind variables.

Method	Description
toNativeSql()	Converts MPDV-SQL into native SQL Return type: String - native SQL Input: String sql – MPDV SQL Exception: In case of an error, an SdiException including a language key and (optional) parameters is delivered.

28.8 Interface IToNativeSqlWithParamsConverter

You can use the interface IToNativeSqlWithParamsConverter to convert MPDV-SQL (SQL with MPDV extensions, e.g. \$lang or individual escapes) into native SQL with bind variables.

Method	Description
toNativeSqlWithParams()	Converts MPDV-SQL into native SQL with bind variables. Return type: NativeSqlData – for a description, refer to the description of the data class

	Input: String sql – MPDV SQL Map<String, MpdvSqlParam> parameterMap – map including bind variables (name of variable value) – for the description of MpdvSqlParam, refer to the data class Exception: In case of an error, an SdiException including a language key and (optional) parameters is delivered.
--	--

28.8.1 Data class MpdvSqlParam

Variable with type	Description
String paramName	Name of the bind variable (placeholder from statement without leading :)
Object Value	Value of the bind variable
int sqlType	SQL type matching the value type of the bind variable (see java.sql.Types)

28.8.2 Data class NativeSqlParam

Variable with type	Description
String paramName	Name of the bind variable (placeholder from statement without leading :)
Object	Value of the bind variable

Value	
int sqlType	SQL type matching the value type of the bind variable (see java.sql.Types)

28.8.3 Data class NativeSqlData

Variable with type	Description
String nativeSql	Native SQL statement - placeholders for bind variables are replaced by ? (as required for java.sql.PreparedStatement)
List<NativeSqlParam> nativeSqlParamList	The values of bind variables (in the correct order)

28.9 Interface IToNativeSqlWithInlinedParamsConverter

You can use the interface IToNativeSqlWithInlinedParamsConverter to convert MPDV-SQL (SQL with MPDV extensions, e.g. \$lang or individual escapes) into native SQL with bind variables. The statement returned can be used by any database tool using JDBC.

Method	Description
toNativeSqlWithInlinedParams ()	Converts MPDV-SQL with bind variables into native SQL including bind variables Return type: String - native SQL including bind variables Input: String sql – MPDV SQL Map<String, MpdvSqlParam> parameterMap – map including bind variables (name of variable value) – for the description of MpdvSqlParam, refer

	to the data class (previous section) Exception: In case of an error, an SdiException including a language key and (optional) parameters is delivered.
--	---

28.10 Interface IServiceConfigProvider

You can use the interface IServiceConfigProvider to import the XML configuration for a service (from the listInterpreter folder). Scopes are taken into account. Using the returned object, you can perform queries via xpath for the configuration.

Method	Description
fetchServiceConfig()	Loads the service configuration for the service and supplies an object to access the configuration via xpath. Return type: IServiceConfig – Object to access configuration via xpath. See description in the next section. Input: String serviceName – name of the service Exception: In case of an error, an SdiException including a language key and (optional) parameters is delivered.

28.11 Interface IServiceConfig

The interface IServiceConfig includes the loaded configuration for a service. You can use the interface to query/access the configuration via xpath.

For further details on the xpath syntax, see e.g. <http://www.w3schools.com/xpath/>

Method	Description
getStringValues()	Applies xpath to the configuration and returns the string values of detected objects. Supported objects (and return values) are: - Elements (tag name) - Attributes (value) - Text (text) - Comments (text) Return type: List<String> - string values of the identified objects Input: String xPath – xPath expression Exception: In case of an error, an SdiException including a language key and (optional) parameters is delivered.

28.12 Interface IHydraCaller

You can use the interface IHydraCaller to build a PDM string and to call a PDM dialog (DLG=...). The results of the call are returned.

Method	Description
addString()	Adds a string value for an acronym to the PDM string. Important: If the value is empty, the complete

	acronym is not passed to the PDM service. If this behavior is not wanted, use the method *NoSkipEmptyValue(). Return type: . Input: String acronym – the acronym String value - the value Exception: -
addInteger()	Adds an integer value for an acronym to the PDM string Important: If the value is empty, the complete acronym is not passed to the PDM service. If this behavior is not wanted, use the method *NoSkipEmptyValue(). Return type: . Input: String acronym – the acronym Integer value - the value Exception: -
addDecimal()	Adds a BigDecimal value for an acronym to the PDM string Important: If the value is empty, the complete acronym is not passed to the PDM service. If this behavior is not wanted, use the method

	*NoSkipEmptyValue(). Return type: . Input: String acronym – the acronym BigDecimal value – the value Exception: -
addBoolean()	Adds a Boolean value for an acronym to the PDM string (J for true and N for false) Important: If the value is empty, the complete acronym is not passed to the PDM service. If this behavior is not wanted, use the method *NoSkipEmptyValue(). Return type: . Input: String acronym – the acronym Boolean value - the value Exception: -
addDate()	Adds a date value for an acronym to the PDM string (format = MM/DD/YYYY) Important: If the value is empty, the complete acronym is not passed to the PDM service. If this behavior is not wanted, use the method *NoSkipEmptyValue().

	Return type: . Input: String acronym – the acronym Calendar value – the value Exception: -
addTime()	Adds a time value for an acronym to the PDM string (in seconds since 00:00:00) Important: If the value is empty, the complete acronym is not passed to the PDM service. If this behavior is not wanted, use the method *NoSkipEmptyValue(). Return type: . Input: String acronym – the acronym Calendar value – the value Exception: -
addTimestamp()	Adds a time stamp value for an acronym to the PDM string (format = MM/DD/YYYY HH:mm:ss) Important: If the value is empty, the complete acronym is not passed to the PDM service. If this behavior is not wanted, use the method *NoSkipEmptyValue(). Return type:

	. Input: String acronym – the acronym Calendar value – the value Exception: -
addPdmString()	Adds a (sub-)PDM string (e.g. ACR1=VAL1 ACR2=VAL2)). Note: This method parses the string into its elements and combines the elements with the other values from other methods later on. Important: If the value is empty, the complete acronym is not passed to the PDM service. If this behavior is not wanted, use the method *NoSkipEmptyValue(). Return type: . Input: String pdmString – the (sub-) PDM string Exception: -
addStringNoSkipEmptyValue()	Adds a string value for an acronym to the PDM string. Return type: . Input: String acronym – the acronym

	String value - the value Exception: -
addIntegerNoSkipEmptyValue()	Adds an integer value for an acronym to the PDM string Return type: . Input: String acronym – the acronym Integer value - the value Exception: -
addDecimalNoSkipEmptyValue()	Adds a BigDecimal value for an acronym to the PDM string Return type: . Input: String acronym – the acronym BigDecimal value – the value Exception: -
addBooleanNoSkipEmptyValue()	Adds a Boolean value for an acronym to the PDM string (J for true and N for false) Return type: .

	Input: String acronym – the acronym Boolean value - the value Exception: -
addDateNoSkipEmptyValue()	Adds a date value for an acronym to the PDM string (format = MM/DD/YYYY) Return type: . Input: String acronym – the acronym Calendar value – the value Exception: -
addTimeNoSkipEmptyValue()	Adds a time value for an acronym to the PDM string (in seconds since 00:00:00) Return type: . Input: String acronym – the acronym Calendar value – the value Exception: -
addTimestampNoSkipEmptyValue()	Adds a time stamp value for an acronym to the PDM string (format = MM/DD/YYYY HH:mm:ss)

	Return type: . Input: String acronym – the acronym Calendar value – the value Exception: -
addPdmStringNoSkipEmptyValue()	Adds a (sub-)PDM string (e.g. ACR1=VAL1 ACR2=VAL2). Note: This method parses the string into its elements and combines the elements with the other values from other methods later on. Return type: . Input: String pdmString – the (sub-) PDM string Exception: -
callHydra()	Executes the PDM call and provides the results. In case of an error, the HydraReturnValue has a return code != 0 Return type: HydraCallResult – description, see below Input: - Exception:

	-
--	---

28.12.1 Data class HydraCallResult

Variable with type	Description
IDataTable resultTable	The payload of the return string / the file of the PDM call All fields of the string type For further details on the type, refer to the data types for "Simple External Services"
HydraReturnValue returnValue	Result of calling (RET/KT/LT/DATEI)

28.12.2 Data class HydraReturnValue

Variable with type	Description
Integer returnCode	Value of the result string of the RET acronym. In case of an error !=0.
String shortText	Value of the result string of the KT acronym
String longText	Value of the result string of the LT acronym
String fileName	Value of the result string of the DATEI acronym

28.13 Interface IPdmStringParser

You can use the interface IPdmStringParser to parse PDM strings. Supported methods are the key/value method and the value method.

Method	Description
parseKeyValueString()	Parses a PDM string in Key/Value format (e.g. ACR1=Wert1 ACR2=Wert2). Escaping reserved characters (e.g. \) is supported. Return type: Map<String, String> - acronyms including their values Input: String input – PDM string Exception: -
parseUnescapedKeyValueString()	Parses a PDM string in Key/Value format (e.g. ACR1=Wert1 ACR2=Wert2). Escaping reserved characters (e.g. \) is NOT supported. Return type: Map<String, String> - acronyms including their values Input: String input – PDM string Exception: -
parseValueOnlyString()	Parses a PDM string including only values (e.g.

	Wert1 Wert2) Escaping reserved characters (e.g. \) is supported. Return type: List<String> - the values Input: String input – PDM string Exception: -
parseUnescapedValueOnlyString()	Parses a PDM string including only values (e.g. Wert1 Wert2) Escaping reserved characters (e.g. \) is NOT supported. Return type: List<String> - the values Input: String input – PDM string Exception: -

28.14 Interface IDataTableBuilder

Use the interface IDataTableBuilder to create DataTables.

Method	Description
setDataTableId()	Sets table ID Return type:

	IDataTableBuilder – reference to the builder Input: String dataTableId – ID of the DataTable Exception: -
addCol()	Adds a column to the table - must not be requested if rows/data have already been added. For further details on the DataType type, refer to the data types of "Simple External Services" Return type: IDataTableBuilder – reference to the builder Input: String colName – column name DataType columnType – column type Exception: IllegalStateException – if data / a row has been added
addRow()	Adds an empty row to the table Return type: IDataTableBuilder – reference to the builder Input: - Exception: -
addRow()	Adds a row including values to the table

	Return type: IDataTableBuilder – reference to the builder Input: Object... values – the values (there must be as many values as columns) Exception: IllegalArgumentException – if the number of values does not match the number of columns
value()	Sets the value in the next column of the current row Return type: IDataTableBuilder – reference to the builder Input: Object value – the value Exception: IllegalStateException – if no row is generated IndexOutOfBoundsException – if the last column is reached and an attempt is made to set a value
build()	Generate DataTable For further details on the type IDataTable, refer to the data types of "Simple External Services" Return type: IDataTable – the table Input: -

	Exception: -
--	------------------------

28.15 Interface IDataTableModifier

You use the interface IDataTableModifier to modify DataTables. A copy is made prior to processing. Consequently, changes are not made to the original DataTable.

Method	Description
setDataTable()	Setting the DataTable Return type: IDataTableModifier – reference to the modifier Input: IDataTable table – the DataTable Exception: -
addRow()	Adds an empty row to the table Return type: IDataTableModifier – reference to the modifier Input: - Exception: -
removeRow()	Removes a row from the table

	Return type: IDataTableModifier – reference to the modifier Input: int rowIndex – index of the row Exception: IndexOutOfBoundsException – if the row with the index does not exist
addColumn()	Adds a column to the table For further details on the DataType type, refer to the data types of "Simple External Services" Return type: IDataTableModifier – reference to the modifier Input: String columnName – column name DataType columnType – column type Exception: IllegalStateException – if a column with the name exists
removeColumn()	Removes a column from the table Return type: IDataTableModifier – reference to the modifier Input: String columnName – column name

	Exception: IllegalStateException – if a column with the name does not exist
setDataTableId()	Sets table ID Return type: IDataTableModifier – reference to the modifier Input: String dataTableId – ID of the DataTable Exception: -
setValue()	Sets the value in a cell Return type: IDataTableModifier – reference to the modifier Input: int rowIndex – index of the row String columnName – name of column Object value – the value Exception: IndexOutOfBoundsException – if the row with the index does not exist IllegalStateException – if a column with the name does not exist
getModifiedTable()	Return of the modified DataTable Return type:

	IDDataTable – the table Input: - Exception: -
--	---

28.16 Interface IFeatureSetChecker

You can use the interface IFeatureSetChecker to check if a feature set is active.

Method	Description
isFeatureSetActive()	Checks if feature set is active Return type: Boolean - true if active, otherwise false Input: String name – name of feature set Exception: SdiException with key IkErrorCheckingFeatureSet if an error occurred during the check

28.17 Interface IDbTypeRetriever

You can use this interface to identify the database type (Oracle or MS SQL Server)

Method	Description
getDbType()	Returns the DB type as string Oracle = "ORACLE" MS SQL Server = "SQL_SERVER"

	Return type: String – the DB type Input: - Exception: -
isOracleDb()	Verifies if it is an Oracle DB Return type: Boolean – true if it is an Oracle DB Input: - Exception: -
isSqlServerDb()	Verifies if it is an MS SQL Server DB Return type: Boolean – true if it is an MS SQL Server DB Input: - Exception: -

28.18 Interface IUserExitProvider



Deprecated as of SP8: can be replaced by IExitMethodExecutionManager.

You can use this interface to generate a user exit object from a user exit class (scope is identified automatically).

Method	Description
fetchUserExit()	Generates a user exit object from the user exit class of the current service. The class is loaded from the scope with the highest priority. Return type: IUserExit – the user exit object Input: - Exception: -
fetchUserExit()	Generates a user exit object from the specified user exit class. The class is loaded from the scope with the highest priority. Return type: IUserExit – the user exit object Input: String userExitId – the full class name of the user exit (including package) Exception: -

28.19 Interface IUserExitFromScopeProvider



Deprecated as of SP8: can be replaced by IExitMethodExecutionManager.

You can use this interface to generate a user exit object from a user exit class (scope is specified).

Method	Description
fetchUserExitFromScope()	Generates a user exit object from the user exit class of the current service. The class is loaded from the specified scope. Return type: IUserExit – the user exit object Input: String scope – the scope (LOCAL, CUSTOM, STANDARD) Exception: -
fetchUserExitFromScope()	Generates a user exit object from the specified user exit class. The class is loaded from the specified scope. Return type: IUserExit – the user exit object Input: String userExitId – the full class name of the user exit (including package) String scope – the scope (LOCAL, CUSTOM, STANDARD)

	Exception: -
--	----------------------------

28.20 Interface IUserExit

You can use this interface to access a loaded user exit class.

Method	Description
getScope()	Identifies the scope that was used to load the user exit class. Return type: String – the scope (LOCAL, CUSTOM, STANDARD) Input: - Exception: -
invoke()	Calls a method of the user exit class Return type: Object - the return object of the user exit (the class is different for each user exit) Input: String methodName – method name Object context – context object (the class depends on the source that called the user exit)

	Object param – parameter object (the class is different for each user exit) Exception: -
--	--

28.21 Interface IAcronymMappingProvider

You can use this interface to access acronym mapping.

Method	Description
getAcronymMapping()	Loads the acronym mapping for the current service Return type: IAcronymMapping – the mapping object Input: - Exception: -

28.21.1 Interface IAcronymMapping

You use this interface to provide access to the acronym mapping.

Method	Description
mapFromServiceAcronym()	Returns mapping for the service acronym Return type: String – the mapping (null if not available) Input:

	String serviceAcronym – the service acronym Exception: -
mapToServiceAcronym()	Returns the service acronym for the mapping Return type: String – the service acronym (null if not available) Input: String mappedAcronym - the mapped acronym Exception: -
getFromServiceAcronymMapping ()	Provides a map with assignment service acronym - > mapped acronym Return type: Map<String, String> - the map including assignment Input: - Exception: -
getServiceAcronymMapping ()	Provides a map with assignment mapped acronym -> service acronym Return type: Map<String, String> - the map including assignment Input:

	- Exception: -
--	-------------------------------------

28.22 Interface IFormulaProvider

You can use this interface to create and work with formulas based on a term.

Method	Description
createFormula ()	Creates the formula object for the term Return type: IFormula – the formula object Input: String formulaTerm – the term MissingParamHandling missingParamHandling – specifies the next steps if one of the placeholders is null or not set in the formula (ERROR or IGNORE are possible). Exception: -

28.22.1 Interface IFormula

You use this interface to provide access to the generated formula

Method	Description
getPlaceholders()	Returns all placeholders (variables) used in the formula Return type:

	List<String> – the placeholders Input: - Exception: -
setStringParam()	Sets a placeholder with string value Return type: - Input: String paramName – placeholder name String value - the value Exception: -
setIntegerParam()	Sets a placeholder with integer value Return type: - Input: String paramName – placeholder name Integer value - the value Exception: -
setDecimalParam()	Sets a placeholder with decimal value Return type: -

	Input: String paramName – placeholder name BigDecimal value – the value Exception: -
setBooleanParam()	Sets a placeholder with Boolean value Return type: - Input: String paramName – placeholder name Boolean value - the value Exception: -
evaluate()	Calculates the formula Return type: IFormulaEvaluationResult – result of the calculation (or null if value is not set - IGNORE mode) Input: - Exception: SdiException – If value is not set (ERROR mode)

28.22.2 Interface IFormulaEvaluationResult

You use this interface to provide access to the result of formula calculation.

Method	Description
getStringResult()	Gets the result as string (if possible) Return type: String - result Input: - Exception: -
getIntegerResult()	Gets the result as integer (if possible) Return type: Integer result Input: - Exception: -
getDecimalResult()	Gets the result as decimal (if possible) Return type: BigDecimal - result Input: - Exception: -
getBooleanResult()	Gets the result as Boolean (if possible)

	Return type: Boolean - result Input: - Exception: -
--	---

28.23 Interface IServiceCaller

You use this interface for the internal request of another service.

Method	Description
callService()	Internal service call Return type: IServiceCallResult – call result Input: String serviceName – name of service ColumnConfigurator colConf – column configurator List<SpecialParam> specialParams – Special Parameters List<FilterParam> filterParams – Filter Parameters (for the Where clause in SQL) Exception: SdiException – If a special parameter or filter parameter is null or if the parameter has a data type that is not supported, if the service provides several tables or additional information on the result, if the service execution results in an error.

28.23.1 Interface IServiceCallResult

You use this interface to provide access to the result of a service call.

Method	Description
getResultTable()	Provides the result table Return type: IDataTable – the table (or null if no table is provided) Input: - Exception: -

28.23.2 Class FilterParam

This is the data type that includes information on a filter parameter. The data of this type cannot be changed.

Method	Description
getAcronym	Provides the acronym of this filter parameter Return type: String
getOperator	Provides the operator of this filter parameter Return type: OperatorType (enum)
getValue	Provides the value of this filter parameter Return type: Object The actual data type depends on the parameter type. These types are possible: - String - Integer

	- BigDecimal - Boolean - Byte[] - Calendar - String[] - Integer[] - BigDecimal[] - Boolean[] - Calendar[]
--	---

28.24 Interface IDataAvailabilityChecker

You can use this interface to check data availability for product(s)/object(s) for a specific period of time.

Method	Description
checkAvailability()	Checks availability Return type: DataAvailability – result of checking Input: DataAvailabilityCheckData checkData – parameters for checking Exception: -

28.24.1 Class DataAvailabilityCheckData

These are the call parameters for the check

Method	Description
getEvaluationType	Type of evaluation (based on shifts or periods of time) SHIFT TIMERANGE

	Return type: String
getBegin	Start of evaluation period Return type: Calendar
getEnd	End of evaluation period Return type: Calendar
isCheckCurrentAvailability	Check if current data actually include values matching the relevant period of time Return type: Boolean
getCheckObjects	Combinations of product/product object Return type: DataAvailabilityObject[]

28.24.2 Class DataAvailabilityObject

These are the call parameters for the check

Method	Description
getProduct	Product: MDE ADE WRM Return type: String
getProductObject	Object pertaining to the product MDEPRO, EREIGMDE, RES_STATUS ADEPRO, ANR WRMPRO, EREIGWRM Return type: String

28.24.3 Class DataAvailability

This is the result of the availability check.

Method	Description
isNoDataAvailable	Flag that specifies if any data is available (if not, you need not select data). Return type: Boolean
isLongTermDataNeeded	Flag that specifies if archive tables must be included Return type: Boolean
getInfoLanguageKey	Additional information if data is only partially available - language key of message Return type: String
getInfoShortMessage	Additional information if data is only partially available - brief information (internal) of message Return type: String
getInfoParams	Additional information if data is only partially available - parameters for the language key Return type: Object[]

28.25 Interface IJhydradirPathProvider

You use this interface to identify the path to the JDIR or the JHYDRADIR directory, which includes the configuration of the WSP.

Method	Description
getPath(): String	 Return type:

	String – path to JDIR Input:
--	--

28.26 Interface IJhydradirInstancePathProvider

This interface is used to identify the path to the instance directory of the JDIR or JHYDRADIR directory in the MOC subfolder where the instance configuration of the WSP is located.

Method	Description
getPath(): String	Return type: String – path to JDIR instance directory. Input:

28.27 Interface IJhydradirTempPathProvider

You use this interface to identify the path to the directory for temporary data of the JDIR or the JHYDRADIR directory.

Method	Description
getPath(): String	Return type: String – path to the temporary directory to JDIR or JHYDRADIR Input:

28.28 Reading settings from the configuration files

28.28.1 WSP configuration files

When the WSP is installed, there are two configuration files "config.properties" where settings are made using key/value pairs.

File	Example
Global configuration file	\\myserver\mip1\jdir\MOC\config.properties
Instance configuration file	\\myserver\mip1\jdir\MOC\1\config.properties

Example:

```
#####
#####INSTANCE CONFIGURATION (1)#####
#####
# Please make sure that after configuration values is no TAB or SPACE

# Config reload timeout in seconds
# After this timeout it is checked, if the instance configuration (this file) has changed and the config must be reloaded
configreload.timeout=10

development.mode=1

#####
#####DATABASE CONFIGURATION#####
#####
# The name of the instance. This is used for configuration of database connection pool
instance.name=mip1

# DB-type (valid values: oracle or sqlserver):
# Needed for the mapping of SQL errors
# db.type=oracle
db.type=sqlserver

# Is the Database a unicode DB? If yes the value must be 1 else 0
db.unicode=1
...
```

28.28.2 Interface IGlobalConfigValueProvider

The IGlobalConfigValueProvider interface allows configuration values to be read from the **global** instance configuration file.

Method	Description
getConfigValue(String key): String	Input: String: key for the configuration value Return type: String – configuration value

28.28.3 Interface IInstanceConfigValueProvider

The IInstanceConfigValueProvider interface enables configuration values to be read from **the instance** configuration file.

Method	Description
getConfigValue(String key): String	Input: String: key of the configuration value Return type: String – configuration value

28.29 Interface IHydraPathReadFileAction

You use this interface to read files that you can reach via configured path.

Method	Description
openFile(String pathName, String subPath): InputStream	Return type: InputStream – opened file stream Input: String pathName: name of the path configuration Paths are configured in the system and identified here by the name of the configuration. String subPath: Subpath in path or NULL, if direct path

28.30 Interface IHydraPathWriteFileAction

You use this interface to write files that you can reach via configured path.

Method	Description
openFile(String pathName, String subPath): void	

	Return type: Input: InputStream fileData: file data to be written String pathName: configuration name of a path. String subPath: Subpath in path or NULL, if direct path
--	--

28.31 Interface IHydraPathRenameFileAction

You use this interface to rename files that you can reach via configured path.

Method	Description
renameFile (String pathName, String sourceName, String targetName): boolean	Return type: Boolean – TRUE if renaming was successful; otherwise FALSE Input: String pathName: configuration name of a path String sourceName: source file name String targetName: target file name

28.32 Interface IHydraPathDeleteFileAction

You use this interface to delete files that you can reach via configured path.

Method	Description
deleteFile(String pathName, String fileName): boolean	Return type: Boolean – TRUE if successful; otherwise FALSE

	Input: String pathName: path name String fileName: file name including potential sub-directories
--	---

28.33 Interface IHydraPathFileExistsAction

You use this interface to check if files exist that you can reach via configured path.

Method	Description
exists(String pathName, String subPath): boolean	Return type: Boolean – TRUE if file exists; otherwise FALSE Input: String pathName: path name String subPath: subpath in the path

28.34 Interface IHydraPathCreateFolderAction

You use this interface to create directories that you can reach via configured path.

Method	Description
createFolder(String pathName, String subPath, String folderName): boolean	Return type: Boolean – TRUE if successful; otherwise FALSE Input: String pathName: path name String subPath: subpath in the path String folderName name of the directory to be

	created
--	---------

28.35 Interface IHydraPathDeleteFolderAction

You use this interface to delete directories that you can reach via configured path.

Method	Description
deleteFolder(String pathName, String subPath, String folderName): boolean	Return type: Boolean – TRUE if successful; otherwise FALSE Input: String pathName: path name String subPath: subpath in the path String folderName name of the directory to be deleted

28.36 Interface IHydraPathDownloadContentAction

You use this interface to download complete directories that you can reach via configured path.

Method	Description
downloadContent(String pathName, String subPath, String localDestinationFolderPath, boolean recursive): void	Return type: Input: String pathName: path name String subPath: subpath in the path String localDestinationFolderPath name of the local target directory Boolean recursive: if TRUE all subdirectories are included, otherwise only files included in this

	directory
--	-----------

28.37 Interface IHydraPathUploadContentAction

You use this interface to upload a local directory to a directory that you can reach via configured path.

Method	Description
uploadContent(String pathName, String subPath, String localSourceFolder): void	Return type: Input: String pathName: path name String subPath: subpath in the path String localSourceFolder name of the local source directory

28.38 Interface IHydraPathListContentAction

You use this interface to list the content of a directory that you can reach via configured path.

Method	Description
uploadContent(String pathName, String subPath, boolean recursive): List<IHydraPathElement>	Return type: List<IHydraPathElement>: List including directory contents Input: String pathName: path name String subPath: subpath in the path Boolean recursive: if TRUE all subdirectories are included, otherwise only files included in this directory

28.39 Interface IHydraPathIsDirectoryAction

You use this interface to check if a path in a directory that you can reach via configured path is a directory or not.

Method	Description
isDirectory(String pathName, String subPath): boolean	Return type: boolean – TRUE if subPath is a directory; otherwise FALSE Input: String pathName: path name String subPath: subpath in the path

28.40 Interface IPersonCardIdChecker

You use this interface to check if a person exists, if the person is not locked and has joined/not yet left the company. The check uses the badge number.

Method	Description
checkCardId()	Checks the person using the badge number. The specified date is used to verify the date of joining/leaving the company. Return type: IPersonCardIdCheckResult – result of the check Input: String cardId – the badge number Calendar checkDate – date to check the date of joining/leaving the company Exception:

	SesException – if an error occurred with database access
checkCardIdForToday()	Performs the check as described for checkCardId and uses the current day as the checkDate. Return type: IPersonCardIdCheckResult – result of the check Input: String cardId – the badge number Exception: SesException – if an error occurred with database access

28.40.1 Interface IPersonCardIdCheckResult

This interface provides the result of the personnel verification using the badge number

Method	Description
getPersonId()	Provides the personnel number for the badge number (if person exists) Return type: Integer – the personnel number Input: - Exception: -
getUserId()	Provides the user assigned to the person (if person exists and a user is assigned) Return type:

	String – the user Input: - Exception: -
getCheckResult()	Provides the result of the check Return type: PersonCardIdCheckState – the actual result of the check Input: - Exception: -

28.40.2 Enum PersonCardIdCheckState

Includes the result of the personnel verification

Value	Description
PERSON_NOT_EXISTING	Person does not exist
PERSON_LOCKED	Person has been locked
PERSON_NOT_JOINED_YET	Person has not yet joined the company by the date of check
PERSON_ALREADY_LEAVED	Person has already left the company by the date of check
CHECK_OK	Everything all right

28.41 Enum CheckResponsibilityAreaMode

Modes to check the authorizations for responsibility areas

Value	Description
VIEW	Single authorization for viewing
USE	Single authorization for using
INSERT	Single authorization for creating
UPDATE	Single authorization for editing
DELETE	Single authorization for deleting
LOCK	Single authorization for locking
COPY	Single authorization for copying
LIST	Single authorization for listing
ANY	There is a row in the table for responsibility areas. All single authorizations may have any value.

28.42 Interface IResponsibilityAreaChecker

You can use this interface to check whether or not a user is authorized for a specific responsibility area.

Method	Description
isResponsibilityAreaPermissionGranted(ResponsibilityAreaCheckerParam param): boolean	Return type: Boolean – TRUE the user has the required authorization Input: ResponsibilityAreaCheckerParam param: parameter structure

28.42.1 Class ResponsibilityAreaCheckerParam

Parameter class to check responsibility areas

Field	Description
String: user	User
String: responsibilityArea	Responsibility area to be checked (if it is allowed for the user)
CheckResponsibilityAreaMode: mode	Mode of checking (optional, if left out, ANY)

28.43 Interface IPersonResponsibilityAreaChecker

You can use this interface to check whether or not a user is authorized for a specific person.

Method	Description
isPersonResponsibilityAreaPermissionGranted(PersonResponsibilityAreaCheckerParam param): PersonResponsibilityAreaCheckerResult	Return type: PersonResponsibilityAreaCheckerResult Input: PersonResponsibilityAreaCheckerParam param: parameter structure

28.43.1 Enum PersonResponsibilityAreaCheckerResult

Parameter class to check personnel responsibility areas

Value	Description
ALLOWED	Authorization available
NOT_ALLOWED	Authorization not available

PERSON_NOT_EXISTING	Person to be checked does not exist
---------------------	-------------------------------------

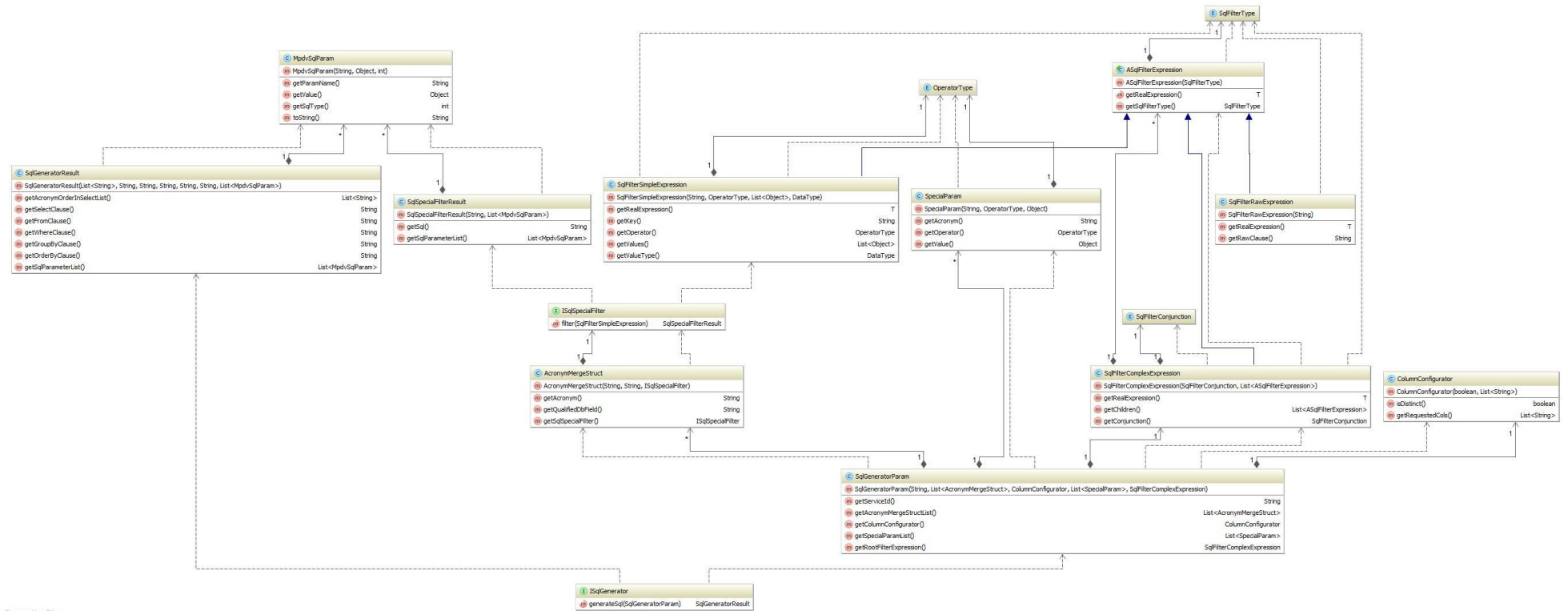
28.44 Interface ISqlGenerator

You use this interface to generate SQL statements using a service configuration. This means: Via the interface, you can generate the SQL that is executed via InterpretedJavaService2, but you do not execute it.

Note: If the acronyms in the service, which is intended to be used as generation basis, include the value "SKIP_INTERPRETER" in the field "Constraints" of the configuration, these acronyms are skipped by the SQL generator.

Method	Description
generateSql(SqlGeneratorParam sqlGeneratorParameter): SqlGeneratorResult	Return type: SqlGeneratorResult Input: SqlGeneratorParam sqlGeneratorParameter: parameter structure

Overview of the classes involved:



Powered by jFiles

28.44.1 Class SqlGeneratorParam

This is the request parameter structure.

Method	Description
getServiceId	Name of the service that is used as basis for generation. Return type: String
getAcronymMergeStructList	List with additional or overwriting acronym configurations: All acronyms passed here overwrite the acronyms that might exist in the service configuration. Return type: List<AcronymMergeStruct>
getColumnConfigurator	Column configuration to be used: Specifies the number and the order of the selected columns. Return type: ColumnConfigurator
getSpecialParamList	List of SpecialParameters to be used Return type: List<SpecialParam>
getRootFilterExpression	Root nodes of the FilterParameter in a tree structure. All filter parameters to be used must be included. Return type: SqlFilterComplexExpression

28.44.2 Class AcronymMergeStruct

You use this structure to overwrite an acronym configuration in the service used for generation.

Method	Description
getAcronym	Acronym of the service parameter

	Return type: String
getQualifiedDbField	Qualified database field. This means that the structure is as follows: <table alias>.<table field>. You may not use %1\$s ! Return type: String
getSqlSpecialFilter	Callback for a special filtering for the current acronym or NULL if no special filtering is required. A special filtering or user-specific filtering is a filtering that does not have the structure <qualified DB field> <operator> <value>. Return type: ISqlSpecialFilter

28.44.3 Interface ISqlSpecialFilter

You use this interface to implement a callback for special filterings or user-defined filterings that do not have the structure <qualified DB field> <operator> <value>.

Method	Description
filter(SqlFilterSimpleExpression simpleExpression) : SqlSpecialFilterResult	Acronym of the service parameter Return type: SqlSpecialFilterResult: may not be NULL Input: SqlFilterSimpleExpression simpleExpression: parameter structure

28.44.4 Class SqlSpecialFilterResult

This structure includes the SQL snippet for the WHERE condition for the filtering.

Method	Description
--------	-------------

getSql()	SQL snippet for WHERE condition. May not be NULL. Return type: String
getSqlParameterList()	List of parameters for the SQL snippet. May not be NULL. Return type: List<MpdvSqlParam>

28.44.5 Class ASqlFilterExpression

Abstract superclass for all filter parameter structures.

Method	Description
getSqlFilterType():SqlFilterType	Type of SQL filter parameter structure Return type: SqlFilterType
getRealExpression(): T	Returns the actual type of the filter parameter structure without external CAST. Return type: T extends ASqlFilterExpression

28.44.6 Enum SqlFilterType

Includes the type of the SQL filter parameter structure

Value	Description
COMPLEX	Node that can have 0 to n child nodes and is either an OR or an AND conjunction.
SIMPLE	Leaf that consists of key, operator and value list.
RAW	Special leaf that includes a blank SQL snippet.

28.44.7 Class SqlFilterComplexExpression

This structure is an AND or an OR conjunction and includes all linked elements as child elements.

Method	Description
getChildren():List<ASqlFilterExpression>	All child elements of this conjunction Return type: List<ASqlFilterExpression>
getConjunction():SqlFilterConjunction	Type of conjunction: AND or OR. Return type: SqlFilterConjunction

28.44.8 Enum SqlFilterConjunction

Includes the type of the SQL conjunction.

Value	Description
AND	AND conjunction
OR	OR conjunction

28.44.9 Class SqlFilterSimpleExpression

This structure is a filter for an acronym.

Method	Description
getKey(): String	Acronym to be filtered by. Return type: String
getOperator(): OperatorType	Operator used for the filtering. Return type: OperatorType
getValues(): List<Object>	List of values to be filtered by. Return type: List<Object>

getValueType(): DataType	Data type of the values to be filtered by. Return type: DataType
--------------------------	--

28.44.10 Class SqlFilterRawExpression

This structure is a special filter that consists of one WHERE snippet only.

Method	Description
getRawClause(): String	SQL snippet to be directly embedded in the WHERE clause. Return type: String

28.45 Interface IMemoryChecker

You can use this utility to register the memory usage in estimated units. A unit can be a cell in a table or 1 kilobyte. If you use this utility, you ensure the stability of Java because the current request is interrupted if the memory demand is too high.

Method	Description
reportAdditionalMemoryConsumption(int units): void	Registers an estimation of the memory demand. This value is added internally to the former estimation. Input: int units – the estimated quantity of additionally used "units". Exception: If the current request requires too much memory, a RuntimeException is thrown and the request is stopped.

28.46 Interface IMandatorySpecialParameterValidator

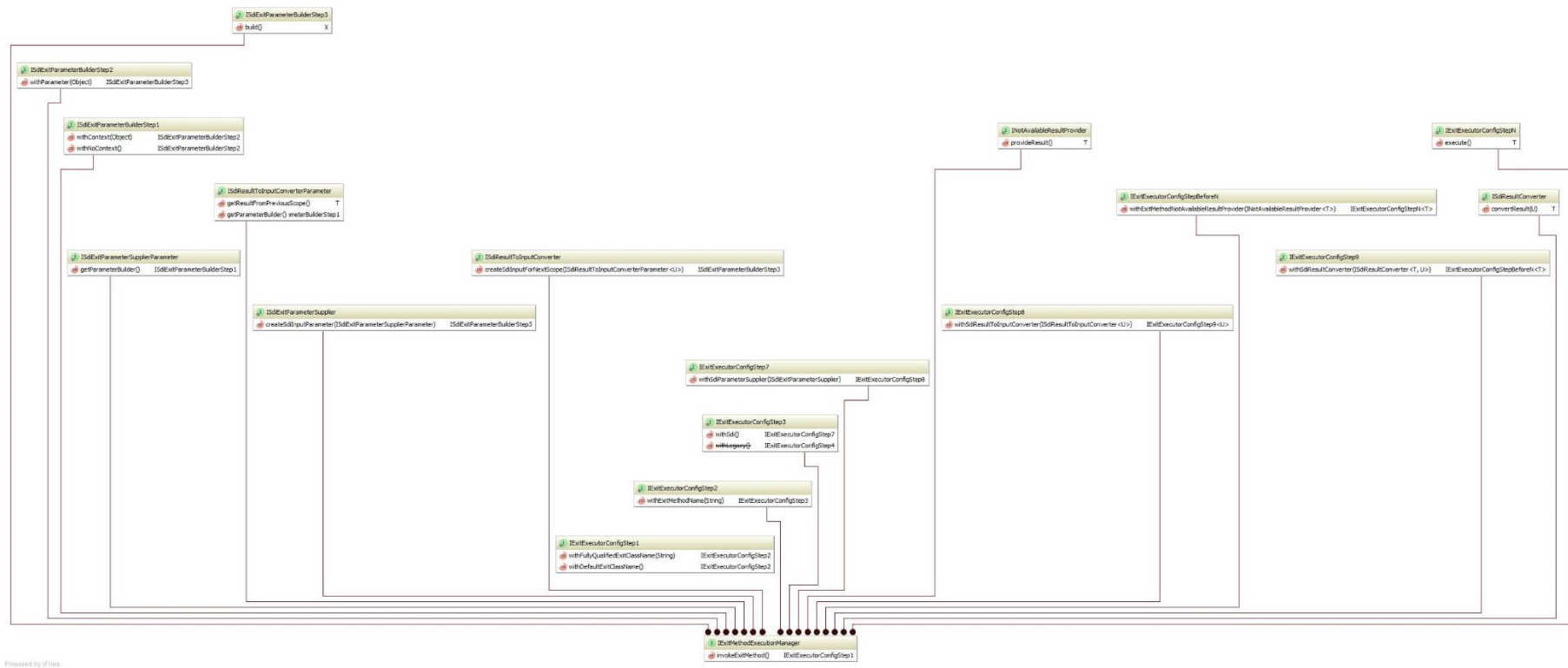
You use this interface to check the configured mandatory parameters of type SpecialParameter in ExternalServices. This interface is interesting for user exits, if other SpecialParameters become mandatory parameters depending on one or several other SpecialParameters.

Method	Description
validateMandatorySpecialParameters(Map<String, SpecialParam> specialParamMap, List<String> additionalMandatoryParameterAcronyms): void	Validates the SpecialParameters using the configuration of the current service. Input: specialParamMap – the SpecialParameters to be validated additionalMandatoryParameterAcronyms – a list including optional additional SpecialParameters to be validated that are not configured as mandatory parameters, but are to be treated as such. Exception: If one or several mandatory parameters are missing
validateMandatorySpecialParameters(String serviceId, Map<String, SpecialParam> specialParamMap, List<String> additionalMandatoryParameterAcronyms): void	Validates the SpecialParameters using the configuration of the service passed (serviceId parameter). Input: serviceId – name of the service whose configuration is to be used for the validation. specialParamMap – the SpecialParameters to be validated additionalMandatoryParameterAcronyms – a list including optional additional SpecialParameters to be validated that are not configured as mandatory parameters, but are to be treated as such.

	Exception: If one or several mandatory parameters are missing
--	---

28.47 Interface IExitMethodExecutionManager

Method	Description
invokeExitMethod: IExitExecutorConfigStep1	Validates the SpecialParameters using the configuration of the current service. Output: IExitExecutorConfigStep1: Step Builder that results in the execution of an exit method via the required steps. After each method, only the next possible step is available and the CodeCompletion in the IDEA is supported step by step when an exit method is called.



```

public void systemUtilFactoryCreatesExitApiWithSdi()
{
    final IExitMethodExecutionManager exitMethodExecutorManager =
this.systemUtilFactory.fetchUtil("ExitMethodExecutionManager");

    final String[] whereConditionHolder = {"b.col6 = 42"};
    final TestResultStructure exitResult = exitMethodExecutorManager.invokeExitMethod()
        .withDefaultExitClassName()
        .withExitMethodName("myExitMethod")
        .withSdi()
        .withSdiParameterSupplier(new ISdiExitParameterSupplier()
        {
            public ISdiExitParameterBuilderStep3 createSdiInputParameter(final
ISdiExitParameterSupplierParameter parameterSupplierParameter)
            {
                return parameterSupplierParameter.getParameterBuilder()
                    .withContext(new InterpretedJavaServiceUeContext(Calendar.getInstance(), "TestUser"))
                    .withParameter(new SdiAugmentSqlParam(
                        "a.col1, b.col2",
                        "testTable a join testTable2 b on a.col3 = b.col4",
                        whereConditionHolder[0],
                        "",
                        "",
                        Collections.<String, SpecialParam>singletonMap(
                            "testService.acronym1", new SpecialParam("testService.acronym1",
OperatorType.EQUAL, Integer.valueOf(42))
                        )
                    ));
            }
        })
        .withSdiResultToInputConverter(new ISdiResultToInputConverter<SdiAugmentSqlResult>()
        {
            public ISdiExitParameterBuilderStep3 createSdiInputForNextScope(final
ISdiResultToInputConverterParameter<SdiAugmentSqlResult> resultToInputConverterParameter)
            {
                final SdiAugmentSqlResult resultFromPreviousScope =
resultToInputConverterParameter.getResultFromPreviousScope();

                // merge the results with the original where condition
                whereConditionHolder[0] = mergeString(whereConditionHolder[0],
resultFromPreviousScope.getWhereSuffix());

                return resultToInputConverterParameter.getParameterBuilder()
                    .withContext(new InterpretedJavaServiceUeContext(Calendar.getInstance(), "TestUser"))
                    .withParameter(new SdiAugmentSqlParam(
                        "a.col1, b.col2",

```

```

        "testTable a join testTable2 b on a.col3 = b.col4",
        whereConditionHolder[0],
        "",
        "",
        Collections.<String, SpecialParam>singletonMap(
            "testService.acronym1", new SpecialParam("testService.acronym1",
OperatorType.EQUAL, Integer.valueOf(42))
        )
    ));
}

}).withSdiResultConverter(new ISdiResultConverter<TestResultStructure, SdiAugmentSqlResult>()
{
    public TestResultStructure convertResult(final SdiAugmentSqlResult sdiExitResultStructure)
    {
        // merge the results with the previous where condition
    }
}

```

28.48 Interface ISdiDataRowCopyUtil

You can use this util to copy an ISdiDataRow.

Method	Description
copyRow(ISdiDataRow „row“): ISdiDataRow	Copies the ISdiDataRow Input: ISdiDataRow „row“ – row to be copied Output: Copy of the row Note: A direct implementation of ISdiDataRow is not allowed!

28.49 Interface IDataTableToStreamConverter

You can use this util to provide the content of an IDataTable as ISdiDataRowStream.

Method	Description
convert(IDataTable table): ISdiDataRowStream	Provides the content of the IDataTable as stream Input: IDataTable table – The table Output: Stream that provides the content of the table

28.50 Interface IStreamToDataTableConverter

You can use this util to convert the content, which is provided by a ISdiDataRowStream, into an IDataTable.

Method	Description
convert(ISdiDataRowStream stream): IDataTable	Iterates all rows that the stream provides and creates an IDataTable. Input: ISdiDataRowStream stream – The stream Output: A table with the content of the stream.

28.51 Interface IResultTransformationManager

You use this util to apply the result transformations on an ISdiDataRowStream using the repository configuration (via service name). You can also apply own additional transformations (optional).

You can also identify the DB data type for all fields with result transformation.

Method	Description
applyResultTransformations(String serviceName, ISdiDataRowStream stream, List<ISdiResultTransformationCallback> additionalTransformations): ISdiDataRowStream	Applies the result transformations on an ISdiDataRowStream using the repository configuration (via service name). You can also use own additional transformations (optional). Input: String serviceName – name of service ISdiDataRowStream stream – The input data stream List<ISdiResultTransformationCallback> additionalTransformations – Own transformation implementations Output: A stream with applied transformations
getDbSelectionTypesForTransformedFields(String serviceName): Map<String, DataType>	Some result transformations change the data type of a field. In this case, the repository includes the data type after the transformation. When you read from the DB, you must perhaps use another data type. This method provides the DB type for all fields with result transformation. Input: String serviceName – name of service Output: A map that includes the DB data type of the acronym (ONLY for fields with result transformations using the repository configuration).

28.52 Auxilliary classes

28.52.1 Class SdiEagerDataRowStream

You can use the SdiEagerDataRowStream class for services to stream rows into the found result set of a service.

Examples on how to use the classes:

„GlobalExits“

Call back function registered with user exit „sdiGlobalAddResultTransformationCallbacks“

„InterpretedJavaService2“

Call back function registered with user exit „sdiAddResultTransformationCallbacks“

The polymorphic constructor of the class allows you to transfer the output rows in various ways:

SdiEagerDataRowStream()

Call without dataRows This variant is used in the above examples of callbacks to delete rows from the result.

SdiEagerDataRowStream(ISdiDataRow dataRow)

Callback with a dataRow. The transferred row is added to the result. This variant is normally used in the above examples of callbacks to modify the row transferred to the callback function and add the modified row to the result.

SdiEagerDataRowStream(List<ISdiDataRow> dataRows)

Callback with a list of dataRows. This list may not be null. The transferred rows are added to the result. This variant is mostly used in the above examples of callbacks to add more rows when individual rows are encountered. Follow the instructions below for creating new rows.

SdiEagerDataRowStream(ISdiDataRow... dataRows)

Callback of several dataRows in form of variable parameter list. The transferred rows are added to the result. This variant is mostly used in the above examples of callbacks to add more rows when individual rows are encountered. Follow the instructions below for creating new rows.



If you want to generate instances from ISdiDataRow, always use a factory method!

Do not implement the interface ISdiDataRow yourself!

- For "GlobalExits": Use "getDataRowBuilder()" or "getDataRowPrototypeFactory()" from "SdiGlobalResultTransformationFunctionParameter" to create new rows. You get the currently viewed row with "getDataRow()".

- For "InterperetedJavaService2": Use "ISdiDataRowFactory" from "SdiAddResultTransformationCallbacksParam".

29 GlobalExits

29.1 Introduction

You can use the global exits to customize programs without reference to the service type. Using global exits, you can redirect a service to another service, you can manipulate the service parameters of a service and/or change the service result.

29.2 Availability

The GlobalExits are available as of SP8.

29.3 Exits

Via exits, the system provides entry points to enable changes to the defined behavior by programming.

29.3.1 Available user exits

User exits provide entry points that are not modified by releases. In case of releases, the interfaces are respected and preserved by MPDV in a backwards compatible manner.

29.3.1.1 sdiGlobalModifyRequest

Using this exit, you can redirect a service to another service and/or change the service parameters. For example, you might want to redirect a standard service to a custom service without having to change the client.

29.3.1.2 sdiGlobalAddResultTransformationCallbacks

Using this exit, you can register callbacks to influence the result processing.

The system requests the callback "ISdiGlobalResultTransformationFunction" for each data record. After the last data record, the system again requests the callback and uses a dummy data record. Because the system uses a dummy data record, you can add new data records even if the service would otherwise not supply any data records.

29.3.1.3 sdiGlobalCleanup

Using this exit, you can close resources like DB connections or files if you have opened the resources in a previous exit. The system always calls this exit, also in case of an error during service execution.

29.3.2 Specifications for the implementation class

Package name: You must include the class in a package that consists of the domain name (in lower case letters). Further subpackages are **not** allowed.

Example: The service name is "MDUserAccountRules.list", consequently the package is called "mduseraccountrules".

Class name: The class must have a name of the following structure: domain name in lower case letters, where the first letter is capitalized, the name of the service function follows and is written in lower case letters, where the first letter is again capitalized.

Example: The service name is "MDUserAccountRules.list", consequently the class is called "MduseraccountrulesList".



For custom class names, the following definition applies:

Custom names include a "_" (see naming conventions):

Remove the underscore and capitalize the first letter after this underscore.

Example: U_CUST_Units_sample.list => UCustUnitsSampleList

Implemented interfaces / methods: no specifications

Other: The class must have a default constructor without parameters.

Compilation: For the compilation, the Jar files *MpdvDomCoreSdiCompileLib.jar* and *MpdvDomCoreUserExitCompileLib.jar* must be included in the class path.

Deployment: File the class file of the exit in the userexit directory including package directory structure.

The user exit directory is

jdir/MOC/<InstanceNo>/userexit/<scope>

or

jhydradir/MOC/<Man InstanceNo dant>/userexit/<scope>.

Example: Exit sdiGlobalCleanup for service "MDUserAccountRules.list":

```
package mduseraccountrules;

import de.mpdv.customization.userExit.IUserExitParam;

/**
 * Sample user exit
 *
 */
public class MduseraccountrulesList
{
    public void sdiGlobalCleanup(final IUserExitParam param)
```

```
{
    // TODO implementation
}
```

Directory structure on the server (Instance 1, Scope custom):

`jdir/MOC/1/userexit/custom/mduseraccountrules/MduseraccountrulesList.class`

29.3.3 Interfaces

29.3.3.1 Class: GlobalUeContext

This context class provides data that is globally useful in the context of exits.

Field	Description
hydraNow	The property "hydraNow" is a time stamp created at the beginning of web service processing and, as a result, can be used as reference time stamp for the current web service call.
userId	Includes the user logged on to the client.

29.3.3.2 Exit sdiGlobalModifyRequest

Parameter key in IUserExitParam:

Key name	Type	Description
param	SdiGlobalModifyRequestParam	Parameter structure of the exit
context	InterpretedJavaServiceUeContext	Context structure for all exits of the interpreted Java Services.
factory	ISystemUtilFactory	Utility class to access system utilities, such as logger or DB connection in exits.

Return key in IUserExitParam:

Key name	Type	Description
result	SdiGlobalModifyRequestResult	Result structure of the user exit

29.3.3.3 Class SdiGlobalModifyRequestParam

Field	Type	Description
-------	------	-------------

columnConfigurator	ColumnConfigurator	Includes the columns requested by the client.
specialParameters	Map<String, SpecialParam>	Assigns acronym => SpecialParameter for all web service parameters of the type "is SpecialParameter"
filterParametersRootExpression	SqlFilterComplexExpression	Root nodes of the filter parameter in a tree structure
serviceId	String	The service ID of the currently executed service
resultBuilder	ISdiGlobalModifyRequestResultBuilder	Builder to create exit results

29.3.3.4 Interface ISdiGlobalModifyRequestResultBuilder

All method requests are optional, except build(). You may only request the replace...() methods including objects that have actually been changed.

Method	Description
replaceColumnConfigurator(modifiedColumnConfigurator: ColumnConfigurator): ISdiGlobalModifyRequestResultBuilder	Overwrites the column configuration.
replaceSpecialParameters(Map<String, SpecialParam>: modifiedSpecialParameters): ISdiGlobalModifyRequestResultBuilder	Overwrites special parameters.
replaceFilterParametersRootExpression(SqlFilterComplexExpression: modifiedFilterParametersRootExpression): ISdiGlobalModifyRequestResultBuilder	Overwrites filter parameters.
replaceServiceId(String: modifiedServiceId): ISdiGlobalModifyRequestResultBuilder	Overwrites the service ID and thus, turns the service.
build():ISdiGlobalModifyRequestResult	Generates the return structure of the exit.

29.3.3.5 Interface ISdiGlobalModifyRequestResult

You must **not** implement this interface. You must generate the concrete instance using the builder of the field "resultBuilder" of the parameter object.

Method	Description
getColumnConfigurator():ColumnConfigurator	The requested column after the exit processing
getSpecialParameters():Map<String, SpecialParam>	The special paramter after exit requests
getFilterParametersRootExpression():SqlFilterComplexExpression	The root node of the filter parameter in the tree structure after being processed in the exit
getServiceId():String	The service ID (also called function ID or service name) of the service after processing in the exit. You can use this method to redirect a service to another service.

29.3.3.6 Exit sdiGlobalAddResultTransformationCallbacks

Parameter key in IUserExitParam:

Key name	Type	Description
param	SdiGlobalAddResultTransformationCallbacksParam	Parameter structure of the exit
context	InterpretedJavaServiceUeContext	Context structure for all exits of the interpreted Java Services
factory	ISystemUtilFactory	Utility class to access system utilities, such as logger or DB connection in exits.

Return key in IUserExitParam:

Key name	Type	Description
result	SdiGlobalAddResultTransformationCallbacksResult	Result structure of the

		user exit
--	--	-----------

29.3.3.7 Class

SdiGlobalAddResultTransformationCallbacksParam

Field	Type	Description
columnMap	Map<String, String>	Includes the assignment of acronym => table column (including alias).
specialParameters	Map<String, SpecialParam>	Assigns acronym => SpecialParameter for all web service parameters of the type "is SpecialParameter"

29.3.3.8 Class

SdiGlobalAddResultTransformationCallbacksResult

Field	Type	Description
callbackList	List<ISdiGlobalResultTransformationFunction>	Includes the list of callbacks that the system calls per result row. The list must never be NULL.

29.3.3.9 Interface

ISdiGlobalResultTransformationFunction

Callback function for the result manipulation.

Method	Description
transform (SdiGlobalResultTransformationFunctionParameter	The system calls this method for each result row and permits to change, delete or add rows. The

functionParameter):ISdiDataRowStream	result row is included in the SdiGlobalResultTransformationFunctionParameter. Note: You must create each row of type ISdiDataRow in ISdiDataRowStream using a factory from SdiGlobalResultTransformationFunctionParameter. You must NOT implement the interface ISdiDataRow. You will find an example further down.
--------------------------------------	---

29.3.3.10 Class

SdiGlobalResultTransformationFunctionParameter

Field	Type	Description
dataRowPrototypeFactory	ISdiDataRowPrototypeFactory	Factory method to create an empty data row using the service configuration. You can only create data rows and no rows AdditionalInfoOnly. If you want to include only AdditionalInfo in a row, you must use the factory "ISdiDataRowBuilder" of the method "getDataRowBuilder()".
dataRow	ISdiDataRow	Current result row. Depending on the row type (DataRowType), not all methods are possible. This should absolutely be checked before processing.
getDataRowBuilder()	ISdiDataRowBuilder	Factory method to create a builder. The user can create AdditionalInfoOnly data rows with

		this builder. Note: You require a new ISdiDataRowBuilder for each row. You cannot use the ISdiDataRowBuilder a second time.
--	--	--

29.3.3.11 Enum DataRowType

Type or result data row.

Value	Description
DATA_RECORD	Normal data row
ADDITIONAL_INFO_ONLY	This row includes only AdditionalInfo.
AFTER_LAST_ROW_DUMMY_RECORD	Dummy row that identifies the end of the result rows. All methods except the DataRowType query throw an exception.

29.3.3.12 Exit sdiGlobalCleanup

Parameter key in IUserExitParam:

Key name	Type	Description
param	SdiGlobalCleanupParam	Parameter structure of the exit
context	InterpretedJavaServiceUeContext	Context structure for all exits of the interpreted Java Services.
factory	ISystemUtilFactory	Utility class to access system utilities, such as logger or DB connection in exits.

No result

29.3.3.13 Class SdiGlobalCleanupParam

For future use. The parameter class is empty.

Field	Type	Description
-------	------	-------------

29.4 HowTos

29.4.1 Redirecting a service request to another service

Sample

scenario:

You want to redirect the service MDUnits.list to your custom service U_MDUnits.list.

You use the exit "sdiGlobalModifyRequest" for the service MDUnits.list (original service). If the class MdUnitsList already exists in your scope, only create the method "sdiGlobalModifyRequest" in the existing class. Otherwise, create the class MdunitsList in the package mdunits.

Details on how to create class and package names are included in the section "Specifications for the implementation class".

```
package mdunits;

import de.mpdv.customization.userexit.IUserExitParam;
import de.mpdv.sdi.data.ue.ISdiGlobalModifyRequestResult;
import de.mpdv.sdi.data.ue.SdiGlobalModifyRequestParam;

public class MdunitsList
{
    public void sdiGlobalModifyRequest(final IUserExitParam exitParam)
    {
        final SdiGlobalModifyRequestParam param = exitParam.get("param");
        final ISdiGlobalModifyRequestResult result = param.getResultBuilder()
            .replaceServiceId("U_MDUnits.list")
            .build();
        exitParam.set("result", result);
    }
}
```

Compile this class and store the compiled class including package directory "mdunits" in the userexit folder.

The user exit folder is located at: jdir/MOC/<InstanceNo>/userexit/<scope>

or here: <JHYDRADIR>/MOC/<InstanceNo>/userexit/<scope>.



The system does not request the global exits of the target service, in the example of U_MDUnits.list. Instead, the system requests the global exits of the original service, in the example MDUnits.list.

If you require a modification of the service result, insert the method sdiGlobalModifyRequest in the exit class of the original service (in the example MdunitsList).

If you want to close resources, insert the method sdiGlobalCleanup in the exit class of the original service (in the example MdunitsList).

29.4.2 Creating additional result rows

The following example shows how to insert additional rows at the end of a list service, for example, for totals rows.

You can find general information on creating user exits in the instructions [Java Userexit with IntelliJ IDEA.pdf](#) and [Java Userexit Privacy Hide Columns.pdf](#). The second documentation also contains another example of global user exits.

Here the service `BOPerson.list` is used. The service lists HR master data. In the example, we insert two additional records at the end, where we set the personnel number to 0 and output a text containing the number of listed persons instead of the name of a person.

Further details follow the source code of the user exit `BopersonList.class`.

```
package boperson;

import de.mpdv.customization.userExit.IUserExitParam;
import de.mpdv.sdi.data.ISdiDataRow;
import de.mpdv.sdi.data.ISdiDataRow.DataRowType;
import de.mpdv.sdi.data.SdiEagerDataRowStream;
import de.mpdv.sdi.data.ue.SdiGlobalAddResultTransformationCallbacksResult;
import de.mpdv.sdi.systemutility.ISdiLogger;
import de.mpdv.sdi.systemutility.ISdiLoggerProvider;
import de.mpdv.sdi.systemutility.ISystemUtilFactory;

import java.util.Calendar;
import java.util.Collections;
import java.util.GregorianCalendar;

public class BopersonList {public class BopersonList {
    private int persCount = 0;

    public void sdiGlobalAddResultTransformationCallbacks(IUserExitParam ueParam) {
        ISystemUtilFactory factory = ueParam.get("factory");
        ISdiLogger logger = factory.<ISdiLoggerProvider>fetchUtil("LoggerProvider").fetchLogger(this.getClass());

        SdiGlobalAddResultTransformationCallbacksResult result = new SdiGlobalAddResultTransformationCallbacksResult(
            Collections.singletonList(functionParameter -> {
                if (functionParameter.getDataRow().getDataRowType() == DataRowType.DATA_RECORD) {
                    this.persCount++;
                }

                if (functionParameter.getDataRow().getDataRowType() == DataRowType.AFTER_LAST_ROW_DUMMY_RECORD) {
                    logger.info("Creating dummy data rows");

                    // Create first additional row
                    ISdiDataRow sumRow1 = functionParameter.getDataRowPrototypeFactory()
                        .createSdiDataRow(" " /* Result set name from ServiceParameter.ResultSet */);
                    sumRow1.setDataTableId(" " /* Result set from DataObject.dataTabLabel */);
                    sumRow1.setCellValue(sumRow1.probeColumnIndex("person.name"), "Dummy row 1, Row count " + this.persCount);
                    sumRow1.setCellValue(sumRow1.probeColumnIndex("person.id"), 99999999);
                    sumRow1.setCellValue(sumRow1.probeColumnIndex("person.valid from"), new GregorianCalendar(1999, Calendar.JANUARY, 1));

                    // Create second additional row
                    ISdiDataRow sumRow2 = functionParameter.getDataRowPrototypeFactory()
                        .createSdiDataRow(" " /* Result set name from ServiceParameter.ResultSet */);
                    sumRow2.setDataTableId(" " /* Result set from DataObject.dataTabLabel */);
                    sumRow2.setCellValue(sumRow1.probeColumnIndex("person.name"), "Dummy row 2, Row count " + this.persCount);
                    sumRow2.setCellValue(sumRow1.probeColumnIndex("person.id"), 99999999);
                    sumRow2.setCellValue(sumRow1.probeColumnIndex("person.valid from"), new GregorianCalendar(1999, Calendar.JANUARY, 1));

                    logger.info("Adding dummy data rows to result");
                    return new SdiEagerDataRowStream(sumRow1, sumRow2);
                } else {
                    return new SdiEagerDataRowStream(functionParameter.getDataRow());
                }
            })
        );
        ueParam.set("result", result);
    }
}
```

Source code explanations

If standard processing of the service is completed, the user exit is called again with a dummy data record.

The dummy data record can be identified by RowType

`DataRowType.AFTER_LAST_ROW_DUMMY_RECORD`.

Additional data records can only be generated using a *ISdiDataRowBuilder* aus der *dataRowPrototypeFactory* . The method *createSdiDataRow(ResultSet)* generates a data record from the service configuration that contains all columns from the service parameters of a *ResultSet*. The *ResultSet* for which the service parameters are to be used is transferred to the method as a filter. The filter refers to the column "Result Set" of the configuration of the service parameters in the repository.

The newly generated data set is then assigned to a *ResultSet* of the service using the method *setDataTableId(dataTabLabel)*. This *ResultSet* usually refers to the column "Data Tab Label" of the configuration of the data object in the repository.



Please note that the *ResultSet* has a different reference for the methods *createSdiDataRow(ResultSet)* and *setDataTableId(dataTabLabel)*. Depending on the type configuration of the service, the *ResultSet*s in the *ServiceParameter* and *Dataobject* can be the same or different. Check the column in the Repository.

The following errors may occur if incorrect information is entered:

- No data record is generated.
- A data record with the incorrect columns is created.
- The data set is not output in the expected *ResultSet*.

If the data record is then generated and assigned to the *ResultSet*, the columns can be filled with values. In the example, the methods *ISdiDataRow.setCellValue(...)* and *ISdiDataRow.probeColumnIndex(...)* are used in combination.



Please bear in mind that the order of the columns in the data record may not be identical to the data records previously generated by standard processing. Always check the index of a column for the newly created records, rather than using an index that you have previously memorized in a variable in the result set when processing an ordinary record.

The newly created records are then output with the auxiliary class *SdiEagerDataRowStream*.

30 Simple External Services

Simple External Services (SES) are a user-friendly interface for the application developer to develop individual services. The SES are part of the Simple Developer Interface (SDI).

30.1 Development process

30.1.1 Repository

You must first create a new service in the repository. The following attributes must be filled:

- Domain
- Function
- Service Type = **ExternalJavaService**

The following attributes of a service parameter are relevant to Simple External Services:

- Acronym
- Web Service Type
- Is result
- Input as an array
- Is Special Parameter (must always be set to Y for input parameters)
- Is mandatory
- Can *
- Transfer Empty Values To Hydra

30.1.2 Specifications for the implementation class

Package name: The class must be included in a package which starts with `de.mpdv.simpleExternalService`. Further subpackages are allowed

Class name: The class should have a name that references the function ID, e.g. *DomainFunction*

Implemented interfaces / methods: The class must implement the interface *ISimpleExternalService* and thus the method

```
public SesResult execute(SesRequest request, SesContext context, ISystemUtilFactory factory)
```

Other: The class must have a default constructor without parameters

Compilation: The Jar file *MpdvDomCoreSdiCompileLib.jar* must be included in the class path for the compile process.

Deployment: The class file of the Simple External Services must be stored in the following directory including package directory structure:

<JHYDRADIR>/MOC/<instance>/externalService/<scope> or

`wsp_config/MOC/<instance>/externalService/<scope>`

Example of a service list for the domain SampleDomain:

```
package de.mpdv.simpleExternalService;

import de.mpdv.sdi.data.SesContext;
import de.mpdv.sdi.data.SesRequest;
import de.mpdv.sdi.data.SesResult;
import de.mpdv.sdi.simpleExternalService.ISimpleExternalService;
import de.mpdv.sdi.systemutility.ISystemUtilFactory;

/**
 * Implementation for simple external service SampleDomain.list
 *
 */
public class SampleDomainList implements ISimpleExternalService
{
    /**
     * Default constructor
     */
    public SampleDomainList()
    {
        // empty
    }

    public SesResult execute(SesRequest request, SesContext context,
ISystemUtilFactory factory)
    {
        // TODO Implementation of simple external service
        return null;
    }
}
```

Directory structure in the server (instance 1, scope custom):

`<JHYDRADIR>/MOC/1/externalService/custom/de/mpdv/simpleExternalService/SampleDomainList.class`

30.1.3 Create mapping

To use the Simple External Service, a mapping of the function ID to the class must exist.

To this end, you must create a file `FunctionID.txt` in the folder `wsp_config/MOC/<instance>/externalServiceMapping/<scope>` or

`<JHYDRADIR>/MOC/<instance>/externalServiceMapping/<scope>`. The file only includes one row that in turn includes the Fully Qualified Class Name (class name including package).

Example (Service list for domain SampleDomain with the class SampleDomainList in the package de.mpdv.simpleExternalService, instance 1, Scope custom):

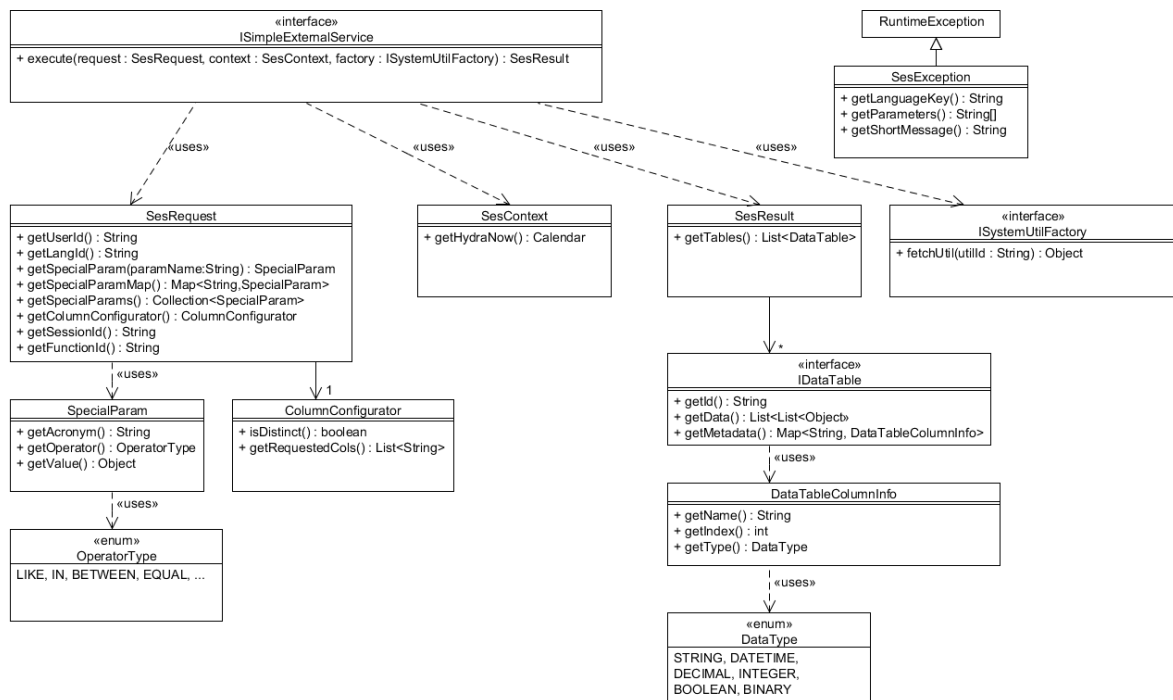
`wsp_config/MOC/1/externalServiceMapping/custom/SampleDomain.list.tx:` or

<JHYDRADIR>/MOC/1/externalServiceMapping/custom/SampleDomain.list.txt:

de.mpdv.simpleExternalService.SampleDomainList

30.2 Interface description

The following UML class diagram shows an overview of the relevant interface data types:



30.2.1 Interface ISimpleExternalService

This is the interface that must implement the class of a Simple External Service. The interface is included in the package *de.mpdv.sdi.simpleExternalService*.

Method	Description
execute	Method for executing the service Return SesResult - the result of the service Input SesRequest request - Client request.

	Includes, among others, SpecialParams, ColumnConfigurator SesContext context - Context object of the service. Includes additional information that might be relevant for the execution of the service. ISystemUtilFactory factory - Factory for System utils, such as logging and DB access
--	--

30.2.2 Interface ISystemUtilFactory

This is the interface that enables access to the system util functions. The interface is included in the package *de.mpdv.sdi.systemutility*.

The system utils are described in a separate section.

30.2.3 Data types

All classes described in the following are included in the package *de.mpdv.sdi.data*.

30.2.3.1 Class SesException

This class has been designed to cancel processing within a Simple External Service by an error (that can be located). It is a so-called unchecked exception (the higher-level type is RuntimeException), i.e. it does not have to be caught explicitly.

It has two constructors (with and without the optional parameter "cause")

Parameter	Type	Description
languageKey	String	Unique language key. It is transferred to the client and re-coded into the actual error message.
shortMessage	String	Brief version of the error message in English. It is logged in the server.
cause	Throwable	OPTIONAL The Exception that has led to

		throwing the SesException.
parameters	String... (0..n values possible)	Parameters for the error message, corresponding to the languageKey.

30.2.3.2 Class SesRequest

This is the data type that includes the request information. The data of this type cannot be changed.

Method	Description
getUserId	Provides the UserId of the user logged on to the client Return type: String
getLangId	Returns the language abbreviation of the client. Return type: String
getSpecialParam	Provides the special parameter for a transferred parameter name Return type: String Input: String paramName – the parameter name
getSpecialParamMap	Provides a map including all special parameters. The parameter name is the key. Return type: Map<String, SpecialParam>
getSpecialParams	Provides a collection including all special parameters. Return type: Collection<SpecialParam>
getColumnConfigurator	Provides the column configurator for the request. It informs on the columns that have been requested from the client.

	Return type: ColumnConfigurator
getFunctionId	Provides the Function ID of the requested service. Return type: String
getSessionId	Provides the session ID of the current client session. Return type: String

30.2.3.3 Class SpecialParam

This is the data type that includes information on a special parameter. The data of this type cannot be changed.

Method	Description
getAcronym	Provides the acronym of this special parameter Return type: String
getOperator	Provides the operator of this special parameter Return type: OperatorType (enum)
getValue	Sends the value of this special parameter Return type: Object The actual data type depends on the parameter type. These types are possible: - String - Integer - BigDecimal - Boolean - Byte[] - Calendar - String[] - Integer[] - BigDecimal[] - Boolean[] - Calendar[]

30.2.3.4 Class OperatorType

It is an **enum** representing different operators. The following values are possible:

Value	Description
LIKE	Like a specific value
EQUAL	Equal to a specific value
BETWEEN	Between two specific values
IN	Included in a quantity of specific values
NOT_EQUAL	Unequal to a specific value
LIKE_OR_NULL	Like a specific value or null
EQUAL_OR_NULL	Equal to a specific value or null
BETWEEN_OR_NULL	Between two specific values or null
IN_OR_NULL	Included in a quantity of specific values or null
NOT_EQUAL_OR_NULL	Unequal to a specific value or null
GT	Greater than a specific value
LT	Less than a specific value
GTE	Greater than or equal to a specific value
LTE	Less than or equal to a specific value
GT_OR_NULL	Greater than a specific value or null
LT_OR_NULL	Less than a specific value or null
GTE_OR_NULL	Greater than or equal to a specific value or null
LTE_OR_NULL	Less than or equal to a specific value or null

30.2.3.5 Class ColumnConfigurator

This is the data type that includes information on the column configurator of a request. The column configurator informs about the columns that have been requested by the client.

Method	Description
getRequestedCols	Provides a list of column names requested by the client. Return type: List<String>
isDistinct	Specifies whether or not double values may be included in the result DataTable of the service. Return type: Boolean

30.2.3.6 Class SesContext

This is the data type that includes additional context information. The data of this type cannot be changed.

Method	Description
getHydraNow	Provides the NOW time for the current service call (HYDRA Now) Return type: Calendar

30.2.3.7 Class SesResult

This is the data type that includes result information on a Simple External Service. The Builder class SesResultBuilder (see section "Class SesResultBuilder") is available for the generation of a SesResult object.

The constructor has the following parameters:

Parameter	Type	Description
tables	List<IDataTable>	List of all DataTables returned to the client.

30.2.3.8 Interface IDataTable

This is the data type that includes a data table. To create a DataTable object, the utility "DataTableBuilder" is available (see section "Interface ISystemUtilFactory").

The constructor has the following parameters:

Parameter	Type	Description
id	String	The unique ID of the data table
data	List<List<Object>>	The content of the data table. An indexed list of data rows. A data row, in turn, is an indexed list of column values. The actual data type of a column value depends on the definition of meta data (see below).

metadata	Map<String, DataTableColumnInfo>	The definition of columns. Column information is mapped including column names as the key.
----------	----------------------------------	--

30.2.3.9 Class DataTableColumnInfo

This is the data type that includes information on a column. DataTableColumnInfo objects are generated implicitly using the utility "DataTableBuilder" (see section "Interface ISystemUtilFactory").

The constructor has the following parameters:

Parameter	Type	Description
Name	String	Column name
index	int	Column index (matches the index within the list<Object>object of a row)
Type	DataType (enum)	The data type of the column content

30.2.3.10 Class DataType

It is an **enum** representing different data types. The following values are possible:

Value	Corresponding Java data type
STRING	java.lang.String
DATETIME	java.util.Calendar
DECIMAL	java.math.BigDecimal
INTEGER	java.lang.Integer
BOOLEAN	java.lang.Boolean
BINARY	byte[]

30.2.3.11 Class SdiException

You use this class to cancel the processing in SDI with an error (that can be located). It is a so-called unchecked exception (the higher-level type is RuntimeException), i.e. it does not have to be caught explicitly.

Description of the interface, see SesException.

30.2.4 Builder

30.2.4.1 Class DataTableBuilder



This class is outdated since SdiCompileLib version 1.12! Do not continue to use! Use instead the utility "DataTableBuilder" of type IDDataTableBuilder of the ISystemUtilFactory. In the class DataTableBuilder, the monitoring of memory usage is not included that should prevent OutOfMemoryExceptions!

Element	Description
Constructor	IMPORTANT: outdated! Use instead the utility DataTableBuilder of the ISystemUtilFactory. Generates the builder Result: DataTableBuilder Input:
Method setDataTableId	Sets the ID of the DataTable Result: DataTableBuilder Input: String dataTableId
Method addCol	Adds a column to the table (meta info). Result: DataTableBuilder Input: String colName – column name

	DataType columnType – column type (enum)
Method addRow	Adds a row to the table and sets the transferred column values. Result: DataTableBuilder Input: Object... values (0..n values) – The values to be set
Method addRow	Adds an (empty) row to the table. You can then set the column values one after the other using the method value. Result: DataTableBuilder Input:
Method value	Adds the value of the next column to the current table row. The number of defined columns specifies how often this method can be called for each row. Result: DataTableBuilder Input: Object value – The value to be set
Method build	Return of the ready-designed DatatTable object. Result: IDataTable Input:

30.2.4.2 Class SesResultBuilder

This is a builder providing support with the creation of a SesResult object. This class is included in the package *de.mpdv.sdi.data*.

Element	Description
Constructor	Generates the builder Result: SesResultBuilder Input:
Method addDataTable	Adds a DataTable. Result: SesResultBuilder Input: IDataTable dataTable
Method build	Return of the ready-designed SesResult object Result: SesResult Input:

30.2.4.3 Class SesExceptionBuilder

This is a builder that you can use to create a SesException with more than one language key (more than one message). This class is included in the package *de.mpdv.sdi.data*.

Element	Description
Constructor	Generates the builder Result:

	SesExceptionBuilder Input:
Method cause	Provides the exception that has led to throwing the SesException (optional). Result: SesExceptionBuilder Input: Throwable pCause
Method languageKey	Adds a new language key. Result: SesExceptionBuilder Input: String langKey
Method shortMessage	Provides the short description of the error for the current language key. Result: SesExceptionBuilder Input: String shortMessage Exception: IllegalStateException if no language key has been added yet.
Method parameters	Provides the parameters for the current language key. Result:

	SesExceptionHandler Input: String... parameters Exception: IllegalStateException if no language key has been added yet.
Method build	Creates the Exception Result: SesException Input: - Exception: IllegalStateException if not a single language key has been added.

30.2.4.4 Class SdiExceptionHandler

This is a builder that you can use to create a SdiException with more than one language key (more than one message). This class is included in the package *de.mpdv.sdi.data*.

Interface `sie SesExceptionHandler` (return is `SdiExceptionHandler / SdiException` instead of `SesExceptionHandler / SesException`)

30.3 Best Practices

30.3.1 Exception Handling in Simple External Services

Relevant areas in Simple External Services should be surrounded by try Catch.

In this context, it is important not to execute a "Catch all", i.e. NOT

```
catch (final IOException e)
```

OR NOT

catch (final Throwable t)

but only to catch the exceptions that may actually occur within try/catch (so-called Checked Exceptions that Eclipse shows for each error even if they are not caught).



This is important to make sure that exceptions from the system's basis provided with a specific language key are actually sent to the client and are not caught in the Simple External Service and replaced by another message.

In general, language keys should always be used for exceptions in order for the client to show localized error messages.

You must store the texts for the language key in the client and the client then shows the text instead of the language key. You can implement that the text of the language key is identified with respect to the language used.

Furthermore, it is important to forward the exception that has been caught.

Example of a general error:

```
throw new SesException("lkErrorAtExternalServiceExecution",  
    "Error at execution of simple external Service.",  
    e);
```

30.3.2 Creating a DataTable

To create a DataTable as the result of the Simple External Service, you can use the utility "DataTableBuilder" of the ISystemUtilFactory of type IDDataTableBuilder. Using the builder, you first set the meta information on the columns.

Example:

```
IDDataTableBuilder dataTableBuilder = factory.fetchUtil("DataTableBuilder");  
dataTableBuilder.setDataTableId("returnTable1");  
dataTableBuilder.addCol("retColumnString2", DataType.STRING);  
dataTableBuilder.addCol("retColumnInt1", DataType.INTEGER);
```

Then the data is added. There are two options.

Example of option 1:

```
dataTableBuilder.addRow();  
dataTableBuilder.value("myString1");  
dataTableBuilder.value(Integer.valueOf(12));  
dataTableBuilder.addRow();  
dataTableBuilder.value("myString2");
```

```
dataTableBuilder.value(Integer.valueOf(22));
```

This option is best if the number of columns is dynamic, i.e. depends on the ColumnConfigurator.

Example of option 2:

```
dataTableBuilder.addRow("myString1", Integer.valueOf(12));  
dataTableBuilder.addRow("myString2", Integer.valueOf(22));
```

This option is best if the number of columns is fixed.

31 ExternalUtils

31.1 Introduction

This document describes the ExternalUtils. You can use the ExternalUtils to store help functions or libraries for all scopes in user exits and ExternalServices.

31.2 Availability

ExternalUtils are available from Service Pack7.

31.3 ExternalUtils

There are two directories where you can store utilities and libraries for all scopes. These two directories are located in the instance directory of the WSP configuration (jdir/MOC/<InstanceNo> or JHYDRADIR/MOC/<InstanceNo>).

31.3.1 Directory ext_util_jar

This directory must only include *.jar files in subfolders. Path: "jdir/MOC/<InstanceNo>/ext_util_jar" bzw. „JHYDRADIR/MOC/<InstanceNo>/ext_util_jar“.

JAR files must be stored in at least one subfolder with the company initials because otherwise MPDV could overwrite the libraries of customers or vice versa. To prevent overwriting within the company, you should use additional subfolders.

Using the directory "ext_util_jar", you can integrate libraries in ExternalJavaServices and user exits that are accessible from all scopes. You can also integrate external libraries. The JAR files are only read once upon the Tomcat start. New files are not recognized later on. Changes to existing files are reloaded when the respective entry point is reloaded (ExternalService or user exit).

Example: A library MyLib.jar in ext_util_jar is used in an ExternalService "MySample.list". The respective class is MySampleList.class. The class MySampleList.class provides the entry point for the request. The class MySampleList would still be the entry point if the library MyLib.jar were used in another class MyHelper.class if MyHelper.class is requested in MySampleList.class.

Class	MySampleList.class	in
jdir/MOC/<InstanceNo>/externalService/<Scope>/de/mpdv/MySampleList.class		or
JHYDRADIR/MOC/<InstanceNo>/externalService/<Scope>/de/mpdv/MySampleList.class		

is reloaded because the time stamp of the file has changed and the development mode is enabled. Then also the library MyLib.jar is reloaded.

The same is true for the user exits.

This directory must not include *.class files!

31.3.2 Directory ext_util_classes

This directory must only include *.class files and configuration files in a JAVA package structure. Path: jdir/MOC/<InstanceNo>/ext_util_classes"or „JHYDRADIR/MOC/<InstanceNo>/ext_util_classes“ The package structure is important, otherwise MPDV might overwrite customers' libraries or a customer might overwrite MPDV's.

You require a basis package that is based on the internet address, but without "www". (Example: Internet address www.mpdv.de => basis package: de.mpdv). Under this basis package, you must at least create one further package with the domain name from the repository, completely in lower case letters.

Under this package, you can create any number of additional packages. You use these packages to keep the *.class files of the own company from being overwritten by accident.

The *.class files are always reloaded when the entry points are reloaded. The changed, the new and the missing *.class files are then considered at runtime. This behavior is only active if the development mode is enabled. Otherwise the *.class files are only read once when the entry points are first requested. For examples and description of the entry points, refer to the "directory ext_util_jar".

This directory must not include *.jar files!

32 MDS-WebUtil

32.1 Purpose

The library WebUtil provides helper functions that you can use to perform http and https calls with little code.

32.2 Requirements

SP16

32.3 Code example

```
String result = new CommonHttpClient()  
    withRequestSettings(urlcon-> urlcon.setRequestMethod("GET"))  
    communicate("https://api.predic8.de/shop/products/", null);  
System.out.println(result);
```

This example calls the URL <https://api.predic8.de/shop/products/> using GET and outputs the result to the command line.

32.4 API reference



You cannot use classes and functions that are not documented when you use the MDS license.

32.4.1 CommonHttpClient

The class CommonHttpClient is immutable. You can create an instance and then, if necessary, overwrite the default values by means of the withXX() methods.

You call the URL by using one of the communicateXX() methods.

32.4.1.1 CommonHttpClient(): CommonHttpClient

Creates an HTTP client that uses system defaults. This client can be used with HTTP or HTTPS. For HTTPS the defaults allow a connection without a truststore. Optionally you can set a truststore by using one of the withTrustStoreFileXX() methods.

32.4.1.2 withRequestSettings(ICheckedConsumer<HttpURLConnection, IOException> settingSetter): CommonHttpClient

Allows to set parameters on the HttpURLConnection like the HTTP verb (GET, POST, DELETE, etc) or other header values like 'Content-Type'

Input:

settingSetter: callback for setting configuration values on the HttpURLConnection

Return:

new instance with applied settings

Sample:

```
new CommonHttpClient()
    .withRequestSettings(urlCon -> {
        urlCon.setRequestMethod("POST");
        urlCon.setRequestProperty("Content-Type", "application/json");
    });
```

32.4.1.3 withBasicAuthData(String user, String password): CommonHttpClient

Allows to specify basic authentication credentials

Input:

user: the user

password: the password

Return:

new instance with applied settings

Sample:

```
new CommonHttpClient()
    .withBasicAuthData("MyUser", "MyPassword");
```

32.4.1.4 withCookieSupport(CookieManager cookieManager): CommonHttpClient

Adds cookie support to the request. Captures cookies and sets them on following requests

Input:

cookieManager: cookie manager

Return:

new instance with applied settings

Sample:

```
new CommonHttpClient()
    .withCookieSupport(cookieManager);
```

32.4.1.5 withTruststoreFile(String truststoreFilePath, String truststorePassword): CommonHttpClient

This method allows to specify a trust store and a truststore password.

Input:

truststoreFilePath: local absolute file path to truststore

truststorePassword: password of truststore

Return:

new instance with applied settings

32.4.1.6 withTruststoreFileStreamProvider(ICheckedSupplier<InputStream, IOException> truststoreFileInputStreamSupplier, String truststorePassword): CommonHttpClient

Allows to specify a truststore and a truststore password. The ICheckedSupplier allows, for example, the usage of system paths.

Input:

truststoreFileInputStreamSupplier: supplier of an InputStream to the truststore content

truststorePassword: password of truststore

Return:

new instance with applied settings

32.4.1.7 withCheckHostNames(): CommonHttpClient

Allows to enable the check of the host name of a certificate against the called target. Default is OFF.

Return:

new instance with applied settings

32.4.1.8 withSslSocketFactory(SSLSocketFactory sslSocketFactory): CommonHttpClient

Allows to use an externally created SSL factory with settings like truststore.

Input:

sslSocketFactory: externally created SSL socket factory

Return:

new instance with applied settings

32.4.1.9 withOverwrittenSecureSocketProtocol(String secureSocketProtocol): CommonHttpClient

This method allows to overwrite the secure socket protocol. The default used is "TLS".

Input:

secureSocketProtocol: the secure socket protocol to be used

Return:

new instance with applied settings

32.4.1.10 withOverwrittenTruststoreType(String truststoreType): CommonHttpClient

This method allows to overwrite the truststore type. The default is `KeyStore.getDefaultType()`.

Input:

truststoreType: the truststore type to be used

Return:

new instance with applied settings

32.4.1.11 withOverwrittenTruststoreAlgorithm(String truststoreAlgorithm): CommonHttpClient

This method allows to overwrite the truststore algorithm. The default is `TrustManagerFactory.getDefaultAlgorithm()`.

Input:

truststoreAlgorithm: the truststore algorithm to be used

Return:

new instance with applied settings

32.4.1.12 communicate(String urlString, String payload):

CommonHttpClient

Executes a HTTP / HTTPS request.

Input:

urlString: URL to communicate with; can be an HTTP or HTTPS URL

payload: payload or NULL: In case of GET there is no payload allowed

Return:

result from URL

Throws:

IOException: if any IO error occurred

HttpException: if the server responds with HTTP code >= 400

Sample:

```
new CommonHttpClient()  
    .withRequestSettings(urlcon-> urlcon.setRequestMethod("GET"))  
    .communicate("https://api.predic8.de/shop/products/", null);
```

32.4.1.13 T communicate(String urlString, String payload, ICheckedFunction<HttpURLConnection, T, IOException> resultTransformer)

Executes a HTTP / HTTPS request.

Input:

urlString: URL to communicate with; can be an HTTP or HTTPS URL

payload: payload or NULL: In case of GET there is no payload allowed

resultTransformer: callback to extract and transform result from HttpURLConnection. This callback can also access and extract result headers.

Return:

result created by resultTransformer

Throws:

IOException: if any IO error occurred

32.4.1.14 void communicateAsStream(String urlString, CheckedConsumer<OutputStream, IOException> requestProcessor, CheckedConsumer<HttpURLConnection, IOException> responseProcessor)

Executes a HTTP / HTTPS request and allows streaming of data.

Input:

urlString: URL to communicate with; can be an HTTP or HTTPS URL

requestProcessor: optional callback to provide payload in streaming manner, can be NULL

responseProcessor: mandatory callback to provide result processing in streaming manner

Throws:

IOException: if any IO error occurred

32.4.2 HttpException

This exception transports the response code from a http request and the error message.

32.4.2.1 getResponseCode(): int

Gets the http response code

Return:

http response code

32.4.2.2 getMessage(): String

Returns the http error result

Return:

http error result

33 JAVA REST Client: Instruction

33.1 Purpose

Use the following instruction if you want to call a web service on a third-party system via Web Service Provider (WSP) from a JAVA program part using http or https.

Here, the WSP acts as client.

33.2 Requirements

The required libraries are available as of service pack 16.



You must ensure that the following files are included in the class path:

MpdvCommonWebUtil.jar, *MpdvDomCoreSdiCompileLib.jar* and
MpdvDomCoreUserExitCompileLib.jar.



This tutorial uses help functions from the library *MpdvCommonWebUtil.jar*. You need not use this library to call REST services. But it is much easier to call REST services with this library than with JAVA.



To create a trust store, you require OpenSSL (<https://www.openssl.org/>) and the JAVA tool "keytool" from a JAVA JDK. For basic tasks, these two programs are not required. They are only required for the extension "version with use of a trust store".

33.3 Task

Use the MES Development Suite to call the REST service "https://api.predic8.de/shop/products/" from the user exit sdiGlobalModifyRequest of the service MDUnit.list. This user exit stores the JSON result of the REST call as file in a system path. The user exit is called before the actual service processing is started.

33.4 Activate the development mode

Edit the file "<InstallDir>\jdir\MOC\1\config.properties" and insert the line "development.mode = 1" below "configreload.timeout=...".

```
#####
#####INSTANCE CONFIGURATION (1)#####
#####
# Please make sure that after configuration values is no TAB or SPACE

# Config reload timeout in seconds
# After this timeout it is checked, if the instance configuration (this file) has changed and the config must be reloaded
configreload.timeout=180

development.mode=1
```

Restart the WSP after the configuration file was changed.

Among other things, the option "development.mode = 1" ensures that changes to user exits take effect the next time the relevant service is called, without you having to restart the WSP each time.

33.5 Create system paths

The tutorial requires 2 system paths. The first path is used to store the JSON result of the REST call. The second path is used to store a JAVA trust store. The JAVA trust store includes the server certificate of the URL called.

1. Create the following folders on the server.
 - a. <InstallDir>\<SystemNo>\custom\rest_out
 - b. <InstallDir>\<SystemNo>\custom\truststore
1. Start the application *Paths* on the client.
2. Create a new system path "RESTOUT".
 - a. Protocol: "file"
 - b. Host: "localhost"
 - c. Port: 0
 - d. URL path: "<InstallDir>\<SystemNo>\custom\rest_out"
 - e. Description: "Storage for REST JSON files"
3. Create a new system path "TRUSTSTR".
 - a. Protocol: "file"
 - b. Host: "localhost"
 - c. Port: 0
 - d. URL path: "<InstallDir>\<SystemNo>\custom\truststore"
 - e. Description: "Storage for REST trust store"

33.6 Create user exit class

Create a class "MdunitsList" in the package "mdunits". Always bear in mind that the names are case sensitive. In the class "MdunitsList", create the method "sdiGlobalModifyRequest".

The result is the following:

```
package mdunits;

import de.mpdv.customization.userExit.IUserExitParam;

public class MdunitsList
{
    public void sdiGlobalModifyRequest(IUserExitParam ueParam)
    {
    }
}
```

33.7 Simple version

In the simple version, the result of an external REST service is output to Stdout.

Workflow:

- When the service "MDUnits.list" is called, the system calls the user exit "sdiGlobalModifyRequest".
- In the user exit "sdiGlobalModifyRequest", we call the REST service with the URL <https://api.predic8.de/shop/products/> using the http method GET.
- The result of the REST service is output to Stdout.

33.7.1 Extend user exit by REST call

Extend the user exit so that the following code is generated:

```
package mdunits;

import de.mpdv.commonwebutil.CommonHttpClient;
import de.mpdv.customization.userExit.IUserExitParam;
import de.mpdv.sdi.data.SesException;

import java.io.IOException;

public class MdunitsList
{
    public void sdiGlobalModifyRequest(final IUserExitParam ueParam)
    {
        try
        {
            final String result = new CommonHttpClient()
                .withRequestSettings(urlcon-> urlcon.setRequestMethod("GET"))
                .communicate("https://api.predic8.de/shop/products/", null);
            System.out.println("REST output: " + result);
        } catch (final IOException exc)
        {
            final String msg = "IO-Error executing REST service";
            throw new SesException("lkInvalidState", msg, exc, msg);
        }
    }
}
```

This code uses the class CommonHttpClient from the library MpdvCommonWebUtil.jar to call the URL <https://api.predic8.de/shop/products/> using GET. The JSON result is then output by system.out.println() to Stdout. All outputs to Stdout are stored in the file <WSP directory>\log\bootlog.txt.

If an error occurs, when the REST service is called, an error message is displayed on the client with the text: "IO-Error executing REST service".



You can only use System.out.println() during development. You must replace it by Logging or remove it at the latest before productive use.

33.7.2 Compile user exit

Compile the class MdunitsList. Be careful to use the correct class path (see section 33.2 "Requirements" above).

33.7.3 Test by calling the service MDUnits.list

1. Copy the compiled class including package on the server to
<InstallDir>\jdir\MOC\1\userexit\custom. If the custom directory does not exist, create this directory.
2. Check if the following file is available after the copy process:
<InstallDir>\jdir\MOC\1\userexit\custom\mdunits\MdunitsList.class
3. Start the application *Units* on the client.
4. Request data.
5. In the log file <InstallDir>\mip<n>\<WSP directory>\log\bootlog.txt (e.g. d:\mip1\WSP1\log\bootlog.txt), you will now find the response of the REST service.

33.8 Version with storage of REST service response in JSON file

This version is based on the preceding version. The user exit additionally stores the JSON response of the REST service as file in a system path. The file gets a unique name with the current time stamp and a 5-digit random number.

33.8.1 Extend the user exit by file storage

```
package mdunits;

import de.mpdv.commonwebutil.CommonHttpClient;
import de.mpdv.customization.userexit.IUserExitParam;
import de.mpdv.sdi.data.SesException;
import de.mpdv.sdi.data.ue.GlobalUeContext;
import de.mpdv.sdi.systemutility.IPathManagementOpenFileForWritingAction;
import de.mpdv.sdi.systemutility.ISystemUtilFactory;

import java.io.IOException;
import java.io.OutputStream;
import java.nio.charset.StandardCharsets;
import java.util.Arrays;
import java.util.Calendar;
import java.util.concurrent.ThreadLocalRandom;

public class MdunitsList
{
    public void sdiGlobalModifyRequest(final IUserExitParam ueParam)
    {
        final ISystemUtilFactory factory = ueParam.get("factory");
        final GlobalUeContext context = ueParam.get("context");
        try
        {
            final String result = new CommonHttpClient()
                .withRequestSettings(urlcon-> urlcon.setRequestMethod("GET"))
                .communicate("https://api.predic8.de/shop/products/", null);
            final String fileName = createFileName(context.getHydraNow());
            writeFile(factory, result, fileName);
        } catch (final IOException exc)
        {
            final String msg = "IO-Error executing REST service";
            throw new SesException("lkInvalidState", msg, exc, msg);
        }
    }

    private static void writeFile(final ISystemUtilFactory factory,
        final String json,
        final String fileName)
    {
        final IPathManagementOpenFileForWritingAction writeAction = factory.fetchUtil("PathManagementOpenFileForWritingAction");
        try (final OutputStream out = writeAction.writeFile("RESTOUT", fileName))
        {
            final byte[] data = json.getBytes(StandardCharsets.UTF_8);
            out.write(data);
            out.flush();
        } catch (final RuntimeException | IOException exc)
        {
            final String msg = "IO-Error persisting JSON";
            throw new SesException("lkInvalidState", msg, exc, msg);
        }
    }
}
```

```

    }

    private static String createFileName(final Calendar now)
    {
        return toTimestampString(now) + fetchFiveDigitRandomNumber() + ".json";
    }

    private static String toTimestampString(final Calendar cal)
    {
        if (cal == null)
        {
            return "null";
        }
        return pad(4, Integer.toString(cal.get(Calendar.YEAR))) +
            pad(2, Integer.toString(cal.get(Calendar.MONTH) + 1)) +
            pad(2, Integer.toString(cal.get(Calendar.DAY_OF_MONTH))) +
            pad(2, Integer.toString(cal.get(Calendar.HOUR_OF_DAY))) +
            pad(2, Integer.toString(cal.get(Calendar.MINUTE))) +
            pad(2, Integer.toString(cal.get(Calendar.SECOND))) +
            pad(3, Integer.toString(cal.get(Calendar.MILLISECOND)));
    }

    private static String fetchFiveDigitRandomNumber()
    {
        return pad(5, String.valueOf(ThreadLocalRandom.current().nextInt(100000)));
    }

    private static String pad(final int length,
        final String str)
    {
        if (str.length() == length)
        {
            return str;
        }
        final char[] paddedChars = new char[length];
        Arrays.fill(paddedChars, '0');
        final char[] strChars = str.toCharArray();
        System.arraycopy(strChars, 0, paddedChars, length - str.length(), strChars.length);
        return new String(paddedChars);
    }
}

```

The method `createFileName()` creates a file name based on a time stamp. The file name then consists of a time stamp, a 5-digit random number and the extension ".json".

The method `writeFile()` uses the SDI function "PathManagementOpenFileForWritingAction" to save the JSON using the system path "RESTOUT". UTF-8 is used as code page for the JSON file.

33.8.2 Compile user exit

Compile the class `MdunitsList`. Be careful to use the correct class path (see section 33.2 "Requirements" above).

33.8.3 Test by calling the service MDUnits.list

- Copy the compiled class including package on the server to
 <InstallDir>\jdir\MOC\1\userexit\custom. If the custom directory does not exist, create this directory.
- Check if the following file is available after the copy process:
 <InstallDir>\jdir\MOC\1\userexit\custom\mdunits\MdunitsList.class
- Start the application *Units* on the client.
- Request data.
- The folder <InstallDir>\<SystemNo>\custom\rest_out on the server now includes a *.json file, which includes the content of the REST service response.
- Open the *.json file using any text editor.

33.9 Version with use of trust store

The preceding versions accepted any server without checking the server certificate. You will extend the user exit in this version so that the concrete certificate is checked. This requires a trust store. A trust store is a storage location for trusted server certificates. With the help of a trust store, a client allows the connection to trusted servers only.

33.9.1 Create the server certificate in PEM encoding



This step is not necessary if you already have a PEM encoded server certificate.

33.9.1.1 Side note: Encoding of certificates

Certificates are mainly encoded in two ways:

- PEM encoding = Base-64 encoded X.509
 - o X.509v3
 - o Text file
 - o Can be identified via prefix in text file: -----BEGIN CERTIFICATE-----
 - o Frequent extensions
 - .cer
 - .crt
 - .pem
- DER encoding
 - o Binary file
 - o Frequent extension
 - .der



Do not rely on the file extension!

Open the certificate using any text editor and check if the file includes "BEGIN CERTIFICATE".

If yes, then it is a PEM encoded certificate, otherwise not.

33.9.1.2 Option 1: Export of server certificate from web browser

1. Use a browser and call the URL <https://api.predic8.de/shop/products/>.
2. Save/export the server certificate including the complete certificate chain.
 - a. This step is browser-specific. Search the internet to find out how you can export server certificates in your browser.

- b. Important: Save the certificate as PEM encoding (also called Base-64 encoded X.509). Use the file name "server.cer".

33.9.1.3 Option 2: Convert from PFX file (PKCS #12)

You require the program keytool from a Java JDK for this step (see section 33.2 "Requirements" above).

1. Identify the alias.
 - a. "<JDK-InstallDir>\bin\keytool.exe" -list -keystore <Keyfile> -storepass <password for keyfile> -v
 - b. Copy the alias or note down the alias of the concrete certificate.
2. Export the certificate.
 - a. "<JDK-InstallDir>\bin\keytool.exe" -export -alias <alias from list command> -file server.cer -keystore server.pfx -storepass <password of keyfile>
3. The result is a PEM encoded file "server.cer".

33.9.1.4 Option 3: Conversion from DER encoded certificate

You require the program openssl from OpenSSL for this step (see section 33.2 "Requirements" above).

It is assumed that the DER encoded certificate file has the file name server.crt.

1. Check whether the file is really DER encoded by opening the file with a text editor. Search for "BEGIN CERTIFICATE".
 - a. If you do not find anything, go on with step 2.
 - b. If you find "BEGIN CERTIFICATE", rename the file in server.cer and go on with section 33.9.2 "Create the trust store".
2. Convert the certificate.
 - a. <OpenSSL-InstallDir>\openssl x509 -inform DER -in server.crt -out server.cer -outform PEM
3. The result is a PEM encoded file "server.cer".

33.9.2 Create the trust store

You require the program keytool from a Java JDK for this step (see section 33.2 "Requirements" above).

In this step, you create a trust store with the file name "truststore.jks" and the password "secret".

1. Create the trust store.
 - a. "<JDK-InstallDir>\bin\keytool.exe" -import -v -trustcacerts -alias my_alias -file server.cer -keystore truststore.jks -storepass secret -noprompt
2. The result is a trust store file "truststore.jks" with password "secret". The certificates are stored under the alias "my_alias" in the trust store.

33.9.3 Extend user exit by trust store

```
package mdunits;

import de.mpdv.commonwebutil.CommonHttpClient;
import de.mpdv.commonwebutil.ICheckedSupplier;
import de.mpdv.customization.userExit.IUserExitParam;
import de.mpdv.sdi.data.SesException;
import de.mpdv.sdi.data.ue.GlobalUeContext;
import de.mpdv.sdi.systemutility.IHydraPathReadFileAction;
import de.mpdv.sdi.systemutility.IPathManagementOpenFileForWritingAction;
import de.mpdv.sdi.systemutility.ISystemUtilFactory;

import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.nio.charset.StandardCharsets;
import java.util.Arrays;
import java.util.Calendar;
import java.util.concurrent.ThreadLocalRandom;

public class MdunitsList
{
    public void sdiGlobalModifyRequest(final IUserExitParam ueParam)
    {
        final ISystemUtilFactory factory = ueParam.get("factory");
        final GlobalUeContext context = ueParam.get("context");
        try
        {
            final String result = new CommonHttpClient()
                .withRequestSettings(urlcon -> urlcon.setRequestMethod("GET"))
                .withTruststoreFileStreamProvider(fetchTruststore(factory), "secret")
                .withCheckHostNames()
                .communicate("https://api.predic8.de/shop/products/", null);
            final String fileName = createFileName(context.getHydraNow());
            writeFile(factory, result, fileName);
        } catch (final IOException exc)
        {
            final String msg = "IO-Error executing REST service";
            throw new SesException("lkInvalidState", msg, exc, msg);
        }
    }

    private static ICheckedSupplier<InputStream, IOException> fetchTruststore(final ISystemUtilFactory factory)
    {
        return () -> factory.<IHydraPathReadFileAction>fetchUtil("HydraPathReadFileAction").openFile("TRUSTSTR",
"truststore.jks");
    }

    private static void writeFile(final ISystemUtilFactory factory,
        final String json,
        final String fileName)
    {
        final IPathManagementOpenFileForWritingAction writeAction = factory.fetchUtil("PathManagementOpenFileForWritingAction");
        try (final OutputStream out = writeAction.writeFile("RESTOUT", fileName))
        {
            final byte[] data = json.getBytes(StandardCharsets.UTF_8);
            out.write(data);
            out.flush();
        } catch (final RuntimeException | IOException exc)
        {
            final String msg = "IO-Error persisting JSON";
            throw new SesException("lkInvalidState", msg, exc, msg);
        }
    }

    private static String createFileName(final Calendar now)
    {
        return toTimestampString(now) + "_" + fetchFiveDigitRandomNumber() + ".json";
    }

    private static String toTimestampString(final Calendar cal)
    {
        if (cal == null)
        {
            return "null";
        }
        return pad(4, Integer.toString(cal.get(Calendar.YEAR))) +
            pad(2, Integer.toString(cal.get(Calendar.MONTH) + 1)) +
            pad(2, Integer.toString(cal.get(Calendar.DAY_OF_MONTH))) +
            pad(2, Integer.toString(cal.get(Calendar.HOUR_OF_DAY))) +
            pad(2, Integer.toString(cal.get(Calendar.MINUTE))) +
            pad(2, Integer.toString(cal.get(Calendar.SECOND))) +
            pad(3, Integer.toString(cal.get(Calendar.MILLISECOND)));
    }

    private static String fetchFiveDigitRandomNumber()
    {
        return pad(5, String.valueOf(ThreadLocalRandom.current().nextInt(100000)));
    }

    private static String pad(final int length,
        final String str)
    {
        if (str.length() == length)
        {
            return str;
        }
    }
}
```

```
}  
    final char[] paddedChars = new char[length];  
    Arrays.fill(paddedChars, '0');  
    final char[] strChars = str.toCharArray();  
    System.arraycopy(strChars, 0, paddedChars, length - str.length(), strChars.length);  
    return new String(paddedChars);  
}  
}
```

The method `withTruststoreFileStreamProvider()` configures the trust store and activates the check of the server certificates. The method `withCheckHostNames()` ensures that the server name matches the server name included in the certificate. Using the function `fetchTruststore()`, the trust store "truststore.jks" is loaded from the system path "TRUSTSTR".

33.9.4 Compile user exit

Compile the class `MdunitsList`. Be careful to use the correct class path (see section 33.2 "Requirements" above).

33.9.5 Test by calling the service MDUnits.list

1. Copy the trust store "truststore.jks" on the server to `<InstallDir>\<SystemNo>\custom\truststore`
2. Check if the following file is available after the copy process:
`<InstallDir>\<SystemNo>\custom\truststore\truststore.jks`
3. Copy the compiled class including package on the server to
`<InstallDir>\jdir\MOC\1\userexit\custom`. If the custom directory does not exist, create this directory.
4. Check if the following file is available after the copy process:
`<InstallDir>\jdir\MOC\1\userexit\custom\mdunits\MdunitsList.class`
5. Start the application *Units* on the client.
6. Request data.
7. The folder `<InstallDir>\<SystemNo>\custom\rest_out` on the server now includes a *.json file, which includes the content of the REST service response.
8. Open the *.json file using any text editor.

34 MDS-MleOutboundUtil

34.1 Purpose

The library MleOutboundUtil provides helper functions that you can use to process the data of the MLE outbound transactions.

34.2 Requirements

To use the library MleOutboundUtil, you require SP16.

34.3 Code example flat output data

This code example shows an ExternalJavaService that processes the MLE output data.

The service reads all segments of type "RESTOUTBOUND1LEVEL". Then the service uses the order and the operation data from the MLE output data to create orders and operations via WSP and REST call.

```
package de.mpdv.extSvc.mleoutboundprocessor;

import static de.mpdv.commonmleoutboundutil.MleOutboundUtil.*;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import com.fasterxml.jackson.databind.ObjectMapper;
import de.mpdv.commonmleoutboundutil.BeginOutboundTaParam;
import de.mpdv.commonmleoutboundutil.ControlRecordData;
import de.mpdv.commonmleoutboundutil.EndOutboundTaParam;
import de.mpdv.commonmleoutboundutil.MleFileData;
import de.mpdv.commonmleoutboundutil.MleOutboundException;
import de.mpdv.commonmleoutboundutil.ProcessingState;
import de.mpdv.commonmleoutboundutil.ProtocolFileData;
import de.mpdv.commonmleoutboundutil.RecordData;
import de.mpdv.commonwebutil.CommonHttpClient;
import de.mpdv.sdi.data.SdiException;
import de.mpdv.sdi.data.SesContext;
import de.mpdv.sdi.data.SesRequest;
import de.mpdv.sdi.data.SesResult;
import de.mpdv.sdi.simpleExternalService.ISimpleExternalService;
import de.mpdv.sdi.systemutility.IDbConnectionProvider;
import de.mpdv.sdi.systemutility.IPathManagementOpenFileForWritingAction;
import de.mpdv.sdi.systemutility.ISystemUtilFactory;

import java.io.IOException;
import java.io.OutputStream;
import java.net.CookieManager;
import java.nio.charset.StandardCharsets;
import java.sql.Connection;
import java.sql.SQLException;
import java.util.Arrays;
import java.util.Calendar;
import java.util.Collections;
import java.util.List;
import java.util.function.ToIntBiFunction;

public class MleOutboundProcessorService implements ISimpleExternalService
{
    @Override
    public SesResult execute(final SesRequest request,
        final SesContext context,
        final ISystemUtilFactory factory)
    {
        final String segmentName = "RESTOUTBOUND1LEVEL";

        final Calendar now = (Calendar) context.getHydraNow().clone();
        final IDbConnectionProvider dbConnectionProvider = factory.fetchUtil("DbConnectionProvider");
        try (final Connection con = dbConnectionProvider.fetchDbConnection())
        {
            ProcessingState state = ProcessingState.DONE;
            Throwable error = null;

            final String processIdentifier = createUniqueIdentifier();
            final List<RecordData> recordDataList = beginOutboundTa(new BeginOutboundTaParam(con, processIdentifier, now,
segmentName));
            if (recordDataList.isEmpty())
            {
                return null;
            }
        }
    }
}
```

```

    }
    try
    {
        callServices(recordDataList);
    } catch (final IOException exc)
    {
        error = exc;
        state = ProcessingState.ERROR;
        final String msg = "Error calling services";
        throw new SdiException("InvalidState", msg, exc, msg);
    } catch (final RuntimeException | Error exc)
    {
        error = exc;
        state = ProcessingState.ERROR;
        throw exc;
    } finally
    {
        final MleFileData protFileData = writeProtocolFile(writeFile(factory), "Protocol data", now);
        final MleFileData errFileData = writeErrorFile(writeFile(factory), "Error data", now);
        final MleFileData dataFileData = writeDataFile(writeFile(factory), "Data", now);
        endOutboundTa(new EndOutboundTaParam(con, processIdentifier,
            Collections.singletonList(
                new ControlRecordData(createUniqueIdentifier(), state)
                    .withProtocolFileDataList(Collections.singletonList(
                        new ProtocolFileData("Test", "TstPrg", protFileData.getFileName(), protFileData.getFileSize(),
                            errFileData.getFileName(), errFileData.getFileSize(), dataFileData.getFileName(),
                            dataFileData.getFileSize(), "myMessage", Integer.valueOf(42))
                    ))
            ),
            error, now, now, segmentName)
        );
    }
    return null;
} catch (final SQLException exc)
{
    final String msg = "Error handling DB connection";
    throw new SdiException("DbError2", msg, exc, msg);
} catch (final MleOutboundException exc)
{
    throw new SdiException(exc.getLanguageKey(), exc.getShortMessage(), exc, exc.getParameters());
}
}

private static void callServices(final List<RecordData> recordDataList) throws IOException
{
    final ObjectMapper mapper = new ObjectMapper();
    final CookieManager cookieManager = new CookieManager();
    final String baseUrl = "https://localhost:8080/data/";
    final String orderUrl = baseUrl + "BOOrder/insert";
    final String operationUrl = baseUrl + "BOOperation/insert";
    for (final RecordData recordData : recordDataList)
    {
        final String orderJson = "{" +
            unwrapJson(mapper.writeValueAsString(new Dummy(Arrays.asList(
                new Param("order.id", extract(recordData.getSapSdata(), 0, 40)),
                new Param("order.ordertype", extract(recordData.getSapSdata(), 40, 5)),
                new Param("order.article", extract(recordData.getSapSdata(), 45, 40))
            )))) +
            ", \"columns\" : []}";
        final String operationJson = "{" +
            unwrapJson(mapper.writeValueAsString(new Dummy(Arrays.asList(
                new Param("operation.id", extract(recordData.getSapSdata(), 0, 40) + extract(recordData.getSapSdata(), 85,
4)),
                new Param("operation.designation", extract(recordData.getSapSdata(), 89, 40)),
                new Param("operation.act.scheduled", extract(recordData.getSapSdata(), 129, 1)),
                new Param("operation.plan.workplace", extract(recordData.getSapSdata(), 130, 8)),
                new Param("operation.processing_code", extract(recordData.getSapSdata(), 138, 6))
            )))) +
            ", \"columns\" : []}";

        new CommonHttpClient()
            .withCookieSupport(cookieManager)
            .withBasicAuthData("12345", "mpdv")
            .withRequestSettings(urlCon -> {
                urlCon.setRequestMethod("POST");
                urlCon.setRequestProperty("Content-Type", "application/json");
            })
            .communicate(orderUrl, orderJson);
        new CommonHttpClient()
            .withCookieSupport(cookieManager)
            .withBasicAuthData("12345", "mpdv")
            .withRequestSettings(urlCon -> {
                urlCon.setRequestMethod("POST");
                urlCon.setRequestProperty("Content-Type", "application/json");
            })
            .communicate(operationUrl, operationJson);
    }
}

private static ToIntBiFunction<CharSequence, String> writeFile(final ISystemUtilFactory factory)
{
    return (content, fileName) -> {
        final IPathManagementOpenFileForWritingAction writeAction =
factory.fetchUtil("PathManagementOpenFileForWritingAction");
        try (final OutputStream out = writeAction.writeFile("MOCLOGS", fileName))
        {
            final byte[] data = content.toString().getBytes(StandardCharsets.UTF_8);
            out.write(data);
            out.flush();
            return data.length;
        }
    };
}

```



```

        } catch (final RuntimeException | IOException ignore)
        {
            return -1;
        }
    };
}

@JsonIgnoreProperties(ignoreUnknown = true)
static class Dummy
{
    private final List<Param> params;

    Dummy(final List<Param> params)
    {
        this.params = params;
    }

    public List<Param> getParams()
    {
        return this.params;
    }
}

@JsonIgnoreProperties(ignoreUnknown = true)
static class Param
{
    private final String acronym;
    private final String value;

    Param(final String acronym,
          final String value)
    {
        this.acronym = acronym;
        this.value = value;
    }

    public String getAcronym()
    {
        return this.acronym;
    }

    public String getValue()
    {
        return this.value;
    }
}
}

```

34.4 Code example hierarchical output data

This code example shows an ExternalJavaService that processes hierarchical MLE output data. The service reads all segments of type "RESTOUTBOUND1LEVELPARENT" and their sub segments. Then the service uses the order and the operation data from the MLE output data to create orders and operations via WSP and REST call.

```

package de.mpdv.extSvc.mleoutboundprocessor;

import static de.mpdv.commonmleoutboundutil.MleOutboundUtil.*;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import com.fasterxml.jackson.databind.ObjectMapper;
import de.mpdv.commonmleoutboundutil.BeginOutboundTaPacketParam;
import de.mpdv.commonmleoutboundutil.BeginOutboundTaParam;
import de.mpdv.commonmleoutboundutil.ControlRecordData;
import de.mpdv.commonmleoutboundutil.EndOutboundTaPacketParam;
import de.mpdv.commonmleoutboundutil.MleFileData;
import de.mpdv.commonmleoutboundutil.MleOutboundException;
import de.mpdv.commonmleoutboundutil.ProcessingState;
import de.mpdv.commonmleoutboundutil.ProtocolFileData;
import de.mpdv.commonmleoutboundutil.RecordData;
import de.mpdv.commonwebutil.CommonHttpClient;
import de.mpdv.sdi.data.SdiException;
import de.mpdv.sdi.data.SesContext;
import de.mpdv.sdi.data.SesRequest;
import de.mpdv.sdi.data.SesResult;
import de.mpdv.sdi.simpleExternalService.ISimpleExternalService;
import de.mpdv.sdi.systemutility.IDbConnectionProvider;
import de.mpdv.sdi.systemutility.IPathManagementOpenFileForWritingAction;
import de.mpdv.sdi.systemutility.ISystemUtilFactory;

import java.io.IOException;
import java.io.OutputStream;
import java.net.CookieManager;
import java.nio.charset.StandardCharsets;
import java.sql.Connection;
import java.sql.SQLException;
import java.util.ArrayList;

```

```

import java.util.Arrays;
import java.util.Calendar;
import java.util.Collections;
import java.util.List;
import java.util.function.ToIntBiFunction;
import java.util.stream.Collectors;
import java.util.stream.Stream;

public class MleOutboundHierarchicalProcessorService implements ISimpleExternalService
{
    @Override
    public SesResult execute(final SesRequest request,
        final SesContext context,
        final ISystemUtilFactory factory)
    {
        final String segmentName = "RESTOUTBOUND1LEVELPARENT";

        final Calendar now = (Calendar) context.getHydraNow().clone();
        final IDbConnectionProvider dbConnectionProvider = factory.fetchUtil("DbConnectionProvider");
        try (final Connection con = dbConnectionProvider.fetchDbConnection())
        {
            final String processIdentifier = createUniqueIdentifier();
            final String taId = createUniqueIdentifier();

            final List<RecordData> parentRecordList = beginOutboundTa(new BeginOutboundTaParam(con, processIdentifier, now,
segmentName));
            final CookieManager cookieManager = new CookieManager();
            final ObjectMapper mapper = new ObjectMapper();
            for (final RecordData parentRecord : parentRecordList)
            {
                ProcessingState state = ProcessingState.DONE;
                Throwable error = null;

                final List<RecordData> childRecordList = beginOutboundTaPacket(
                    new BeginOutboundTaPacketParam(con, processIdentifier, parentRecord.getInternalId(), now));
                final List<Integer> successfulRecords = new ArrayList<>();
                try
                {
                    callParentService(mapper, cookieManager, parentRecord);
                    successfulRecords.add(Integer.valueOf(parentRecord.getInternalId()));
                    for (final RecordData childRecord : childRecordList)
                    {
                        callChildService(mapper, cookieManager, parentRecord, childRecord);
                        successfulRecords.add(Integer.valueOf(childRecord.getInternalId()));
                    }
                } catch (final RuntimeException | Error exc)
                {
                    error = exc;
                    state = ProcessingState.ERROR;
                    throw exc;
                } catch (final IOException exc)
                {
                    error = exc;
                    state = ProcessingState.ERROR;
                    final String msg = "Error calling services";
                    throw new SdiException("lkInvalidState", msg, exc, msg);
                } finally
                {
                    final MleFileData protFileData = writeProtocolFile(writeFile(factory), "Protocol data", now);
                    final MleFileData errFileData = writeErrorFile(writeFile(factory), "Error data", now);
                    final MleFileData dataFileData = writeDataFile(writeFile(factory), "Data", now);

                    endOutboundTaPacket(
                        new EndOutboundTaPacketParam(con, processIdentifier, parentRecord.getInternalId(),
Collections.singletonList(
                            new ControlRecordData(taId, state).withProtocolFileDataList(Collections.singletonList(
                                new ProtocolFileData("Test", "TstPrg", protFileData.getFileName(), protFileData.getFileSize(),
errFileData.getFileName(), errFileData.getFileSize(), dataFileData.getFileName(),
dataFileData.getFileSize(),
"myMessage", Integer.valueOf(43))
                            )
                        ), error, now, now, segmentName)
                    .withSuccessfulRecords(createListOfAllRecords(parentRecord, childRecordList), successfulRecords)
                );
            }
        }
        return null;
    } catch (final SQLException exc)
    {
        final String msg = "Error handling DB connection";
        throw new SdiException("lkDbError2", msg, exc, msg);
    } catch (final MleOutboundException exc)
    {
        throw new SdiException(exc.getLanguageKey(), exc.getShortMessage(), exc, exc.getParameters());
    }
}

private static List<Integer> createListOfAllRecords(final RecordData parentRecord,
    final List<RecordData> childRecordList)
{
    return Stream.concat(Stream.of(parentRecord), childRecordList.stream())
        .map(record -> Integer.valueOf(record.getInternalId()))
        .collect(Collectors.toList());
}

private static void callParentService(final ObjectMapper mapper,
    final CookieManager cookieManager,
    final RecordData parentRecord) throws IOException

```

```

{
    final String orderUrl = "https://localhost:8080/data/BOOrder/insert";
    final String orderJson = "{" +
        unwrapJson(mapper.writeValueAsString(new Dummy(Arrays.asList(
            new Param("order.id", extract(parentRecord.getSapSdata(), 0, 40)),
            new Param("order.ordertype", extract(parentRecord.getSapSdata(), 40, 5)),
            new Param("order.article", extract(parentRecord.getSapSdata(), 45, 40))
        )))) +
        ", \"columns\" : []}";

    new CommonHttpClient()
        .withCookieSupport(cookieManager)
        .withBasicAuthData("12345", "mpdv")
        .withRequestSettings(urlCon -> {
            urlCon.setRequestMethod("POST");
            urlCon.setRequestProperty("Content-Type", "application/json");
        })
        .communicate(orderUrl, orderJson);
}

private static void callChildService(final ObjectMapper mapper,
    final CookieManager cookieManager,
    final RecordData parentRecord,
    final RecordData childRecord) throws IOException
{
    final String operationUrl = "https://localhost:8080/data/BOOperation/insert";
    final String operationJson = "{" +
        unwrapJson(mapper.writeValueAsString(new Dummy(Arrays.asList(
            new Param("operation.id", extract(parentRecord.getSapSdata(), 0, 40) + extract(childRecord.getSapSdata(), 0, 4)),
            new Param("operation.designation", extract(childRecord.getSapSdata(), 4, 40)),
            new Param("operation.act.scheduled", extract(childRecord.getSapSdata(), 44, 1)),
            new Param("operation.plan.workplace", extract(childRecord.getSapSdata(), 45, 8)),
            new Param("operation.processing_code", extract(childRecord.getSapSdata(), 53, 6))
        )))) +
        ", \"columns\" : []}";

    new CommonHttpClient()
        .withCookieSupport(cookieManager)
        .withBasicAuthData("12345", "mpdv")
        .withRequestSettings(urlCon -> {
            urlCon.setRequestMethod("POST");
            urlCon.setRequestProperty("Content-Type", "application/json");
        })
        .communicate(operationUrl, operationJson);
}

private static ToIntBiFunction<CharSequence, String> writeFile(final ISystemUtilFactory factory)
{
    return (content, fileName) -> {
        final IPathManagementOpenFileForWritingAction writeAction =
        factory.fetchUtil("PathManagementOpenFileForWritingAction");
        try (final OutputStream out = writeAction.writeFile("MOCLOGS", fileName))
        {
            final byte[] data = content.toString().getBytes(StandardCharsets.UTF_8);
            out.write(data);
            out.flush();
            return data.length;
        } catch (final RuntimeException | IOException ignore)
        {
            return -1;
        }
    };
}

@JsonIgnoreProperties(ignoreUnknown = true)
static class Dummy
{
    private final List<Param> params;

    Dummy(final List<Param> params)
    {
        this.params = params;
    }

    public List<Param> getParams()
    {
        return this.params;
    }
}

@JsonIgnoreProperties(ignoreUnknown = true)
static class Param
{
    private final String acronym;
    private final String value;

    Param(final String acronym,
        final String value)
    {
        this.acronym = acronym;
        this.value = value;
    }

    public String getAcronym()
    {
        return this.acronym;
    }

    public String getValue()
    {
        return this.value;
    }
}

```

```
}  
}  
}
```

34.5 API reference



You cannot use classes and functions that are not documented when you use the MDS license.



The API reference below is only available in English.

34.5.1 MleOutboundUtil

34.5.1.1 createUniqueIdentifier(): String

Creates an UUID encoded as Base64 URL encoded string of size 22 characters. Useful as processing identifier and TA-IDs.

Return:

Base64 URL encoded UUID

34.5.1.2 extract(String sapSdata, int start, int length): String

Extracts and trims a substring from sapSdata

Input:

sapSdata: data record string

start: start position

length: length

Return:

extracted string

NullPointerException: if *sapSdata* is NULL

34.5.1.3 unwrapJson(String json): String

Removes the curly bracket from start and end of JSON string to create a JSON snippet that is embeddable

Input:

json: JSON string

Return:

JSON string without first and last character

NullPointerException: if *json* is NULL

34.5.1.4 beginOutboundTa(BeginOutboundTaParam param): List<RecordData>

Hierarchical and flat records: Starts the processing of all records of a given segment name and returns the list of open records matching the given segment name. Marks all records with the processing identifier

Input:

param: parameter structure

Return:

list of outbound records matching the provided segment name. Empty list, if no open records exist

Throws:

MleOutboundException: if an error occurs

NullPointerException: if *param* is NULL

34.5.1.5 endOutboundTa(EndOutboundTaParam param): void

Only flat records: Ends the processing of one or all records marked with the provided processing identifier.

Input:

param: parameter structure

Throws:

MleOutboundException: if an error occurs

NullPointerException: if *param* is NULL

34.5.1.6 beginOutboundTaPacket(BeginOutboundTaPacketParam param): List<RecordData>

Only hierarchical records: Begins a processing of all children of a provided parent internal ID

Input:

param: parameter structure

Return:

child records

Throws:

MleOutboundException: if an error occurs

NullPointerException: if *param* is NULL

34.5.1.7 endOutboundTaPacket(EndOutboundTaPacketParam param): void

Only hierarchical records: Ends the processing of all children

Input:

param: parameter structure

Throws:

MleOutboundException: if an error occurs

NullPointerException: if *param* is NULL

34.5.1.8 fetchOutboundConfigStructure(FetchOutboundConfigStructureParam param): Optional<MleOutboundConfigStruct>

Fetches MLE outbound configuration data from distribution model, logical system and logical system configuration

Input:

param: parameter structure

Return:

MLE outbound configuration data or empty structure if no or only inactive logical systems exist

Throws:

MleOutboundException: if an error occurs

NullPointerException: if *param* is NULL

34.5.1.9 writeProtocolFile(ToIntBiFunction<CharSequence, String> writeFunction, CharSequence content, Calendar now): MleFileData

Writes a protocol file by creating a file name matching following pattern: "PRO-yyyy-MM-dd HH:mm:ss.SSS\d\d\d\d\d.txt". The pattern of 5 times \d means a five digit random number.

Input:

writeFunction: function to write the file

content: file content

now: timestamp used for file name

Return:

data structure containing file name and file size

34.5.1.10 writeErrorFile(ToIntBiFunction<CharSequence, String> writeFunction, CharSequence content, Calendar now): MleFileData

Writes an error file by creating a file name matching following pattern: "ERR-yyyy-MM-dd HH:mm:ss.SSS\d\d\d\d\d.txt". The pattern of 5 times \d means a five digit random number.

Input:

writeFunction: function to write the file

content: file content

now: timestamp used for file name

Return:

data structure containing file name and file size

34.5.1.11 writeDataFile(ToIntBiFunction<CharSequence, String> writeFileFunction, CharSequence content, Calendar now): MleFileData

Writes a data file by creating a file name matching following pattern: "DAT-yyyy-MM-dd HH:mm:ss.SSS\d\d\d\d\d.txt". The pattern of 5 times \d means a five digit random number.

Input:

writeFileFunction: function to write the file

content: file content

now: timestamp used for file name

Return:

data structure containing file name and file size

34.5.1.12 resetAllInProcessDataRecords(ResetAllInProcessData RecordsParam param): void

Hierarchical and flat records: Resets all data records to status TODO (001) that are marked with the provided processing identifier.

Input:

parameter structure

34.5.1.13 resetInProcessDataRecordRecursive(ResetInProcessD ataRecordRecursiveParam param): void

Hierarchical and flat records: Resets provided data record and their children to status TODO (001) that are marked with the provided processing identifier.

Input:

parameter structure

34.5.2 MleOutboundException

This exception carries the error as a translatable key and corresponding parameters. Additionally this exception contains an english short message.

34.5.2.1 `getLanguageKey(): String`

Gets the language key that is used for translation. The language key is a string that is used to lookup a translated text.

Return:

language key that is used to lookup translated text

34.5.2.2 `getShortMessage(): String`

Gets the English error message that is meant for logging

Return:

English error message

34.5.2.3 `getParameterList(): List<String>`

Gets the parameters for the translation text as List

Return:

parameter list

34.5.2.4 `getParameters(): List<String>`

Gets the parameters for the language key as array.

Return:

parameter array

34.5.3 `MleFileData`

This data structure contains a file name and a file size

34.5.3.1 `getFileName(): String`

Gets the file name without path

Return:

file name without path

34.5.3.2 `getFileSize(): int`

Gets the file size

Return:

file size

34.5.4 RecordData

This data structure contains the data of a MLE outbound data record and is immutable.

**34.5.4.1 RecordData(String dbTald,
String recordStatus,
Calendar recordSaveTs,
Calendar recordWorkTs,
String recordSourceSystem,
String sapSegnam,
String sapMandt,
String sapDocnum,
String sapSegnum,
String sapPsgnum,
String sapHlevel,
String sapSdata,
String pid,
String param2,
int internalId,
Integer parentInternalId)**

Constructor

34.5.5 BeginOutboundTaParam

This data structure contains all parameters for `MleOutboundUtil.beginOutboundTa()` and is immutable..

34.5.5.1 BeginOutboundTaParam(Connection con, String processingIdentifier, Calendar workTs, String segmentName)

Constructor

Input:

con: DB connection

processingIdentifier: marker for the current processing thread: unique identifier

workTs: the work timestamp to use for the records

segmentName: segment name

Throws:

IllegalArgumentException: if any mandatory string value is NULL or empty

NullPointerException: if any non-string value is NULL

34.5.5.2 withPropagateTransactionMode()

Enables the use of the auto commit mode from caller. This can be used to participate in another transaction.

Return:

new instance with applied settings

34.5.6 EndOutboundTaParam

This data structure contains all parameters for `MleOutboundUtil.endOutboundTa()` and is immutable.

34.5.6.1 EndOutboundTaParam(Connection con, String processingIdentifier, List<ControlRecordData> controlRecordDataList, Throwable exceptionOfTryBlock, Calendar workTs, Calendar saveTs, String segmentName)

Constructor: The values of IDoc type, CIM type, message type, message code and message function are taken from the configuration of the segment.

Input:

con: DB Connection

processingIdentifier: unique identifier that was used to mark the records

controlRecordDataList: list of of data necessary for control record

exceptionOfTryBlock: exception of the try block. If the exception of the try block is not passed in then any exception of this method hides the exception of the try block

workTs: work timestamp

saveTs: save timestamp

segmentName: segment name

Throws:

IllegalArgumentException: if any mandatory string value is NULL or empty

NullPointerException: if any mandatory non-string value is NULL

**34.5.6.2 EndOutboundTaParam(Connection con,
String processingIdentifier,
List<ControlRecordData> controlRecordDataList,
Throwable exceptionOfTryBlock,
Calendar workTs,
Calendar saveTs,
String segmentName,
String dmSapIdotyp,
String dmSapCimtyp,
String dmSapMesTyp,
String dmSapMescod,
String dmSapMesfct)**

Constructor

Input:

con: DB connection

processingIdentifier: unique identifier that was used to mark the records

controlRecordDataList: list of data necessary for control record

exceptionOfTryBlock: exception of the try block. If the exception of the try block is not passed in then any exception of this method hides the exception of the try block

workTs: work timestamp

saveTs: save timestamp

segmentName: segment name

dmSapIdotyp: IDoc type

dmSapCimtyp: CIM type

dmSapMesTyp: message type

dmSapMescod: message code

dmSapMesfct: message function

34.5.6.3 withSuccessfulRecords(List<Integer> allRecords, List<Integer> successfulRecords): EndOutboundTaParam

Sets the state of successful records to DONE while all other records get the state of the control record.
The invocation of this method is only possible if there is only one control record.

Input:

allRecords: all internal IDs of all records

successfulRecords: internal IDs of successful records only

Return:

new instance with applied settings

Throws:

NullPointerException: if any of the parameters is NULL

IllegalStateException: if the invocation of this method is not allowed due to other method calls

34.5.6.4 withDataRecordClassifierList(List<DataRecordClassifier> dataRecordClassifierList): EndOutboundTaParam

Sets individual state codes to each data record

Input:

dataRecordClassifierList list of data record internal ID to state

Return:

new instance with applied settings

Throws:

NullPointerException: if the parameter is NULL

IllegalStateException: if the invocation of this method is not allowed due to other method calls

34.5.6.5 withPropagateTransactionMode(): EndOutboundTaParam

Enables the use of the auto commit mode from caller. This can be used to participate in another transaction.

Return:

new instance with applied settings

34.5.7 BeginOutboundTaPacketParam

This data structure contains all parameters for `MleOutboundUtil.beginOutboundTaPacket()` and is immutable.

34.5.7.1 BeginOutboundTaPacketParam(Connection con, String processingIdentifier, int parentInternalId, Calendar workTs)

Constructor

Input:

con: DB connection

processingIdentifier: unique identifier that was used to mark the records

parentInternalId: internal ID of parent record

workTs: work timestamp

Throws:

IllegalArgumentException: if any mandatory string value is NULL or empty

NullPointerException: if any mandatory non-string value is NULL

34.5.7.2 withPropagateTransactionMode(): BeginOutboundTaPacketParam

Enables the use of the auto commit mode from caller. This can be used to participate in another transaction.

Return:

new instance with applied settings

34.5.8 EndOutboundTaPacketParam

This data structure contains all parameters for `MleOutboundUtil.endOutboundTaPacket()` and is immutable.

34.5.8.1 EndOutboundTaPacketParam(Connection con, String processingIdentifier, int parentInternalId, List<ControlRecordData> controlRecordDataList, Throwable exceptionOfTryBlock, Calendar workTs, Calendar saveTs, String segmentName)

Constructor: The values of IDoc type, CIM type, message type, message code and message function are taken from the configuration of the segment..

Input:

con: DB connection

processingIdentifier: unique identifier that was used to mark the records

parentInternalId: internal ID of parent record

controlRecordDataList: list of of data necessary for control record

exceptionOfTryBlock: exception of the try block. If the exception of the try block is not passed in then any exception of this method hides the exception of the try block

workTs: work timestamp

saveTs: save timestamp

segmentName: segment name

Throws:

IllegalArgumentException: if any mandatory string value is NULL or empty

NullPointerException: if any mandatory non-string value is NULL

**34.5.8.2 EndOutboundTaPacketParam(Connection con,
String processingIdentifier,
int parentInternalId,
List<ControlRecordData> controlRecordDataList,
Throwable exceptionOfTryBlock,
Calendar workTs,
Calendar saveTs,
String segmentName,
String dmSapIdotyp,
String dmSapCimtyp,
String dmSapMesTyp,
String dmSapMescod,
String dmSapMesfct)**

Constructor

Input:

con: DB connection

processingIdentifier: unique identifier that was used to mark the records

parentInternalId: internal ID of parent record

controlRecordDataList: list of of data necessary for control record

exceptionOfTryBlock: exception of the try block. If the exception of the try block is not passed in then any exception of this method hides the exception of the try block

workTs: work timestamp

saveTs: save timestamp

segmentName: segment name

dmSapIdotyp: IDoc type

dmSapCimtyp: CIM type

dmSapMesTyp: message type

dmSapMescod: message code

dmSapMesfct: message function

Throws:

IllegalArgumentException: if any mandatory string value is NULL or empty

NullPointerException: if any mandatory non-string value is NULL

34.5.8.3 withSuccessfulRecords(List<Integer> allRecords, List<Integer> successfulRecords): EndOutboundTaPacketParam

Sets the state of successful records to DONE while all other records get the state of the control record.
The invocation of this method is only possible if there is only one control record.

Input:

allRecords: all internal IDs of all records

successfulRecords: internal IDs of successful records only

Return:

new instance with applied settings

Throws:

NullPointerException: if any parameter is NULL

IllegalStateException: if the invocation of this method is not allowed due to other method calls

34.5.8.4 withDataRecordClassifierList(List<DataRecordClassifier> dataRecordClassifierList): EndOutboundTaPacketParam

Sets individual states to each data record

Input:

dataRecordClassifierList: list of data record internal ID to state

Return:

new instance with applied settings

Throws:

NullPointerException: if parameter is NULL

IllegalStateException: if the invocation of this method is not allowed due to other method calls

34.5.8.5 withPropagateTransactionMode(): EndOutboundTaPacketParam

Enables the use of the auto commit mode from caller. This can be used to participate in another transaction.

Return:

new instance with applied settings

34.5.9 ControlRecordData

This data structure contains data for MLE outbound control records and is immutable. This data structure contains optionally also a list of file records.

34.5.9.1 ControlRecordData(String tald, ProcessingState state)

Constructor

Input:

tald: transaction ID of the control record

state: processing code of the control record

Throws:

IllegalArgumentException: if any mandatory string value is NULL or empty

NullPointerException: if any mandatory non-string value is NULL

34.5.9.2 withProtocolFileDataList(List<ProtocolFileData> protocolDataList): ControlRecordData

Adds file records to the outbound control record.

Input:

protocolDataList: list of file records. Each file record may contain a protocol file, an error file and a data file

Return:

new instance with applied settings

Throws:

NullPointerException: if the parameter is NULL

34.5.10 ProtocolFileData

This data structure contains data for the file record of outbound MLE. This data structure is immutable.

34.5.10.1 ProtocolFileData(String designation, String program, String protocolFileName, int protocolFileSize, String errorFileName, int errorFileSize, String dataFileName, int dataFileSize, String message, Integer textNo)

Constructor

Input:

designation: mandatory description of the file record

program: optional program name

protocolFileName: optional file name of the protocol file

protocolFileSize: file size of the protocol file or 0 if no protocol file is provided

errorFileName: optional file name of the error file

errorFileSize: file size of the protocol file or 0 if no error file is provided

dataFileName: optional file name of the data file

dataFileSize: file size of the protocol file or 0 if no data file is provided

message: optional message assigned to the file record

textNo: optional text number assigned to the file record

Throws:

IllegalArgumentException: if the designation is NULL or empty

34.5.11 DataRecordClassifier

This data structure allows to set individual state codes at each data record. This data structure is immutable.

34.5.11.1 DataRecordClassifier(String talId, ProcessingState dataState, Integer internalId): ControlRecordData

Constructor

Input:

talId: transaction ID of the data record

dataState: processing code of the data record

internalId: internal ID of the data record

Throws:

IllegalArgumentException: if talId is NULL or empty

NullPointerException: if dataState or internalId is NULL

34.5.12 ProcessingState

Enumeration of processing codes for MLE outbound

34.5.12.1 getControlRecordCode(): String

Gets the processing code for a control record

34.5.12.2 getDataRecordCode(): String

Gets the processing code for a data record

34.5.13 FetchOutboundConfigStructureParam

This data structure contains all parameters for `MleOutboundUtil.fetchOutboundParameterStructure()` and is immutable.

34.5.13.1 FetchOutboundConfigStructureParam(Connection con, String segmentName, boolean isEncryptedPassword)

Constructor

Input:

con: DB connection

segmentName: segment name

isEncryptedPassword: is the password in the DB encrypted

Throws:

IllegalArgumentException: if any mandatory string value is NULL or empty

NullPointerException: if any mandatory non-string value is NULL

34.5.14 MleOutboundConfigStruct

This data structure contains all data from the MLE outbound configuration and is immutable.

34.5.14.1 getDmDirect(): String

Getter

Return:

direction: I for input; O for output

34.5.14.2 getDmDesc(): String

Getter

Return:

description

34.5.14.3 getDmSapMestyp(): String

Getter

Return:

message type

34.5.14.4 getDmSapIdoctyp(): String

Getter

Return:

IDoc type

34.5.14.5 getDmSapCimtyp(): String

Getter

Return:

CIM type

34.5.14.6 getDmSapMescod(): String

Getter

Return:

message code

34.5.14.7 getDmSapMesfct(): String

Getter

Return:

message function

34.5.14.8 getDmSourcesys(): String

Getter

Return:

SAP target system

34.5.14.9 getDmSapMescod(): String

Getter

Return:

message code

34.5.14.10 getDmSapSegnam01(): String

Getter

Return:

segment name for upload

34.5.14.11 getDmSapSegnam02(): String

Getter

Return:

segment name 2

34.5.14.12getDmSapSegnam03(): String

Getter

Return:

segment name 3

34.5.14.13getDmSapSegnam04(): String

Getter

Return:

segment name 4

34.5.14.14getDmSapSegnam05(): String

Getter

Return:

segment name 5

34.5.14.15getDmSapSegnam06(): String

Getter

Return:

segment name 6

34.5.14.16getDmSapSegnam07(): String

Getter

Return:

segment name 7

34.5.14.17getDmSapSegnam08(): String

Getter

Return:

segment name 8

34.5.14.18getDmSapSegnam09(): String

Getter

Return:

segment name 9

34.5.14.19getDmSapSegnam10(): String

Getter

Return:

segment name 10

34.5.14.20getDmSapTest(): String

Getter

Return:

test flag

34.5.14.21getDmDest(): String

Getter

Return:

logical target system for output process

34.5.14.22getDmParam1(): String

Getter

Return:

value of parameter 1

34.5.14.23getDmParam2(): String

Getter

Return:

value of parameter 2

34.5.14.24getLsLogsys(): String

Getter

Return:

logical system

34.5.14.25getLsDesc(): String

Getter

Return:

description of logical system

34.5.14.26getLsParam1(): String

Getter

Return:

communication mode

34.5.14.27getLsParam2(): String

Getter

Return:

value of parameter 2

34.5.14.28getLsActRole(): String

Getter

Return:

active role

34.5.14.29getLscParamName01(): String

Getter

Return:

parameter name 1

34.5.14.30getLscParamVal01(): String

Getter

Return:

parameter value 1

34.5.14.31getLscParamName02(): String

Getter

Return:

parameter name 2

34.5.14.32getLscParamVal02(): String

Getter

Return:

parameter value 2

34.5.14.33getLscParamName03(): String

Getter

Return:

parameter name 3

34.5.14.34getLscParamVal03(): String

Getter

Return:

parameter value 3

34.5.14.35getLscParamName04(): String

Getter

Return:

parameter name 4

34.5.14.36getLscParamVal04(): String

Getter

Return:

parameter value 4

34.5.14.37getLscParamName05(): String

Getter

Return:

parameter name 5

34.5.14.38getLscParamVal05(): String

Getter

Return:

parameter value 5

34.5.14.39getLscParamName06(): String

Getter

Return:

parameter name 6

34.5.14.40getLscParamVal06(): String

Getter

Return:

parameter value 6

34.5.14.41getLscParamName07(): String

Getter

Return:

parameter name 7

34.5.14.42getLscParamVal07(): String

Getter

Return:

parameter value 7

34.5.14.43getLscParamName08(): String

Getter

Return:

parameter name 8

34.5.14.44getLscParamVal08(): String

Getter

Return:

parameter value 8

34.5.14.45getLscParamName09(): String

Getter

Return:

parameter name 9

34.5.14.46getLscParamVal09(): String

Getter

Return:

parameter value 9

34.5.14.47getLscParamName10(): String

Getter

Return:

parameter name 10

34.5.14.48getLscParamVal10(): String

Getter

Return:

parameter value 10

34.5.14.49getLscParamName11(): String

Getter

Return:

parameter name 11

34.5.14.50getLscParamVal11(): String

Getter

Return:

parameter value 11

34.5.14.51getLscParamName12(): String

Getter

Return:

parameter name 12

34.5.14.52getLscParamVal12(): String

Getter

Return:

parameter value 12

34.5.14.53getLscParamName13(): String

Getter

Return:

parameter name 13

34.5.14.54getLscParamVal13(): String

Getter

Return:

parameter value 13

34.5.14.55getLscParamName14(): String

Getter

Return:

parameter name 14

34.5.14.56getLscParamVal14(): String

Getter

Return:

parameter value 14

34.5.14.57getLscParamName15(): String

Getter

Return:

parameter name 15

34.5.14.58getLscParamVal15(): String

Getter

Return:

parameter value 15

34.5.14.59getLscSapSndPor(): String

Getter

Return:

sending system port

34.5.14.60getLscSapSndPrt(): String

Getter

Return:

partner type of sending system

34.5.14.61getLscSapSndPrn(): String

Getter

Return:

partner number of sender

34.5.14.62getLscSapRcvPor(): String

Getter

Return:

receiver port

34.5.14.63getLscSapRcvPrt(): String

Getter

Return:

partner type of receiver

34.5.14.64getLscSapRcvPrn(): String

Getter

Return:

partner number of receiver

34.5.14.65getLscParam1(): String

Getter

Return:

value of parameter 1

34.5.14.66getLscParam2(): String

Getter

Return:

value of parameter 2

34.5.14.67getLscProgTyp(): String

Getter

Return:

program type either ?C for client or ?S for server

34.5.15 ResetAllInProcessDataRecordsParam

This data structure contains all parameters for `MleOutboundUtil.resetAllInProcessDataRecords()` and is immutable.

34.5.15.1 ResetAllInProcessDataRecordsParam(Connection con, String processingIdentifier)

Constructor

Input:

con: DB Connection

processingIdentifier: unique identifier that was used to mark the records

Throws:

`IllegalArgumentException`: if any mandatory string value is NULL or empty

`NullPointerException`: if any mandatory non-string value is NULL

34.5.16 ResetInProcessDataRecordRecursiveParam

This data structure contains all parameters for `MleOutboundUtil.resetInProcessDataRecordRecursive()` and is immutable.

34.5.16.1 ResetInProcessDataRecordRecursiveParam(Connection con, String processingIdentifier, int internalId)

Constructor

Input:

con: DB Connection

processingIdentifier: unique identifier that was used to mark the records

internalId: internal ID of record to reset

Throws:

`IllegalArgumentException`: if any mandatory string value is NULL or empty

`NullPointerException`: if any mandatory non-string value is NULL

35 Instructions Java Userexits mit IntelliJ IDEA

35.1 Purpose

Use this guide if you want to use the free-to-use IntelliJ IDEA Community development environment to develop user exits for the server.

This guide shows you how to install and setup a development area.

This guide also shows you how to test the created user exits.

35.2 Requirements

35.2.1 Versions

This guide was prepared in February, 2020 with the help of the IntelliJ IDEA Community Version 2019.3.3 and Amazon Corretto JDK Version 1.8.0_232. This guide can be used from MW 3x or with the MIP 1.0.

35.2.2 Basics

The guide assumes that you have basic knowledge of the Java programming language and a Java development environment.

The author of the guide assumes that you know how to create Java user exits. In order to do so, you should have participated in a suitable training course from MPDV (e.g. CUT-MOC training or an individual training course).

35.2.3 Licenses

Java user exits can only be used if a valid customizing license (e.g. MDS-BAS, MIP-SDK) or a valid runtime license (e.g. MDS-RTL) is available on the system.

35.2.4 Software and files

- You need the „**MPDV Compile Libraries**“. It is a JAR file from MPDV that you have to integrate as external modules to compile the user exits.
You receive the JARs during the training. Current version can be downloaded from the MPDV Support Portal (download area).
- Download **Amazon Corretto JDK 8** (Windows x64).
- Download **IntelliJ IDEA Community Edition**.

35.3 How to proceed

35.3.1 Install JDK Amazon Corretto.

You receive a ZIP file when you download Amazon Corretto JDK. Copy the content of the ZIP file in one of your directories and remember the location for later.

Here, all required files can be found in the „e:\DevSrc\CustUserexitDemo“ directory. The JDK is located for our example in „e:\DevSrc\CustUserexitDemo\jdk1.8.0_232“.

35.3.2 Provide MPDV Compile Libraries

You require JAR files from MPDV in order to integrate these as external modules to compile the user exits.

You receive the JARs during the training. Current version can be downloaded from the MPDV Support Portal (download area).

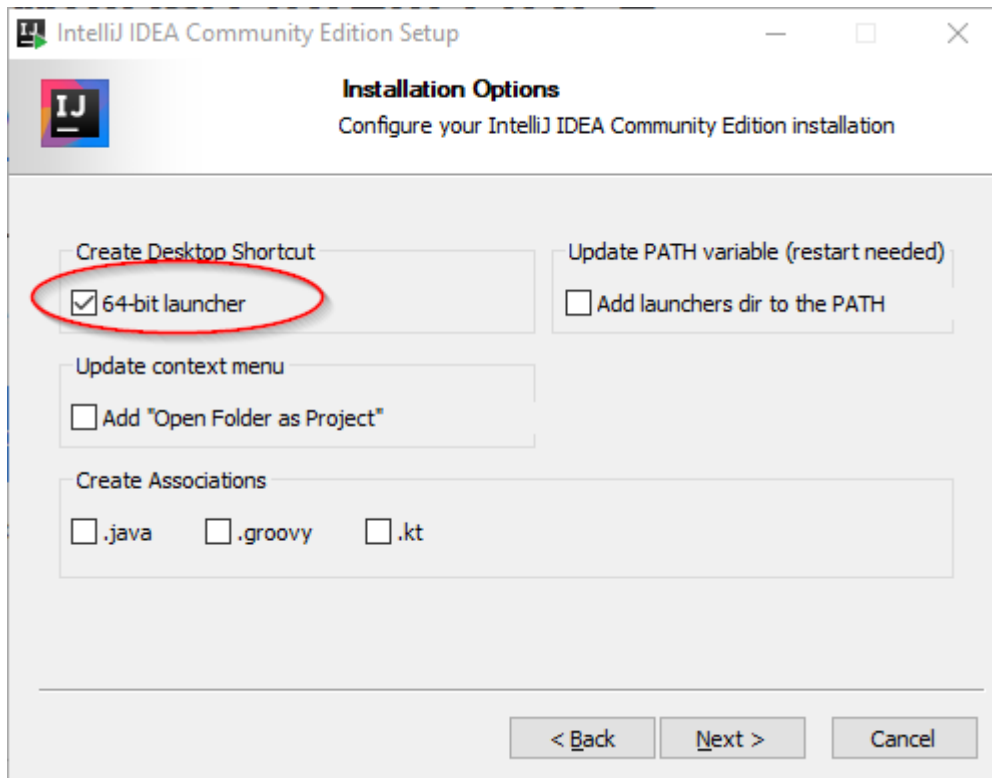
Copy the content of the ZIP file with the JARs in one of your directories and remember the location for later.

Here, all required files can be found in the „e:\DevSrc\CustUserexitDemo“ directory. The JAR files are located for our example in „e:\DevSrc\CustUserexitDemo\Java-Lib“.

35.3.3 Install and setup the IntelliJ IDEA Community

35.3.3.1 Installing IntelliJ IDEA

Start installing the downloads in the IntelliJ IDEA Community. Follow the instructions of the installation program. We recommend to activate the option „64-bit-launcher“.



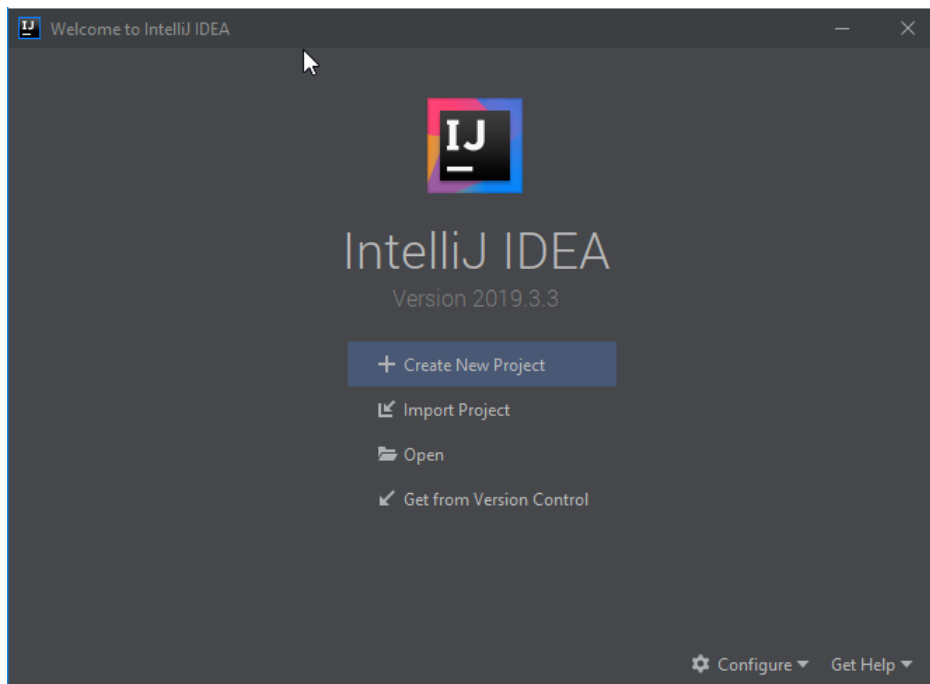
35.3.3.2 During the first start

Start IntelliJ IDEA.

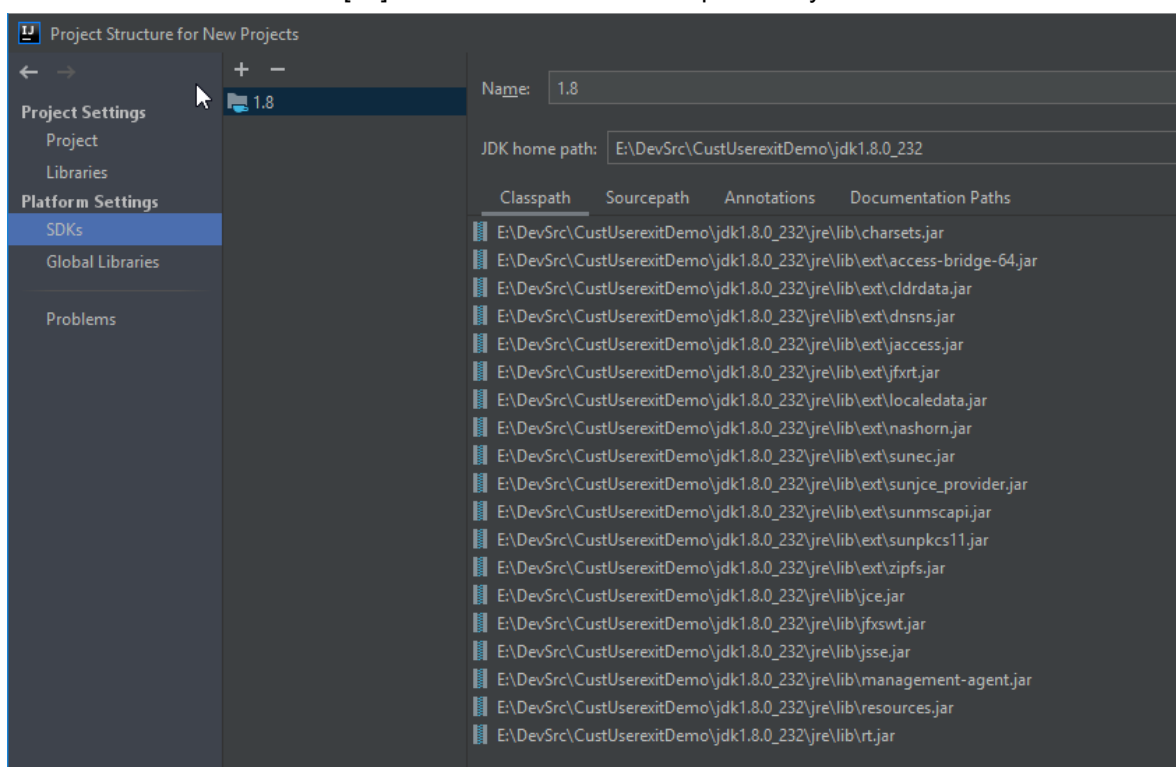
During the first start, basic settings of the IDE are requested. We also specify requirements to create new projects.

- The "Import IntelliJ IDEA Settings From ..." opens after the first start. Select "Do not import settings".
- „Customize IntelliJ“
 - „Set UI theme“:
You can choose according to your personal taste "Dracula" or "Light".
 - „Default Plugins“: You don't need default plugins.
 - „Featured Plugins“: You don't need featured plugins.
- Select now „Start using IntelliJ IDEA“

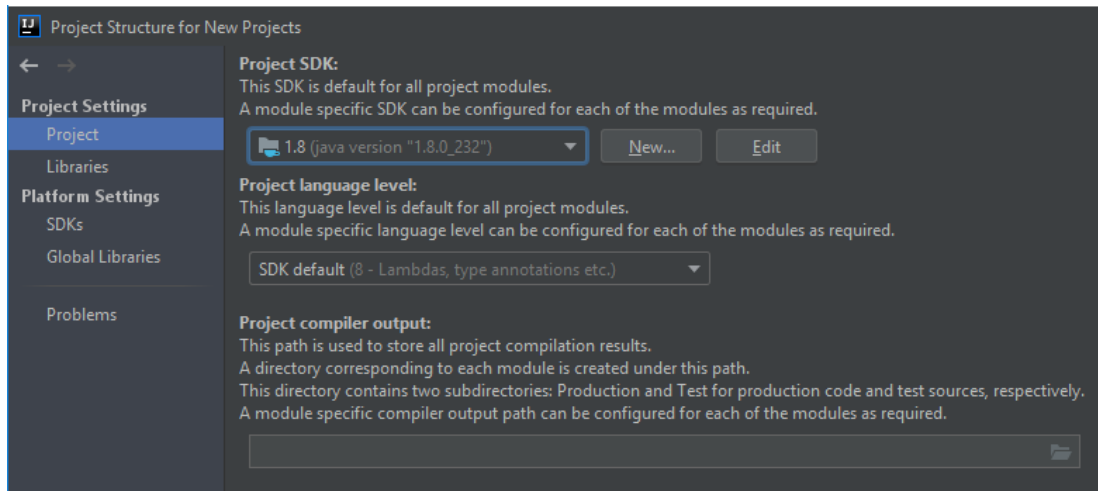
"Welcome to IntelliJ IDEA":



- Select "Configure" which is located at the lower edge.
 - Select "Structure for new projects"
 - Select "Platform Settings"
 - Select the SDKs.
 - [-]: Delete any default SDKs, if existing.
 - [+]: Select "JDK" and add the previously installed JDK Amazon Corretto 8.



- Select "Project Settings"
 - Select now "Project"
 - "Project SDK": select Amazon Corretto 1.8

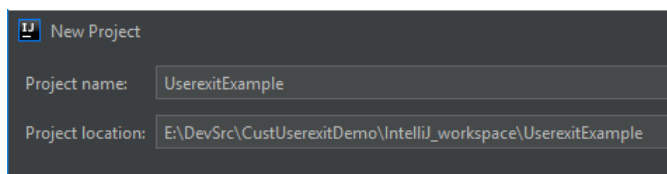


- Confirm the settings with [OK]
- Back in the dialog "Welcome to IntelliJ IDEA", select the option "Create New Project".

35.3.4 The first project

35.3.4.1 Create project

- "Create New Project"
 - Select "Java".
 - You don't need any additional libraries and frameworks.
 - Select [Next].
 - You might have to deactivate "Create project from template".
 - Select [Next].
 - "Project name": choose a name as you see fit. We use here "UserexitExample".
 - Select "Project location". We store the project in "e:\DevSrc\CustUserexitDemo\IntelliJ_workspace\UserexitExample" in our example. Choose. You can also create new folders for the directory in the selection dialog.

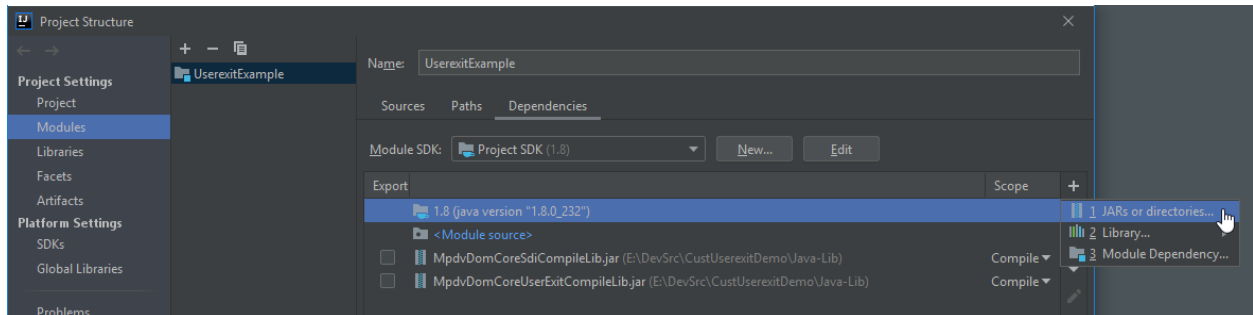


- Select [Finish].

35.3.4.2 Add MPDV compile libraries

In the next step we will release the compile libraries for user exits provided by MPDV.

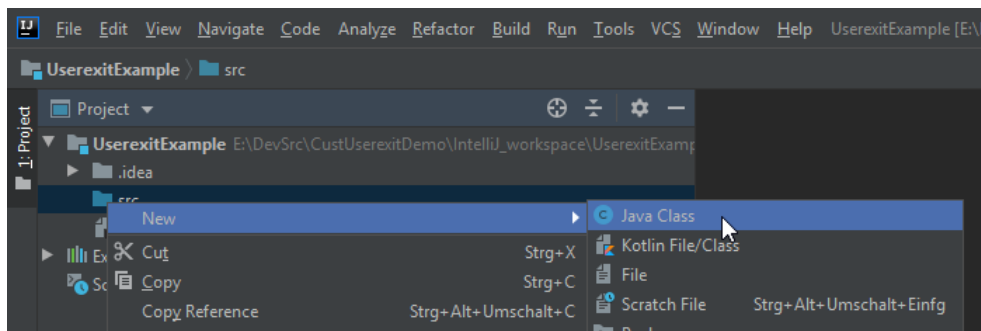
- Select in the menu "File / project structure".
- Select "Project settings" / "Modules"
- Select the created project. Here it is "UserexitExample".
- Select the tab "Dependencies".
- [+] / "JARs or directories"
 - o Add the following libraries including the JAR files you might require:
„MpdvDomCoreSdiCompileLib.jar“ and
„MpdvDomCoreUserExitCompileLib.jar“



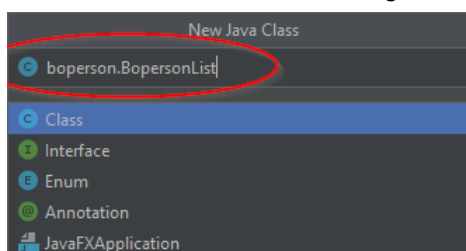
- Confirm with [OK]

35.3.5 Create user exits

- Open with the right mouse button the context menu on the "src" path of the project and select "New" / "Java Class".

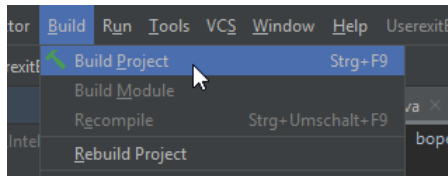


- Enter the „Fully qualified class name“. In this example, we create a user exit for the service "BOPerson.List" in the domain "BOPerson". The "Fully qualified class name" is deduced from this: "boperson.BopersonList". Please observe the upper/lower case and the position of points. The rules for forming the "fully qualified class name" are not the subject of this guide. Details on how to form the "fully qualified class name" can be found in the product documentation (e.g. MDS-BAS) and in the documentation of the training courses attended.



- Confirm the entry of the "fully qualified class name" [Enter].

- Enter the source code. This manual does not cover the source code in detail. You can use the source code from the tutorial "Hiding columns depending on a function authorization" as an example.
- Build the project.



35.3.6 Test the user exits

35.3.6.1 Activate the development mode

You can also activate the developer mode on the Web Service Provider (WSP) with the option "development.mode = 1". Among other things, the developer mode ensures that changes to user exits take effect the next time the corresponding service is called, without you having to restart the WSP each time.

Edit the file "<InstallDir>\jdir\MOC\1\config.properties" and insert the line "development.mode = 1" below "configreload.timeout=...".

```
#####
#####INSTANCE CONFIGURATION (1)#####
#####
# Please make sure that after configuration values is no TAB or SPACE

# Config reload timeout in seconds
# After this timeout it is checked, if the instance configuration
# (this file) has changed and the config must be reloaded
configreload.timeout=180

development.mode=1
```

Restart the WSP after the configuration file was changed to activate it.

35.3.6.2 Set log level

The Web Service Provider WSP supports the following log levels:

1. "trace" (contains most information and all other levels)
2. "debug" (contains "info", "warn", and "error")
3. "info" (contains "warn" and "error")
4. "warn" (contains "error")
5. "error"

6. "all" (all levels, equals "trace")

You can change the log level. The log level is set to "error" in standard installations.

Edit the file "<InstallDir>\jdir\MOC\1\config.properties". You can change the log level on entries beginning with "log4j.category".

```
...
# The level of logging can be changed here
# ...
# If needed replace error with another level
log4j.category.de.mpdv=debug, FileAppender
log4j.category.de.mpdv.common.configManagement=error, FileAppender
log4j.category.de.mpdv.formulaEvaluator=error, FileAppender
log4j.category.de.mpdv.hydra.common.management=error, FileAppender
...
```

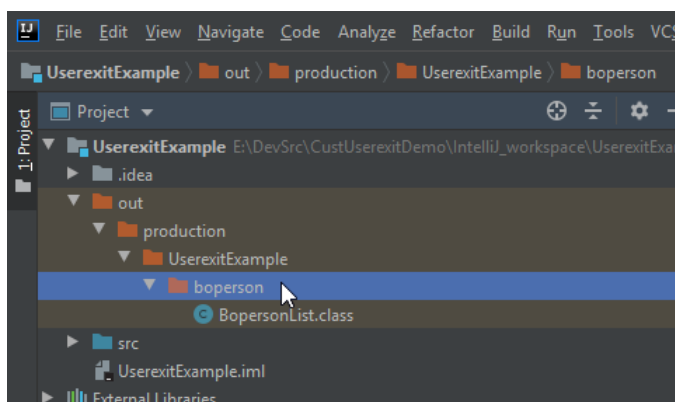
If you develop user exits for services, you must create additional entries in the logging configuration for them. The reason for it is that user exits that are not in the MPDV's namespace, have other class paths. Example: the following goes for a user exit of the "BOPerson.list" service:

```
...
log4j.category.de.mpdv.formulaEvaluator=error, FileAppender
log4j.category.de.mpdv.hydra.common.management=error, FileAppender
log4j.category.boperson.BopersonList=debug, FileAppender
...
```

Restart the WSP after the configuration file was changed to activate it.

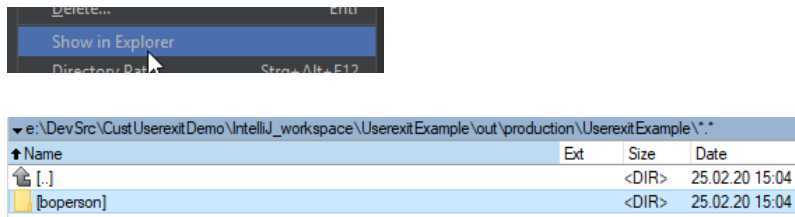
35.3.6.3 Copy class file to the server

After a successful "Build", there is a "class" file available on the "out" directory.



Right-click with the mouse button on the context menu on the package level (one level above the class file). In the example you open the context menu on "boperson".

Select "Show in Explorer".



Copy the directory (in the example "boperson") from the Explorer to the server directory for user exits in the local scope.

```
<InstalDir>/jdir/moc/1/userexit/local
```

You must create the "local" directory if it does not exist.

Here in our example, the file is located on the server at

```
\\MyServer\mip1\jdir\MOC\1\userexit\local\boperson\BopersonList.class
```

35.4 Result

Call the service that triggers the user exit. You can use the relevant application of the client for this or the service interface.

You can check in the log file of the WSP if the user exit is loaded and executed. If you have set at least log level "debug" and the user exit is successfully loaded and executed, you will find lines in the log file like:

```
... invoke exit method: [boperson.BopersonList.sdiGlobalAddResultTransformationCallbacks] scope: [LOCAL]}
```

If you use the logger in the user exit itself, you will also find your own log output in this log file, depending on the set log level.

In case of an error you will find in the log file "Exceptions" references that might indicate the problem.

36 Instructions Java User exit

Hide columns

36.1 Purpose

This guide shows you how to use a Java user exit to hide the contents of certain columns in a service. Contents are hidden if a user is not permitted to access a user-defined function.

User exits of this kind make it possible to suppress the output of personal data without depriving users of access to the entire client application.

36.2 Requirements

36.2.1 Versions

This guide can be used from MW 3x or MIP 1.0.

36.2.2 Licenses

Java user exits can only be used if a valid customizing license (e.g. MDS-BAS, MIP-SDK) or a valid runtime license (e.g. MDS-RTL) is available on the system.

36.2.3 Basics

The guide assumes that you have a basic knowledge of the Java programming language and a Java development environment.

The author of the guide assumes that you know how to create Java user exits. In order to do so, you should have participated in a suitable training course from MPDV (e.g. CUT-MOC training or an individual training course).

36.2.4 Java development environment

You require a Java development environment to compile the class file.

Setting up a development environment is not part of this guide. This document shows you how to do it [Java Userexit with IntelliJ IDEA](#) . This document also describes how to set up the IntelliJ IDEA Community development environment and create a project where the required class can be generated.

36.3 How to proceed

36.3.1 Task

Only those users who are authorized for the function authorization "u_person_all" should be able to see all columns in the client application "HR master data".

The following rules should apply for all other users:

- A WhiteList defines that the following columns should be displayed as usual:
 - o Personnel number
 - o Badge number
 - o Name
 - o Date "Valid from"
 - o Date "Valid until"
 - o Company
 - o Cost center
- A fixed value is defined with a replacement list for the following columns:
 - o The entry date for all persons should be the 01.01.1999.
 - o The telephone number of the company's head office should be used as the "Telephone number company" for all staff.
 - o The e-mail address of the company's head office should be used as the "E-mail address company" for all staff.
- Depending on the data type of the columns, default values should be displayed for all other columns:
 - o All alphanumeric columns (strings) should be displayed as "X" for all staff.
 - o All other columns are set to empty.

36.3.2 Source code boperson.BopersonList

```
package boperson;

import de.mpdv.customization.userExit.IUserExitParam;
import de.mpdv.sdi.data.*;
import de.mpdv.sdi.data.ISdiDataRow.DataRowType;
import de.mpdv.sdi.data.ue.GlobalUeContext;
import de.mpdv.sdi.data.ue.SdiGlobalAddResultTransformationCallbacksResult;
import de.mpdv.sdi.systemutility.*;
import java.util.*;

public class BopersonList {

    public static final String AUTHORIZATION_KEY = "u_person_all";

    public void sdiGlobalAddResultTransformationCallbacks(final IUserExitParam ueParam) {
        final GlobalUeContext context = ueParam.get("context");
        final ISystemUtilFactory factory = ueParam.get("factory");
        final ISdiLoggerProvider logProvider = factory.fetchUtil("LoggerProvider");
        final ISdiLogger logger = logProvider.fetchLogger(this.getClass());

        if (!checkUserAuthorization(factory, context.getUserId())) {
            logger.info("Replacing values because of missing authorization: [" + AUTHORIZATION_KEY + "]");
            final SdiGlobalAddResultTransformationCallbacksResult result = new SdiGlobalAddResultTransformationCallbacksResult(
                Arrays.asList(functionParameter -> {
                    if (functionParameter.getDataRow().getDataRowType() != DataRowType.DATA_RECORD) {
                        return new SdiEagerDataRowStream(functionParameter.getDataRow());
                    }
                })
            );
        }
    }
}
```

```

    }

    final List<String> whiteList = Arrays.asList(
        "person.id", "person.card_id",
        "person.firstname", "person.lastname",
        "person.middlename", "person.name",
        "person.valid_from", "person.valid_to",
        "person.costcenter", "person.company");

    final Map<String, Object> contentReplacement = new HashMap<String, Object>() {
        {
            put("person.date_of_joining", new GregorianCalendar(1999, Calendar.JANUARY, 1));
            put("person.telephone_number.business", "01234 56789-0");
            put("person.email_company", "info@mycompany.com");
        }
    };

    for (int colIdx = 0; colIdx < functionParameter.getDataRow().getColumnCount(); colIdx++) {
        if (!whiteList.contains(functionParameter.getDataRow().getColumnName(colIdx))) {
            replaceValues(functionParameter.getDataRow(), colIdx, contentReplacement);
        }
    }
    return new SdiEagerDataRowStream(functionParameter.getDataRow());
}
})
);
ueParam.set("result", result);
}
}

private static boolean checkUserAuthorization(final ISystemUtilFactory factory,
                                             final String userId) {
    final IServiceCaller serviceCaller = factory.fetchUtil("ServiceCaller");
    final IServiceCallResult result = serviceCaller.callService(
        "SYSAuthorization.checkAuthorization",
        new ColumnConfigurator(false, Collections.singletonList("authorization.is_authorized")),
        Arrays.asList(
            new SpecialParam("user.id", OperatorType.EQUAL, userId),
            new SpecialParam("authorization.key", OperatorType.EQUAL, AUTHORIZATION_KEY)
        ),
        Collections.emptyList()
    );
    return Optional.ofNullable(result)
        .map(IServiceCallResult::getResultTable)
        .map(IDataTable::getData)
        .filter(rows -> !rows.isEmpty())
        .map(rows -> rows.get(0))
        .filter(row -> !row.isEmpty())
        .map(row -> (Boolean) row.get(0))
        .orElse(Boolean.FALSE);
}

private void replaceValues(final ISdiDataRow dataRow, final int colIdx, Map<String, Object> overwriteMap) {
    if (overwriteMap.containsKey(dataRow.getColumnName(colIdx))) {
        dataRow.setCellValue(colIdx, overwriteMap.get(dataRow.getColumnName(colIdx)));
    } else {
        switch (dataRow.getDataType(colIdx)) {
            case STRING:
                dataRow.setCellValue(colIdx, "X");
                break;
            case DATETIME:
            case DECIMAL:
            case INTEGER:
            case LONG:
            case BOOLEAN:
            case BINARY:
            default:
                dataRow.setCellValue(colIdx, null);
                break;
        }
    }
}
}
}
}
}

```

36.3.3 Source code explanations

36.3.3.1 Basics

The implementation takes place with a global user exit called "GlobalExit". You can find details about the "GlobalExits" and the used interfaces and classes in the documents [MDS GlobalExits](#) and [MDS SystemUtilities](#).

36.3.3.2 Imports

First, the classes are imported from the "MPDV Compile Libraries".

36.3.3.3 Class „BopersonList“

When defining the class "BopersonList", the constant the AUTHORIZATION_KEY is defined at the outset. It contains the function authorization a user needs to view all columns in the result set.

36.3.3.4 The registration of the callback function

Then a callback function is registered with the method "sdiGlobalAddResultTransformationCallbacks". The callback function is specified as a Lambda expression.

The callback function is later called for each row in the result set and can change its contents. The callback function later replaces the contents of the columns in the row if the user is not authorized for all columns.

The callback function is only registered if the logged in user is not authorized for all result columns. If the user is authorized for all columns, no callback function is registered and the service is executed without further intervention. The authorization check is performed with "checkUserAuthorization", which is described below.

The callback function only executes something if it is called for a "DataRowType.DATA_RECORD". For other "DataRowTypes" the data record is output unchanged.

The "whiteList" defines those columns that remain unchanged, even if the logged-in user is not authorized for all columns of the result set.

The map "contentReplacement" specifies a fixed value for particular columns. If the logged user is not authorized for all columns of the result set, the values of these columns are replaced by fixed values.

The loop at the end of the callback function then runs through all columns of the result set. If the column is not in the "whiteList", the method "replaceValues" is called. This method obtains the map „contentReplacement“ as parameter. The method itself is described further below.

The changed "DataRow" is output at the end of the callback function.

36.3.3.5 Method checkUserAuthorization

The method "checkUserAuthorization" checks if the logged user is authorized for all columns of the result set.

To do so, the method calls the service "SYSAuthorization.checkAuthorization". The service has as the parameter the logged user from the service and the function authorization that you defined as a constant at the beginning of the class.

The SYSAuthorization.checkAuthorization service has only one cell in the result set: a row with an authorization.is_authorized column of the type BOOLEAN. The content is "true" if the user is authorized, other than that it is "false".

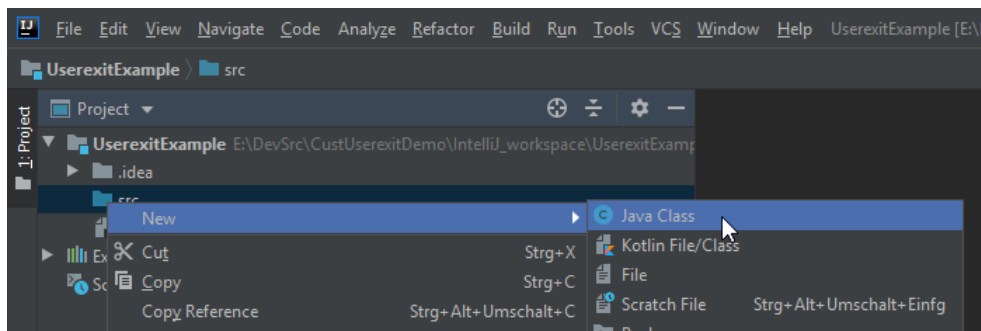
36.3.3.6 Method replaceValues

The method "replaceValues" executes the actual replacement of the value in a column of a result row. This method is called for all columns that are not on the "whiteList". If the column is in the map "contentReplacement", the content is overwritten with the defined value in the map.

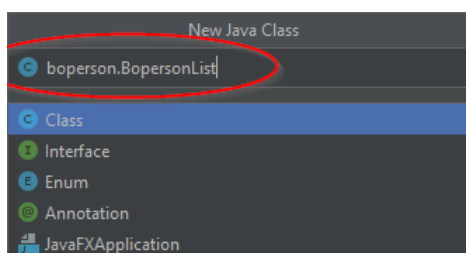
Otherwise the content is overwritten with the one defined in a function, depending on the data type

36.3.4 Create a class

- Open with the right mouse button the context menu on the "src" path of the project and select "New" / "Java Class".



- Enter the fully qualified class name. We create in this example a user exit for the service "BOPerson.List" in the domain "BOPerson". The fully qualified class name is deduced from this: "boperson.BopersonList". Please observe the upper/lower case and the position of points. The rules for forming the "fully qualified class name" are not the subject of this guide. Details on how to form the "fully qualified class name" can be found in the product documentation (e.g. MDS-BAS) and in the documentation of the training courses attended.



- Confirm the entry of the "fully qualified class name" [Enter].

- Now enter the source code from the previous chapter.

36.3.5 Test the user exits

36.3.5.1 Activate development modus

You can also activate the developer mode on the Web Service Provider (WSP) with the option "development.mode = 1". Among other things, the developer mode ensures that changes to user exits take effect the next time the corresponding service is called, without you having to restart the WSP each time.

Edit the file "<InstallDir>\jdir\MOC\1\config.properties" and insert the line "development.mode = 1" below "configreload.timeout=...".

```
#####
#####INSTANCE CONFIGURATION (1)#####
#####
# Please make sure that after configuration values is no TAB or SPACE

# Config reload timeout in seconds
# After this timeout it is checked, if the instance configuration
# (this file) has changed and the config must be reloaded
configreload.timeout=180

development.mode=1
```

Restart the WSP after the configuration file was changed to activate it.

36.3.5.2 Set log level

The Web Service Provider WSP supports the following log levels:

7. "trace" (contains most information and all other levels)
8. "debug" (contains "info", "warn", and "error")
9. "info" (contains "warn" and "error")
10. "warn" (contains "error")
11. "error"
12. "all" (all levels, equals "trace")

You can change the log level. The log level is set to "error" in standard installations.

Edit the file "<InstallDir>\jdir\MOC\1\config.properties". You can change the log level on entries beginning with "log4j.category".

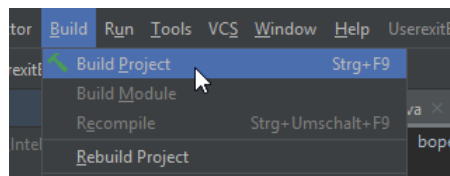
You need to create an additional entry in the logging configuration for the user exit in this guide. The reason for it is that user exits for services that are not in the MPDV's namespace have other class paths. Example: the following goes for a user exit of the "BOPerson.list" service:

```
# The level of logging can be changed here
# ...
# If needed replace error with another level
log4j.category.de.mpdv=error, FileAppender
log4j.category.de.mpdv.common.configManagement=error, FileAppender
log4j.category.de.mpdv.formulaEvaluator=error, FileAppender
log4j.category.de.mpdv.hydra.common.management=error, FileAppender
log4j.category.boperson.BopersonList=debug, FileAppender
...
```

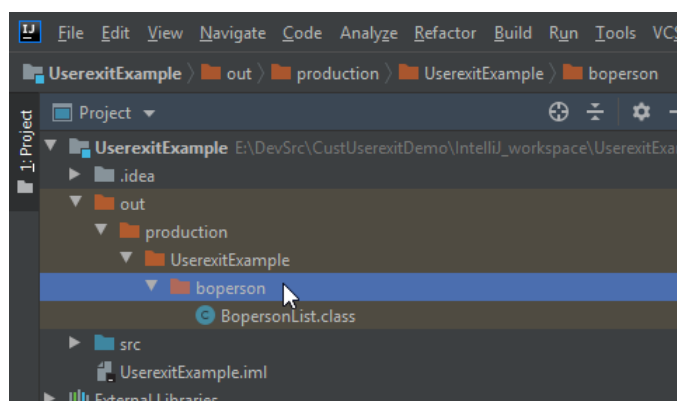
Restart the WSP after the configuration file was changed to activate it.

36.3.5.3 Copy class file to the server

Build project

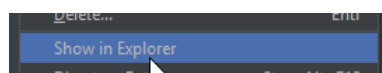


After a successful "Build", there is a "class" file available on the "out" directory.



Right-click with the mouse button on the context menu on the package level (one level above the class file). In the example you open the context menu on "boperson".

Select "Show in Explorer".



Name	Ext	Size	Date
[.]		<DIR>	25.02.20 15:04
[boperson]		<DIR>	25.02.20 15:04

Copy the directory (in the example "boperson") from the Explorer to the server directory for user exits in the local scope.

```
<InstalDir>/jdir/moc/1/userexit/local
```

You must create the "local" directory if it does not exist.

Here in our example, the file is located on the server at

```
\\MyServer\mip1\jdir\MOC\1\userexit\local\boperson\BopersonList.class
```

36.4 Result

Call the service that triggers the user exit. You can use the "Staff" application in the client or the BOPerson.List service with the service interface.

The contents of columns are then changed according to the task.

person.area	person.card_id	person.costcenter	person.date_of_joining	person.email_company	person.email_...	person.id	person.name	person.valid_from	person.valid_to
X	9600	105	1999-01-01T00:00:00+01:00	info@mycompany.com	X	906000	Emmerich, Anton	1900-01-01T00:00:00+01:00	2049-12-31T00:00:00+01:00
X	9600	105	1999-01-01T00:00:00+01:00	info@mycompany.com	X	906000	Emmerich, Anton	2050-01-01T00:00:00+01:00	
X	9998	105	1999-01-01T00:00:00+01:00	info@mycompany.com	X	999998	Meier, Hans	1900-01-01T00:00:00+01:00	
X	9999	105	1999-01-01T00:00:00+01:00	info@mycompany.com	X	999999	Schultz, Werner	1900-01-01T00:00:00+01:00	

You can check in the log file of the WSP if the user exit is loaded and executed. If you have set at least log level "debug" and the user exit is successfully loaded and executed, you will find lines in the log file like:

```
... invoke exit method: [boperson.BopersonList.sdiGlobalAddResultTransformationCallbacks] scope: [LOCAL]}
```

If you use the logger in the user exit itself, you will also find your own log output in this log file, depending on the set log level.

In case of an error you will find in the log file "Exceptions" references that might indicate the problem.

36.5 Further information

If you want to similarly deploy the solution used in the example for the HR master data columns for multiple services, it is not very elegant to implement the same methods in each user exit class of each service in the same way. You can store classes that you want to use in other classes as so-called "ExternalUtils" in a central location and use them in different classes. You can find further details at [MDS ExternalUtils](#).



ExternalUtils can be used for all scopes. It is therefore essential that your packages used as ExternalUtils have unique names.

37 Add Specific Metrics

37.1 General Notes

37.1.1 Introduction

The Web Service Provider (WSP) uses the micrometer framework (<https://micrometer.io>) to collect metrics. The micrometer framework represents an abstraction for several metric systems. MPDV uses the metric system Prometheus (<https://prometheus.io>).

37.1.2 Requirements

The required libraries are available as of service pack 15.



Make sure that you have the files HdrHistogram-2.1.11.jar, LatencyUtils-2.0.3.jar, micrometer-core-1.1.3.jar and micrometer-spring-legacy-1.1.3.jar in the classpath.

You can obtain the files if you visit a training on MES Development Suite at MPDV. Current versions of these files are also available for download from the MPDV Support Portal. They are in the "Java libraries" for the MES Development Suite.



Make sure that you have the license SIS-CSA.

37.1.3 Naming

This chapter is a short repetition of the naming from the [System metrics](#). This repetition is supplemented by information important for you if you want to add specific metrics.



Note the conventions for namespaces when creating your own metrics:

- „u_??“, if you define metrics yourself as end customer.
- „n_NamespaceId_??“, if you define metrics as partners.

The "path" to a metric must include the namespace at least once. Examples of valid metrics (end customer):

```
wsp.service.mdunits.list.u_unitmetric  
wsp.services.subset.u_partofmypath.myfirstmetric  
wsp.services.subset.u_partofmypath.u_mysecondmetric
```

- **wsp.services.....**
 - **Reserved** for metrics provided by the basic system functions.
- **wsp.services.subset.<namespace>....**
 - Metrics provided for a subset of services.
- **wsp.service.<Service name in lower cases>.<namespace>...**
 - Metrics only provided for specific services.

The metrics you create should always include the following tags used across systems:

- **device_id="<Client-Geräte-Ident>"**
- **user="<User-Ident>"**
- **processing_result="processed|failed|empty|..."**
- **distribution_id="<Verteilungs-Id>"** (if technically feasible)

All "parts" of the metric name should be separated by a dot instead of an underscore when the API is called internally to write system metrics (based on micrometers). This applies in deviation from the description of the aforementioned websites. The interface to Prometheus replaces the dots of the metric names with an underscore to make the result match the Prometheus description.

Metric data types

This chapter is a short repetition of data types of metrics from the [System metrics](#). This repetition is supplemented by information important for you if you want to add specific metrics.

- Summary
 - Summary is used to totalize values.
(Example: number of processed data records, processing time)
 - Individual parameter
 - **<Name der Metrik>_sum**
 - **<Name der Metrik>_count**
 - **<Name der Metrik>_max**
 - These individual parameters are automatically generated when using the micrometer framework. If metrics are to be output by other means, these individual parameters must be generated elsewhere.
- Counters
 - Metric, which is incremented with each call.
(Example: number of calls, number of errors)
- Timer
 - Timer is used to add up times (in seconds).
(Example: length of a service)

- Individual parameter
 - `<Name of the metric>_seconds`
- This individual parameter is automatically generated when using the micrometer framework. If metrics are to be output by other means, these individual parameters must be generated elsewhere.
- Gauge
 - Gauge is an actual value and is not added up.
(Example: current memory usage, current table usage, alive status)



Generally, you must bear in mind that only the currently available content of a metric is transmitted at the time Prometheus collects the data.

Some of the data provided, e.g. with gauge, are not "collected" but transferred individually. Instead, only the last transferred value (which had overwritten the predecessor values) is transferred. This is why it does not make sense to include metrics for logging or tracing.

37.2 Example

37.2.1 Purpose

Use the following instructions if you want to enter specific metrics at the Web Service Provider (WSP) in a Java program part.

37.2.2 Task

You want to collect the runtime of a service with a longer runtime per metric and visualize it for example in Grafana. In this instruction the longer runtime is simulated by a `Sleep()`, which randomly pauses the service between 30 seconds and 45 seconds. The instruction shows, exemplary for the creation of a metric, the 2 different options of the micrometer framework to measure time spans: `Timer` and `LongRunningTimer`.

37.2.3 Activate the development mode

Edit the file "`<InstallDir>\jdir\MOC\1\config.properties`" and insert the line "`development.mode = 1`" below "`configreload.timeout=...`".

```
#####
#####INSTANCE CONFIGURATION (1)#####
#####
# Please make sure that after configuration values is no TAB or SPACE

# Config reload timeout in seconds
# After this timeout it is checked, if the instance configuration (this file) has changed and the config must be reloaded
configreload.timeout=180

development.mode=1
```

Restart the WSP after the configuration file was changed.

Among other things, the option "development.mode = 1" ensures that changes to user exits take effect the next time the relevant service is called, without you having to restart the WSP each time.

37.2.4 Create user exit class

Create a class "MdunitsList" in the package "mdunits". Pay attention to the correct upper/lower case. In the MdunitsList class, create the method "sdiGlobalModifyRequest", "sdiGlobalAddResultTransformationCallbacks" and "sdiGlobalCleanup".

The result is as follows:

```
package mdunits;

import de.mpdv.customization.userExit.IUserExitParam;

public class MdunitsList
{
    public void sdiGlobalModifyRequest(final IUserExitParam ueParam)
    {
    }

    public void sdiGlobalAddResultTransformationCallbacks(final IUserExitParam ueParam)
    {
    }

    public void sdiGlobalCleanup(final IUserExitParam ueParam)
    {
    }
}
```

37.2.5 Version with timer

The system starts the timer when the service is started in the version with timer. When the service ends, the system stops the timer. The timer does not pass on the value to the metric system until the service has ended. In contrast, the LongRunningTimer passes the value to the metric system at runtime. Another difference is that the timer in the metric system keeps increasing. The LongRunningTimer on the other hand only displays a value at runtime and then drops back to 0.

Workflow:

- When the service "MDUnits.list" is called, the system calls the user exit "sdiGlobalModifyRequest".
- The user exit "sdiGlobalModifyRequest" starts the timer.
- The system executes the service.
- The system executes the user exit "sdiGlobalCleanup" after the actual service is ended.
- The user exit „sdiGlobalCleanup“ ends the timer.

37.2.5.1 Extend user exit by metric

Extend the user exits to create the following code:

```

package mdunits;

import de.mpdv.customization.userExit.IUserExitParam;
import de.mpdv.sdi.data.SdiEagerDataRowStream;
import de.mpdv.sdi.data.ue.GlobalUeContext;
import de.mpdv.sdi.data.ue.SdiGlobalAddResultTransformationCallbacksResult;
import io.micrometer.core.instrument.Metrics;
import io.micrometer.core.instrument.Timer;

import java.util.Collections;
import java.util.concurrent.ThreadLocalRandom;

public class MdunitsList
{
    private Timer.Sample start;
    private boolean successfulExecution = false;

    public void sdiGlobalModifyRequest(final IUserExitParam ueParam)
    {
        this.start = Timer.start();
        try
        {
            Thread.sleep(ThreadLocalRandom.current().nextLong(30 * 1000L, 45 * 1000L));
        } catch (final InterruptedException exc)
        {
            Thread.currentThread().interrupt();
            throw new IllegalStateException("Thread was interrupted", exc);
        }
    }

    public void sdiGlobalAddResultTransformationCallbacks(final IUserExitParam ueParam)
    {
        ueParam.set("result", new SdiGlobalAddResultTransformationCallbacksResult(Collections.singletonList(
            callBackParam -> {
                if(callBackParam.getDataRow().getDataRowType() == ISdiDataRow.DataRowType.AFTER_LAST_ROW_DUMMY_RECORD)
                {
                    this.successfulExecution = true;
                }
                return new SdiEagerDataRowStream(callBackParam.getDataRow());
            }
        )));
    }

    public void sdiGlobalCleanup(final IUserExitParam ueParam)
    {
        final GlobalUeContext context = ueParam.get("context");
        this.start.stop(Metrics.globalRegistry
            .timer("wsp.service.mdunitslist.u_timer", "device_id", context.getDeviceId(), "user", context.getUserId(),
                "processing_result", this.successfulExecution ? "processed" : "failed"));
    }
}

```

This code starts the timer "wsp.service.mdunitslist.u_timer" in user exit "sdiGlobalModifyRequest", pauses randomly between 30 and 45 seconds and ends the timer user exit "sdiGlobalCleanup". The evaluation dimensions or tags used are "device_id", "user" and "processing_result". The "processing_result" is specified if the callback in the exit sdiGlobalAddResultTransformationCallbacks is called or not. If an error occurs in the service, the callback is not executed.

37.2.5.2 Compile user exit

Compile the class MdunitsList. Make sure you use the correct class path (see section 33.2 "Requirements" above).

37.2.5.3 Test by calling the service MDUnits.list

6. Copy the compiled class including package on the server to
 <InstallDir>\jdir\MOC\1\userexit\custom. If the custom directory does not exist, create this directory.
7. Check if the following file is available after the copy process:
 <InstallDir>\jdir\MOC\1\userexit\custom\mdunits\MdunitsList.class
8. Start the application *Units* on the client.

9. Request data.
10. Call the URL with the browser <WSP-Server>:8080/ for the first WSP, 1 for the second WSP, etc. >/prometheus (example for a WSP: <http://myserver:8080/prometheus>)
 - a. You will find the new entry "wsp_service_mdunitslist_u_timer" => The dot in the name has been converted to an underscore by Micrometer/Prometheus!

37.2.6 Version with timer and LongTaskTimer

In the version with timer and LongTaskTimer the system starts the timer and the LongTaskTimer when the service is started. If the service finishes, the system stops both timer. The LongTaskTimer passes the duration to the metric system already during the call, but drops back to 0 after the service is finished. The timer passes the duration to the metric system only after it is finished, but collects all previous durations. You can see this difference very clearly in a visualization, for example in Grafana.

37.2.6.1 Extend user exit with a LongTaskTimer-metric

Extend the user exits to create the following code:

```
package mdunits;

import de.mpdv.customization.userExit.IUserExitParam;
import de.mpdv.sdi.data.SdiEagerDataRowStream;
import de.mpdv.sdi.data.ue.GlobalUeContext;
import de.mpdv.sdi.data.ue.SdiGlobalAddResultTransformationCallbacksResult;
import io.micrometer.core.instrument.LongTaskTimer;
import io.micrometer.core.instrument.Metrics;
import io.micrometer.core.instrument.Timer;

import java.util.Collections;
import java.util.concurrent.ThreadLocalRandom;

public class MdunitsList
{
    private Timer.Sample start;
    private LongTaskTimer.Sample longRunningTaskSample;
    private boolean successfulExecution = false;

    public void sdiGlobalModifyRequest(final IUserExitParam ueParam)
    {
        final GlobalUeContext context = ueParam.get("context");
        this.longRunningTaskSample = Metrics.globalRegistry.more().longTaskTimer("wsp.service.mdunitslist.u_running_timer",
            "device_id", context.getDeviceId(), "user", context.getUserId())
            .start();
        this.start = Timer.start();
        try
        {
            Thread.sleep(ThreadLocalRandom.current().nextLong(30 * 1000L, 45 * 1000L));
        } catch (final InterruptedException exc)
        {
            Thread.currentThread().interrupt();
            throw new IllegalStateException("Thread was interrupted", exc);
        }
    }

    public void sdiGlobalAddResultTransformationCallbacks(final IUserExitParam ueParam)
    {
        ueParam.set("result", new SdiGlobalAddResultTransformationCallbacksResult(Collections.singletonList(
            callBackParam -> {
                if (callBackParam.getDataRow().getDataRowType() == ISdiDataRow.DataRowType.AFTER_LAST_ROW_DUMMY_RECORD)
                {
                    this.successfulExecution = true;
                }
                return new SdiEagerDataRowStream(callBackParam.getDataRow());
            }
        )));
    }

    public void sdiGlobalCleanup(final IUserExitParam ueParam)
    {
        final GlobalUeContext context = ueParam.get("context");
        this.start.stop(Metrics.globalRegistry
            .timer("wsp.service.mdunitslist.u_timer", "device_id", context.getDeviceId(), "user", context.getUserId(),
                "processing_result", this.successfulExecution ? "processed" : "failed"));
        this.longRunningTaskSample.stop();
    }
}
```


This code starts the timer "wsp.service.mdunitslist.u_timer" and the LongTaskTimer "wsp.service.mdunitslist.u_running_timer" in the user exit "sdiGlobalModifyRequest", pauses randomly between 30 and 45 seconds and ends the timer user exit "sdiGlobalCleanup". The evaluation dimensions or tags used are "device_id", "user" and "processing_result". The "processing_result" is specified if the callback in the exit sdiGlobalAddResultTransformationCallbacks is called or not.

37.2.6.2 Compile user exit

Compile the class MdunitsList. Make sure to use the correct class path (see section 33.2 "Requirements" above).

37.2.6.3 Test by calling the service MDUnits.list

11. Copy the compiled class including package on the server to
<InstallDir>\jdir\MOC\1\userexit\custom. If the custom directory does not exist, create this directory.
12. Check whether the following file exists after the copy action and has a current time stamp:
<InstallDir>\jdir\MOC\1\userexit\custom\mdunits\MdunitsList.class
13. Start the application *Units* on the client.
14. Request data.
15. Call the URL with the browser <WSP-Server>:8080/ for the first WSP, 1 for the second WSP, etc.>/prometheus (example for a WSP: <http://myserver:8080/prometheus>)
 - a. In this list you will find the new entry "wsp_service_mdunitslist_u_running_timer" => The dot in the name was converted to an underscore by Micrometer/Prometheus!

38 Calling Services via PDM Dialogs

38.1 Overview

Purpose

You can call services via dialogs that are sent to the dialog data interface, e.g. from the AIP.



Better call the service directly via the service interface SCS-SIF and not via a PDM dialog. Only call a service via a PDM dialog if the direct call via service interface cannot be used.

Requirements

- Service pack 9. At least Maintenance Pack 555.
- Know-how of the services in use.

Restrictions

- You cannot map list requests (DLG=LIST;n|...) to services.
- You cannot map string commands (DLG=SCMD;n|...) to services.
- Except the return code RET, you cannot pass any results (result parameters, result sets, lists) to the PDM world.

How to proceed

Generate a mapping file for each service that you want to call via a dialog string. The mapping file contains the information which dialog is mapped to which service and any number of conversion rules for the service parameters.

38.2 Tutorial

The tutorial shows how a customer-specific database table is populated with data via a terminal dialog.

In the example you generate a table and a service to create data records in the table. You also create a mapping to a PDM dialog. This way, you can also fill the table with data via an AIP terminal.



This document deals with mapping. As a prerequisite, you need to know how to create tables and how to generate and deploy services.

Customer-specific database table

Use the following SQL statement to create a customer-specific database table. The SQL syntax is "HYDRA-SQL". You must execute the statement with either the utility program hysql.exe or hysql.out or as a patch.

```
create table u_mapping_example
(
    example_ts datetime,
    example_int integer,
    example_char char(500),
    example_date date,
    example_time integer,
    example_option_yn char(1),
    example_type char(20)
);
revoke all on u_mapping_example from "public";

create unique index "hydadm".u_ix_example_1 on "hydadm".u_mapping_example( example_int );
```

Service

The service U_CUST_MappingTest.insert is created as InterpretedBapiService in the repository client.

For demonstration purposes, the service contains fields of different data types.

The column "my_int" is labeled as unique key in the constraints by "KEY=1".

The column "is_option" is defined as type boolean and is stored via the constraints with "BOOL=Y;N;null;string|" in the database field char(1).

Domain...	Domain	Name	Function	Service Type	DLG	List Mode	Description	System Call	Parent	Modified	Is Invalid
Local	U_CUST_MappingTest	U_CUST_MappingTest.insert	insert	InterpretedBapiService					Local->U_CUST_MappingTest->Services		

Domain Set	Domain	Service	Acron...	Web Service Type	DB Alias	DB Tabelle	DB Field	Is Special Parameter	Can Equal	Constrai...	Transformation Type	Re
Local	U_CUST_MappingTest	U_CUST_MappingTest.insert	my_int	integer	me	u_mapping_example	example_int	Y	Y	KEY=1		
Local	U_CUST_MappingTest	U_CUST_MappingTest.insert	is_option	boolean	me	u_mapping_example	example_option_yn	Y	Y		FCT=BOOLTRANSFORMATION TRUEINVAL=<S FALSEINVAL=W	
Local	U_CUST_MappingTest	U_CUST_MappingTest.insert	my_date	datetime	me	u_mapping_example	example_date	Y	Y			
Local	U_CUST_MappingTest	U_CUST_MappingTest.insert	my_string	string	me	u_mapping_example	example_char	Y	Y			
Local	U_CUST_MappingTest	U_CUST_MappingTest.insert	my_time	integer	me	u_mapping_example	example_time	Y	Y			
Local	U_CUST_MappingTest	U_CUST_MappingTest.insert	my_ts	datetime	me	u_mapping_example	example_ts	Y	Y			
Local	U_CUST_MappingTest	U_CUST_MappingTest.insert	my_type	string	me	u_mapping_example	example_type	Y	Y			

...Interprete\U_CUST_MappingTest.Configuration.xml for Copy&Paste:

```
<?xml version="1.0"?>
<domains xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <domain name="U_CUST_MappingTest">
    <service name="U_CUST_MappingTest.insert" Function="insert" ServiceType="InterpretedBapiService" listMode="" DLG=""
      SystemCall="">
      <parameter Acronym="is_option" ResultSet="" WebServiceType="boolean" DefaultValue="" IsResult="" IsDynamicResult=""
        InputAsArray="" IsSpecialParameter="Y" IsFilterParameter="" IsMandatory="" CanEqual="Y" CanLike="" CanBetween="" CanIn=""
        CanNotEqual="" CanEqualOrNull="" CanLikeOrNull="" CanBetweenOrNull="" CanInOrNull="" CanNotEqualOrNull="" CanGt="" CanLt=""
        CanGte="" CanLte="" CanGtOrNull="" CanLtOrNull="" CanGteOrNull="" CanLteOrNull="" HydraAcronym="" HydraResultAcronym=""
        TransferEmptyValuesToHydra="" HydraShiftPart="" Reference="" TransformationType="" PlugName="" DBField="example_option_yn"
        DBAlias="me" DBTabelle="u_mapping_example" DBFieldAlternative="" DataObjectName="" ConditionalFieldKey=""
        Constraints="BOOL=Y;N;null;string|" />
    </service>
  </domain>
</domains>
```

```

<parameter Acronym="my_date" ResultSet="" WebServiceType="datetime" DefaultValue="" IsResult="" IsDynamicResult=""
InputAsArray="" IsSpecialParameter="Y" IsFilterParameter="" IsMandatory="" CanEqual="Y" CanLike="" CanBetween="" CanIn=""
CanNotEqual="" CanEqualOrNull="" CanLikeOrNull="" CanBetweenOrNull="" CanInOrNull="" CanNotEqualOrNull="" CanGt="" CanLt=""
CanGte="" CanLte="" CanGtOrNull="" CanLtOrNull="" CanGteOrNull="" CanLteOrNull="" HydraAcronym="" HydraResultAcronym=""
TransferEmptyValuesToHydra="" HydraShiftPart="" Reference="" TransformationType="" PlugName="" DBField="example_date"
DBAlias="me" DBTable="u_mapping_example" DBFieldAlternative="" DataObjectName="" ConditionalFieldKey="" Constraints="" />
<parameter Acronym="my_int" ResultSet="" WebServiceType="integer" DefaultValue="" IsResult="" IsDynamicResult=""
InputAsArray="" IsSpecialParameter="Y" IsFilterParameter="" IsMandatory="" CanEqual="Y" CanLike="" CanBetween="" CanIn=""
CanNotEqual="" CanEqualOrNull="" CanLikeOrNull="" CanBetweenOrNull="" CanInOrNull="" CanNotEqualOrNull="" CanGt="" CanLt=""
CanGte="" CanLte="" CanGtOrNull="" CanLtOrNull="" CanGteOrNull="" CanLteOrNull="" HydraAcronym="" HydraResultAcronym=""
TransferEmptyValuesToHydra="" HydraShiftPart="" Reference="" TransformationType="" PlugName="" DBField="example_int" DBAlias="me"
DBTable="u_mapping_example" DBFieldAlternative="" DataObjectName="" ConditionalFieldKey="" Constraints="KEY=1" />
<parameter Acronym="my_string" ResultSet="" WebServiceType="string" DefaultValue="" IsResult="" IsDynamicResult=""
InputAsArray="" IsSpecialParameter="Y" IsFilterParameter="" IsMandatory="" CanEqual="Y" CanLike="" CanBetween="" CanIn=""
CanNotEqual="" CanEqualOrNull="" CanLikeOrNull="" CanBetweenOrNull="" CanInOrNull="" CanNotEqualOrNull="" CanGt="" CanLt=""
CanGte="" CanLte="" CanGtOrNull="" CanLtOrNull="" CanGteOrNull="" CanLteOrNull="" HydraAcronym="" HydraResultAcronym=""
TransferEmptyValuesToHydra="" HydraShiftPart="" Reference="" TransformationType="" PlugName="" DBField="example_char"
DBAlias="me" DBTable="u_mapping_example" DBFieldAlternative="" DataObjectName="" ConditionalFieldKey="" Constraints="" />
<parameter Acronym="my_time" ResultSet="" WebServiceType="integer" DefaultValue="" IsResult="" IsDynamicResult=""
InputAsArray="" IsSpecialParameter="Y" IsFilterParameter="" IsMandatory="" CanEqual="Y" CanLike="" CanBetween="" CanIn=""
CanNotEqual="" CanEqualOrNull="" CanLikeOrNull="" CanBetweenOrNull="" CanInOrNull="" CanNotEqualOrNull="" CanGt="" CanLt=""
CanGte="" CanLte="" CanGtOrNull="" CanLtOrNull="" CanGteOrNull="" CanLteOrNull="" HydraAcronym="" HydraResultAcronym=""
TransferEmptyValuesToHydra="" HydraShiftPart="" Reference="" TransformationType="" PlugName="" DBField="example_time"
DBAlias="me" DBTable="u_mapping_example" DBFieldAlternative="" DataObjectName="" ConditionalFieldKey="" Constraints="" />
<parameter Acronym="my_ts" ResultSet="" WebServiceType="datetime" DefaultValue="" IsResult="" IsDynamicResult=""
InputAsArray="" IsSpecialParameter="Y" IsFilterParameter="" IsMandatory="" CanEqual="Y" CanLike="" CanBetween="" CanIn=""
CanNotEqual="" CanEqualOrNull="" CanLikeOrNull="" CanBetweenOrNull="" CanInOrNull="" CanNotEqualOrNull="" CanGt="" CanLt=""
CanGte="" CanLte="" CanGtOrNull="" CanLtOrNull="" CanGteOrNull="" CanLteOrNull="" HydraAcronym="" HydraResultAcronym=""
TransferEmptyValuesToHydra="" HydraShiftPart="" Reference="" TransformationType="" PlugName="" DBField="example_ts" DBAlias="me"
DBTable="u_mapping_example" DBFieldAlternative="" DataObjectName="" ConditionalFieldKey="" Constraints="" />
<parameter Acronym="my_type" ResultSet="" WebServiceType="string" DefaultValue="" IsResult="" IsDynamicResult=""
InputAsArray="" IsSpecialParameter="Y" IsFilterParameter="" IsMandatory="" CanEqual="Y" CanLike="" CanBetween="" CanIn=""
CanNotEqual="" CanEqualOrNull="" CanLikeOrNull="" CanBetweenOrNull="" CanInOrNull="" CanNotEqualOrNull="" CanGt="" CanLt=""
CanGte="" CanLte="" CanGtOrNull="" CanLtOrNull="" CanGteOrNull="" CanLteOrNull="" HydraAcronym="" HydraResultAcronym=""
TransferEmptyValuesToHydra="" HydraShiftPart="" Reference="" TransformationType="" PlugName="" DBField="example_type"
DBAlias="me" DBTable="u_mapping_example" DBFieldAlternative="" DataObjectName="" ConditionalFieldKey="" Constraints="" />
</service>
</domain>
</domains>

```

Mapping file

The PDM dialogs are sent to the server via the AIP terminal. If you want to redirect the dialogs to the service, you must create a mapping file. The mapping file defines which PDM dialog is "redirected" to which service and which PDM dialog identifications are assigned to which service parameters.



Create the mapping file using the encoding "UTF8 without BOM".

Use the dialog "UMAPTEST.INSERT" to collect data via the AIP.

Name of the mapping file

The mapping file is named after the dialog and is in the JSON format: "UMAPTEST.INSERT.json"

Deployment

You must deploy the mapping file on the server in the JHydradir directory "legacyReplacementMapping"; in the example in the custom scope.

"\\server\\Hydra3\\jhydradir\\MOC\\1\\legacyReplacementMapping\\custom\\UMAPTEST.INSERT.json"

or

"\\server\\mip\\jdir\\MOC\\1\\legacyReplacementMapping\\custom\\UMAPTEST.INSERT.json".

Row no.	Contents
1	{
2	"serviceName": "U_CUST_MappingTest.insert",
3	"dlgName": "UMAPTEST.INSERT",
4	"rules": [
5	{
6	"serviceParam": "my_ts",
7	"ruleType": "timestamp",
8	"dlgParamDate": "DAT",
9	"dlgParamTime": "ZEI"
10	},
11	{
12	"serviceParam": "my_date",
13	"ruleType": "timestamp",
14	"dlgParamDate": "DAT"
15	},
16	{
17	"serviceParam": "my_time",
18	"ruleType": "simple",
19	"dlgParam": "ZEI"
20	},
21	{
22	"serviceParam": "my_int",
23	"ruleType": "simple",
24	"dlgParam": "Z AHL"
25	},
26	{
27	"serviceParam": "is_option",
28	"ruleType": "simple",
29	"dlgParam": "ANAUS"
30	},
31	{
32	"serviceParam": "my_type",
33	"ruleType": "replace_from_to",
34	"dlgParam": "TYP",
35	"fromToMap": {
36	"1": "TYPE_FIRST",
37	"2": "TYPE_SECOND",
38	"3": "TYPE_THIRD"
39	},
40	"defaultValue": "TYPE_UNDEFINED",
41	"onError": "DEFAULT_VALUE"
42	},
43	{
44	"serviceParam": "my_string",
45	"ruleType": "set_constant",
46	"constantValue": "<unspecified>"
47	},
48	{
49	"serviceParam": "my_int",
50	"secondServiceParam": "my_string",
51	"ruleType": "simple_pair",
52	"dlgParam": "VAL_UNIT",
53	"splitter": ";"
54	}
55	]
56	}

Calling the dialog

You can now call the service using a dialog, e.g. via the AIP, or you can simulate the call in the command line of the server.

```
hymw -u1109 -c"DLG=UMAPTEST.INSERT|DAT=05/31/2016|ZEI=86400|ZAHL=-12|ANAU=Y|TYP=2|VAL_UNIT=19;Meter|"
```

Further notes on the mapping file

The first information in the mapping file is: which dialog calls which service (rows 2 and 3).

This information is followed by rules for transferring PDM dialog parameters to service parameters. These rules are processed in the order they are listed in the file.

A rule is only then executed, if the PDM dialog parameter is defined and the service parameter does exist. You can show scenarios like e.g. the service parameter `my_int` is first of all populated by the dialog parameter `ZAHL`. But if the dialog parameter `VAL_UNIT` exists, this dialog parameter overwrites the service parameter `my_int` (rows 21 to 25 and 48 to 54).

Transfer rules

There are different types of rules. The `ruleType` defines the rules:

"ruleType": "simple" (rows 21 to 25)

The `dlgParam` specified is passed to the `serviceParam` specified. Implicit type conversions from the usual forms of presentation are performed. The dot is the decimal separator of decimal numbers.

If the data type of the `serviceParam` is "boolean", the type is converted into "true", if the first character of the `dlgParam` is "1", "J", "j", "Y", "y", "T" or "t" ("T" for "TRUE") (rows 26 to 30).

Service parameters of type "timeStamp" cannot be converted with "ruleType": "simple". For service parameters of type "timeStamp", you must use "ruleType": "timestamp".

"ruleType": "timestamp" (rows 5 to 10)

Service parameters can have the type "timeStamp", which contains a complete time stamp with date and time. Dialog parameters do not know the type timeStamp. Here, date and time are included in two separate dialog parameters.

The specified `serviceParam` is populated with the date from `dlgParamDate` and with the time from `dlgParamTime`.

The date must be in the MM/DD/YYYY format, the time must be in seconds.

TimeStamps containing only a date are equally transferred using the ruleType timeStamp. In this case, only the `dlgParamDate` is used (rows 11 to 15).

The dialog parameters and the service parameters show times in seconds as integers. Times are transferred using the ruleType "simple" (rows 16 to 20).

"ruleType": "replace_from_to"

You use this ruleType to replace values. The dlgParam specified is passed to the serviceParam specified. The value of dlgParam is replaced using the specified table (fromToMap). In the example (rows 31 to 42) the value "1" is replaced with "TYPE_FIRST".

If the text in dlgParam could not be found in the table, the action specified as onError is carried out:

NO_CHANGE

The text in dlgParam is transferred unchanged to serviceParam.

DEFAULT_VALUE

The value specified as defaultValue is transferred to serviceParam.

SKIP_RULE

The rule is skipped, the serviceParam is not changed.

Following these replacement rules, the implicit type conversions are processed as with the rule "simple".

"ruleType": "set_constant"

The serviceParam specified is populated with the constantValue specified (rows 43 to 47).

"ruleType": "simple_pair"

In rare cases, one dialog parameter transfers two separated values. These values must be assigned to two different service parameters. For example: In case of |VAL_UNIT=19;Meter|, the service parameter my_int should be populated with "19" and my_string with "Meter". The rule must define serviceParam, secondServiceParam, dlgParam and splitter.

"ruleType": "user_field_66"

This rule transfers the usual 66 user fields from the dialog parameters to the service parameters. Using this rule, you need not specify 66 individual rules. Accordingly, you enter the placeholder "[n]" as the user field number in the dialog and service parameters. The dialog parameters replace this placeholder with the simple user field number without leading zero. The service parameters replace the placeholder with the corresponding double-digit number, if necessary with leading zero. The placeholder can be at any place in the dialog or service parameter. Each transfer implies the same type conversions as with "ruleType": "simple".

Example:

```
{
  "ruleType": "user_field_66",
  "dlgParam": "CNR.FU.[n]",
```

```
"serviceParam": "batch.userfield.[n] "
},
```

This rule results in the following single transfer rules, for example:

CNR.FU.3	batch.userfield03
CNR.FU.33	batch.userfield33
CNR.FU.44	batch.userfield44

"ruleType": "counter"

This rule is reserved to transfer the standard dialog parameters for "counters' data collection" to service parameters in the future.

"ruleType": "quantities"

This rule is reserved to transfer the standard dialog parameters for "recording of quantities" to service parameters in the future.

Dialog appendices:

In the PDM dialogs, you can add appendices, separated by semicolon, to the actual dialog (identification DLG=...), e.g. DLG=XXX;ASYNC| and DLG=XXX;PLAUS|.

The mapping is performed for the actual dialog only, without appendix. The appendix is virtually transferred to an additional dialog parameter DLG_APPENDIX and can then be respected during parameter conversion. If you need the dialog appendix as service parameter, you must define a transfer rule for this service parameter with dialog parameter DLG_APPENDIX.

Example:

The dialog "DLG=A_UN;ASYNC|ANR=abc|..." is processed as "DLG=A_UN|DLG_APPENDIX=ASYNC|ANR=abc|..." in the mapping.

Error

If the service called detects an error (validation error), the return value for the dialog is -1. The dialog identifications KT and LT provide further information. Especially the language key returned in LT can help to find the error cause.

```
|RET=-1|KT=Validation error occurred|LT=lkInsertAlreadyExists|
```

If a technical runtime error occurs on running the service, the return value for the dialog is again -1. The dialog identifications KT and LT provide further information.


```
|RET=-1|KT=RuntimeException at class: DbUtil at method: getSqlParameter at line:
190: "Unsupported class type for getSqlParameter. Offending type: class
java.lang.Boolean"|LT=1kUnspecifiedError|
```

Use the log file of the JavaServer for a more detailed analysis.

Notes on use

The PDM services (hymw) use a database table (*rwsc_callback_mapping*) to decide which dialogs are redirected to which services. This database table is populated by the Web Service Provider (WSP). The table is not populated immediately after reloading or changing a mapping file.

There are three ways to update this table:

- Automatic: When the Web Service Provider (WSP) is started, the table is automatically updated.
- Service: If you call the service "ServiceDictionary.refresh", the update is started manually.
- Calling the PDM: You can start the update manually by calling the PDM command `DLG=RWSC.MSG|RWSC.ACTION=LEGREPLMAPRENEW|`. You can also execute the command in the command line of the server:

```
hymw -u9999 -c"DLG=RWSC.MSG|RWSC.ACTION=LEGREPLMAPRENEW|"
```